Docker & Kubernetes

A Complete Guide

From First Container to Production Deployment -- With Docker Compose and Kubernetes

Containers • Compose • Orchestration

2025 Edition • 32 Chapters

Table of Contents

Chapter 1 — What Is Docker?
Chapter 2 — Installing Docker
Chapter 3 — Your First Container
Chapter 4 — How Containers Work
Chapter 5 — Docker Images
Chapter 6 — Your First Dockerfile
Chapter 7 — Dockerfile Best Practices
Chapter 8 — Building Real App Images
Chapter 9 — Container Lifecycle
Chapter 10 — Environment Variables and Configuration
Chapter 11 — Volumes
Chapter 12 — Networking
Chapter 13 — Logs and Debugging
Chapter 14 — Introduction to Docker Compose
Chapter 15 — Multi-Container Applications
Chapter 16 — Compose Networking and Volumes
Chapter 17 — Compose for Development
Chapter 18 — Compose in Practice
Chapter 19 — Docker Hub and Registries
Chapter 20 — Image Tagging
Chapter 21 — CI/CD with Docker
Chapter 22 — Docker Security
Chapter 23 — Docker in Production
Chapter 24 — Docker Swarm
Chapter 25 — Optimizing Images
Chapter 26 — What Is Kubernetes?
Chapter 27 — Kubernetes Setup

Chapter 28 — Pods, Deployments, and Services
Chapter 29 — ConfigMaps, Secrets, and Volumes
Chapter 30 — Deploy a Real App to Kubernetes
Chapter 31 — Appendix: Cheat Sheet
Chapter 32 — Appendix: Project

Chapter 1: What Is Docker?

What You Will Learn

- What Docker is and why it exists
- The problem Docker solves (the "works on my machine" problem)
- How Docker compares to virtual machines
- The main parts of the Docker ecosystem
- Real-world use cases for Docker
- Who uses Docker and why

Why This Chapter Matters

Imagine you bake a perfect cake at home. You write down the recipe. You give the recipe to a friend. They follow it exactly. But the cake tastes different.

Why? Different oven. Different brand of flour. Different altitude. Different kitchen.

Software has the same problem. A program works perfectly on your computer. But it breaks on your coworker's computer. It breaks on the server. It breaks in production.

Docker fixes this problem. It is one of the most important tools in modern software development. Before you learn how to use it, you need to understand what it is and why millions of developers depend on it every day.

The Shipping Container Analogy

Before the 1950s, shipping goods across the ocean was a nightmare.

Each product had a different shape. Bananas were different from furniture. Furniture was different from electronics. Workers had to load each item by hand. Things broke. Things got lost. It was slow and expensive.

Then someone invented the shipping container. A standard metal box. The same size. The same shape. Every time.

It did not matter what was inside. Bananas, furniture, or electronics -- they all went into the same kind of box. Ships, trucks, and trains were built to carry these standard boxes. Loading became fast. Shipping became cheap. Global trade exploded.

```
Before Shipping Containers:
+--------+    +----+    +------------+
| Banana |    | TV |    | Chair      |
| Crate  |    +----+    | (awkward   |
+--------+              |  shape)    |
                         +------------+
  Each item loaded separately. Slow. Expensive. Fragile.

After Shipping Containers:
+-------------------+  +-------------------+  +-------------------+
|  [Bananas]        |  |  [TVs]            |  |  [Chairs]         |
|  Standard Box     |  |  Standard Box     |  |  Standard Box     |
+-------------------+  +-------------------+  +-------------------+
  Same size. Same shape. Stack them. Ship them. Done.
```

Docker is the shipping container for software.

Your application, its settings, its libraries, its dependencies -- everything goes into one standard package called a container. That container runs the same way everywhere. On your laptop. On your coworker's laptop. On a server in the cloud.

The "Works on My Machine" Problem

This is the most famous problem in software development. Let us walk through a real example.

The Scenario

Alice is a developer. She builds a web application using Python 3.11 and a database called PostgreSQL 15.

On her laptop:

- Python 3.11 is installed
- PostgreSQL 15 is installed
- The application works perfectly

She sends the code to Bob. Bob tries to run it.

On Bob's laptop:

- Python 3.9 is installed (older version)
- PostgreSQL 13 is installed (older version)
- The application crashes

Bob says, "Your code is broken."

Alice says, "It works on my machine."

```
Alice's Laptop                     Bob's Laptop
+-------------------+             +-------------------+
| Python 3.11       |             | Python 3.9        |
| PostgreSQL 15     |             | PostgreSQL 13     |
| Linux libraries   |             | Different libs    |
| App config v2     |             | App config v1     |
|                   |             |                   |
| App: WORKS!       |             | App: CRASHES!     |
+-------------------+             +-------------------+
        |                                 |
        v                                 v
   "Ship it!"                     "It's broken!"
```

Why Does This Happen?

Software depends on many things:

- Programming language version -- Python 3.11 has features that Python 3.9 does not.
- Libraries -- Small pieces of code written by other people. Your app uses specific versions.
- Operating system -- Windows, Mac, and Linux behave differently.
- System settings -- Environment variables, file paths, permissions.
- Other software -- Databases, web servers, message queues.

All of these are called dependencies. Dependencies are the things your software needs in order to run.

When any dependency is different, your software can break.

How Docker Fixes This

With Docker, Alice packages everything together:

```
Docker Container
+-----------------------------------+
|  Python 3.11                      |
|  PostgreSQL 15                    |
|  All required libraries           |
|  All configuration files          |
|  Alice's application code         |
|                                   |
|  Everything the app needs to run  |
+-----------------------------------+
```

She gives this container to Bob. Bob runs it. It works. Because the container carries its own Python, its own PostgreSQL, its own everything.

The container does not care what is installed on Bob's computer. It brings its own world with it.

Virtual Machines vs Containers

Before Docker, people solved the "works on my machine" problem with virtual machines (VMs). To understand containers, you need to understand how they are different from VMs.

What Is a Virtual Machine?

A virtual machine is a complete computer running inside your computer. It has its own operating system, its own memory, and its own storage.

Think of it like building a house inside a house. The inner house has its own foundation, its own walls, its own plumbing. It works. But it is heavy. It uses a lot of space. It takes a long time to build.

What Is a Container?

A container is like an apartment in a building. Each apartment is separate. You cannot see into your neighbor's apartment. You have your own space. But you share the building's foundation, plumbing, and electrical system.

Containers share the host computer's operating system. They only carry what is different -- the application and its dependencies.

Side-by-Side Comparison

```
Virtual Machines                     Containers
+---+---+---+                       +---+---+---+
|App|App|App|                       |App|App|App|
+---+---+---+                       +---+---+---+
|Lib|Lib|Lib|                       |Lib|Lib|Lib|
+---+---+---+                       +---+---+---+
|OS |OS |OS |                       Container Engine
+---+---+---+                       +--------------+
  Hypervisor                        |   Host OS    |
+--------------+                    +--------------+
|   Host OS    |                    |  Hardware    |
+--------------+                    +--------------+
|  Hardware    |
+--------------+

Each VM has its OWN              Containers SHARE
operating system.                the host OS.
Heavy. Slow to start.           Light. Starts in seconds.
```

Key Differences

Feature	Virtual Machine	Container
Size	Gigabytes (1-10 GB)	Megabytes (10-500 MB)
Startup time	Minutes	Seconds
Operating system	Each VM has its own	Shares host OS
Isolation	Very strong	Strong (but shares kernel)
Resource usage	Heavy	Lightweight
How many can run	A few per machine	Dozens or hundreds

When to Use Each

Use virtual machines when:

- You need to run a completely different operating system (like Windows on a Mac)
- You need the strongest possible isolation (security-sensitive work)
- You are running untrusted code

Use containers when:

- You want fast startup times
- You want to run many applications on one machine
- You want consistent environments for development, testing, and production
- You are building microservices (small, independent services)

The Docker Ecosystem

Docker is not just one tool. It is a family of tools that work together. Here are the main parts.

Docker Engine

This is the core of Docker. It is the software that actually builds and runs containers. It runs in the background on your computer. You do not interact with it directly most of the time.

Think of it as the engine in a car. You do not touch the engine. You use the steering wheel and pedals. But without the engine, nothing works.

Docker Desktop

This is the application you install on your Mac, Windows, or Linux computer. It gives you a nice graphical interface to manage your containers. It also includes Docker Engine, so installing Docker Desktop gives you everything you need.

Think of it as the car's dashboard. It shows you what the engine is doing in a friendly way.

Docker Hub

This is a website where people share Docker images. An image is a blueprint for creating a container. Docker Hub is like an app store for containers.

Want to run a database? Download the PostgreSQL image from Docker Hub. Want to run a web server? Download the Nginx image. Thousands of ready-to-use images are available for free.

Website: https://hub.docker.com

Docker Compose

This tool lets you run multiple containers together. Most real applications need more than one container. A web app might need a web server container, a database container, and a cache

container.

Docker Compose lets you define all of them in one file and start them with one command.

Docker CLI

This is the command-line tool you use to talk to Docker. You type commands like `docker run` and `docker build` in your terminal. We will use this tool throughout the entire book.

```
The Docker Ecosystem:

+-----------------------------------------------------+
|                Docker Desktop                       |
|  (Graphical interface for your computer)            |
|                                                     |
|  +-----------------------------------------------+  |
|  |             Docker Engine                     |  |
|  |  (Builds and runs containers)                 |  |
|  +-----------------------------------------------+  |
|                                                     |
|  +--------------------+ +----------------------+  |
|  |    Docker CLI      | |  Docker Compose      |  |
|  | (Command-line tool) | | (Multi-container)    |  |
|  +--------------------+ +----------------------+  |
+-----------------------------------------------------+
         |
         | pulls images from
         v
+-----------------------------------------------------+
|                Docker Hub                           |
|  (Online library of images)                         |
|  nginx, python, node, postgres, redis, ...          |
+-----------------------------------------------------+
```

Real-World Use Cases

Docker is not just a toy for developers. It is used in serious, large-scale situations.

1. Consistent Development Environments

A team of 10 developers all work on the same project. Without Docker, each person sets up their own environment. "I use Python 3.10." "I use Python 3.11." "My database is version 14." "Mine is version 15."

With Docker, everyone runs the same container. Same Python. Same database. Same everything. No more "works on my machine."

2. Microservices

Large applications are often split into small, independent services. Netflix, for example, has hundreds of services. Each service runs in its own container. Each service can be updated independently.

```
Monolithic Application              Microservices with Docker
+--------------------+              +------+ +------+ +------+
|                    |              | User | | Cart | | Pay  |
|  Everything in     |              | Svc  | | Svc  | | Svc  |
|  one big app       |              +------+ +------+ +------+
|                    |              +------+ +------+ +------+
|                    |              |Search| |Email | |Auth  |
+--------------------+              | Svc  | | Svc  | | Svc  |
                                    +------+ +------+ +------+
One failure breaks                  Each service runs in its
everything.                         own container. Independent.
```

3. CI/CD Pipelines

When developers push code, automated systems build it, test it, and deploy it. These systems are called CI/CD pipelines (Continuous Integration / Continuous Deployment).

Docker makes sure the build environment is always the same. Tests run in containers. The production deployment uses the same container that was tested.

4. Cloud Deployment

Cloud services like AWS, Google Cloud, and Azure all support Docker containers. You build your container once and deploy it to any cloud provider.

5. Learning and Experimentation

Want to try a new database? Run it in a container. When you are done, delete the container. Nothing was installed on your computer. Nothing to clean up.

Who Uses Docker?

Docker is used by millions of developers and thousands of companies.

- Startups use Docker to move fast and keep environments consistent.
- Large companies like Google, Netflix, and Spotify use Docker to run thousands of services.
- DevOps engineers use Docker to automate deployments.
- Data scientists use Docker to share reproducible experiments.
- Students use Docker to set up development environments quickly.

If you work in software, you will encounter Docker. It is not optional anymore. It is a fundamental skill.

Common Mistakes

1. Thinking Docker is a virtual machine. It is not. Containers share the host operating system. VMs do not. This is the most important difference.
2. Thinking Docker only works on Linux. Docker Desktop runs on Mac, Windows, and Linux. The containers themselves use Linux internally, but Docker Desktop handles this for you.
3. Thinking you need to learn Docker all at once. You do not. Start with the basics. Run a container. Then build an image. Then learn Compose. Step by step.
4. Confusing images and containers. An image is a blueprint. A container is a running instance of that blueprint. We will explain this in detail in Chapter 4.

Best Practices

1. Start using Docker early in a project. Do not wait until deployment. Use Docker from the first day. Your development environment should match your production environment.
2. Use official images from Docker Hub. Do not reinvent the wheel. If you need Python, use the official Python image. If you need PostgreSQL, use the official PostgreSQL image.
3. Keep containers small. A container should do one thing well. Do not put your web server, database, and cache in the same container.
4. Learn the command line. Docker Desktop is nice, but the command line is more powerful and more portable. Most Docker tutorials and documentation use command-line examples.

Quick Summary

Docker is a tool that packages your application and everything it needs into a standard unit called a container. Containers run the same way on any computer. This solves the "works on my machine" problem.

Containers are not virtual machines. They are lighter, faster, and more efficient. They share the host operating system instead of running their own.

The Docker ecosystem includes Docker Engine (the core), Docker Desktop (the GUI), Docker Hub (image library), Docker Compose (multi-container tool), and the Docker CLI (command-line interface).

Key Points

- Docker packages software into containers that run consistently everywhere.
- The "works on my machine" problem happens because of different dependencies across environments.
- Containers share the host OS and are lightweight. VMs have their own OS and are heavy.
- Docker images are blueprints. Containers are running instances of images.
- Docker Hub is an online library where you can find thousands of ready-to-use images.
- Docker Compose manages multiple containers working together.
- Docker is used for development environments, microservices, CI/CD, cloud deployment, and learning.

Practice Questions

1. In your own words, explain how the shipping container analogy relates to Docker. What does Docker "standardize"?
2. Alice writes a Node.js application that works on her laptop. She sends the code to Bob, but it crashes on his laptop because he has a different version of Node.js. How would

Docker solve this problem?

3. Name three differences between virtual machines and containers. When would you choose a VM over a container?
4. What is Docker Hub? How is it similar to an app store?
5. A company has a large application with a user service, a payment service, and an email service. How could Docker help them manage these services?

Exercises

Exercise 1: Research Docker Hub

Go to https://hub.docker.com in your web browser. Search for "python" and answer these questions:

- How many pulls does the official Python image have?
- What tags (versions) are available?
- Who maintains the official image?

Exercise 2: Spot the Problem

Read this scenario and identify what could go wrong:

> A team of three developers is building a Java application. Dev 1 uses Java 17 on Mac. Dev 2 uses Java 11 on Windows. Dev 3 uses Java 21 on Linux. They do not use Docker.

Write down at least three problems this team might face. Then explain how Docker would solve each one.

Exercise 3: Draw It Out

On a piece of paper or in a drawing tool, draw two diagrams:

1. Three virtual machines running on one physical server. Label each layer (hardware, host OS, hypervisor, guest OS, app).
2. Three containers running on one physical server. Label each layer (hardware, host OS, container engine, app).

Compare the two diagrams. Which one has more layers? Which one would use more resources?

What Is Next?

Now you know what Docker is and why it matters. In the next chapter, you will install Docker on your computer and run your very first container. You will type your first Docker command and see a container come to life. Let us get started.

Chapter 2: Installing Docker

What You Will Learn

- How to install Docker Desktop on Mac, Windows, and Linux
- How to verify that Docker is installed correctly
- How to create a Docker Hub account
- How to run your very first Docker command
- How to navigate the Docker Desktop interface

Why This Chapter Matters

You cannot learn to drive by reading about cars. You need to sit in the driver's seat, start the engine, and move.

Docker is the same. You need it installed on your computer before you can do anything. This chapter walks you through every step. By the end, you will have Docker running and you will have launched your first container.

Do not skip this chapter. Follow along. Type every command. See every output. This is where your hands-on Docker journey begins.

What You Need Before You Start

Before installing Docker, make sure your computer meets these requirements:

For Mac:

- macOS 12 (Monterey) or newer
- At least 4 GB of RAM

- Apple Silicon (M1, M2, M3, M4) or Intel processor

For Windows:

- Windows 10 version 21H2 or newer, or Windows 11
- At least 4 GB of RAM
- 64-bit processor
- Hardware virtualization enabled in BIOS (most modern computers have this)

For Linux:

- A 64-bit distribution (Ubuntu, Fedora, Debian, etc.)
- At least 4 GB of RAM
- A user account with sudo privileges

Installing Docker Desktop on Mac

Step 1: Download Docker Desktop

Open your web browser and go to:

```
https://www.docker.com/products/docker-desktop/
```

Click the Download for Mac button. The website will detect whether you have an Apple Silicon or Intel Mac and offer the correct download.

If you are not sure which chip you have, click the Apple menu in the top-left corner of your screen, then click About This Mac. If it says "Apple M1" or "Apple M2" or similar, you have Apple Silicon. If it says "Intel," you have an Intel chip.

Step 2: Install the Application

1. Open the downloaded `.dmg` file. It will show a window with the Docker icon and an Applications folder.
2. Drag the Docker icon into the Applications folder.
3. Wait for the copy to finish.

```
+-----------------------------------------+
|                                         |
|   [ Docker Icon ]  --->  [ Applications ] |
```

```
|                                          |
|   Drag Docker to Applications            |
+------------------------------------------+
```

Step 3: Start Docker Desktop

1. Open your Applications folder.
2. Double-click Docker.
3. macOS may ask, "Are you sure you want to open it?" Click Open.
4. Docker may ask for your password. Enter it. Docker needs this to set up networking.
5. You will see the Docker icon (a whale) appear in your menu bar at the top of the screen.

Step 4: Wait for Docker to Start

The whale icon in your menu bar will animate (the containers on the whale will move). This means Docker is starting up. Wait until the animation stops. This usually takes 30 to 60 seconds.

When Docker is ready, the whale icon will be still.

Step 5: Accept the Terms

Docker Desktop will show you a service agreement. Read it and click Accept to continue.

Installing Docker Desktop on Windows

Step 1: Check Virtualization

Docker on Windows needs a feature called hardware virtualization. Most modern computers have it enabled. To check:

1. Press `Ctrl + Shift + Esc` to open Task Manager.
2. Click the Performance tab.
3. Click CPU on the left.
4. Look for "Virtualization" at the bottom. It should say Enabled.

If it says "Disabled," you need to enable it in your computer's BIOS settings. Search online for "enable virtualization [your computer model]" for specific instructions.

Step 2: Enable WSL 2

Docker Desktop on Windows uses WSL 2 (Windows Subsystem for Linux). This lets Docker run Linux containers on Windows.

Open PowerShell as Administrator:

1. Right-click the Start button.
2. Click Terminal (Admin) or PowerShell (Admin).

Then type this command:

```powershell
wsl --install
```

What this does: It installs WSL 2 and a default Linux distribution (usually Ubuntu). This is the foundation Docker needs to run containers on Windows.

Restart your computer after this command finishes.

Step 3: Download Docker Desktop

Open your web browser and go to:

```
https://www.docker.com/products/docker-desktop/
```

Click Download for Windows.

Step 4: Run the Installer

1. Double-click the downloaded `Docker Desktop Installer.exe`.
2. Make sure Use WSL 2 instead of Hyper-V is checked. This is the recommended option.
3. Click OK to start the installation.
4. Wait for the installation to finish. This may take a few minutes.
5. Click Close and restart when prompted.

Step 5: Start Docker Desktop

After your computer restarts:

1. Find Docker Desktop in your Start menu.
2. Click to open it.

3. Docker may take a minute or two to start. You will see the Docker whale icon in your system tray (bottom-right corner of the screen).
4. Accept the service agreement when prompted.

Installing Docker on Linux

On Linux, you can install Docker Engine directly or use Docker Desktop. We will show Docker Engine because it is more common on Linux.

Ubuntu / Debian

Open a terminal and run these commands one by one:

```bash
# Step 1: Update your package list
sudo apt-get update
```

What this does: Downloads the latest list of available software packages. Think of it as refreshing the catalog at a store.

```bash
# Step 2: Install required packages
sudo apt-get install -y ca-certificates curl
```

What this does: Installs tools Docker needs to download files securely from the internet.

```bash
# Step 3: Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

What this does: A GPG key is like a digital signature. It proves that the Docker software you download is genuine and has not been tampered with.

```bash
# Step 4: Add the Docker repository
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

What this does: Tells your system where to find Docker packages. It is like adding a new store to your shopping list.

```bash
# Step 5: Update the package list again
sudo apt-get update
```

What this does: Refreshes the catalog now that we added the Docker store.

```bash
# Step 6: Install Docker
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

What this does: Installs Docker Engine and all related tools. Here is what each package is:

- `docker-ce` -- Docker Community Edition (the engine)
- `docker-ce-cli` -- The command-line tool
- `containerd.io` -- The container runtime (manages container processes)
- `docker-buildx-plugin` -- Advanced image building tool
- `docker-compose-plugin` -- Docker Compose (for multi-container apps)

```bash
# Step 7: Add your user to the docker group
sudo usermod -aG docker $USER
```

What this does: Lets you run Docker commands without typing `sudo` every time. You need to log out and log back in for this to take effect.

Fedora / RHEL

```bash
# Step 1: Install the dnf-plugins-core package
sudo dnf -y install dnf-plugins-core

# Step 2: Add the Docker repository
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/fedora/docker-ce.repo

# Step 3: Install Docker
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# Step 4: Start Docker
sudo systemctl start docker
sudo systemctl enable docker
```

```
# Step 5: Add your user to the docker group
sudo usermod -aG docker $USER
```

Log out and log back in after running these commands.

Verifying Your Installation

Now let us make sure Docker is installed correctly. Open your terminal (Mac/Linux) or PowerShell (Windows).

Check the Docker Version

```bash
docker --version
```

Expected output:

```
Docker version 27.4.1, build b9d17ea
```

Your version number may be different. That is fine. As long as you see a version number, Docker is installed.

Line-by-line explanation:

- `docker` -- The Docker command-line tool
- `--version` -- A flag that asks "What version are you?"
- The output shows the version number and a build hash (a unique identifier for that specific build)

Check Docker System Information

```bash
docker info
```

Expected output (abbreviated):

```
Client:
 Version:    27.4.1
 Context:    desktop-linux

Server:
 Containers: 0
  Running: 0
  Paused:  0
  Stopped: 0
 Images: 0
 Server Version: 27.4.1
 Storage Driver: overlay2
 Operating System: Docker Desktop
 Total Memory: 7.659GiB
 CPUs: 4
```

Line-by-line explanation:

- `Client` -- Information about the Docker CLI (the tool you are typing commands into)
- `Server` -- Information about Docker Engine (the background service)
- `Containers: 0` -- You have no containers yet (expected for a fresh install)
- `Images: 0` -- You have no images yet (also expected)
- `Storage Driver: overlay2` -- The file system technology Docker uses for images and containers
- `Total Memory` -- How much memory Docker can use
- `CPUs` -- How many CPU cores Docker can use

If you see an error like "Cannot connect to the Docker daemon," make sure Docker Desktop is running.

Creating a Docker Hub Account

Docker Hub is the default place where Docker looks for images. You can use it without an account, but having one lets you:

- Download images faster (higher rate limits)
- Store your own images
- Access private repositories

Step 1: Go to Docker Hub

Open your browser and visit:

```
https://hub.docker.com
```

Step 2: Sign Up

1. Click Sign Up.
2. Enter a username, email, and password.
3. Complete the verification.
4. Check your email and verify your account.

Step 3: Log In from the Command Line

Back in your terminal, type:

```bash
docker login
```

Expected output:

```
Log in with your Docker ID or email address to push and pull images.
Username: yourusername
Password:
Login Succeeded
```

What this does: Connects your terminal to your Docker Hub account. Now Docker can download images faster and push your own images.

Your password will not show as you type it. This is normal security behavior. Just type it and press Enter.

Running Your First Container: hello-world

This is the moment. You are about to run your first Docker container. Type this command:

```bash
docker run hello-world
```

Expected output:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:c41088499908a59aae30...
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Congratulations! You just ran your first container!

What Just Happened? Step by Step

Let us break down exactly what Docker did:

```
Step 1: You typed "docker run hello-world"
         |
         v
Step 2: Docker looked for the "hello-world" image on your computer.
        It did not find it.
        "Unable to find image 'hello-world:latest' locally"
         |
         v
Step 3: Docker went to Docker Hub and downloaded the image.
        "Pulling from library/hello-world"
        "Pull complete"
         |
         v
Step 4: Docker created a container from the image.
         |
         v
Step 5: Docker started the container.
        The container ran a small program that printed the
```

```
        "Hello from Docker!" message.
         |
         v
Step 6: The program finished. The container stopped.
```

```
Your Computer                        Docker Hub
+------------------+                 +------------------+
|                  |   1. "Do you    |                  |
|  Docker Engine   | ---have this?-->|  hello-world     |
|                  |                 |  image           |
|                  |   2. "Here     |                  |
|  [hello-world    | <--it is!"-----|                  |
|   image saved]   |                 +------------------+
|                  |
|  3. Create and   |
|     run container|
|                  |
|  4. "Hello from  |
|     Docker!"     |
+------------------+
```

Understanding Each Line of Output

`Unable to find image 'hello-world:latest' locally` Docker first checks your computer for the image. Since this is your first time, it does not find it. The word `latest` is a tag -- it means "the newest version."

`latest: Pulling from library/hello-world` Docker is downloading the image from Docker Hub. The `library/` prefix means this is an official image maintained by Docker.

`e6590344b1a5: Pull complete` The image has been downloaded. The long string of characters is a unique identifier for this image layer.

`Digest: sha256:c41088499908a59aae30...` This is a fingerprint of the image. It guarantees you got exactly the right image and nothing was changed during download.

`Status: Downloaded newer image for hello-world:latest` Docker confirms the download is complete.

`Hello from Docker!` The container ran successfully and printed this message.

Docker Desktop UI Overview

Docker Desktop gives you a graphical way to see what Docker is doing. Let us take a quick tour.

Opening Docker Desktop

- Mac: Click the whale icon in the menu bar, then click Dashboard.
- Windows: Click the whale icon in the system tray, then click Dashboard.
- Linux: Open Docker Desktop from your applications menu.

The Main Sections

```
+-------------------------------------------------------------+
|  Docker Desktop                                             |
+-------------------------------------------------------------+
|                                                             |
|  [ Containers ]  [ Images ]  [ Volumes ]  [ Builds ]     |
|                                                             |
|  Containers Tab:                                            |
|  +-------------------------------------------------------+ |
|  | NAME          IMAGE         STATUS      PORT         | |
|  +-------------------------------------------------------+ |
|  | (your containers will appear here)                    | |
|  +-------------------------------------------------------+ |
|                                                             |
+-------------------------------------------------------------+
```

Containers tab: Shows all your containers (running and stopped). You can start, stop, and delete containers from here.

Images tab: Shows all Docker images on your computer. You can see how much space each image uses.

Volumes tab: Shows data storage areas that persist even when containers are deleted. We will learn about volumes later.

Builds tab: Shows the history of images you have built.

Container Details

When you click on a container, you can see:

- Logs: What the container printed to the screen (like the "Hello from Docker!" message)
- Inspect: Detailed technical information about the container
- Terminal: An interactive terminal inside the container
- Stats: CPU and memory usage

You do not need to use Docker Desktop for this book. Everything can be done from the command line. But it is useful to have a visual way to see what is happening.

Troubleshooting Common Installation Issues

"Cannot connect to the Docker daemon"

What it means: Docker Engine is not running.

Fix: Make sure Docker Desktop is open and running. Look for the whale icon in your menu bar (Mac) or system tray (Windows). If you are on Linux without Docker Desktop, start Docker with:

```bash
sudo systemctl start docker
```

"Permission denied" on Linux

What it means: Your user is not in the `docker` group.

Fix: Run these commands and then log out and back in:

```bash
sudo usermod -aG docker $USER
```

Then close your terminal, log out, log back in, and open a new terminal.

"WSL 2 is not installed" on Windows

What it means: Docker needs WSL 2 to run on Windows.

Fix: Open PowerShell as Administrator and run:

```powershell
wsl --install
```

Restart your computer after this finishes.

Docker Desktop is slow to start

What it means: This is normal the first time. Docker needs to set up its environment.

Fix: Wait 1-2 minutes. If it consistently takes more than 5 minutes, try these steps:

1. Quit Docker Desktop completely.
2. Restart your computer.
3. Open Docker Desktop again.

"No space left on device"

What it means: Docker has used up the disk space allocated to it.

Fix: Clean up unused Docker data:

```bash
docker system prune
```

What this does: Removes all stopped containers, unused networks, and dangling images. It frees up disk space. Docker will ask you to confirm before deleting anything.

Common Mistakes

1. Forgetting to start Docker Desktop. Docker commands will not work if Docker Desktop is not running. Always check that the whale icon is visible.
2. Not logging out after adding your user to the docker group (Linux). The group change does not take effect until you log out and log back in.
3. Running commands in the wrong terminal. Make sure you are in a regular terminal (Mac/Linux) or PowerShell (Windows), not inside another application.
4. Typing `sudo` before every Docker command on Mac/Windows. You do not need `sudo` with Docker Desktop on Mac or Windows. Only Linux users need `sudo` (unless they added themselves to the docker group).
5. Ignoring the Docker Desktop service agreement. Docker Desktop requires you to accept the terms of service. The application will not work until you do.

Best Practices

1. Keep Docker Desktop updated. New versions fix bugs, add features, and patch security issues. Enable automatic updates if your system supports it.
2. Configure resource limits. Docker Desktop lets you set how much memory and CPU it can use. Go to Settings > Resources. For most development work, 4 GB of memory and 2 CPUs is sufficient.
3. Clean up regularly. Docker images and containers take up disk space. Run `docker system prune` periodically to remove unused data.
4. Use `docker info` to diagnose problems. If something is not working, `docker info` tells you a lot about the state of your Docker installation.

Quick Summary

Installing Docker means installing Docker Desktop (on Mac/Windows) or Docker Engine (on Linux). After installation, you verify it works with `docker --version` and `docker info`. Creating a Docker Hub account gives you access to thousands of pre-built images. Running `docker run hello-world` is the traditional first step that confirms everything is working.

Key Points

- Docker Desktop is the easiest way to install Docker on Mac and Windows.
- Linux users can install Docker Engine directly from Docker's repository.
- `docker --version` checks that Docker is installed.
- `docker info` shows detailed information about your Docker installation.
- `docker run hello-world` downloads and runs a test container.
- Docker Hub is the default image registry. Create a free account for better access.
- When you run an image for the first time, Docker downloads it from Docker Hub automatically.
- Docker Desktop provides a graphical interface, but everything can be done from the command line.

Practice Questions

1. What command do you use to check which version of Docker is installed?
2. When you run `docker run hello-world`, Docker says "Unable to find image locally." What does Docker do next?
3. What is WSL 2 and why does Docker need it on Windows?
4. After running `sudo usermod -aG docker $USER` on Linux, why do you need to log out and log back in?
5. What does the `docker system prune` command do? When would you use it?

Exercises

Exercise 1: Verify Your Installation

Run these three commands and write down the output:

```bash
docker --version
docker info
docker run hello-world
```

If any command fails, use the troubleshooting section to fix the issue.

Exercise 2: Explore Docker Hub

1. Log in to Docker Hub at https://hub.docker.com.
2. Search for the `nginx` image.
3. Find out how many pulls the official Nginx image has.
4. Look at the tags. What is the difference between `nginx:latest` and `nginx:alpine`?

Exercise 3: Explore Docker Desktop

1. Open Docker Desktop.
2. Go to the Containers tab. Do you see the `hello-world` container? What is its status?
3. Go to the Images tab. Do you see the `hello-world` image? How large is it?
4. Go to Settings > Resources. How much memory and how many CPUs are allocated to Docker?

What Is Next?

Your Docker installation is working. You have run your first container. In the next chapter, we will run real containers -- a web server, an interactive Linux shell, and more. You will learn how to control containers: start them, stop them, name them, and clean them up.

Chapter 3: Your First Real Container

What You Will Learn

- How to run a real web server in a container (Nginx)
- How to run containers in the background (detached mode)
- How to map ports so you can access containers from your browser
- How to list, stop, and remove containers
- How to run interactive containers (like a Linux shell)
- How to name your containers
- How to start and restart stopped containers

Why This Chapter Matters

In Chapter 2, you ran `hello-world`. It printed a message and stopped. That was a test.

Now it is time to do something real. You are going to run a web server inside a container. You will open it in your browser. You will control it -- start it, stop it, remove it. These are the commands you will use every single day when working with Docker.

Think of this chapter as your first driving lesson. You are going to start the car, drive around the block, park, and turn off the engine. Simple, but essential.

Running a Web Server with Nginx

Nginx (pronounced "engine-x") is one of the most popular web servers in the world. Let us run it in a container.

The Basic Run

```bash
docker run nginx
```

Expected output:

```
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
2d429b9e73a6: Pull complete
20c8b3871098: Pull complete
06da587a7970: Pull complete
...
Digest: sha256:56b388b0d79c738f4cf...
Status: Downloaded newer image for nginx:latest
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to
perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
...
2024/01/15 10:30:00 [notice] 1#1: start worker process 36
```

What happened:

1. Docker could not find the Nginx image on your computer.
2. Docker downloaded it from Docker Hub.
3. Docker created a container from the image.
4. Docker started the container.
5. Nginx started running inside the container.

But there is a problem. Your terminal is stuck. It is showing Nginx log messages. You cannot type anything else. The container is running in the foreground.

Press `Ctrl + C` to stop the container and get your terminal back.

Detached Mode: Running in the Background

You do not want your terminal to be taken over by a container. You want containers to run in the background, like background music. This is called detached mode.

```bash
docker run -d nginx
```

Expected output:

```
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6a7b8c9d0
```

Line-by-line explanation:

- `docker run` -- Create and start a container
- `-d` -- Run in detached mode (in the background)
- `nginx` -- The image to use
- The long string is the container ID. It is a unique identifier for this container. Think of it like a license plate for your container.

Your terminal is free. The container is running in the background. But how do you see it in your browser?

Port Mapping: Connecting the Outside World to Your Container

Right now, Nginx is running inside the container. But the container is isolated. It is like a room with no door. The web server is running, but nothing outside the container can reach it.

You need to map a port. A port is like a door number. Web servers listen on port 80. You need to connect a port on your computer to port 80 inside the container.

```bash
docker run -d -p 8080:80 nginx
```

Expected output:

```
f4e3d2c1b0a9876543210fedcba9876543210fedcba9876543210fedcba987654
```

Line-by-line explanation:

- `-p 8080:80` -- Map port 8080 on your computer to port 80 in the container
- The format is `-p HOST_PORT:CONTAINER_PORT`
- `8080` is the port on YOUR computer (the host)
- `80` is the port INSIDE the container (where Nginx listens)

```
Your Computer (Host)                    Container
+-------------------+                  +-------------------+
|                   |   port mapping   |                   |
|  Port 8080  ------+------------------+------  Port 80    |
|                   |                  |                   |
|  Browser visits   |                  |   Nginx listens   |
|  localhost:8080   |                  |   on port 80      |
+-------------------+                  +-------------------+
```

Now open your web browser and go to:

```
http://localhost:8080
```

You should see the Nginx welcome page! It says "Welcome to nginx!" This web page is being served from inside your Docker container.

Why Not Use Port 80 on Both Sides?

You could. This works:

```bash
docker run -d -p 80:80 nginx
```

Then you would visit `http://localhost` (port 80 is the default for web browsers). But port 80 might already be used by another application on your computer. Using 8080 avoids conflicts.

Running Multiple Containers on Different Ports

You can run multiple Nginx containers at the same time, each on a different port:

```bash
docker run -d -p 8081:80 nginx
docker run -d -p 8082:80 nginx
```

Now you have three web servers running:

- `http://localhost:8080` -- First Nginx
- `http://localhost:8081` -- Second Nginx
- `http://localhost:8082` -- Third Nginx

Each container is independent. They do not know about each other.

```
Your Computer
+-----------------------------------------+
```

```
|                                          |
|  Port 8080 ----> Container 1 (Nginx)     |
|  Port 8081 ----> Container 2 (Nginx)     |
|  Port 8082 ----> Container 3 (Nginx)     |
|                                          |
+------------------------------------------+
```

Listing Containers: docker ps

How do you see which containers are running? Use `docker ps`.

```bash
docker ps
```

Expected output:

```
CONTAINER ID   IMAGE   COMMAND                  CREATED         STATUS         PORTS    ...
f4e3d2c1b0a9   nginx   "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes   0.0.0.0:...
a1b2c3d4e5f6   nginx   "/docker-entrypoint.…"   5 minutes ago   Up 5 minutes   80/tcp   ...
```

Column explanations:

Column	Meaning
`CONTAINER ID`	The first 12 characters of the full container ID
`IMAGE`	The image the container was created from
`COMMAND`	The command running inside the container
`CREATED`	When the container was created
`STATUS`	Whether it is running and for how long
`PORTS`	Port mappings (host -> container)
`NAMES`	A random name Docker assigned

Notice the `NAMES` column. Docker gives each container a random name like `eager_pike` or `zen_hopper`. These names are fun, but not very useful. We will learn to give our own names soon.

Seeing All Containers (Including Stopped Ones)

`docker ps` only shows running containers. To see ALL containers, including stopped ones:

```bash
docker ps -a
```

Expected output:

```
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          ...
f4e3d2c1b0a9   nginx         "/docker-entrypoint.…"   5 minutes ago    Up 5 minutes    ...
a1b2c3d4e5f6   nginx         "/docker-entrypoint.…"   8 minutes ago    Up 8 minutes    ...
b3c4d5e6f7g8   hello-world   "/hello"                 20 minutes ago   Exited (0) 20 mi...
```

What `-a` means: The `-a` flag stands for "all." It shows every container, including ones that have stopped. The `hello-world` container from Chapter 2 is here too. Its status is "Exited (0)," which means it finished running successfully (exit code 0 means success).

Stopping Containers: docker stop

To stop a running container, use `docker stop` followed by the container ID or name.

```bash
docker stop f4e3d2c1b0a9
```

Expected output:

```
f4e3d2c1b0a9
```

Docker prints the container ID to confirm it was stopped.

What this does: Sends a polite "please stop" signal to the container. The container has 10 seconds to shut down gracefully. If it does not stop in time, Docker forces it to stop.

You can also use the container name:

```bash
docker stop eager_pike
```

You do not need to type the full container ID. The first few characters are enough, as long as they are unique:

```bash
docker stop f4e
```

Tip: If `f4e` matches only one container, Docker knows which one you mean. If it matches multiple containers, Docker will tell you it is ambiguous.

Verify the Container Stopped

```bash
docker ps
```

The stopped container should no longer appear in the list. Use `docker ps -a` to see it with a status of "Exited."

Removing Containers: docker rm

Stopping a container does not delete it. It is like turning off a car. The car still exists. You can start it again later.

To actually delete a container:

```bash
docker rm f4e3d2c1b0a9
```

Expected output:

```
f4e3d2c1b0a9
```

What this does: Permanently deletes the container. It is gone. You cannot start it again.

Important: You can only remove stopped containers. If you try to remove a running container, Docker will refuse:

```bash
docker rm eager_pike
```

Error output:

```
Error response from daemon: You cannot remove a running container f4e3d2c1b0a9.
Stop the container before attempting removal or force remove.
```

Force Remove a Running Container

If you want to stop AND remove in one step:

```bash
docker rm -f eager_pike
```

The `-f` flag means force. It stops the container immediately and then removes it.

Remove All Stopped Containers

Over time, you will have many stopped containers. Clean them all up at once:

```bash
docker container prune
```

Expected output:

```
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
b3c4d5e6f7g8
a1b2c3d4e5f6

Total reclaimed space: 5.12kB
```

Stop and Remove: A Quick Workflow

Here is a common workflow you will use constantly:

```bash
# 1. Run a container in the background with port mapping
docker run -d -p 8080:80 nginx

# 2. Check that it is running
docker ps
```

```
# 3. Visit http://localhost:8080 in your browser

# 4. When you are done, stop the container
docker stop <container_id>

# 5. Remove the container
docker rm <container_id>
```

Or do steps 4 and 5 together:

```bash
docker rm -f <container_id>
```

There is also a handy flag to automatically remove a container when it stops:

```bash
docker run -d -p 8080:80 --rm nginx
```

The `--rm` flag tells Docker: "When this container stops, delete it automatically." This is great for temporary containers.

Interactive Containers: docker run -it

Not every container runs a web server. Sometimes you want to go INSIDE a container and look around. This is called running an interactive container.

```bash
docker run -it alpine sh
```

Expected output:

```
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
4abcf2066143: Pull complete
Digest: sha256:c5b1261d6d3e43071626931fc004f70149baeba2c8ec672bd4f27761f8e1ad6b
Status: Downloaded newer image for alpine:latest
/ #
```

Line-by-line explanation:

- `docker run` -- Create and start a container

- `-i` -- Interactive mode. Keep the input stream open so you can type commands.
- `-t` -- TTY mode. Give you a terminal with a command prompt.
- `-it` -- The two flags combined. Almost always used together.
- `alpine` -- A very small Linux distribution (about 7 MB). Perfect for experimenting.
- `sh` -- The command to run inside the container. `sh` is a shell (a command-line interface).

You are now INSIDE the container! The `/ #` prompt means you are the root user inside the Alpine Linux container.

Explore Inside the Container

Try some commands:

```bash
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run
sbin   srv    sys    tmp    usr    var
```

This is the file system inside the container. It is a tiny Linux system.

```bash
/ # cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.19.0
PRETTY_NAME="Alpine Linux v3.19"
```

You are running Alpine Linux inside a container. Your host computer could be Mac, Windows, or any Linux distribution. The container has its own operating system.

```bash
/ # echo "Hello from inside a container!"
Hello from inside a container!
```

```bash
/ # whoami
root
```

You are the root user (administrator) inside this container. But this does not give you root access to your host computer. The container is isolated.

Exit the Container

Type `exit` to leave the container:

```bash
/ # exit
```

You are back to your regular terminal. The container has stopped because the shell (`sh`) was the main process, and when it ended, the container stopped too.

Interactive Containers with Other Images

You can run interactive sessions with any image. Here is Ubuntu:

```bash
docker run -it ubuntu bash
```

Expected output:

```
root@a1b2c3d4e5f6:/#
```

Now you have a full Ubuntu Linux environment. You can install software, run scripts, and experiment without affecting your host computer:

```bash
root@a1b2c3d4e5f6:/# apt-get update
root@a1b2c3d4e5f6:/# apt-get install -y curl
root@a1b2c3d4e5f6:/# curl --version
curl 8.5.0 (x86_64-pc-linux-gnu)...
```

Type `exit` to leave.

Important: Anything you install inside a container is lost when the container is removed. Containers are ephemeral (temporary by design). We will learn how to persist data in a later chapter.

Naming Your Containers: --name

Docker gives random names to containers. Names like `eager_pike` and `zen_hopper` are hard to remember. You can choose your own name.

```bash
docker run -d -p 8080:80 --name my-web-server nginx
```

Expected output:

```
c9d8e7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8
```

Now check the container:

```bash
docker ps
```

Expected output:

```
CONTAINER ID   IMAGE   COMMAND                  CREATED         STATUS         PORTS   ...
c9d8e7f6a5b4   nginx   "/docker-entrypoint.…"   5 seconds ago   Up 4 seconds   0.0.0.0:...
```

The `NAMES` column now shows `my-web-server`. Much easier to remember.

You can use this name everywhere you would use a container ID:

```bash
docker stop my-web-server
docker start my-web-server
docker rm my-web-server
```

Naming Rules

- Names must be unique. You cannot have two containers with the same name.
- Names can contain letters, numbers, hyphens, and underscores.
- Names cannot start with a hyphen.
- If you try to create a container with a name that already exists, Docker will refuse.

```bash
docker run -d --name my-app nginx
docker run -d --name my-app nginx
```

Error output:

```
docker: Error response from daemon: Conflict. The container name "/my-app"
is already in use by container "c9d8e7f6a5b4". You have to remove (or rename)
that container to be able to reuse that name.
```

To fix this, remove the old container first:

```bash
docker rm -f my-app
docker run -d --name my-app nginx
```

Starting and Restarting Containers

When you stop a container, it is not deleted. It still exists. You can start it again.

Starting a Stopped Container

```bash
# First, stop a container
docker stop my-web-server

# Check that it is stopped
docker ps -a
```

Expected output:

```
CONTAINER ID   IMAGE   COMMAND                  CREATED         STATUS                  ...
c9d8e7f6a5b4   nginx   "/docker-entrypoint.…"   10 minutes ago  Exited (0) 5 seconds ag...
```

Now start it again:

```bash
docker start my-web-server
```

Expected output:

```
my-web-server
```

The container is running again with the same settings (same ports, same name).

Restarting a Container

`docker restart` stops and starts a container in one command:

```bash
docker restart my-web-server
```

Expected output:

```
my-web-server
```

This is useful when a container is misbehaving and you want to give it a fresh start without reconfiguring everything.

The Difference Between run, start, and restart

Command	What It Does
`docker run`	Creates a NEW container from an image and starts it
`docker start`	Starts an EXISTING stopped container
`docker restart`	Stops and starts an EXISTING container

This is a common point of confusion. `docker run` always creates a new container. If you want to start an existing one, use `docker start`.

```
docker run = Create + Start (new container)
docker start = Start (existing container)
docker restart = Stop + Start (existing container)
```

Viewing Container Logs

When a container runs in the background, you might want to see what it is doing. Use `docker logs`.

```bash
docker logs my-web-server
```

Expected output:

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to
perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
...
```

```
2024/01/15 10:30:00 [notice] 1#1: start worker processes
2024/01/15 10:30:00 [notice] 1#1: start worker process 29
```

Follow Logs in Real Time

```bash
docker logs -f my-web-server
```

The `-f` flag means follow. It shows new log entries as they appear, like `tail -f` on Linux. Press `Ctrl + C` to stop following.

Show Only the Last Few Lines

```bash
docker logs --tail 5 my-web-server
```

This shows only the last 5 lines. Useful when a container has been running for a long time and has thousands of log lines.

A Real-World Example: Python Web App

Let us run a Python application in a container. Docker Hub has a simple Python HTTP server we can use.

```bash
docker run -d -p 9000:8000 --name python-app python:3.12-slim \
  python -m http.server 8000
```

Line-by-line explanation:

- `-d` -- Run in the background
- `-p 9000:8000` -- Map your port 9000 to the container's port 8000
- `--name python-app` -- Name it `python-app`
- `python:3.12-slim` -- Use the Python 3.12 image (slim version, which is smaller)
- `python -m http.server 8000` -- The command to run inside the container. This starts Python's built-in web server on port 8000.

Now visit `http://localhost:9000` in your browser. You will see a directory listing. This is Python's simple HTTP server running inside a container.

To clean up:

```bash
docker rm -f python-app
```

A Real-World Example: Node.js

Let us run a quick Node.js example too:

```bash
docker run -d -p 3000:3000 --name node-app node:20-slim \
  sh -c "echo 'const http = require(\"http\"); http.createServer((req, res) => { res.en...
```

What this does: Creates a tiny Node.js web server that responds with "Hello from Node.js in Docker!" on port 3000.

Visit `http://localhost:3000` in your browser to see the message.

To clean up:

```bash
docker rm -f node-app
```

Container Lifecycle Diagram

Here is the complete lifecycle of a container:

```
                    docker run
                        |
                        v
                  +-----------+
                  |  CREATED  |
                  +-----------+
                        |
                        | (auto-starts)
                        v
```

```
                  +-----------+          docker stop
                  |  RUNNING  | ------------------------> +-----------+
                  +-----------+                           |  STOPPED  |
                        ^                                 +-----------+
                        |                                       |
                        | docker start                          |
                        | docker restart                        |
                        +---------------------------------------+
                                                                |
                                                                | docker rm
                                                                v
                                                          +-----------+
                                                          |  REMOVED  |
                                                          +-----------+
                                                          (gone forever)
```

Command Reference

Here is a summary of every command from this chapter:

Command	What It Does
`docker run IMAGE`	Create and start a container
`docker run -d IMAGE`	Run in background (detached)
`docker run -p HOST:CONTAINER IMAGE`	Map ports
`docker run -it IMAGE COMMAND`	Run interactively
`docker run --name NAME IMAGE`	Give the container a name
`docker run --rm IMAGE`	Auto-remove when stopped
`docker ps`	List running containers
`docker ps -a`	List ALL containers
`docker stop CONTAINER`	Stop a container
`docker start CONTAINER`	Start a stopped container
`docker restart CONTAINER`	Restart a container
`docker rm CONTAINER`	Remove a stopped container
`docker rm -f CONTAINER`	Force remove (even if running)

`docker logs CONTAINER`	View container logs
`docker logs -f CONTAINER`	Follow logs in real time
`docker container prune`	Remove all stopped containers

Common Mistakes

1. Forgetting the `-d` flag. Running `docker run nginx` without `-d` will lock your terminal. Always use `-d` for long-running containers like web servers.
2. Getting the port order wrong. It is `-p HOST:CONTAINER`, not `-p CONTAINER:HOST`. The host port comes first. Remember: "Outside:Inside."
3. Using `docker run` instead of `docker start`. `docker run` creates a NEW container every time. If you already have a stopped container, use `docker start` to restart it.
4. Forgetting to remove old containers. Stopped containers take up disk space. Use `docker ps -a` to see them and `docker container prune` to clean them up.
5. Port conflicts. If port 8080 is already in use, Docker will show an error. Choose a different port. Common alternatives: 8081, 8082, 9000, 3000.

Best Practices

1. Always name your containers. Use `--name` so you can easily reference them. Names like `my-web-server` are much better than `eager_pike`.
2. Use `--rm` for temporary containers. If you are just experimenting, use `--rm` so the container is automatically deleted when it stops.
3. Use `-d` for servers. Web servers, databases, and other long-running services should run in detached mode.
4. Use `-it` for exploration. When you want to explore an image or debug something, use interactive mode with a shell.
5. Clean up regularly. Run `docker ps -a` to see stopped containers. Run `docker container prune` to remove them.

Quick Summary

You can run containers in the foreground or background (detached mode with `-d`). Port mapping (`-p HOST:CONTAINER`) lets your browser connect to services inside containers. `docker ps` lists running containers. `docker stop` and `docker rm` control the container lifecycle. Interactive containers (`-it`) let you explore and run commands inside a container. Naming containers (`--name`) makes them easier to manage.

Key Points

- `docker run -d` runs a container in the background so your terminal stays free.
- `-p 8080:80` maps port 8080 on your computer to port 80 inside the container.
- `docker ps` shows running containers. `docker ps -a` shows all containers.
- `docker stop` gracefully stops a container. `docker rm` deletes it.
- `docker run -it alpine sh` gives you an interactive shell inside a container.
- `--name` lets you give a container a human-readable name.
- `docker start` restarts a stopped container. `docker run` creates a new one.
- `--rm` automatically removes a container when it stops.
- Containers are ephemeral. Anything installed inside a container is lost when it is removed.

Practice Questions

1. What is the difference between running a container with and without the `-d` flag?
2. You run `docker run -d -p 3000:80 nginx`. What URL would you visit in your browser to see the Nginx welcome page?
3. What is the difference between `docker stop` and `docker rm`? Can you remove a running container?
4. You run `docker run -it ubuntu bash` and install Python inside the container. You type `exit`, then `docker start` the container. Is Python still installed? What if you `docker rm` the container and `docker run` a new one?
5. Why would you use the `--name` flag when running a container?

Exercises

Exercise 1: Web Server Challenge

1. Run an Nginx container in the background on port 9090.
2. Give it the name `exercise-web`.
3. Verify it is running with `docker ps`.
4. Visit it in your browser.
5. View its logs.
6. Stop it.
7. Start it again (do not use `docker run`).
8. Finally, remove it.

Write down every command you used.

Exercise 2: Multiple Containers

1. Run three Nginx containers in the background on ports 8001, 8002, and 8003.
2. Name them `web-1`, `web-2`, and `web-3`.
3. Use `docker ps` to verify all three are running.
4. Stop only `web-2`.
5. Use `docker ps` and `docker ps -a` to see the difference.
6. Remove all three containers.

Exercise 3: Interactive Exploration

1. Run an interactive Alpine container with `docker run -it alpine sh`.
2. Inside the container, run `uname -a` to see the operating system information.
3. Run `ls /` to see the root file system.
4. Create a file: `echo "I was here" > /tmp/myfile.txt`
5. Verify the file exists: `cat /tmp/myfile.txt`
6. Exit the container.
7. Start a NEW Alpine container with `docker run -it alpine sh`.
8. Check if `/tmp/myfile.txt` exists. Why or why not?

What Is Next?

You can now run, stop, and manage containers. But how do containers actually work? What happens behind the scenes? In the next chapter, we will look under the hood and understand images, layers, namespaces, and cgroups -- the technology that makes containers possible.

Chapter 4: How Containers Work

What You Will Learn

- The difference between images and containers (with multiple analogies)
- How image layers work and why they matter
- What a union filesystem is (simplified)
- How namespaces provide isolation
- How cgroups limit resources
- Why containers start faster and use fewer resources than virtual machines

Why This Chapter Matters

In the previous chapters, you ran containers. You started them, stopped them, and removed them. But you treated Docker like a black box. You typed commands and things happened.

Now it is time to open the box and look inside.

Understanding how containers work will help you:

- Debug problems when things go wrong
- Make better decisions about how to build and run containers
- Have intelligent conversations with your team about container architecture
- Understand why certain Docker commands exist

You do not need to be a kernel engineer to understand this chapter. We will use simple analogies and clear diagrams. Let us look under the hood.

Images vs Containers

This is the most important concept in Docker. If you understand this, everything else becomes easier.

Analogy 1: Recipe vs Cake

An image is like a recipe. A container is like a cake made from that recipe.

- The recipe (image) describes what ingredients to use and how to combine them.
- The cake (container) is the actual thing you eat.
- You can make many cakes from one recipe.
- Each cake is independent. Eating one cake does not affect the others.
- The recipe itself never changes when you bake a cake.

```
      Image (Recipe)                    Containers (Cakes)
+--------------------+          +--------+  +--------+  +--------+
| - Python 3.12      |   run    |  Cake  |  |  Cake  |  |  Cake  |
| - Flask library    | ------> |   1    |  |   2    |  |   3    |
| - Your app code    |   run    |        |  |        |  |        |
| - Config files     | ------> | Running|  |Running |  |Stopped |
+--------------------+   run   +--------+  +--------+  +--------+
                        ------>
  One image.                     Many containers.
  Never changes.                 Each is independent.
```

Analogy 2: Class vs Object

If you know programming, this analogy will click immediately.

An image is like a class in object-oriented programming. A container is like an object (an instance of that class).

```python
# The Image (like a class definition)
class WebServer:
    python_version = "3.12"
    framework = "Flask"
    code = "app.py"

# Containers (like objects created from the class)
server_1 = WebServer()   # docker run my-app
server_2 = WebServer()   # docker run my-app
server_3 = WebServer()   # docker run my-app
```

```java
// In Java terms:
// Image = the .class file (blueprint)
// Container = the object in memory (instance)

WebServer server1 = new WebServer();  // docker run my-app
WebServer server2 = new WebServer();  // docker run my-app
```

Each container has its own state. Changing one container does not affect the others, just like changing one object does not affect other objects of the same class.

Analogy 3: Stamp vs Impression

An image is like a rubber stamp. A container is like the impression the stamp makes on paper.

- One stamp can make many impressions.
- Each impression is on a different piece of paper.
- Changing one impression does not change the stamp.
- The stamp itself remains the same.

The Technical Reality

An image is a read-only template. It contains:

- An operating system (usually a minimal Linux distribution)
- Your application code
- All libraries and dependencies
- Configuration files
- Instructions for how to start the application

A container is a running (or stopped) instance of an image. It contains:

- Everything from the image (read-only)
- A thin writable layer on top (for any changes)
- Its own process space
- Its own network interface
- Its own file system view

```
Container
+-----------------------------------+
|  Writable Layer (your changes)    |  <-- Only in the container
+-----------------------------------+
|  Image Layer 3 (app code)         |  <-- Read-only, from image
+-----------------------------------+
|  Image Layer 2 (libraries)        |  <-- Read-only, from image
```

```
+-----------------------------------+
|  Image Layer 1 (base OS)          |  <-- Read-only, from image
+-----------------------------------+
```

Image Layers: Building Blocks

Docker images are not single, monolithic files. They are made of layers, stacked on top of each other. This is one of Docker's most clever design decisions.

What Is a Layer?

A layer is a set of file system changes. Think of it like a transparent sheet of paper.

Imagine you are creating a picture using transparent overlays:

1. Layer 1: Draw the background (a blue sky). This is the base operating system.
2. Layer 2: Add some trees. These are the libraries your app needs.
3. Layer 3: Add a house. This is your application code.

When you stack them together, you see the complete picture.

```
Layer 3: App Code          +-----+
                           |house|
Layer 2: Libraries    +---+|     |+---+
                      |tree||     ||tree|
Layer 1: Base OS   ===+===++=====++===+=====
                      blue sky background

Stacked together: A complete scene
```

How Layers Work in Docker

When you build a Docker image, each instruction creates a new layer:

```
Instruction 1: Start with Ubuntu          -> Layer 1 (Ubuntu base, ~75 MB)
Instruction 2: Install Python             -> Layer 2 (Python files, ~50 MB)
Instruction 3: Install Flask library      -> Layer 3 (Flask files, ~5 MB)
Instruction 4: Copy your application code -> Layer 4 (Your code, ~1 MB)
```

```
+----------------------------------------+
|  Layer 4: Your app code (1 MB)         |
+----------------------------------------+
|  Layer 3: Flask library (5 MB)         |
+----------------------------------------+
|  Layer 2: Python (50 MB)               |
+----------------------------------------+
|  Layer 1: Ubuntu base (75 MB)          |
+----------------------------------------+
Total image size: ~131 MB
```

Why Layers Matter: Sharing and Caching

Here is the brilliant part. Layers are shared between images.

Suppose you have two applications:

- App A uses Python 3.12 + Flask
- App B uses Python 3.12 + Django

Both apps need Python 3.12. Docker does not store Python twice. Both images share the same Python layer.

```
App A Image              App B Image
+----------------+      +----------------+
| Flask + App A  |      | Django + App B |
+----------------+      +----------------+
|   Python 3.12  | <--- SHARED ---> |   Python 3.12  |
+----------------+      +----------------+
|   Ubuntu base  | <--- SHARED ---> |   Ubuntu base  |
+----------------+      +----------------+

Without sharing: 131 MB + 135 MB = 266 MB
With sharing:    131 MB + 9 MB   = 140 MB (Python & Ubuntu stored once)
```

This saves enormous amounts of disk space. If you have 10 Python applications, the Python layer is stored only once.

Layers also make downloads faster. If you already have the Python layer and you download a new image that uses Python, Docker skips the Python layer. It only downloads the layers you do not have.

```bash
# Downloading App A (first time, everything is new)
Pulling from library/app-a
a1b2c3: Pull complete    # Ubuntu layer
d4e5f6: Pull complete    # Python layer
```

```
g7h8i9: Pull complete    # Flask layer
j0k1l2: Pull complete    # App A code

# Downloading App B (shares layers with App A)
Pulling from library/app-b
a1b2c3: Already exists   # Ubuntu layer (already have it!)
d4e5f6: Already exists   # Python layer (already have it!)
m3n4o5: Pull complete    # Django layer (new)
p6q7r8: Pull complete    # App B code (new)
```

Union Filesystem: Seeing Layers as One

You might wonder: if an image is made of multiple layers, how does the container see one file system?

The answer is a union filesystem (also called an overlay filesystem).

A union filesystem takes multiple layers and presents them as a single, unified view. It is like stacking transparent sheets. You see through all the layers at once, and the result looks like one picture.

How It Works

```
What Docker stores:              What the container sees:
(separate layers)                (one unified view)

Layer 3: /app/server.py          /
Layer 2: /usr/bin/python         ├── app/
Layer 1: /bin/sh, /etc/...       │   └── server.py
                                 ├── usr/
         UNION FILESYSTEM        │   └── bin/
              ------>            │       └── python
                                 ├── bin/
                                 │   └── sh
                                 └── etc/
                                     └── ...
```

The container does not know about layers. It sees a normal file system. You can `ls`, `cd`, `cat`, and use any command as if everything is in one place.

The Writable Layer

When a container modifies a file, the change goes into a thin writable layer on top. The original image layers are never changed.

This is called copy-on-write. When a container needs to change a file from an image layer, Docker copies that file into the writable layer first. Then the container modifies the copy.

```
Container's View:

+-----------------------------------+
|  Writable Layer                   |  <-- Changes go here
|  (modified /etc/config.txt)       |
+-----------------------------------+
|  Image Layer 3 (app code)         |  <-- Read-only
+-----------------------------------+
|  Image Layer 2 (libraries)        |  <-- Read-only
+-----------------------------------+
|  Image Layer 1 (base OS)          |  <-- Read-only
|  (original /etc/config.txt)       |
+-----------------------------------+

When the container reads /etc/config.txt, it gets the MODIFIED
version from the writable layer (the top layer wins).
```

Important consequence: When you delete a container, its writable layer is deleted too. Any changes you made inside the container are gone. This is why containers are called ephemeral (temporary).

Namespaces: Isolation Rooms

Containers feel like separate machines. Each container has its own processes, its own network, its own file system. But they all run on the same physical computer with the same kernel.

How? The answer is namespaces.

What Is a Namespace?

A namespace is a Linux kernel feature that creates an isolated view of a system resource. Think of it as a one-way mirror.

Imagine a building with many offices. Each office has a window. But the windows are one-way mirrors. You can see your own office clearly. You cannot see into other offices. And you do not even know other offices exist.

That is what namespaces do for containers.

Types of Namespaces

Docker uses several types of namespaces:

1. PID Namespace (Process Isolation)

Each container has its own process list. Inside the container, the main process has PID 1 (process ID 1). It cannot see processes from other containers or from the host.

```
Host Computer:
PID 1: systemd (init)
PID 100: Docker Engine
PID 200: Container A's process (nginx)
PID 201: Container A's worker
PID 300: Container B's process (python)

Container A sees:              Container B sees:
PID 1: nginx                  PID 1: python
PID 2: worker                 (no other processes)
(no other processes)

Each container thinks it is the only thing running.
```

2. Network Namespace (Network Isolation)

Each container has its own network stack. Its own IP address. Its own ports. Container A can listen on port 80, and Container B can also listen on port 80. They do not conflict because they have different network namespaces.

```
Host Network:  172.17.0.1

Container A Network:  172.17.0.2
  Port 80: nginx

Container B Network:  172.17.0.3
  Port 80: apache

Container C Network:  172.17.0.4
  Port 3000: node app

Each container has its own IP address and its own ports.
Port 80 in Container A is different from port 80 in Container B.
```

3. Mount Namespace (Filesystem Isolation)

Each container has its own view of the filesystem. Container A sees its own files. Container B sees its own files. They cannot see each other's files.

```
Container A's filesystem:     Container B's filesystem:
/                             /
├── app/                      ├── app/
│   └── server.py             │   └── index.js
├── usr/                      ├── usr/
│   └── bin/python             │   └── bin/node
└── etc/                      └── etc/

Completely separate views. Container A cannot see
Container B's files, and vice versa.
```

4. User Namespace (User Isolation)

A process can be root (administrator) inside a container but a regular user on the host. This is important for security.

5. UTS Namespace (Hostname Isolation)

Each container has its own hostname. You can name your container anything without affecting other containers or the host.

Namespaces Summary

```
+---Container A----+    +---Container B----+
|                  |    |                  |
| PID namespace:   |    | PID namespace:   |
|  PID 1: nginx    |    |  PID 1: python   |
|                  |    |                  |
| Net namespace:   |    | Net namespace:   |
|  172.17.0.2      |    |  172.17.0.3      |
|                  |    |                  |
| Mnt namespace:   |    | Mnt namespace:   |
|  /app/server.py  |    |  /app/main.py    |
|                  |    |                  |
| UTS namespace:   |    | UTS namespace:   |
|  web-server      |    |  api-server      |
+------------------+    +------------------+
         |                       |
         |    Shared Kernel      |
         +-----------+-----------+
                     |
              +------+------+
              | Linux Kernel |
              | (one kernel  |
              |  for all)    |
```

```
                +-------------+
```

Cgroups: Resource Limits

Namespaces handle isolation (who can see what). Cgroups handle resource limits (who can use how much).

What Is a Cgroup?

Cgroup stands for control group. It is a Linux kernel feature that limits how much of a resource (CPU, memory, disk, network) a process can use.

The Analogy

Imagine an apartment building with shared utilities:

- Water, electricity, and internet are shared among all apartments.
- Without limits, one apartment could use all the water pressure.
- The building manager sets limits: each apartment gets a maximum amount of water, electricity, and bandwidth.

Cgroups are the building manager for containers.

Memory Limits

You can tell Docker exactly how much memory a container can use:

```bash
docker run -d --memory=512m nginx
```

What this does: The container can use at most 512 megabytes of RAM. If it tries to use more, the kernel can terminate the process inside the container (known as an OOM kill -- Out Of Memory kill).

```
Host Memory: 16 GB
+-----------------------------------------------------+
|                                                     |
|  Container A: max 512 MB  [=====     ]              |
|  Container B: max 1 GB    [==========          ] |
```

```
|  Container C: max 256 MB  [===       ]           |
|                                                  |
|  Remaining: available for host and other apps    |
+--------------------------------------------------+
```

CPU Limits

You can limit how much CPU a container can use:

```bash
docker run -d --cpus=1.5 nginx
```

What this does: The container can use at most 1.5 CPU cores. If the host has 4 cores, this container can use up to 37.5% of the total CPU.

```bash
docker run -d --cpus=0.5 nginx
```

What this does: The container gets at most half of one CPU core.

Why Cgroups Matter

Without cgroups, one misbehaving container could:

- Use all available memory, causing other containers to crash
- Use 100% of the CPU, making other containers slow
- Write to disk so fast that other containers cannot read or write

Cgroups prevent one container from being a "noisy neighbor."

```
Without Cgroups:                 With Cgroups:
+-------------------+           +-------------------+
| Container A:      |           | Container A:      |
|   USES ALL CPU!   |           |   Limited to 1 CPU|
|   100% memory!    |           |   Max 512 MB RAM  |
+-------------------+           +-------------------+
| Container B:      |           | Container B:      |
|   Starving...     |           |   Gets fair share |
|   No resources!   |           |   of resources    |
+-------------------+           +-------------------+
```

Why Containers Are Fast (vs VMs)

Now you understand the pieces. Let us put it all together and see why containers are so much faster than virtual machines.

Virtual Machine Startup

When you start a VM:

1. The hypervisor allocates hardware resources.
2. A complete operating system boots up (kernel, init system, services).
3. The OS loads drivers, starts system services.
4. Finally, your application starts.

This takes minutes.

```
VM Startup Timeline:
0s -------- 30s -------- 60s -------- 90s -------- 120s
|           |            |            |             |
Boot BIOS   Load kernel  Start       Start          App
            & init       services    networking     ready

Total: 1-3 minutes
```

Container Startup

When you start a container:

1. Docker creates namespaces (microseconds).
2. Docker sets up cgroups (microseconds).
3. Docker mounts the union filesystem (microseconds).
4. Docker starts your application process.

That is it. No OS boot. No drivers. No system services. Just your application.

This takes seconds (often less than one second).

```
Container Startup Timeline:
0s -------- 0.1s -------- 0.5s -------- 1s
|           |             |             |
Create      Mount FS      Start app     App
namespaces  + cgroups     process       ready
& network
```

```
Total: Under 1 second
```

Resource Comparison

```
Virtual Machine:                    Container:
+-------------------------+        +-------------------------+
|  Your App               |        |  Your App               |
+-------------------------+        +-------------------------+
|  Libraries              |        |  Libraries              |
+-------------------------+        +-------------------------+
|  Guest OS               |        |  (no guest OS!)         |
|  (1-10 GB)              |        |                         |
|  - Kernel               |        +-------------------------+
|  - System services      |        Uses host kernel directly.
|  - Drivers              |        Total: 10-500 MB
|  - Everything an OS has |
+-------------------------+
|  Hypervisor             |
+-------------------------+
Total: 1-10 GB
```

Side-by-Side Summary

Aspect	Virtual Machine	Container
Startup	Minutes	Seconds (or less)
Size	1-10 GB	10-500 MB
RAM overhead	512 MB - 2 GB per VM	A few MB per container
Isolation	Hardware-level (very strong)	Kernel-level (strong)
OS	Each has its own	Shares host OS kernel
Density	5-20 per server	100s-1000s per server
Boot	Full OS boot	Start a process

The Key Insight

A virtual machine virtualizes hardware. It creates a fake computer with fake CPU, fake RAM, and fake disk. Then it installs a real operating system on this fake hardware.

A container virtualizes the operating system. It creates an isolated environment using the host's kernel. No fake hardware. No second OS. Just isolation.

This is why containers are fast. They skip the most expensive step: booting an operating system.

Putting It All Together

Here is the complete picture of how a container works:

```
+-----------------------------------------------------+
|                   Container                         |
|                                                     |
|  +-----------------------------------------------+ |
|  |  Writable Layer (copy-on-write)               | |
|  +-----------------------------------------------+ |
|  |  Image Layers (read-only, shared)             | |
|  |  [App Code] [Libraries] [Base OS]             | |
|  +-----------------------------------------------+ |
|                                                     |
|  Isolated by:                                       |
|  - PID namespace    (own process list)              |
|  - Network namespace (own IP + ports)               |
|  - Mount namespace   (own filesystem)               |
|  - User namespace    (own user IDs)                 |
|  - UTS namespace     (own hostname)                 |
|                                                     |
|  Limited by:                                        |
|  - Memory cgroup  (max RAM)                         |
|  - CPU cgroup     (max CPU)                         |
|  - I/O cgroup     (max disk speed)                  |
+-----------------------------------------------------+
                         |
                  Uses host kernel
                         |
                +-------+-------+
                |  Linux Kernel |
                +---------------+
                |   Hardware    |
                +---------------+
```

When you type `docker run nginx`:

1. Docker pulls the Nginx image (if not already downloaded).
2. Docker stacks the image layers using a union filesystem.
3. Docker adds a writable layer on top.

4. Docker creates namespaces for process, network, filesystem, user, and hostname isolation.
5. Docker creates cgroups to limit resource usage (if specified).
6. Docker starts the Nginx process inside this isolated environment.

The result: an isolated, resource-controlled environment that starts in under a second. That is a container.

Common Mistakes

1. Thinking containers are small VMs. They are not. There is no guest operating system. There is no hypervisor. Containers are isolated processes sharing the host kernel.
2. Thinking container isolation is the same as VM isolation. VM isolation is stronger because VMs have their own kernel. Container isolation is good for most use cases but not suitable for running untrusted code from unknown sources.
3. Forgetting that container changes are temporary. Changes in the writable layer are lost when the container is removed. Use volumes (covered in a later chapter) to persist data.
4. Confusing image layers with container layers. Image layers are read-only and shared. The container has one additional writable layer. Only the writable layer is unique to that container.
5. Thinking each container has its own kernel. All containers on a host share the same Linux kernel. This is the fundamental difference between containers and VMs.

Best Practices

1. Think of containers as disposable. Design your applications so that containers can be created and destroyed at will. Do not store important data inside containers.
2. Keep images small. Fewer layers and smaller layers mean faster downloads, less disk space, and faster startup. Use minimal base images like Alpine when possible.
3. Set resource limits in production. Always use `--memory` and `--cpus` in production to prevent one container from consuming all resources.
4. Use official base images. Official images are maintained by Docker and the software vendors. They are optimized, secure, and well-tested.
5. Understand the layer cache. When building images, Docker caches layers. Put things that change rarely (like OS packages) in early layers and things that change often (like

your code) in later layers. This makes rebuilds faster.

Quick Summary

Images are read-only blueprints made of stacked layers. Containers are running instances of images with an additional writable layer. Union filesystems combine layers into a single view. Namespaces provide isolation (process, network, filesystem, user, hostname). Cgroups limit resource usage (CPU, memory, I/O). Containers are fast because they share the host kernel and skip the OS boot process that VMs require.

Key Points

- An image is a read-only template (like a recipe or a class). A container is a running instance (like a cake or an object).
- Images are built from layers. Each layer represents a set of filesystem changes.
- Layers are shared between images, saving disk space and download time.
- A union filesystem merges layers into one unified view.
- Each container gets a thin writable layer on top of the image layers (copy-on-write).
- Namespaces isolate containers from each other (processes, network, filesystem, users, hostname).
- Cgroups limit how much CPU, memory, and I/O a container can use.
- Containers share the host kernel. VMs have their own kernel. This is why containers start in seconds and VMs take minutes.

Practice Questions

1. Explain the difference between an image and a container using your own analogy (not one from this chapter).
2. An image has 4 layers. You create 3 containers from this image. How many copies of the image layers exist on disk? Why?

3. What happens when a container modifies a file that comes from an image layer? What mechanism handles this?
4. Name three types of namespaces and explain what each one isolates.
5. You have a server with 16 GB of RAM. You want to run 4 containers, each limited to 2 GB. What Docker flag would you use? What happens if a container tries to use 3 GB?

Exercises

Exercise 1: Observe Image Layers

Run these commands and observe the layers:

```bash
docker pull nginx
docker image inspect nginx
```

Look at the output of `docker image inspect`. Find the "Layers" section. How many layers does the Nginx image have? What does each layer hash represent?

Exercise 2: Test Isolation

Open two terminals. In each one, run an interactive Alpine container:

```bash
# Terminal 1
docker run -it --name box-a alpine sh

# Terminal 2
docker run -it --name box-b alpine sh
```

In box-a, create a file:

```bash
echo "I am box A" > /tmp/identity.txt
```

In box-b, try to read that file:

```bash
cat /tmp/identity.txt
```

What happens? Why?

Exercise 3: Resource Limits

Run a container with memory limits:

```bash
docker run -d --memory=100m --name limited-box nginx
```

Then inspect it:

```bash
docker inspect limited-box | grep -i memory
```

What memory limit is shown? Try running a container with `--memory=10m`. Does Nginx start? What happens?

What Is Next?

You now understand how containers work on the inside. In the next chapter, we will take a closer look at Docker images -- how to find them, inspect them, manage them, and understand tags and digests. Images are the foundation of everything in Docker, and mastering them is the next step in your Docker journey.

Chapter 5: Docker Images

What You Will Learn

- What is inside a Docker image
- How to list images on your computer
- How to download images from Docker Hub
- How tags work (:latest, :3.9, :alpine, and more)
- What image IDs and digests are
- How to inspect image details
- How to remove images
- How to browse Docker Hub effectively
- The difference between official and community images
- How to compare image sizes and choose wisely

Why This Chapter Matters

Images are the foundation of Docker. Every container starts from an image. Every deployment ships an image. Every team shares images.

If images are the ingredients in your kitchen, this chapter teaches you how to shop. Where to find the best ingredients. How to read the labels. How to pick the right version. How to clean out your fridge when it gets full.

You already know that images exist. Now you will learn to manage them like a professional.

What Is Inside a Docker Image?

An image contains everything your application needs to run. Let us look at what a typical Python web application image includes:

```
Docker Image: my-python-app
+-------------------------------------------+
|  Layer 5: Your application code           |
|           app.py, requirements.txt        |
+-------------------------------------------+
|  Layer 4: Python packages                 |
|           Flask, requests, gunicorn       |
+-------------------------------------------+
|  Layer 3: Python runtime                  |
|           python3, pip                    |
+-------------------------------------------+
|  Layer 2: System packages                 |
|           libssl, libffi, gcc             |
+-------------------------------------------+
|  Layer 1: Base operating system           |
|           Debian slim (minimal Linux)     |
+-------------------------------------------+
```

An image does NOT contain:

- A full operating system kernel (it uses the host's kernel)
- Hardware drivers
- A boot loader
- System services like SSH or cron (unless you specifically add them)

This is why images are small compared to virtual machine disks.

Concrete Examples

Here is what common images contain:

Node.js image (`node:20`):

- Debian Linux (base)
- Node.js 20 runtime
- npm package manager
- Common build tools

Python image (`python:3.12`):

- Debian Linux (base)
- Python 3.12 interpreter
- pip package manager
- Common C libraries for building packages

Nginx image (`nginx:latest`):

- Debian Linux (base)
- Nginx web server
- Default configuration files
- Default HTML page

Alpine image (`alpine:latest`):

- Alpine Linux (minimal, ~7 MB)
- BusyBox utilities (basic Linux commands)
- apk package manager
- Almost nothing else

Listing Images: docker images

To see all images stored on your computer:

```bash
docker images
```

Expected output:

```
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    a8758716bb6a   2 weeks ago    187MB
python        3.12      dfb5b0152abc   3 weeks ago    1.01GB
alpine        latest    05455a08881e   4 weeks ago    7.38MB
node          20-slim   b1d4de7d95e2   4 weeks ago    220MB
hello-world   latest    d2c94e258dcb   9 months ago   13.3kB
```

Column explanations:

Column	Meaning
`REPOSITORY`	The name of the image (like `nginx` or `python`)
`TAG`	The version or variant (like `latest` or `3.12`)
`IMAGE ID`	A unique 12-character identifier (short form)
`CREATED`	When the image was built (not when you downloaded it)

`SIZE`	How much disk space the image uses

Alternative Command

You can also use:

```bash
docker image ls
```

This does the same thing. Docker has both older-style commands (`docker images`) and newer-style commands (`docker image ls`). Both work.

Downloading Images: docker pull

When you run `docker run nginx`, Docker automatically downloads the image if you do not have it. But you can also download images manually with `docker pull`.

```bash
docker pull python:3.12
```

Expected output:

```
3.12: Pulling from library/python
6a299ae1ccee: Pull complete
839dbcc8d89f: Pull complete
d3cd37f7e8c1: Pull complete
7c5af6fa66a5: Pull complete
c5f20a7b5a88: Pull complete
Digest: sha256:3966b81808d864099f802080d897cef36c01550472ab3955fdd716d1c665acd6
Status: Downloaded newer image for python:3.12
docker.io/library/python:3.12
```

Line-by-line explanation:

- `3.12: Pulling from library/python` -- Docker is downloading version 3.12 from the official Python repository on Docker Hub.
- `6a299ae1ccee: Pull complete` -- Each line represents a layer being downloaded. The hex string is the layer's unique ID.
- `Digest: sha256:3966b8...` -- A cryptographic hash that uniquely identifies this exact image. Think of it as a fingerprint.

- `Status: Downloaded newer image` -- Confirms the download is complete.
- `docker.io/library/python:3.12` -- The full address of the image. `docker.io` is Docker Hub. `library` means it is an official image.

Why Pull Manually?

You might want to pull an image ahead of time:

- To speed up `docker run` later (no download needed)
- To make sure you have the latest version
- To prepare a machine before deploying containers
- To download during off-peak hours

Understanding Tags

Tags are labels attached to images. They usually indicate a version, a variant, or both.

The :latest Tag

If you do not specify a tag, Docker uses `:latest` by default.

```bash
docker pull nginx
```

This is the same as:

```bash
docker pull nginx:latest
```

Warning: `:latest` does not always mean "the newest version." It is just a tag name. Most image maintainers point it to the newest stable version, but this is a convention, not a guarantee.

Version Tags

Version tags specify an exact version:

```bash
docker pull python:3.12
docker pull python:3.11
docker pull python:3.10
docker pull node:20
docker pull node:18
```

These are the most common tags. They tell you exactly which version you are getting.

Variant Tags

Some images have variants. The most common ones:

`:slim` -- A smaller version with fewer pre-installed packages.

```bash
docker pull python:3.12-slim
```

`:alpine` -- Built on Alpine Linux. Very small. Minimal packages.

```bash
docker pull python:3.12-alpine
docker pull node:20-alpine
```

`:bookworm` or `:bullseye` -- Built on specific Debian Linux releases.

```bash
docker pull python:3.12-bookworm
docker pull python:3.12-bullseye
```

Combining Version and Variant

Tags can combine version and variant information:

```bash
docker pull python:3.12-slim-bookworm
```

This means: Python 3.12, slim variant, on Debian Bookworm.

Tag Comparison Example

Let us see how tags affect image size for the Python image:

```bash
docker pull python:3.12
docker pull python:3.12-slim
docker pull python:3.12-alpine
docker images | grep python
```

Expected output:

```
REPOSITORY   TAG            IMAGE ID       CREATED       SIZE
python       3.12           dfb5b0152abc   3 weeks ago   1.01GB
python       3.12-slim      a1b2c3d4e5f6   3 weeks ago   143MB
python       3.12-alpine    f6e5d4c3b2a1   3 weeks ago   51.8MB
```

```
Image Size Comparison:

python:3.12          [================================================] 1.01 GB
python:3.12-slim     [=======]                                          143 MB
python:3.12-alpine   [===]                                              51.8 MB

The alpine variant is about 20x smaller than the full image!
```

Which Tag Should You Use?

Tag	Use When
`:latest`	Quick experiments, testing (NOT production)
`:3.12` (version)	Production, when you need a specific version
`:3.12-slim`	Production, when you want smaller images
`:3.12-alpine`	When size matters most, and you can handle Alpine's differences

Best practice: Always specify a version tag in production. Never use `:latest` in production because it can change without warning.

```
Bad:   docker run python              # What version? Who knows!
Good:  docker run python:3.12-slim    # Python 3.12, slim variant. Clear.
```

Image IDs and Digests

Every image has two types of identifiers: an ID and a digest.

Image ID

The image ID is a unique identifier generated from the image contents. You see it in `docker images` output:

```
REPOSITORY   TAG      IMAGE ID       CREATED       SIZE
nginx        latest   a8758716bb6a   2 weeks ago   187MB
```

`a8758716bb6a` is the short form. The full ID is much longer:

```bash
docker inspect nginx:latest --format='{{.Id}}'
```

Expected output:

```
sha256:a8758716bb6aa4d90071160d27028fe4eaee7ce8166221a97d30440c8eac2be6
```

The short ID is the first 12 characters of the full ID.

Image Digest

A digest is a cryptographic hash of the image manifest. It uniquely identifies the image on a registry (like Docker Hub).

```bash
docker images --digests
```

Expected output:

```
REPOSITORY  TAG     DIGEST                                                    IM...
nginx       latest  sha256:56b388b0d79c738f4cf51bbaf184a14fa82d...  a8758716bb6a  187MB
```

Why Digests Matter

Tags can change. Today `python:3.12` points to one image. Tomorrow, when a security patch is released, `python:3.12` might point to a different image.

Digests never change. The digest `sha256:3966b818...` will always refer to the exact same image. If you need absolute certainty about which image you are running, use the digest:

```bash
docker pull python@sha256:3966b81808d864099f802080d897cef36c01550472ab3955fdd716d1c665acd6
```

This guarantees you get the exact same image every time, no matter what.

```
Tags can move:                    Digests are permanent:

python:3.12                       python@sha256:abc123...
  |                                |
  v                                v
  Image A (Jan 2024)              Image A (forever)
  |
  v (tag updated!)
  Image B (Feb 2024)              Digest never changes.
                                  It always points to
Tag now points to a               the same image.
different image!
```

Inspecting Images: docker image inspect

Want to see the details of an image? Use `docker image inspect`.

```bash
docker image inspect nginx:latest
```

Expected output (abbreviated):

```json
[
    {
        "Id": "sha256:a8758716bb6a...",
        "Created": "2024-01-10T00:00:00.000000000Z",
        "Architecture": "amd64",
        "Os": "linux",
        "Size": 187365382,
        "Config": {
            "Env": [
                "PATH=/usr/local/sbin:/usr/local/bin:...",
                "NGINX_VERSION=1.25.3"
            ],
            "ExposedPorts": {
```

```
                "80/tcp": {}
            },
            "Cmd": ["nginx", "-g", "daemon off;"]
        },
        "RootFS": {
            "Type": "layers",
            "Layers": [
                "sha256:7292cf786aa8...",
                "sha256:43d8c25e73d4...",
                "sha256:5e99f539e0d2...",
                "sha256:02b80e18f1b4...",
                "sha256:6d1f20c1c6fd...",
                "sha256:29db1f0a7b32...",
                "sha256:e3c2f1048ab6..."
            ]
        }
    }
]
```

Key fields explained:

Field	Meaning
`Id`	The full image ID
`Created`	When the image was built
`Architecture`	What CPU architecture it is built for (amd64, arm64)
`Os`	What operating system (always `linux` for Docker)
`Size`	Image size in bytes
`Env`	Environment variables set in the image
`ExposedPorts`	Ports the application listens on
`Cmd`	The default command that runs when you start a container
`Layers`	List of layer hashes that make up the image

Getting Specific Information

You can extract specific fields:

```bash
# Get the default command
docker image inspect nginx:latest --format='{{.Config.Cmd}}'
```

Output:

```
[nginx -g daemon off;]
```

```bash
# Get the number of layers
docker image inspect nginx:latest --format='{{len .RootFS.Layers}}'
```

Output:

```
7
```

```bash
# Get the architecture
docker image inspect nginx:latest --format='{{.Architecture}}'
```

Output:

```
amd64
```

Removing Images: docker rmi

Images take up disk space. You will want to remove ones you no longer need.

Remove a Specific Image

```bash
docker rmi hello-world
```

Expected output:

```
Untagged: hello-world:latest
Untagged: hello-world@sha256:c41088499908a59aae30...
Deleted: sha256:d2c94e258dcb3c5ac2798d32e1249e42ef01cba4841c2234249495f87264ac5a
Deleted: sha256:ac28800ec8bb38d5c35b49d45a6ac4777544941199075dff8c4eb63e093aa81e
```

Line-by-line explanation:

- `Untagged: hello-world:latest` -- Removes the tag from the image.
- `Untagged: hello-world@sha256:...` -- Removes the digest reference.
- `Deleted: sha256:d2c94e...` -- Deletes the image layer from disk.
- `Deleted: sha256:ac2880...` -- Deletes another layer.

You Cannot Remove an Image That Is in Use

If a container (even a stopped one) was created from an image, Docker will refuse to remove the image:

```bash
docker rmi nginx
```

Error output:

```
Error response from daemon: conflict: unable to remove repository reference
"nginx:latest" (must force) - container a1b2c3d4 is using its referenced image
```

Fix: Remove the container first, then the image:

```bash
docker rm a1b2c3d4
docker rmi nginx
```

Or force it:

```bash
docker rmi -f nginx
```

Remove Multiple Images

```bash
docker rmi python:3.12 node:20 alpine:latest
```

Remove All Unused Images

```bash
docker image prune
```

Expected output:

```
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:f6e5d4c3b2a1...
deleted: sha256:a1b2c3d4e5f6...

Total reclaimed space: 250MB
```

What is a "dangling" image? A dangling image is one that has no tag. This happens when you build a new version of an image with the same tag. The old version loses its tag and becomes dangling.

Remove ALL Unused Images (More Aggressive)

```bash
docker image prune -a
```

This removes ALL images that are not used by at least one container. It reclaims more space but removes more images. You will need to pull them again if you need them.

Browsing Docker Hub

Docker Hub (https://hub.docker.com) is the largest public registry of Docker images. Let us learn how to navigate it effectively.

Searching from the Command Line

```bash
docker search python
```

Expected output:

```
NAME                        DESCRIPTION                                     STARS   OF...
python                      Python is an interpreted, interactive, ...       9456    [OK]
pypy                        PyPy is a fast, compliant alternative ...        380     [OK]
circleci/python             Python is a high-level programming ...           55
cimg/python                 Community-maintained Python images ...           19
bitnami/python              Bitnami container image for Python              26
```

Column explanations:

Column	Meaning
`NAME`	Image name
`DESCRIPTION`	A short description
`STARS`	How many users have "starred" this image (like GitHub stars)
`OFFICIAL`	Whether this is an official image maintained by Docker or the software vendor

Using the Docker Hub Website

The website gives you much more information than the command line:

1. Overview tab: Description, quick start instructions, supported tags.
2. Tags tab: All available versions and variants with their sizes and architectures.
3. Docker Pull Command: A copy-paste command to download the image.
4. Source: Link to the Dockerfile (the recipe used to build the image).
5. Vulnerability scanning: Security reports showing known vulnerabilities.

Official vs Community Images

This is an important distinction. Choose wisely.

Official Images

Official images are curated and maintained by Docker in partnership with the software vendors (or by dedicated maintainers).

How to recognize them:

- The name has no prefix: `python`, `nginx`, `node`, `postgres`
- They show `[OK]` in the OFFICIAL column when you search
- On Docker Hub, they have a "Docker Official Image" badge

Why use them:

- Regularly updated with security patches
- Follow best practices

- Well-documented
- Scanned for vulnerabilities
- Trusted by the community

Examples:

```bash
docker pull python        # Official Python image
docker pull nginx         # Official Nginx image
docker pull node          # Official Node.js image
docker pull postgres      # Official PostgreSQL image
docker pull redis         # Official Redis image
docker pull openjdk       # Official Java image
```

Community Images

Community images are created by individuals or organizations. They have a username or organization prefix.

```bash
docker pull bitnami/python     # Bitnami's Python image
docker pull circleci/python    # CircleCI's Python image
docker pull myuser/myapp       # Someone's custom image
```

When to use them:

- When no official image exists for the software you need
- When a community image has specific features you need (like pre-configured tools)
- When a trusted organization maintains it (like Bitnami, CircleCI, or Google)

When to be careful:

- Unknown authors
- Low star counts
- Images not updated in a long time
- No documentation
- No linked source code (Dockerfile)

```
Choosing an Image:

Official Images                     Community Images
+----------------------+          +----------------------+
| python               |          | randomuser/python    |
| nginx                |          | myorg/custom-nginx   |
| node                 |          | bitnami/node         |
+----------------------+          +----------------------+
```

```
| - Trusted            |          | - May have extras    |
| - Updated regularly  |          | - Quality varies     |
| - Scanned for bugs   |          | - Check the author   |
| - Well documented    |          | - Check last update  |
+----------------------+          +----------------------+
| USE FIRST            |          | USE WHEN NEEDED      |
+----------------------+          +----------------------+
```

Image Size Comparison

Image size matters. Smaller images mean:

- Faster downloads
- Faster deployments
- Less disk space used
- Smaller attack surface (fewer packages means fewer potential vulnerabilities)

Here is a real comparison of common images:

```bash
docker pull ubuntu:22.04
docker pull debian:bookworm-slim
docker pull alpine:latest
docker images
```

Expected output:

```
REPOSITORY   TAG              SIZE
ubuntu       22.04            77.9MB
debian       bookworm-slim    74.8MB
alpine       latest           7.38MB
```

```
Base Image Size Comparison:

ubuntu:22.04         [==========================]           77.9 MB
debian:bookworm-slim [=========================]            74.8 MB
alpine:latest        [===]                                  7.38 MB

Alpine is about 10x smaller than Ubuntu or Debian!
```

Application Image Sizes

The same application can have dramatically different sizes depending on the base image:

```
Node.js Image Variants:

node:20              [================================================]  1.1 GB
node:20-slim         [==========]                                       220 MB
node:20-alpine       [======]                                           135 MB

Python Image Variants:

python:3.12          [================================================]  1.01 GB
python:3.12-slim     [=======]                                          143 MB
python:3.12-alpine   [===]                                              51.8 MB

Java Image Variants:

eclipse-temurin:21   [================================================]  422 MB
eclipse-temurin:21-alpine [========]                                    185 MB
```

When to Use Alpine

Alpine images are the smallest, but there are trade-offs:

Advantages:

- Tiny size (7 MB base)
- Faster to download and deploy
- Smaller attack surface

Disadvantages:

- Uses `musl` instead of `glibc` (a different C library). Some software may not work.
- Uses `apk` instead of `apt` (different package manager). Package names are different.
- Fewer pre-installed tools. You may need to install more things manually.
- Some Python packages need extra compilation steps on Alpine.

Rule of thumb: Start with slim variants. Try Alpine if you need smaller images and are willing to handle compatibility issues.

Checking Disk Space

Over time, images accumulate and use significant disk space. Check how much space Docker is using:

```bash
docker system df
```

Expected output:

```
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          8         3         3.2GB     2.1GB (65%)
Containers      5         2         50MB      30MB (60%)
Local Volumes   3         2         500MB     200MB (40%)
Build Cache     0         0         0B        0B
```

Column explanations:

Column	Meaning
`TOTAL`	Total number of items
`ACTIVE`	Number currently in use by running containers
`SIZE`	Total disk space used
`RECLAIMABLE`	Space you could free up by removing unused items

In this example, you could free up 2.1 GB by removing unused images.

A Complete Image Workflow

Here is a typical workflow for working with images:

```bash
# 1. Search for an image
docker search postgres

# 2. Pull a specific version
docker pull postgres:16

# 3. List your images
docker images

# 4. Inspect the image
docker image inspect postgres:16

# 5. Run a container from it
```

```
docker run -d --name my-db -e POSTGRES_PASSWORD=mysecret postgres:16

# 6. When done, stop and remove the container
docker stop my-db
docker rm my-db

# 7. If you no longer need the image, remove it
docker rmi postgres:16

# 8. Clean up all unused images
docker image prune
```

Command Reference

Command	What It Does
`docker images`	List all images
`docker image ls`	List all images (alternative syntax)
`docker pull IMAGE:TAG`	Download an image
`docker image inspect IMAGE`	Show detailed image information
`docker rmi IMAGE`	Remove an image
`docker rmi -f IMAGE`	Force remove an image
`docker image prune`	Remove dangling images
`docker image prune -a`	Remove all unused images
`docker search TERM`	Search Docker Hub
`docker images --digests`	List images with digests
`docker system df`	Show Docker disk usage

Common Mistakes

1. Using `:latest` in production. The `:latest` tag can change at any time. Always pin to a specific version like `:3.12` or `:3.12-slim` in production environments.
2. Not cleaning up old images. Images accumulate quickly. A developer's machine can easily have 20-50 GB of Docker images. Run `docker image prune` regularly.
3. Choosing the full image when slim would work. The full `python:3.12` image is over 1 GB. The slim version is 143 MB. Unless you specifically need the extra packages, use slim.
4. Pulling images without checking the source. Always verify that an image is official or from a trusted publisher before using it. Unverified images could contain malware.
5. Confusing image ID with container ID. They look similar (both are hex strings) but refer to different things. Use `docker images` for images and `docker ps` for containers.

Best Practices

1. Always specify a tag. Use `python:3.12-slim`, not `python`. This ensures reproducibility. Your container will be the same today, tomorrow, and next year.
2. Use official images as your starting point. Even if you need a custom image, start from an official base. Build on top of `python:3.12-slim`, not some random image.
3. Choose the smallest image that works. Start with `-slim`. Try `-alpine` if you can. Only use the full image if you need specific tools that are not in the slim version.
4. Clean up regularly. Add `docker system prune` to your weekly routine. Or set up automatic cleanup in your CI/CD pipeline.
5. Use digests for critical deployments. When deploying to production, consider using the image digest instead of a tag for absolute certainty about what is running.
6. Check image vulnerabilities. Docker Hub shows vulnerability scans for official images. Review them before using an image in production.

Quick Summary

Docker images are read-only templates stored as layers. You list them with `docker images`, download them with `docker pull`, and remove them with `docker rmi`. Tags identify specific versions and variants (`:latest`, `:3.12`, `:3.12-slim`, `:3.12-alpine`). Image IDs are unique hashes. Digests provide immutable references that never change. Official images are maintained by Docker and software vendors and should be your first choice. Image size varies dramatically by variant -- Alpine images are the smallest but have compatibility

trade-offs.

Key Points

- Images are read-only templates made of layers. Containers are running instances.
- `docker images` lists all images on your computer.
- `docker pull IMAGE:TAG` downloads an image from Docker Hub.
- Tags identify versions and variants: `:latest`, `:3.12`, `:slim`, `:alpine`.
- Always use explicit version tags in production. Never rely on `:latest`.
- Image IDs are unique identifiers. Digests are immutable references.
- `docker image inspect` shows detailed metadata about an image.
- `docker rmi` removes images. `docker image prune` removes unused images.
- Official images are trusted, regularly updated, and your best starting point.
- Slim and Alpine variants are much smaller than full images.
- `docker system df` shows how much disk space Docker is using.

Practice Questions

1. What is the difference between `docker pull python:3.12` and `docker pull python:3.12-alpine`? Which one is smaller and why?
2. Why should you avoid using the `:latest` tag in production? What could go wrong?
3. What is the difference between an image ID and a digest? When would you use a digest instead of a tag?
4. You run `docker rmi nginx` and get an error about a container using the image. What should you do?
5. How would you find out the default command that an image runs when you start a container from it?

Exercises

Exercise 1: Compare Image Sizes

Pull these three variants of the Node.js image:

```bash
docker pull node:20
docker pull node:20-slim
docker pull node:20-alpine
```

Use `docker images` to compare their sizes. Calculate how much smaller the Alpine variant is compared to the full image (as a percentage).

Exercise 2: Inspect and Compare

Inspect the `nginx:latest` and `python:3.12-slim` images:

```bash
docker image inspect nginx:latest
docker image inspect python:3.12-slim
```

For each image, find:

- How many layers does it have?
- What is the default command?
- What environment variables are set?

Exercise 3: Clean Up Challenge

1. Run `docker system df` to see your current disk usage.
2. Pull five different images (your choice).
3. Run `docker system df` again. How much did the disk usage increase?
4. Remove all the images you just pulled.
5. Run `docker image prune` to clean up any dangling images.
6. Run `docker system df` one final time. Is your disk usage back to where it started?

What Is Next?

You now understand Docker images -- how to find them, inspect them, manage them, and choose the right ones. In the next chapter, you will learn how to create your OWN images by writing a Dockerfile. You will package your own application into a Docker image and share it

with the world. That is where Docker starts to become truly powerful.

Chapter 6: Your First Dockerfile

What You Will Learn

By the end of this chapter, you will be able to:

- Explain what a Dockerfile is and why it exists
- Write a Dockerfile from scratch using common instructions
- Understand each instruction: FROM, WORKDIR, COPY, RUN, CMD, ENTRYPOINT, and EXPOSE
- Build a Docker image using `docker build`
- Understand what build context means and how to control it
- Use a `.dockerignore` file to exclude unnecessary files
- Explain how Docker layer caching works and why it matters
- Build and run a complete Node.js application using a Dockerfile

Why This Chapter Matters

Up until now, you have been pulling images that other people created. That is like eating at restaurants -- someone else made the food for you. But what if you want to cook your own meal? What if you have your own application and you want to package it into a Docker image?

That is where the Dockerfile comes in. It is the recipe that tells Docker exactly how to build your custom image. Every professional developer who works with Docker writes Dockerfiles. It is one of the most important skills in modern software development.

Without Dockerfiles, you would have to manually set up every container by hand. Imagine having to install software, copy files, and configure settings every single time you wanted to run your application. A Dockerfile automates all of that into a single, repeatable file.

What Is a Dockerfile?

A Dockerfile is a plain text file that contains a series of instructions. Each instruction tells Docker to perform a specific action, like installing software or copying files. When you run `docker build`, Docker reads these instructions one by one and creates an image.

Think of it like a cooking recipe:

```
RECIPE: Chocolate Cake

START WITH:    A basic cake pan (the base)
ADD:           2 cups flour
ADD:           1 cup sugar
MIX IN:        3 eggs
BAKE AT:       350 degrees for 30 minutes
SERVE:         On a plate with frosting
```

A Dockerfile works the same way:

```
Dockerfile: My Application

START WITH:    A Linux operating system with Node.js installed
SET UP:        A working directory inside the container
COPY:          My application code into the container
INSTALL:       My application dependencies
EXPOSE:        The port my app listens on
RUN:           My application when the container starts
```

Here is what a real Dockerfile looks like:

```dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Do not worry if this looks confusing right now. We will go through every single line in detail.

Key Rules About Dockerfiles

1. The file must be named exactly `Dockerfile` (capital D, no file extension)
2. Each instruction is written in UPPERCASE by convention (FROM, RUN, COPY, etc.)
3. Docker executes instructions from top to bottom, in order

4. Each instruction creates a new layer in the image (more on this later)

The FROM Instruction: Your Starting Point

Every Dockerfile must start with a `FROM` instruction. It tells Docker which base image to use as the foundation.

Real-life analogy: If you are building a house, FROM is like choosing the foundation. You would not pour concrete from scratch every time. Instead, you start with an existing foundation and build on top of it.

Syntax

```dockerfile
FROM image_name:tag
```

Examples

```dockerfile
# Start with Ubuntu Linux
FROM ubuntu:22.04

# Start with a lightweight Linux distribution
FROM alpine:3.18

# Start with Node.js pre-installed
FROM node:18-alpine

# Start with Python pre-installed
FROM python:3.11-slim

# Start with Java pre-installed
FROM eclipse-temurin:17-jre-alpine
```

What Happens Behind the Scenes

When Docker encounters the FROM instruction:

```
+-----------------------------------------------------+
|          What FROM Does                             |
+-----------------------------------------------------+
```

```
|                                                    |
|  FROM node:18-alpine                               |
|       |                                            |
|       v                                            |
|  1. Check if image exists locally                  |
|       |                                            |
|       v                                            |
|  2. If not, pull from Docker Hub                   |
|       |                                            |
|       v                                            |
|  3. Use it as the starting layer                   |
|       |                                            |
|       v                                            |
|  4. All subsequent instructions build on top       |
|                                                    |
+----------------------------------------------------+
```

Choosing a Base Image

The base image you choose matters a lot. Here is a comparison:

```
+-------------------+----------+---------------------------+
| Base Image        | Size     | What Is Included          |
+-------------------+----------+---------------------------+
| ubuntu:22.04      | ~77 MB   | Full Ubuntu OS            |
| debian:12-slim    | ~74 MB   | Minimal Debian OS         |
| alpine:3.18       | ~7 MB    | Tiny Linux OS             |
| node:18           | ~350 MB  | Debian + Node.js          |
| node:18-slim      | ~180 MB  | Slim Debian + Node.js     |
| node:18-alpine    | ~120 MB  | Alpine + Node.js          |
| scratch           | 0 MB     | Completely empty          |
+-------------------+----------+---------------------------+
```

The `-alpine` variants use Alpine Linux as their base. Alpine is a tiny Linux distribution (about 7 MB) that is perfect for containers because it keeps your images small.

The `-slim` variants remove unnecessary packages from the standard image, giving you a middle ground between full and Alpine.

The `scratch` image is completely empty. It is used for compiled languages like Go where the binary includes everything it needs.

The WORKDIR Instruction: Setting Your Working Directory

`WORKDIR` sets the working directory inside the container. All subsequent instructions (like COPY, RUN, and CMD) will execute relative to this directory.

Real-life analogy: WORKDIR is like walking into a specific room in your house before you start working. If you say `WORKDIR /app`, it is like saying "go to the /app room. Everything I do from now on happens in this room."

Syntax

```dockerfile
WORKDIR /path/to/directory
```

Example

```dockerfile
FROM node:18-alpine

# Set the working directory to /app
WORKDIR /app

# Now all paths are relative to /app
# This copies package.json to /app/package.json
COPY package.json .
```

What If the Directory Does Not Exist?

Docker creates it automatically. You do not need to run `mkdir` first.

```dockerfile
# Docker will create /app/src if it does not exist
WORKDIR /app/src
```

Multiple WORKDIR Instructions

You can use WORKDIR multiple times. Relative paths build on the previous WORKDIR:

```dockerfile
WORKDIR /app
# Current directory: /app

WORKDIR src
# Current directory: /app/src

WORKDIR ../config
# Current directory: /app/config
```

The COPY Instruction: Adding Your Files

`COPY` copies files and directories from your computer (the build context) into the image.

Real-life analogy: COPY is like packing items from your desk into a moving box. You pick specific items from your desk (your computer) and place them in the box (the container image).

Syntax

```dockerfile
COPY source destination
```

Examples

```dockerfile
# Copy a single file to the current WORKDIR
COPY package.json .

# Copy a single file to a specific location
COPY package.json /app/package.json

# Copy everything from the current directory
COPY . .

# Copy a specific directory
COPY src/ /app/src/

# Copy multiple files
COPY package.json package-lock.json ./
```

Understanding the Dot (.)

The dot `.` means "current directory." But which current directory?

- On the left side (source): The `.` refers to the build context directory on your computer
- On the right side (destination): The `.` refers to the WORKDIR inside the container

```dockerfile
WORKDIR /app

# Source: everything in the build context on your computer
# Destination: /app/ inside the container
COPY . .
```

The --chown Flag

You can change the ownership of copied files:

```dockerfile
COPY --chown=node:node package.json .
```

This makes the `node` user (instead of root) the owner of the file.

The RUN Instruction: Executing Commands During Build

`RUN` executes a command during the image build process. It is commonly used to install software packages, create directories, or download files.

Real-life analogy: RUN is like following a step in a recipe while cooking. "Mix the ingredients" happens during cooking time, not when you serve the dish. Similarly, RUN commands happen during build time, not when the container starts.

Syntax

There are two forms:

```dockerfile
# Shell form (runs in /bin/sh -c)
RUN command

# Exec form (runs directly, no shell processing)
RUN ["executable", "param1", "param2"]
```

Examples

```dockerfile
# Install a package on Ubuntu/Debian
RUN apt-get update && apt-get install -y curl

# Install a package on Alpine
RUN apk add --no-cache curl

# Install Node.js dependencies
RUN npm install

# Create a directory
RUN mkdir -p /app/logs

# Download a file
RUN curl -o /app/data.json https://example.com/data.json
```

Combining RUN Commands

Each RUN instruction creates a new layer. To keep your image small, combine related commands:

```dockerfile
# BAD: Three separate layers
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y wget

# GOOD: One layer
RUN apt-get update && \
    apt-get install -y curl wget && \
    rm -rf /var/lib/apt/lists/*
```

The backslash `\` at the end of a line means "this command continues on the next line." It makes long commands easier to read.

The `&&` means "run the next command only if the previous one succeeded."

CMD vs ENTRYPOINT: What Runs When the Container Starts

This is where many beginners get confused. Both CMD and ENTRYPOINT define what happens when a container starts, but they work differently.

CMD: The Default Command

`CMD` sets the default command that runs when a container starts. Users can override it easily.

Real-life analogy: CMD is like setting a default radio station in your car. You can always change the station, but if you just turn on the radio without thinking, it plays the default.

```dockerfile
# Shell form
CMD node server.js

# Exec form (preferred)
CMD ["node", "server.js"]
```

Important: There can only be one CMD instruction in a Dockerfile. If you write multiple CMD instructions, only the last one takes effect.

Overriding CMD

When you run a container, you can override the CMD:

```bash
# Uses the default CMD from the Dockerfile
$ docker run my-app

# Overrides CMD with "node test.js"
$ docker run my-app node test.js

# Overrides CMD with a shell
$ docker run -it my-app /bin/sh
```

ENTRYPOINT: The Fixed Command

`ENTRYPOINT` sets a command that always runs. It cannot be easily overridden.

Real-life analogy: ENTRYPOINT is like the ignition system in your car. No matter where you want to go, you always need to turn the key (or push the start button). The destination (CMD) might change, but the ignition (ENTRYPOINT) stays the same.

```dockerfile
ENTRYPOINT ["node"]
CMD ["server.js"]
```

With this setup:

```bash
# Runs: node server.js (ENTRYPOINT + CMD)
$ docker run my-app

# Runs: node test.js (ENTRYPOINT + override)
$ docker run my-app test.js

# Runs: node --version (ENTRYPOINT + override)
$ docker run my-app --version
```

Comparison Table

```
+---------------+---------------------------+---------------------------+
| Feature       | CMD                       | ENTRYPOINT                |
+---------------+---------------------------+---------------------------+
| Purpose       | Default command/args      | Fixed command              |
| Override      | Easy (just add args)      | Requires --entrypoint     |
| Multiple      | Last one wins             | Last one wins              |
| Used for      | Flexible containers       | Single-purpose containers  |
| Common with   | General apps              | CLI tools, wrappers        |
+---------------+---------------------------+---------------------------+
```

Using CMD and ENTRYPOINT Together

The most common pattern is to use both:

```dockerfile
# ENTRYPOINT defines the executable
ENTRYPOINT ["python"]

# CMD provides default arguments
CMD ["app.py"]
```

```
+---------------------------------------------------+
|     How ENTRYPOINT + CMD Work Together            |
+---------------------------------------------------+
|                                                   |
|  ENTRYPOINT: ["python"]   (the program)           |
|  CMD:        ["app.py"]   (default argument)      |
|                                                   |
|  docker run my-app                                |
|  => python app.py                                 |
|                                                   |
|  docker run my-app test.py                        |
|  => python test.py   (CMD is overridden)          |
|                                                   |
|  docker run my-app --version                      |
|  => python --version (CMD is overridden)          |
```

```
|                                                    |
+----------------------------------------------------+
```

When to Use Which?

Use CMD when:

- You want users to easily override the command
- Your container is a general-purpose application
- You are building a development container

Use ENTRYPOINT when:

- You want the container to act as a specific executable
- You are building a CLI tool (like a linter or test runner)
- You always want the same program to run, just with different arguments

Use both when:

- You want a fixed program (ENTRYPOINT) with default arguments (CMD)
- You want users to easily change arguments but not the program

---

The EXPOSE Instruction: Documenting Ports

`EXPOSE` tells Docker (and anyone reading the Dockerfile) which ports the application inside the container listens on.

Real-life analogy: EXPOSE is like putting a sign on a building's door that says "Entrance on Port 3000." It does not actually open the door. It just tells people which door to use.

Important Distinction

`EXPOSE` does not actually publish the port. It is documentation. To actually make the port accessible, you still need the `-p` flag when running the container:

```bash
# EXPOSE 3000 in Dockerfile tells you the app uses port 3000
# But you still need -p to publish it
$ docker run -p 3000:3000 my-app
```

Syntax

```dockerfile
# Expose a single port
EXPOSE 3000

# Expose multiple ports
EXPOSE 3000
EXPOSE 8080

# Expose UDP port
EXPOSE 53/udp
```

Why Bother with EXPOSE?

Even though EXPOSE does not publish ports, it serves important purposes:

1. Documentation: Other developers know which ports the app uses
2. Docker Compose: Some tools use EXPOSE information automatically
3. Best practice: It makes your Dockerfile self-documenting

Building an Image: docker build

Now that you understand the instructions, let us build an image.

The docker build Command

```bash
$ docker build -t image_name:tag path_to_build_context
```

Breaking this down:

- `docker build` -- tells Docker to build an image
- `-t image_name:tag` -- gives the image a name and tag (like a version)
- `path_to_build_context` -- the directory containing the Dockerfile and source files

Examples

```bash
# Build from the current directory, name it "my-app"
$ docker build -t my-app .

# Build with a specific tag
$ docker build -t my-app:1.0 .

# Build with a specific Dockerfile name
$ docker build -t my-app -f Dockerfile.dev .

# Build from a different directory
$ docker build -t my-app ./my-project
```

Understanding Build Output

When you run `docker build`, Docker shows you each step:

```
$ docker build -t my-app .

[+] Building 15.2s (10/10) FINISHED
 => [internal] load build definition from Dockerfile          0.0s
 => [internal] load .dockerignore                             0.0s
 => [internal] load metadata for docker.io/library/node:18    1.2s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:abc   3.4s
 => [2/5] WORKDIR /app                                       0.1s
 => [3/5] COPY package.json .                                 0.1s
 => [4/5] RUN npm install                                     8.3s
 => [5/5] COPY . .                                            0.2s
 => exporting to image                                        1.9s
 => => naming to docker.io/library/my-app                     0.0s
```

Each line starting with `=>` is a build step. The numbers `[1/5]`, `[2/5]`, etc., correspond to the instructions in your Dockerfile.

Build Context: What Docker Can See

The build context is the set of files and directories that Docker can access during the build. When you run `docker build .`, the `.` (dot) tells Docker that the current directory is the build context.

Real-life analogy: The build context is like the counter space in your kitchen. You can only cook with ingredients that are on the counter. Files outside the build context are like ingredients still in the pantry -- Docker cannot reach them.

How It Works

```
+-----------------------------------------------------+
|    Your Computer                                    |
|                                                     |
|    my-project/              <-- Build Context       |
|    +-- Dockerfile                                   |
|    +-- package.json                                 |
|    +-- server.js                                    |
|    +-- src/                                         |
|    |   +-- app.js                                   |
|    |   +-- routes.js                                |
|    +-- node_modules/        <-- Sent to Docker!     |
|    +-- .git/                <-- Sent to Docker!     |
|    +-- test-data.zip        <-- Sent to Docker!     |
|                                                     |
+-----------------------------------------------------+
```

Everything in the build context directory gets sent to the Docker daemon before the build starts. This is why large directories (like `node_modules` or `.git`) can slow down your build.

The Problem

```bash
$ docker build -t my-app .
Sending build context to Docker daemon  245.8MB
```

245 MB! That is because Docker is sending `node_modules`, `.git`, and other large directories that you do not need in the image.

The .dockerignore File: Keeping Builds Fast

A `.dockerignore` file tells Docker which files and directories to exclude from the build context. It works exactly like `.gitignore`.

Real-life analogy: When you pack a suitcase for a trip, you do not pack everything in your closet. You leave behind things you do not need. The `.dockerignore` file is your "do not pack" list.

Creating a .dockerignore File

Create a file named `.dockerignore` in the same directory as your Dockerfile:

```
# .dockerignore

# Dependencies (will be installed inside the container)
node_modules

# Version control
.git
.gitignore

# IDE files
.vscode
.idea
*.swp
*.swo

# OS files
.DS_Store
Thumbs.db

# Docker files
Dockerfile
docker-compose.yml

# Documentation
README.md
LICENSE

# Test files
coverage
*.test.js

# Environment files with secrets
.env
.env.local
```

Before and After .dockerignore

```
Without .dockerignore:
$ docker build -t my-app .
Sending build context to Docker daemon  245.8MB
Build time: 45 seconds

With .dockerignore:
$ docker build -t my-app .
Sending build context to Docker daemon  1.2MB
Build time: 12 seconds
```

That is a huge improvement. The build context went from 245 MB to 1.2 MB.

Pattern Syntax

```
# Ignore a specific file
secret.key

# Ignore a directory
node_modules

# Ignore by extension
*.log
*.tmp

# Ignore with wildcards
temp*

# Exception (do NOT ignore this)
!important.log

# Ignore in subdirectories
**/*.test.js
```

Layer Caching: Why Build Order Matters

This is one of the most important concepts in Docker. Understanding layer caching can make your builds go from minutes to seconds.

What Is a Layer?

Every instruction in a Dockerfile creates a new layer. A layer is like a transparent sheet that contains only the changes made by that instruction.

```
+-----------------------------------------------------+
|    How Layers Stack Up                              |
+-----------------------------------------------------+
|                                                     |
|    CMD ["node", "server.js"]    <- Layer 6          |
|    EXPOSE 3000                  <- Layer 5          |
|    COPY . .                     <- Layer 4          |
|    RUN npm install              <- Layer 3          |
|    COPY package.json .          <- Layer 2          |
|    WORKDIR /app                 <- Layer 1          |
|    FROM node:18-alpine          <- Base Layer       |
|                                                     |
+-----------------------------------------------------+
```

How Caching Works

Docker caches each layer. When you rebuild an image, Docker checks if anything has changed for each instruction:

- If nothing changed: Docker reuses the cached layer (instant)
- If something changed: Docker rebuilds that layer AND all layers after it

```
+---------------------------------------------------+
|    Layer Caching in Action                        |
+---------------------------------------------------+
|                                                   |
|    First Build:                                   |
|    FROM node:18-alpine      [BUILD] 3s            |
|    WORKDIR /app             [BUILD] 0.1s          |
|    COPY package.json .      [BUILD] 0.1s          |
|    RUN npm install          [BUILD] 15s           |
|    COPY . .                 [BUILD] 0.2s          |
|    CMD ["node", "server.js"][BUILD] 0.1s          |
|    Total: ~18.5 seconds                           |
|                                                   |
|    Second Build (only code changed):              |
|    FROM node:18-alpine      [CACHED] instant      |
|    WORKDIR /app             [CACHED] instant      |
|    COPY package.json .      [CACHED] instant      |
|    RUN npm install          [CACHED] instant      |
|    COPY . .                 [BUILD] 0.2s          |
|    CMD ["node", "server.js"][BUILD] 0.1s          |
|    Total: ~0.3 seconds                            |
|                                                   |
+---------------------------------------------------+
```

The second build is almost instant because Docker reused the cached layers for everything before the code change.

The Cache-Busting Problem

Here is the key rule: When a layer changes, all subsequent layers are rebuilt.

```dockerfile
# BAD ORDER - cache busted on every code change
FROM node:18-alpine
WORKDIR /app
COPY . .                    # <-- Changes every time you edit code
RUN npm install             # <-- Must reinstall every time!
CMD ["node", "server.js"]
```

```dockerfile
# GOOD ORDER - npm install is cached unless package.json changes
FROM node:18-alpine
WORKDIR /app
COPY package.json .         # <-- Only changes when dependencies change
RUN npm install             # <-- Cached if package.json unchanged
COPY . .                    # <-- Changes when code changes
CMD ["node", "server.js"]
```

```
+-----------------------------------------------------+
|    Why Order Matters                                |
+-----------------------------------------------------+
|                                                     |
|    BAD: COPY everything, then npm install           |
|    - Edit server.js                                 |
|    - COPY . . invalidates cache                     |
|    - npm install MUST re-run (15 seconds)           |
|    - Total rebuild: ~18 seconds                     |
|                                                     |
|    GOOD: COPY package.json, install, then code      |
|    - Edit server.js                                 |
|    - COPY package.json is still cached              |
|    - npm install is still cached                    |
|    - Only COPY . . re-runs (0.2 seconds)            |
|    - Total rebuild: ~0.3 seconds                    |
|                                                     |
+-----------------------------------------------------+
```

Complete Example: Node.js Application

Let us put everything together and build a real Node.js application.

Step 1: Create the Project Structure

```
my-node-app/
+-- Dockerfile
+-- .dockerignore
+-- package.json
+-- server.js
```

Step 2: Create package.json

```json
{
  "name": "my-node-app",
  "version": "1.0.0",
  "description": "A simple Node.js app in Docker",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step 3: Create server.js

```javascript
const express = require('express');
const app = express();
const PORT = 3000;

// A simple route that returns a greeting
app.get('/', (req, res) => {
  res.json({
    message: 'Hello from Docker!',
    timestamp: new Date().toISOString(),
    hostname: require('os').hostname()
  });
});

// A health check endpoint
app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

// Start the server
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Step 4: Create .dockerignore

```
node_modules
.git
.gitignore
README.md
.DS_Store
*.log
```

Step 5: Write the Dockerfile

```dockerfile
# Step 1: Start with Node.js on Alpine Linux
# Alpine is a tiny Linux distribution (~7 MB)
# node:18-alpine gives us Node.js 18 on Alpine
FROM node:18-alpine

# Step 2: Set the working directory inside the container
# All subsequent commands will run from /app
# Docker creates /app if it does not exist
WORKDIR /app

# Step 3: Copy ONLY package.json first
# We do this before copying all code for better caching
# If package.json has not changed, npm install will be cached
COPY package.json .

# Step 4: Install dependencies
# npm install reads package.json and installs everything listed
# This layer is cached unless package.json changes
RUN npm install

# Step 5: Copy the rest of the application code
# This is done AFTER npm install for caching reasons
# Changes to server.js will not trigger a reinstall
COPY . .

# Step 6: Document which port the app uses
# This does not actually publish the port
# It tells other developers "this app listens on 3000"
EXPOSE 3000

# Step 7: Define the command to start the application
# When the container starts, it runs: node server.js
# Using exec form (array) is best practice
CMD ["node", "server.js"]
```

Step 6: Build the Image

```bash
$ cd my-node-app

$ docker build -t my-node-app:1.0 .
```

Expected output:

```
[+] Building 18.4s (10/10) FINISHED
 => [internal] load build definition from Dockerfile          0.0s
 => [internal] load .dockerignore                             0.0s
```

```
 => [internal] load metadata for docker.io/library/node:18    1.1s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:abc   4.2s
 => [2/5] WORKDIR /app                                       0.1s
 => [3/5] COPY package.json .                                 0.0s
 => [4/5] RUN npm install                                    11.3s
 => [5/5] COPY . .                                            0.1s
 => exporting to image                                        1.6s
 => => naming to docker.io/library/my-node-app:1.0            0.0s
```

Step 7: Run the Container

```bash
$ docker run -d -p 3000:3000 --name my-app my-node-app:1.0
a1b2c3d4e5f6...
```

Breaking down the flags:

- `-d` -- run in detached mode (in the background)
- `-p 3000:3000` -- map port 3000 on your computer to port 3000 in the container
- `--name my-app` -- give the container a friendly name
- `my-node-app:1.0` -- the image to run

Step 8: Test the Application

```bash
$ curl http://localhost:3000
{
  "message": "Hello from Docker!",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "hostname": "a1b2c3d4e5f6"
}

$ curl http://localhost:3000/health
{
  "status": "healthy"
}
```

Your application is running inside a Docker container.

Step 9: Verify the Layer Cache

Make a small change to `server.js` (change the message text) and rebuild:

```bash
$ docker build -t my-node-app:1.1 .
```

Expected output:

```
[+] Building 1.2s (10/10) FINISHED
 => [internal] load build definition from Dockerfile          0.0s
 => [internal] load .dockerignore                             0.0s
 => [internal] load metadata for docker.io/library/node:18    0.3s
 => CACHED [1/5] FROM docker.io/library/node:18-alpine        0.0s
 => CACHED [2/5] WORKDIR /app                                 0.0s
 => CACHED [3/5] COPY package.json .                          0.0s
 => CACHED [4/5] RUN npm install                              0.0s
 => [5/5] COPY . .                                            0.2s
 => exporting to image                                        0.7s
 => => naming to docker.io/library/my-node-app:1.1            0.0s
```

Notice the word `CACHED` on steps 1 through 4. Only step 5 (copying the code) and the export had to run. The build went from 18 seconds to 1.2 seconds.

Step 10: Clean Up

```bash
$ docker stop my-app
my-app

$ docker rm my-app
my-app
```

Complete Dockerfile Anatomy

Here is a visual reference for all the instructions we covered:

```
+---------------------------------------------------+
|                 DOCKERFILE ANATOMY                |
+---------------------------------------------------+
|                                                   |
|  FROM node:18-alpine                              |
|  ^                                                |
|  |  Base image (required, must be first)          |
|                                                   |
|  WORKDIR /app                                     |
|  ^                                                |
|  |  Set working directory                         |
|                                                   |
|  COPY package.json .                              |
|  ^                                                |
```

```
|  |  Copy files from build context to image       |
|                                                  |
|  RUN npm install                                 |
|  ^                                               |
|  |  Execute command during build                 |
|                                                  |
|  COPY . .                                        |
|  ^                                               |
|  |  Copy remaining files                         |
|                                                  |
|  EXPOSE 3000                                     |
|  ^                                               |
|  |  Document the port (informational only)       |
|                                                  |
|  CMD ["node", "server.js"]                       |
|  ^                                               |
|  |  Default command when container starts        |
|                                                  |
+--------------------------------------------------+
```

Common Mistakes

Mistake 1: Forgetting to Name the File Correctly

```bash
# WRONG file names:
dockerfile        # lowercase 'd'
Dockerfile.txt    # has an extension
DockerFile        # wrong capitalization

# CORRECT:
Dockerfile        # capital D, no extension
```

Mistake 2: Copying Everything Before Installing Dependencies

```dockerfile
# BAD: Busts cache on every code change
COPY . .
RUN npm install

# GOOD: Caches npm install when only code changes
COPY package.json package-lock.json ./
RUN npm install
COPY . .
```

Mistake 3: Using CMD When You Mean RUN

```dockerfile
# WRONG: CMD runs when container STARTS, not during build
CMD apt-get update && apt-get install -y curl

# RIGHT: RUN executes during BUILD
RUN apt-get update && apt-get install -y curl
```

Mistake 4: Not Using .dockerignore

```bash
# Without .dockerignore, you might accidentally:
# - Send node_modules (100+ MB) to the build context
# - Include .git directory (can be very large)
# - Copy .env files with secrets into the image
```

Mistake 5: Confusing EXPOSE with -p

```dockerfile
# EXPOSE does NOT publish the port
EXPOSE 3000

# You STILL need -p when running:
# docker run -p 3000:3000 my-app
```

Best Practices

1. Always use a `.dockerignore` file to keep your build context small and avoid copying secrets into the image.
2. Copy dependency files first, then install, then copy code. This maximizes layer caching.
3. Use specific base image tags like `node:18-alpine` instead of `node:latest`. This ensures reproducible builds.
4. Use exec form for CMD (`CMD ["node", "server.js"]` instead of `CMD node server.js`). Exec form handles signals properly.
5. Keep your Dockerfile readable with comments explaining each step.
6. Minimize the number of layers by combining related RUN commands.
7. Do not install unnecessary packages. Every package increases image size and potential security vulnerabilities.

8. Use `WORKDIR` instead of `RUN mkdir && cd`. WORKDIR is cleaner and persists across instructions.

Quick Summary

A Dockerfile is a text file with instructions for building a Docker image. Each instruction performs a specific action:

- `FROM` sets the base image (required, must be first)
- `WORKDIR` sets the working directory inside the container
- `COPY` copies files from your computer into the image
- `RUN` executes commands during the build process
- `CMD` sets the default command when a container starts (can be overridden)
- `ENTRYPOINT` sets a fixed command when a container starts (hard to override)
- `EXPOSE` documents which port the application uses

You build images with `docker build -t name:tag .` and Docker uses layer caching to speed up rebuilds. A `.dockerignore` file excludes unnecessary files from the build context.

Key Points

- A Dockerfile is a recipe for building Docker images
- Every Dockerfile must start with `FROM`
- Each instruction creates a layer; layers are cached for faster rebuilds
- Put things that change rarely at the top, and things that change often at the bottom
- `CMD` is the default command (easily overridden); `ENTRYPOINT` is the fixed command
- `EXPOSE` is documentation only -- it does not publish ports
- The build context is everything Docker can see during the build
- `.dockerignore` keeps the build context small and prevents secrets from leaking
- Layer caching can make rebuilds nearly instant if you order instructions wisely

---

Practice Questions

1. What is the difference between `CMD` and `ENTRYPOINT`? When would you use each one?
2. Why is it important to copy `package.json` and run `npm install` before copying the rest of the application code? What happens if you copy everything first?
3. You have a Dockerfile with `EXPOSE 8080`. You run the container with `docker run my-app`. Can you access the application on port 8080 from your browser? Why or why not?
4. What does the `.` (dot) at the end of `docker build -t my-app .` mean? What is it called?
5. You change one line of code in `server.js` and rebuild the image. Which layers will Docker rebuild, and which will come from the cache?

Exercises

Exercise 1: Build Your First Dockerfile

Create a simple Dockerfile that:

1. Uses `node:18-alpine` as the base image
2. Sets the working directory to `/app`
3. Copies a simple JavaScript file that prints "Hello, Docker!"
4. Runs the script using `CMD`

Build the image and run it. Verify the output.

Exercise 2: Experiment with Layer Caching

1. Create a Node.js project with a `package.json` and a `server.js`
2. Write a Dockerfile with proper instruction ordering
3. Build the image and note the build time
4. Change only the `server.js` file
5. Rebuild and observe which steps say CACHED
6. Now change `package.json` (add a dependency)
7. Rebuild and observe which steps are rebuilt

Exercise 3: CMD vs ENTRYPOINT

1. Create a Dockerfile with `CMD ["echo", "Hello from CMD"]`
2. Build and run it. Note the output.
3. Run it again with `docker run my-app echo "Custom message"`. What happens?

4. Change CMD to `ENTRYPOINT ["echo"]` and `CMD ["Hello from ENTRYPOINT"]`
5. Build and run it. Then try `docker run my-app "Custom message"`. What is different?

What Is Next?

You now know how to write a Dockerfile and build your own images. In the next chapter, we will learn Dockerfile best practices that professional developers use to create smaller, faster, and more secure images. You will learn about multi-stage builds, running as non-root users, and techniques that can cut your image size by 90 percent or more.

Chapter 7: Dockerfile Best Practices

What You Will Learn

By the end of this chapter, you will be able to:

- Use multi-stage builds to dramatically reduce image size
- Minimize the number of layers by combining RUN commands
- Order Dockerfile instructions for optimal cache performance
- Run containers as a non-root user for better security
- Write effective `.dockerignore` files
- Choose specific image tags instead of `:latest`
- Understand the difference between COPY and ADD
- Add health checks to your containers
- Compare image sizes before and after applying best practices

Why This Chapter Matters

In the previous chapter, you learned how to write a Dockerfile and build an image. But writing a Dockerfile that works and writing a Dockerfile that works well are two very different things.

A poorly written Dockerfile might produce an image that is 1.2 GB in size, takes 5 minutes to build, and runs as the root user (a serious security risk). The same application, with a well-written Dockerfile, might produce a 50 MB image that builds in 10 seconds and runs securely.

In professional environments, these differences matter enormously. Larger images mean slower deployments, higher storage costs, and more security vulnerabilities. This chapter teaches you the techniques that separate amateur Dockerfiles from professional ones.

Multi-Stage Builds: The Biggest Win

Multi-stage builds are the single most powerful technique for reducing image size. They let you use one image for building your application and a different, much smaller image for running it.

Real-life analogy: Think about baking a cake. You need a big messy kitchen with mixing bowls, a whisk, measuring cups, and an oven to make the cake. But when you serve the cake, you only need the cake itself and a plate. You do not bring the entire kitchen to the dinner table. Multi-stage builds work the same way -- you use a big "kitchen" image to build your app, then copy only the finished product to a small "plate" image.

The Problem Without Multi-Stage Builds

Consider building a React application:

```dockerfile
# Single-stage build (the old way)
FROM node:18

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm install

COPY . .
RUN npm run build

# The built files are in /app/build
# But we also have node_modules, source code, build tools...

CMD ["npx", "serve", "-s", "build"]
```

This image contains:

- Node.js runtime (~350 MB)
- `node_modules` directory (~200 MB)
- Source code (~5 MB)
- Build tools (~100 MB)
- The actual built files we need (~2 MB)

Total image size: ~660 MB for an app that only needs 2 MB of files.

The Solution: Multi-Stage Builds

```dockerfile
# ============================================
# Stage 1: Build (the kitchen)
# ============================================
FROM node:18 AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm install

COPY . .
RUN npm run build

# At this point, /app/build contains our static files

# ============================================
# Stage 2: Production (the plate)
# ============================================
FROM nginx:alpine

# Copy ONLY the built files from stage 1
COPY --from=builder /app/build /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Breaking this down:

Stage 1 (`builder`):

- Uses the full Node.js image with all build tools
- Installs dependencies and builds the application
- Named `builder` using `AS builder`

Stage 2 (production):

- Starts fresh with a tiny nginx:alpine image
- Copies only the built files from stage 1 using `COPY --from=builder`
- Does not include Node.js, node_modules, or source code

```
+-----------------------------------------------------+
|    Multi-Stage Build Flow                           |
+-----------------------------------------------------+
|                                                     |
|    Stage 1: builder (node:18)                       |
|    +-------------------------------------+          |
|    | node_modules/     200 MB            |          |
|    | src/              5 MB              |          |
|    | build tools       100 MB            |          |
|    | build/            2 MB  ----+       |          |
|    +-------------------------------------+  |       |
```

```
|                                          |  |     |
|                            Only this     |  |     |
|                            gets copied -+  |     |
|                                            |     |
|    Stage 2: production (nginx:alpine)      v     |
|    +-----------------------------------+        |
|    | nginx             10 MB           |        |
|    | build/            2 MB            |        |
|    +-----------------------------------+        |
|                                                 |
|    Final image size: ~12 MB                     |
|    (down from ~660 MB!)                         |
|                                                 |
+-------------------------------------------------+
```

Size Comparison

```
+-----------------------+-------------+
| Approach              | Image Size  |
+-----------------------+-------------+
| Single stage          | ~660 MB     |
| Multi-stage           | ~12 MB      |
| Reduction             | 98%         |
+-----------------------+-------------+
```

Multi-Stage Build for a Go Application

Go is even more dramatic because Go compiles to a single binary:

```dockerfile
# Stage 1: Build
FROM golang:1.21 AS builder

WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /app/server

# Stage 2: Production
FROM scratch

COPY --from=builder /app/server /server

EXPOSE 8080
ENTRYPOINT ["/server"]
```

The `scratch` image is completely empty (0 bytes). The final image contains only the compiled Go binary.

```
+-----------------------+-------------+
| Approach              | Image Size  |
+-----------------------+-------------+
| Single stage (golang) | ~850 MB     |
| Multi-stage (scratch) | ~8 MB       |
| Reduction             | 99%         |
+-----------------------+-------------+
```

Minimize Layers: Combine RUN Commands

Every RUN instruction creates a new layer. Unnecessary layers increase image size and slow down pulls.

Real-life analogy: Imagine you go to the grocery store three times in one day -- once for bread, once for milk, and once for eggs. That is three trips. It would be much more efficient to buy all three things in a single trip. Combining RUN commands is the same idea.

The Problem

```dockerfile
# BAD: 7 separate layers
FROM ubuntu:22.04

RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y wget
RUN apt-get install -y git
RUN apt-get install -y vim
RUN apt-get clean
RUN rm -rf /var/lib/apt/lists/*
```

Each RUN creates a layer. Even though we clean up in the last two commands, the packages still exist in earlier layers. Docker layers are additive -- deleting something in a later layer does not remove it from earlier layers.

```
+-----------------------------------------------------+
|    Why Separate Layers Waste Space                  |
+-----------------------------------------------------+
|                                                     |
```

```
|    Layer 7: rm -rf /var/lib/apt/lists/*  (+0 MB) |
|    Layer 6: apt-get clean                (+0 MB) |
|    Layer 5: install vim                  (+30 MB) |
|    Layer 4: install git                  (+50 MB) |
|    Layer 3: install wget                 (+5 MB)  |
|    Layer 2: install curl                 (+5 MB)  |
|    Layer 1: apt-get update               (+25 MB) |
|    Base:    ubuntu:22.04                 (+77 MB) |
|                                                   |
|    Total: ~192 MB                                 |
|    (cleanup in layers 6-7 does NOT reduce         |
|     the size of layers 1-5!)                      |
|                                                   |
+---------------------------------------------------+
```

The Solution

```dockerfile
# GOOD: 1 layer for all package operations
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y \
        curl \
        wget \
        git \
        vim && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

Now the update, installation, and cleanup all happen in a single layer. The temporary files from `apt-get update` never get stored in a separate layer.

```
+---------------------------------------------------+
|    Combined Layer                                 |
+---------------------------------------------------+
|                                                   |
|    Layer 1: update + install + clean    (+70 MB) |
|    Base:    ubuntu:22.04                (+77 MB) |
|                                                   |
|    Total: ~147 MB (saved ~45 MB)                  |
|                                                   |
+---------------------------------------------------+
```

Formatting Tips

Use backslashes and alphabetical ordering to keep combined RUN commands readable:

```dockerfile
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        curl \
        git \
        vim \
        wget && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

The `--no-install-recommends` flag tells apt not to install suggested packages, which further reduces image size.

Order Instructions for Cache: Dependencies Before Code

We touched on this in Chapter 6, but it is worth emphasizing because it is one of the most impactful best practices.

The rule: Put things that change rarely at the top of your Dockerfile, and things that change frequently at the bottom.

Why It Matters

Remember: when a layer changes, all subsequent layers are rebuilt. So if your code changes (which happens many times a day), you want to make sure expensive operations like `npm install` are above the code copy and can remain cached.

The Pattern

```dockerfile
FROM node:18-alpine

WORKDIR /app

# 1. Copy ONLY dependency files (changes rarely)
COPY package.json package-lock.json ./

# 2. Install dependencies (expensive, but cached if step 1 unchanged)
RUN npm ci --only=production
```

```
# 3. Copy application code (changes frequently)
COPY . .

# 4. Configuration (changes rarely)
EXPOSE 3000
CMD ["node", "server.js"]
```

Cache Performance Comparison

```
+-----------------------------------------------------+
|    Developer's Typical Day                          |
+-----------------------------------------------------+
|                                                     |
|    Action              Good Order   Bad Order       |
|    ------------------------------------------       |
|    Change code         0.5s build   30s build       |
|    Change code         0.5s build   30s build       |
|    Change code         0.5s build   30s build       |
|    Add dependency      30s build    30s build       |
|    Change code         0.5s build   30s build       |
|    Change code         0.5s build   30s build       |
|    ------------------------------------------       |
|    Total (6 builds)    62s          180s            |
|    Time saved          118 seconds                  |
|                                                     |
+-----------------------------------------------------+
```

Over a day with many rebuilds, good ordering saves minutes. Over a week, it saves hours.

Python Example

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# Copy dependency file first
COPY requirements.txt .

# Install dependencies (cached if requirements.txt unchanged)
RUN pip install --no-cache-dir -r requirements.txt

# Copy code last
COPY . .

CMD ["python", "app.py"]
```

Java Example

```dockerfile
FROM maven:3.9-eclipse-temurin-17

WORKDIR /app

# Copy dependency definition first
COPY pom.xml .

# Download dependencies (cached if pom.xml unchanged)
RUN mvn dependency:go-offline

# Copy source code last
COPY src ./src

RUN mvn package -DskipTests

CMD ["java", "-jar", "target/app.jar"]
```

Non-Root USER: Security First

By default, containers run as the root user. This is a security risk. If an attacker breaks into your container, they have root access and can potentially escape to the host system.

Real-life analogy: Running a container as root is like giving every employee the master key to the entire building. They only need access to their own office. Give them a key that opens just their office door.

The Problem

```bash
# By default, processes run as root inside containers
$ docker run node:18-alpine whoami
root

$ docker run node:18-alpine id
uid=0(root) gid=0(root)
```

The Solution: Add a USER Instruction

```dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --only=production

COPY . .

# Create a non-root user
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup

# Change ownership of the app directory
RUN chown -R appuser:appgroup /app

# Switch to the non-root user
USER appuser

EXPOSE 3000
CMD ["node", "server.js"]
```

Some Base Images Include Non-Root Users

Many official images come with a non-root user already created:

```dockerfile
# node images have a user called "node"
FROM node:18-alpine

WORKDIR /app

COPY --chown=node:node package.json package-lock.json ./
RUN npm ci --only=production

COPY --chown=node:node . .

# Use the built-in "node" user
USER node

EXPOSE 3000
CMD ["node", "server.js"]
```

The `--chown=node:node` flag in COPY sets the owner of the files as they are copied, which is cleaner than running `chown` afterward.

Verifying It Works

```bash
$ docker build -t secure-app .

$ docker run secure-app whoami
node

$ docker run secure-app id
uid=1000(node) gid=1000(node)
```

The process is no longer running as root.

Where to Place USER

Place `USER` after all operations that require root privileges (like installing packages) but before the `CMD`:

```dockerfile
FROM node:18-alpine

WORKDIR /app

# These operations might need root
COPY package.json package-lock.json ./
RUN npm ci --only=production
COPY . .

# Switch to non-root AFTER setup is done
USER node

# These run as the non-root user
EXPOSE 3000
CMD ["node", "server.js"]
```

Use .dockerignore Effectively

We covered `.dockerignore` in Chapter 6, but here is a more comprehensive example:

```
# Version control
.git
.gitignore
.gitattributes

# CI/CD
.github
.gitlab-ci.yml
.circleci
```

```
Jenkinsfile

# Docker (no need to include these in the image)
Dockerfile
Dockerfile.*
docker-compose*.yml
.dockerignore

# Dependencies (installed inside the container)
node_modules
vendor
__pycache__
*.pyc
.venv
venv

# IDE and editor files
.vscode
.idea
*.swp
*.swo
*~

# OS files
.DS_Store
Thumbs.db
desktop.ini

# Test and coverage
coverage
.nyc_output
*.test.js
*.spec.js
test
tests
__tests__

# Documentation
README.md
CHANGELOG.md
LICENSE
docs

# Environment and secrets
.env
.env.*
*.pem
*.key

# Build artifacts
dist
build
*.log
tmp
temp
```

Impact of a Good .dockerignore

```
+-----------------------------------------+
| File/Directory    | Size   | Needed?      |
+-----------------------------------------+
| node_modules/     | 200 MB | No (reinstalled)|
| .git/             | 50 MB  | No           |
| coverage/         | 10 MB  | No           |
| .vscode/          | 1 MB   | No           |
| .env              | 1 KB   | DANGEROUS!   |
| src/              | 2 MB   | Yes          |
| package.json      | 1 KB   | Yes          |
| server.js         | 5 KB   | Yes          |
+-----------------------------------------+
| Without .dockerignore: ~263 MB sent        |
| With .dockerignore:    ~2 MB sent          |
+-----------------------------------------+
```

Specific Tags, Not :latest

Always use specific version tags for your base images. Never use `:latest` in production Dockerfiles.

Real-life analogy: Imagine ordering "the latest model" of a car. Today that might be a sedan. Next month, it might be an SUV. If your parking garage only fits sedans, you have a problem. Specify the exact model you want.

The Problem with :latest

```dockerfile
# BAD: What version of Node.js will you get?
FROM node:latest

# It could be Node 18 today and Node 22 tomorrow
# Your app might not work with Node 22
# Your builds are not reproducible
```

Use Specific Tags

```dockerfile
# GOOD: Always gets Node.js 18 on Alpine
FROM node:18-alpine

# BETTER: Specific minor version
FROM node:18.19-alpine

# BEST: Specific patch version (most reproducible)
FROM node:18.19.0-alpine3.18
```

Comparison

```
+-----------------------------------------------+
|    Tag Specificity                            |
+-----------------------------------------------+
|                                               |
|    node:latest         Could be anything!     |
|    node:18             Node 18, any minor/patch |
|    node:18-alpine      Node 18 on Alpine Linux |
|    node:18.19-alpine   Specific minor version |
|    node:18.19.0-alpine Exact version (best)   |
|                                               |
|    More specific = More reproducible          |
|                                               |
+-----------------------------------------------+
```

COPY vs ADD: Know the Difference

Both `COPY` and `ADD` copy files into the image, but they have different capabilities.

COPY (Recommended)

`COPY` does exactly one thing: it copies files and directories from the build context into the image.

```dockerfile
COPY package.json .
COPY src/ /app/src/
```

ADD (Use Sparingly)

`ADD` does everything COPY does, plus:

1. It can extract compressed archives (`.tar.gz`, `.tar.bz2`, etc.)
2. It can download files from URLs

```dockerfile
# ADD can extract tar files automatically
ADD app.tar.gz /app/

# ADD can download from URLs (but DON'T do this)
ADD https://example.com/file.txt /app/
```

Why COPY Is Better

```
+-----------------------------------------------------+
|    COPY vs ADD                                      |
+-----------------------------------------------------+
|                                                     |
|    Feature          | COPY    | ADD                |
|    -----------------+---------+-------------------+
|    Copy files       | Yes     | Yes                |
|    Copy directories | Yes     | Yes                |
|    Extract archives | No      | Yes (auto)         |
|    Download URLs    | No      | Yes (no caching)   |
|    Predictable      | Yes     | Surprising         |
|    Recommended      | Yes     | Rarely             |
|                                                     |
|    Rule: Use COPY unless you specifically need    |
|    ADD's auto-extraction feature.                  |
|                                                     |
+-----------------------------------------------------+
```

The problem with ADD is its "magic" behavior. If you `ADD` a file called `data.tar.gz`, it gets automatically extracted. This can be surprising if you just wanted to copy the archive without extracting it.

For downloading files, use `RUN curl` or `RUN wget` instead of `ADD`, because:

- Downloaded files are not cached
- You cannot verify checksums
- You cannot extract and clean up in the same layer

```dockerfile
# BAD: ADD for downloading
ADD https://example.com/big-file.tar.gz /app/

# GOOD: RUN curl for downloading
RUN curl -fsSL https://example.com/big-file.tar.gz | tar -xz -C /app/ && \
    rm -f /app/big-file.tar.gz
```

HEALTHCHECK: Is Your Container Healthy?

A `HEALTHCHECK` instruction tells Docker how to check if your container is actually working, not just running.

Real-life analogy: A heartbeat monitor does not just check if a patient is in the room. It checks if their heart is actually beating. HEALTHCHECK does the same for your container.

The Problem Without HEALTHCHECK

```bash
$ docker ps
CONTAINER ID   IMAGE     STATUS
a1b2c3d4e5f6   my-app    Up 2 hours
```

The container says "Up" but the application inside might have crashed, run out of memory, or be stuck in an infinite loop. Docker does not know the difference.

Adding a HEALTHCHECK

```dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --only=production
COPY . .

# Health check: try to reach the /health endpoint every 30 seconds
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:3000/health || exit 1

USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Breaking down the options:

- `--interval=30s` -- Check every 30 seconds
- `--timeout=10s` -- Wait up to 10 seconds for a response
- `--start-period=5s` -- Give the app 5 seconds to start up before checking
- `--retries=3` -- Fail after 3 consecutive failed checks
- `CMD curl -f http://localhost:3000/health || exit 1` -- The actual check command

How It Looks in docker ps

```bash
$ docker ps
CONTAINER ID   IMAGE     STATUS
a1b2c3d4       my-app    Up 2 min (healthy)
b5c6d7e8       my-app    Up 5 min (unhealthy)
f9g0h1i2       my-app    Up 10 sec (health: starting)
```

Docker now shows three possible health states:

```
+-----------------------------------------------------+
|    Container Health States                          |
+-----------------------------------------------------+
|                                                     |
|    starting   -> Just started, waiting for          |
|                  first successful check             |
|                                                     |
|    healthy    -> Health check is passing            |
|                                                     |
|    unhealthy  -> Health check is failing            |
|                  (3+ consecutive failures)          |
|                                                     |
+-----------------------------------------------------+
```

Health Check Without curl

If your image does not include `curl` (to keep it small), you can use other approaches:

```dockerfile
# For Node.js: use a custom health check script
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD node -e "require('http').get('http://localhost:3000/health', (r) => { process.e...

# For Python: use wget (available on Alpine)
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:5000/health || exit 1
```

Before and After: Putting It All Together

Let us take a real Dockerfile and apply all the best practices we have learned.

Before: The Naive Dockerfile

```dockerfile
FROM node:latest

COPY . /app
WORKDIR /app

RUN npm install
RUN npm run build

EXPOSE 3000
CMD npm start
```

Problems:

- Uses `:latest` (not reproducible)
- Copies everything before installing (cache busting)
- No `.dockerignore`
- Separate RUN commands
- No non-root user
- No health check
- Shell form CMD
- Single stage (includes build tools in production)

After: The Professional Dockerfile

```dockerfile
# Stage 1: Build
FROM node:18-alpine AS builder

WORKDIR /app

# Copy dependency files first for cache
COPY package.json package-lock.json ./

# Install ALL dependencies (including devDependencies for build)
RUN npm ci

# Copy source code
COPY . .

# Build the application
RUN npm run build

# Stage 2: Production
FROM node:18-alpine

WORKDIR /app
```

```
# Copy dependency files
COPY package.json package-lock.json ./

# Install ONLY production dependencies
RUN npm ci --only=production && \
    npm cache clean --force

# Copy built application from builder stage
COPY --from=builder /app/dist ./dist

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:3000/health || exit 1

# Run as non-root user
USER node

EXPOSE 3000

# Exec form CMD
CMD ["node", "dist/server.js"]
```

Size and Build Time Comparison

```
+---------------------------------------------------+
|    Before vs After Comparison                     |
+---------------------------------------------------+
|                                                   |
|    Metric              Before       After         |
|    ----------------------------------------------
|    Base image          node:latest  node:18-alpine|
|    Image size          950 MB       120 MB       |
|    Build (first)       60 sec       45 sec       |
|    Build (code change) 55 sec       3 sec        |
|    Runs as             root         node (safe)  |
|    Health check        No           Yes          |
|    Reproducible        No           Yes          |
|    Build tools in prod Yes          No           |
|                                                   |
|    Size reduction: 87%                            |
|    Rebuild speedup: 18x faster                    |
|                                                   |
+---------------------------------------------------+
```

Checking Image Sizes

You can compare image sizes using `docker images`:

```bash
$ docker images
REPOSITORY     TAG        SIZE
my-app         naive      950MB
my-app         optimized  120MB
```

Or for more detail, use `docker history`:

```bash
$ docker history my-app:optimized
IMAGE          CREATED        SIZE      COMMENT
a1b2c3d4       2 min ago      0B        CMD ["node", "dist/server.js"]
e5f6g7h8       2 min ago      0B        EXPOSE 3000
i9j0k1l2       2 min ago      0B        USER node
m3n4o5p6       2 min ago      0B        HEALTHCHECK ...
q7r8s9t0       2 min ago      1.5MB     COPY dist from builder
u1v2w3x4       2 min ago      45MB      npm ci --only=production
y5z6a7b8       3 min ago      3KB       COPY package*.json
c9d0e1f2       3 min ago      0B        WORKDIR /app
g3h4i5j6       2 weeks ago    75MB      node:18-alpine base
```

Best Practices Checklist

Here is a quick reference checklist for writing professional Dockerfiles:

```
+----------------------------------------------------+
|    Dockerfile Best Practices Checklist             |
+----------------------------------------------------+
|                                                    |
|  [ ] Use multi-stage builds                        |
|  [ ] Use specific base image tags (not :latest)    |
|  [ ] Use -alpine or -slim variants                 |
|  [ ] Copy dependency files before code             |
|  [ ] Combine RUN commands with &&                  |
|  [ ] Clean up in the same RUN layer                |
|  [ ] Use COPY instead of ADD                       |
|  [ ] Add a .dockerignore file                      |
|  [ ] Run as non-root USER                          |
|  [ ] Use exec form for CMD                         |
|  [ ] Add a HEALTHCHECK                             |
|  [ ] Do not install unnecessary packages           |
|  [ ] Add comments explaining each section          |
|                                                    |
+----------------------------------------------------+
```

Common Mistakes

Mistake 1: Building Without a .dockerignore

```bash
# Without .dockerignore, secrets can leak into the image
$ docker build -t my-app .
# .env file with API keys is now inside the image!
# Anyone who pulls your image can extract those secrets
```

Mistake 2: Running npm install Instead of npm ci

```dockerfile
# BAD: npm install can modify package-lock.json
RUN npm install

# GOOD: npm ci uses exact versions from package-lock.json
RUN npm ci
```

`npm ci` (clean install) is designed for automated environments. It:

- Installs exact versions from `package-lock.json`
- Is faster than `npm install`
- Deletes `node_modules` before installing
- Fails if `package-lock.json` and `package.json` are out of sync

Mistake 3: Not Cleaning Up in the Same Layer

```dockerfile
# BAD: cleanup in separate layer does not reduce size
RUN apt-get update && apt-get install -y gcc
RUN rm -rf /var/lib/apt/lists/*

# GOOD: cleanup in same layer removes files before layer is saved
RUN apt-get update && \
    apt-get install -y gcc && \
    rm -rf /var/lib/apt/lists/*
```

Mistake 4: Using ADD When COPY Would Work

```dockerfile
# BAD: ADD has surprising auto-extraction behavior
ADD config.tar.gz /app/

# GOOD: Be explicit about what you want
COPY config.tar.gz /app/
RUN tar -xzf /app/config.tar.gz -C /app/ && rm /app/config.tar.gz
```

Mistake 5: Ignoring HEALTHCHECK

Without HEALTHCHECK, Docker only knows if the main process is running. It does not know if your application is actually responding to requests. A crashed web server that keeps the process alive will appear "healthy" without a proper health check.

Quick Summary

Professional Dockerfiles use several techniques to produce small, fast, and secure images:

- Multi-stage builds separate the build environment from the production environment, often reducing image size by 80-99%
- Combining RUN commands prevents temporary files from being stored in separate layers
- Ordering for cache means putting dependencies before code so that expensive installations are cached
- Non-root USER prevents attackers from having root access if they break into the container
- Specific tags make builds reproducible by locking down the base image version
- COPY over ADD keeps behavior predictable
- HEALTHCHECK lets Docker (and orchestrators) know if your application is truly healthy

---

Key Points

- Multi-stage builds are the most effective way to reduce image size
- Each RUN instruction creates a layer; combine related commands
- Put things that change rarely at the top of the Dockerfile
- Always run containers as a non-root user in production

- Never use `:latest` in production Dockerfiles
- Use `COPY` unless you specifically need `ADD`'s auto-extraction
- HEALTHCHECK tells Docker how to verify your application is working
- A good `.dockerignore` speeds up builds and prevents secret leaks
- Use `npm ci` instead of `npm install` in Dockerfiles

Practice Questions

1. What is a multi-stage build, and why does it reduce image size? What keyword connects the stages?
2. You have a Dockerfile with three separate RUN commands that install packages. Why is this worse than combining them into one RUN command? Give an example of the combined version.
3. Why should you always run your production containers as a non-root user? What instruction changes the user?
4. A developer uses `FROM node:latest` in their Dockerfile. Their application works today but fails after they rebuild the image next month. What likely happened?
5. What is the difference between COPY and ADD? When would you use ADD?

Exercises

Exercise 1: Multi-Stage Build

Take the following single-stage Dockerfile and convert it to a multi-stage build:

```dockerfile
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
CMD ["node", "dist/server.js"]
```

Build both versions and compare the image sizes using `docker images`.

Exercise 2: Apply All Best Practices

Write a Dockerfile for a Python Flask application that:

1. Uses `python:3.11-slim` as the base
2. Copies `requirements.txt` before the rest of the code
3. Installs dependencies with `pip install --no-cache-dir`
4. Runs as a non-root user
5. Includes a health check
6. Uses exec form CMD

Exercise 3: Measure the Difference

1. Create a simple Node.js application
2. Write a "naive" Dockerfile (single stage, no `.dockerignore`, `:latest` tag)
3. Write an "optimized" Dockerfile (multi-stage, `.dockerignore`, specific tag, non-root user)
4. Build both and compare:
    - Image size (`docker images`)
    - Build time (use `time docker build`)
    - User inside container (`docker run IMAGE whoami`)

What Is Next?

Now that you know how to write professional Dockerfiles, the next chapter puts these skills to work. You will build complete, production-ready Docker images for three different applications: a Node.js Express API, a Python Flask API, and a Java Spring Boot application. Each one will use the best practices you learned in this chapter.

Chapter 8: Building Real Application Images

What You Will Learn

By the end of this chapter, you will be able to:

- Build a complete Docker image for a Node.js Express API
- Build a complete Docker image for a Python Flask API
- Build a complete Docker image for a Java Spring Boot application
- Use multi-stage builds for compiled languages like Java
- Test each containerized application with curl
- Compare image sizes across different languages and approaches
- Understand how project structure affects Dockerfile design

Why This Chapter Matters

The previous two chapters taught you how to write Dockerfiles and follow best practices. But real applications are messier than simple examples. They have dependencies, configuration files, build steps, and specific runtime requirements.

This chapter bridges the gap between "hello world" Dockerfiles and real production images. You will build three complete applications from scratch, each in a different programming language. By the end, you will have the confidence to Dockerize any application you encounter.

Think of it this way: the previous chapters taught you how to drive a car in a parking lot. This chapter takes you out on the real road with traffic, intersections, and highway merges.

Application 1: Node.js Express API

Express is the most popular web framework for Node.js. Let us build a simple REST API and package it in Docker.

Project Structure

```
node-api/
+-- Dockerfile
+-- .dockerignore
+-- package.json
+-- package-lock.json
+-- src/
    +-- server.js
    +-- routes/
        +-- users.js
```

package.json

```json
{
  "name": "node-docker-api",
  "version": "1.0.0",
  "description": "A Node.js Express API in Docker",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",
    "helmet": "^7.1.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

A quick explanation of the dependencies:

- express -- The web framework that handles HTTP requests
- cors -- Middleware that allows cross-origin requests (when your frontend is on a different domain)
- helmet -- Middleware that sets security-related HTTP headers

- nodemon -- A development tool that restarts the server when code changes (not needed in production)

src/server.js

```javascript
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const userRoutes = require('./routes/users');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(helmet());
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.json({
    service: 'Node.js Express API',
    version: '1.0.0',
    status: 'running'
  });
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy', uptime: process.uptime() });
});

app.use('/api/users', userRoutes);

// Start server
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
```

src/routes/users.js

```javascript
const express = require('express');
const router = express.Router();

// In-memory data (in a real app, this would be a database)
const users = [
  { id: 1, name: 'Alice', email: 'alice@example.com' },
  { id: 2, name: 'Bob', email: 'bob@example.com' },
  { id: 3, name: 'Charlie', email: 'charlie@example.com' }
```

```
];

// GET all users
router.get('/', (req, res) => {
  res.json(users);
});

// GET user by ID
router.get('/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }
  res.json(user);
});

module.exports = router;
```

.dockerignore

```
node_modules
.git
.gitignore
README.md
.DS_Store
*.log
.env
.env.*
coverage
.nyc_output
Dockerfile
docker-compose.yml
```

Dockerfile

```dockerfile
# Use Node.js 18 on Alpine Linux for a small image
FROM node:18-alpine

# Set the working directory
WORKDIR /app

# Copy dependency files first (for layer caching)
# If these files do not change, npm ci will be cached
COPY package.json package-lock.json ./

# Install ONLY production dependencies
```

```
# npm ci is faster and more reliable than npm install
# --only=production skips devDependencies like nodemon
RUN npm ci --only=production

# Copy the application source code
COPY src/ ./src/

# Add a health check so Docker knows if the app is working
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:3000/health || exit 1

# Run as non-root user (node user is built into the node image)
USER node

# Document the port
EXPOSE 3000

# Start the application using exec form
CMD ["node", "src/server.js"]
```

Let us go through each line:

1. `FROM node:18-alpine` -- We use Alpine Linux to keep the image small (~120 MB base instead of ~350 MB).
2. `WORKDIR /app` -- Sets up our working directory inside the container.
3. `COPY package.json package-lock.json ./` -- Copies dependency files first for caching.
4. `RUN npm ci --only=production` -- Installs only the packages needed to run the app (not dev tools like nodemon).
5. `COPY src/ ./src/` -- Copies only the source code, not the entire project directory.
6. `HEALTHCHECK` -- Uses wget (available on Alpine) to check the /health endpoint.
7. `USER node` -- Switches to the built-in non-root user.
8. `EXPOSE 3000` -- Documents the port.
9. `CMD ["node", "src/server.js"]` -- Starts the server.

Build and Run

```bash
# Build the image
$ docker build -t node-api:1.0 .
[+] Building 12.3s (10/10) FINISHED
 => [internal] load build definition from Dockerfile          0.0s
 => [internal] load .dockerignore                             0.0s
 => [internal] load metadata for docker.io/library/node:18    0.8s
 => [1/4] FROM docker.io/library/node:18-alpine               0.0s
 => [2/4] WORKDIR /app                                        0.0s
 => [3/4] COPY package.json package-lock.json ./               0.0s
```

```
 => [4/4] RUN npm ci --only=production                        9.8s
 => [5/5] COPY src/ ./src/                                    0.1s
 => exporting to image                                        1.6s

# Check image size
$ docker images node-api
REPOSITORY   TAG    SIZE
node-api     1.0    135MB

# Run the container
$ docker run -d -p 3000:3000 --name node-api node-api:1.0
a1b2c3d4e5f6...
```

Test with curl

```bash
# Test the root endpoint
$ curl http://localhost:3000
{
  "service": "Node.js Express API",
  "version": "1.0.0",
  "status": "running"
}

# Test the health endpoint
$ curl http://localhost:3000/health
{
  "status": "healthy",
  "uptime": 12.345
}

# Get all users
$ curl http://localhost:3000/api/users
[
  { "id": 1, "name": "Alice", "email": "alice@example.com" },
  { "id": 2, "name": "Bob", "email": "bob@example.com" },
  { "id": 3, "name": "Charlie", "email": "charlie@example.com" }
]

# Get a specific user
$ curl http://localhost:3000/api/users/1
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com"
}

# Test a non-existent user
$ curl http://localhost:3000/api/users/99
{
  "error": "User not found"
}
```

Clean Up

```bash
$ docker stop node-api && docker rm node-api
```

Application 2: Python Flask API

Flask is a lightweight web framework for Python. It is simple to learn and widely used for APIs.

Project Structure

```
python-api/
+-- Dockerfile
+-- .dockerignore
+-- requirements.txt
+-- app/
    +-- __init__.py
    +-- main.py
    +-- routes/
        +-- __init__.py
        +-- books.py
```

requirements.txt

```
flask==3.0.0
gunicorn==21.2.0
```

A quick explanation:

- flask -- The web framework (like Express for Node.js)
- gunicorn -- A production-grade web server for Python apps. Flask's built-in server is only meant for development. Gunicorn can handle multiple requests at the same time.

Real-life analogy: Flask's built-in server is like a food truck that can serve one customer at a time. Gunicorn is like a full restaurant with multiple waiters serving many customers simultaneously.

app/main.py

```python
import os
from flask import Flask, jsonify
from app.routes.books import books_bp

def create_app():
    app = Flask(__name__)

    # Register blueprints (groups of related routes)
    app.register_blueprint(books_bp, url_prefix='/api/books')

    @app.route('/')
    def index():
        return jsonify({
            'service': 'Python Flask API',
            'version': '1.0.0',
            'status': 'running'
        })

    @app.route('/health')
    def health():
        return jsonify({'status': 'healthy'})

    return app

# Create the app instance
app = create_app()

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port)
```

app/_\init_.py

```python
# This file makes the app directory a Python package
```

app/routes/_\init_.py

```python
# This file makes the routes directory a Python package
```

app/routes/books.py

```python
from flask import Blueprint, jsonify, request

books_bp = Blueprint('books', __name__)

# In-memory data
books = [
    {'id': 1, 'title': 'The Docker Book', 'author': 'James Turnbull', 'year': 2014},
    {'id': 2, 'title': 'Kubernetes in Action', 'author': 'Marko Luksa', 'year': 2018},
    {'id': 3, 'title': 'Site Reliability Engineering', 'author': 'Betsy Beyer', 'year':...
]

@books_bp.route('/', methods=['GET'])
def get_books():
    return jsonify(books)

@books_bp.route('/<int:book_id>', methods=['GET'])
def get_book(book_id):
    book = next((b for b in books if b['id'] == book_id), None)
    if book is None:
        return jsonify({'error': 'Book not found'}), 404
    return jsonify(book)

@books_bp.route('/', methods=['POST'])
def add_book():
    data = request.get_json()
    if not data or 'title' not in data or 'author' not in data:
        return jsonify({'error': 'Title and author are required'}), 400

    new_book = {
        'id': max(b['id'] for b in books) + 1 if books else 1,
        'title': data['title'],
        'author': data['author'],
        'year': data.get('year', 2024)
    }
    books.append(new_book)
    return jsonify(new_book), 201
```

.dockerignore

```
__pycache__
*.pyc
*.pyo
.git
.gitignore
.venv
venv
env
.env
.env.*
README.md
Dockerfile
```

```
docker-compose.yml
*.log
.pytest_cache
.coverage
tests
```

Dockerfile

```dockerfile
# Use Python 3.11 slim image
# slim is smaller than the full image but has more tools than Alpine
# Python on Alpine can have compatibility issues with some packages
FROM python:3.11-slim

# Set environment variables
# PYTHONDONTWRITEBYTECODE: prevents Python from writing .pyc files
# PYTHONUNBUFFERED: ensures print statements appear in Docker logs immediately
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Set working directory
WORKDIR /app

# Copy dependency file first for caching
COPY requirements.txt .

# Install dependencies
# --no-cache-dir: do not save downloaded packages (saves space)
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app/ ./app/

# Create a non-root user
RUN addgroup --system appgroup && \
    adduser --system --ingroup appgroup appuser && \
    chown -R appuser:appgroup /app

# Health check using wget (available on slim images)
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000...

# Switch to non-root user
USER appuser

# Document the port
EXPOSE 5000

# Run with gunicorn (production server)
# -w 4: use 4 worker processes
# -b 0.0.0.0:5000: listen on all interfaces, port 5000
```

```
# app.main:app: the Flask app is in app/main.py, variable named "app"
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app.main:app"]
```

Key differences from the Node.js Dockerfile:

1. Environment variables: `PYTHONDONTWRITEBYTECODE` and `PYTHONUNBUFFERED` are Python-specific optimizations for containers.
2. We use `python:3.11-slim` instead of `alpine`: Python on Alpine can cause issues because Alpine uses `musl` instead of `glibc`. Some Python packages need to be compiled from source on Alpine, which is slow. The `-slim` variant avoids these problems.
3. Gunicorn instead of Flask's dev server: In production, you should always use a proper WSGI server like Gunicorn.

Build and Run

```bash
# Build the image
$ docker build -t python-api:1.0 .
[+] Building 15.1s (11/11) FINISHED
 => [internal] load build definition from Dockerfile          0.0s
 => [internal] load .dockerignore                             0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim             0.0s
 => [2/5] WORKDIR /app                                        0.0s
 => [3/5] COPY requirements.txt .                             0.0s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt  12.3s
 => [5/5] COPY app/ ./app/                                    0.1s
 => exporting to image                                        2.5s

# Check image size
$ docker images python-api
REPOSITORY    TAG    SIZE
python-api    1.0    155MB

# Run the container
$ docker run -d -p 5000:5000 --name python-api python-api:1.0
b2c3d4e5f6g7...
```

Test with curl

```bash
# Test root endpoint
$ curl http://localhost:5000
{
  "service": "Python Flask API",
  "version": "1.0.0",
  "status": "running"
```

```
}

# Test health endpoint
$ curl http://localhost:5000/health
{
  "status": "healthy"
}

# Get all books
$ curl http://localhost:5000/api/books
[
  {"author": "James Turnbull", "id": 1, "title": "The Docker Book", "year": 2014},
  {"author": "Marko Luksa", "id": 2, "title": "Kubernetes in Action", "year": 2018},
  {"author": "Betsy Beyer", "id": 3, "title": "Site Reliability Engineering", "year": 2...
]

# Get a specific book
$ curl http://localhost:5000/api/books/2
{
  "author": "Marko Luksa",
  "id": 2,
  "title": "Kubernetes in Action",
  "year": 2018
}

# Add a new book
$ curl -X POST http://localhost:5000/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Docker Deep Dive", "author": "Nigel Poulton", "year": 2023}'
{
  "author": "Nigel Poulton",
  "id": 4,
  "title": "Docker Deep Dive",
  "year": 2023
}
```

Clean Up

```bash
$ docker stop python-api && docker rm python-api
```

Application 3: Java Spring Boot (with Multi-Stage Build)

Java applications are different from Node.js and Python because they need to be compiled before they can run. This makes multi-stage builds essential.

Real-life analogy: Node.js and Python are like reading a book aloud -- you read the source code directly. Java is like translating a book into another language first (compilation), then reading the translation. You need the translation tools during build time, but not when reading the final book.

Project Structure

```
java-api/
+-- Dockerfile
+-- .dockerignore
+-- pom.xml
+-- src/
    +-- main/
        +-- java/
        |   +-- com/
        |       +-- example/
        |           +-- api/
        |               +-- Application.java
        |               +-- controller/
        |                   +-- ProductController.java
        |                   +-- HealthController.java
        +-- resources/
            +-- application.properties
```

pom.xml (Maven Build Configuration)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.1</version>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>docker-api</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>
```

```
    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

What is Maven? Maven is a build tool for Java (like npm for Node.js or pip for Python). The `pom.xml` file lists your dependencies and tells Maven how to build your project.

What is a JAR? A JAR (Java ARchive) file is a compiled Java application packaged into a single file. It contains all your code and is what you actually run. Think of it like a compiled executable.

Application.java

```java
package com.example.api;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

controller/ProductController.java

```java
package com.example.api.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.*;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final List<Map<String, Object>> products = new ArrayList<>(Arrays.asList(
        createProduct(1, "Laptop", 999.99, "Electronics"),
        createProduct(2, "Headphones", 79.99, "Electronics"),
        createProduct(3, "Coffee Maker", 49.99, "Kitchen")
    ));

    @GetMapping
    public List<Map<String, Object>> getAllProducts() {
        return products;
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getProduct(@PathVariable int id) {
        return products.stream()
            .filter(p -> (int) p.get("id") == id)
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    private static Map<String, Object> createProduct(
            int id, String name, double price, String category) {
        Map<String, Object> product = new HashMap<>();
        product.put("id", id);
        product.put("name", name);
        product.put("price", price);
        product.put("category", category);
        return product;
    }
}
```

controller/HealthController.java

```java
package com.example.api.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import java.util.Map;

@RestController
public class HealthController {

    @GetMapping("/")
    public Map<String, String> index() {
        return Map.of(
            "service", "Java Spring Boot API",
            "version", "1.0.0",
            "status", "running"
        );
    }

    @GetMapping("/health")
    public Map<String, String> health() {
        return Map.of("status", "healthy");
    }
}
```

application.properties

```properties
server.port=8080
management.endpoints.web.exposure.include=health,info
```

.dockerignore

```
.git
.gitignore
target
*.class
*.jar
*.log
.idea
*.iml
.vscode
README.md
Dockerfile
docker-compose.yml
.env
```

Dockerfile (Multi-Stage)

This is where Java Dockerfiles really differ from Node.js and Python. We must use a multi-stage build because:

1. We need the JDK (Java Development Kit) and Maven to build the application
2. We only need the JRE (Java Runtime Environment) to run it
3. The JDK + Maven image is ~400 MB, but the JRE Alpine image is ~100 MB

dockerfile
```dockerfile
# ============================================
# Stage 1: Build the application
# ============================================
# Use Maven with JDK 17 to compile the application
# This stage includes build tools we do NOT need in production
FROM maven:3.9-eclipse-temurin-17 AS builder

WORKDIR /app

# Copy the Maven configuration file first
# This file lists all dependencies
# If it does not change, Maven will use cached dependencies
COPY pom.xml .

# Download all dependencies (cached if pom.xml unchanged)
# go-offline tells Maven to download everything it needs
RUN mvn dependency:go-offline -B

# Copy the source code
COPY src ./src

# Build the application
# -DskipTests: skip running tests during the build
# The result is a JAR file in /app/target/
RUN mvn package -DskipTests -B

# ============================================
# Stage 2: Run the application
# ============================================
# Use only the Java Runtime (JRE), not the full JDK
# Alpine variant for the smallest possible image
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Copy ONLY the compiled JAR from the build stage
# The JAR contains everything the app needs to run
COPY --from=builder /app/target/*.jar app.jar

# Create a non-root user
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:8080/health || exit 1

# Switch to non-root user
USER appuser
```

```
# Document the port
EXPOSE 8080

# Run the JAR file
# java -jar tells Java to run the compiled application
CMD ["java", "-jar", "app.jar"]
```

Key differences from the previous Dockerfiles:

1. Two FROM instructions: Stage 1 uses Maven+JDK for building. Stage 2 uses only the JRE for running.
2. `mvn dependency:go-offline`: Downloads all dependencies. Like `npm install`, this is cached if `pom.xml` has not changed.
3. `mvn package`: Compiles the Java code into a JAR file.
4. `COPY --from=builder`: Copies only the compiled JAR from the build stage.
5. `--start-period=30s`: Java applications take longer to start than Node.js or Python, so we give them more time.

```
+-----------------------------------------------------+
|    Multi-Stage Build for Java                       |
+-----------------------------------------------------+
|                                                     |
|    Stage 1: builder (maven:3.9-eclipse-temurin)     |
|    +----------------------------------------+       |
|    | Maven build tools     150 MB           |       |
|    | JDK (full)            250 MB           |       |
|    | Downloaded deps       100 MB           |       |
|    | Source code             1 MB           |       |
|    | Compiled JAR          30 MB  ---+      |       |
|    +----------------------------------------+ |     |
|                                            |   |     |
|                            Only the JAR    |   |     |
|                            gets copied ---+   |     |
|                                               |     |
|    Stage 2: production (temurin:17-jre)      v     |
|    +----------------------------------------+       |
|    | JRE (runtime only)    100 MB           |       |
|    | app.jar               30 MB            |       |
|    +----------------------------------------+       |
|                                                     |
|    Final image: ~130 MB                             |
|    (instead of ~530 MB with single stage)           |
|                                                     |
+-----------------------------------------------------+
```

Build and Run

```bash
# Build the image (this takes longer because Maven downloads dependencies)
$ docker build -t java-api:1.0 .
[+] Building 85.2s (13/13) FINISHED
 => [builder 1/5] FROM maven:3.9-eclipse-temurin-17           5.2s
 => [builder 2/5] WORKDIR /app                                0.0s
 => [builder 3/5] COPY pom.xml .                              0.0s
 => [builder 4/5] RUN mvn dependency:go-offline               45.3s
 => [builder 5/5] COPY src ./src                              0.1s
 => [builder 6/6] RUN mvn package -DskipTests                 25.1s
 => [stage-1 1/3] FROM eclipse-temurin:17-jre-alpine          3.2s
 => [stage-1 2/3] WORKDIR /app                                0.0s
 => [stage-1 3/3] COPY --from=builder /app/target/*.jar ...   0.5s
 => exporting to image                                        5.8s

# Check image size
$ docker images java-api
REPOSITORY   TAG    SIZE
java-api     1.0    195MB

# Run the container
$ docker run -d -p 8080:8080 --name java-api java-api:1.0
c3d4e5f6g7h8...

# Wait a few seconds for Spring Boot to start
# (Java apps take longer to start than Node.js or Python)
```

Test with curl

```bash
# Test root endpoint
$ curl http://localhost:8080
{
  "service": "Java Spring Boot API",
  "version": "1.0.0",
  "status": "running"
}

# Test health endpoint
$ curl http://localhost:8080/health
{
  "status": "healthy"
}

# Get all products
$ curl http://localhost:8080/api/products
[
  {"id": 1, "name": "Laptop", "price": 999.99, "category": "Electronics"},
  {"id": 2, "name": "Headphones", "price": 79.99, "category": "Electronics"},
  {"id": 3, "name": "Coffee Maker", "price": 49.99, "category": "Kitchen"}
]
```

```
# Get a specific product
$ curl http://localhost:8080/api/products/1
{
  "id": 1,
  "name": "Laptop",
  "price": 999.99,
  "category": "Electronics"
}

# Test non-existent product
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/api/products/99
404
```

Clean Up

```bash
$ docker stop java-api && docker rm java-api
```

Image Size Comparison

Now let us compare all three applications:

```bash
$ docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" | grep -E "node-a...
```

```
+-----------------------------------------------------+
|    Image Size Comparison                            |
+-----------------------------------------------------+
|                                                     |
|    Application        Image Size   Base Image       |
|    ----------------------------------------------   |
|    Node.js Express    ~135 MB      node:18-alpine|
|    Python Flask       ~155 MB      python:3.11-slim|
|    Java Spring Boot   ~195 MB      temurin:17-jre |
|                                                     |
+-----------------------------------------------------+
```

Why the Sizes Differ

```
+-----------------------------------------------------+
|    What Makes Up Each Image                         |
+-----------------------------------------------------+
|                                                     |
|    Node.js (135 MB):                                |
|    +-- Alpine Linux base        7 MB                |
|    +-- Node.js runtime        110 MB                |
|    +-- npm packages            15 MB                |
|    +-- Application code         3 MB                |
|                                                     |
|    Python (155 MB):                                 |
|    +-- Debian slim base        74 MB                |
|    +-- Python runtime          50 MB                |
|    +-- pip packages            28 MB                |
|    +-- Application code         3 MB                |
|                                                     |
|    Java (195 MB):                                   |
|    +-- Alpine base              7 MB                |
|    +-- JRE runtime            100 MB                |
|    +-- Spring Boot JAR         85 MB                |
|    +-- Application code         3 MB                |
|                                                     |
+-----------------------------------------------------+
```

Single Stage vs Multi-Stage Comparison (Java)

The Java application benefits the most from multi-stage builds:

```
+-----------------------------------------------------+
|    Java: Single Stage vs Multi-Stage                |
+-----------------------------------------------------+
|                                                     |
|    Single stage (maven + JDK):  ~530 MB             |
|    Multi-stage (JRE only):      ~195 MB             |
|    Savings:                     ~335 MB (63%)       |
|                                                     |
|    The single stage includes:                       |
|    - Full JDK (not needed at runtime)               |
|    - Maven (not needed at runtime)                  |
|    - Downloaded source dependencies                 |
|    - Compiled .class files AND the JAR              |
|                                                     |
+-----------------------------------------------------+
```

Side-by-Side Dockerfile Comparison

Here is a quick reference comparing the Dockerfile patterns for each language:

```
+-------------------+-------------------+-------------------+
| Node.js           | Python            | Java              |
+-------------------+-------------------+-------------------+
| FROM node:alpine  | FROM python:slim  | FROM maven AS build|
|                   |                   | FROM jre-alpine    |
+-------------------+-------------------+-------------------+
| COPY package.json | COPY requirements | COPY pom.xml      |
| RUN npm ci        | RUN pip install   | RUN mvn deps      |
+-------------------+-------------------+-------------------+
| COPY . .          | COPY app/ ./app/  | COPY src ./src    |
|                   |                   | RUN mvn package   |
+-------------------+-------------------+-------------------+
| USER node         | USER appuser      | USER appuser      |
+-------------------+-------------------+-------------------+
| CMD ["node",...]  | CMD ["gunicorn",.]| CMD ["java","-jar"|
+-------------------+-------------------+-------------------+
| Multi-stage: No   | Multi-stage: No   | Multi-stage: Yes  |
| (interpreted)     | (interpreted)     | (compiled)        |
+-------------------+-------------------+-------------------+
```

Node.js and Python are interpreted languages -- the runtime reads and executes the source code directly. Java is a compiled language -- you must compile the source code into bytecode first, then the JRE executes the bytecode.

This is why Java benefits most from multi-stage builds: the compilation tools are large and not needed at runtime.

Common Mistakes

Mistake 1: Using the Development Server in Production

```dockerfile
# BAD: Flask's built-in server cannot handle production traffic
CMD ["python", "app.py"]

# GOOD: Use gunicorn for production
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app.main:app"]
```

Mistake 2: Not Using Multi-Stage for Java

```dockerfile
# BAD: Final image includes Maven, JDK, and all build tools
FROM maven:3.9-eclipse-temurin-17
COPY . .
RUN mvn package
CMD ["java", "-jar", "target/app.jar"]
# Image size: ~530 MB

# GOOD: Final image has only JRE and the JAR
# (use multi-stage as shown above)
# Image size: ~195 MB
```

Mistake 3: Copying Everything Instead of What You Need

```dockerfile
# BAD: Copies tests, docs, config files you do not need
COPY . .

# GOOD: Copy only what the application needs
COPY src/ ./src/
COPY package.json package-lock.json ./
```

Mistake 4: Not Setting PYTHONDONTWRITEBYTECODE

```dockerfile
# BAD: Python writes .pyc files that waste space in the container
FROM python:3.11-slim

# GOOD: Prevent .pyc files and ensure logs appear immediately
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
```

Mistake 5: Forgetting That Java Takes Longer to Start

```dockerfile
# BAD: Java may not be ready in 5 seconds
HEALTHCHECK --start-period=5s CMD wget ...

# GOOD: Give Java enough time to start
HEALTHCHECK --start-period=30s CMD wget ...
```

Best Practices

1. Choose the right base image for your language. Alpine for Node.js, slim for Python, JRE-alpine for Java runtime.
2. Use multi-stage builds for compiled languages. Java and Go applications especially benefit from this.
3. Use production-grade servers. Gunicorn for Python, not Flask's dev server. In Node.js, Express is fine for production but consider PM2 for process management.
4. Copy only what you need. Specify exact files and directories instead of `COPY . .` when possible.
5. Set language-specific environment variables. Like `PYTHONDONTWRITEBYTECODE` for Python or `NODE_ENV=production` for Node.js.
6. Adjust health check timing for your language. Java needs a longer `--start-period` than Node.js.
7. Always test your containerized application with curl before deploying to make sure all endpoints work correctly.

Quick Summary

In this chapter, you built three complete Docker images for real applications:

1. Node.js Express API -- Used a single-stage build with `node:18-alpine`, installed dependencies with `npm ci`, and ran with the default Node.js runtime.
2. Python Flask API -- Used `python:3.11-slim` with Gunicorn as the production server. Set Python-specific environment variables for container optimization.
3. Java Spring Boot API -- Used a multi-stage build with Maven+JDK for compilation and JRE-alpine for runtime. Multi-stage reduced the image size by 63%.

Each application followed best practices: layer caching, non-root users, health checks, and `.dockerignore` files.

Key Points

- Different languages require different Dockerfile patterns
- Compiled languages (Java, Go) benefit most from multi-stage builds
- Always use production-grade servers (Gunicorn for Python, not Flask's dev server)
- Copy dependency files before source code for optimal caching

- Set language-specific environment variables (PYTHONDONTWRITEBYTECODE, NODE_ENV)
- Adjust health check timing based on application startup speed
- Image sizes vary by language: Node.js (~135 MB) < Python (~155 MB) < Java (~195 MB) in our examples
- Multi-stage builds can reduce Java images from ~530 MB to ~195 MB
- Always test endpoints with curl after building and running

Practice Questions

1. Why do we use `python:3.11-slim` instead of `python:3.11-alpine` for the Flask application? What problems can Alpine cause with Python?
2. Explain why Java applications benefit more from multi-stage builds than Node.js or Python applications. What is different about Java?
3. Why do we use Gunicorn instead of Flask's built-in server in the Dockerfile? What is the real-life analogy?
4. In the Java Dockerfile, what does `COPY --from=builder /app/target/*.jar app.jar` do? What would happen if we did not use multi-stage and just had a single FROM?
5. Looking at the image size comparison, what accounts for most of the size in each image? Is it the application code or something else?

Exercises

Exercise 1: Build and Test All Three

Follow the instructions in this chapter to build and test all three applications. For each one:

1. Create the project structure and files
2. Build the Docker image
3. Run the container
4. Test all endpoints with curl
5. Check the image size with `docker images`
6. Compare the sizes

Exercise 2: Add a New Endpoint

Choose one of the three applications and add a new endpoint. For example:

- Node.js: Add a `POST /api/users` endpoint that creates a new user
- Python: Add a `DELETE /api/books/:id` endpoint that removes a book
- Java: Add a `PUT /api/products/:id` endpoint that updates a product

Rebuild the image and test the new endpoint.

Exercise 3: Optimize the Java Image Further

The Java image we built is ~195 MB. Try these optimizations:

1. Use `eclipse-temurin:17-jre-alpine` with jlink to create a custom JRE
2. Add `spring-boot-maven-plugin` layered JAR support
3. Compare the image sizes

What Is Next?

You now know how to build real application images for three popular languages. But building images is only half the story. In the next chapter, you will learn about the container lifecycle -- how containers are created, started, stopped, restarted, and removed. You will learn essential commands for managing containers in development and production.

Chapter 9: The Container Lifecycle

What You Will Learn

By the end of this chapter, you will be able to:

- Understand the complete container lifecycle and all possible states
- Create containers without starting them using `docker create`
- Start, stop, restart, pause, and unpause containers
- Remove containers and clean up resources
- View real-time logs with `docker logs -f`
- Execute commands inside running containers with `docker exec`
- Attach to a running container's process with `docker attach`
- Copy files between your computer and containers with `docker cp`
- Monitor container resource usage with `docker stats`
- View running processes inside containers with `docker top`

Why This Chapter Matters

So far, you have been using `docker run` to create and start containers, and `docker stop` to shut them down. But containers have a much richer lifecycle than just "running" and "stopped."

Real-life analogy: Think of a container like a car. You do not just start it and park it. You can idle it, pause it at a red light, turn off the engine while keeping the car in place, and eventually sell it for scrap. Understanding all these states gives you full control over your containers.

In production, you need to debug containers that are misbehaving, check their logs, copy files in and out, and monitor their resource usage. This chapter teaches you every tool you need for day-to-day container management.

Container States

A Docker container can be in one of several states. Here is the complete lifecycle:

```
+-----------------------------------------------------+
|           Container Lifecycle Diagram               |
+-----------------------------------------------------+
|                                                     |
|                docker create                        |
|                     |                               |
|                     v                               |
|               +-----------+                         |
|               | CREATED   |                         |
|               +-----------+                         |
|                     |                               |
|               docker start                          |
|                     |                               |
|                     v                               |
|               +-----------+    docker pause         |
|          +--->| RUNNING   |----------------+        |
|          |    +-----------+                |        |
|          |       |    |                    v        |
|          |       |    |              +-----------+|
|          |       |    |              | PAUSED    ||
|          |       |    |              +-----------+|
|          |       |    |                    |        |
|          |       |    |           docker unpause   |
|          |       |    |                    |        |
|          |       |    +<-------------------+        |
|          |       |                                 |
|    docker|  docker stop                            |
|  restart |  (or app exits)                         |
|          |       |                                 |
|          |       v                                 |
|          |  +-----------+                          |
|          +--| STOPPED   |                          |
|             +-----------+                          |
|                  |                                 |
|             docker rm                              |
|                  |                                 |
|                  v                                 |
|             +-----------+                          |
|             | REMOVED   |                          |
|             +-----------+                          |
|                                                    |
+-----------------------------------------------------+
```

State Descriptions

```
+-------------+-------------------------------------------------+
| State       | Description                                     |
+-------------+-------------------------------------------------+
| Created     | Container exists but has never been started.    |
|             | Like a car that has been built but never        |
|             | had the engine turned on.                       |
+-------------+-------------------------------------------------+
| Running     | Container is actively executing its process.    |
|             | Like a car with the engine running.             |
+-------------+-------------------------------------------------+
| Paused      | Container's processes are suspended (frozen).   |
|             | Like pressing pause on a video. Everything      |
|             | freezes but nothing is lost.                    |
+-------------+-------------------------------------------------+
| Stopped     | Container's main process has exited. The        |
|             | container still exists with all its data.       |
|             | Like a car with the engine turned off but       |
|             | still parked in the garage.                     |
+-------------+-------------------------------------------------+
| Removed     | Container has been deleted. It no longer        |
|             | exists. Like selling a car for scrap.           |
+-------------+-------------------------------------------------+
```

---

docker create: Creating Without Starting

`docker create` creates a new container but does not start it. The container sits in the "Created" state.

Real-life analogy: It is like setting up a tent at a campsite but not going inside yet. The tent is there, ready to be used, but nobody is in it.

Syntax

```bash
$ docker create [options] image [command]
```

Example

```bash
# Create a container without starting it
$ docker create --name my-web -p 8080:80 nginx:alpine
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6
```

```
# Check its status
$ docker ps -a --filter "name=my-web"
CONTAINER ID   IMAGE          STATUS    NAMES
a1b2c3d4e5f6   nginx:alpine   Created   my-web
```

Notice the status says "Created," not "Up."

When to Use docker create

- When you want to set up a container in advance and start it later
- When you need to inspect a container's configuration before running it
- When building automation scripts that prepare containers first

---

docker start: Starting a Container

`docker start` starts a container that is in the "Created" or "Stopped" state.

Syntax

```bash
$ docker start [options] container_name_or_id
```

Example

```bash
# Start the container we created above
$ docker start my-web
my-web

# Verify it is running
$ docker ps
CONTAINER ID   IMAGE          STATUS          NAMES
a1b2c3d4e5f6   nginx:alpine   Up 3 seconds   my-web
```

Start with Attached Output

By default, `docker start` runs in the background. To see the output:

```bash
# Start and attach to see output (like docker run without -d)
$ docker start -a my-web
/docker-entrypoint.sh: Configuration complete; ready for start up
```

The `-a` flag attaches your terminal to the container's output, so you can see what it prints.

---

docker stop: Stopping a Container Gracefully

`docker stop` sends a SIGTERM signal to the main process in the container, giving it time to shut down gracefully. If the process does not stop within 10 seconds (by default), Docker sends a SIGKILL to force it to stop.

Real-life analogy: `docker stop` is like telling someone "Please finish what you are doing and leave the room." You give them time to clean up. If they do not leave within 10 seconds, security escorts them out.

Syntax

```bash
$ docker stop [options] container_name_or_id
```

Examples

```bash
# Stop a container (waits up to 10 seconds)
$ docker stop my-web
my-web

# Stop with a custom timeout (wait up to 30 seconds)
$ docker stop -t 30 my-web
my-web

# Stop multiple containers at once
$ docker stop web-1 web-2 web-3
web-1
web-2
web-3
```

What Happens During Stop

```
+-----------------------------------------------------+
|    What docker stop Does                            |
+-----------------------------------------------------+
|                                                     |
|    1. Docker sends SIGTERM to the main process      |
|       "Please shut down gracefully"                 |
|                                                     |
|    2. Application receives the signal               |
|       - Closes database connections                 |
|       - Finishes processing current requests        |
|       - Saves state if needed                       |
|       - Exits cleanly                               |
|                                                     |
|    3. If still running after timeout (10s):         |
|       Docker sends SIGKILL                          |
|       "You are being forced to stop NOW"            |
|                                                     |
+-----------------------------------------------------+
```

docker stop vs docker kill

```bash
# docker stop: graceful shutdown (SIGTERM, then SIGKILL after timeout)
$ docker stop my-web

# docker kill: immediate shutdown (SIGKILL, no grace period)
$ docker kill my-web
```

Use `docker stop` in normal situations. Use `docker kill` only when a container is unresponsive and will not stop gracefully.

---

docker restart: Stop and Start in One Step

`docker restart` stops a running container and immediately starts it again.

Real-life analogy: It is like turning your computer off and back on again. The classic "have you tried restarting it?" fix.

Syntax

```bash
$ docker restart [options] container_name_or_id
```

Examples

```bash
# Restart a container
$ docker restart my-web
my-web

# Restart with a custom stop timeout
$ docker restart -t 5 my-web
my-web
```

What Happens During Restart

```
+---------------------------------------------------+
|    Restart Process                                |
+---------------------------------------------------+
|                                                   |
|    RUNNING -----> STOPPED -----> RUNNING          |
|              stop           start                 |
|                                                   |
|    1. Send SIGTERM (graceful stop)                |
|    2. Wait for process to exit                    |
|    3. Start the container again                   |
|    4. Container gets a new process                |
|       (but keeps the same filesystem)             |
|                                                   |
+---------------------------------------------------+
```

Important: Restarting a container does NOT reset its filesystem. Any files created or modified inside the container are still there after the restart. However, the process starts fresh (new PID, reset memory).

---

docker pause and docker unpause: Freezing a Container

`docker pause` freezes all processes in a container. The container is still in memory, but nothing executes. `docker unpause` resumes the processes exactly where they left off.

Real-life analogy: Pause is like pressing the pause button on a video game. The game freezes completely but nothing is lost. When you unpause, everything continues exactly where it was.

Syntax

```bash
$ docker pause container_name_or_id
$ docker unpause container_name_or_id
```

Example

```bash
# Start a container
$ docker run -d --name my-web -p 8080:80 nginx:alpine
a1b2c3d4...

# Verify it is running
$ curl http://localhost:8080
<!DOCTYPE html>...  (nginx welcome page)

# Pause the container
$ docker pause my-web
my-web

# Check the status
$ docker ps
CONTAINER ID   IMAGE          STATUS                 NAMES
a1b2c3d4       nginx:alpine   Up 1 min (Paused)      my-web

# Try to access the website (it will hang/timeout)
$ curl --max-time 3 http://localhost:8080
curl: (28) Operation timed out

# Unpause the container
$ docker unpause my-web
my-web

# It works again!
$ curl http://localhost:8080
<!DOCTYPE html>...  (nginx welcome page)
```

When to Use Pause

- Temporarily stop a container from using CPU without stopping it
- During backup operations where you need a consistent snapshot
- When debugging and you want to freeze state at a specific point

Pause vs Stop

```
+-----------------------------------------------------+
|    Pause vs Stop Comparison                         |
+-----------------------------------------------------+
|                                                     |
|    Feature          Pause         Stop              |
|    ---------------- ------------- --------------- |
|    Process state    Frozen        Terminated        |
|    Memory           Preserved     Released          |
|    Resume speed     Instant       Needs restart     |
|    Network          Frozen        Closed            |
|    Filesystem       Unchanged     Unchanged         |
|    Use case         Temporary     Longer term       |
|                     freeze        shutdown          |
|                                                     |
+-----------------------------------------------------+
```

---

docker rm: Removing Containers

`docker rm` permanently deletes a container. This is irreversible -- once removed, the container and any data stored in its writable layer are gone forever.

Real-life analogy: `docker rm` is like throwing away a cardboard box. Once it is in the trash and the garbage truck takes it, you cannot get it back.

Syntax

```bash
$ docker rm [options] container_name_or_id
```

Examples

```bash
# Remove a stopped container
$ docker rm my-web
my-web

# Try to remove a running container (this fails!)
$ docker rm my-web
Error response from daemon: cannot remove container: container is running

# Force remove a running container (stops it first)
$ docker rm -f my-web
```

```
my-web

# Remove multiple containers
$ docker rm web-1 web-2 web-3

# Remove all stopped containers
$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
a1b2c3d4e5f6
b2c3d4e5f6g7
Total reclaimed space: 25.3MB
```

The --rm Flag: Auto-Remove

You can tell Docker to automatically remove a container when it stops:

```bash
# Container is automatically removed when it exits
$ docker run --rm --name temp-web -p 8080:80 nginx:alpine

# When you stop it, it is automatically removed
$ docker stop temp-web
# No need to docker rm -- it is already gone!

$ docker ps -a | grep temp-web
# (nothing -- the container was auto-removed)
```

This is very useful for temporary containers that you do not need to keep.

---

docker logs: Viewing Container Output

`docker logs` shows you what a container has been printing to its standard output (stdout) and standard error (stderr). This is essential for debugging.

Real-life analogy: `docker logs` is like checking a security camera recording. You can see what happened inside the container, even if you were not watching at the time.

Syntax

```bash
$ docker logs [options] container_name_or_id
```

Basic Examples

```bash
# View all logs
$ docker logs my-web

# View the last 20 lines
$ docker logs --tail 20 my-web

# View logs with timestamps
$ docker logs -t my-web
2024-01-15T10:30:00.123Z  Server running on port 3000
2024-01-15T10:30:05.456Z  GET / 200 12ms
2024-01-15T10:30:10.789Z  GET /api/users 200 8ms
```

Following Logs in Real Time

The `-f` flag (follow) streams logs in real time, like `tail -f`:

```bash
# Follow logs in real time (press Ctrl+C to stop)
$ docker logs -f my-web
Server running on port 3000
GET / 200 12ms
GET /api/users 200 8ms
... (new logs appear as they happen) ...
```

Combining Options

```bash
# Show the last 10 lines and follow new ones
$ docker logs --tail 10 -f my-web

# Show logs since a specific time
$ docker logs --since 2024-01-15T10:00:00 my-web

# Show logs from the last 5 minutes
$ docker logs --since 5m my-web

# Show logs with timestamps, last 5 lines, following
$ docker logs -t --tail 5 -f my-web
```

Practical Debugging Example

```bash
# Your application is crashing. Let us investigate:

# Step 1: Check if the container is running
$ docker ps -a --filter "name=my-app"
CONTAINER ID   IMAGE    STATUS                     NAMES
a1b2c3d4       my-app   Exited (1) 2 minutes ago   my-app

# Status "Exited (1)" means the app crashed (exit code 1)

# Step 2: Check the logs to see what happened
$ docker logs my-app
Server starting on port 3000
Connected to database
Error: Cannot read properties of undefined (reading 'id')
    at /app/src/routes/users.js:15:28
    at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:...

# Now you can see the exact error and file/line number!
```

---

docker exec: Running Commands Inside a Container

`docker exec` lets you run a command inside a running container. This is one of the most frequently used Docker commands for debugging and administration.

Real-life analogy: `docker exec` is like reaching through a window into a room and pressing buttons. The room (container) is already there and running. You are just interacting with it from the outside.

Syntax

```bash
$ docker exec [options] container command [arguments]
```

Common Examples

```bash
# Run a simple command
$ docker exec my-web ls /app
node_modules
package.json
```

```
server.js

# Check the current user
$ docker exec my-web whoami
node

# Check environment variables
$ docker exec my-web env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NODE_VERSION=18.19.0
...

# Check running processes
$ docker exec my-web ps aux
PID   USER     COMMAND
    1 node     node server.js
   25 node     ps aux
```

Interactive Shell Access

The most common use of `docker exec` is to open an interactive shell inside the container:

```bash
# Open a bash shell inside the container
$ docker exec -it my-web /bin/bash
root@a1b2c3d4:/app#

# If the container does not have bash (like Alpine), use sh
$ docker exec -it my-web /bin/sh
/app $
```

Breaking down the flags:

- `-i` (interactive) -- Keeps stdin open so you can type commands
- `-t` (tty) -- Allocates a terminal so you see a prompt and can use terminal features

Together, `-it` gives you an interactive terminal session inside the container.

Inside the Container

Once inside, you can explore:

```bash
# You are now INSIDE the container
/app $ ls -la
total 32
drwxr-xr-x  1 node node 4096 Jan 15 10:30 .
drwxr-xr-x  1 root root 4096 Jan 15 10:30 ..
```

```
drwxr-xr-x 50 node node 4096 Jan 15 10:30 node_modules
-rw-r--r--  1 node node  320 Jan 15 10:30 package.json
-rw-r--r--  1 node node  580 Jan 15 10:30 server.js

# Check memory usage
/app $ free -m
              total       used       free
Mem:           7972       1234       5678

# Check disk usage
/app $ df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay         59G   12G    44G  22% /

# Check network connectivity
/app $ wget -q -O- http://localhost:3000/health
{"status":"healthy"}

# Exit the container shell
/app $ exit
```

Running Commands as a Different User

```bash
# Run as root (even if the container normally runs as non-root)
$ docker exec -u root my-web whoami
root

# Run as a specific user
$ docker exec -u node my-web whoami
node
```

Setting Environment Variables

```bash
# Run a command with an additional environment variable
$ docker exec -e DEBUG=true my-web env | grep DEBUG
DEBUG=true
```

---

docker attach: Connecting to the Main Process

`docker attach` connects your terminal to the container's main process (PID 1). This is different from `docker exec` because you are connecting to the existing process, not starting a new one.

Real-life analogy: `docker exec -it bash` is like opening a new window into a room. `docker attach` is like picking up an already-connected phone call.

Syntax

```bash
$ docker attach [options] container_name_or_id
```

Example

```bash
# Start a container
$ docker run -d --name my-web -p 3000:3000 node-api:1.0

# Attach to it (you will see its output)
$ docker attach my-web
Server running on port 3000
GET / 200 12ms
GET /api/users 200 8ms
```

Important Warning About docker attach

When you press `Ctrl+C` while attached, it sends the signal to the container's main process. This can stop the container:

```bash
# Attached to my-web
$ docker attach my-web
Server running on port 3000
^C   # <-- Pressing Ctrl+C STOPS the container!
```

To detach without stopping the container, use the key sequence `Ctrl+P, Ctrl+Q`:

```bash
# Detach without stopping
$ docker attach my-web
Server running on port 3000
# Press Ctrl+P, then Ctrl+Q
read escape sequence
$  # Back to your terminal, container still running
```

When to Use attach vs exec

```
+---------------------------------------------------+
|    attach vs exec                                 |
+---------------------------------------------------+
|                                                   |
|    Use docker attach when:                        |
|    - You want to see the main process output      |
|    - You want to interact with the main process   |
|    - You are running an interactive app           |
|                                                   |
|    Use docker exec when:                          |
|    - You want to run a NEW command                |
|    - You want to debug (open a shell)             |
|    - You want to inspect files or processes       |
|    - You do NOT want to risk stopping the         |
|      container (safer option)                     |
|                                                   |
+---------------------------------------------------+
```

In practice, `docker exec` is used much more frequently than `docker attach`.

---

docker cp: Copying Files Between Host and Container

`docker cp` copies files and directories between your computer (the host) and a container. The container can be running or stopped.

Real-life analogy: `docker cp` is like passing items through a window. You can hand things in (host to container) or take things out (container to host).

Syntax

```bash
# Copy from host to container
$ docker cp local_path container:container_path

# Copy from container to host
$ docker cp container:container_path local_path
```

Examples: Host to Container

```bash
# Copy a file into a running container
$ docker cp config.json my-web:/app/config.json

# Copy a directory into a container
$ docker cp ./logs my-web:/app/logs

# Verify the file was copied
$ docker exec my-web ls /app/config.json
/app/config.json
```

Examples: Container to Host

```bash
# Copy a log file from the container to your computer
$ docker cp my-web:/app/logs/error.log ./error.log

# Copy the entire app directory from the container
$ docker cp my-web:/app ./container-app

# Copy from a stopped container (this also works!)
$ docker stop my-web
$ docker cp my-web:/app/data.db ./backup-data.db
```

Practical Use Cases

```bash
# Use Case 1: Quick config update without rebuilding
$ docker cp new-nginx.conf my-web:/etc/nginx/nginx.conf
$ docker restart my-web

# Use Case 2: Extract logs for analysis
$ docker cp my-web:/var/log/app.log ./app.log

# Use Case 3: Debug a crash by copying the core dump
$ docker cp crashed-app:/app/core.dump ./core.dump

# Use Case 4: Add test data to a running container
$ docker cp test-data.json my-web:/app/data/test-data.json
```

Warning: Copying files into a container is a temporary fix. The changes will be lost when the container is removed. For permanent changes, update your Dockerfile and rebuild.

---

docker stats: Monitoring Resource Usage

`docker stats` shows real-time resource usage statistics for running containers, similar to the `top` command on Linux.

Real-life analogy: `docker stats` is like the dashboard in your car. It shows you how much fuel (CPU) you are using, the engine temperature (memory), and how fast you are going (network traffic).

Syntax

```bash
$ docker stats [options] [container_names...]
```

Example

```bash
$ docker stats
CONTAINER ID   NAME        CPU %   MEM USAGE / LIMIT     MEM %   NET I/O          BLOCK...
a1b2c3d4       node-api    0.15%   45.2MiB / 7.77GiB     0.57%   1.2kB / 648B     0B / 0B
b2c3d4e5       python-api  0.08%   62.1MiB / 7.77GiB     0.78%   1.1kB / 580B     0B / 0B
c3d4e5f6       java-api    0.52%   185MiB / 7.77GiB      2.32%   2.3kB / 1.1kB    4.1MB...
```

Understanding the Output

```
+--------------------------------------------------+
|    docker stats Columns Explained                |
+--------------------------------------------------+
|                                                  |
|    CONTAINER ID  Unique container identifier     |
|    NAME          Container name                  |
|    CPU %         CPU usage as a percentage       |
|    MEM USAGE     Current memory used             |
|    / LIMIT       Maximum memory allowed          |
|    MEM %         Memory usage as a percentage    |
|    NET I/O       Network data in / out           |
|    BLOCK I/O     Disk read / write               |
|                                                  |
+--------------------------------------------------+
```

Monitor Specific Containers

```bash
# Monitor only specific containers
$ docker stats node-api python-api

# Show stats once (not streaming)
$ docker stats --no-stream
CONTAINER ID   NAME        CPU %   MEM USAGE / LIMIT     MEM %
a1b2c3d4       node-api    0.15%   45.2MiB / 7.77GiB     0.57%
b2c3d4e5       python-api  0.08%   62.1MiB / 7.77GiB     0.78%

# Custom format
$ docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
NAME          CPU %     MEM USAGE / LIMIT
node-api      0.15%     45.2MiB / 7.77GiB
python-api    0.08%     62.1MiB / 7.77GiB
```

Comparing Resource Usage Across Languages

```
+-----------------------------------------------------+
|    Resource Usage by Language                       |
+-----------------------------------------------------+
|                                                     |
|    Application     Idle CPU  Idle Memory            |
|    ------------------------------------------- |
|    Node.js Express  ~0.1%    ~45 MB                 |
|    Python Flask      ~0.1%    ~62 MB                |
|    Java Spring Boot  ~0.5%    ~185 MB               |
|                                                     |
|    Note: Java uses more memory because the JVM      |
|    pre-allocates memory for the heap.               |
|                                                     |
+-----------------------------------------------------+
```

---

docker top: Viewing Processes Inside a Container

`docker top` shows the running processes inside a container, similar to the `ps` command.

Real-life analogy: `docker top` is like looking through a window to see who is inside a room and what they are doing.

Syntax

```bash
$ docker top container_name_or_id
```

Example

```bash
# View processes in a Node.js container
$ docker top node-api
UID    PID    PPID   C   STIME   TTY   TIME       CMD
node   1234   1233   0   10:30   ?     00:00:02   node src/server.js

# View processes in a Python container (with gunicorn workers)
$ docker top python-api
UID      PID    PPID   C   STIME   TTY   TIME       CMD
appuser  2345   2344   0   10:31   ?     00:00:01   gunicorn: master [app.main:app]
appuser  2346   2345   0   10:31   ?     00:00:00   gunicorn: worker [app.main:app]
appuser  2347   2345   0   10:31   ?     00:00:00   gunicorn: worker [app.main:app]
appuser  2348   2345   0   10:31   ?     00:00:00   gunicorn: worker [app.main:app]
appuser  2349   2345   0   10:31   ?     00:00:00   gunicorn: worker [app.main:app]
```

Using Custom ps Options

You can pass custom formatting options:

```bash
# Show specific fields
$ docker top my-web -o pid,user,%cpu,%mem,command
PID    USER    %CPU   %MEM   COMMAND
1234   node    0.1    0.5    node src/server.js
```

Putting It All Together: A Debugging Session

Here is a realistic scenario that combines several commands:

```bash
# Scenario: Your web application is not responding to requests

# Step 1: Check if the container is running
$ docker ps -a --filter "name=my-app"
CONTAINER ID   IMAGE    STATUS        NAMES
a1b2c3d4       my-app   Up 2 hours    my-app

# It says "Up" but let us dig deeper

# Step 2: Check resource usage
```

```
$ docker stats --no-stream my-app
NAME     CPU %   MEM USAGE / LIMIT     MEM %
my-app   98.5%   750MiB / 1GiB         73.2%

# CPU is at 98.5%! Something is using too much CPU

# Step 3: Check what processes are running
$ docker top my-app
UID    PID    CMD
node   1234   node src/server.js
node   1235   node src/worker.js   # <-- suspicious!

# Step 4: Check the logs
$ docker logs --tail 50 my-app
...
[ERROR] Worker stuck in infinite loop processing job #4521
[WARN] Memory usage approaching limit
[ERROR] Worker stuck in infinite loop processing job #4521
...

# Step 5: Get a shell inside the container for investigation
$ docker exec -it my-app /bin/sh

/app $ ls -la src/worker.js
-rw-r--r-- 1 node node 2048 Jan 15 10:30 src/worker.js

/app $ cat src/worker.js | head -20
# (examine the file to find the bug)

/app $ exit

# Step 6: Copy the problematic file out for editing
$ docker cp my-app:/app/src/worker.js ./worker.js

# Step 7: Fix the file and copy it back
$ vim ./worker.js  # (fix the infinite loop)
$ docker cp ./worker.js my-app:/app/src/worker.js

# Step 8: Restart the container
$ docker restart my-app

# Step 9: Verify it is working
$ docker stats --no-stream my-app
NAME     CPU %   MEM USAGE / LIMIT     MEM %
my-app   0.2%    45MiB / 1GiB          4.4%

# CPU is back to normal!

# Step 10: Follow logs to confirm
$ docker logs -f --tail 10 my-app
Server running on port 3000
Worker processing job #4522 - completed
Worker processing job #4523 - completed
...
```

Quick Reference: Container Management Commands

```
+-----------------------------------------------------+
|    Container Lifecycle Commands                     |
+-----------------------------------------------------+
|                                                     |
|    CREATE:                                          |
|    docker create    Create without starting         |
|    docker run       Create and start (combined)     |
|                                                     |
|    STATE CHANGES:                                   |
|    docker start     Created/Stopped -> Running      |
|    docker stop      Running -> Stopped              |
|    docker restart   Stop then Start                 |
|    docker pause     Running -> Paused               |
|    docker unpause   Paused -> Running               |
|    docker kill      Force stop immediately          |
|                                                     |
|    REMOVE:                                          |
|    docker rm        Delete a stopped container      |
|    docker rm -f     Force delete (even running)     |
|    docker prune     Delete ALL stopped containers|
|                                                     |
|    INSPECT:                                         |
|    docker logs      View container output           |
|    docker logs -f   Follow logs in real time        |
|    docker exec      Run command in container        |
|    docker attach    Connect to main process         |
|    docker cp        Copy files in/out               |
|    docker stats     Monitor resource usage          |
|    docker top       View running processes          |
|                                                     |
+-----------------------------------------------------+
```

Common Mistakes

Mistake 1: Trying to Remove a Running Container

```bash
# This fails
$ docker rm my-web
Error: cannot remove running container

# Solution 1: Stop first, then remove
$ docker stop my-web && docker rm my-web

# Solution 2: Force remove
$ docker rm -f my-web
```

Mistake 2: Pressing Ctrl+C in docker attach

```bash
# This STOPS the container!
$ docker attach my-web
^C  # Container stops!

# Instead, detach with Ctrl+P, Ctrl+Q
# Or use docker exec instead of attach
```

Mistake 3: Forgetting --rm for Temporary Containers

```bash
# BAD: creates containers that pile up
$ docker run -d nginx:alpine
$ docker run -d nginx:alpine
$ docker run -d nginx:alpine
# Now you have 3 containers to clean up

# GOOD: auto-remove when done
$ docker run --rm -d nginx:alpine
```

Mistake 4: Not Checking Logs When Something Fails

```bash
# Container keeps restarting? Check the logs!
$ docker logs my-app
Error: Cannot find module 'express'
# The issue is clear: express is not installed
```

Mistake 5: Using docker attach Instead of docker exec

```bash
# docker attach connects to PID 1 -- can be dangerous
# docker exec creates a NEW process -- much safer

# PREFERRED for debugging:
$ docker exec -it my-web /bin/sh

# ONLY use attach when you specifically need to interact
# with the main process (rare)
```

Best Practices

1. Use `--rm` for temporary containers to prevent a buildup of stopped containers that consume disk space.
2. Use `docker exec` instead of `docker attach` for debugging. It is safer and more flexible.
3. Always check logs first when debugging container issues. Most problems are visible in the logs.
4. Use `docker stats` to monitor resource usage and catch runaway processes.
5. Name your containers with `--name` so you can refer to them easily instead of using container IDs.
6. Use `docker logs --tail N -f` to follow recent logs without seeing the entire log history.
7. Stop containers gracefully with `docker stop` before removing them. Only use `docker kill` or `docker rm -f` as a last resort.
8. Regularly prune stopped containers with `docker container prune` to free up disk space.

Quick Summary

Containers have a complete lifecycle with five states: Created, Running, Paused, Stopped, and Removed. Docker provides commands to transition between each state. For debugging, `docker logs` shows output, `docker exec` lets you run commands inside containers, and `docker stats` monitors resource usage. `docker cp` copies files between your computer and containers. Understanding these commands is essential for managing containers in both development and production.

Key Points

- Containers have five states: Created, Running, Paused, Stopped, Removed
- `docker create` makes a container without starting it; `docker run` does both
- `docker stop` sends SIGTERM for graceful shutdown; `docker kill` forces immediate stop
- `docker pause` freezes processes in memory; `docker unpause` resumes them
- `docker logs -f` follows container output in real time
- `docker exec -it container /bin/sh` opens an interactive shell for debugging
- `docker attach` connects to PID 1 (be careful with Ctrl+C)
- `docker cp` works in both directions and even on stopped containers
- `docker stats` shows CPU, memory, and network usage
- Use `--rm` for temporary containers to prevent clutter

Practice Questions

1. What is the difference between `docker stop` and `docker kill`? When would you use each?
2. You need to look at a configuration file inside a running container. Which command would you use: `docker exec`, `docker attach`, or `docker cp`? Why?
3. A container shows "Up 2 hours" in `docker ps` but the application inside is not responding. How would you investigate the problem? List the commands you would use.
4. What is the difference between `docker pause` and `docker stop`? What happens to memory in each case?
5. You press Ctrl+C while running `docker attach my-web`. What happens? How can you detach without stopping the container?

Exercises

Exercise 1: Lifecycle Walk-Through

Walk through the complete container lifecycle:

1. Create a container with `docker create` (do not start it)
2. Start it with `docker start`

3. Pause it and verify it is paused with `docker ps`
4. Unpause it
5. Stop it
6. Start it again (restart from stopped state)
7. Remove it

Exercise 2: Debug a Misbehaving Container

1. Run a container: `docker run -d --name debug-me nginx:alpine`
2. Use `docker exec` to get a shell inside the container
3. From inside the container, modify the nginx welcome page
4. From outside the container, use `docker cp` to copy the modified file to your computer
5. Check the container's resource usage with `docker stats`
6. View the container's processes with `docker top`

Exercise 3: Log Analysis

1. Run a container with a Node.js or Python application
2. Make several requests to the application with curl
3. Use `docker logs --tail 5` to see the last 5 log entries
4. Use `docker logs --since 1m` to see logs from the last minute
5. Use `docker logs -f` to follow logs in real time while making more requests

---

What Is Next?

Now that you understand the container lifecycle and know how to manage, debug, and monitor containers, the next chapter covers environment variables and configuration. You will learn how to pass configuration to your containers without hardcoding values in your Dockerfile -- a critical skill for running the same image in development, staging, and production environments.

Chapter 10: Environment Variables and Configuration

What You Will Learn

By the end of this chapter, you will be able to:

- Explain why configuration should live outside your code
- Pass environment variables to containers using the `-e` flag
- Pass multiple environment variables at once
- Use `.env` files to organize environment variables
- Load environment variables from a file with `--env-file`
- Set default environment variables in a Dockerfile with `ENV`
- Use build arguments with `ARG` and `--build-arg`
- Understand the difference between runtime and build-time configuration
- Configure a database connection string using environment variables

Why This Chapter Matters

Imagine you build a web application that connects to a database. In development, the database runs on your laptop at `localhost:5432`. In production, it runs on a cloud server at `db.mycompany.com:5432`. How do you handle this difference?

The wrong answer is to hardcode the database address in your code and change it every time you deploy. That is error-prone, insecure, and makes your application impossible to share.

The right answer is environment variables. You store the database address in an environment variable and read it at runtime. The same Docker image works everywhere -- you just change the environment variable.

Real-life analogy: Think of environment variables as the settings on your phone. The same phone (your Docker image) can work on different cell networks (environments) just by changing the settings (environment variables). You do not need a different phone for each network.

This chapter teaches you how to use environment variables -- one of the most fundamental skills for building professional, production-ready containers.

Why Configuration Belongs Outside Code

The Twelve-Factor App

The Twelve-Factor App is a set of principles for building modern applications. Factor number three states:

> Store configuration in the environment.

Configuration means anything that changes between environments:

- Database connection strings
- API keys and secrets
- Port numbers
- Feature flags (on/off switches for features)
- Log levels (how much detail to log)
- External service URLs

What Happens When You Hardcode Configuration

```javascript
// BAD: Configuration hardcoded in the source code
const db = require('pg');
const connection = db.connect({
  host: 'localhost',       // Only works on your laptop!
  port: 5432,
  database: 'myapp',
  user: 'admin',
  password: 'secret123'   // Password visible in source code!
});
```

Problems with this approach:

1. It only works in one environment. Change from development to production and the app breaks.
2. Secrets are exposed. The password is visible in the source code, in version control, and in the Docker image.
3. You need different code for each environment. Development, staging, production -- three different versions of the same file.

The Right Way: Environment Variables

```javascript
// GOOD: Configuration read from environment variables
const db = require('pg');
const connection = db.connect({
  host: process.env.DB_HOST,       // Read from environment
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD
});
```

Now the same code works everywhere. You just change the environment variables:

```
+--------------------------------------------------+
|    Same Code, Different Environments             |
+--------------------------------------------------+
|                                                  |
|    Development:                                  |
|    DB_HOST=localhost                              |
|    DB_PORT=5432                                   |
|    DB_PASSWORD=devpass123                         |
|                                                  |
|    Staging:                                      |
|    DB_HOST=staging-db.internal                   |
|    DB_PORT=5432                                   |
|    DB_PASSWORD=stagingP@ss                       |
|                                                  |
|    Production:                                   |
|    DB_HOST=prod-db.mycompany.com                 |
|    DB_PORT=5432                                   |
|    DB_PASSWORD=sup3rS3cur3P@ss!                  |
|                                                  |
|    Same Docker image, same code,                 |
|    different configuration.                      |
|                                                  |
+--------------------------------------------------+
```

The -e Flag: Setting Environment Variables

The simplest way to pass an environment variable to a container is the `-e` (or `--env`) flag.

Syntax

```bash
$ docker run -e VARIABLE_NAME=value image_name
```

Example

```bash
# Set a single environment variable
$ docker run -e MY_NAME=Alice alpine echo "Hello, my name is $MY_NAME"
Hello, my name is Alice
```

Let us use a more practical example. Create a simple Node.js app that reads environment variables:

```javascript
// server.js
const express = require('express');
const app = express();

const PORT = process.env.PORT || 3000;
const APP_ENV = process.env.APP_ENV || 'development';
const DB_HOST = process.env.DB_HOST || 'localhost';

app.get('/', (req, res) => {
  res.json({
    environment: APP_ENV,
    port: PORT,
    database_host: DB_HOST
  });
});

app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running in ${APP_ENV} mode on port ${PORT}`);
});
```

Now run it with different configurations:

```bash
# Development configuration
$ docker run -d -p 3000:3000 \
  -e APP_ENV=development \
```

```
  -e DB_HOST=localhost \
  --name dev-app my-app

$ curl http://localhost:3000
{
  "environment": "development",
  "port": "3000",
  "database_host": "localhost"
}

$ docker stop dev-app && docker rm dev-app

# Production configuration (same image, different env vars)
$ docker run -d -p 3000:3000 \
  -e APP_ENV=production \
  -e DB_HOST=prod-db.mycompany.com \
  --name prod-app my-app

$ curl http://localhost:3000
{
  "environment": "production",
  "port": "3000",
  "database_host": "prod-db.mycompany.com"
}
```

Same image, different behavior -- all controlled by environment variables.

Passing a Variable from Your Host

If the variable already exists on your computer, you can pass it without a value:

```bash
# Set a variable on your computer
$ export MY_SECRET=abc123

# Pass it to the container (Docker reads it from your environment)
$ docker run -e MY_SECRET alpine echo $MY_SECRET
abc123
```

When you write `-e MY_SECRET` without `=value`, Docker passes the value from your host's environment.

Multiple Environment Variables

You can set multiple variables by using multiple `-e` flags:

```bash
$ docker run -d -p 3000:3000 \
  -e APP_ENV=production \
  -e DB_HOST=db.example.com \
  -e DB_PORT=5432 \
  -e DB_NAME=myapp \
  -e DB_USER=admin \
  -e DB_PASSWORD=s3cret \
  -e LOG_LEVEL=info \
  -e CACHE_TTL=3600 \
  --name my-app my-app
```

This works, but as you can see, it gets long and hard to manage. This is where `.env` files come in.

.env Files: Organizing Environment Variables

A `.env` file is a simple text file where each line contains one environment variable in the format `KEY=VALUE`.

Real-life analogy: A `.env` file is like a settings sheet. Instead of telling someone each setting one by one, you hand them a sheet of paper with all the settings listed.

Creating a .env File

```bash
# .env
APP_ENV=production
PORT=3000

# Database configuration
DB_HOST=db.example.com
DB_PORT=5432
DB_NAME=myapp
DB_USER=admin
DB_PASSWORD=s3cret

# Application settings
LOG_LEVEL=info
CACHE_TTL=3600
MAX_CONNECTIONS=100
```

Rules for .env Files

```
+-----------------------------------------------------+
|    .env File Syntax Rules                           |
+-----------------------------------------------------+
|                                                     |
|    KEY=VALUE          Basic format                  |
|    KEY=               Empty value (valid)           |
|    # This is a        Comments start with #         |
|    comment                                          |
|    KEY="hello world"  Quotes for spaces             |
|    KEY='hello world'  Single quotes work too        |
|                                                     |
|    NOT supported:                                   |
|    export KEY=VALUE   (no export keyword)           |
|    KEY = VALUE        (no spaces around =)          |
|                                                     |
+-----------------------------------------------------+
```

The --env-file Flag: Loading From a File

Instead of listing each variable with `-e`, you can load all variables from a file:

Syntax

```bash
$ docker run --env-file path/to/.env image_name
```

Example

Create different `.env` files for different environments:

```bash
# .env.development
APP_ENV=development
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_NAME=myapp_dev
DB_USER=dev_user
DB_PASSWORD=dev_password
LOG_LEVEL=debug
```

```bash
# .env.production
APP_ENV=production
PORT=3000
DB_HOST=prod-db.mycompany.com
DB_PORT=5432
DB_NAME=myapp_prod
DB_USER=prod_user
DB_PASSWORD=sup3rS3cur3P@ss
LOG_LEVEL=warn
```

Now run with different environments:

```bash
# Run with development settings
$ docker run -d -p 3000:3000 --env-file .env.development --name dev my-app

# Run with production settings
$ docker run -d -p 3001:3000 --env-file .env.production --name prod my-app
```

Using Multiple .env Files

You can combine `--env-file` with `-e` flags, and use multiple `--env-file` flags:

```bash
# Load base config, then override specific values
$ docker run -d -p 3000:3000 \
  --env-file .env.base \
  --env-file .env.production \
  -e LOG_LEVEL=debug \
  --name my-app my-app
```

When the same variable appears multiple times, the last one wins:

```
+-----------------------------------------------------+
|    Variable Precedence (Last Wins)                  |
+-----------------------------------------------------+
|                                                     |
|    .env.base:        LOG_LEVEL=info                 |
|    .env.production:  LOG_LEVEL=warn     (wins       |
|    -e flag:          LOG_LEVEL=debug     over       |
|                                          these)     |
|                                                     |
|    Result: LOG_LEVEL=debug                          |
|    (the -e flag was last, so it wins)               |
|                                                     |
+-----------------------------------------------------+
```

Verifying Environment Variables Inside a Container

```bash
# Check all environment variables
$ docker exec my-app env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
APP_ENV=production
DB_HOST=prod-db.mycompany.com
DB_PORT=5432
LOG_LEVEL=debug
...

# Check a specific variable
$ docker exec my-app printenv DB_HOST
prod-db.mycompany.com
```

ENV in Dockerfile: Setting Default Values

The `ENV` instruction in a Dockerfile sets default values for environment variables. These defaults can be overridden at runtime with `-e`.

Real-life analogy: `ENV` in a Dockerfile is like the factory settings on your phone. They give you reasonable defaults, but you can change them anytime.

Syntax

```dockerfile
# Single variable
ENV KEY=value

# Multiple variables (old syntax, still works)
ENV KEY1=value1 \
    KEY2=value2 \
    KEY3=value3
```

Example Dockerfile

```dockerfile
FROM node:18-alpine

WORKDIR /app
```

```
# Set default environment variables
# These can be overridden with docker run -e
ENV NODE_ENV=production \
    PORT=3000 \
    LOG_LEVEL=info

COPY package.json package-lock.json ./
RUN npm ci --only=production

COPY . .

USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

How ENV Defaults Work

```
+--------------------------------------------------+
|    ENV Default Behavior                          |
+--------------------------------------------------+
|                                                  |
|    Dockerfile:    ENV PORT=3000                   |
|                                                  |
|    docker run my-app                             |
|    => PORT=3000 (uses default from Dockerfile)   |
|                                                  |
|    docker run -e PORT=8080 my-app                |
|    => PORT=8080 (override wins)                  |
|                                                  |
|    Priority:                                     |
|    1. -e flag (highest)                          |
|    2. --env-file                                 |
|    3. ENV in Dockerfile (lowest, default)        |
|                                                  |
+--------------------------------------------------+
```

ENV Variables Are Visible in the Image

Important security note: Variables set with `ENV` are baked into the image. Anyone who has access to the image can see them:

```bash
$ docker inspect my-app | grep -A 5 "Env"
"Env": [
    "NODE_ENV=production",
    "PORT=3000",
    "LOG_LEVEL=info",
```

```
    "PATH=/usr/local/sbin:..."
]
```

Never put secrets (passwords, API keys, tokens) in ENV instructions. Use runtime `-e` flags or `--env-file` instead.

```dockerfile
# BAD: Secret is visible in the image!
ENV DB_PASSWORD=my_secret_password

# GOOD: Set a placeholder. Override at runtime.
ENV DB_PASSWORD=changeme

# Or better yet, do not set it at all.
# Let the application fail clearly if the variable is missing.
```

ARG and --build-arg: Build-Time Variables

`ARG` defines variables that are only available during the build process. They are not available when the container runs.

Real-life analogy: `ARG` is like instructions you give to a contractor while building your house. "Use oak for the floors" and "Paint the walls blue" are building instructions. Once the house is built, the contractor's instructions are gone. The homeowner (the running container) does not see them.

Syntax

```dockerfile
# In the Dockerfile
ARG VARIABLE_NAME
ARG VARIABLE_NAME=default_value
```

```bash
# When building
$ docker build --build-arg VARIABLE_NAME=value -t image_name .
```

Example: Choosing a Node.js Version at Build Time

```dockerfile
# ARG defines a build-time variable
ARG NODE_VERSION=18

# Use the ARG value in FROM
FROM node:${NODE_VERSION}-alpine

WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --only=production
COPY . .

EXPOSE 3000
CMD ["node", "server.js"]
```

Now you can build with different Node.js versions without changing the Dockerfile:

```bash
# Build with Node.js 18 (the default)
$ docker build -t my-app:node18 .

# Build with Node.js 20
$ docker build --build-arg NODE_VERSION=20 -t my-app:node20 .

# Build with Node.js 16
$ docker build --build-arg NODE_VERSION=16 -t my-app:node16 .
```

ARG vs ENV: The Key Difference

```
+-----------------------------------------------------+
|    ARG vs ENV Comparison                            |
+-----------------------------------------------------+
|                                                     |
|    Feature          ARG            ENV             |
|    ------------------------------------------       |
|    Available when   Build only     Build + Run     |
|    Set by           --build-arg    -e / --env      |
|    In Dockerfile    ARG NAME=val   ENV NAME=val    |
|    Visible in       Build output   Image + Run     |
|    running container   No          Yes             |
|    Persists in image   No          Yes             |
|    Use for          Version nums,  App config,     |
|                     build opts     ports, modes    |
|                                                     |
+-----------------------------------------------------+
```

Visualizing Build-Time vs Runtime

```
+-----------------------------------------------------+
|    Build Time vs Runtime                            |
+-----------------------------------------------------+
|                                                     |
|    BUILD TIME (docker build)                        |
|    +-------------------------------------------+    |
|    |  ARG variables available here              |    |
|    |  ENV variables set here                    |    |
|    |  RUN commands execute here                 |    |
|    +-------------------------------------------+    |
|                                                     |
|         Image is created                            |
|              |                                      |
|              v                                      |
|                                                     |
|    RUNTIME (docker run)                             |
|    +-------------------------------------------+    |
|    |  ARG variables are GONE                    |    |
|    |  ENV variables available here              |    |
|    |  -e variables available here               |    |
|    |  CMD/ENTRYPOINT executes here              |    |
|    +-------------------------------------------+    |
|                                                     |
+-----------------------------------------------------+
```

Using ARG and ENV Together

A common pattern is to use ARG to set a build-time default that becomes an ENV variable:

```dockerfile
# Accept a version at build time
ARG APP_VERSION=1.0.0

# Make it available at runtime too
ENV APP_VERSION=${APP_VERSION}

# Now APP_VERSION is available both during build and at runtime
```

```bash
# Build with a custom version
$ docker build --build-arg APP_VERSION=2.1.0 -t my-app:2.1.0 .

# The version is available inside the running container
$ docker run my-app:2.1.0 printenv APP_VERSION
2.1.0
```

ARG Scope

ARG has a specific scope. An ARG defined before FROM is only available in the FROM instruction:

```dockerfile
# This ARG is only available for the FROM line
ARG NODE_VERSION=18
FROM node:${NODE_VERSION}-alpine

# This will NOT work -- NODE_VERSION is out of scope!
RUN echo "Building with Node ${NODE_VERSION}"

# You need to re-declare it after FROM
ARG NODE_VERSION
RUN echo "Building with Node ${NODE_VERSION}"
```

Runtime vs Build-Time Configuration

Understanding when to use each type of configuration is crucial:

Build-Time Configuration (ARG)

Use ARG for things that affect how the image is built:

```dockerfile
# Which version of the base image to use
ARG NODE_VERSION=18
FROM node:${NODE_VERSION}-alpine

# Whether to install development dependencies
ARG INSTALL_DEV_DEPS=false
RUN if [ "$INSTALL_DEV_DEPS" = "true" ]; then \
      npm ci; \
    else \
      npm ci --only=production; \
    fi

# Which registry to pull packages from
ARG NPM_REGISTRY=https://registry.npmjs.org
RUN npm config set registry ${NPM_REGISTRY} && npm ci
```

Runtime Configuration (ENV / -e)

Use ENV or -e for things that affect how the application runs:

```dockerfile
ENV NODE_ENV=production \
    PORT=3000 \
    LOG_LEVEL=info
```

```bash
$ docker run -e DB_HOST=prod-db.example.com \
             -e DB_PASSWORD=s3cret \
             -e REDIS_URL=redis://cache:6379 \
             my-app
```

Decision Guide

```
+-----------------------------------------------------+
|    When to Use ARG vs ENV vs -e                     |
+-----------------------------------------------------+
|                                                     |
|    Question: Does the value affect how the          |
|    image is built?                                  |
|                                                     |
|    YES -> Use ARG                                   |
|           Examples:                                 |
|           - Base image version                      |
|           - Build-time flags                        |
|           - Package registry URLs                   |
|                                                     |
|    NO -> Does the value contain a secret?           |
|                                                     |
|      YES -> Use -e or --env-file at runtime         |
|             NEVER put secrets in ENV or ARG         |
|             Examples:                               |
|             - Database passwords                    |
|             - API keys                              |
|             - Auth tokens                           |
|                                                     |
|      NO -> Does it have a sensible default?         |
|                                                     |
|        YES -> Use ENV in Dockerfile                 |
|               Override with -e when needed          |
|               Examples:                             |
|               - PORT=3000                           |
|               - LOG_LEVEL=info                      |
|               - NODE_ENV=production                 |
|                                                     |
|        NO -> Use -e at runtime                      |
|              Examples:                              |
|              - DB_HOST (different per env)          |
|              - API_URL (different per env)          |
|                                                     |
+-----------------------------------------------------+
```

Practical Example: Database Connection String

Let us put it all together with a realistic example. We will build an application that connects to a PostgreSQL database using environment variables.

The Application Code

```javascript
// server.js
const express = require('express');
const app = express();

// Read ALL configuration from environment variables
const config = {
  port: process.env.PORT || 3000,
  env: process.env.NODE_ENV || 'development',
  db: {
    host: process.env.DB_HOST || 'localhost',
    port: parseInt(process.env.DB_PORT || '5432'),
    name: process.env.DB_NAME || 'myapp',
    user: process.env.DB_USER || 'postgres',
    password: process.env.DB_PASSWORD || 'postgres',
  }
};

// Construct the connection string from individual variables
const connectionString =
  `postgresql://${config.db.user}:${config.db.password}` +
  `@${config.db.host}:${config.db.port}/${config.db.name}`;

app.get('/', (req, res) => {
  res.json({
    environment: config.env,
    database: {
      host: config.db.host,
      port: config.db.port,
      name: config.db.name,
      // Never expose the password in API responses!
      password: '********'
    }
  });
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(config.port, '0.0.0.0', () => {
```

```
  console.log(`Server running on port ${config.port}`);
  console.log(`Environment: ${config.env}`);
  console.log(`Database: ${config.db.host}:${config.db.port}/${config.db.name}`);
  // Never log the password!
});
```

The Dockerfile

```dockerfile
FROM node:18-alpine

WORKDIR /app

# Set defaults for non-sensitive configuration
# These can be overridden at runtime with -e
ENV NODE_ENV=production \
    PORT=3000 \
    DB_PORT=5432

# Do NOT set defaults for sensitive values like DB_PASSWORD
# The app should fail clearly if they are not provided

COPY package.json package-lock.json ./
RUN npm ci --only=production

COPY . .

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:3000/health || exit 1

USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Environment Files

```bash
# .env.development
NODE_ENV=development
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_NAME=myapp_dev
DB_USER=dev_user
DB_PASSWORD=dev_password_123
```

```bash
# .env.production
NODE_ENV=production
PORT=3000
DB_HOST=prod-db.mycompany.com
DB_PORT=5432
DB_NAME=myapp_production
DB_USER=prod_admin
DB_PASSWORD=Pr0d_Sup3r_S3cur3!
```

Running in Different Environments

```bash
# Build the image once
$ docker build -t my-app:1.0 .

# Run in development
$ docker run -d -p 3000:3000 \
  --env-file .env.development \
  --name app-dev my-app:1.0

$ curl http://localhost:3000
{
  "environment": "development",
  "database": {
    "host": "localhost",
    "port": 5432,
    "name": "myapp_dev",
    "password": "********"
  }
}

$ docker stop app-dev && docker rm app-dev

# Run in production
$ docker run -d -p 3000:3000 \
  --env-file .env.production \
  --name app-prod my-app:1.0

$ curl http://localhost:3000
{
  "environment": "production",
  "database": {
    "host": "prod-db.mycompany.com",
    "port": 5432,
    "name": "myapp_production",
    "password": "********"
  }
}
```

The Complete Configuration Flow

```
+-----------------------------------------------------+
|    Configuration Flow                               |
+-----------------------------------------------------+
|                                                     |
|    1. Dockerfile (ENV) sets defaults                |
|       NODE_ENV=production                           |
|       PORT=3000                                     |
|       DB_PORT=5432                                  |
|                                                     |
|    2. .env file provides environment-specific       |
|       values (--env-file)                           |
|       DB_HOST=prod-db.mycompany.com                 |
|       DB_USER=prod_admin                            |
|       DB_PASSWORD=Pr0d_Sup3r_S3cur3!                |
|                                                     |
|    3. -e flags override individual values           |
|       -e LOG_LEVEL=debug                            |
|                                                     |
|    4. Application reads process.env.*               |
|       and uses the final merged values              |
|                                                     |
|    Priority: -e > --env-file > ENV (Dockerfile)     |
|                                                     |
+-----------------------------------------------------+
```

Security Best Practices for Environment Variables

```
+-----------------------------------------------------+
|    Environment Variable Security                    |
+-----------------------------------------------------+
|                                                     |
|  DO:                                                |
|  - Use --env-file for secrets at runtime            |
|  - Add .env files to .gitignore                     |
|  - Use different credentials per environment        |
|  - Use Docker secrets or vault in production        |
|  - Set non-sensitive defaults in Dockerfile ENV     |
|                                                     |
|  DO NOT:                                            |
|  - Put secrets in Dockerfile ENV instructions       |
|  - Commit .env files to version control             |
|  - Log or print secret values                       |
|  - Return secrets in API responses                  |
|  - Use ARG for secrets (visible in build history)|
|                                                     |
+-----------------------------------------------------+
```

Why ARG Is Not Safe for Secrets

```bash
# BAD: Using ARG for a secret
$ docker build --build-arg DB_PASSWORD=s3cret -t my-app .

# Anyone can see the build arg in the image history!
$ docker history my-app
IMAGE        CREATED    SIZE   COMMENT
a1b2c3d4     1 min      0B     ARG DB_PASSWORD=s3cret  <-- EXPOSED!
```

Build arguments are visible in `docker history`. Never use them for secrets.

Common Mistakes

Mistake 1: Hardcoding Configuration in Source Code

```javascript
// BAD: Hardcoded values
const DB_HOST = 'localhost';
const DB_PASSWORD = 'my_password';

// GOOD: Read from environment
const DB_HOST = process.env.DB_HOST;
const DB_PASSWORD = process.env.DB_PASSWORD;
```

Mistake 2: Putting Secrets in Dockerfile ENV

```dockerfile
# BAD: Secret is baked into the image
ENV DB_PASSWORD=my_super_secret_password

# GOOD: No default for secrets. Set at runtime with -e
# ENV DB_PASSWORD is intentionally omitted
```

Mistake 3: Committing .env Files to Git

```bash
# Always add .env files to .gitignore!
$ echo ".env" >> .gitignore
$ echo ".env.*" >> .gitignore
```

Mistake 4: Not Providing Required Variables

```bash
# Your app expects DB_HOST but you forgot to set it
$ docker run my-app
# App crashes: "Error: DB_HOST environment variable is required"

# Solution: Always validate required environment variables
```

Here is how to validate in code:

```javascript
// Validate required environment variables at startup
const required = ['DB_HOST', 'DB_USER', 'DB_PASSWORD', 'DB_NAME'];

for (const varName of required) {
  if (!process.env[varName]) {
    console.error(`ERROR: Required environment variable ${varName} is not set`);
    process.exit(1);
  }
}
```

Mistake 5: Using the Wrong Type of Variable

```dockerfile
# BAD: Using ARG for runtime config
ARG PORT=3000
# PORT is NOT available when the container runs!

# GOOD: Using ENV for runtime config
ENV PORT=3000
# PORT IS available when the container runs
```

Best Practices

1. Read all configuration from environment variables. Never hardcode values that change between environments.
2. Set sensible defaults with ENV for non-sensitive configuration like PORT and LOG_LEVEL.
3. Never put secrets in ENV or ARG. Pass them at runtime with `-e` or `--env-file`.
4. Use separate .env files for each environment (development, staging, production).
5. Add .env files to .gitignore to prevent accidentally committing secrets.

6. Validate required variables at application startup and fail fast with clear error messages.
7. Never log or expose secrets in responses, error messages, or console output.
8. Use ARG for build-time decisions only, like choosing a base image version.

Quick Summary

Environment variables are the standard way to configure Docker containers. The `ENV` instruction in a Dockerfile sets defaults that can be overridden at runtime. The `-e` flag sets individual variables, while `--env-file` loads them from a file. Build-time variables use `ARG` and `--build-arg` and are only available during the build process. Following the Twelve-Factor App principle, configuration should always live outside your code.

Key Points

- Configuration should live outside your code (Twelve-Factor App, Factor 3)
- `-e KEY=VALUE` sets a single environment variable at runtime
- `--env-file .env` loads multiple variables from a file
- `ENV` in a Dockerfile sets defaults that can be overridden at runtime
- `ARG` in a Dockerfile defines build-time variables (not available at runtime)
- Priority order: `-e` flag > `--env-file` > `ENV` in Dockerfile
- Never put secrets in `ENV` or `ARG` instructions
- Never commit `.env` files to version control
- Always validate required environment variables at application startup
- Use separate `.env` files for each environment

Practice Questions

1. What is the Twelve-Factor App principle about configuration? Why should configuration live outside your code?
2. What is the difference between `ARG` and `ENV` in a Dockerfile? Give an example of when you would use each.

3. You have a Dockerfile with `ENV PORT=3000` and you run the container with `docker run -e PORT=8080 my-app`. What port does the application see? Why?
4. Why should you never put passwords or API keys in a Dockerfile's `ENV` instruction? What should you do instead?
5. You run `docker build --build-arg SECRET_KEY=abc123 -t my-app .`. Is this safe? Why or why not?

Exercises

Exercise 1: Environment Variable Basics

1. Create a simple application that reads three environment variables: `APP_NAME`, `APP_VERSION`, and `GREETING`
2. Write a Dockerfile with `ENV` defaults for `APP_NAME` and `APP_VERSION`
3. Build the image and run it three ways:
    - With no overrides (use defaults)
    - With `-e` overriding `GREETING`
    - With `--env-file` providing all three values

Exercise 2: Multi-Environment Setup

1. Create an application that connects to a database (you can fake the connection)
2. Create three `.env` files: `.env.development`, `.env.staging`, `.env.production`
3. Each file should have different values for `DB_HOST`, `DB_NAME`, and `DB_USER`
4. Run the same image with each `.env` file and verify the different configurations

Exercise 3: Build Arguments

1. Create a Dockerfile that uses `ARG` to specify the Node.js version
2. Build the image three times with different Node.js versions (16, 18, 20)
3. Verify each image uses the correct version by running `docker run image node --version`
4. Try to access the ARG variable at runtime. What happens?

What Is Next?

Now you know how to configure your containers without hardcoding values. But there is another important problem to solve: data persistence. When a container is removed, all its data is lost. In the next chapter, you will learn about Docker volumes -- the solution for persistent data. You will learn how to keep your database data, uploaded files, and logs safe even when containers come and go.

Chapter 11: Volumes and Persistent Data

What You Will Learn

By the end of this chapter, you will be able to:

- Explain why containers lose data when they are removed
- Describe the three types of Docker storage: volumes, bind mounts, and tmpfs mounts
- Create and manage named volumes with `docker volume create`
- Use bind mounts to share files between your computer and a container
- Inspect volumes to see where Docker stores data
- Persist database data using a PostgreSQL container with a volume
- Back up and restore volume data
- Use read-only volumes for security

Why This Chapter Matters

Here is a problem that surprises every Docker beginner: you start a PostgreSQL database in a container, create tables, insert data, and everything works perfectly. Then you stop and remove the container. You start a new one, and all your data is gone. Every table. Every row. Vanished.

This happens because containers are ephemeral by design -- they are meant to be temporary. When a container is removed, everything inside it is deleted, including any files the application created.

Real-life analogy: A container is like a whiteboard. You can write anything on it, but when you erase the whiteboard (remove the container), everything is gone. Volumes are like a notebook -- even if you throw away the whiteboard, the notebook keeps your notes safe.

This chapter teaches you how to use volumes to persist data beyond the lifetime of a container. This is essential for databases, uploaded files, application logs, and any data you cannot afford to lose.

Why Containers Lose Data

Every container has a writable layer that sits on top of the read-only image layers. When the application writes files, they go into this writable layer.

```
+-----------------------------------------------------+
|    Container Filesystem                             |
+-----------------------------------------------------+
|                                                     |
|    +----------------------------------------+       |
|    | Writable Layer (container-specific)    |       |
|    | - Application logs                     |       |
|    | - Database files                       |       |
|    | - Uploaded files                       |       |
|    | - Temp files                           |       |
|    +----------------------------------------+       |
|    | Read-only Image Layer 4 (CMD)          |       |
|    +----------------------------------------+       |
|    | Read-only Image Layer 3 (COPY code)    |       |
|    +----------------------------------------+       |
|    | Read-only Image Layer 2 (npm install)  |       |
|    +----------------------------------------+       |
|    | Read-only Image Layer 1 (FROM node)    |       |
|    +----------------------------------------+       |
|                                                     |
|    When the container is removed,                   |
|    the writable layer is DELETED.                   |
|    The image layers remain untouched.               |
|                                                     |
+-----------------------------------------------------+
```

Demonstration: Data Loss

```bash
# Start a container and create a file
$ docker run -it --name test alpine /bin/sh
/ # echo "Important data!" > /my-data.txt
/ # cat /my-data.txt
Important data!
/ # exit
```

```
# The container is stopped but still exists
$ docker ps -a | grep test
CONTAINER ID   IMAGE    STATUS                     NAMES
a1b2c3d4       alpine   Exited (0) 5 seconds ago  test

# The data is still there in the stopped container
$ docker start test
$ docker exec test cat /my-data.txt
Important data!
$ docker stop test

# Now REMOVE the container
$ docker rm test
test

# Start a new container from the same image
$ docker run -it --name test alpine /bin/sh
/ # cat /my-data.txt
cat: can't open '/my-data.txt': No such file or directory
/ # exit

# The data is gone!
```

This is by design. Containers are meant to be disposable. But obviously, we need a way to keep important data. That is where volumes come in.

Three Types of Docker Storage

Docker provides three ways to persist data:

```
+-----------------------------------------------------+
|    Docker Storage Types                             |
+-----------------------------------------------------+
|                                                     |
|    1. VOLUMES                                       |
|       Managed by Docker in a special area           |
|       Best for production data                      |
|       /var/lib/docker/volumes/                      |
|                                                     |
|    2. BIND MOUNTS                                   |
|       Map a host directory to a container path      |
|       Best for development (live code reload)       |
|       Any path on the host machine                  |
|                                                     |
|    3. TMPFS MOUNTS                                  |
|       Stored in memory only (RAM)                   |
```

```
|       Never written to disk                      |
|       Best for sensitive temporary data          |
|                                                  |
+--------------------------------------------------+
```

Visual Comparison

```
+--------------------------------------------------+
|                     Host Machine                 |
|                                                  |
|  Filesystem:                Memory:              |
|  /var/lib/docker/volumes/   +----------+         |
|  +----------------+         | tmpfs    |         |
|  | Docker Volume  |         | mount    |         |
|  +-------+--------+         +----+-----+         |
|          |                       |               |
|  /home/user/project/             |               |
|  +----------------+              |               |
|  | Bind Mount     |              |               |
|  +-------+--------+              |               |
|          |                       |               |
|          v                       v               |
|  +-----------------------------------------+   |
|  |            Container                     |   |
|  |                                          |   |
|  |  /data     /code        /tmp/secrets     |   |
|  |  (volume)  (bind mount) (tmpfs)          |   |
|  +-----------------------------------------+   |
|                                                  |
+--------------------------------------------------+
```

Named Volumes: Docker-Managed Storage

Named volumes are the recommended way to persist data. Docker creates and manages them in a special directory on your host machine.

Real-life analogy: A named volume is like a storage locker. You rent a locker (create a volume), put things in it (write data), and the locker keeps your stuff safe even if you move to a different apartment (remove the container). You can even give a new apartment (new container) the key to the same locker.

Creating a Volume

```bash
$ docker volume create my-data
my-data
```

That is it. Docker creates a volume named `my-data` and manages it for you.

Listing Volumes

```bash
$ docker volume ls
DRIVER    VOLUME NAME
local     my-data
```

Using a Volume with a Container

You attach a volume to a container using the `-v` (or `--mount`) flag:

```bash
# Syntax: -v volume_name:container_path
$ docker run -d \
  -v my-data:/app/data \
  --name app1 \
  alpine sleep 3600
```

This mounts the `my-data` volume to `/app/data` inside the container. Any files written to `/app/data` are actually stored in the volume.

Demonstration: Data Persists

```bash
# Create a volume
$ docker volume create demo-data
demo-data

# Start container 1 and write data
$ docker run -it --name writer -v demo-data:/data alpine /bin/sh
/ # echo "Hello from container 1!" > /data/message.txt
/ # echo "This data will survive!" >> /data/message.txt
/ # cat /data/message.txt
Hello from container 1!
This data will survive!
/ # exit

# Remove container 1
$ docker rm writer
writer
```

```
# Start container 2 with the SAME volume
$ docker run -it --name reader -v demo-data:/data alpine /bin/sh
/ # cat /data/message.txt
Hello from container 1!
This data will survive!
/ # exit

# The data survived! The volume persists even after container removal.
```

```
+---------------------------------------------------+
|    Volume Persistence                             |
+---------------------------------------------------+
|                                                   |
|    Container 1 (writer):                          |
|    Writes data to /data/message.txt               |
|    Container is removed ---> data remains!        |
|                                                   |
|    demo-data volume: [message.txt survives]       |
|                                                   |
|    Container 2 (reader):                          |
|    Mounts same volume to /data                    |
|    Reads /data/message.txt ---> data is there!    |
|                                                   |
+---------------------------------------------------+
```

Auto-Creating Volumes

If you use `-v` with a volume name that does not exist, Docker creates it automatically:

```bash
# Docker creates "auto-volume" automatically
$ docker run -v auto-volume:/data alpine echo "Volume created!"

$ docker volume ls
DRIVER    VOLUME NAME
local     auto-volume
local     demo-data
```

Volume Inspection

You can inspect a volume to see its details:

```bash
$ docker volume inspect my-data
[
    {
        "CreatedAt": "2024-01-15T10:30:00Z",
        "Driver": "local",
        "Labels": {},
        "Mountpoint": "/var/lib/docker/volumes/my-data/_data",
        "Name": "my-data",
        "Options": {},
        "Scope": "local"
    }
]
```

Understanding the Output

```
+-----------------------------------------------------+
|    Volume Inspect Fields                            |
+-----------------------------------------------------+
|                                                     |
|    Name        The volume name                      |
|    Driver      Storage driver (usually "local")     |
|    Mountpoint  Where Docker stores the data on      |
|                your host machine                    |
|    Scope       "local" means this machine only      |
|    CreatedAt   When the volume was created          |
|                                                     |
+-----------------------------------------------------+
```

The Mountpoint shows you where the volume data physically lives on your host machine. On Linux, it is typically under `/var/lib/docker/volumes/`. On Docker Desktop (Mac/Windows), this path is inside the Docker VM.

Bind Mounts: Sharing Host Directories

Bind mounts map a specific directory on your computer to a directory inside the container. Unlike volumes, you control exactly where the data is stored.

Real-life analogy: A bind mount is like putting a window between two rooms. Whatever is in the room on one side of the window is visible from the other side. If you add something from either side, both sides see it.

Syntax

```bash
# Using -v flag (short form)
$ docker run -v /host/path:/container/path image

# Using --mount flag (long form, more explicit)
$ docker run --mount type=bind,source=/host/path,target=/container/path image
```

Example: Development with Live Code Reload

This is the most common use of bind mounts -- mounting your source code into a container so changes you make on your computer are instantly visible inside the container:

```bash
# Your project is at /home/user/my-project
$ ls /home/user/my-project
Dockerfile  package.json  server.js  src/

# Mount it into the container
$ docker run -d \
  -p 3000:3000 \
  -v /home/user/my-project:/app \
  --name dev-server \
  node:18-alpine \
  sh -c "cd /app && npm install && npx nodemon server.js"
```

Now when you edit `server.js` on your computer, the changes are instantly visible inside the container. If you are using a tool like nodemon, it will detect the change and restart the server automatically.

```
+-----------------------------------------------------+
|    Bind Mount for Development                       |
+-----------------------------------------------------+
|                                                     |
|    Host Machine           Container                 |
|    /home/user/my-project  /app                      |
|    +------------------+   +------------------+     |
|    | server.js        |<->| server.js        |     |
|    | package.json     |<->| package.json     |     |
|    | src/             |<->| src/             |     |
|    +------------------+   +------------------+     |
|                                                     |
|    Edit server.js on     Container sees the         |
|    your computer  -----> change immediately         |
|                                                     |
+-----------------------------------------------------+
```

Using $(pwd) for the Current Directory

Instead of typing the full path, you can use `$(pwd)` to refer to the current directory:

```bash
# Mount the current directory
$ cd /home/user/my-project
$ docker run -d \
  -p 3000:3000 \
  -v $(pwd):/app \
  --name dev-server \
  node:18-alpine \
  sh -c "cd /app && npm install && npx nodemon server.js"
```

`$(pwd)` is a shell command that prints the current working directory. It gets replaced with the full path before Docker sees it.

Bind Mount vs Volume

```
+---------------------------------------------------+
|    Bind Mounts vs Volumes                         |
+---------------------------------------------------+
|                                                   |
|    Feature          Volume        Bind Mount      |
|    ----------------------------------------       |
|    Managed by       Docker        You             |
|    Location         Docker area   Anywhere        |
|    Create           docker volume Just use -v     |
|    Portable         Yes           No (host path)  |
|    Best for         Production    Development     |
|    Pre-populate     Yes           No              |
|    Backup           docker cmds   cp/tar/rsync    |
|    Performance      Optimized     Direct access   |
|                                                   |
|    Use VOLUMES for production data                |
|    Use BIND MOUNTS for development                |
|                                                   |
+---------------------------------------------------+
```

An Important Difference

When you mount a volume to a container path that already has data, the existing data in the image is copied to the volume (if the volume is empty). This is called pre-population.

With bind mounts, the host directory replaces whatever was in the container at that path. If the host directory is empty, the container path will be empty too, even if the image had files there.

```
+---------------------------------------------------+
|    Pre-Population Behavior                        |
```

```
+-----------------------------------------------------+
|                                                     |
|    Image has /app/config with default files         |
|                                                     |
|    Volume (empty):                                  |
|    -v my-vol:/app/config                            |
|    Result: Volume gets the default files             |
|    (pre-populated from the image)                   |
|                                                     |
|    Bind Mount (empty directory):                    |
|    -v /host/empty:/app/config                       |
|    Result: Container sees empty directory!           |
|    (host directory overrides image content)          |
|                                                     |
+-----------------------------------------------------+
```

---

tmpfs Mounts: Memory-Only Storage

tmpfs mounts store data in the host's memory (RAM). The data is never written to disk and disappears when the container stops.

Real-life analogy: A tmpfs mount is like a whiteboard in your office. You can write on it while you are there, but when you leave (the container stops), everything is erased. Nothing was ever saved permanently.

Syntax

```bash
$ docker run --tmpfs /path/in/container image
# or
$ docker run --mount type=tmpfs,destination=/path/in/container image
```

When to Use tmpfs

- Sensitive data that should never be written to disk (like encryption keys)
- Temporary files that do not need to persist
- Performance for high-speed temporary storage

Example

```bash
$ docker run -d \
  --tmpfs /app/temp \
  --name secure-app \
  my-app

# Data in /app/temp is stored in RAM only
# It disappears when the container stops
```

Database Persistence with PostgreSQL

The most important use of volumes is persisting database data. Let us walk through a complete example with PostgreSQL.

The Problem: Data Loss Without Volumes

```bash
# Start PostgreSQL WITHOUT a volume
$ docker run -d \
  --name postgres-no-vol \
  -e POSTGRES_PASSWORD=mypassword \
  -p 5432:5432 \
  postgres:16-alpine

# Wait for it to start
$ sleep 5

# Create a table and insert data
$ docker exec -it postgres-no-vol psql -U postgres -c "
  CREATE TABLE users (id SERIAL PRIMARY KEY, name TEXT);
  INSERT INTO users (name) VALUES ('Alice'), ('Bob');
  SELECT * FROM users;
"
 id | name
----+-------
  1 | Alice
  2 | Bob
(2 rows)

# Remove the container
$ docker stop postgres-no-vol && docker rm postgres-no-vol

# Start a new PostgreSQL container
$ docker run -d \
  --name postgres-no-vol \
  -e POSTGRES_PASSWORD=mypassword \
  -p 5432:5432 \
```

```
  postgres:16-alpine

$ sleep 5

# Try to read the data
$ docker exec -it postgres-no-vol psql -U postgres -c "SELECT * FROM users;"
ERROR:  relation "users" does not exist

# Data is gone!
$ docker stop postgres-no-vol && docker rm postgres-no-vol
```

The Solution: Volumes

```bash
# Create a volume for PostgreSQL data
$ docker volume create postgres-data

# Start PostgreSQL WITH a volume
$ docker run -d \
  --name postgres-vol \
  -e POSTGRES_PASSWORD=mypassword \
  -p 5432:5432 \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:16-alpine

$ sleep 5

# Create a table and insert data
$ docker exec -it postgres-vol psql -U postgres -c "
  CREATE TABLE users (id SERIAL PRIMARY KEY, name TEXT);
  INSERT INTO users (name) VALUES ('Alice'), ('Bob'), ('Charlie');
  SELECT * FROM users;
"
 id |  name
----+---------
  1 | Alice
  2 | Bob
  3 | Charlie
(3 rows)

# Remove the container (the volume stays!)
$ docker stop postgres-vol && docker rm postgres-vol

# Start a NEW container with the SAME volume
$ docker run -d \
  --name postgres-new \
  -e POSTGRES_PASSWORD=mypassword \
  -p 5432:5432 \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:16-alpine
```

```
$ sleep 5

# The data is still there!
$ docker exec -it postgres-new psql -U postgres -c "SELECT * FROM users;"
 id |  name
----+---------
  1 | Alice
  2 | Bob
  3 | Charlie
(3 rows)
```

The data survived because it was stored in the `postgres-data` volume, not in the container's writable layer.

```
+-----------------------------------------------------+
|    PostgreSQL with Volume                           |
+-----------------------------------------------------+
|                                                     |
|    Container 1 (postgres-vol):                      |
|    /var/lib/postgresql/data -> postgres-data vol  |
|    - Creates tables, inserts rows                   |
|    - Container is removed                           |
|                                                     |
|    postgres-data volume:                            |
|    [Tables and rows are safely stored here]         |
|                                                     |
|    Container 2 (postgres-new):                      |
|    /var/lib/postgresql/data -> postgres-data vol  |
|    - Same volume, same data!                        |
|    - Tables and rows are all there                  |
|                                                     |
+-----------------------------------------------------+
```

How Did We Know to Use /var/lib/postgresql/data?

The PostgreSQL Docker image documentation tells you where it stores data. Every database image documents its data directory:

```
+-----------------------------------------------------+
|    Common Database Data Directories                 |
+-----------------------------------------------------+
|                                                     |
|    Database        Data Directory                   |
|    -------------------------------------------------
|    PostgreSQL      /var/lib/postgresql/data       |
|    MySQL           /var/lib/mysql                 |
|    MongoDB         /data/db                         |
|    Redis           /data                            |
```

```
|                                                    |
+----------------------------------------------------+
```

Backup and Restore

Backing up volume data is essential. Here are two common approaches.

Method 1: Copy From a Container

```bash
# Start a temporary container that mounts the volume
# and copies data to a tar archive
$ docker run --rm \
  -v postgres-data:/source \
  -v $(pwd):/backup \
  alpine \
  tar czf /backup/postgres-backup.tar.gz -C /source .

# This creates postgres-backup.tar.gz in your current directory
$ ls -lh postgres-backup.tar.gz
-rw-r--r--  1 user  staff   15M Jan 15 12:00 postgres-backup.tar.gz
```

Let us break down what this command does:

1. `docker run --rm` -- Start a temporary container that auto-removes when done
2. `-v postgres-data:/source` -- Mount the volume we want to back up to `/source`
3. `-v $(pwd):/backup` -- Mount the current directory on our host to `/backup`
4. `alpine` -- Use a tiny Alpine image (we just need the tar command)
5. `tar czf /backup/postgres-backup.tar.gz -C /source .` -- Create a compressed archive of everything in `/source` and save it to `/backup`

Method 2: Restore From Backup

```bash
# Create a new volume
$ docker volume create postgres-restored

# Restore the backup into the new volume
$ docker run --rm \
  -v postgres-restored:/target \
  -v $(pwd):/backup \
```

```
  alpine \
  sh -c "cd /target && tar xzf /backup/postgres-backup.tar.gz"

# Use the restored volume with a new container
$ docker run -d \
  --name restored-db \
  -e POSTGRES_PASSWORD=mypassword \
  -p 5432:5432 \
  -v postgres-restored:/var/lib/postgresql/data \
  postgres:16-alpine
```

Backup Flow Diagram

```
+---------------------------------------------------+
|    Volume Backup and Restore Flow                 |
+---------------------------------------------------+
|                                                   |
|    BACKUP:                                        |
|    postgres-data volume                           |
|         |                                         |
|         v                                         |
|    [temp container reads volume]                  |
|         |                                         |
|         v                                         |
|    tar czf backup.tar.gz                          |
|         |                                         |
|         v                                         |
|    Host: ./backup.tar.gz                          |
|                                                   |
|    RESTORE:                                       |
|    Host: ./backup.tar.gz                          |
|         |                                         |
|         v                                         |
|    [temp container writes to new volume]          |
|         |                                         |
|         v                                         |
|    postgres-restored volume                       |
|         |                                         |
|         v                                         |
|    New container uses restored volume             |
|                                                   |
+---------------------------------------------------+
```

Read-Only Volumes

You can mount a volume as read-only by adding `:ro` to the mount:

```bash
# Mount config as read-only
$ docker run -d \
  -v app-config:/app/config:ro \
  --name my-app \
  my-app
```

The container can read files in `/app/config` but cannot modify them. Any attempt to write will fail:

```bash
$ docker exec my-app touch /app/config/test.txt
touch: /app/config/test.txt: Read-only file system
```

When to Use Read-Only Mounts

```
+-----------------------------------------------------+
|    Read-Only Mount Use Cases                        |
+-----------------------------------------------------+
|                                                     |
|    Configuration files:                             |
|    -v ./nginx.conf:/etc/nginx/nginx.conf:ro         |
|    (App can read config but not modify it)          |
|                                                     |
|    SSL certificates:                                |
|    -v certs:/etc/ssl/certs:ro                       |
|    (App can use certs but not change them)          |
|                                                     |
|    Static assets:                                   |
|    -v static-files:/app/public:ro                   |
|    (Web server serves files but cannot alter)       |
|                                                     |
|    Source code (CI/CD):                             |
|    -v $(pwd):/app:ro                                |
|    (Build tools read code but cannot modify)        |
|                                                     |
+-----------------------------------------------------+
```

Volume Cleanup

Volumes persist even after all containers using them are removed. Over time, unused volumes can accumulate and waste disk space.

Removing a Specific Volume

```bash
# Remove a volume (must not be in use by any container)
$ docker volume rm my-data
my-data

# Try to remove a volume that is in use
$ docker volume rm postgres-data
Error: volume is in use

# You must stop and remove the container first
$ docker stop postgres-new && docker rm postgres-new
$ docker volume rm postgres-data
postgres-data
```

Removing All Unused Volumes

```bash
$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
abc123def456
ghi789jkl012

Total reclaimed space: 256.3MB
```

Warning: `docker volume prune` deletes unused volumes permanently. Make sure you have backed up any data you need before running this command.

Listing All Volumes

```bash
$ docker volume ls
DRIVER    VOLUME NAME
local     postgres-data
local     postgres-restored
local     demo-data
local     auto-volume

# Filter for specific volumes
$ docker volume ls --filter "name=postgres"
DRIVER    VOLUME NAME
local     postgres-data
local     postgres-restored
```

Quick Reference: Volume Commands

```
+--------------------------------------------------+
|    Volume Commands                               |
+--------------------------------------------------+
|                                                  |
|    CREATE:                                       |
|    docker volume create NAME                     |
|                                                  |
|    LIST:                                         |
|    docker volume ls                              |
|                                                  |
|    INSPECT:                                      |
|    docker volume inspect NAME                    |
|                                                  |
|    REMOVE:                                       |
|    docker volume rm NAME                         |
|                                                  |
|    REMOVE ALL UNUSED:                            |
|    docker volume prune                           |
|                                                  |
|    USE WITH CONTAINER:                           |
|    docker run -v NAME:/path image                |
|    docker run -v NAME:/path:ro image (read-only) |
|    docker run -v /host:/path image (bind mount)  |
|    docker run --tmpfs /path image (memory only)  |
|                                                  |
+--------------------------------------------------+
```

Common Mistakes

Mistake 1: Forgetting to Use a Volume for Database Containers

```bash
# BAD: No volume -- data is lost when container is removed
$ docker run -d -e POSTGRES_PASSWORD=pass postgres:16-alpine

# GOOD: Volume preserves data
$ docker run -d -e POSTGRES_PASSWORD=pass \
  -v pgdata:/var/lib/postgresql/data postgres:16-alpine
```

Mistake 2: Using a Bind Mount When You Need a Volume

```bash
# BAD for production: bind mount depends on host path
$ docker run -v /home/user/data:/data my-app

# GOOD for production: named volume is portable
$ docker run -v app-data:/data my-app
```

Bind mounts are great for development (live code editing) but volumes are better for production because they are managed by Docker and do not depend on a specific host directory.

Mistake 3: Not Backing Up Volumes Before Removing Them

```bash
# DANGEROUS: Are you sure there is nothing important in these volumes?
$ docker volume prune -f

# SAFER: List volumes first, back up important ones, then prune
$ docker volume ls
$ docker run --rm -v important-data:/source -v $(pwd):/backup alpine \
    tar czf /backup/important-backup.tar.gz -C /source .
$ docker volume prune
```

Mistake 4: Mounting Over Important Container Directories

```bash
# BAD: This replaces the entire /app directory, including node_modules!
$ docker run -v $(pwd):/app my-node-app
# The container's /app/node_modules is gone because the bind mount replaced it

# SOLUTION: Use an anonymous volume for node_modules
$ docker run \
  -v $(pwd):/app \
  -v /app/node_modules \
  my-node-app
```

The second `-v /app/node_modules` (without a colon) creates an anonymous volume for `node_modules` inside the container, preventing the bind mount from overwriting it.

Mistake 5: Confusing Volume Names with Host Paths

```bash
# This creates a VOLUME named "mydata"
$ docker run -v mydata:/app/data alpine

# This creates a BIND MOUNT from ./mydata on your host
```

```
$ docker run -v ./mydata:/app/data alpine

# This creates a BIND MOUNT from /mydata on your host
$ docker run -v /mydata:/app/data alpine
```

The rule: if the source contains a `/` (slash), it is a bind mount. If it is just a name (no slashes), it is a volume.

Best Practices

1. Always use volumes for database containers. PostgreSQL, MySQL, MongoDB, Redis -- any database needs a volume to persist data.
2. Use named volumes for production data. Named volumes are portable and managed by Docker.
3. Use bind mounts for development. Bind mounts let you edit code on your host and see changes immediately in the container.
4. Back up volumes regularly. Use the tar-based backup method shown in this chapter.
5. Use read-only mounts for configuration files and certificates that the container should not modify.
6. Clean up unused volumes periodically with `docker volume prune` to free disk space.
7. Know your database's data directory. Check the image documentation to know where to mount the volume.
8. Do not store secrets in volumes. Use Docker secrets or environment variables for sensitive data.

Quick Summary

Containers are ephemeral -- their writable layer is deleted when they are removed. Docker provides three storage options: volumes (managed by Docker, best for production), bind mounts (map host directories, best for development), and tmpfs mounts (memory-only, for temporary sensitive data). Named volumes persist data beyond the lifetime of any container. Database containers should always use volumes. Volumes can be backed up by mounting them in a temporary container and creating a tar archive.

Key Points

- Container data is lost when the container is removed (ephemeral filesystem)
- Docker volumes persist data independently of containers
- Three storage types: volumes (Docker-managed), bind mounts (host paths), tmpfs (memory)
- Create volumes with `docker volume create` or let Docker auto-create them
- Mount volumes with `-v volume_name:/container/path`
- Bind mounts use `-v /host/path:/container/path`
- Add `:ro` for read-only mounts
- Always use volumes for databases (PostgreSQL, MySQL, MongoDB)
- Back up volumes by mounting them in a temporary container and using tar
- Names without slashes are volumes; paths with slashes are bind mounts
- Clean up with `docker volume prune` (be careful -- this is permanent)

Practice Questions

1. What happens to data inside a container when you run `docker rm`? Why does this happen?
2. What are the three types of Docker storage? When would you use each one?
3. You start a PostgreSQL container without a volume, create a table, insert data, and then remove and recreate the container. What happens to your data? How would you fix this?
4. What is the difference between a named volume and a bind mount? Give an example of when you would use each.
5. You mount a volume as read-only with `-v config:/app/config:ro`. What happens when the application tries to write a file to `/app/config`?

Exercises

Exercise 1: Volume Basics

1. Create a named volume called `exercise-data`
2. Start a container that mounts the volume at `/data`
3. Write a file to `/data/hello.txt` inside the container
4. Remove the container
5. Start a new container with the same volume
6. Verify the file is still there
7. Clean up: remove the container and the volume

Exercise 2: Database Persistence

1. Create a volume called `mysql-data`
2. Start a MySQL container with the volume mounted at `/var/lib/mysql`
3. Create a database and a table with some data
4. Stop and remove the container
5. Start a new MySQL container with the same volume
6. Verify your database, table, and data are still intact

Exercise 3: Backup and Restore

1. Create a volume and populate it with several files using a container
2. Back up the volume to a tar.gz file on your host
3. Remove the original volume
4. Create a new volume and restore the backup into it
5. Verify all files are present in the restored volume

What Is Next?

You now understand how to persist data in Docker using volumes and bind mounts. This is a fundamental skill for running stateful applications like databases. In the next chapter, we will explore Docker networking -- how containers communicate with each other and the outside world. You will learn how to connect a web application container to a database container, creating multi-container applications.

Chapter 12: Docker Networking

What You Will Learn

By the end of this chapter, you will be able to:

- Explain how Docker containers communicate with each other
- Understand the default bridge network and its limitations
- Map container ports to your host machine using `-p`
- Create custom bridge networks for better container communication
- Use container names as hostnames (DNS resolution)
- Connect an application container to a database container
- List, inspect, connect, and disconnect networks
- Choose the right network driver for your use case

Why This Chapter Matters

Imagine you are building a web application. You have a web server running in one container and a database running in another container. How do they talk to each other? Can the web server find the database? Can the outside world reach your web server?

These are networking questions, and understanding Docker networking is essential for building real applications. Without networking knowledge, your containers are like isolated rooms with no doors or windows. This chapter gives you the keys to connect them.

Think of it this way. In the real world, a restaurant has a kitchen (the database) and a dining room (the web server). Customers (the outside world) enter through the front door (port mapping), and the waiters carry food between the kitchen and dining room through an internal hallway (the Docker network). This chapter teaches you how to build those hallways and doors.

How Docker Networking Works

When you install Docker, it automatically creates a networking system that allows containers to communicate. Let us start by understanding the big picture.

```
┌──────────────────────────────────────────────────────────┐
│                    Your Computer (Host)                  │
│                                                          │
│    ┌──────────────┐         ┌──────────────┐            │
│    │  Container A  │         │  Container B  │           │
│    │  (Web App)    │         │  (Database)   │           │
│    │  Port 3000    │         │  Port 5432    │           │
│    └──────┬───────┘         └──────┬───────┘            │
│           │                         │                    │
│    ┌──────┴─────────────────────────┴──────┐             │
│    │          Docker Network               │             │
│    │     (Internal communication)          │             │
│    └──────────────────┬────────────────────┘             │
│                       │                                  │
│              ┌────────┴────────┐                         │
│              │   Port Mapping  │                         │
│              │  Host:Container │                         │
│              └────────┬────────┘                         │
│                       │                                  │
└───────────────────────┴──────────────────────────────────┘
                        │
                  Outside World
                 (Your Browser)
```

Docker networking has three main concepts:

1. Container-to-container communication: Containers on the same network can talk to each other.
2. Port mapping: You expose container ports to your host machine so the outside world can reach them.
3. Network isolation: Containers on different networks cannot communicate unless you explicitly connect them.

The Default Bridge Network

When you install Docker, it creates a network called `bridge`. This is the default bridge network. Every container you start without specifying a network gets connected to this default bridge network automatically.

Let us see it in action.

Listing Docker Networks

Run this command to see all networks on your system:

```bash
docker network ls
```

Output:

```
NETWORK ID     NAME      DRIVER    SCOPE
a1b2c3d4e5f6   bridge    bridge    local
f6e5d4c3b2a1   host      host      local
1a2b3c4d5e6f   none      null      local
```

Let us break down each column:

- NETWORK ID: A unique identifier for the network. Docker generates this automatically.
- NAME: The human-readable name of the network.
- DRIVER: The type of network. We will explain drivers later in this chapter.
- SCOPE: Where the network is available. `local` means it exists only on this machine.

You see three networks that Docker created automatically:

1. bridge: The default network for containers. Think of it as a shared hallway.
2. host: Removes network isolation between the container and the host. The container uses the host's network directly.
3. none: No networking at all. The container is completely isolated.

Running Containers on the Default Bridge

Let us start two containers and see how they communicate.

```bash
docker run -d --name web nginx
```

Output:

```
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6
```

```bash
docker run -d --name api nginx
```

Output:

```
b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6a1
```

Both containers are now running on the default bridge network. Let us verify:

```bash
docker network inspect bridge
```

Output (simplified):

```json
[
    {
        "Name": "bridge",
        "Driver": "bridge",
        "Containers": {
            "a1b2c3...": {
                "Name": "web",
                "IPv4Address": "172.17.0.2/16"
            },
            "b2c3d4...": {
                "Name": "api",
                "IPv4Address": "172.17.0.3/16"
            }
        }
    }
]
```

Both containers have IP addresses on the same network (172.17.0.x). They can communicate using these IP addresses.

Testing Communication on the Default Bridge

Let us go inside the `web` container and try to reach the `api` container:

```bash
docker exec -it web bash
```

Now inside the container, install `ping` (it is not included by default in the nginx image):

```bash
apt-get update && apt-get install -y iputils-ping
```

Try pinging the `api` container by its IP address:

```bash
ping -c 3 172.17.0.3
```

Output:

```
PING 172.17.0.3 (172.17.0.3) 56(84) bytes of data.
64 bytes from 172.17.0.3: icmp_seq=1 ttl=64 time=0.089 ms
64 bytes from 172.17.0.3: icmp_seq=2 ttl=64 time=0.065 ms
64 bytes from 172.17.0.3: icmp_seq=3 ttl=64 time=0.071 ms

--- 172.17.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

It works. The containers can communicate by IP address.

Now try pinging by container name:

```bash
ping -c 3 api
```

Output:

```
ping: api: Name or service not known
```

This fails. The default bridge network does not provide DNS resolution (the ability to use container names as addresses). You can only use IP addresses.

Type `exit` to leave the container:

```bash
exit
```

The Problem with the Default Bridge Network

The default bridge network has two significant limitations:

1. No DNS resolution: You cannot use container names to communicate. You must use IP addresses.

2. IP addresses change: Every time you restart a container, Docker might assign it a different IP address. This means your application would break because it is trying to connect to the old address.

Think of it like this. The default bridge network is like a building where apartments have numbers, but there is no directory listing who lives where. If your friend moves to a different apartment, you have no way to find them unless they tell you their new number.

```
┌──────────────────────────────────────────┐
│        Default Bridge Network            │
│                                          │
│   Container "web"    Container "api"     │
│   IP: 172.17.0.2     IP: 172.17.0.3      │
│                                          │
│   web -> api:  Must use 172.17.0.3       │
│   web -> api:  Cannot use name "api"     │
│                                          │
│   After restart:                         │
│   IP: 172.17.0.4     IP: 172.17.0.2      │
│   (IPs may change!)                      │
└──────────────────────────────────────────┘
```

This is why Docker recommends using user-defined bridge networks instead. Let us learn about those next.

Clean up:

```bash
docker rm -f web api
```

Output:

```
web
api
```

Port Mapping: Letting the Outside World In

Before we move to custom networks, let us understand port mapping. By default, containers are isolated from the outside world. If you run a web server inside a container, you cannot access it from your browser. Port mapping creates a tunnel from your host machine into the container.

What Is a Port?

A port is a numbered doorway on a computer where network traffic enters or exits. Think of your computer as an apartment building. Each apartment has a door number. Port 80 is the door for web traffic (HTTP). Port 443 is the door for secure web traffic (HTTPS). Port 5432 is the door for PostgreSQL databases.

When a container runs a web server on port 80 inside the container, that port 80 only exists inside the container. Your browser on the host machine cannot reach it. Port mapping connects a port on your host to a port in the container.

Using the -p Flag

The `-p` flag (short for `--publish`) maps a host port to a container port.

```bash
docker run -d --name my-web -p 8080:80 nginx
```

Let us break down `-p 8080:80`:

- 8080: The port on your host machine (your computer).
- 80: The port inside the container where nginx is listening.
- The colon separates host from container: `host:container`.

```
┌──────────────────────────────────────────────────┐
│              Your Computer (Host)                 │
│                                                   │
│   Browser ──> localhost:8080                      │
│                    │                               │
│                    │  (port mapping)               │
│                    ▼                               │
│    ┌──────────────────────────┐                   │
│    │      Container "my-web"  │                   │
│    │      nginx on port 80    │                   │
│    └──────────────────────────┘                   │
└──────────────────────────────────────────────────┘
```

Now open your browser and go to `http://localhost:8080`. You will see the nginx welcome page.

Common Port Mapping Patterns

Here are some common port mapping examples:

```bash
# Map host port 3000 to container port 3000 (same port)
docker run -d -p 3000:3000 my-node-app

# Map host port 5433 to container port 5432 (different ports)
docker run -d -p 5433:5432 postgres

# Map multiple ports
docker run -d -p 8080:80 -p 8443:443 nginx

# Map to a specific host IP address
docker run -d -p 127.0.0.1:8080:80 nginx

# Let Docker choose a random host port
docker run -d -p 80 nginx
```

Let us explain each:

- Same port (`3000:3000`): The simplest case. Host and container use the same port number.
- Different ports (`5433:5432`): Useful when the default port is already in use on your host. Here, PostgreSQL listens on 5432 inside the container, but you access it via 5433 on your host.
- Multiple ports (`-p 8080:80 -p 8443:443`): Maps more than one port. Useful for servers that use both HTTP and HTTPS.
- Specific IP (`127.0.0.1:8080:80`): Only allows connections from your own machine, not from other computers on your network.
- Random port (`-p 80`): Docker picks an available host port automatically. Use `docker port` to see which port was assigned.

Checking Port Mappings

To see which ports are mapped for a running container:

```bash
docker port my-web
```

Output:

```
80/tcp -> 0.0.0.0:8080
```

This tells you that port 80 in the container is mapped to port 8080 on all host interfaces (`0.0.0.0` means all interfaces).

Clean up:

```bash
docker rm -f my-web
```

User-Defined Bridge Networks

User-defined bridge networks solve the problems of the default bridge network. They provide:

1. Automatic DNS resolution: Containers can find each other by name.
2. Better isolation: Only containers you explicitly add can communicate.
3. Live connection management: You can connect and disconnect containers from networks without restarting them.

Creating a Custom Network

Use `docker network create` to make a new network:

```bash
docker network create my-app-network
```

Output:

```
c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0t1u2v3w4x5y6z7a8b9
```

Docker returns the network ID. Now verify it exists:

```bash
docker network ls
```

Output:

```
NETWORK ID     NAME             DRIVER    SCOPE
a1b2c3d4e5f6   bridge           bridge    local
f6e5d4c3b2a1   host             host      local
c4d5e6f7g8h9   my-app-network   bridge    local
1a2b3c4d5e6f   none             null      local
```

Your new network `my-app-network` appears in the list. It uses the `bridge` driver by default.

Running Containers on a Custom Network

Use the `--network` flag to start a container on your custom network:

```bash
docker run -d --name web --network my-app-network nginx
```

```bash
docker run -d --name api --network my-app-network nginx
```

Now let us test DNS resolution. Go inside the `web` container:

```bash
docker exec -it web bash
apt-get update && apt-get install -y iputils-ping
```

Try pinging the `api` container by name:

```bash
ping -c 3 api
```

Output:

```
PING api (172.18.0.3) 56(84) bytes of data.
64 bytes from api.my-app-network (172.18.0.3): icmp_seq=1 ttl=64 time=0.065 ms
64 bytes from api.my-app-network (172.18.0.3): icmp_seq=2 ttl=64 time=0.058 ms
64 bytes from api.my-app-network (172.18.0.3): icmp_seq=3 ttl=64 time=0.061 ms

--- api ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

It works. The container name `api` is automatically resolved to its IP address. This is the key advantage of user-defined bridge networks.

```
┌──────────────────────────────────────────────────┐
│        User-Defined Bridge Network               │
│        "my-app-network"                          │
│                                                  │
│   Container "web"      Container "api"           │
│   IP: 172.18.0.2       IP: 172.18.0.3            │
│                                                  │
│   web -> api:  Can use name "api"     ✓          │
│   web -> api:  Can use IP 172.18.0.3  ✓          │
│                                                  │
│   Docker DNS resolves "api" -> 172.18.0.3        │
│   Even after restart, name stays the same!       │
└──────────────────────────────────────────────────┘
```

Exit the container:

```bash
exit
```

Clean up:

```bash
docker rm -f web api
```

Default Bridge vs. User-Defined Bridge

Here is a comparison to make the differences clear:

```
┌──────────────────────────┬──────────────────────────────┐
│   Default Bridge         │   User-Defined Bridge        │
├──────────────────────────┼──────────────────────────────┤
│ Created automatically    │ You create it manually       │
│ No DNS resolution        │ Automatic DNS resolution     │
│ All containers join      │ Only specified containers    │
│   by default             │   join                       │
│ Less secure (shared      │ More secure (isolated        │
│   by all containers)     │   to your app)               │
│ Cannot connect/          │ Can connect/disconnect       │
│   disconnect live        │   containers at any time     │
│ Use IP addresses only    │ Use container names          │
└──────────────────────────┴──────────────────────────────┘
```

Bottom line: Always use user-defined bridge networks for your applications. The default bridge network should only be used for quick, throwaway experiments.

Connecting an Application to a Database

Let us put everything together with a real-world example. We will run a PostgreSQL database and connect a simple application to it.

Step 1: Create the Network

```bash
docker network create app-network
```

Step 2: Start the Database

```bash
docker run -d \
  --name my-postgres \
  --network app-network \
  -e POSTGRES_USER=myuser \
  -e POSTGRES_PASSWORD=mypassword \
  -e POSTGRES_DB=myapp \
  postgres:16
```

Let us break down this command:

- `-d`: Run in the background (detached mode).
- `--name my-postgres`: Give the container the name "my-postgres". Other containers will use this name to connect.
- `--network app-network`: Attach the container to our custom network.
- `-e POSTGRES_USER=myuser`: Set the database username.
- `-e POSTGRES_PASSWORD=mypassword`: Set the database password.
- `-e POSTGRES_DB=myapp`: Create a database called "myapp".
- `postgres:16`: Use the official PostgreSQL 16 image.

Step 3: Start an Application Container

Let us start a simple container and connect to the database from it:

```bash
docker run -it --rm \
  --name my-app \
  --network app-network \
  postgres:16 \
  psql -h my-postgres -U myuser -d myapp
```

Let us break this down:

- `-it`: Interactive mode with a terminal (so we can type commands).
- `--rm`: Remove the container when we exit.
- `--network app-network`: Connect to the same network as the database.
- `postgres:16`: We use the postgres image because it includes the `psql` client.
- `psql -h my-postgres -U myuser -d myapp`: Connect to the host `my-postgres` (the container name), as user `myuser`, to the database `myapp`.

Output:

```
Password for user myuser:
```

Type `mypassword` and press Enter:

```
psql (16.1)
Type "help" for help.

myapp=>
```

You are now connected to the PostgreSQL database running in another container. The key thing to notice is that we used `my-postgres` as the hostname. Docker's DNS resolved that name to the database container's IP address automatically.

Let us run a quick SQL command to verify:

```sql
SELECT version();
```

Output:

```
                           version
-----------------------------------------------------------
 PostgreSQL 16.1 on x86_64-pc-linux-gnu, compiled by ...
(1 row)
```

Type `\q` to exit psql, or press Ctrl+D.

```
┌──────────────────────────────────────────────────────────┐
│              app-network                                 │
│                                                          │
│   ┌──────────────────┐    ┌──────────────────┐          │
│   │    my-app        │    │   my-postgres    │          │
│   │                  │    │                  │          │
│   │  psql connects   │───>│  PostgreSQL      │          │
│   │  to "my-postgres"│    │  Port 5432       │          │
│   │                  │    │                  │          │
│   └──────────────────┘    └──────────────────┘          │
│                                                          │
│   DNS: "my-postgres" resolves to 172.18.0.2              │
└──────────────────────────────────────────────────────────┘
```

Step 4: Expose the Database (Optional)

If you also want to connect to the database from your host machine (for example, using a database GUI tool), add port mapping:

```bash
docker rm -f my-postgres

docker run -d \
  --name my-postgres \
  --network app-network \
  -p 5432:5432 \
  -e POSTGRES_USER=myuser \
  -e POSTGRES_PASSWORD=mypassword \
  -e POSTGRES_DB=myapp \
  postgres:16
```

Now you can connect from your host at `localhost:5432` AND other containers can still connect using the name `my-postgres`.

Clean up:

```bash
docker rm -f my-postgres
docker network rm app-network
```

Network Management Commands

Here is a complete reference for managing Docker networks.

docker network ls — List All Networks

```bash
docker network ls
```

Output:

```
NETWORK ID     NAME      DRIVER    SCOPE
a1b2c3d4e5f6   bridge    bridge    local
f6e5d4c3b2a1   host      host      local
1a2b3c4d5e6f   none      null      local
```

docker network inspect — View Network Details

```bash
docker network inspect bridge
```

This shows detailed information about a network, including which containers are connected, the subnet, the gateway, and more.

docker network create — Create a Network

```bash
# Basic creation
docker network create my-network

# With a specific subnet
docker network create --subnet 192.168.1.0/24 my-network

# With a specific driver
docker network create --driver bridge my-network
```

docker network connect — Add a Container to a Network

You can add a running container to a network without restarting it:

```bash
docker network connect my-network my-container
```

This is useful when you need a container to communicate with containers on multiple networks.

docker network disconnect — Remove a Container from a Network

```bash
docker network disconnect my-network my-container
```

The container keeps running but can no longer communicate with other containers on that network.

docker network rm — Delete a Network

```bash
docker network rm my-network
```

You cannot delete a network that has containers connected to it. You must disconnect or remove all containers first.

docker network prune — Remove Unused Networks

```bash
docker network prune
```

Output:

```
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
my-old-network
test-network
```

This removes all custom networks that have no containers connected to them. The default bridge, host, and none networks are never removed.

Connecting a Container to Multiple Networks

A container can belong to more than one network. This is useful when you want a container to communicate with different groups of containers.

```bash
# Create two networks
docker network create frontend-network
docker network create backend-network

# Start containers
docker run -d --name web --network frontend-network nginx
docker run -d --name api --network backend-network nginx
docker run -d --name database --network backend-network postgres:16

# Connect the API to the frontend network too
docker network connect frontend-network api
```

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│   frontend-network          backend-network              │
│   ┌──────────────┐          ┌──────────────┐             │
│   │              │          │              │             │
│   │   web ◄──────►│ api ◄──►│   database   │             │
│   │              │          │              │             │
│   └──────────────┘          └──────────────┘             │
│                                                          │
```

```
│   web can reach api:     Yes (frontend-network)      │
│   api can reach database: Yes (backend-network)      │
│   web can reach database: No  (different networks)   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

In this setup:

- The `web` container can reach `api` because they share `frontend-network`.
- The `api` container can reach `database` because they share `backend-network`.
- The `web` container cannot reach `database` because they are on different networks.

This is a common pattern for security. The database is only accessible from the API layer, never directly from the web layer.

Clean up:

```bash
docker rm -f web api database
docker network rm frontend-network backend-network
```

Network Driver Types

Docker supports different network drivers, each designed for a specific use case. Think of network drivers like different types of roads — a highway, a private road, and no road at all.

Bridge Driver (Default)

The bridge driver creates an isolated network on your host machine. Containers on the same bridge network can communicate. This is the most common driver and the one you will use for most applications.

```bash
# Bridge is the default driver
docker network create my-bridge-network

# Explicit driver specification (same result)
docker network create --driver bridge my-bridge-network
```

When to use: For most single-host applications. This is the right choice 99% of the time during development.

Host Driver

The host driver removes the network isolation between the container and the host. The container shares the host's network directly. This means the container does not get its own IP address — it uses the host's IP address.

```bash
docker run -d --network host nginx
```

With the host network, there is no port mapping needed. If nginx listens on port 80 inside the container, it is available at port 80 on your host directly.

```
┌──────────────────────────────────────┐
│   Bridge Driver                      │
│                                      │
│   Host (port 8080) ──> Container     │
│   (port mapping required)            │
│                                      │
│   Container has own IP: 172.17.0.2   │
└──────────────────────────────────────┘


┌──────────────────────────────────────┐
│   Host Driver                        │
│                                      │
│   Host = Container                   │
│   (no port mapping needed)           │
│                                      │
│   Container uses host IP directly    │
└──────────────────────────────────────┘
```

When to use: When you need maximum network performance and do not need network isolation. Common in performance-critical applications. Note that the host network driver only works on Linux, not on Docker Desktop for Mac or Windows.

None Driver

The none driver gives the container no network access at all. The container is completely isolated.

```bash
docker run -d --network none nginx
```

The container cannot communicate with other containers, the host, or the internet.

When to use: For security-sensitive containers that should not have any network access, such as a container that only processes local files.

Summary of Network Drivers

```
┌──────────────┬──────────────────────────┬──────────────────────────┐
│  Driver      │  Description             │  Use Case                │
├──────────────┼──────────────────────────┼──────────────────────────┤
│  bridge      │  Isolated network on the │  Most applications       │
│              │  host. Containers get own │  (default choice)        │
│              │  IP addresses.           │                          │
├──────────────┼──────────────────────────┼──────────────────────────┤
│  host        │  Container uses the host │  Performance-            │
│              │  network directly. No    │  critical apps.          │
│              │  isolation.              │  Linux only.             │
├──────────────┼──────────────────────────┼──────────────────────────┤
│  none        │  No networking at all.   │  Security-sensitive      │
│              │  Complete isolation.     │  workloads.              │
└──────────────┴──────────────────────────┴──────────────────────────┘
```

Common Mistakes

Mistake 1: Using the Default Bridge Network for Applications

```bash
# Wrong: relies on IP addresses that can change
docker run -d --name db postgres:16
docker run -d --name app my-app
# Inside app, connecting to db by IP 172.17.0.2 — fragile!

# Right: use a custom network with DNS
docker network create app-net
docker run -d --name db --network app-net postgres:16
docker run -d --name app --network app-net my-app
# Inside app, connecting to "db" by name — reliable!
```

Mistake 2: Forgetting Port Mapping

```bash
# Wrong: no port mapping, cannot access from browser
docker run -d nginx

# Right: map a host port
docker run -d -p 8080:80 nginx
```

Mistake 3: Port Already in Use

```bash
# This will fail if port 8080 is already in use
docker run -d -p 8080:80 nginx
```

Error:

```
Error response from daemon: driver failed programming external connectivity:
Bind for 0.0.0.0:8080 failed: port is already allocated
```

Solution: Use a different host port:

```bash
docker run -d -p 8081:80 nginx
```

Mistake 4: Containers on Different Networks

```bash
# These containers cannot communicate!
docker network create network-a
docker network create network-b
docker run -d --name web --network network-a nginx
docker run -d --name api --network network-b nginx
# web cannot reach api - they are on different networks
```

Solution: Put them on the same network, or use `docker network connect` to add the container to an additional network.

Mistake 5: Confusing Host Port and Container Port Order

```bash
# Wrong order thinking: this maps host 80 to container 3000
# Actual: this maps host 3000 to container 80
docker run -d -p 3000:80 nginx
```

Remember: it is always `host:container`. Think of it as `outside:inside`.

Best Practices

1. Always use user-defined bridge networks for multi-container applications. Never rely on the default bridge network.
2. Use meaningful network names that describe their purpose: `frontend-net`, `backend-net`, `app-network`.
3. Separate concerns with multiple networks. Put your web tier and database tier on different networks. Connect the API to both.
4. Use container names as hostnames in your application configuration. For example, set your database host to `my-postgres` instead of an IP address.
5. Only expose ports you need. Do not map ports unless you need to access the container from outside Docker.
6. Use specific host IPs for security. Instead of `-p 8080:80` (accessible from anywhere), use `-p 127.0.0.1:8080:80` (accessible only from your machine).
7. Clean up unused networks regularly with `docker network prune`.

Quick Summary

Docker networking allows containers to communicate with each other and with the outside world. The default bridge network connects all containers but does not support DNS resolution. User-defined bridge networks solve this by letting containers find each other by name. Port mapping with `-p host:container` lets traffic from the outside world reach containers. Network drivers (bridge, host, none) provide different levels of isolation. For real applications, always create custom networks and use container names as hostnames.

Key Points

- Docker creates three default networks: bridge, host, and none.
- The default bridge network does not support DNS resolution (you cannot use container names).
- User-defined bridge networks provide automatic DNS resolution and better isolation.
- Port mapping (`-p host:container`) lets the outside world reach containers.
- The format is always `host_port:container_port` — think "outside:inside."
- Use `docker network create` to make a custom network.
- Use `docker network connect` and `docker network disconnect` to manage containers at runtime.

- The bridge driver is the right choice for most applications.
- Containers on different networks cannot communicate unless explicitly connected.
- Always use container names (not IP addresses) for inter-container communication.

Practice Questions

1. What is the difference between the default bridge network and a user-defined bridge network? Why does Docker recommend user-defined networks?
2. You run `docker run -d -p 5000:3000 my-app`. What does this command do? Which port do you use in your browser to access the app?
3. You have three containers: `frontend`, `api`, and `database`. You want the frontend to communicate with the api, and the api to communicate with the database, but the frontend should NOT be able to reach the database directly. How would you set up the networks?
4. What happens if you try to delete a network that still has containers connected to it?
5. Explain the difference between the bridge, host, and none network drivers. When would you use each one?

Exercises

Exercise 1: Build a Two-Container Network

1. Create a custom network called `exercise-net`.
2. Start an nginx container named `webserver` on `exercise-net` with port 8080 mapped to port 80.
3. Start another nginx container named `backend` on `exercise-net`.
4. Go inside `webserver` and verify you can ping `backend` by name.
5. Access the webserver from your browser at `http://localhost:8080`.
6. Clean up all containers and the network.

Exercise 2: Multi-Network Isolation

1. Create two networks: `public-net` and `private-net`.
2. Start a container named `web` on `public-net`.

3. Start a container named `api` on both `public-net` and `private-net`.
4. Start a container named `db` on `private-net`.
5. Verify that `web` can reach `api` but cannot reach `db`.
6. Verify that `api` can reach both `web` and `db`.
7. Clean up everything.

Exercise 3: Connect an App to a Database

1. Create a network called `db-net`.
2. Start a PostgreSQL container named `mydb` on `db-net` with username `admin`, password `secret123`, and database `testdb`.
3. Start another container on the same network and use `psql` to connect to `mydb` by name.
4. Create a table and insert some data to verify the connection works.
5. Clean up everything.

What Is Next?

You now know how to connect containers and let them communicate. But when something goes wrong, how do you figure out what happened? In the next chapter, you will learn how to read container logs, debug issues, inspect container internals, and systematically solve common problems. These debugging skills will save you hours of frustration as you build more complex applications.

Chapter 13: Logs and Debugging

What You Will Learn

By the end of this chapter, you will be able to:

- Read container logs using `docker logs`
- Filter logs by time, count, and follow them in real time
- Get inside a running container with `docker exec`
- Inspect container details like IP address, mounts, and environment variables
- Add health checks to your Dockerfiles
- Monitor Docker events in real time
- Diagnose and fix the most common container problems
- Follow a systematic debugging flowchart when things go wrong

Why This Chapter Matters

Things will go wrong. Your container will crash. Your application will not start. Your database connection will fail. A file will be missing. A port will be in use. This is not a question of "if" but "when."

The difference between a frustrated developer and a productive one is not that the productive developer never has problems. It is that they know how to find and fix problems quickly. This chapter gives you that superpower.

Think of debugging like being a detective. When a crime happens, a detective does not panic. They follow a process: check the evidence (logs), examine the scene (inspect), interview witnesses (events), and look for patterns (common issues). By the end of this chapter, you will be a Docker detective.

Reading Container Logs with docker logs

Every application produces output — messages about what it is doing, warnings about potential problems, and error messages when something breaks. When an application runs inside a Docker container, Docker captures all of this output and stores it for you.

Basic Log Reading

Let us start a container and look at its logs:

```bash
docker run -d --name my-web -p 8080:80 nginx
```

Now visit `http://localhost:8080` in your browser a few times. Then read the logs:

```bash
docker logs my-web
```

Output:

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform conf...
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
...
2024-01-15 10:30:00 172.17.0.1 - - [15/Jan/2024:10:30:00 +0000] "GET / HTTP/1.1" 200 61...
2024-01-15 10:30:01 172.17.0.1 - - [15/Jan/2024:10:30:01 +0000] "GET /favicon.ico HTTP/...
```

Each line is a log entry. For nginx, you can see:

- Startup messages: Lines that start with `/docker-entrypoint.sh` show nginx initializing.
- Access logs: Lines showing HTTP requests. The `200` means "OK" (the page was found). The `404` means "Not Found" (the favicon was not found, which is normal).

Understanding What Docker Logs Captures

Docker captures two streams of output:

1. stdout (standard output): Normal messages. Think of this as the application talking normally.
2. stderr (standard error): Error messages. Think of this as the application shouting about problems.

```
┌──────────────────────────────────┐
│        Your Container            │
│                                  │
│   Application                    │
│     │                            │
│     ├── stdout ──> "Started OK"  │──> docker logs
│     │                            │
│     └── stderr ──> "Error!"      │──> docker logs
│                                  │
└──────────────────────────────────┘
```

Both streams appear when you run `docker logs`. This is important to understand because some applications write their logs to files instead of stdout/stderr. If your application writes to `/var/log/app.log` inside the container, `docker logs` will not show those logs. You would need to go inside the container to read them.

Showing Only the Last N Lines with --tail

When a container has been running for a long time, it may have thousands of log lines. You usually only care about the most recent ones.

```bash
docker logs --tail 10 my-web
```

This shows only the last 10 lines. You can change the number to whatever you need:

```bash
# Last 5 lines
docker logs --tail 5 my-web

# Last 100 lines
docker logs --tail 100 my-web

# Last 1 line
docker logs --tail 1 my-web
```

Filtering by Time with --since

You can show logs from a specific time onward:

```bash
# Logs from the last 5 minutes
docker logs --since 5m my-web

# Logs from the last 1 hour
```

```
docker logs --since 1h my-web

# Logs from the last 30 seconds
docker logs --since 30s my-web

# Logs since a specific timestamp
docker logs --since "2024-01-15T10:30:00" my-web
```

The time units are:

- `s` for seconds
- `m` for minutes
- `h` for hours

You can also use `--until` to show logs up to a certain time:

```bash
# Logs between 1 hour ago and 30 minutes ago
docker logs --since 1h --until 30m my-web
```

Following Logs in Real Time with -f

The `-f` flag (short for `--follow`) streams logs continuously, just like `tail -f` on Linux. New log lines appear as they happen.

```bash
docker logs -f my-web
```

Now open your browser and visit `http://localhost:8080` again. You will see new log lines appear immediately in your terminal.

To stop following, press `Ctrl+C`. This stops the log stream but does not stop the container.

Combining Flags

You can combine flags for precise log viewing:

```bash
# Follow logs, starting from the last 5 lines
docker logs -f --tail 5 my-web

# Show last 20 lines from the past hour
docker logs --tail 20 --since 1h my-web
```

Showing Timestamps with -t

Add timestamps to each log line:

```bash
docker logs -t my-web
```

Output:

```
2024-01-15T10:30:00.123456789Z 172.17.0.1 - - "GET / HTTP/1.1" 200 615
2024-01-15T10:30:01.234567890Z 172.17.0.1 - - "GET /favicon.ico HTTP/1.1" 404 153
```

The `Z` at the end means the timestamp is in UTC (Coordinated Universal Time).

Clean up:

```bash
docker rm -f my-web
```

Getting Inside a Container with docker exec

Sometimes reading logs is not enough. You need to go inside the container to investigate — check files, test commands, or look at the environment. The `docker exec` command lets you run commands inside a running container.

Opening a Shell

The most common use of `docker exec` is opening an interactive shell:

```bash
docker run -d --name my-web nginx
docker exec -it my-web bash
```

Let us break down the flags:

- `-i` (interactive): Keeps the input stream open so you can type commands.
- `-t` (terminal): Allocates a terminal so the output is formatted nicely.
- `my-web`: The name of the container.
- `bash`: The command to run (a Bash shell).

You are now inside the container. Your prompt changes to something like:

```
root@a1b2c3d4e5f6:/#
```

You can explore:

```bash
# See what is in the root directory
ls /

# Check the nginx configuration
cat /etc/nginx/nginx.conf

# See running processes
ps aux

# Check the hostname
hostname

# Check environment variables
env
```

Type `exit` to leave:

```bash
exit
```

When bash Is Not Available

Some minimal container images do not include `bash`. In that case, use `sh` (the Bourne shell), which is almost always available:

```bash
docker exec -it my-web sh
```

If the container is based on Alpine Linux (a very small Linux distribution), `sh` is your only option because Alpine does not include `bash` by default.

Running a Single Command

You do not need to open a full shell. You can run a single command and get the output:

```bash
# Check the nginx version
docker exec my-web nginx -v
```

Output:

```
nginx version: nginx/1.25.3
```

```bash
# List files in a directory
docker exec my-web ls /etc/nginx/

# Check the IP address
docker exec my-web hostname -i

# Read a specific file
docker exec my-web cat /etc/hostname
```

Notice that for single commands, we do not need `-it` because we are not typing interactively.

Clean up:

```bash
docker rm -f my-web
```

Inspecting Containers with docker inspect

The `docker inspect` command gives you a detailed JSON document with everything Docker knows about a container. It is like reading a container's medical record — every detail is there.

Basic Inspection

```bash
docker run -d --name my-web -p 8080:80 nginx
docker inspect my-web
```

This produces a large amount of JSON output. The most useful sections are:

- NetworkSettings: IP address, ports, networks
- Mounts: Volumes and bind mounts
- Config.Env: Environment variables
- State: Running, paused, exit code, error messages

Finding Specific Information with --format

Instead of reading through pages of JSON, you can extract exactly what you need using the `--format` flag with Go template syntax.

Get the Container's IP Address

```bash
docker inspect --format '{{.NetworkSettings.IPAddress}}' my-web
```

Output:

```
172.17.0.2
```

Get the Container's Status

```bash
docker inspect --format '{{.State.Status}}' my-web
```

Output:

```
running
```

Get Environment Variables

```bash
docker inspect --format '{{.Config.Env}}' my-web
```

Output:

```
[PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin NGINX_VERSION=1.25.3...
```

Get Mount Points

```bash
docker inspect --format '{{json .Mounts}}' my-web
```

Output:

```json
[]
```

An empty array means no volumes are mounted.

Get the Container's Restart Count

```bash
docker inspect --format '{{.RestartCount}}' my-web
```

Output:

```
0
```

Get the Port Mappings

```bash
docker inspect --format '{{json .NetworkSettings.Ports}}' my-web
```

Output:

```json
{"80/tcp":[{"HostIp":"0.0.0.0","HostPort":"8080"}]}
```

Common docker inspect Queries Reference

```
┌──────────────────────────────────────┬──────────────────────────────────────┐
│  What You Want                       │  Format String                       │
├──────────────────────────────────────┼──────────────────────────────────────┤
│  IP Address                          │  {{.NetworkSettings.IPAddress}}      │
│  Status                              │  {{.State.Status}}                   │
│  Exit Code                           │  {{.State.ExitCode}}                 │
│  Start Time                          │  {{.State.StartedAt}}                │
│  Image                               │  {{.Config.Image}}                   │
│  Command                             │  {{.Config.Cmd}}                     │
│  Environment Variables               │  {{.Config.Env}}                     │
│  Mounted Volumes                     │  {{json .Mounts}}                    │
│  Port Mappings                       │  {{json .NetworkSettings.Ports}}     │
│  Restart Count                       │  {{.RestartCount}}                   │
│  Container PID                       │  {{.State.Pid}}                      │
└──────────────────────────────────────┴──────────────────────────────────────┘
```

Clean up:

```bash
docker rm -f my-web
```

Health Checks in Dockerfiles

A health check is a command that Docker runs periodically to check if your application is working correctly. Think of it like a doctor regularly checking a patient's pulse. The container might be "running" (the process exists), but the application inside might have crashed, frozen, or become unresponsive.

Why Health Checks Matter

Without a health check, Docker only knows if the main process is running. It does not know if the process is actually doing useful work. For example:

- A web server process might be running but returning errors for every request.
- A database might be running but not accepting connections.
- An API might be running but stuck in an infinite loop.

A health check tells Docker to periodically verify that the application is truly healthy.

Adding a HEALTHCHECK to Your Dockerfile

```dockerfile
FROM nginx:latest

# Copy your website files
COPY ./html /usr/share/nginx/html

# Health check: curl the homepage every 30 seconds
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost/ || exit 1
```

Let us break down each part:

- `HEALTHCHECK`: The Dockerfile instruction that defines a health check.
- `--interval=30s`: Run the check every 30 seconds. Docker waits this long between checks.
- `--timeout=10s`: If the check takes longer than 10 seconds, consider it failed.
- `--start-period=5s`: Give the application 5 seconds to start up before beginning health checks. This prevents false failures during startup.
- `--retries=3`: The container is marked "unhealthy" after 3 consecutive failures. One failure alone does not mark it unhealthy.
- `CMD curl -f http://localhost/ || exit 1`: The actual check command. `curl -f` makes a request to the homepage. If it fails, exit with code 1 (unhealthy). If it succeeds, exit with code 0 (healthy).

Health Check States

A container with a health check goes through these states:

```
┌──────────────────────────────────────────────────┐
│                                                  │
│   starting ──────> healthy ──────> unhealthy     │
│      │                │               │          │
│      │                │               │          │
│      │                └───────────────┘          │
│      │           (can recover back to healthy)  │
│      │                                           │
│   (during start-period,                          │
│    failures are ignored)                         │
│                                                  │
└──────────────────────────────────────────────────┘
```

- starting: The container just started and is within the start period. Health check failures are ignored.
- healthy: The health check is passing. The application is working correctly.
- unhealthy: The health check has failed the specified number of retries in a row. Something is wrong.

Checking Health Status

You can see the health status in `docker ps`:

```bash
docker ps
```

Output:

```
CONTAINER ID   IMAGE     STATUS                    PORTS
a1b2c3d4e5f6   my-app    Up 2 minutes (healthy)    0.0.0.0:8080->80/tcp
```

Notice the `(healthy)` in the STATUS column.

You can also get detailed health information with `docker inspect`:

```bash
docker inspect --format '{{json .State.Health}}' my-app
```

Output (formatted):

```json
{
  "Status": "healthy",
  "FailingStreak": 0,
  "Log": [
    {
      "Start": "2024-01-15T10:30:00.123Z",
      "End": "2024-01-15T10:30:00.456Z",
      "ExitCode": 0,
      "Output": "..."
    }
  ]
}
```

Health Check Examples for Different Applications

```dockerfile
# Node.js application
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1

# PostgreSQL database
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD pg_isready -U postgres || exit 1

# Redis
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD redis-cli ping || exit 1

# Simple check if a process is running
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD pgrep -x my-process || exit 1
```

Monitoring Docker Events

The `docker events` command shows you a real-time stream of everything happening in Docker — containers starting, stopping, dying, networks being created, volumes being mounted, and more.

Watching Events

```bash
docker events
```

Now open a second terminal and run:

```bash
docker run --name test-events nginx
```

Back in your first terminal, you will see events like:

```
2024-01-15T10:30:00.000 container create a1b2c3... (image=nginx, name=test-events)
2024-01-15T10:30:00.100 container attach a1b2c3... (image=nginx, name=test-events)
2024-01-15T10:30:00.200 network connect a1b2c3... (name=bridge, container=test-events)
2024-01-15T10:30:00.300 container start a1b2c3... (image=nginx, name=test-events)
```

Each event tells you:

- When it happened (timestamp)
- What happened (create, attach, connect, start)
- Which container was involved
- Additional details (image name, network name)

Filtering Events

You can filter events to show only what you care about:

```bash
# Only container events
docker events --filter type=container

# Only events for a specific container
docker events --filter container=my-web

# Only specific event types
docker events --filter event=start
docker events --filter event=die

# Combine filters
docker events --filter type=container --filter event=die
```

Viewing Past Events

By default, `docker events` shows events in real time. You can also look at past events:

```bash
# Events from the last 10 minutes
docker events --since 10m
```

```
# Events from a specific time
docker events --since "2024-01-15T10:00:00"
```

Press `Ctrl+C` to stop watching events.

Common Issues and How to Fix Them

Here are the most frequent problems you will encounter with Docker containers and how to diagnose and fix each one.

Issue 1: Port Conflict

Symptom: You get an error when starting a container with port mapping.

```
Error response from daemon: driver failed programming external connectivity:
Bind for 0.0.0.0:8080 failed: port is already allocated
```

Cause: Another container or application is already using that port on your host.

Diagnosis:

```bash
# Find what is using the port
# On macOS/Linux:
lsof -i :8080

# Or check which Docker container is using it:
docker ps --format "table {{.Names}}\t{{.Ports}}" | grep 8080
```

Fix: Either stop the other container or use a different host port:

```bash
# Stop the conflicting container
docker rm -f other-container

# Or use a different port
docker run -d -p 8081:80 my-app
```

Issue 2: Container Exits Immediately

Symptom: You start a container and it exits right away.

```bash
docker run -d --name my-app my-image
docker ps
# Container is not listed (not running)

docker ps -a
# Container shows "Exited (1) 2 seconds ago"
```

Diagnosis:

```bash
# Check the exit code
docker inspect --format '{{.State.ExitCode}}' my-app

# Read the logs for error messages
docker logs my-app
```

Common exit codes:

```
┌──────────────┬──────────────────────────────┐
│  Exit Code   │  Meaning                     │
├──────────────┼──────────────────────────────┤
│  0           │  Normal exit (application finished) │
│  1           │  General error               │
│  2           │  Misuse of command           │
│  126         │  Command not executable      │
│  127         │  Command not found           │
│  137         │  Killed (OOM or docker kill) │
│  139         │  Segmentation fault          │
│  143         │  Terminated (docker stop)    │
└──────────────┴──────────────────────────────┘
```

Fix: Read the logs to understand the error. Common causes include:

- Wrong command in the Dockerfile `CMD`
- Missing environment variables
- Missing files or dependencies
- Wrong working directory

Issue 3: Missing Files Inside the Container

Symptom: Your application cannot find a file it needs.

```
Error: ENOENT: no such file or directory, open '/app/config.json'
```

Diagnosis:

```bash
# Go inside the container and check
docker exec -it my-app sh

# List the files in the app directory
ls -la /app/

# Check the current working directory
pwd
```

Fix:

- Make sure the `COPY` instruction in your Dockerfile includes the file.
- Check that the `WORKDIR` matches where your application expects files.
- Verify that the `.dockerignore` file is not excluding the file.

Issue 4: Permission Denied

Symptom: Your application cannot read or write a file.

```
Error: EACCES: permission denied, open '/app/data/output.txt'
```

Diagnosis:

```bash
# Check file permissions
docker exec my-app ls -la /app/data/

# Check which user the container runs as
docker exec my-app whoami
docker exec my-app id
```

Fix: In your Dockerfile, make sure the files are owned by the right user:

```dockerfile
# Option 1: Change ownership
COPY --chown=node:node . /app

# Option 2: Change permissions
RUN chmod -R 755 /app/data
```

Issue 5: OOM Killed (Out of Memory)

Symptom: The container suddenly stops and the exit code is 137.

Diagnosis:

```bash
# Check the exit code
docker inspect --format '{{.State.ExitCode}}' my-app
# Returns: 137

# Check if it was killed due to memory
docker inspect --format '{{.State.OOMKilled}}' my-app
# Returns: true
```

Fix: Either increase the memory limit or fix the memory leak in your application:

```bash
# Run with more memory (512MB limit)
docker run -d --memory=512m my-app

# Run with no memory limit (not recommended for production)
docker run -d my-app
```

Issue 6: Cannot Connect to Database

Symptom: Your application cannot connect to the database container.

```
Error: connect ECONNREFUSED 127.0.0.1:5432
```

Cause: The application is trying to connect to `localhost` or `127.0.0.1`, which refers to the application container itself, not the database container.

Fix:

1. Make sure both containers are on the same Docker network.
2. Use the database container's name as the hostname, not `localhost`.

```bash
# Wrong: connecting to localhost
DATABASE_HOST=localhost

# Right: connecting to the container name
DATABASE_HOST=my-postgres
```

Systematic Debugging Flowchart

When something goes wrong, follow this flowchart. It will guide you through the most effective debugging steps in the right order.

```
                    ┌──────────────────┐
                    │  Container Problem?  │
                    └──────────────────┘
                              │
                              ▼
                    ┌──────────────────┐
                    │ Is the container      │
                    │ running?              │
                    │ docker ps             │
                    └──────────────────┘
                         │         │
                  Yes    │         │  No
                         │         │
                         ▼         ▼
            ┌──────────┐  ┌──────────────────┐
            │ Check logs   │  │ Check exited         │
            │ docker logs  │  │ container            │
            │ my-container │  │ docker ps -a         │
            └──────────┘  └──────────────────┘
                  │                  │
                  │                  ▼
                  │          ┌──────────────────┐
                  │          │ Read the logs        │
                  │          │ docker logs          │
                  │          │ my-container         │
                  │          └──────────────────┘
                  │                  │
                  ▼                  ▼
            ┌──────────┐  ┌──────────────────┐
            │ Can you      │  │ Check exit code      │
            │ reach the    │  │ docker inspect       │
            │ service?     │  │ --format             │
            │ curl/wget    │  │ '{{.State.ExitCode}}' │
            └──────────┘  └──────────────────┘
                  │  │                  │
            Yes   │  │ No               ▼
                  │  │          ┌──────────────────┐
                  │  │          │ Fix the issue:       │
                  │  │          │ - Code error (1)     │
                  ▼  │          │ - Missing cmd (127)  │
            ┌──────┐ │          │ - OOM killed (137)   │
            │ App works │ │          │ - Permissions (126)  │
            │ Check for │ │          └──────────────────┘
            │ specific  │ │
            │ errors in │ │
            │ logs      │ │
            └──────┘ │
                     ▼
            ┌──────────────────┐
            │ Check:               │
            │ 1. Port mapping (-p) │
            │ 2. Network settings  │
            │ 3. Firewall rules    │
            │ 4. App listening on  │
            │    0.0.0.0 not       │
            │    127.0.0.1         │
            └──────────────────┘
```

Step-by-Step Debugging Process

Follow these steps in order when you encounter a problem:

Step 1: Is the container running?

```bash
docker ps
```

If your container is not in the list, it is not running. Check stopped containers:

```bash
docker ps -a
```

Step 2: Read the logs

```bash
docker logs my-container
docker logs --tail 50 my-container
```

Most problems will be explained in the logs. Read the error messages carefully.

Step 3: Check the exit code (if the container stopped)

```bash
docker inspect --format '{{.State.ExitCode}}' my-container
```

The exit code tells you what kind of failure occurred.

Step 4: Go inside and investigate

```bash
docker exec -it my-container sh
```

Check files, permissions, environment variables, and network connectivity from inside the container.

Step 5: Check network settings

```bash
# What network is the container on?
docker inspect --format '{{json .NetworkSettings.Networks}}' my-container

# Can the container reach other containers?
docker exec my-container ping -c 3 other-container
```

Step 6: Check resource usage

```bash
docker stats my-container
```

This shows real-time CPU, memory, and network usage. If memory usage is near the limit, the container might get OOM killed.

Step 7: Watch events

```bash
docker events --filter container=my-container
```

This shows what Docker is doing with your container in real time.

Common Mistakes

Mistake 1: Not Reading the Logs

Many beginners skip `docker logs` and go straight to trying random fixes. Always read the logs first. The answer is usually right there.

Mistake 2: Using docker exec on a Stopped Container

```bash
# This will fail — the container must be running
docker exec -it my-stopped-container sh
```

Error:

```
Error response from daemon: Container ... is not running
```

If the container is stopped, you cannot exec into it. Instead, start it in a way that keeps it running:

```bash
# Start with an interactive shell to investigate
docker run -it my-image sh
```

Mistake 3: Forgetting -it Flags

```bash
# This will not work interactively
docker exec my-container bash
```

Without `-it`, you cannot type commands. Always use `-it` for interactive sessions.

Mistake 4: Ignoring Health Check Failures

If `docker ps` shows `(unhealthy)`, do not ignore it. Check the health check logs:

```bash
docker inspect --format '{{json .State.Health}}' my-container
```

Mistake 5: Looking at the Wrong Container

When running multiple containers, make sure you are reading logs from the correct one. Use `docker ps` to list all running containers and their names.

Best Practices

1. Always check logs first. The `docker logs` command should be your first step when debugging.
2. Add health checks to every production Dockerfile. They help orchestrators (like Docker Compose and Kubernetes) know when your application is truly ready.
3. Use `--tail` and `--since` to filter logs. Do not scroll through thousands of lines when you only need the recent ones.
4. Write application logs to stdout/stderr. This ensures `docker logs` captures them. Avoid writing to log files inside the container.
5. Use meaningful container names. `--name my-postgres` is much easier to debug than a random container ID.
6. Monitor with `docker stats` to keep an eye on resource usage before problems occur.
7. Set restart policies for production containers so they recover from crashes automatically:

```bash
docker run -d --restart unless-stopped my-app
```

8. Keep health check intervals reasonable. Checking every second is too aggressive. Every 30 seconds is usually fine.

Quick Summary

Docker provides powerful tools for debugging containers. Use `docker logs` to read application output, with `--tail`, `--since`, and `-f` to filter and follow logs. Use `docker exec -it` to get inside running containers and investigate. Use `docker inspect` to extract specific details like IP addresses, mounts, and environment variables. Add `HEALTHCHECK` instructions to your Dockerfiles so Docker can monitor your application's actual health. Use `docker events` to see what Docker is doing in real time. When problems occur, follow the debugging flowchart: check if the container is running, read the logs, check the exit code, investigate from inside, check network and resources.

Key Points

- `docker logs` shows stdout and stderr output from a container.
- `--tail N` shows the last N lines, `--since` filters by time, `-f` follows in real time.
- `docker exec -it container sh` opens a shell inside a running container.
- `docker inspect` returns detailed JSON about a container; use `--format` to extract specific fields.
- `HEALTHCHECK` in a Dockerfile lets Docker monitor if your application is truly working.
- Health check states are: starting, healthy, and unhealthy.
- `docker events` shows a real-time stream of all Docker activity.
- Common issues include port conflicts, immediate exits, missing files, permission denied, and OOM kills.
- Exit code 137 usually means the container was killed due to memory limits.
- Always check logs first when debugging; the answer is usually there.

Practice Questions

1. Your container shows "Exited (137)" in `docker ps -a`. What does exit code 137 mean, and how would you investigate further?
2. What is the difference between `docker logs my-container` and `docker logs -f my-container`? When would you use each one?

3. Why does Docker recommend that applications write logs to stdout/stderr instead of log files? What happens if an application writes to a log file inside the container?
4. Explain what each part of this HEALTHCHECK instruction does:

```dockerfile
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1
```

5. You start a container and it exits immediately. Walk through the steps you would take to diagnose the problem.

Exercises

Exercise 1: Log Investigation

1. Run an nginx container named `log-test` with port 8080 mapped to port 80.
2. Visit `http://localhost:8080` in your browser 5 times.
3. View only the last 3 log lines.
4. View logs from the last 2 minutes.
5. Follow the logs in real time and visit the page again to see new entries appear.
6. Clean up.

Exercise 2: Container Inspection

1. Run a PostgreSQL container named `inspect-test` with environment variables for user, password, and database.
2. Use `docker inspect` with `--format` to find the container's IP address.
3. Use `docker inspect` to find the environment variables.
4. Use `docker exec` to go inside the container and verify the database was created.
5. Clean up.

Exercise 3: Health Check Debugging

1. Create a Dockerfile based on nginx that includes a HEALTHCHECK which curls localhost.
2. Build and run the container.
3. Wait 30 seconds and check the health status using `docker ps` and `docker inspect`.

4. Now create another Dockerfile with a health check that will fail (for example, curl a port that is not listening).
5. Build and run it, then observe the health status change to unhealthy.
6. Check the health check log using `docker inspect`.
7. Clean up.

What Is Next?

You can now debug individual containers like a pro. But real applications have multiple containers — a web server, an API, a database, maybe a cache. Managing all of these separately with `docker run` commands is tedious and error-prone. In the next chapter, you will learn about Docker Compose, a tool that lets you define and run multi-container applications with a single file and a single command.

Chapter 14: Introduction to Docker Compose

What You Will Learn

By the end of this chapter, you will be able to:

- Explain why Docker Compose exists and what problem it solves
- Understand the structure of a `docker-compose.yml` file
- Define services, ports, and environment variables in Compose
- Start and stop multi-container applications with a single command
- Use `docker compose up`, `down`, `ps`, `logs`, and `exec`
- Compare Compose commands to their individual Docker equivalents

Why This Chapter Matters

In previous chapters, you learned how to run individual containers with `docker run`. That works fine when you have one or two containers. But real applications usually have more:

- A web server
- An API server
- A database
- Maybe a cache like Redis
- Maybe a message queue

Running each of these with separate `docker run` commands means remembering long commands, creating networks manually, making sure containers start in the right order, and typing the same thing every time you restart your application. This gets tedious fast.

Think of it like cooking a meal. If you have one dish, you can remember the recipe. But if you are preparing a five-course dinner, you need a written menu with instructions for every dish. Docker Compose is that menu — a single file that describes your entire application, and a single command to bring it all to life.

What Is Docker Compose?

Docker Compose is a tool for defining and running multi-container Docker applications. Instead of typing multiple `docker run` commands, you describe your entire application in a YAML file called `docker-compose.yml`, and then start everything with one command: `docker compose up`.

The Problem Without Compose

Let us say you have a web application that needs nginx and PostgreSQL. Without Compose, you would type:

```bash
# Step 1: Create a network
docker network create my-app-network

# Step 2: Start the database
docker run -d \
  --name my-postgres \
  --network my-app-network \
  -e POSTGRES_USER=myuser \
  -e POSTGRES_PASSWORD=mypassword \
  -e POSTGRES_DB=myapp \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

# Step 3: Start the web server
docker run -d \
  --name my-web \
  --network my-app-network \
  -p 8080:80 \
  nginx
```

That is three commands with many flags. And when you want to stop everything:

```bash
docker rm -f my-web my-postgres
docker network rm my-app-network
```

Now imagine doing this every day. Or imagine adding three more services. It becomes unmanageable.

The Solution with Compose

The same setup in a `docker-compose.yml` file:

```yaml
services:
  web:
    image: nginx
    ports:
      - "8080:80"

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: myapp
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

And to start everything:

```bash
docker compose up -d
```

That is it. One file, one command. Docker Compose creates the network automatically, starts both containers, and connects them. To stop everything:

```bash
docker compose down
```

Understanding the docker-compose.yml File

The `docker-compose.yml` file is written in YAML (Yet Another Markup Language). YAML is a human-readable format for configuration files. If you have never seen YAML before, here are the basics:

- YAML uses indentation (spaces, not tabs) to show structure.

- A key-value pair looks like `key: value`.
- A list uses dashes: `- item`.
- Comments start with `#`.

The Structure

Every `docker-compose.yml` file follows this structure:

```yaml
services:
  service-name-1:
    # Configuration for service 1
  service-name-2:
    # Configuration for service 2

volumes:
  # Named volumes used by services

networks:
  # Custom networks (optional)
```

Let us break down each section.

The services Section

The `services` section is the heart of the file. Each service represents one container. The service name is what you choose — it becomes the container's hostname on the Docker network.

```yaml
services:
  web:
    image: nginx
    ports:
      - "8080:80"

  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://myuser:mypassword@database:5432/myapp

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: myapp
```

This defines three services:

1. web: Uses the official `nginx` image and maps port 8080 to port 80.
2. api: Builds from a Dockerfile in the `./api` directory and passes an environment variable.
3. database: Uses the official `postgres:16` image with configuration via environment variables.

image vs. build

There are two ways to specify what a service runs:

Using `image`: Pull a pre-built image from a registry (like Docker Hub):

```yaml
services:
  web:
    image: nginx:latest
```

Using `build`: Build an image from a Dockerfile in your project:

```yaml
services:
  api:
    build: ./api
```

This tells Compose to look for a Dockerfile in the `./api` directory and build the image.

You can also specify more build options:

```yaml
services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile.dev
```

- `context`: The directory containing the Dockerfile and files to copy.
- `dockerfile`: The name of the Dockerfile (if it is not the default `Dockerfile`).

ports

The `ports` section maps host ports to container ports, just like `-p` in `docker run`:

```yaml
services:
  web:
```

```
    image: nginx
    ports:
      - "8080:80"        # Host port 8080 -> Container port 80
      - "8443:443"       # Host port 8443 -> Container port 443
```

Always use quotes around port mappings in YAML. Without quotes, YAML might interpret `80:80` as a base-60 number (a quirk of the YAML specification).

environment

The `environment` section sets environment variables inside the container, just like `-e` in `docker run`:

```yaml
services:
  database:
    image: postgres:16
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: myapp
```

You can also write it as a list:

```yaml
services:
  database:
    image: postgres:16
    environment:
      - POSTGRES_USER=myuser
      - POSTGRES_PASSWORD=mypassword
      - POSTGRES_DB=myapp
```

Both formats are equivalent. Choose the one you find more readable.

volumes

The `volumes` section mounts storage, just like `-v` in `docker run`:

```yaml
services:
  database:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data    # Named volume
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql  # Bind mount
```

```
volumes:
  pgdata:    # Declare the named volume
```

Named volumes (like `pgdata`) must be declared in a top-level `volumes` section at the bottom of the file.

A Complete Example

Here is a complete `docker-compose.yml` with all the pieces together:

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - api

  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://myuser:mypassword@database:5432/myapp
      REDIS_URL: redis://cache:6379
    depends_on:
      - database
      - cache

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: myapp
    volumes:
      - pgdata:/var/lib/postgresql/data

  cache:
    image: redis:7

volumes:
  pgdata:
```

```
┌──────────────────────────────────────────────────────────┐
│              docker-compose.yml                          │
│                                                          │
```

```
│    ┌─────────┐     ┌─────────┐     ┌──────────┐        │
│    │  web    │──>  │  api    │──>  │ database │        │
│    │ :8080   │     │ :3000   │     │ postgres │        │
│    │ nginx   │     │ node    │     │ :5432    │        │
│    └─────────┘     └─────────┘     └──────────┘        │
│                        │                               │
│                        │                               │
│                        └──────────>┌──────────┐        │
│                                    │  cache   │        │
│                                    │  redis   │        │
│                                    │  :6379   │        │
│                                    └──────────┘        │
│                                                        │
│   All services share one auto-created network          │
└────────────────────────────────────────────────────────┘
```

Essential Compose Commands

Now that you understand the file, let us learn the commands to work with it.

docker compose up — Start Everything

```bash
docker compose up -d
```

This command:

1. Reads `docker-compose.yml` in the current directory.
2. Creates a network for the project (if one does not exist).
3. Builds images (for services with `build`).
4. Creates and starts all containers.
5. The `-d` flag runs everything in the background (detached mode).

Output:

```
[+] Running 4/4
 ✔ Network myapp_default     Created
 ✔ Container myapp-cache-1    Started
 ✔ Container myapp-database-1 Started
 ✔ Container myapp-api-1      Started
 ✔ Container myapp-web-1      Started
```

Notice that Docker Compose names things with a prefix. The prefix is the name of the directory your `docker-compose.yml` is in. If your project is in a folder called `myapp`, the containers are named `myapp-web-1`, `myapp-database-1`, and so on.

Without `-d`, Compose runs in the foreground and you see logs from all services in your terminal. Press `Ctrl+C` to stop everything.

```bash
# Foreground mode (logs stream to terminal)
docker compose up

# Background mode (runs silently)
docker compose up -d
```

docker compose down — Stop Everything

```bash
docker compose down
```

This command:

1. Stops all containers defined in the Compose file.
2. Removes the containers.
3. Removes the network.
4. Does not remove volumes (your data is safe).

Output:

```
[+] Running 5/5
 ✔ Container myapp-web-1      Removed
 ✔ Container myapp-api-1      Removed
 ✔ Container myapp-cache-1    Removed
 ✔ Container myapp-database-1 Removed
 ✔ Network myapp_default      Removed
```

To also remove volumes (delete all data):

```bash
docker compose down -v
```

The `-v` flag removes named volumes. Use this carefully — it deletes your database data.

docker compose ps — List Running Services

```bash
docker compose ps
```

Output:

```
NAME                 IMAGE        COMMAND                  SERVICE    STATUS    PORTS
myapp-api-1          myapp-api    "node server.js"         api        running   0.0.0.0...
myapp-cache-1        redis:7      "docker-entrypoint..."   cache      running   6379/tcp
myapp-database-1     postgres:16  "docker-entrypoint..."   database   running   5432/tcp
myapp-web-1          nginx:latest "/docker-entrypoint..."  web        running   0.0.0.0...
```

This shows all services, their status, and port mappings.

docker compose logs — View Logs

```bash
# Logs from all services
docker compose logs

# Logs from a specific service
docker compose logs api

# Follow logs in real time
docker compose logs -f

# Follow logs for a specific service
docker compose logs -f api

# Last 20 lines from all services
docker compose logs --tail 20

# Last 10 lines from a specific service, following
docker compose logs -f --tail 10 database
```

Compose logs are color-coded by service, making it easy to tell which service produced which line:

```
myapp-api-1       | Server started on port 3000
myapp-database-1  | PostgreSQL init process complete
myapp-cache-1     | Ready to accept connections
myapp-web-1       | nginx started
```

docker compose exec — Run Commands in a Service

```bash
# Open a shell in the api service
docker compose exec api sh

# Run a single command in the database service
docker compose exec database psql -U myuser -d myapp

# Check the nginx configuration
docker compose exec web nginx -t
```

Notice the difference from `docker exec`:

- `docker exec` uses the container name: `docker exec -it myapp-api-1 sh`
- `docker compose exec` uses the service name: `docker compose exec api sh`

The service name is simpler and does not depend on the project prefix.

docker compose build — Build Images

```bash
# Build all services that have "build" defined
docker compose build

# Build a specific service
docker compose build api

# Build with no cache (fresh build)
docker compose build --no-cache
```

docker compose restart — Restart Services

```bash
# Restart all services
docker compose restart

# Restart a specific service
docker compose restart api
```

Compose vs. Individual Docker Commands

Here is a side-by-side comparison to show how much simpler Compose makes things:

Without Compose	With Compose

`docker network create mynet`
 	
`docker run -d`
`  --name db`
`  --network mynet`
`  -e POSTGRES_USER=user`
`  -e POSTGRES_PASSWORD=pass`
`  -v pgdata:/var/lib/...`
`  postgres:16`
 	
`docker run -d`
`  --name api`
`  --network mynet`
`  -p 3000:3000`
`  -e DATABASE_URL=...`
`  my-api`
	`(automatic)`
	
	`docker compose up -d`
	
	`(Everything defined in`
	`  docker-compose.yml)`

`docker logs db`
`docker logs api`
	`docker compose logs database`
	`docker compose logs api`
	`docker compose logs (all)`

`docker exec -it db psql ...`	`docker compose exec database psql`
`docker rm -f api db` `docker network rm mynet`	`docker compose down`
`(repeat all commands` `  to start again)`	`docker compose up -d` `(one command restarts everything)`

The key advantages of Compose:

1. Declarative: You describe what you want, not how to do it.
2. Repeatable: The same file produces the same result every time.
3. Shareable: You can share the file with your team, and everyone gets the same setup.
4. Version controlled: The file goes into Git, so changes are tracked.
5. One command: Start, stop, and manage everything with simple commands.

A Hands-On Example

Let us build a simple example from scratch. We will run nginx with a custom HTML page and a PostgreSQL database.

Step 1: Create the Project Directory

```bash
mkdir ~/compose-demo
cd ~/compose-demo
```

Step 2: Create a Simple HTML Page

```bash
mkdir html
```

Create `html/index.html`:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Compose Demo</title>
</head>
<body>
    <h1>Hello from Docker Compose!</h1>
    <p>This page is served by nginx running in a Docker container.</p>
    <p>A PostgreSQL database is running in another container.</p>
</body>
</html>
```

Step 3: Create the docker-compose.yml

Create `docker-compose.yml` in the project root:

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo123
      POSTGRES_DB: demoapp
    volumes:
      - db-data:/var/lib/postgresql/data
```

```
volumes:
  db-data:
```

Let us explain every line:

- `services:` — Start of the services section.
- `web:` — A service named "web."
- `image: nginx:latest` — Use the official nginx image.
- `ports: - "8080:80"` — Map host port 8080 to container port 80.
- `volumes: - ./html:/usr/share/nginx/html:ro` — Mount the local `html` directory into the container. The `:ro` means read-only (the container cannot modify your files).
- `database:` — A service named "database."
- `image: postgres:16` — Use PostgreSQL 16.
- `environment:` — Set database configuration via environment variables.
- `volumes: - db-data:/var/lib/postgresql/data` — Persist database data in a named volume.
- `volumes: db-data:` — Declare the named volume at the top level.

Step 4: Start the Application

```bash
docker compose up -d
```

Output:

```
[+] Running 3/3
 ✔ Network compose-demo_default       Created
 ✔ Container compose-demo-database-1  Started
 ✔ Container compose-demo-web-1       Started
```

Step 5: Verify Everything Is Running

```bash
docker compose ps
```

Output:

```
NAME                        IMAGE         STATUS    PORTS
compose-demo-database-1     postgres:16   running   5432/tcp
compose-demo-web-1          nginx:latest  running   0.0.0.0:8080->80/tcp
```

Visit `http://localhost:8080` in your browser. You should see your HTML page.

Step 6: Connect to the Database

```bash
docker compose exec database psql -U demo -d demoapp
```

Output:

```
psql (16.1)
Type "help" for help.

demoapp=>
```

You are connected. Type `\q` to exit.

Step 7: View the Logs

```bash
docker compose logs
```

You will see logs from both services, color-coded.

Step 8: Stop Everything

```bash
docker compose down
```

Output:

```
[+] Running 3/3
 ✔ Container compose-demo-web-1       Removed
 ✔ Container compose-demo-database-1  Removed
 ✔ Network compose-demo_default       Removed
```

Everything is stopped and cleaned up. Your database data is preserved in the `db-data` volume and will be there when you run `docker compose up -d` again.

Common Mistakes

Mistake 1: Using Tabs Instead of Spaces in YAML

YAML does not allow tabs for indentation. Use spaces only. Most editors can be configured to insert spaces when you press the Tab key.

```yaml
# Wrong (uses a tab)
services:
	web:    # <-- This is a tab. YAML will reject this.

# Right (uses spaces)
services:
  web:     # <-- This uses 2 spaces. YAML accepts this.
```

Mistake 2: Wrong Indentation Level

YAML is very strict about indentation. Every level must be indented consistently.

```yaml
# Wrong: ports is at the wrong level
services:
  web:
    image: nginx
  ports:              # <-- Wrong! This should be under "web", not at the same level
    - "8080:80"

# Right
services:
  web:
    image: nginx
    ports:            # <-- Correct! Indented under "web"
      - "8080:80"
```

Mistake 3: Forgetting to Quote Port Numbers

```yaml
# Potentially wrong (YAML may misinterpret this)
ports:
  - 80:80

# Right (always quote port mappings)
```

```
ports:
  - "80:80"
```

Mistake 4: Forgetting the Top-Level volumes Section

```yaml
# Wrong: uses a named volume but does not declare it
services:
  database:
    volumes:
      - pgdata:/var/lib/postgresql/data
# Missing: volumes section at the bottom

# Right: declare the volume
services:
  database:
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Mistake 5: Running Compose from the Wrong Directory

Docker Compose looks for `docker-compose.yml` in the current directory. If you run `docker compose up` from the wrong directory, it will not find the file.

```bash
# Wrong: not in the project directory
cd ~
docker compose up
# Error: no configuration file provided

# Right: navigate to the project directory first
cd ~/myproject
docker compose up
```

Best Practices

1. Use meaningful service names. Name services by their role: `web`, `api`, `database`, `cache`, not `service1`, `service2`.

2. Always specify image versions. Use `postgres:16` instead of `postgres:latest`. This ensures everyone gets the same version.
3. Use named volumes for data you want to keep. Database data should always be in a named volume.
4. Use environment variables for configuration. Do not hardcode values that might change between environments.
5. Keep your `docker-compose.yml` in version control. Commit it to Git so your team can use it.
6. Use `docker compose down` instead of stopping containers individually. This ensures clean shutdown and network removal.
7. Organize your project with the Compose file at the root and subdirectories for each service.

```
my-project/
├── docker-compose.yml
├── api/
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── web/
│   └── index.html
└── database/
    └── init.sql
```

Quick Summary

Docker Compose lets you define multi-container applications in a single YAML file and manage them with simple commands. The `docker-compose.yml` file defines services (containers), their images or build instructions, ports, environment variables, and volumes. Use `docker compose up -d` to start everything, `docker compose down` to stop everything, `docker compose ps` to list services, `docker compose logs` to view logs, and `docker compose exec` to run commands inside services. Compose automatically creates a network for your services, so they can find each other by name.

Key Points

- Docker Compose replaces multiple `docker run` commands with a single YAML file.
- The `docker-compose.yml` file has three main sections: `services`, `volumes`, and `networks`.
- Each service can use `image` (pre-built) or `build` (from Dockerfile).
- Port mappings use the same `host:container` format as `docker run -p`.
- Compose automatically creates a default network for all services.
- `docker compose up -d` starts all services in the background.
- `docker compose down` stops and removes all services and networks.
- `docker compose logs` shows output from all services.
- `docker compose exec service-name command` runs commands inside a service.
- YAML uses spaces for indentation, never tabs.

Practice Questions

1. What problem does Docker Compose solve? Why would you use it instead of individual `docker run` commands?
2. What is the difference between `image` and `build` in a service definition? When would you use each one?
3. You run `docker compose up -d` and then make changes to your `docker-compose.yml`. What command do you run to apply the changes?
4. What does `docker compose down -v` do differently from `docker compose down`? When would you use the `-v` flag, and when would you avoid it?
5. In a `docker-compose.yml` file, the `database` service uses a named volume called `pgdata`. Where must `pgdata` be declared, and why?

Exercises

Exercise 1: Your First Compose File

1. Create a new directory called `exercise-compose`.
2. Write a `docker-compose.yml` that runs nginx with port 9090 mapped to port 80.
3. Start the service with `docker compose up -d`.
4. Verify it is running with `docker compose ps`.
5. Visit `http://localhost:9090` in your browser.

6. View the logs with `docker compose logs`.
7. Stop everything with `docker compose down`.

Exercise 2: Two Services

1. Create a `docker-compose.yml` with two services: nginx (web) and Redis (cache).
2. Map nginx to port 8080.
3. Start both services.
4. Verify both are running.
5. Use `docker compose exec` to connect to Redis and run `redis-cli ping`.
6. View logs from only the cache service.
7. Stop everything.

Exercise 3: Build and Run

1. Create a simple Node.js application (a file called `server.js` that responds with "Hello from Compose").
2. Create a Dockerfile for it.
3. Write a `docker-compose.yml` that uses `build: .` to build and run your app.
4. Start the application with `docker compose up -d`.
5. Test it by visiting the appropriate URL.
6. Make a change to `server.js`, rebuild with `docker compose build`, and restart.
7. Stop everything.

What Is Next?

You now know how to define multi-container applications with Docker Compose. In the next chapter, you will put this knowledge to work by building a real application with three services: a Node.js API, a PostgreSQL database, and a Redis cache. You will see how to wire everything together, handle service dependencies, and test the full stack.

Chapter 15: Building Multi-Container Applications

What You Will Learn

By the end of this chapter, you will be able to:

- Build a real multi-container application from scratch
- Create a Node.js API that connects to PostgreSQL and Redis
- Write a complete `docker-compose.yml` for a three-service stack
- Use `depends_on` with health checks to control startup order
- Understand the wait-for-it pattern for database readiness
- Connect services through shared Docker networks
- Test and debug a full multi-container application
- View and analyze logs from all services together

Why This Chapter Matters

In the previous chapter, you learned the basics of Docker Compose. Now it is time to build something real. This chapter walks you through creating a complete application with three services working together — the kind of architecture you will encounter in professional software development.

Think of this chapter as building your first real house. The previous chapter taught you what a blueprint is and how to read it. Now you are actually going to lay the foundation, put up the walls, install the plumbing, and wire the electricity. By the end, you will have a working building.

The application we will build is a simple task management API. Users can create, read, and delete tasks. The tasks are stored in PostgreSQL, and recently accessed tasks are cached in Redis for faster retrieval.

The Architecture

Our application has three services:

```
┌──────────────────────────────────────────────────────────┐
│                     Docker Compose                       │
│                                                          │
│   ┌──────────────┐                                       │
│   │   api        │  Node.js + Express                    │
│   │   Port 3000  │  Handles HTTP requests                │
│   └──────────────┘                                       │
│          │   │                                           │
│          │   │                                           │
│   ┌──────▼───┴──┐     ┌──────────────┐                  │
│   │   database  │     │   cache      │                  │
│   │   Port 5432 │     │   Port 6379  │                  │
│   │   PostgreSQL│     │   Redis      │                  │
│   │             │     │              │                  │
│   │   Stores    │     │   Caches     │                  │
│   │   tasks     │     │   recent     │                  │
│   │   permanently│    │   tasks      │                  │
│   └──────────────┘     └──────────────┘                  │
│                                                          │
│   All services on the same network                       │
│   API connects to "database" and "cache" by name         │
└──────────────────────────────────────────────────────────┘
```

- api: A Node.js application that exposes REST endpoints for managing tasks.
- database: A PostgreSQL database that stores tasks permanently.
- cache: A Redis instance that caches recently accessed tasks for speed.

Setting Up the Project

Step 1: Create the Project Structure

```bash
mkdir ~/task-manager
cd ~/task-manager
mkdir api
```

Your project will look like this:

```
task-manager/
├── docker-compose.yml
├── api/
│   ├── Dockerfile
│   ├── package.json
│   ├── server.js
│   └── wait-for-it.sh
└── database/
    └── init.sql
```

Step 2: Create the Node.js API

Create `api/package.json`:

```json
{
  "name": "task-manager-api",
  "version": "1.0.0",
  "description": "A simple task manager API",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.3",
    "redis": "^4.6.10"
  }
}
```

Let us explain each dependency:

- express: A popular web framework for Node.js. It makes it easy to create API endpoints.
- pg: A PostgreSQL client for Node.js. It lets us connect to and query the database.
- redis: A Redis client for Node.js. It lets us store and retrieve cached data.

Create `api/server.js`:

```javascript
const express = require('express');
const { Pool } = require('pg');
const { createClient } = require('redis');

// Create the Express application
const app = express();
app.use(express.json());

// PostgreSQL connection
// "database" is the service name from docker-compose.yml
// Docker DNS resolves "database" to the container's IP address
const pool = new Pool({
  host: process.env.DB_HOST || 'database',
  port: process.env.DB_PORT || 5432,
  user: process.env.DB_USER || 'taskuser',
  password: process.env.DB_PASSWORD || 'taskpass',
  database: process.env.DB_NAME || 'taskdb',
});

// Redis connection
// "cache" is the service name from docker-compose.yml
const redisClient = createClient({
  url: process.env.REDIS_URL || 'redis://cache:6379',
});

// Connect to Redis when the app starts
redisClient.on('error', (err) => console.error('Redis error:', err));
redisClient.connect().then(() => {
  console.log('Connected to Redis');
});

// Health check endpoint
// This tells Docker (and other tools) that the API is working
app.get('/health', async (req, res) => {
  try {
    // Check database connection
    await pool.query('SELECT 1');
    // Check Redis connection
    await redisClient.ping();
    res.json({ status: 'healthy', database: 'connected', cache: 'connected' });
  } catch (error) {
    res.status(500).json({ status: 'unhealthy', error: error.message });
  }
});

// GET /tasks - Retrieve all tasks
app.get('/tasks', async (req, res) => {
  try {
    // First, check if tasks are cached in Redis
    const cachedTasks = await redisClient.get('all_tasks');
    if (cachedTasks) {
      console.log('Returning tasks from cache');
      return res.json(JSON.parse(cachedTasks));
    }
```

```
    // If not cached, query the database
    console.log('Querying tasks from database');
    const result = await pool.query(
      'SELECT * FROM tasks ORDER BY created_at DESC'
    );

    // Cache the result in Redis for 60 seconds
    await redisClient.setEx('all_tasks', 60, JSON.stringify(result.rows));

    res.json(result.rows);
  } catch (error) {
    console.error('Error fetching tasks:', error.message);
    res.status(500).json({ error: error.message });
  }
});

// POST /tasks - Create a new task
app.post('/tasks', async (req, res) => {
  try {
    const { title, description } = req.body;

    if (!title) {
      return res.status(400).json({ error: 'Title is required' });
    }

    const result = await pool.query(
      'INSERT INTO tasks (title, description) VALUES ($1, $2) RETURNING *',
      [title, description || '']
    );

    // Clear the cache because the data has changed
    await redisClient.del('all_tasks');

    console.log('Created task:', result.rows[0].id);
    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Error creating task:', error.message);
    res.status(500).json({ error: error.message });
  }
});

// DELETE /tasks/:id - Delete a task
app.delete('/tasks/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const result = await pool.query(
      'DELETE FROM tasks WHERE id = $1 RETURNING *',
      [id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Task not found' });
    }
```

```
    // Clear the cache because the data has changed
    await redisClient.del('all_tasks');

    console.log('Deleted task:', id);
    res.json({ message: 'Task deleted', task: result.rows[0] });
  } catch (error) {
    console.error('Error deleting task:', error.message);
    res.status(500).json({ error: error.message });
  }
});

// Start the server
const PORT = process.env.PORT || 3000;
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Task Manager API running on port ${PORT}`);
});
```

Let us walk through the key parts of this code:

1. Database connection: We connect to PostgreSQL using the hostname `database`. This is the service name in our `docker-compose.yml`. Docker's DNS automatically resolves this to the database container's IP address.
2. Redis connection: We connect to Redis using `redis://cache:6379`. Again, `cache` is the service name.
3. Health check endpoint (`/health`): This tests both the database and Redis connections. Docker will use this to know if the API is truly healthy.
4. GET /tasks: First checks Redis for cached results. If found, returns them immediately (fast). If not found, queries PostgreSQL and caches the result for 60 seconds.
5. POST /tasks: Creates a new task in PostgreSQL and clears the Redis cache so the next GET returns fresh data.
6. DELETE /tasks/:id: Deletes a task from PostgreSQL and clears the cache.

Step 3: Create the Dockerfile for the API

Create `api/Dockerfile`:

```dockerfile
FROM node:20-alpine

# Set the working directory
WORKDIR /app

# Copy package files first (for better caching)
COPY package.json ./

# Install dependencies
```

```
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose the port the app runs on
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

# Start the application
CMD ["npm", "start"]
```

We use `wget` instead of `curl` for the health check because Alpine Linux includes `wget` by default but not `curl`.

Step 4: Create the Database Initialization Script

Create a directory and file:

```bash
mkdir database
```

Create `database/init.sql`:

```sql
-- This script runs automatically when the PostgreSQL container
-- starts for the first time. It creates the tasks table.

CREATE TABLE IF NOT EXISTS tasks (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT DEFAULT '',
    completed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert some sample data so the API has something to show
INSERT INTO tasks (title, description) VALUES
    ('Learn Docker basics', 'Understand images, containers, and volumes'),
    ('Learn Docker Compose', 'Build multi-container applications'),
    ('Deploy to production', 'Use Docker in a real project');
```

This SQL script runs only when the PostgreSQL container starts for the very first time (when the data volume is empty). If the volume already has data, the script is skipped.

The Complete docker-compose.yml

Now for the main event. Create `docker-compose.yml` in the project root:

```yaml
services:
  # ─────────────────────────────────────────────────────
  # API Service — Node.js application
  # ─────────────────────────────────────────────────────
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DB_HOST: database
      DB_PORT: 5432
      DB_USER: taskuser
      DB_PASSWORD: taskpass
      DB_NAME: taskdb
      REDIS_URL: redis://cache:6379
    depends_on:
      database:
        condition: service_healthy
      cache:
        condition: service_healthy
    networks:
      - app-network

  # ─────────────────────────────────────────────────────
  # Database Service — PostgreSQL
  # ─────────────────────────────────────────────────────
  database:
    image: postgres:16
    environment:
      POSTGRES_USER: taskuser
      POSTGRES_PASSWORD: taskpass
      POSTGRES_DB: taskdb
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskuser -d taskdb"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    networks:
      - app-network

  # ─────────────────────────────────────────────────────
```

```
  # Cache Service — Redis
  # ──────────────────────────────────────────────────
  cache:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

# Named volumes for persistent data
volumes:
  db-data:

# Custom network
networks:
  app-network:
    driver: bridge
```

Let us go through each service in detail.

The api Service Explained

```yaml
api:
  build: ./api
```

Build the Docker image from the `./api` directory, using the Dockerfile we created there.

```yaml
  ports:
    - "3000:3000"
```

Map host port 3000 to container port 3000. This lets us access the API from our browser or tools like curl.

```yaml
  environment:
    DB_HOST: database
    DB_PORT: 5432
    DB_USER: taskuser
    DB_PASSWORD: taskpass
    DB_NAME: taskdb
    REDIS_URL: redis://cache:6379
```

These environment variables tell the API how to connect to the database and cache. Notice that `DB_HOST` is set to `database` — the service name. And `REDIS_URL` uses `cache` — another

service name. Docker's DNS handles the rest.

```yaml
  depends_on:
    database:
      condition: service_healthy
    cache:
      condition: service_healthy
```

This is crucial. The `depends_on` section tells Docker Compose that the API should not start until both the database and cache are healthy. Without this, the API might start before the database is ready, causing connection errors.

The database Service Explained

```yaml
database:
  image: postgres:16
```

Use the official PostgreSQL 16 image.

```yaml
  volumes:
    - db-data:/var/lib/postgresql/data
    - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
```

Two volume mounts:

1. `db-data:/var/lib/postgresql/data` — A named volume that persists the database data. Without this, all data would be lost when the container stops.
2. `./database/init.sql:/docker-entrypoint-initdb.d/init.sql` — A bind mount that copies our initialization script into the container. PostgreSQL automatically runs any SQL file in `/docker-entrypoint-initdb.d/` when it starts for the first time.

```yaml
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U taskuser -d taskdb"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
```

The health check uses `pg_isready`, a built-in PostgreSQL utility that checks if the database is accepting connections. Docker will run this command every 10 seconds. After the database reports healthy, the API service (which depends on it) can start.

The cache Service Explained

```yaml
cache:
  image: redis:7-alpine
```

Use Redis 7 on Alpine Linux (a minimal, small image).

```yaml
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
```

The health check uses `redis-cli ping`, which returns `PONG` if Redis is working. This is the standard way to check Redis health.

Understanding depends_on and Health Checks

The `depends_on` section controls startup order. Without it, Docker Compose starts all services simultaneously. This is a problem because:

1. The database needs several seconds to initialize before it accepts connections.
2. The API tries to connect to the database immediately when it starts.
3. If the database is not ready, the API crashes.

```
┌──────────── Without depends_on ────────────┐
│                                            │
│   Time 0s:  database starts initializing   │
│   Time 0s:  api starts and tries to connect │
│   Time 0s:  api CRASHES (database not ready)│
│   Time 3s:  database becomes ready         │
│                                            │
│   Result: API crashed before DB was ready  │
└────────────────────────────────────────────┘

┌──────────── With depends_on + health ──────┐
│                                            │
│   Time 0s:  database starts initializing   │
│   Time 0s:  cache starts initializing      │
│   Time 2s:  cache is healthy               │
│   Time 3s:  database is healthy            │
│   Time 3s:  api starts (both deps healthy) │
```

```
│   Time 3s:  api connects successfully       │
│                                             │
│   Result: Everything works!                 │
└─────────────────────────────────────────────┘
```

depends_on Without Health Checks

A simple `depends_on` only waits for the container to start, not for the application inside to be ready:

```yaml
# This only waits for the container to start, NOT for the database to be ready
depends_on:
  - database
```

depends_on With Health Checks (Recommended)

Adding `condition: service_healthy` waits for the health check to pass:

```yaml
# This waits until the database health check passes
depends_on:
  database:
    condition: service_healthy
```

This is the correct approach. Always use health checks with `depends_on` for reliable startup.

The wait-for-it Pattern

Sometimes you cannot add health checks (for example, when using third-party images). An alternative approach is the wait-for-it script — a shell script that waits for a network port to become available before starting your application.

Create `api/wait-for-it.sh`:

```bash
#!/bin/sh
# wait-for-it.sh - Wait for a service to become available

HOST="$1"
```

```
PORT="$2"
shift 2
CMD="$@"

echo "Waiting for $HOST:$PORT to be ready..."

while ! nc -z "$HOST" "$PORT" 2>/dev/null; do
  echo "  $HOST:$PORT is not ready. Waiting..."
  sleep 1
done

echo "$HOST:$PORT is ready!"
exec $CMD
```

Make it executable:

```bash
chmod +x api/wait-for-it.sh
```

Use it in the Dockerfile:

```dockerfile
# Copy the wait script
COPY wait-for-it.sh /wait-for-it.sh
RUN chmod +x /wait-for-it.sh

# Wait for the database before starting
CMD ["/wait-for-it.sh", "database", "5432", "npm", "start"]
```

The script repeatedly tries to connect to `database:5432`. Once the connection succeeds, it runs `npm start`.

However, with Docker Compose's `depends_on` and health checks (as we did in our example), the wait-for-it approach is usually not needed. The health check approach is simpler and more reliable.

Building and Running the Application

Step 1: Build and Start

From the `task-manager` directory:

```bash
docker compose up -d --build
```

The `--build` flag forces Docker to build the images. Without it, Docker uses cached images if they exist.

Output:

```
[+] Building 15.2s (10/10) FINISHED
 => [api] Building...
 => ...
[+] Running 4/4
 ✔ Network task-manager_app-network  Created
 ✔ Container task-manager-cache-1     Started
 ✔ Container task-manager-database-1  Started
 ✔ Container task-manager-api-1       Started
```

Step 2: Check Service Status

```bash
docker compose ps
```

Output:

```
NAME                        IMAGE              STATUS                    PORTS
task-manager-api-1          task-manager-api   Up 10s (healthy)          0.0.0.0:3000->3...
task-manager-cache-1        redis:7-alpine     Up 15s (healthy)          6379/tcp
task-manager-database-1     postgres:16        Up 15s (healthy)          5432/tcp
```

All three services are running and healthy.

Step 3: Test the API

Let us test our API using `curl` (a command-line tool for making HTTP requests).

Get all tasks (should return the sample data from init.sql):

```bash
curl http://localhost:3000/tasks
```

Output:

```json
[
  {
    "id": 3,
    "title": "Deploy to production",
    "description": "Use Docker in a real project",
    "completed": false,
    "created_at": "2024-01-15T10:30:00.000Z"
  },
  {
    "id": 2,
    "title": "Learn Docker Compose",
    "description": "Build multi-container applications",
    "completed": false,
    "created_at": "2024-01-15T10:30:00.000Z"
  },
  {
    "id": 1,
    "title": "Learn Docker basics",
    "description": "Understand images, containers, and volumes",
    "completed": false,
    "created_at": "2024-01-15T10:30:00.000Z"
  }
]
```

Create a new task:

```bash
curl -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Read Chapter 15", "description": "Multi-container apps"}'
```

Output:

```json
{
  "id": 4,
  "title": "Read Chapter 15",
  "description": "Multi-container apps",
  "completed": false,
  "created_at": "2024-01-15T10:35:00.000Z"
}
```

Delete a task:

```bash
curl -X DELETE http://localhost:3000/tasks/1
```

Output:

```json
{
  "message": "Task deleted",
  "task": {
    "id": 1,
    "title": "Learn Docker basics",
    "description": "Understand images, containers, and volumes",
    "completed": false
  }
}
```

Check the health endpoint:

```bash
curl http://localhost:3000/health
```

Output:

```json
{
  "status": "healthy",
  "database": "connected",
  "cache": "connected"
}
```

Step 4: Observe Caching in Action

Run `GET /tasks` twice and watch the logs:

```bash
curl http://localhost:3000/tasks
curl http://localhost:3000/tasks
```

Now check the API logs:

```bash
docker compose logs api --tail 10
```

Output:

```
task-manager-api-1  | Querying tasks from database
task-manager-api-1  | Returning tasks from cache
```

The first request queried the database. The second request returned data from Redis cache. This is faster because Redis is an in-memory store.

Working with Logs

Docker Compose makes it easy to view logs from all services.

Viewing All Logs

```bash
docker compose logs
```

This shows logs from all three services, color-coded:

```
task-manager-database-1  | PostgreSQL init process complete; ready for start up.
task-manager-database-1  | database system is ready to accept connections
task-manager-cache-1     | Ready to accept connections
task-manager-api-1       | Connected to Redis
task-manager-api-1       | Task Manager API running on port 3000
```

Following Logs in Real Time

```bash
docker compose logs -f
```

Press `Ctrl+C` to stop following.

Logs for a Specific Service

```bash
# Only database logs
docker compose logs database

# Only API logs with follow
docker compose logs -f api

# Last 20 lines from all services
docker compose logs --tail 20
```

Debugging a Specific Service

If something is not working, check the service logs first:

```bash
# Check if the database initialized correctly
docker compose logs database | head -30

# Check if the API connected to its dependencies
docker compose logs api

# Check if Redis is accepting connections
docker compose logs cache
```

Stopping and Restarting

Stop and Remove Everything

```bash
docker compose down
```

This stops all containers, removes them, and removes the network. The database data is preserved in the `db-data` volume.

Stop Without Removing

```bash
docker compose stop
```

This stops containers but does not remove them. You can restart with `docker compose start`.

Restart Everything

```bash
docker compose up -d
```

The database still has all its data because we used a named volume.

Full Reset (Delete Everything Including Data)

```bash
docker compose down -v
```

The `-v` flag removes the named volumes, which deletes all database data. The next `docker compose up -d` will re-run the `init.sql` script because the volume is fresh.

Common Mistakes

Mistake 1: Not Using depends_on with Health Checks

```yaml
# Wrong: API starts before the database is ready
services:
  api:
    build: ./api
    depends_on:
      - database

# Right: API waits for the database health check to pass
services:
  api:
    build: ./api
    depends_on:
      database:
        condition: service_healthy
```

Mistake 2: Using localhost in Connection Strings

```javascript
// Wrong: "localhost" refers to the API container itself, not the database
const pool = new Pool({ host: 'localhost', ... });

// Right: Use the service name as the hostname
const pool = new Pool({ host: 'database', ... });
```

Mistake 3: Forgetting to Rebuild After Code Changes

```bash
# Wrong: this uses the old cached image
docker compose up -d

# Right: rebuild the image after code changes
docker compose up -d --build
```

Mistake 4: Not Declaring Named Volumes

```yaml
# Wrong: volume "db-data" is used but not declared
services:
  database:
    volumes:
      - db-data:/var/lib/postgresql/data

# Right: declare the volume in the top-level volumes section
services:
  database:
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

Mistake 5: Hardcoding Credentials in Source Code

```javascript
// Wrong: credentials hardcoded in source code
const pool = new Pool({
  host: 'database',
  user: 'taskuser',
  password: 'taskpass',
});

// Right: use environment variables
const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});
```

And set the values in `docker-compose.yml`:

```yaml
environment:
  DB_HOST: database
  DB_USER: taskuser
  DB_PASSWORD: taskpass
```

Best Practices

1. Always use health checks for services that other services depend on. This prevents race conditions during startup.
2. Use environment variables for all configuration. Never hardcode hostnames, ports, or credentials.
3. Use named volumes for any data you want to persist (databases, file uploads, etc.).
4. Use `--build` when you change application code to ensure the image is rebuilt.
5. Start with `docker compose logs` when debugging. The logs almost always contain the answer.
6. Use meaningful service names that describe the role: `database` instead of `postgres`, `cache` instead of `redis`.
7. Keep the init script idempotent. Use `CREATE TABLE IF NOT EXISTS` instead of `CREATE TABLE` so the script does not fail if run multiple times.
8. Test the health endpoint manually with curl to make sure it works before relying on it for health checks.

Quick Summary

In this chapter, you built a real multi-container application with a Node.js API, PostgreSQL database, and Redis cache. The `docker-compose.yml` file defined all three services, their configurations, and their dependencies. You used `depends_on` with `condition: service_healthy` to ensure services start in the correct order. Health checks used `pg_isready` for PostgreSQL and `redis-cli ping` for Redis. The API connected to other services using their service names as hostnames, which Docker's DNS resolved automatically. You tested the API with curl and observed how Redis caching speeds up repeated requests.

Key Points

- Multi-container applications use Docker Compose to define and run multiple services together.
- Services communicate using their service names as hostnames (Docker DNS resolution).
- `depends_on` with `condition: service_healthy` ensures services start only after their dependencies are ready.

- Health checks should verify that the application is truly functional, not just that the process is running.
- PostgreSQL runs SQL files in `/docker-entrypoint-initdb.d/` during first-time initialization.
- Named volumes persist database data across container restarts.
- Redis provides fast in-memory caching to reduce database load.
- Use `docker compose up -d --build` to rebuild images after code changes.
- Use `docker compose logs` as the first debugging step.
- Always use environment variables for connection strings and credentials.

Practice Questions

1. Why do we use `condition: service_healthy` instead of just listing the service in `depends_on`? What would happen without health checks?
2. In the API code, the database connection uses `host: 'database'`. How does Docker know which IP address `database` refers to?
3. What is the purpose of the `init.sql` file, and where must it be mounted in the PostgreSQL container? When does it run?
4. Explain how Redis caching works in this application. What happens when you create a new task — why do we clear the cache?
5. If you run `docker compose down -v` and then `docker compose up -d`, what happens to the data in the database?

Exercises

Exercise 1: Extend the API

1. Clone the task manager project from this chapter.
2. Add a `PUT /tasks/:id` endpoint that updates a task's title and description.
3. Make sure to clear the Redis cache when a task is updated.
4. Rebuild and test with curl.

Exercise 2: Add a Fourth Service

1. Add an Adminer service (a web-based database management tool) to the `docker-compose.yml`:

```yaml
  adminer:
    image: adminer
    ports:
      - "8080:8080"
```

2. Start the stack and access Adminer at `http://localhost:8080`.
3. Log in with the database credentials and browse the tasks table.

Exercise 3: Simulate a Failure

1. Start the full stack.
2. Stop only the database service: `docker compose stop database`.
3. Try to create a task via curl. What error do you get?
4. Check the API logs. What do they say?
5. Check the health endpoint. What status does it report?
6. Restart the database: `docker compose start database`.
7. Verify that everything recovers.

What Is Next?

You have built a real multi-container application. But we glossed over some important details about networking and volumes in Compose. In the next chapter, you will take a deep dive into Compose networking (how services discover each other, custom networks, and network isolation) and Compose volumes (named volumes, volume declarations, and sharing volumes between services). These concepts are essential for building secure and well-organized applications.

Chapter 16: Compose Networking and Volumes

What You Will Learn

By the end of this chapter, you will be able to:

- Explain how the default Compose network works
- Create and configure custom networks in Compose
- Use service names as hostnames for service discovery
- Declare and use named volumes in Compose
- Share volumes between multiple services
- Use external volumes created outside of Compose
- Isolate services using multiple networks
- Understand network isolation between different Compose projects

Why This Chapter Matters

In the previous chapters, networking and volumes "just worked" because Compose handled them automatically. But as your applications grow more complex, you need to understand what is happening behind the scenes. You need to answer questions like:

- How do my services find each other?
- Can I prevent certain services from communicating?
- Where is my data actually stored?
- Can two different Compose projects share the same database?

Think of this chapter as learning how the plumbing and electrical wiring work in your house. When everything is fine, you do not think about them. But when you need to add a new room, fix a leak, or understand why the lights went out, you need to know how the systems work.

The Default Compose Network

When you run `docker compose up`, Compose automatically creates a network for your project. Every service in the Compose file is connected to this network. You do not have to do anything — it just happens.

How the Default Network Is Named

The default network name follows this pattern:

```
<project-directory-name>_default
```

For example, if your project is in a folder called `my-app`:

```bash
cd ~/my-app
docker compose up -d
```

Docker creates a network called `my-app_default`.

You can verify this:

```bash
docker network ls
```

Output:

```
NETWORK ID     NAME               DRIVER    SCOPE
a1b2c3d4e5f6   bridge             bridge    local
f6e5d4c3b2a1   host               host      local
d7e8f9a0b1c2   my-app_default     bridge    local
1a2b3c4d5e6f   none               null      local
```

What the Default Network Provides

The default network gives you:

1. Automatic DNS resolution: Every service can reach every other service by its service name.
2. Isolation from other projects: Services in one Compose project cannot reach services in another.
3. Isolation from standalone containers: Standalone containers (started with `docker run`) are on the default bridge network, not your Compose network.

```
┌──────────────────────────────────────────┐
│          my-app_default network          │
│                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐ │
│   │   web    │  │   api    │  │   db     │ │
│   │          ◄──►          ◄──►          │ │
│   └──────────┘  └──────────┘  └──────────┘ │
│                                          │
│   All services can reach each other      │
│   by name: "web", "api", "db"            │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│       other-project_default network      │
│                                          │
│   ┌──────────┐  ┌──────────┐             │
│   │ frontend│  │ backend │              │
│   │          ◄──►         │              │
│   └──────────┘  └──────────┘             │
│                                          │
│   Cannot reach services in my-app        │
└──────────────────────────────────────────┘
```

Service Discovery: Service Name = Hostname

This is one of the most important concepts in Docker Compose networking. When you define a service in `docker-compose.yml`, the service name automatically becomes a hostname that other services can use.

How It Works

```yaml
services:
  api:
```

```
    image: my-api
    environment:
      DATABASE_HOST: database    # <-- use the service name
      REDIS_HOST: cache          # <-- use the service name

  database:
    image: postgres:16

  cache:
    image: redis:7
```

Inside the `api` container:

- The hostname `database` resolves to the IP address of the `database` container.
- The hostname `cache` resolves to the IP address of the `cache` container.

This works because Docker runs a built-in DNS server for each Compose network. When a container asks "what is the IP address of `database`?", Docker's DNS server responds with the correct IP address.

```
┌──────────────────────────────────────────┐
│   Inside the "api" container             │
│                                          │
│   ping database                          │
│     │                                    │
│     ▼                                    │
│   Docker DNS Server                      │
│     │                                    │
│     ▼                                    │
│   "database" = 172.18.0.3                │
│     │                                    │
│     ▼                                    │
│   PING 172.18.0.3 ... reply!             │
│                                          │
│   The service name IS the hostname       │
└──────────────────────────────────────────┘
```

Connection String Examples

Here is how you would configure common applications to connect to other services:

```yaml
services:
  app:
    environment:
      # PostgreSQL connection
      DATABASE_URL: postgres://user:pass@database:5432/mydb
```

```
      # Redis connection
      REDIS_URL: redis://cache:6379

      # MongoDB connection
      MONGO_URL: mongodb://mongo:27017/mydb

      # MySQL connection
      MYSQL_HOST: mysql-db
      MYSQL_PORT: 3306
```

In every case, the hostname is the service name from the Compose file. The port is the container's internal port (not any mapped host port).

An Important Distinction: Internal vs. External Ports

When services communicate with each other inside the Compose network, they use the container port (the internal port). Port mapping (`ports:`) only applies to access from outside Docker (your host machine).

```yaml
services:
  api:
    ports:
      - "8080:3000"    # External: access via localhost:8080
    environment:
      DATABASE_URL: postgres://user:pass@database:5432/mydb
      # Internal: connects to database:5432 (container port)

  database:
    image: postgres:16
    # No ports mapping needed! The API connects internally.
    # Only map ports if you need access from your host machine.
```

```
┌──────────────────────────────────────────────────┐
│                                                  │
│   Your Browser ──> localhost:8080 ──> api:3000   │
│   (external access uses mapped port 8080)        │
│                                                  │
│   api container ──> database:5432                │
│   (internal access uses container port 5432)     │
│                                                  │
│   api container ──> cache:6379                   │
│   (internal access uses container port 6379)     │
│                                                  │
└──────────────────────────────────────────────────┘
```

Custom Networks in Compose

While the default network works for simple applications, custom networks give you more control — especially when you need to isolate certain services from each other.

Defining Custom Networks

```yaml
services:
  web:
    image: nginx
    networks:
      - frontend

  api:
    image: my-api
    networks:
      - frontend
      - backend

  database:
    image: postgres:16
    networks:
      - backend

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

This creates two networks:

- frontend: Contains `web` and `api`.
- backend: Contains `api` and `database`.

```
┌──────────────────────────────────────────────────┐
│                                                  │
│   frontend network         backend network       │
│   ┌──────────────┐         ┌──────────────┐      │
│   │              │         │              │      │
│   │   web ◄──────►│ api ◄──►│  database    │      │
│   │              │         │              │      │
│   └──────────────┘         └──────────────┘      │
│                                                  │
│   web can reach api:       Yes (frontend)        │
│   api can reach database:  Yes (backend)         │
│   web can reach database:  No (different nets)   │
│                                                  │
└──────────────────────────────────────────────────┘
```

The key benefit: the `web` service cannot directly communicate with the `database`. It must go through the `api` service. This is a security best practice — the database is only accessible from the services that need it.

Why Isolate Services?

Consider a security scenario. If an attacker compromises your web server, they can reach anything on the same network. With network isolation:

- If `web` is compromised, the attacker can only reach `api`, not `database`.
- The `database` is protected behind the `backend` network.
- The `api` acts as a gatekeeper between the two networks.

Network Configuration Options

You can configure networks with additional options:

```yaml
networks:
  frontend:
    driver: bridge
    name: my-frontend-net     # Custom name instead of project prefix

  backend:
    driver: bridge
    ipam:                      # IP Address Management
      config:
        - subnet: 172.28.0.0/16
```

- `name`: Override the auto-generated network name.
- `ipam`: Customize the IP address range for the network.

Named Volumes in Compose

Named volumes are Docker-managed storage that persist data across container restarts. In Compose, you declare named volumes in two places:

1. In the service definition (where the volume is mounted).
2. In the top-level `volumes` section (where the volume is declared).

Declaring and Using Named Volumes

```yaml
services:
  database:
    image: postgres:16
    volumes:
      - db-data:/var/lib/postgresql/data

  redis:
    image: redis:7
    volumes:
      - redis-data:/data

volumes:
  db-data:
  redis-data:
```

Let us trace what happens:

1. Compose sees `db-data:/var/lib/postgresql/data` in the `database` service.
2. It looks for `db-data` in the top-level `volumes` section.
3. If the volume does not exist, Docker creates it.
4. Docker mounts the volume at `/var/lib/postgresql/data` inside the container.
5. Any data PostgreSQL writes to that directory is stored in the volume.
6. When you run `docker compose down`, the containers are removed but the volume stays.
7. When you run `docker compose up` again, the same volume is mounted, and all data is still there.

Volume Lifecycle

```
┌──────────────── Volume Lifecycle ────────────────┐
│                                                  │
│   docker compose up -d                           │
│     └──> Volume "db-data" created (if new)       │
│     └──> Volume mounted to container             │
│                                                  │
│   Application writes data                        │
│     └──> Data stored in volume                   │
│                                                  │
│   docker compose down                            │
│     └──> Container removed                       │
│     └──> Volume KEPT (data preserved)            │
│                                                  │
│   docker compose up -d (again)                   │
│     └──> Same volume mounted                     │
│     └──> All data still there!                   │
│                                                  │
│   docker compose down -v                         │
│     └──> Container removed                       │
```

```
│     └──> Volume DELETED (data gone!)              │
│                                                   │
└───────────────────────────────────────────────────┘
```

Volume Configuration Options

Named volumes can have configuration:

```yaml
volumes:
  db-data:
    driver: local              # The default driver

  important-data:
    driver: local
    name: my-production-data   # Custom name (not prefixed with project name)

  existing-volume:
    external: true             # Volume must already exist
```

- `driver`: The storage driver. `local` is the default and stores data on the host filesystem.
- `name`: Override the auto-generated name. By default, Compose names volumes as `<project>_<volume>`.
- `external: true`: The volume must already exist before you run `docker compose up`. Compose will not create it.

Sharing Volumes Between Services

Sometimes multiple services need access to the same files. You can mount the same volume in multiple services.

Example: Shared Upload Directory

```yaml
services:
  api:
    image: my-api
    volumes:
      - uploads:/app/uploads     # API writes uploaded files here

  processor:
```

```
    image: my-processor
    volumes:
      - uploads:/data/incoming   # Processor reads files from here

volumes:
  uploads:
```

```
┌──────────────────────────────────────────┐
│                                          │
│   ┌──────────┐     ┌──────────┐         │
│   │   api    │     │ processor │         │
│   │          │     │           │         │
│   │ /app/uploads│  │/data/incoming│      │
│   │     │    │     │     │     │         │
│   └─────┼────┘     └─────┼─────┘         │
│         │                │               │
│         └────────┬───────┘               │
│                  │                       │
│          ┌───────▼────────┐              │
│          │ uploads volume │              │
│          │ (shared data)  │              │
│          └────────────────┘              │
│                                          │
│   api writes a file -> processor can     │
│   read the same file immediately         │
└──────────────────────────────────────────┘
```

Both containers see the same files. When the `api` service saves an uploaded file, the `processor` service can immediately read it.

Example: Shared Configuration

```yaml
services:
  web:
    image: nginx
    volumes:
      - shared-config:/etc/nginx/conf.d:ro

  api:
    image: my-api
    volumes:
      - shared-config:/etc/app/config:ro

  config-generator:
    image: my-config-tool
    volumes:
      - shared-config:/output     # This service writes the config
```

```
volumes:
  shared-config:
```

The `config-generator` writes configuration files to the shared volume. Both `web` and `api` read from it. The `:ro` (read-only) flag prevents them from accidentally modifying the configuration.

Read-Only Volumes

Append `:ro` to a volume mount to make it read-only inside the container:

```yaml
services:
  web:
    volumes:
      - static-files:/usr/share/nginx/html:ro
```

The container can read files from this volume but cannot create, modify, or delete them. This is a good security practice for services that only need to read data.

External Volumes

An external volume is a volume that exists outside of Docker Compose — you created it manually or it belongs to another project. Compose will not create or delete it.

Creating an External Volume

```bash
docker volume create shared-database
```

Using an External Volume in Compose

```yaml
services:
  database:
    image: postgres:16
    volumes:
      - shared-database:/var/lib/postgresql/data

volumes:
```

```
  shared-database:
    external: true
```

The `external: true` flag tells Compose:

- Do not create this volume — it must already exist.
- Do not delete this volume on `docker compose down -v`.

If the volume does not exist, `docker compose up` will fail with an error:

```
ERROR: Volume shared-database declared as external, but could not be found.
```

Why Use External Volumes?

1. Sharing data between Compose projects: Two different Compose projects can use the same external volume to share data.
2. Protecting critical data: External volumes are not deleted by `docker compose down -v`.
3. Using pre-populated volumes: You can create a volume, populate it with data, and then use it in Compose.

Sharing a Volume Between Compose Projects

Project A (`~/project-a/docker-compose.yml`):

```yaml
services:
  writer:
    image: my-writer-app
    volumes:
      - shared-data:/data

volumes:
  shared-data:
    external: true
```

Project B (`~/project-b/docker-compose.yml`):

```yaml
services:
  reader:
    image: my-reader-app
    volumes:
      - shared-data:/data:ro
```

```
volumes:
  shared-data:
    external: true
```

Both projects use the same external volume `shared-data`. Project A writes to it, and Project B reads from it.

```
┌──────────────────┐         ┌──────────────────┐
│   Project A      │         │   Project B      │
│                  │         │                  │
│   writer ────────┼─────────┼──── reader       │
│   /data (rw)     │         │   /data (ro)     │
│                  │         │                  │
└──────────────────┘         └──────────────────┘
          │                           │
          └───────────────────────────┘
                      │
                      ▼
            ┌─────────────────┐
            │  shared-data    │
            │  (external vol) │
            └─────────────────┘
```

Network Isolation Between Compose Projects

By default, each Compose project gets its own isolated network. Services in Project A cannot reach services in Project B, and vice versa.

Demonstrating Isolation

Project A (`~/project-a/docker-compose.yml`):

```yaml
services:
  web-a:
    image: nginx
```

Project B (`~/project-b/docker-compose.yml`):

```yaml
services:
  web-b:
    image: nginx
```

When both are running:

```bash
# Start both projects
cd ~/project-a && docker compose up -d
cd ~/project-b && docker compose up -d

# Check networks
docker network ls
```

Output:

```
NETWORK ID     NAME                  DRIVER    SCOPE
...
d1e2f3a4b5c6   project-a_default     bridge    local
a6b5c4d3e2f1   project-b_default     bridge    local
```

Two separate networks. `web-a` cannot reach `web-b`:

```bash
# From inside web-a, try to reach web-b
docker compose -f ~/project-a/docker-compose.yml exec web-a ping -c 1 web-b
# ping: web-b: Name or service not known
```

Connecting Compose Projects with External Networks

If you need services from different Compose projects to communicate, use an external network:

Step 1: Create the external network

```bash
docker network create shared-network
```

Step 2: Use it in both projects

Project A (`~/project-a/docker-compose.yml`):

```yaml
services:
  api-a:
    image: my-api
    networks:
      - shared

networks:
```

```
  shared:
    external: true
    name: shared-network
```

Project B (`~/project-b/docker-compose.yml`):

```yaml
services:
  api-b:
    image: my-api
    networks:
      - shared

networks:
  shared:
    external: true
    name: shared-network
```

Now `api-a` and `api-b` are on the same network and can communicate using their service names.

Putting It All Together: A Complete Example

Let us build a Compose configuration that uses custom networks, named volumes, and shared volumes.

```yaml
services:
  # ──────────────────────────────────────────
  # Frontend — Nginx serving static files
  # ──────────────────────────────────────────
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - static-assets:/usr/share/nginx/html:ro
    networks:
      - frontend
    depends_on:
      - api

  # ──────────────────────────────────────────
  # API — Node.js application
  # ──────────────────────────────────────────
```

```
  api:
    build: ./api
    ports:
      - "3000:3000"
    volumes:
      - uploads:/app/uploads
    environment:
      DB_HOST: database
      REDIS_URL: redis://cache:6379
    networks:
      - frontend
      - backend
    depends_on:
      database:
        condition: service_healthy
      cache:
        condition: service_healthy

  # ──────────────────────────────────────────
  # Database — PostgreSQL
  # ──────────────────────────────────────────
  database:
    image: postgres:16
    volumes:
      - db-data:/var/lib/postgresql/data
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
      POSTGRES_DB: appdb
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - backend

  # ──────────────────────────────────────────
  # Cache — Redis
  # ──────────────────────────────────────────
  cache:
    image: redis:7-alpine
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - backend

  # ──────────────────────────────────────────
  # Asset Builder — Generates static files
  # ──────────────────────────────────────────
```

```
  asset-builder:
    build: ./assets
    volumes:
      - static-assets:/output
    profiles:
      - build

# ──────────────────────────────────────────
# Volumes
# ──────────────────────────────────────────
volumes:
  db-data:          # PostgreSQL data
  redis-data:       # Redis persistence
  uploads:          # User-uploaded files
  static-assets:    # Built frontend assets (shared between builder and web)

# ──────────────────────────────────────────
# Networks
# ──────────────────────────────────────────
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

Let us trace the network isolation:

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│   frontend network              backend network          │
│   ┌──────────┐                  ┌──────────┐            │
│   │          │                  │          │            │
│   │  web  ◄──┼──► api ◄─────────┼──► database│          │
│   │  :8080   │                  │  :5432   │            │
│   │          │                  │          │            │
│   └──────────┘                  │  cache   │            │
│                                 │  :6379   │            │
│                                 │          │            │
│                                 └──────────┘            │
│                                                          │
│   web -> api:        Yes (frontend)                      │
│   web -> database:   No  (different networks)            │
│   web -> cache:      No  (different networks)            │
│   api -> database:   Yes (backend)                       │
│   api -> cache:      Yes (backend)                       │
│   api -> web:        Yes (frontend)                      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

And the volume sharing:

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│   asset-builder ──writes──> static-assets <──reads── web │
│                                                          │
│   api ──writes/reads──> uploads                          │
│                                                          │
│   database ──writes/reads──> db-data                     │
│                                                          │
│   cache ──writes/reads──> redis-data                     │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Common Mistakes

Mistake 1: Not Declaring Volumes at the Top Level

```yaml
# Wrong: volume is used but not declared
services:
  database:
    volumes:
      - my-data:/var/lib/postgresql/data
# Error: volume "my-data" is not declared

# Right: declare it
services:
  database:
    volumes:
      - my-data:/var/lib/postgresql/data

volumes:
  my-data:
```

Mistake 2: Using Mapped Ports for Inter-Service Communication

```yaml
services:
  api:
    ports:
      - "8080:3000"
    environment:
      # Wrong: using the mapped host port
      DATABASE_URL: postgres://user:pass@database:5433/mydb
```

```
  database:
    image: postgres:16
    ports:
      - "5433:5432"
    # Right: use the container port (5432), not the host port (5433)
```

Other services should always use the container's internal port. The mapped port is only for access from your host machine.

Mistake 3: Expecting External Volumes to Be Auto-Created

```yaml
volumes:
  my-volume:
    external: true
# If "my-volume" does not exist, docker compose up will fail!
```

Create the volume first:

```bash
docker volume create my-volume
```

Mistake 4: Forgetting Network Assignment

```yaml
# Wrong: web is on frontend but api is not — they cannot communicate
services:
  web:
    image: nginx
    networks:
      - frontend
  api:
    image: my-api
    networks:
      - backend     # Missing "frontend"! web cannot reach api.
```

If a service needs to communicate with services on multiple networks, it must be on all of those networks.

Mistake 5: Assuming Compose Projects Can See Each Other

Services in different Compose projects are on different networks by default. They cannot communicate unless you use an external network.

Best Practices

1. Use custom networks for security. Separate your frontend and backend tiers. Only the API should have access to the database.
2. Always declare named volumes. Named volumes persist data and have clear names. Avoid anonymous volumes.
3. Use `:ro` (read-only) mounts when a service only needs to read data. This prevents accidental modifications.
4. Use `external: true` for critical data. This prevents accidental deletion with `docker compose down -v`.
5. Use service names as hostnames. Never hardcode IP addresses. Service names are stable and handled by Docker DNS.
6. Use internal ports for service-to-service communication. Only map ports to the host when you need external access.
7. Document your network topology. Add comments in your `docker-compose.yml` explaining which networks serve which purpose.
8. Regularly clean up unused volumes with `docker volume prune`. Old volumes can consume significant disk space.

Quick Summary

Docker Compose automatically creates a default network where all services can reach each other by name. For more control, you can define custom networks to isolate services — for example, keeping the database on a backend network that the web server cannot reach. Named volumes persist data across container restarts and must be declared in both the service definition and the top-level `volumes` section. Volumes can be shared between services for scenarios like file uploads or shared configuration. External volumes and networks allow sharing resources between different Compose projects while protecting critical data from accidental deletion.

Key Points

- Compose automatically creates a default network named `<project>_default`.
- Service names are automatically registered as DNS hostnames on the Compose network.
- Services communicate using container ports (internal), not mapped host ports.
- Custom networks in Compose provide isolation between service groups.
- A service can belong to multiple networks to bridge between them.
- Named volumes must be declared in the top-level `volumes:` section.
- Volumes can be shared between services by mounting the same volume in multiple services.
- Use `:ro` to mount volumes as read-only when a service only needs to read data.
- `external: true` means the volume or network must already exist and will not be managed by Compose.
- Different Compose projects are network-isolated by default; use external networks to connect them.

Practice Questions

1. You have a Compose file with services `web`, `api`, and `database`. Without defining any custom networks, can `web` communicate with `database`? Why or why not?
2. What is the difference between a named volume and an external volume? When would you use each?
3. Your `api` service needs to connect to PostgreSQL on port 5432. In the Compose file, the `database` service maps port `5433:5432`. What host and port should the `api` service use to connect? Why?
4. Explain how you would prevent a `frontend` service from directly accessing a `database` service while still allowing an `api` service to access both.
5. Two Compose projects need to share a PostgreSQL database. How would you set this up?

Exercises

Exercise 1: Network Isolation

1. Create a `docker-compose.yml` with three services: `web` (nginx), `api` (nginx), and `db` (nginx).
2. Create two networks: `public` and `private`.
3. Put `web` on `public`, `db` on `private`, and `api` on both.
4. Start the stack and verify:
    - `web` can ping `api`
    - `api` can ping `db`
    - `web` cannot ping `db`

Exercise 2: Shared Volumes

1. Create a Compose file with two services.
2. Service `writer` runs `busybox` and writes a file to a shared volume every 5 seconds.
3. Service `reader` runs `busybox` and reads the file from the shared volume.
4. Verify that `reader` can see the files written by `writer`.

Exercise 3: External Resources

1. Create a Docker network manually: `docker network create my-external-net`.
2. Create a Docker volume manually: `docker volume create my-external-data`.
3. Write a `docker-compose.yml` that uses both of these external resources.
4. Start the stack and verify everything works.
5. Run `docker compose down -v` and verify the external volume was NOT deleted.

What Is Next?

You now understand how Compose manages networks and volumes. In the next chapter, you will learn how to use Docker Compose for development workflows — bind mounts for live code reloading, watch mode, development vs. production configurations, and debugging inside containers. These techniques will make your daily development workflow much more efficient.

Chapter 17: Docker Compose for Development

What You Will Learn

By the end of this chapter, you will be able to:

- Use bind mounts for live code reloading during development
- Set up hot reload for Node.js, Python, and other frameworks
- Use `docker compose watch` for automatic sync and rebuild
- Create separate development and production Compose configurations
- Use `compose.override.yml` for environment-specific settings
- Combine multiple Compose files with `docker compose -f`
- Extend and override services for different environments
- Debug applications running inside containers
- Attach a debugger to a containerized application

Why This Chapter Matters

Up to this point, every time you changed your application code, you had to rebuild the Docker image and restart the container. That workflow is fine for production, but it is painfully slow for development. Imagine having to wait 30 seconds to see the effect of a one-line change. You would give up on Docker very quickly.

This chapter shows you how to set up a development workflow where code changes appear instantly inside your container — no rebuild needed. It is the difference between driving to the post office to send a letter and sending an email. The result is the same, but one is dramatically faster.

Think of it this way. In production, your application is sealed inside a container like food sealed in a can. That is great for shipping and storage. But during development, you need to taste and adjust the recipe constantly. This chapter teaches you how to keep the kitchen (your development environment) open while still using containers.

Bind Mounts for Live Code Reloading

A bind mount maps a directory on your host machine directly into the container. Unlike a named volume (which Docker manages), a bind mount uses a directory you specify. When you change a file on your host, the change appears inside the container immediately.

How Bind Mounts Work

```
┌──────────────────────────────────────────────┐
│   Your Computer (Host)                       │
│                                              │
│   ~/my-project/src/server.js                 │
│         │                                    │
│         │  bind mount                        │
│         │                                    │
│   ┌─────▼──────────────────────────┐         │
│   │   Container                    │         │
│   │                                │         │
│   │   /app/src/server.js           │         │
│   │   (same file! not a copy)      │         │
│   │                                │         │
│   │   Changes on host appear here  │         │
│   │   instantly — no rebuild!      │         │
│   └────────────────────────────────┘         │
│                                              │
└──────────────────────────────────────────────┘
```

The key difference from the production workflow:

- Production: Code is copied into the image with `COPY` during build. Changes require a rebuild.
- Development: Code is mounted from your host. Changes are reflected immediately.

Setting Up Bind Mounts in Compose

Here is a Node.js example:

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
    volumes:
      - ./api:/app                    # Mount source code
      - /app/node_modules             # Prevent overwriting node_modules
    environment:
      NODE_ENV: development
```

Let us explain each volume line:

- `./api:/app` — Mount the `./api` directory from your host into `/app` in the container. Every file in `./api` on your host is now also at `/app` in the container.
- `/app/node_modules` — This is an anonymous volume that preserves the container's `node_modules` directory. Without this, the bind mount would overwrite the container's `node_modules` with whatever is (or is not) in your host's `node_modules`.

The node_modules Problem

This is a common source of confusion. Let us walk through what happens:

1. Your Dockerfile has `RUN npm install`, which installs dependencies inside the container at `/app/node_modules`.
2. You add a bind mount `./api:/app`, which maps your host directory over `/app` in the container.
3. Your host directory might not have a `node_modules` folder (or it has one built for a different OS).
4. The bind mount overwrites the container's `/app` completely, including the `node_modules` that `npm install` created.
5. Your application fails because the dependencies are gone.

The fix: add an anonymous volume for `node_modules`:

```yaml
volumes:
  - ./api:/app              # Mount source code from host
  - /app/node_modules       # Keep container's node_modules
```

```
┌──────────────────────────────────────────────┐
│   Without /app/node_modules volume:          │
│                                              │
│   Host ./api/         ──mount──>  /app/      │
│   ├── src/                        ├── src/   │
```

```
│   ├── package.json              ├── package.json
│   └── (no node_modules)         └── (empty!) │
│                                              │
│   node_modules is OVERWRITTEN — app crashes!  │
├──────────────────────────────────────────────┤
│   With /app/node_modules volume:             │
│                                              │
│   Host ./api/         ──mount──>  /app/      │
│   ├── src/                    ├── src/     │
│   ├── package.json            ├── package.json
│                               └── node_modules/
│                                   (preserved!)
│                                              │
│   node_modules is kept from the build step    │
└──────────────────────────────────────────────┘
```

Hot Reload for Node.js with nodemon

For Node.js, the file watcher tool nodemon automatically restarts your application when it detects file changes.

Update `api/package.json`:

```json
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

Update your Compose file to use the dev script:

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
    volumes:
      - ./api:/app
      - /app/node_modules
    command: npm run dev      # Override CMD from Dockerfile
    environment:
      NODE_ENV: development
```

The `command:` line overrides whatever `CMD` is set in the Dockerfile. Instead of `npm start` (which runs `node server.js`), it runs `npm run dev` (which runs `nodemon server.js`).

Now the workflow is:

1. Start the stack: `docker compose up -d`
2. Edit `api/server.js` on your host machine.
3. Save the file.
4. nodemon inside the container detects the change and restarts the server.
5. Refresh your browser to see the change — no rebuild needed.

Hot Reload for Python with Flask

```yaml
services:
  web:
    build: ./web
    ports:
      - "5000:5000"
    volumes:
      - ./web:/app
    environment:
      FLASK_ENV: development
      FLASK_DEBUG: 1
    command: flask run --host=0.0.0.0 --reload
```

Flask's `--reload` flag watches for file changes and automatically restarts the server.

Hot Reload for Frontend Applications

Frontend frameworks like React, Vue, and Angular have built-in hot reload. They work with Docker bind mounts:

```yaml
services:
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    command: npm run dev -- --host 0.0.0.0
    environment:
      CHOKIDAR_USEPOLLING: "true"    # Needed for file watching in Docker
```

The `CHOKIDAR_USEPOLLING` environment variable tells the file watcher to use polling instead of native file system events. This is sometimes necessary because Docker's file system layer

may not pass native change events from the host to the container.

Docker Compose Watch

Docker Compose introduced a `watch` feature that provides a more sophisticated approach to syncing files during development. Instead of mounting your entire source directory, `watch` lets you define specific sync rules and trigger actions when files change.

What docker compose watch Does

The `watch` command monitors your source files and performs actions when changes are detected:

- sync: Copy the changed file into the container.
- rebuild: Rebuild the image and recreate the container.
- sync+restart: Copy the file and restart the service.

Setting Up Watch

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
    develop:
      watch:
        # Sync source code changes into the container
        - action: sync
          path: ./api/src
          target: /app/src

        # Rebuild when package.json changes (new dependencies)
        - action: rebuild
          path: ./api/package.json

        # Sync and restart when config changes
        - action: sync+restart
          path: ./api/config
          target: /app/config
```

Let us explain each rule:

1. sync for source code: When any file in `./api/src` changes, it is copied to `/app/src` in the container. The server detects the change and hot-reloads (if you are using nodemon or a similar tool).
2. rebuild for dependencies: When `package.json` changes (you added a new dependency), the entire image is rebuilt and the container is recreated. This ensures `npm install` runs with the new dependencies.
3. sync+restart for config: When configuration files change, they are copied to the container and the service is restarted. This is for files that require a restart to take effect but not a full rebuild.

Running Watch Mode

```bash
docker compose watch
```

This starts the services and begins watching for file changes. The output shows what actions are triggered:

```
[+] Running 2/2
 ✔ Container myapp-api-1       Started
 ✔ Container myapp-database-1  Started
Watching ./api/src for changes...
Watching ./api/package.json for changes...
Watching ./api/config for changes...
```

When you edit a file:

```
Syncing ./api/src/server.js to /app/src/server.js
```

Watch vs. Bind Mounts

```
┌──────────────────────────┬──────────────────────────┐
│   Bind Mounts            │   Docker Compose Watch   │
├──────────────────────────┼──────────────────────────┤
│ Mount entire directory   │ Sync specific paths      │
│ Real-time (instant)      │ Near real-time (slight   │
│                          │  delay for sync)          │
│ node_modules hack needed │ No hack needed           │
│ Works with any Compose   │ Requires newer Compose   │
│  version                 │  versions                │
│ Can cause permission     │ Fewer permission issues  │
│  issues across OS        │                          │
```

```
│ Simpler to set up         │ More configuration       │
│ Cannot trigger rebuilds   │ Can trigger rebuilds     │
│                           │  on dependency changes   │
└───────────────────────────┴──────────────────────────┘
```

Both approaches are valid. Bind mounts are simpler and work immediately. Watch mode is more flexible and avoids some common issues with bind mounts.

Development vs. Production Compose Files

Your development environment needs things that production does not:

- Bind mounts for code syncing
- Debug ports open
- Hot reload commands
- Verbose logging
- Development environment variables

Your production environment needs things that development does not:

- Optimized images
- No source code mounted
- Production commands
- Minimal logging
- Production secrets

Docker Compose lets you handle this with multiple files.

The compose.override.yml Pattern

Docker Compose automatically loads two files if they exist:

1. `docker-compose.yml` (or `compose.yml`) — The base configuration.
2. `docker-compose.override.yml` (or `compose.override.yml`) — Overrides for development.

The override file is merged on top of the base file. Values in the override file replace or extend values in the base file.

Base file: `docker-compose.yml`

This contains configuration shared between development and production:

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DB_HOST: database
      DB_USER: appuser
      DB_NAME: appdb
    depends_on:
      database:
        condition: service_healthy

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  db-data:
```

Override file: `docker-compose.override.yml`

This adds development-specific settings:

```yaml
services:
  api:
    volumes:
      - ./api:/app
      - /app/node_modules
    command: npm run dev
    environment:
      NODE_ENV: development
      DEBUG: "true"

  database:
    ports:
      - "5432:5432"    # Expose DB port for local GUI tools
```

When you run `docker compose up -d`, both files are merged automatically:

```
┌──────────────────────────────────────────────────┐
│   docker-compose.yml + docker-compose.override.yml│
│                                                  │
│   api:                                           │
│     build: ./api                  (from base)    │
│     ports: ["3000:3000"]          (from base)    │
│     volumes:                      (from override) │
│       - ./api:/app                               │
│       - /app/node_modules                        │
│     command: npm run dev          (from override) │
│     environment:                                 │
│       DB_HOST: database           (from base)    │
│       DB_USER: appuser            (from base)    │
│       DB_NAME: appdb              (from base)    │
│       NODE_ENV: development       (from override) │
│       DEBUG: "true"               (from override) │
│                                                  │
│   database:                                      │
│     image: postgres:16            (from base)    │
│     ports: ["5432:5432"]          (from override) │
│     volumes, healthcheck...       (from base)    │
│                                                  │
└──────────────────────────────────────────────────┘
```

The override file does not replace the entire service — it merges. Lists like `environment` are combined. Single values like `command` are replaced.

Production File: compose.production.yml

For production, create a separate file:

`compose.production.yml`:

```yaml
services:
  api:
    command: npm start
    environment:
      NODE_ENV: production
    restart: always

  database:
    restart: always
    # No port mapping — database not accessible from outside
```

To use the production file instead of the override:

```bash
docker compose -f docker-compose.yml -f compose.production.yml up -d
```

The `-f` flag specifies which files to use. When you provide `-f` explicitly, the automatic loading of `docker-compose.override.yml` is skipped.

Using Multiple Compose Files with -f

The `-f` flag lets you combine any number of Compose files. Files are applied in order — later files override earlier ones.

Common Patterns

```bash
# Development (default: base + override)
docker compose up -d

# Production (base + production overrides)
docker compose -f docker-compose.yml -f compose.production.yml up -d

# Testing (base + test overrides)
docker compose -f docker-compose.yml -f compose.test.yml up -d

# Production with monitoring (base + production + monitoring)
docker compose -f docker-compose.yml \
  -f compose.production.yml \
  -f compose.monitoring.yml \
  up -d
```

Example: Test Configuration

`compose.test.yml`:

```yaml
services:
  api:
    command: npm test
    environment:
      NODE_ENV: test
      DB_NAME: testdb

  database:
    environment:
      POSTGRES_DB: testdb
    # Use a temporary volume for test data
    volumes:
      - test-data:/var/lib/postgresql/data
```

```
volumes:
  test-data:
```

Run tests:

```bash
docker compose -f docker-compose.yml -f compose.test.yml up --abort-on-container-exit
```

The `--abort-on-container-exit` flag stops all services when any one service exits. This is perfect for test runs — when the test runner finishes, everything shuts down.

Extending Services

Docker Compose supports extending services to avoid repeating configuration.

Using YAML Anchors

YAML anchors let you define a block once and reuse it:

```yaml
x-common-env: &common-env
  DB_HOST: database
  DB_USER: appuser
  DB_PASSWORD: apppass
  DB_NAME: appdb

services:
  api:
    build: ./api
    environment:
      <<: *common-env
      PORT: 3000

  worker:
    build: ./worker
    environment:
      <<: *common-env
      QUEUE_NAME: tasks
```

Let us break down the syntax:

- `x-common-env:` — Any top-level key starting with `x-` is an extension field. Compose ignores it.

- `&common-env` — This creates an anchor named `common-env` that points to the block of environment variables.
- `<<: *common-env` — This merges the anchored block into the current position. The `<<` means "merge" and `*common-env` references the anchor.

The result is equivalent to:

```yaml
services:
  api:
    environment:
      DB_HOST: database
      DB_USER: appuser
      DB_PASSWORD: apppass
      DB_NAME: appdb
      PORT: 3000

  worker:
    environment:
      DB_HOST: database
      DB_USER: appuser
      DB_PASSWORD: apppass
      DB_NAME: appdb
      QUEUE_NAME: tasks
```

Using Extension Fields for Service Templates

```yaml
x-service-base: &service-base
  restart: unless-stopped
  logging:
    driver: json-file
    options:
      max-size: "10m"
      max-file: "3"

services:
  api:
    <<: *service-base
    build: ./api
    ports:
      - "3000:3000"

  worker:
    <<: *service-base
    build: ./worker
```

Both `api` and `worker` inherit the `restart` and `logging` configuration from the `x-service-base` template.

Debugging Inside Containers

When your application runs inside a container, you need a way to debug it. Here are several approaches.

Approach 1: docker compose exec

The simplest way to investigate is to get a shell inside the container:

```bash
docker compose exec api sh
```

From inside, you can:

```bash
# Check environment variables
env | grep DB

# Test the database connection
ping database

# Check file permissions
ls -la /app/

# Read application logs
cat /app/logs/error.log

# Check what processes are running
ps aux

# Test an endpoint from inside the container
wget -qO- http://localhost:3000/health
```

Approach 2: Adding Debug Tools to Development Images

Your production Dockerfile should be minimal, but your development setup can include debugging tools:

```yaml
services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile.dev    # Use a development Dockerfile
```

`api/Dockerfile.dev`:

```dockerfile
FROM node:20

# Install debugging and diagnostic tools
RUN apt-get update && apt-get install -y \
    curl \
    wget \
    net-tools \
    iputils-ping \
    vim \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000
EXPOSE 9229    # Node.js debug port

CMD ["npm", "run", "dev"]
```

Approach 3: Attaching a Node.js Debugger

Node.js has a built-in debugging protocol. You can attach VS Code or Chrome DevTools to a running Node.js process inside a container.

Step 1: Start Node.js in debug mode

Update `package.json`:

```json
{
  "scripts": {
    "dev": "nodemon server.js",
    "debug": "nodemon --inspect=0.0.0.0:9229 server.js"
  }
}
```

The `--inspect=0.0.0.0:9229` flag tells Node.js to:

- Start the debug server.
- Listen on all interfaces (`0.0.0.0`), not just localhost. This is necessary because the debugger connects from outside the container.
- Use port 9229 (the default Node.js debug port).

Step 2: Expose the debug port in Compose

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
      - "9229:9229"    # Debug port
    volumes:
      - ./api:/app
      - /app/node_modules
    command: npm run debug
```

Step 3: Configure VS Code

Create `.vscode/launch.json` in your project root:

```json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Attach to Docker",
      "type": "node",
      "request": "attach",
      "port": 9229,
      "address": "localhost",
      "localRoot": "${workspaceFolder}/api",
      "remoteRoot": "/app",
      "restart": true
    }
  ]
}
```

Let us explain the key fields:

- `request: "attach"`: Attach to an already-running process (not launch a new one).
- `port: 9229`: The debug port we exposed.
- `localRoot`: Where the source code lives on your host machine.
- `remoteRoot`: Where the source code lives inside the container.
- `restart: true`: Reattach if the server restarts (from nodemon).

Step 4: Start debugging

1. Run `docker compose up -d`.
2. In VS Code, open the Run and Debug panel (Ctrl+Shift+D or Cmd+Shift+D).
3. Select "Attach to Docker" and click the green play button.
4. Set breakpoints in your code.

5. Make a request to your API. VS Code will pause at the breakpoint.

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│   VS Code (your host)                                │
│   ┌──────────────────┐                               │
│   │ Breakpoint at    │                               │
│   │ line 42 of       │                               │
│   │ server.js        │                               │
│   └──────────────────┘                               │
│            │                                         │
│            │  Debug protocol (port 9229)             │
│            ▼                                         │
│   ┌──────────────────┐                               │
│   │ Container "api"  │                               │
│   │                  │                               │
│   │ Node.js paused at │                              │
│   │ line 42          │                               │
│   │                  │                               │
│   │ Variables, call  │                               │
│   │ stack visible in │                               │
│   │ VS Code          │                               │
│   └──────────────────┘                               │
│                                                      │
└──────────────────────────────────────────────────────┘
```

A Complete Development Setup

Here is a full development Compose configuration that brings everything together:

`docker-compose.yml` (base):

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DB_HOST: database
      DB_USER: appuser
      DB_PASSWORD: apppass
      DB_NAME: appdb
      REDIS_URL: redis://cache:6379
    depends_on:
      database:
        condition: service_healthy
```

```
      cache:
        condition: service_healthy
    networks:
      - app-network

  database:
    image: postgres:16
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  cache:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

volumes:
  db-data:

networks:
  app-network:
```

`docker-compose.override.yml` (development overrides, loaded automatically):

```yaml
services:
  api:
    volumes:
      - ./api:/app
      - /app/node_modules
    ports:
      - "9229:9229"          # Debug port
    command: npm run debug
    environment:
      NODE_ENV: development
      DEBUG: "*"

  database:
```

```
    ports:
      - "5432:5432"          # Access from host for DB GUI tools

  cache:
    ports:
      - "6379:6379"          # Access from host for Redis CLI
```

`compose.production.yml` (production overrides, loaded explicitly):

```yaml
services:
  api:
    command: npm start
    environment:
      NODE_ENV: production
    restart: always
    deploy:
      resources:
        limits:
          memory: 512M

  database:
    restart: always
    # No host port mapping

  cache:
    restart: always
    # No host port mapping
```

Daily Development Workflow

```bash
# Start development environment
docker compose up -d

# Watch logs while developing
docker compose logs -f api

# After changing code: nothing to do!
# nodemon restarts automatically

# After adding a new npm package:
docker compose exec api npm install new-package
docker compose up -d --build    # Rebuild to include new package

# Run tests
docker compose exec api npm test

# Debug: set breakpoint in VS Code and attach debugger
```

```
# Stop at end of day
docker compose down

# Start fresh (reset database)
docker compose down -v
docker compose up -d
```

Common Mistakes

Mistake 1: Forgetting to Bind Mount During Development

```yaml
# Wrong: code is baked into the image, changes require rebuild
services:
  api:
    build: ./api
    command: npm run dev
    # Missing volumes! Every change needs `docker compose build`

# Right: bind mount for instant changes
services:
  api:
    build: ./api
    volumes:
      - ./api:/app
      - /app/node_modules
    command: npm run dev
```

Mistake 2: Not Handling node_modules

```yaml
# Wrong: bind mount overwrites node_modules
services:
  api:
    volumes:
      - ./api:/app          # This wipes out /app/node_modules!

# Right: protect node_modules with an anonymous volume
services:
  api:
    volumes:
      - ./api:/app
      - /app/node_modules   # Preserve container's node_modules
```

Mistake 3: Not Binding to 0.0.0.0

Many development servers bind to `127.0.0.1` (localhost) by default. Inside a container, `127.0.0.1` means "inside this container only," so connections from outside the container are refused.

```bash
# Wrong: server only accessible inside the container
node server.js --host 127.0.0.1

# Right: server accessible from outside the container
node server.js --host 0.0.0.0
```

Or in your application code:

```javascript
// Wrong
app.listen(3000, '127.0.0.1');

// Right
app.listen(3000, '0.0.0.0');
```

Mistake 4: Using Production Compose File for Development

```bash
# Wrong: uses the production file which has no bind mounts
docker compose -f docker-compose.yml -f compose.production.yml up -d

# Right: just use docker compose up (loads override automatically)
docker compose up -d
```

Mistake 5: Leaving Debug Ports Open in Production

```yaml
# DANGER: debug port exposed in production
services:
  api:
    ports:
      - "3000:3000"
      - "9229:9229"    # This should ONLY be in the dev override!
```

Always put debug ports in the override file, never in the base file.

Best Practices

1. Keep the base Compose file environment-agnostic. It should work for both development and production. Put environment-specific settings in override files.
2. Use `docker-compose.override.yml` for development. It is loaded automatically, so your development workflow is just `docker compose up -d`.
3. Use bind mounts only in development. Production should use the code baked into the image.
4. Always protect language-specific dependency directories (`node_modules`, `vendor`, `venv`, etc.) with anonymous volumes when using bind mounts.
5. Use hot reload tools (nodemon, Flask debug mode, etc.) to avoid manual restarts.
6. Expose debug ports only in development. Never expose debug ports in production.
7. Use YAML anchors to avoid duplicating configuration across services.
8. Add `.dockerignore` to prevent unnecessary files from being included in builds (especially `node_modules`, `.git`, and IDE files).

Quick Summary

Docker Compose provides powerful tools for development workflows. Bind mounts sync your source code from the host into containers for instant feedback. The `docker compose watch` feature offers file syncing with automatic rebuild triggers. Override files (`docker-compose.override.yml`) let you maintain separate development and production configurations without duplicating the base setup. YAML anchors reduce configuration repetition. For debugging, you can attach VS Code or Chrome DevTools to Node.js running inside containers by exposing the debug port and configuring the source mapping.

Key Points

- Bind mounts (`./src:/app`) sync host files into containers for live code reloading.
- Protect dependency directories (like `node_modules`) with anonymous volumes.
- `docker compose watch` provides file sync with rebuild triggers for dependency changes.
- `docker-compose.override.yml` is automatically loaded and should contain development settings.

- Use `docker compose -f` to explicitly specify which files to combine.
- Override files merge with (not replace) the base file.
- Hot reload tools (nodemon, Flask debug mode) automatically restart on code changes.
- Bind your development server to `0.0.0.0`, not `127.0.0.1`, for access from outside the container.
- Expose debug ports only in development override files, never in the base or production files.
- YAML anchors (`&name` and `*name`) reduce configuration duplication.

Practice Questions

1. Why do you need an anonymous volume for `/app/node_modules` when using bind mounts? What would happen without it?
2. Explain the difference between `docker compose watch` with `action: sync` and `action: rebuild`. When would you use each one?
3. You have a base `docker-compose.yml` and a `docker-compose.override.yml`. If both files define an `environment` section for the same service, what happens? Are the values merged or replaced?
4. Why must a development server bind to `0.0.0.0` instead of `127.0.0.1` inside a Docker container?
5. Describe how you would set up a debugging workflow for a Node.js application running inside a Docker container.

Exercises

Exercise 1: Hot Reload Setup

1. Create a simple Node.js Express application that returns "Hello World" on GET /.
2. Write a `docker-compose.yml` with bind mounts and nodemon for hot reload.
3. Start the stack and verify the page works.
4. Change the response text to "Hello Docker" without restarting.
5. Refresh the browser and verify the change appears.

Exercise 2: Development and Production Configs

1. Create a base `docker-compose.yml` for a Node.js app with PostgreSQL.
2. Create a `docker-compose.override.yml` that adds bind mounts, debug port, and `npm run dev`.
3. Create a `compose.production.yml` that uses `npm start`, `NODE_ENV=production`, and no bind mounts.
4. Start in development mode and verify bind mounts work.
5. Stop, then start in production mode using `-f` flags and verify there are no bind mounts.

Exercise 3: Attaching a Debugger

1. Set up a Node.js application in Docker with the debug port exposed (9229).
2. Configure VS Code with a `launch.json` that attaches to the containerized app.
3. Set a breakpoint in your request handler.
4. Make a request and verify VS Code pauses at the breakpoint.
5. Inspect variables and step through the code.

What Is Next?

You now have a professional development workflow with Docker Compose. In the next chapter, you will build a complete full-stack project from scratch: a React frontend, a Spring Boot API, a PostgreSQL database, and an Nginx reverse proxy. This is the kind of setup you will encounter in real-world projects, and you will create the entire stack with Docker Compose.

Chapter 18: Docker Compose in Practice

What You Will Learn

By the end of this chapter, you will be able to:

- Build a complete full-stack application with Docker Compose
- Create a React frontend served by Nginx
- Build a Spring Boot REST API with Gradle
- Set up a PostgreSQL database with initialization scripts
- Configure Nginx as a reverse proxy
- Write production-ready Dockerfiles for each service
- Create environment-specific Compose overrides
- Test the entire stack end-to-end
- Understand how all the pieces fit together in a real project

Why This Chapter Matters

This chapter brings together everything you have learned. You will build a full-stack application that mirrors what you would encounter at a real company. It has a frontend, a backend API, a database, and a reverse proxy — four services working together.

Think of this as your graduation project. You have learned about images, containers, Dockerfiles, volumes, networks, Compose, debugging, and development workflows. Now you are combining all of those skills into one cohesive project. When you finish this chapter, you will have a template you can use for any full-stack project.

The Architecture

We are building a simple bookstore application. Users can browse books and add new ones. Here is how the pieces fit together:

```
┌──────────────────────────────────────────────────────────┐
│                      Docker Compose                      │
│                                                          │
│   Browser Request                                        │
│        │                                                 │
│        ▼                                                 │
│   ┌──────────────┐                                       │
│   │   nginx      │  Reverse Proxy                        │
│   │   Port 80    │  Routes requests:                     │
│   │              │  /api/* ──> api:8080                  │
│   │              │  /*     ──> frontend files            │
│   └──────────────┘                                       │
│       │      │                                           │
│       ▼      ▼                                           │
│   ┌────────┐ ┌──────────────┐                            │
│   │frontend│ │   api        │  Spring Boot               │
│   │ (files)│ │   Port 8080  │  REST API                  │
│   │ React  │ │              │  /api/books                │
│   │ static │ └──────────────┘                            │
│   │ files  │        │                                    │
│   └────────┘        ▼                                    │
│              ┌──────────────┐                            │
│              │  database    │  PostgreSQL                │
│              │  Port 5432   │  Stores books              │
│              └──────────────┘                            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Why Nginx as a reverse proxy?

A reverse proxy is a server that sits in front of your other servers and routes requests to the right place. Without a reverse proxy, your frontend and API would run on different ports (for example, `localhost:3000` for the frontend and `localhost:8080` for the API). This causes cross-origin issues in browsers.

With Nginx as a reverse proxy, everything goes through a single port (80). Nginx decides where to send each request:

- Requests starting with `/api/` go to the Spring Boot API.
- All other requests serve the React frontend files.

```
┌──────────────────────────────────────────────┐
│   Without Reverse Proxy:                     │
│                                              │
│   Browser -> localhost:3000  (frontend)      │
│   Browser -> localhost:8080  (api)           │
│   Problem: Cross-origin! Browser blocks it.  │
│                                              │
├──────────────────────────────────────────────┤
│   With Reverse Proxy:                        │
│                                              │
│   Browser -> localhost:80    (nginx)         │
│   nginx  -> /api/* -> api:8080               │
│   nginx  -> /*     -> static files           │
│   Everything through one port. No CORS!      │
│                                              │
└──────────────────────────────────────────────┘
```

Project Structure

Here is the complete project layout:

```
bookstore/
├── docker-compose.yml
├── compose.override.yml
├── compose.production.yml
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── public/
│   │   └── index.html
│   └── src/
│       ├── App.js
│       ├── App.css
│       └── index.js
├── api/
│   ├── Dockerfile
│   ├── build.gradle
│   ├── settings.gradle
│   └── src/
│       └── main/
│           ├── java/
│           │   └── com/
│           │       └── bookstore/
│           │           ├── BookstoreApplication.java
│           │           ├── Book.java
│           │           ├── BookRepository.java
│           │           └── BookController.java
│           └── resources/
```

```
│               └── application.properties
├── nginx/
│   └── nginx.conf
└── database/
    └── init.sql
```

Let us build each piece step by step.

Step 1: The Database

Database Initialization Script

Create `database/init.sql`:

```sql
-- Create the books table
CREATE TABLE IF NOT EXISTS books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    isbn VARCHAR(20),
    published_year INTEGER,
    description TEXT DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert sample data
INSERT INTO books (title, author, isbn, published_year, description) VALUES
    ('Clean Code', 'Robert C. Martin', '978-0132350884', 2008,
     'A handbook of agile software craftsmanship'),
    ('The Pragmatic Programmer', 'David Thomas, Andrew Hunt', '978-0135957059', 2019,
     'Your journey to mastery'),
    ('Design Patterns', 'Gang of Four', '978-0201633610', 1994,
     'Elements of reusable object-oriented software'),
    ('Refactoring', 'Martin Fowler', '978-0134757599', 2018,
     'Improving the design of existing code'),
    ('Domain-Driven Design', 'Eric Evans', '978-0321125217', 2003,
     'Tackling complexity in the heart of software');
```

This script runs the first time the PostgreSQL container starts with an empty data volume. It creates the `books` table and inserts five sample books.

Step 2: The Spring Boot API

Build Configuration

Create `api/build.gradle`:

```groovy
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.2.1'
    id 'io.spring.dependency-management' version '1.1.4'
}

group = 'com.bookstore'
version = '1.0.0'

java {
    sourceCompatibility = '17'
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
    runtimeOnly 'org.postgresql:postgresql'
}
```

Let us explain each dependency:

- spring-boot-starter-web: Provides everything needed to build REST APIs (embedded web server, JSON handling, etc.).
- spring-boot-starter-data-jpa: Provides database access using JPA (Java Persistence API), which lets you work with database rows as Java objects.
- spring-boot-starter-actuator: Provides health check endpoints (`/actuator/health`) that Docker can use for health checks.
- postgresql: The PostgreSQL JDBC driver that lets Java talk to PostgreSQL.

Create `api/settings.gradle`:

```groovy
rootProject.name = 'bookstore-api'
```

Application Properties

Create `api/src/main/resources/application.properties`:

```properties
# Server configuration
server.port=8080

# Database configuration
# "database" is the Docker Compose service name
spring.datasource.url=jdbc:postgresql://${DB_HOST:database}:${DB_PORT:5432}/${DB_NAME:b...
spring.datasource.username=${DB_USER:bookuser}
spring.datasource.password=${DB_PASSWORD:bookpass}

# JPA configuration
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false

# Actuator health endpoint
management.endpoints.web.exposure.include=health
```

The `${DB_HOST:database}` syntax means "use the `DB_HOST` environment variable, or default to `database` if it is not set." This lets us configure the application through Docker Compose environment variables.

Java Source Code

Create `api/src/main/java/com/bookstore/BookstoreApplication.java`:

```java
package com.bookstore;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookstoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(BookstoreApplication.class, args);
    }
}
```

This is the entry point for the Spring Boot application. The `@SpringBootApplication` annotation tells Spring to auto-configure everything.

Create `api/src/main/java/com/bookstore/Book.java`:

```java
package com.bookstore;

import jakarta.persistence.*;
```

```
import java.time.LocalDateTime;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    private String isbn;

    @Column(name = "published_year")
    private Integer publishedYear;

    private String description;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Default constructor (required by JPA)
    public Book() {}

    // Constructor for creating new books
    public Book(String title, String author, String isbn,
                Integer publishedYear, String description) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.publishedYear = publishedYear;
        this.description = description;
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public Integer getPublishedYear() { return publishedYear; }
    public void setPublishedYear(Integer publishedYear) {
        this.publishedYear = publishedYear;
```

```
    }

    public String getDescription() { return description; }
    public void setDescription(String description) {
        this.description = description;
    }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }
}
```

This is a JPA entity. It maps the `books` table in PostgreSQL to a Java class. Each column in the table corresponds to a field in the class. The `@Entity` annotation tells Spring this class represents a database table.

Create `api/src/main/java/com/bookstore/BookRepository.java`:

```java
package com.bookstore;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface BookRepository extends JpaRepository<Book, Long> {
    List<Book> findByAuthorContainingIgnoreCase(String author);
    List<Book> findByTitleContainingIgnoreCase(String title);
}
```

This is a Spring Data repository. By extending `JpaRepository`, Spring automatically provides methods like `findAll()`, `findById()`, `save()`, and `deleteById()`. The custom methods (`findByAuthorContainingIgnoreCase`) are automatically implemented by Spring based on the method name.

Create `api/src/main/java/com/bookstore/BookController.java`:

```java
package com.bookstore;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/books")
```

```
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    // GET /api/books - List all books
    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    // GET /api/books/{id} - Get a single book
    @GetMapping("/{id}")
    public ResponseEntity<Book> getBook(@PathVariable Long id) {
        return bookRepository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // POST /api/books - Create a new book
    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book book) {
        Book saved = bookRepository.save(book);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    // DELETE /api/books/{id} - Delete a book
    @DeleteMapping("/{id}")
    public ResponseEntity<Map<String, String>> deleteBook(@PathVariable Long id) {
        if (!bookRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        bookRepository.deleteById(id);
        return ResponseEntity.ok(Map.of("message", "Book deleted"));
    }

    // GET /api/books/search?title=xxx - Search books by title
    @GetMapping("/search")
    public List<Book> searchBooks(@RequestParam(required = false) String title,
                                  @RequestParam(required = false) String author) {
        if (title != null) {
            return bookRepository.findByTitleContainingIgnoreCase(title);
        }
        if (author != null) {
            return bookRepository.findByAuthorContainingIgnoreCase(author);
        }
        return bookRepository.findAll();
    }
}
```

This is the REST controller. It defines the API endpoints:

- `GET /api/books` — Returns all books.
- `GET /api/books/{id}` — Returns one book.
- `POST /api/books` — Creates a new book.
- `DELETE /api/books/{id}` — Deletes a book.
- `GET /api/books/search?title=xxx` — Searches by title or author.

API Dockerfile

Create `api/Dockerfile`:

```dockerfile
# ── Stage 1: Build ────────────────────────────────
FROM gradle:8-jdk17 AS builder

WORKDIR /app

# Copy build files first (for dependency caching)
COPY build.gradle settings.gradle ./

# Download dependencies (cached unless build files change)
RUN gradle dependencies --no-daemon

# Copy source code
COPY src ./src

# Build the application
RUN gradle bootJar --no-daemon

# ── Stage 2: Run ──────────────────────────────────
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Copy the built JAR from the builder stage
COPY --from=builder /app/build/libs/*.jar app.jar

# Expose the application port
EXPOSE 8080

# Health check using Spring Actuator
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:8080/actuator/health || exit 1

# Run the application
CMD ["java", "-jar", "app.jar"]
```

This is a multi-stage build. Let us explain:

Stage 1 (builder): Uses a full Gradle image with JDK 17 to compile the Java application into a JAR file. The `gradle bootJar` command creates an executable JAR that contains everything the application needs to run.

Stage 2 (runtime): Uses a minimal JRE image (just the Java runtime, no build tools). It copies only the compiled JAR from Stage 1. This makes the final image much smaller — around 200MB instead of 800MB+.

The `start_period: 30s` gives Spring Boot 30 seconds to start up. Java applications take longer to start than Node.js apps because the JVM needs to initialize.

Step 3: The React Frontend

Package Configuration

Create `frontend/package.json`:

```json
{
  "name": "bookstore-frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  },
  "browserslist": {
    "production": [">0.2%", "not dead", "not op_mini all"],
    "development": ["last 1 chrome version"]
  }
}
```

HTML Template

Create `frontend/public/index.html`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
```

```
    <title>Bookstore</title>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```

React Application

Create `frontend/src/index.js`:

```javascript
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './App.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Create `frontend/src/App.js`:

```javascript
import React, { useState, useEffect } from 'react';

function App() {
  const [books, setBooks] = useState([]);
  const [title, setTitle] = useState('');
  const [author, setAuthor] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Fetch books on page load
  useEffect(() => {
    fetchBooks();
  }, []);

  const fetchBooks = async () => {
    try {
      setLoading(true);
      const response = await fetch('/api/books');
      if (!response.ok) throw new Error('Failed to fetch books');
      const data = await response.json();
      setBooks(data);
      setError(null);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
```

```
  };

  const addBook = async (e) => {
    e.preventDefault();
    if (!title || !author) return;

    try {
      const response = await fetch('/api/books', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title, author }),
      });
      if (!response.ok) throw new Error('Failed to add book');
      setTitle('');
      setAuthor('');
      fetchBooks();
    } catch (err) {
      setError(err.message);
    }
  };

  const deleteBook = async (id) => {
    try {
      const response = await fetch(`/api/books/${id}`, {
        method: 'DELETE',
      });
      if (!response.ok) throw new Error('Failed to delete book');
      fetchBooks();
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div className="app">
      <header>
        <h1>Bookstore</h1>
        <p>A full-stack Docker Compose application</p>
      </header>

      <section className="add-book">
        <h2>Add a Book</h2>
        <form onSubmit={addBook}>
          <input
            type="text"
            placeholder="Title"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            required
          />
          <input
            type="text"
            placeholder="Author"
            value={author}
            onChange={(e) => setAuthor(e.target.value)}
```

```
            required
          />
          <button type="submit">Add Book</button>
        </form>
      </section>

      <section className="book-list">
        <h2>All Books</h2>
        {loading && <p>Loading books...</p>}
        {error && <p className="error">Error: {error}</p>}
        {!loading && books.length === 0 && <p>No books found.</p>}
        <div className="books">
          {books.map((book) => (
            <div key={book.id} className="book-card">
              <h3>{book.title}</h3>
              <p className="author">by {book.author}</p>
              {book.publishedYear && (
                <p className="year">Published: {book.publishedYear}</p>
              )}
              {book.description && (
                <p className="description">{book.description}</p>
              )}
              <button
                className="delete-btn"
                onClick={() => deleteBook(book.id)}
              >
                Delete
              </button>
            </div>
          ))}
        </div>
      </section>
    </div>
  );
}

export default App;
```

Notice that the API calls use relative URLs like `/api/books` instead of `http://localhost:8080/api/books`. This works because Nginx (our reverse proxy) handles the routing. The browser sends all requests to Nginx, and Nginx forwards `/api/*` requests to the Spring Boot API.

Create `frontend/src/App.css`:

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

```
body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background-color: #f5f5f5;
  color: #333;
}

.app {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

header {
  text-align: center;
  margin-bottom: 30px;
  padding: 20px;
  background: #2c3e50;
  color: white;
  border-radius: 8px;
}

header h1 {
  margin-bottom: 5px;
}

header p {
  opacity: 0.8;
  font-size: 14px;
}

.add-book {
  background: white;
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 20px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.add-book h2 {
  margin-bottom: 15px;
}

.add-book form {
  display: flex;
  gap: 10px;
}

.add-book input {
  flex: 1;
  padding: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 14px;
}
```

```
.add-book button {
  padding: 10px 20px;
  background: #2c3e50;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
}

.add-book button:hover {
  background: #34495e;
}

.book-list h2 {
  margin-bottom: 15px;
}

.books {
  display: grid;
  gap: 15px;
}

.book-card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.book-card h3 {
  margin-bottom: 5px;
  color: #2c3e50;
}

.book-card .author {
  color: #666;
  font-style: italic;
  margin-bottom: 5px;
}

.book-card .year {
  color: #888;
  font-size: 13px;
  margin-bottom: 5px;
}

.book-card .description {
  color: #555;
  font-size: 14px;
  margin-bottom: 10px;
}

.delete-btn {
```

```
  padding: 5px 15px;
  background: #e74c3c;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 13px;
}

.delete-btn:hover {
  background: #c0392b;
}

.error {
  color: #e74c3c;
  padding: 10px;
  background: #ffeaea;
  border-radius: 4px;
}
```

Frontend Dockerfile

Create `frontend/Dockerfile`:

```dockerfile
# ── Stage 1: Build ──────────────────────────────
FROM node:20-alpine AS builder

WORKDIR /app

# Copy package files
COPY package.json ./

# Install dependencies
RUN npm install

# Copy source code
COPY public ./public
COPY src ./src

# Build the React app (creates /app/build)
RUN npm run build

# ── Stage 2: Serve ──────────────────────────────
FROM nginx:alpine

# Copy the built React app to nginx's serve directory
COPY --from=builder /app/build /usr/share/nginx/html

# Expose port 80
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

This is another multi-stage build:

Stage 1: Uses Node.js to build the React application. `npm run build` creates optimized static files (HTML, CSS, JavaScript) in the `/app/build` directory.

Stage 2: Uses a minimal Nginx image to serve the static files. The built files are copied to Nginx's default serve directory. The final image contains no Node.js, no source code, and no `node_modules` — just the optimized static files and Nginx.

Step 4: The Nginx Reverse Proxy

Create `nginx/nginx.conf`:

```nginx
# Main Nginx configuration for the Bookstore application

# Run with a single worker process
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    # Include MIME types so files are served with correct Content-Type
    include       /etc/nginx/mime.types;
    default_type  application/octet-stream;

    # Logging format
    log_format main '$remote_addr - $remote_user [$time_local] '
                    '"$request" $status $body_bytes_sent '
                    '"$http_referer" "$http_user_agent"';

    access_log /var/log/nginx/access.log main;
    error_log  /var/log/nginx/error.log warn;

    # Performance settings
    sendfile        on;
    keepalive_timeout  65;

    # ── Upstream: Spring Boot API ────────────────────
    # "api" is the Docker Compose service name
    upstream api_backend {
```

```
        server api:8080;
    }

    # ── Server Configuration ──────────────────────────
    server {
        listen 80;
        server_name localhost;

        # ── API requests: forward to Spring Boot ──────
        # Any request starting with /api/ goes to the backend
        location /api/ {
            proxy_pass http://api_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # ── Health check endpoint for the API ─────────
        location /actuator/ {
            proxy_pass http://api_backend;
        }

        # ── Frontend: serve React static files ────────
        location / {
            root /usr/share/nginx/html;
            index index.html;
            # For single-page app: serve index.html for all routes
            try_files $uri $uri/ /index.html;
        }

        # ── Error pages ───────────────────────────────
        error_page 500 502 503 504 /50x.html;
        location = /50x.html {
            root /usr/share/nginx/html;
        }
    }
}
```

Let us walk through the key sections:

`upstream api_backend`: Defines the backend server. `api:8080` uses the Compose service name `api` and the port the Spring Boot app listens on. Docker's DNS resolves `api` to the correct container IP.

`location /api/`: Any request with a URL starting with `/api/` is forwarded to the Spring Boot API. The `proxy_set_header` lines pass useful information about the original request to the backend.

`location /`: All other requests serve files from `/usr/share/nginx/html`, which is where the React build files live. The `try_files` directive is important for single-page apps — if the requested file does not exist, it serves `index.html`, allowing React Router to handle the URL.

Step 5: The Docker Compose Files

Base Configuration

Create `docker-compose.yml`:

```yaml
services:
  # ─────────────────────────────────────────────────────
  # Nginx — Reverse Proxy
  # Routes /api/* to the Spring Boot API
  # Serves React frontend files for everything else
  # ─────────────────────────────────────────────────────
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - frontend-build:/usr/share/nginx/html:ro
    depends_on:
      api:
        condition: service_healthy
      frontend:
        condition: service_completed_successfully
    networks:
      - frontend-net

  # ─────────────────────────────────────────────────────
  # Frontend — React Application
  # Builds the React app and outputs to a shared volume
  # This is a "build" service that exits after building
  # ─────────────────────────────────────────────────────
  frontend:
    build: ./frontend
    volumes:
      - frontend-build:/usr/share/nginx/html

  # ─────────────────────────────────────────────────────
  # API — Spring Boot Application
  # REST API at /api/books
  # ─────────────────────────────────────────────────────
  api:
    build: ./api
    environment:
      DB_HOST: database
      DB_PORT: 5432
      DB_USER: bookuser
```

```
      DB_PASSWORD: bookpass
      DB_NAME: bookstore
    depends_on:
      database:
        condition: service_healthy
    networks:
      - frontend-net
      - backend-net

  # ────────────────────────────────────────────────────
  # Database — PostgreSQL
  # ────────────────────────────────────────────────────
  database:
    image: postgres:16
    environment:
      POSTGRES_USER: bookuser
      POSTGRES_PASSWORD: bookpass
      POSTGRES_DB: bookstore
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bookuser -d bookstore"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    networks:
      - backend-net

# ────────────────────────────────────────────────────
# Volumes
# ────────────────────────────────────────────────────
volumes:
  db-data:             # Persistent database storage
  frontend-build:      # Built React files shared between frontend and nginx

# ────────────────────────────────────────────────────
# Networks
# ────────────────────────────────────────────────────
networks:
  frontend-net:        # nginx <-> api communication
    driver: bridge
  backend-net:         # api <-> database communication
    driver: bridge
```

Let us explain the architecture decisions:

Network isolation: Two networks separate concerns. The `frontend-net` connects nginx and the api. The `backend-net` connects the api and the database. Nginx cannot reach the database directly — it must go through the api.

The frontend service: This service builds the React app and exits. It uses the `service_completed_successfully` condition in nginx's `depends_on` so nginx waits for the build to finish before starting. The built files are placed in the `frontend-build` volume, which nginx reads from.

Shared volume for frontend files: The `frontend-build` volume is shared between the `frontend` (which writes the built files) and `nginx` (which reads and serves them).

Development Override

Create `compose.override.yml`:

```yaml
services:
  api:
    ports:
      - "8080:8080"      # Direct API access for debugging
    environment:
      SPRING_JPA_SHOW_SQL: "true"     # Show SQL queries in logs

  database:
    ports:
      - "5432:5432"      # Direct database access for GUI tools
```

Production Configuration

Create `compose.production.yml`:

```yaml
services:
  nginx:
    restart: always

  api:
    restart: always
    environment:
      SPRING_JPA_SHOW_SQL: "false"
    deploy:
      resources:
        limits:
          memory: 512M

  database:
    restart: always
    # No port mapping in production
    deploy:
      resources:
        limits:
          memory: 256M
```

Step 6: Build and Run

Building Everything

From the `bookstore` directory:

```bash
docker compose up -d --build
```

Output:

```
[+] Building 45.2s (25/25) FINISHED
 => [frontend] Building...
 => [api] Building...
[+] Running 5/5
 ✔ Network bookstore_frontend-net  Created
 ✔ Network bookstore_backend-net   Created
 ✔ Container bookstore-database-1  Started
 ✔ Container bookstore-frontend-1  Started
 ✔ Container bookstore-api-1       Started
 ✔ Container bookstore-nginx-1     Started
```

The first build takes a few minutes because it downloads dependencies, compiles Java code, and builds the React app. Subsequent builds are much faster due to Docker's layer caching.

Checking Status

```bash
docker compose ps
```

Output:

```
NAME                     IMAGE              STATUS                      PORTS
bookstore-api-1          bookstore-api      Up 30s (healthy)            0.0.0.0:8080->8...
bookstore-database-1     postgres:16        Up 45s (healthy)            0.0.0.0:5432->5...
bookstore-frontend-1     bookstore-frontend Exited (0) 20s ago
bookstore-nginx-1        nginx:alpine       Up 10s                      0.0.0.0:80->80/tcp
```

Notice that the `frontend` service shows "Exited (0)" — this is expected. It built the React app, placed the files in the shared volume, and exited. Exit code 0 means it completed

successfully.

Step 7: Testing the Application

Access the Frontend

Open your browser and go to `http://localhost`. You should see the Bookstore application with the five sample books.

Test the API Directly

```bash
# List all books
curl http://localhost/api/books
```

Output:

```json
[
  {
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "978-0132350884",
    "publishedYear": 2008,
    "description": "A handbook of agile software craftsmanship",
    "createdAt": "2024-01-15T10:00:00"
  },
  ...
]
```

```bash
# Add a new book
curl -X POST http://localhost/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Docker in Practice", "author": "Ian Miell"}'
```

```bash
# Search for a book
curl "http://localhost/api/books/search?title=clean"
```

```bash
# Check API health
curl http://localhost/actuator/health
```

Output:

```json
{
  "status": "UP"
}
```

Check the Logs

```bash
# All services
docker compose logs --tail 20

# Just the API
docker compose logs api

# Follow nginx access logs
docker compose logs -f nginx
```

Verify Network Isolation

```bash
# The API can reach the database
docker compose exec api ping -c 1 database
# Should succeed

# Nginx cannot reach the database
docker compose exec nginx ping -c 1 database
# Should fail: "bad address" or "Name does not resolve"
```

Step 8: Environment-Specific Deployment

Development

```bash
# Uses docker-compose.yml + compose.override.yml automatically
docker compose up -d --build
```

Features: API port exposed, database port exposed, SQL logging enabled.

Production

```bash
# Explicitly specify production file (skips override)
docker compose -f docker-compose.yml -f compose.production.yml up -d --build
```

Features: Only port 80 exposed, auto-restart enabled, memory limits set, no SQL logging.

How All the Pieces Connect

Let us trace a complete request through the system:

```
1. User opens http://localhost in browser

2. Browser sends GET / to nginx (port 80)

3. nginx sees "/" and serves the React index.html
   from /usr/share/nginx/html

4. Browser loads React app and JavaScript

5. React app calls GET /api/books

6. Browser sends GET /api/books to nginx (port 80)

7. nginx sees "/api/" prefix and forwards to api:8080

8. Spring Boot receives GET /api/books

9. Spring Boot queries PostgreSQL at database:5432

10. PostgreSQL returns the book rows

11. Spring Boot converts rows to JSON and responds

12. nginx forwards the JSON response to the browser

13. React renders the book list on the page
```

```
┌──────────────────────────────────────────────────────────┐
│                   Request Flow                           │
│                                                          │
```

```
│   Browser                                              │
│     │                                                  │
│     │ GET /api/books                                   │
│     ▼                                                  │
│   nginx:80                                             │
│     │                                                  │
│     │ proxy_pass (URL starts with /api/)               │
│     ▼                                                  │
│   api:8080                                             │
│     │                                                  │
│     │ JDBC query                                       │
│     ▼                                                  │
│   database:5432                                        │
│     │                                                  │
│     │ SQL result                                       │
│     ▼                                                  │
│   api:8080                                             │
│     │                                                  │
│     │ JSON response                                    │
│     ▼                                                  │
│   nginx:80                                             │
│     │                                                  │
│     │ Forward response                                 │
│     ▼                                                  │
│   Browser                                              │
│     │                                                  │
│     │ Render book list                                 │
│     ▼                                                  │
│   User sees the books!                                 │
│                                                        │
└────────────────────────────────────────────────────────┘
```

Common Mistakes

Mistake 1: Using Absolute API URLs in React

```javascript
// Wrong: hardcoded URL breaks in Docker
const response = await fetch('http://localhost:8080/api/books');

// Right: relative URL works with reverse proxy
const response = await fetch('/api/books');
```

Relative URLs let Nginx handle the routing. The browser sends the request to wherever the page was loaded from (Nginx on port 80), and Nginx forwards it to the API.

Mistake 2: Forgetting try_files for Single-Page Apps

```nginx
# Wrong: direct URLs like /about return 404
location / {
    root /usr/share/nginx/html;
}

# Right: fallback to index.html for client-side routing
location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html;
}
```

Without `try_files`, if a user navigates directly to `http://localhost/about`, Nginx looks for a file called `/about` which does not exist. `try_files` falls back to `index.html`, which loads the React app, and React Router handles the URL.

Mistake 3: Not Waiting for Build Services

```yaml
# Wrong: nginx starts before frontend finishes building
nginx:
  depends_on:
    - frontend    # Only waits for the container to start, not finish

# Right: wait for the build to complete
nginx:
  depends_on:
    frontend:
      condition: service_completed_successfully
```

Mistake 4: Missing Network Connections

```yaml
# Wrong: api and nginx on different networks — nginx cannot reach api
api:
  networks:
    - backend-net

nginx:
  networks:
    - frontend-net

# Right: api must be on both networks
api:
  networks:
    - frontend-net    # So nginx can reach it
    - backend-net     # So it can reach the database
```

Mistake 5: Exposing Database Ports in Production

```yaml
# DANGER: database accessible from the internet
database:
  ports:
    - "5432:5432"    # Only do this in development!
```

In production, the database should only be accessible from other containers on the same Docker network. Never expose database ports to the host in production.

Best Practices

1. Use multi-stage builds for all services. This keeps final images small and production-ready.
2. Use a reverse proxy (Nginx) in front of your application. It simplifies URL routing, handles static files efficiently, and avoids cross-origin issues.
3. Separate networks by tier — frontend, backend, database. Only the API should bridge between networks.
4. Use `service_completed_successfully` for build-only services that need to finish before dependent services start.
5. Keep credentials in environment variables, never in source code or Dockerfiles.
6. Use `.dockerignore` files in every service directory to exclude unnecessary files from the build context (node_modules, .git, build artifacts).
7. Use read-only mounts (`:ro`) for configuration files and frontend assets. Services that only need to read data should not be able to write.
8. Test each service independently before assembling the full stack. Verify the API works by itself before adding the frontend and proxy.
9. Start with simple and add complexity. Get two services working before adding the third and fourth.
10. Document your architecture with diagrams. As the number of services grows, it becomes harder to remember how they connect.

Quick Summary

In this chapter, you built a complete full-stack application using Docker Compose with four services: a React frontend (built and served as static files), a Spring Boot REST API, a PostgreSQL database, and an Nginx reverse proxy. The Nginx reverse proxy routes API requests to Spring Boot and serves the React frontend files for everything else. Network isolation ensures the database is only accessible from the API. Multi-stage builds keep all images small. Environment-specific Compose files let you switch between development and production configurations.

Key Points

- A reverse proxy (Nginx) routes requests to the correct backend service and serves static frontend files.
- Use relative URLs (`/api/books`) in frontend code so the reverse proxy can handle routing.
- Multi-stage Docker builds separate the build environment from the runtime environment.
- The `service_completed_successfully` condition waits for build services to finish.
- Network isolation (separate frontend and backend networks) improves security.
- Shared volumes let build services output files that other services consume.
- Spring Boot uses `application.properties` with `${ENV_VAR:default}` syntax for configuration.
- `try_files $uri $uri/ /index.html` in Nginx is required for single-page app routing.
- Development overrides expose extra ports; production overrides add restart policies and resource limits.
- Never expose database ports in production.

Practice Questions

1. Explain why the frontend service exits after building. How does Nginx get the React build files if the frontend container is no longer running?
2. Why do we use two separate networks (frontend-net and backend-net) instead of putting all services on one network?

3. A user navigates directly to `http://localhost/books/123` in their browser. Trace the request through the system. What role does `try_files` play?
4. The API service is on both `frontend-net` and `backend-net`. Why does it need to be on both networks? What would happen if it were only on `backend-net`?
5. Compare the development and production Compose configurations. What are the key differences, and why does each difference matter?

Exercises

Exercise 1: Add a Search Feature

1. Start the bookstore application.
2. Modify the React frontend to add a search bar that calls `GET /api/books/search?title=xxx`.
3. Rebuild only the frontend: `docker compose build frontend`.
4. Restart: `docker compose up -d`.
5. Test the search functionality in your browser.

Exercise 2: Add a Redis Cache

1. Add a Redis service to the `docker-compose.yml`.
2. Put Redis on the `backend-net` network.
3. Configure the Spring Boot API to use Redis for caching (add Spring Cache and Spring Data Redis dependencies).
4. Verify that repeated requests are served from the cache.

Exercise 3: Add an Admin Dashboard

1. Add an Adminer service to the Compose file for database management.
2. Put it on the `backend-net` network.
3. Optionally add a route in Nginx to proxy `/admin/` to Adminer.
4. Access the admin dashboard and manage books directly in the database.
5. Verify that changes in the database appear in the React frontend.

What Is Next?

Congratulations. You have built a complete, production-ready, full-stack application with Docker Compose. You understand how to create Dockerfiles, configure networks, manage volumes, set up reverse proxies, and handle different environments. In the upcoming chapters, you will learn how to publish your images to Docker Hub and other registries, set up continuous integration and deployment (CI/CD) pipelines, and eventually scale your applications with orchestration tools like Docker Swarm and Kubernetes.

Chapter 19: Docker Hub and Container Registries

What You Will Learn

By the end of this chapter, you will be able to:

- Understand what Docker Hub is and why it exists
- Create a Docker Hub account and log in from the terminal
- Tag images correctly for pushing to a registry
- Push your own images to Docker Hub
- Create and manage repositories on Docker Hub
- Set up automated builds from GitHub
- Work with private repositories
- Use alternative registries like GitHub Container Registry, AWS ECR, and Google GCR
- Run your own self-hosted registry

Why This Chapter Matters

Up until now, every image you have built lives only on your computer. If your hard drive crashes, those images are gone. If a teammate needs your image, they cannot get it. If you want to deploy to a server, you have no way to transfer the image.

Container registries solve all of these problems. Think of a registry as an app store for Docker images. Just like you download apps from the App Store or Google Play, Docker pulls images from registries. And just like developers publish apps to those stores, you can publish your images to a registry.

Docker Hub is the default registry — the one Docker talks to when you run `docker pull nginx` without specifying a location. But it is not the only option. Understanding registries is essential

because every production deployment starts with pushing an image to a registry and pulling it on the target server.

What Is Docker Hub?

Docker Hub is a cloud-based registry service provided by Docker, Inc. It is the world's largest container image library and the default registry for Docker.

Think of it this way:

```
+---------------------------------------------------+
|              The App Store Analogy                |
|                                                   |
|   App Store / Google Play  =  Docker Hub          |
|   Apps                     =  Docker Images       |
|   App Developer            =  Image Publisher      |
|   Downloading an App       =  docker pull          |
|   Publishing an App        =  docker push          |
|   App Listing              =  Repository           |
+---------------------------------------------------+
```

When you run `docker pull nginx`, Docker contacts Docker Hub at `hub.docker.com`, finds the `nginx` repository, and downloads the image. You have been using Docker Hub all along without realizing it.

Official Images vs User Images

Docker Hub hosts two types of images:

Official images have no username prefix. They are maintained by Docker or the software vendor:

```
nginx           <-- Official NGINX image
postgres        <-- Official PostgreSQL image
node            <-- Official Node.js image
python          <-- Official Python image
```

User images include a username or organization prefix:

```
myusername/webapp        <-- Your personal image
companyname/api-server   <-- An organization's image
```

```
bitnami/redis            <-- Bitnami's Redis image
```

The naming pattern is:

```
[registry-host/] [username/] image-name [:tag]

Examples:
  nginx:latest                              <-- Official, Docker Hub (default)
  myuser/webapp:v1.0                        <-- User image, Docker Hub
  ghcr.io/myuser/webapp:v1.0               <-- GitHub Container Registry
  123456789.dkr.ecr.us-east-1.amazonaws.com/webapp:v1.0  <-- AWS ECR
```

Creating a Docker Hub Account

Before you can push images, you need a free Docker Hub account.

Step 1: Go to `https://hub.docker.com` and click "Sign Up."

Step 2: Choose a Docker ID (username). This becomes your namespace for all images. Choose carefully — it is hard to change later.

Step 3: Verify your email address.

Your Docker ID is important because every image you push will be prefixed with it:

```
your-docker-id/image-name:tag
```

Logging In from the Terminal

To push images, you must authenticate with Docker Hub from your terminal.

docker login

```bash
docker login
```

Output:

```
Login with your Docker ID to push and pull images from Docker Hub.
If you don't have a Docker ID, head over to https://hub.docker.com

Username: myusername
Password:
Login Succeeded
```

Line-by-line explanation:

- `docker login` — Tells Docker to authenticate with the default registry (Docker Hub)
- `Username:` — Enter the Docker ID you created during sign-up
- `Password:` — Enter your password (characters will not appear as you type — this is normal and a security feature)
- `Login Succeeded` — You are now authenticated and can push images

Where Are Credentials Stored?

Docker stores your login credentials in a configuration file:

```bash
cat ~/.docker/config.json
```

Output:

```json
{
  "auths": {
    "https://index.docker.io/v1/": {
      "auth": "bXl1c2VybmFtZTpteXBhc3N3b3Jk"
    }
  }
}
```

> Security Warning: The `auth` field is your username and password encoded in Base64 — it is NOT encrypted. Anyone who can read this file can decode your credentials. On shared machines, use `docker login` only when needed and `docker logout` when finished.

Using Access Tokens (Recommended)

Instead of your password, use a Personal Access Token (PAT). Tokens are safer because:

- You can revoke a token without changing your password
- You can create tokens with limited permissions (read-only, read-write)

- You can create separate tokens for different machines or CI systems

To create a token:

1. Go to Docker Hub → Account Settings → Security → New Access Token
2. Give it a description (like "My Laptop" or "CI Server")
3. Choose the access level (Read, Read/Write, or Read/Write/Delete)
4. Copy the token immediately (you will not see it again)

Use the token instead of your password:

```bash
docker login -u myusername
```

When prompted for a password, paste the access token.

Logging Out

```bash
docker logout
```

Output:

```
Removing login credentials for https://index.docker.io/v1/
```

This removes the stored credentials from `~/.docker/config.json`.

Tagging Images for a Registry

Before you can push an image, it must be tagged with the correct name format. The tag tells Docker where to push the image.

The Tagging Format

```
registry-host/username/image-name:tag
```

For Docker Hub, the registry host is optional (it is the default):

```
username/image-name:tag
```

Using docker tag

The `docker tag` command creates a new name (tag) that points to an existing image. It does NOT create a copy — both names point to the same image data.

```bash
# First, build your image with any name
docker build -t my-webapp .

# Then tag it for Docker Hub
docker tag my-webapp myusername/my-webapp:v1.0
```

Think of it like a file shortcut or alias. The image data stays the same — you are just giving it an additional name that includes your Docker Hub username.

```
+-----------------------------------------+
|          Image Tagging Analogy          |
|                                         |
|   Original name:   my-webapp            |
|   Tagged name:     myusername/my-webapp:v1.0|
|                                         |
|   Both names point to the SAME image.   |
|   Like two shortcuts to the same file.  |
+-----------------------------------------+
```

Verify the Tag

```bash
docker images
```

Output:

```
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
my-webapp               latest    a1b2c3d4e5f6   5 minutes ago    150MB
myusername/my-webapp     v1.0      a1b2c3d4e5f6   5 minutes ago    150MB
```

Notice that both entries have the same IMAGE ID. They are the same image with two different names.

Tag During Build

You can skip the separate tagging step by using the correct name during the build:

```bash
docker build -t myusername/my-webapp:v1.0 .
```

This builds the image and immediately gives it the registry-ready name.

Pushing Images to Docker Hub

Once your image is tagged correctly, pushing is straightforward.

docker push

```bash
docker push myusername/my-webapp:v1.0
```

Output:

```
The push refers to repository [docker.io/myusername/my-webapp]
5f70bf18a086: Pushed
a3ed95caeb02: Pushed
e3b0c44298fc: Mounted from library/node
v1.0: digest: sha256:abc123def456... size: 1572
```

Line-by-line explanation:

- `The push refers to repository` — Confirms where the image is being sent
- `Pushed` — This layer was uploaded to Docker Hub
- `Mounted from library/node` — This layer already exists on Docker Hub (from the base image). Docker does not re-upload it — saving time and bandwidth
- `v1.0: digest: sha256:...` — A unique fingerprint for this exact image version

```
+-------------------------------------------------------+
|              Push Flow                                |
|                                                       |
|   Your Computer              Docker Hub               |
|   +-------------+           +------------------+      |
|   | my-webapp   |  push --> | myusername/      |      |
|   | v1.0        |           | my-webapp:v1.0   |      |
|   +-------------+           +------------------+      |
|                                                       |
```

```
|   Only NEW layers are uploaded.                        |
|   Layers from base images are shared (mounted).        |
+-------------------------------------------------------+
```

Pushing Multiple Tags

You can push multiple tags for the same image:

```bash
docker tag my-webapp myusername/my-webapp:v1.0
docker tag my-webapp myusername/my-webapp:latest

docker push myusername/my-webapp:v1.0
docker push myusername/my-webapp:latest
```

Or push all tags at once:

```bash
docker push myusername/my-webapp --all-tags
```

Understanding Repositories

A repository on Docker Hub is a collection of related images, identified by different tags. Think of it as a folder that holds all versions of one application.

```
+-----------------------------------------+
|   Repository: myusername/my-webapp        |
|                                           |
|   Tags (versions):                        |
|     v1.0    - First release               |
|     v1.1    - Bug fix                     |
|     v2.0    - Major update                |
|     latest  - Most recent                 |
|                                           |
|   All these are in ONE repository.        |
+-----------------------------------------+
```

Creating a Repository on Docker Hub

Repositories are created automatically when you push an image. But you can also create them manually through the web interface:

1. Go to Docker Hub → Repositories → Create Repository
2. Enter a name (like `my-webapp`)
3. Add a description
4. Choose visibility: Public (anyone can pull) or Private (only you)
5. Click Create

Repository Settings

Each repository has settings you can configure:

- Description — What this image does
- README — Detailed documentation (supports Markdown)
- Collaborators — Other users who can push to this repository
- Webhooks — Trigger actions when a new image is pushed
- Build settings — Configure automated builds from source code

---

Automated Builds

Automated builds link a Docker Hub repository to a source code repository on GitHub or Bitbucket. Whenever you push code changes, Docker Hub automatically builds a new image.

```
+-----------------------------------------------------------+
|              Automated Build Flow                         |
|                                                           |
|   GitHub Repo          Docker Hub                         |
|   +-----------+        +------------------+               |
|   | Push code | -----> | Detect change    |               |
|   | to main   |        | Build image      |               |
|   +-----------+        | Push to registry |               |
|                        +------------------+               |
|                                                           |
|   You push code. Docker Hub builds the image for you.     |
+-----------------------------------------------------------+
```

Setting Up Automated Builds

1. Go to Docker Hub → Your Repository → Builds
2. Link your GitHub or Bitbucket account
3. Select the source repository

4. Configure build rules:
    - Source branch: `main` → Docker tag: `latest`
    - Source branch: `/^v[0-9.]+$/` → Docker tag: `{sourceref}`
5. Enable "Build on push"

Build Rules Example

Source Type	Source	Docker Tag	Dockerfile Location
Branch	main	latest	/Dockerfile
Tag	/^v.*$/	{sourceref}	/Dockerfile
Branch	develop	dev	/Dockerfile

What this means:

- Pushing to the `main` branch builds an image tagged `latest`
- Creating a Git tag like `v1.2.3` builds an image tagged `v1.2.3`
- Pushing to `develop` builds an image tagged `dev`

> Note: Docker Hub's free automated builds have been limited. Many teams now use GitHub Actions or other CI systems to build and push images instead. We cover this approach in Chapter 21.

Private Repositories

By default, Docker Hub repositories are public — anyone can pull your images. For proprietary applications, you need private repositories.

Free Tier Limitations

Docker Hub's free tier includes:

- Unlimited public repositories
- One private repository (this may change — check Docker Hub for current limits)

Paid plans offer more private repositories and additional features like team management and vulnerability scanning.

Creating a Private Repository

When creating a repository, select Private for visibility:

```bash
# This will push to a private repo (if configured on Docker Hub)
docker push myusername/my-secret-app:v1.0
```

Pulling from a Private Repository

Anyone pulling a private image must be logged in and have access:

```bash
# Must be logged in first
docker login

# Now you can pull
docker pull myusername/my-secret-app:v1.0
```

Without logging in, you get an error:

```
Error response from daemon: pull access denied for myusername/my-secret-app,
repository does not exist or may require 'docker login'
```

Alternative Registries

Docker Hub is not the only registry. Here are the most popular alternatives and when to use each.

GitHub Container Registry (GHCR)

GitHub Container Registry stores images alongside your source code on GitHub. It integrates naturally with GitHub Actions for CI/CD.

```
Registry URL: ghcr.io
Image format: ghcr.io/username/image-name:tag
```

Login:

```bash
# Create a Personal Access Token on GitHub with "write:packages" scope
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin
```

Tag and push:

```bash
docker tag my-webapp ghcr.io/myusername/my-webapp:v1.0
docker push ghcr.io/myusername/my-webapp:v1.0
```

Why choose GHCR:

- Your code and images live in the same place
- GitHub Actions can push images without extra credentials
- Free for public repositories
- Generous free tier for private repositories

AWS Elastic Container Registry (ECR)

AWS ECR is Amazon's fully managed registry. It integrates with other AWS services like ECS, EKS, and Lambda.

```
Registry URL: <account-id>.dkr.ecr.<region>.amazonaws.com
Image format: 123456789.dkr.ecr.us-east-1.amazonaws.com/my-webapp:v1.0
```

Login (requires AWS CLI):

```bash
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS \
  --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com
```

Create a repository first (ECR requires this):

```bash
aws ecr create-repository --repository-name my-webapp --region us-east-1
```

Tag and push:

```bash
docker tag my-webapp 123456789.dkr.ecr.us-east-1.amazonaws.com/my-webapp:v1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/my-webapp:v1.0
```

Why choose AWS ECR:

- You deploy to AWS services (ECS, EKS, Fargate)
- You need fine-grained IAM permissions
- You want images stored close to your AWS infrastructure

Google Container Registry (GCR) / Artifact Registry

Google Artifact Registry (successor to GCR) stores images for Google Cloud deployments.

```
Registry URL: us-docker.pkg.dev (Artifact Registry)
              gcr.io (legacy GCR)
Image format: us-docker.pkg.dev/project-id/repo-name/image-name:tag
```

Login:

```bash
gcloud auth configure-docker us-docker.pkg.dev
```

Tag and push:

```bash
docker tag my-webapp us-docker.pkg.dev/my-project/my-repo/my-webapp:v1.0
docker push us-docker.pkg.dev/my-project/my-repo/my-webapp:v1.0
```

Why choose Artifact Registry:

- You deploy to Google Cloud (GKE, Cloud Run)
- You need vulnerability scanning integrated with Google Cloud
- You want multi-format support (Docker, npm, Maven, etc.)

Comparison Table

Feature	Docker Hub	GHCR	AWS ECR	Google AR
Free public repos	Yes	Yes	No	No
Free private repos	Limited	Yes	No	No
CI/CD integration	Limited	GitHub	AWS	Google
Vulnerability scan	Paid	Free	Free	Free
Custom domains	No	No	Yes	Yes
Best for	Open source	GitHub projects	AWS deploys	GCP deploys

Running a Self-Hosted Registry

Sometimes you need a registry that runs on your own servers. Reasons include:

- Air-gapped environments — No internet access
- Compliance — Data must stay in your network
- Speed — Local registry for faster pulls in your data center
- Cost — No per-image charges

Starting a Local Registry

Docker provides an official registry image. Setting up a basic registry takes one command:

```bash
docker run -d \
  --name my-registry \
  -p 5000:5000 \
  --restart always \
  registry:2
```

Line-by-line explanation:

- `docker run -d` — Run in detached mode (background)
- `--name my-registry` — Give it a readable name
- `-p 5000:5000` — Map port 5000 on your machine to port 5000 in the container
- `--restart always` — Automatically restart if it crashes or if Docker restarts
- `registry:2` — Use version 2 of the official Docker registry image

Using the Local Registry

```bash
# Tag an image for the local registry
docker tag my-webapp localhost:5000/my-webapp:v1.0

# Push to the local registry
docker push localhost:5000/my-webapp:v1.0

# Pull from the local registry
docker pull localhost:5000/my-webapp:v1.0
```

Adding Persistent Storage

Without a volume, pushed images are lost when the container stops:

```bash
docker run -d \
  --name my-registry \
  -p 5000:5000 \
  --restart always \
  -v registry-data:/var/lib/registry \
  registry:2
```

The `-v registry-data:/var/lib/registry` flag stores image data in a named volume that survives container restarts.

Checking What Is in the Registry

```bash
# List all repositories
curl http://localhost:5000/v2/_catalog
```

Output:

```json
{"repositories":["my-webapp"]}
```

```bash
# List tags for a repository
curl http://localhost:5000/v2/my-webapp/tags/list
```

Output:

```json
{"name":"my-webapp","tags":["v1.0"]}
```

```
+-----------------------------------------------------+
|           Self-Hosted Registry Architecture           |
|                                                       |
|   Developer Machine        Your Server                |
|   +----------------+      +-------------------+       |
|   | docker push    | ---> | Registry:2        |       |
|   | localhost:5000 |      | Port 5000         |       |
|   +----------------+      | Volume: data      |       |
|                            +-------------------+       |
|                                  |                     |
|   Production Server              |                     |
|   +----------------+             |                     |
```

```
|   | docker pull    | <-----------+                    |
|   | server:5000    |                                  |
|   +----------------+                                  |
+-------------------------------------------------------+
```

> Security Warning: A local registry on `localhost:5000` works without HTTPS. If you expose the registry to a network, you MUST configure TLS (HTTPS) and authentication. Without them, anyone on your network can push and pull images.

Common Mistakes

Mistake 1: Forgetting to Log In Before Pushing

```bash
docker push myusername/my-webapp:v1.0
```

Error:

```
denied: requested access to the resource is denied
```

Fix: Run `docker login` first and enter your credentials.

Mistake 2: Pushing Without the Correct Tag Format

```bash
# This will NOT work - no username prefix
docker push my-webapp:v1.0
```

Error:

```
An image does not exist locally with the tag: my-webapp
```

Fix: Tag the image with your username first:

```bash
docker tag my-webapp myusername/my-webapp:v1.0
docker push myusername/my-webapp:v1.0
```

Mistake 3: Using the Wrong Registry URL

```bash
# Trying to push to GHCR without the correct prefix
docker push myusername/my-webapp:v1.0  # Goes to Docker Hub, not GHCR
```

Fix: Include the full registry URL:

```bash
docker push ghcr.io/myusername/my-webapp:v1.0  # Goes to GHCR
```

Mistake 4: Leaving Credentials on Shared Machines

After using `docker login` on a shared server, your credentials remain in `~/.docker/config.json`. Anyone with access to that file can use your account.

Fix: Always run `docker logout` when finished on shared machines.

Mistake 5: Pushing Sensitive Data in Image Layers

If your Dockerfile copies `.env` files or secrets into the image, those secrets are visible to anyone who pulls the image — even if you delete them in a later layer.

Fix: Use `.dockerignore` to exclude sensitive files, and use Docker secrets or environment variables for configuration.

Best Practices

1. Use access tokens instead of passwords — Tokens can be revoked individually and scoped to specific permissions
2. Always log out on shared machines — Protect your credentials with `docker logout`
3. Tag images with specific versions — Avoid relying solely on `latest` (more on this in Chapter 20)
4. Use private repositories for proprietary code — Never publish internal applications to public repositories
5. Add a README to your Docker Hub repositories — Help users understand what your image does and how to use it
6. Set up webhooks — Notify your deployment pipeline when new images are pushed

7. Choose the registry closest to your infrastructure — AWS ECR for AWS deployments, GHCR for GitHub projects, and so on
8. Scan images for vulnerabilities — Enable scanning on your registry (covered in Chapter 22)
9. Use `.dockerignore` — Prevent sensitive files from being included in images

---

Quick Summary

Docker Hub is the default registry for Docker images, acting like an app store where you can publish and download container images. You log in with `docker login`, tag images with your username prefix, and push with `docker push`. Repositories hold multiple tagged versions of an image. For private or proprietary images, use private repositories. Alternative registries like GitHub Container Registry, AWS ECR, and Google Artifact Registry integrate with their respective platforms. For air-gapped or compliance scenarios, you can run a self-hosted registry with the official `registry:2` image.

Key Points

- Docker Hub is the default registry — `docker pull nginx` talks to Docker Hub automatically
- docker login authenticates you with a registry
- docker tag creates a new name for an image — it does not copy the image
- docker push uploads an image to a registry, sending only new layers
- Repositories group multiple tagged versions of one image
- Private repositories require authentication to pull
- GHCR, AWS ECR, and Google Artifact Registry are popular alternatives, each integrated with their cloud platform
- Self-hosted registries run on your own servers using the `registry:2` image
- Access tokens are safer than passwords for authentication
- Never store sensitive data in image layers

Practice Questions

1. What command do you use to authenticate with Docker Hub from the terminal? What is a safer alternative to using your password?
2. If your Docker Hub username is `devjane` and you have an image called `api-server`, what is the correct tag format to push it to Docker Hub with version `2.1.0`?
3. You run `docker push myapp:v1.0` and get a "denied" error. What are two possible reasons for this error?
4. What is the difference between a public and private repository on Docker Hub? How does a private repository affect `docker pull`?
5. You need to store Docker images inside your company network with no internet access. What solution would you use?

Exercises

Exercise 1: Push Your First Image

1. Create a Docker Hub account if you do not have one
2. Log in from the terminal using `docker login`
3. Build a simple image (use any Dockerfile from a previous chapter)
4. Tag it with your Docker Hub username and version `v1.0`
5. Push it to Docker Hub
6. Verify it appears on your Docker Hub profile page
7. Log out with `docker logout`
8. Pull the image to confirm it works: `docker pull yourusername/image:v1.0`

Exercise 2: Set Up a Local Registry

1. Start a local registry using `docker run -d -p 5000:5000 registry:2`
2. Tag an existing image for the local registry: `docker tag nginx localhost:5000/my-nginx:v1.0`
3. Push the image to your local registry
4. Delete the local image: `docker rmi localhost:5000/my-nginx:v1.0`
5. Pull it back from the local registry
6. Use `curl` to list the repositories in your local registry

Exercise 3: Explore Alternative Registries

1. Create a GitHub Personal Access Token with `write:packages` scope
2. Log in to GitHub Container Registry using the token
3. Tag an image for GHCR and push it
4. Verify the image appears in your GitHub profile under "Packages"
5. Compare the experience with pushing to Docker Hub

What Is Next?

You now know how to share images through registries. But we glossed over an important detail: how to choose the right tags for your images. In the next chapter, we dive deep into image tagging strategies — why `latest` can be dangerous, how semantic versioning works for Docker images, and how teams organize their tags for reliable deployments.

Chapter 20: Image Tagging Strategies

What You Will Learn

By the end of this chapter, you will be able to:

- Understand why image tagging matters for reliable deployments
- Apply semantic versioning to Docker images
- Explain the dangers of relying on the `:latest` tag
- Use Git commit SHAs as image tags for traceability
- Implement immutable tags for production safety
- Design tagging strategies that work for teams
- Apply multi-tag approaches (v1.2.3, v1.2, v1, latest)
- Set up automated tagging in CI/CD pipelines

Why This Chapter Matters

Imagine you deploy an application to production using the tag `myapp:latest`. Everything works. A week later, someone pushes a new version — also tagged `latest`. Your production server pulls the image, and the application breaks.

What version was running before? What changed? You have no idea, because `latest` was overwritten.

Tags are the version labels on your Docker images. Good tagging is like labeling moving boxes clearly: "Kitchen - Plates" tells you exactly what is inside, while "Stuff" tells you nothing. In production, unclear tags lead to broken deployments, wasted debugging hours, and sleepless nights.

This chapter teaches you how to tag images so that every deployment is traceable, repeatable, and safe.

What Is an Image Tag?

A tag is a label attached to a Docker image that identifies a specific version. The full image reference looks like this:

```
repository:tag

Examples:
  nginx:1.25.3
  node:20-alpine
  myapp:v2.1.0
  myapp:abc123f
```

If you omit the tag, Docker assumes `:latest`:

```bash
docker pull nginx          # Same as: docker pull nginx:latest
docker build -t myapp .    # Same as: docker build -t myapp:latest .
```

Tags Are Pointers, Not Copies

A tag is like a bookmark in a book. The bookmark points to a specific page. You can move the bookmark to a different page at any time — the pages themselves do not change.

```
+-------------------------------------------+
|            Tags Are Pointers              |
|                                           |
|   Tag "v1.0"  -----> Image abc123         |
|   Tag "v1.1"  -----> Image def456         |
|   Tag "latest" ----> Image def456         |
|                                           |
|   "latest" and "v1.1" point to the SAME   |
|   image. Moving "latest" does not delete   |
|   the old image.                           |
+-------------------------------------------+
```

This means you can have multiple tags pointing to the same image, and you can reassign a tag to point to a different image at any time.

The :latest Trap

The `:latest` tag is the most misunderstood concept in Docker. Let us clear up the confusion.

What :latest Actually Is

`:latest` is NOT:

- Automatically updated to the newest version
- A guarantee that you have the most recent image
- Special or magical in any way

`:latest` IS:

- Just the default tag when no tag is specified
- A regular, mutable tag that can point to any image
- Frequently overwritten, making it unreliable

Why :latest Is Dangerous in Production

```
+-----------------------------------------------------------+
|              The :latest Problem                          |
|                                                           |
|   Monday:    myapp:latest --> Image A (v1.0, works fine)  |
|   Tuesday:   myapp:latest --> Image B (v1.1, has a bug)   |
|   Wednesday: myapp:latest --> Image C (v1.2, different bug)|
|                                                           |
|   Which version is running on your server?                |
|   When did it change?                                     |
|   How do you roll back?                                   |
|                                                           |
|   Answer: You have NO IDEA.                               |
+-----------------------------------------------------------+
```

Problem 1: No traceability. If production breaks, you cannot tell which version of the code is running.

Problem 2: No reproducibility. Running `docker pull myapp:latest` on Monday gives you a different image than on Tuesday.

Problem 3: No safe rollback. You cannot roll back to "the previous latest" because it has been overwritten.

Problem 4: Cache confusion. If a server already has `myapp:latest` cached, it might not pull the new version unless you force it.

When :latest Is Acceptable

- Local development — You are the only user and you know what version you are running
- Quick testing — Pulling an image to try it out
- Tutorials and examples — Simplicity matters more than precision

When :latest Is NOT Acceptable

- Production deployments — Always use specific version tags
- Docker Compose files — Pin exact versions for reproducibility
- CI/CD pipelines — Use commit SHAs or semantic versions
- Shared environments — Team members need to know what version they are running

---

Semantic Versioning for Docker Images

Semantic versioning (SemVer) is a widely-used system for version numbers. It uses three numbers separated by dots:

```
MAJOR.MINOR.PATCH

Examples:
  1.0.0   - First release
  1.0.1   - Bug fix
  1.1.0   - New feature, backward compatible
  2.0.0   - Breaking change
```

What Each Number Means

```
+-----------------------------------------------------+
|           Semantic Versioning Explained             |
|                                                     |
|   v 2 . 3 . 1                                      |
|     |   |   |                                      |
|     |   |   +-- PATCH: Bug fixes only              |
|     |   |       "We fixed a crash."                |
|     |   |       Safe to update.                    |
```

```
|     |   |                                          |
|     |   +------ MINOR: New features added          |
|     |           "We added dark mode."              |
|     |           Old features still work.           |
|     |           Usually safe to update.            |
|     |                                              |
|     +---------- MAJOR: Breaking changes            |
|                 "We redesigned the API."           |
|                 Old code might break.              |
|                 Read the changelog first.          |
+----------------------------------------------------+
```

Applying SemVer to Docker Images

```bash
# Initial release
docker build -t myapp:1.0.0 .

# Bug fix — increment PATCH
docker build -t myapp:1.0.1 .

# New feature — increment MINOR, reset PATCH
docker build -t myapp:1.1.0 .

# Breaking change — increment MAJOR, reset MINOR and PATCH
docker build -t myapp:2.0.0 .
```

Real-World Example

Suppose you have a web API. Here is how versions might progress:

Version	What Changed	Impact on Users
1.0.0	Initial release	First version
1.0.1	Fixed login timeout bug	No changes needed
1.0.2	Fixed memory leak	No changes needed
1.1.0	Added search endpoint	New feature, old API works
1.2.0	Added pagination to list API	New feature, old API works
2.0.0	Changed authentication to OAuth	Clients must update

Git SHA Tags

A Git SHA tag uses the Git commit hash as the image tag. This creates a direct link between an image and the exact source code that produced it.

```bash
# Get the short Git commit hash
GIT_SHA=$(git rev-parse --short HEAD)
echo $GIT_SHA
# Output: a1b2c3d

# Use it as the image tag
docker build -t myapp:$GIT_SHA .
docker build -t myapp:a1b2c3d .
```

Why Git SHAs Are Useful

```
+-------------------------------------------------------------+
|           Git SHA Traceability                               |
|                                                              |
|   Image tag:  myapp:a1b2c3d                                  |
|                  |                                            |
|                  v                                            |
|   Git commit: a1b2c3d "Fix payment processing bug"           |
|                  |                                            |
|                  v                                            |
|   Exact code: You can see EVERY line of code                  |
|               that went into this image.                      |
|                                                              |
|   Production breaks? Check commit a1b2c3d.                   |
|   Need to reproduce? Build from commit a1b2c3d.              |
+-------------------------------------------------------------+
```

Advantage 1: Perfect traceability. Given an image tag, you can find the exact commit, see the diff, read the commit message, and know who made the change.

Advantage 2: Unique by nature. Every commit has a unique hash, so every image tag is unique. No overwrites.

Advantage 3: Easy debugging. If production has `myapp:a1b2c3d`, you run `git log a1b2c3d` to see what changed.

Limitations of Git SHAs

- Not human-readable. `myapp:a1b2c3d` tells you nothing about the version or features
- No ordering. You cannot tell if `a1b2c3d` is newer than `f4e5d6c` just by looking at them

- Best combined with semantic versions. Use both: `myapp:1.2.3` and `myapp:a1b2c3d`

Immutable Tags

An immutable tag is a tag that, once assigned, cannot be overwritten. If you try to push a new image with the same tag, the registry rejects it.

Why Immutability Matters

```
+-------------------------------------------------------------+
|          Mutable vs Immutable Tags                          |
|                                                             |
|   Mutable (default):                                        |
|     Push myapp:v1.0 (Image A)                               |
|     Push myapp:v1.0 (Image B)  <-- Overwrites Image A!    |
|     Anyone pulling v1.0 gets Image B now.                   |
|                                                             |
|   Immutable:                                                |
|     Push myapp:v1.0 (Image A)                               |
|     Push myapp:v1.0 (Image B)  <-- REJECTED by registry   |
|     v1.0 ALWAYS means Image A. Forever.                     |
+-------------------------------------------------------------+
```

Enabling Immutable Tags

AWS ECR supports immutable tags natively:

```bash
aws ecr put-image-tag-mutability \
  --repository-name my-webapp \
  --image-tag-mutability IMMUTABLE
```

Other registries may not support immutable tags at the registry level. In that case, enforce immutability through your CI/CD pipeline by checking if a tag exists before pushing:

```bash
# Check if tag already exists on Docker Hub
check_tag() {
  local image=$1
  local tag=$2
  if docker manifest inspect "${image}:${tag}" > /dev/null 2>&1; then
    echo "ERROR: Tag ${tag} already exists. Cannot overwrite."
```

```
    exit 1
  fi
}

check_tag "myusername/myapp" "v1.0.0"
docker push myusername/myapp:v1.0.0
```

Which Tags Should Be Immutable?

Tag Type	Should Be Immutable?	Why
v1.2.3	Yes	Release versions must never change
a1b2c3d	Yes	Git SHAs are inherently unique
latest	No	Meant to be updated with each release
v1.2	No	Points to latest patch (v1.2.0 then v1.2.1)
dev	No	Updated with every development build

Tagging Strategies for Teams

When multiple developers work on the same project, you need a consistent tagging strategy. Here are the most common approaches.

Strategy 1: SemVer Only

```
myapp:1.0.0
myapp:1.0.1
myapp:1.1.0
myapp:2.0.0
```

Best for: Small teams with manual releases. Downside: No link to source code. Requires discipline to bump versions correctly.

Strategy 2: Git SHA Only

```
myapp:a1b2c3d
myapp:b2c3d4e
myapp:c3d4e5f
```

Best for: Teams that deploy every commit. Works well with continuous deployment. Downside: Not human-readable. Hard to tell which version is "stable."

Strategy 3: SemVer + Git SHA (Recommended)

```
myapp:1.2.3
myapp:a1b2c3d
```

Both tags point to the same image. You get the best of both worlds:

- Use `1.2.3` for human communication ("deploy version 1.2.3")
- Use `a1b2c3d` for debugging ("what code is in production?")

Strategy 4: Branch-Based Tags

```
myapp:main-a1b2c3d      <-- Built from main branch
myapp:develop-b2c3d4e   <-- Built from develop branch
myapp:feature-auth-c3d  <-- Built from feature branch
```

Best for: Teams using GitFlow or similar branching strategies. Downside: Tags get long and numerous. Feature branch tags may never be used in production.

Strategy 5: Environment-Based Tags

```
myapp:staging
myapp:production
```

Warning: This approach has the same problems as `:latest`. The tag `staging` gets overwritten with every deployment. Avoid this in favor of SemVer or Git SHAs.

Multi-Tag Strategy

The most robust approach is to apply multiple tags to the same image. This gives users flexibility in how they pin their versions.

How Multi-Tagging Works

When you release version `1.2.3`, you create four tags:

```bash
docker build -t myapp:1.2.3 .
docker tag myapp:1.2.3 myapp:1.2
docker tag myapp:1.2.3 myapp:1
docker tag myapp:1.2.3 myapp:latest
```

All four tags point to the same image:

```
+-----------------------------------------------------------+
|              Multi-Tag Strategy                           |
|                                                           |
|   myapp:1.2.3  ----+                                      |
|   myapp:1.2    ----+--> Same Image (abc123)               |
|   myapp:1      ----+                                      |
|   myapp:latest ----+                                      |
|                                                           |
|   Users choose their risk level:                          |
|     :1.2.3   = "I want exactly this version"             |
|     :1.2     = "Give me the latest patch for 1.2"        |
|     :1       = "Give me the latest 1.x version"          |
|     :latest  = "Give me whatever is newest"              |
+-----------------------------------------------------------+
```

What Happens When You Release 1.2.4

```bash
docker build -t myapp:1.2.4 .
docker tag myapp:1.2.4 myapp:1.2      # Now points to 1.2.4
docker tag myapp:1.2.4 myapp:1        # Now points to 1.2.4
docker tag myapp:1.2.4 myapp:latest   # Now points to 1.2.4
# myapp:1.2.3 still points to the OLD image — it is NOT overwritten
```

```
+-----------------------------------------------------------+
|           After Releasing 1.2.4                           |
|                                                           |
|   myapp:1.2.3  ---------> Old Image (abc123)             |
```

```
|                                                            |
|   myapp:1.2.4  ----+                                       |
|   myapp:1.2    ----+--> New Image (def456)                 |
|   myapp:1      ----+                                       |
|   myapp:latest ----+                                       |
|                                                            |
|   Users pinned to :1.2.3 are unaffected.                  |
|   Users on :1.2 automatically get 1.2.4.                  |
+------------------------------------------------------------+
```

Real-World Example: Node.js Tags

The official `node` image on Docker Hub demonstrates this perfectly:

```
node:20.10.0          <-- Exact version
node:20.10            <-- Latest patch for 20.10
node:20               <-- Latest 20.x version
node:lts              <-- Current long-term support
node:latest           <-- Most recent release
node:20.10.0-alpine   <-- Exact version, Alpine base
node:20-alpine        <-- Latest 20.x, Alpine base
```

Automated Tagging in CI/CD

Manually tagging images is error-prone. Automate it in your CI/CD pipeline.

GitHub Actions Example

```yaml
# .github/workflows/docker-tag.yml
name: Build and Tag Docker Image

on:
  push:
    branches: [main]
    tags: ['v*']  # Trigger on version tags like v1.2.3

env:
  IMAGE_NAME: myusername/myapp

jobs:
  build-and-push:
```

```
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Extract metadata for tagging
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.IMAGE_NAME }}
          tags: |
            # Tag with Git SHA for every push
            type=sha,prefix=
            # Tag with branch name
            type=ref,event=branch
            # Tag with semver from Git tag (v1.2.3 -> 1.2.3, 1.2, 1)
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=semver,pattern={{major}}
            # Tag as latest for default branch
            type=raw,value=latest,enable={{is_default_branch}}

      - name: Build and push image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
```

What this does:

When you push to `main`:

- Tags with the Git SHA: `myapp:a1b2c3d`
- Tags with the branch name: `myapp:main`
- Tags as `myapp:latest`

When you create a Git tag `v1.2.3`:

- Tags as `myapp:1.2.3`
- Tags as `myapp:1.2`
- Tags as `myapp:1`
- Tags with the Git SHA: `myapp:a1b2c3d`

Automated Tagging Script (Without CI/CD)

If you are not using a CI/CD system, here is a shell script:

```bash
#!/bin/bash
# tag-and-push.sh - Tag and push a Docker image with multiple tags

set -e

IMAGE="myusername/myapp"
VERSION=$1  # Pass version as argument: ./tag-and-push.sh 1.2.3

if [ -z "$VERSION" ]; then
  echo "Usage: ./tag-and-push.sh <version>"
  echo "Example: ./tag-and-push.sh 1.2.3"
  exit 1
fi

# Extract major and minor versions
MAJOR=$(echo "$VERSION" | cut -d. -f1)
MINOR=$(echo "$VERSION" | cut -d. -f1-2)
GIT_SHA=$(git rev-parse --short HEAD)

echo "Building image..."
docker build -t "$IMAGE:$VERSION" .

echo "Creating additional tags..."
docker tag "$IMAGE:$VERSION" "$IMAGE:$MINOR"
docker tag "$IMAGE:$VERSION" "$IMAGE:$MAJOR"
docker tag "$IMAGE:$VERSION" "$IMAGE:$GIT_SHA"
docker tag "$IMAGE:$VERSION" "$IMAGE:latest"

echo "Pushing all tags..."
docker push "$IMAGE" --all-tags

echo "Done! Pushed tags: $VERSION, $MINOR, $MAJOR, $GIT_SHA, latest"
```

Usage:

```bash
chmod +x tag-and-push.sh
./tag-and-push.sh 1.2.3
```

Output:

```
Building image...
Creating additional tags...
Pushing all tags...
Done! Pushed tags: 1.2.3, 1.2, 1, a1b2c3d, latest
```

Common Mistakes

Mistake 1: Using :latest in Production

```yaml
# docker-compose.yml — DO NOT DO THIS
services:
  web:
    image: myapp:latest  # Which version is this? Nobody knows.
```

Fix:

```yaml
# docker-compose.yml — DO THIS
services:
  web:
    image: myapp:1.2.3  # Exact version, reproducible
```

Mistake 2: Forgetting to Tag Before Pushing

```bash
docker build -t myapp .
docker push myapp  # Pushes as "myapp:latest" — not versioned!
```

Fix:

```bash
docker build -t myusername/myapp:1.0.0 .
docker push myusername/myapp:1.0.0
```

Mistake 3: Overwriting Release Tags

```bash
# Version 1.0.0 already exists on the registry
docker build -t myapp:1.0.0 .   # Different code!
docker push myapp:1.0.0         # Overwrites the original!
```

Fix: Never reuse a release tag. If you need to fix a bug, release `1.0.1`.

Mistake 4: Inconsistent Tagging Across the Team

Developer A uses `v1.2.3`, Developer B uses `1.2.3`, Developer C uses `release-1.2.3`. Chaos.

Fix: Document your tagging convention and enforce it in CI/CD:

```
# Our tagging convention (document this in your README):
# Release tags: 1.2.3 (no "v" prefix)
# Git SHA tags: a1b2c3d (short SHA, 7 characters)
# Branch tags: main, develop (branch name only)
```

Mistake 5: Not Cleaning Up Old Tags

Over time, hundreds of tags accumulate in your registry, consuming storage and making it hard to find relevant versions.

Fix: Set up a retention policy. Most registries support lifecycle rules:

```bash
# AWS ECR lifecycle policy — keep only last 30 images
aws ecr put-lifecycle-policy \
  --repository-name myapp \
  --lifecycle-policy-text '{
    "rules": [{
      "rulePriority": 1,
      "selection": {
        "tagStatus": "any",
        "countType": "imageCountMoreThan",
        "countNumber": 30
      },
      "action": { "type": "expire" }
    }]
  }'
```

Best Practices

1. Never use `:latest` in production — Always pin a specific version tag
2. Use semantic versioning — MAJOR.MINOR.PATCH communicates the type of change
3. Add Git SHA tags — Create a direct link between images and source code
4. Apply multiple tags — Give users flexibility (exact version, minor, major, latest)
5. Make release tags immutable — Once `1.2.3` is pushed, it should never change
6. Document your tagging convention — Write it down so the whole team follows it
7. Automate tagging in CI/CD — Humans make mistakes; automation does not
8. Clean up old tags — Set retention policies to avoid unbounded storage growth
9. Use consistent formats — Choose `1.2.3` or `v1.2.3` and stick with it everywhere

Quick Summary

Image tags identify specific versions of Docker images. The `:latest` tag is a trap in production because it is mutable and provides no traceability. Semantic versioning (MAJOR.MINOR.PATCH) communicates the impact of changes. Git SHA tags link images to exact source code commits. Immutable tags prevent accidental overwrites of release versions. The multi-tag strategy (1.2.3 + 1.2 + 1 + latest) gives users flexibility while maintaining precision. Automate tagging in CI/CD pipelines to ensure consistency and eliminate human error.

Key Points

- Tags are pointers — Multiple tags can point to the same image, and tags can be moved
- `:latest` is not special — It is just the default tag, and it is mutable (overwritable)
- Semantic versioning uses MAJOR.MINOR.PATCH to communicate the type of change
- Git SHA tags create a traceable link from an image to its source code
- Immutable tags prevent release versions from being overwritten
- Multi-tagging (1.2.3, 1.2, 1, latest) gives users control over their update risk
- Automate tagging in CI/CD to ensure consistency
- Document your convention so the team follows the same rules

Practice Questions

1. What does the `:latest` tag actually mean? Why is it dangerous to use in production?
2. In semantic versioning, what does it mean when the MAJOR version number changes? Give an example with Docker image tags.
3. What is a Git SHA tag, and what problem does it solve that semantic version tags do not?
4. You release version `1.2.3` of your application. Which tags should you create following the multi-tag strategy?

5. What is an immutable tag? Which types of tags should be immutable and which should remain mutable?

Exercises

Exercise 1: Multi-Tag a Release

1. Build a simple Docker image from any previous chapter
2. Pretend this is version `2.1.0` of your application
3. Create all four tags: `2.1.0`, `2.1`, `2`, and `latest`
4. Run `docker images` and verify all four tags show the same IMAGE ID
5. Now pretend you release `2.1.1` (a bug fix). Create tags for `2.1.1` and update `2.1`, `2`, and `latest`
6. Run `docker images` again. Which tags now point to the new image? Which still point to the old one?

Exercise 2: Git SHA Tagging

1. Initialize a Git repository in a project directory (or use an existing one)
2. Make a commit with a simple change
3. Get the short Git SHA with `git rev-parse --short HEAD`
4. Build a Docker image tagged with this SHA
5. Make another commit and build again with the new SHA
6. You now have two images. Given an image tag, can you find the exact commit?

Exercise 3: Write a Tagging Convention

1. Write a document (plain text is fine) that defines your team's tagging convention
2. Include: what format to use for release tags, whether to include a "v" prefix, when to use Git SHAs, what `:latest` should point to, and how often to clean up old tags
3. Create a simple shell script that enforces this convention (validates that the tag matches your pattern before pushing)

What Is Next?

You now know how to tag images for clarity, safety, and traceability. But tagging is only one piece of the puzzle. In the next chapter, we explore CI/CD with Docker — how to automate the entire process of building, tagging, and pushing images every time you push code. No more manual builds. No more forgotten tags. Just push your code and let the pipeline handle the rest.

Chapter 21: CI/CD with Docker

What You Will Learn

By the end of this chapter, you will be able to:

- Understand what CI/CD is and why it matters
- Set up a GitHub Actions workflow that builds Docker images
- Push images to Docker Hub and GitHub Container Registry automatically
- Use caching to speed up Docker builds in CI
- Build multi-platform images for amd64 and arm64 architectures
- Deploy different images to different environments (staging, production)
- Write a complete `.github/workflows/docker.yml` from scratch

Why This Chapter Matters

Up until now, you have been building and pushing Docker images by hand. You run `docker build`, then `docker tag`, then `docker push`. This works for learning, but it falls apart in a real team:

- Someone forgets to build before pushing
- Someone pushes an image without running tests
- Someone tags the image incorrectly
- Someone pushes from their laptop with uncommitted changes

CI/CD eliminates all of these problems. It automates the entire process: every time you push code, a server builds the image, runs tests, tags it correctly, and pushes it to a registry. No manual steps. No human errors.

Think of CI/CD as an assembly line in a factory:

```
+-------------------------------------------------------------+
|              The Assembly Line Analogy                       |
|                                                             |
|   Raw Materials (code)                                      |
|        |                                                    |
|        v                                                    |
|   [Station 1: Build]        "Assemble the product"          |
|        |                     docker build                    |
|        v                                                    |
|   [Station 2: Test]         "Quality check"                 |
|        |                     run tests                       |
|        v                                                    |
|   [Station 3: Package]      "Label and box it"              |
|        |                     docker tag                      |
|        v                                                    |
|   [Station 4: Ship]         "Send to warehouse"             |
|        |                     docker push                     |
|        v                                                    |
|   Registry (warehouse)                                      |
|        |                                                    |
|        v                                                    |
|   Servers pull and deploy                                   |
+-------------------------------------------------------------+
```

You write code and push it to GitHub. The assembly line does everything else.

What Is CI/CD?

CI stands for Continuous Integration. It means automatically building and testing your code every time someone pushes a change. The word "continuous" means it happens on every push — not once a week or once a month.

CD stands for Continuous Delivery (or Continuous Deployment). It means automatically delivering your tested code to a registry or server.

- Continuous Delivery = Automatically prepare a release (build, test, push image). A human clicks a button to deploy.
- Continuous Deployment = Automatically deploy to production. No human in the loop.

```
+-------------------------------------------------------------+
|                CI/CD Pipeline                                |
|                                                             |
|   Developer pushes code                                     |
|        |                                                    |
|        v                                                    |
```

```
|   +------------------+                                  |
|   | Continuous       |  Build code                      |
|   | Integration (CI) |  Run tests                       |
|   |                  |  Build Docker image              |
|   +------------------+                                  |
|        |                                                |
|        v  (tests pass)                                  |
|   +------------------+                                  |
|   | Continuous       |  Tag image                       |
|   | Delivery (CD)    |  Push to registry                |
|   |                  |  Deploy to staging/production     |
|   +------------------+                                  |
|        |                                                |
|        v                                                |
|   Application is live                                   |
+---------------------------------------------------------+
```

Popular CI/CD Tools

Tool	Where It Runs	Best For
GitHub Actions	GitHub.com	Projects hosted on GitHub
GitLab CI	GitLab.com	Projects hosted on GitLab
Jenkins	Self-hosted	Full control, complex pipelines
CircleCI	Cloud	Fast builds, easy setup
Travis CI	Cloud	Open source projects

This chapter uses GitHub Actions because it is free for public repositories, requires no extra accounts, and integrates directly with GitHub.

GitHub Actions Basics

GitHub Actions is GitHub's built-in CI/CD system. You define workflows in YAML files inside the `.github/workflows/` directory of your repository.

Key Concepts

```
+-------------------------------------------------------------+
|              GitHub Actions Terminology                     |
|                                                             |
|   Workflow:   A YAML file that defines an automation        |
|               (.github/workflows/docker.yml)                |
|                                                             |
|   Event:      What triggers the workflow                    |
|               (push, pull request, schedule, manual)        |
|                                                             |
|   Job:        A set of steps that run on the same machine   |
|               (build, test, deploy)                         |
|                                                             |
|   Step:       A single task within a job                    |
|               (checkout code, build image, push image)      |
|                                                             |
|   Runner:     The machine that executes the job             |
|               (ubuntu-latest, windows-latest, macos-latest)|
|                                                             |
|   Action:     A reusable step created by the community      |
|               (docker/build-push-action, actions/checkout)  |
+-------------------------------------------------------------+
```

Your First Workflow

Create this file in your repository:

```yaml
# .github/workflows/docker.yml
name: Build Docker Image

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t myapp:test .

      - name: Run tests
        run: docker run --rm myapp:test npm test
```

Line-by-line explanation:

- `name: Build Docker Image` — A human-readable name shown in the GitHub Actions UI

- `on: push: branches: [main]` — Run this workflow when code is pushed to the `main` branch
- `jobs: build:` — Define a job called "build"
- `runs-on: ubuntu-latest` — Run on a fresh Ubuntu virtual machine
- `uses: actions/checkout@v4` — Clone your repository onto the runner
- `run: docker build -t myapp:test .` — Build the Docker image
- `run: docker run --rm myapp:test npm test` — Start a container and run tests

This workflow runs on every push to `main`. If the build or tests fail, GitHub shows a red X on the commit. If everything passes, you get a green checkmark.

Building and Pushing to Docker Hub

Let us extend the workflow to push images to Docker Hub.

Step 1: Add Docker Hub Credentials as Secrets

1. Go to your GitHub repository → Settings → Secrets and variables → Actions
2. Click "New repository secret"
3. Add `DOCKERHUB_USERNAME` with your Docker Hub username
4. Add `DOCKERHUB_TOKEN` with your Docker Hub access token (not your password)

Why secrets? Secrets are encrypted and never shown in logs. Hardcoding credentials in a YAML file would expose them to anyone who can read your repository.

Step 2: The Workflow

```yaml
# .github/workflows/docker.yml
name: Build and Push to Docker Hub

on:
  push:
    branches: [main]
    tags: ['v*']
  pull_request:
    branches: [main]

env:
  IMAGE_NAME: myusername/myapp
```

```yaml
jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Get the code
      - name: Checkout code
        uses: actions/checkout@v4

      # Step 2: Set up Docker Buildx (needed for advanced features)
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      # Step 3: Log in to Docker Hub
      - name: Log in to Docker Hub
        if: github.event_name != 'pull_request'
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      # Step 4: Generate tags and labels
      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.IMAGE_NAME }}
          tags: |
            type=sha,prefix=
            type=ref,event=branch
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=semver,pattern={{major}}
            type=raw,value=latest,enable={{is_default_branch}}

      # Step 5: Build and push
      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
```

What each step does:

1. Checkout — Clones your code onto the runner
2. Setup Buildx — Enables advanced Docker build features (caching, multi-platform builds)
3. Login — Authenticates with Docker Hub. The `if` condition skips login on pull requests (because PRs should not push images)
4. Metadata — Automatically generates tags based on the Git context (branch, tag, SHA)

5. Build and push — Builds the image and pushes it. On pull requests, it only builds (no push) to verify the build works

What Tags Are Generated?

Git Event	Tags Generated
Push to `main`	`main`, `latest`, `a1b2c3d`
Git tag `v1.2.3`	`1.2.3`, `1.2`, `1`, `a1b2c3d`
Pull request	No push (build only)

Building and Pushing to GitHub Container Registry

If you prefer keeping images next to your code on GitHub, push to GHCR instead.

```yaml
# .github/workflows/docker-ghcr.yml
name: Build and Push to GHCR

on:
  push:
    branches: [main]
    tags: ['v*']

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
```

```
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
```

Key differences from Docker Hub:

- `registry: ghcr.io` — Points to GitHub Container Registry
- `username: ${{ github.actor }}` — Uses the GitHub username of whoever triggered the workflow
- `password: ${{ secrets.GITHUB_TOKEN }}` — Automatically provided by GitHub Actions (no manual secret needed)
- `permissions: packages: write` — Grants permission to push to GHCR

Caching Docker Builds

Docker builds in CI can be slow because the runner starts fresh every time — there is no layer cache from previous builds. Caching fixes this.

Why Caching Matters

```
+-------------------------------------------------------------+
|              Without Caching                                |
|                                                             |
|   Build 1: Install dependencies    (2 minutes)              |
|   Build 2: Install dependencies    (2 minutes) <-- again!   |
|   Build 3: Install dependencies    (2 minutes) <-- again!   |
|                                                             |
```

```
|              With Caching                                  |
|                                                            |
|   Build 1: Install dependencies    (2 minutes)             |
|   Build 2: Use cached layer        (5 seconds)  <-- fast!  |
|   Build 3: Use cached layer        (5 seconds)  <-- fast!  |
+-----------------------------------------------------------+
```

GitHub Actions Cache

The `docker/build-push-action` supports caching through GitHub's cache storage:

```yaml
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

What this does:

- `cache-from: type=gha` — Pull cached layers from GitHub Actions cache
- `cache-to: type=gha,mode=max` — Push all layers to the cache after building
- `mode=max` — Cache ALL layers, not just the final image layers. This maximizes cache hits for multi-stage builds

Registry-Based Cache

Alternatively, store the cache in the registry itself:

```yaml
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=registry,ref=${{ env.IMAGE_NAME }}:buildcache
          cache-to: type=registry,ref=${{ env.IMAGE_NAME }}:buildcache,mode=max
```

This stores a special cache image in your registry alongside your regular images.

Multi-Platform Builds

Modern applications need to run on different CPU architectures. The most common are:

- amd64 (also called x86_64) — Standard Intel/AMD processors (most servers, PCs)
- arm64 (also called aarch64) — ARM processors (Apple Silicon Macs, AWS Graviton, Raspberry Pi)

Why Multi-Platform Matters

```
+-------------------------------------------------------------+
|              The Architecture Problem                       |
|                                                             |
|   You build on your Mac (Apple Silicon = arm64)             |
|   You deploy to a server (Intel = amd64)                    |
|                                                             |
|   Without multi-platform:                                   |
|     Your image only works on arm64.                         |
|     The server cannot run it.                               |
|                                                             |
|   With multi-platform:                                      |
|     Docker automatically picks the right version            |
|     for each architecture.                                  |
+-------------------------------------------------------------+
```

Multi-Platform Workflow

```yaml
      # Required for multi-platform builds
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build and push multi-platform
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          platforms: linux/amd64,linux/arm64
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

What the new parts do:

- `docker/setup-qemu-action` — Installs QEMU, which lets the amd64 runner emulate arm64 (and vice versa). QEMU stands for "Quick Emulator."
- `platforms: linux/amd64,linux/arm64` — Build the image for both architectures

When someone runs `docker pull myapp:1.0.0` on an Intel server, Docker automatically downloads the amd64 version. On an Apple Silicon Mac, it downloads the arm64 version. This happens transparently — the user does not need to know.

> Note: Multi-platform builds are slower because the runner must emulate the non-native architecture. The arm64 build on an amd64 runner can take 2-5 times longer than a native build.

Environment-Specific Deployments

In real projects, you deploy to multiple environments:

```
+-------------------------------------------------------------+
|              Environment Pipeline                           |
|                                                             |
|   Push to main branch                                       |
|        |                                                    |
|        v                                                    |
|   Build image -> Push to registry                           |
|        |                                                    |
|        v                                                    |
|   Deploy to STAGING (automatic)                             |
|        |                                                    |
|        v                                                    |
|   Manual approval                                           |
|        |                                                    |
|        v                                                    |
|   Deploy to PRODUCTION (after approval)                     |
+-------------------------------------------------------------+
```

Workflow with Staging and Production

```yaml
# .github/workflows/deploy.yml
name: Build, Push, and Deploy

on:
  push:
```

```
    branches: [main]
    tags: ['v*']

env:
  IMAGE_NAME: myusername/myapp

jobs:
  # Job 1: Build and push the image
  build:
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.IMAGE_NAME }}
          tags: |
            type=sha,prefix=
            type=semver,pattern={{version}}
      - uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # Job 2: Deploy to staging (automatic)
  deploy-staging:
    needs: build
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - name: Deploy to staging
        run: |
          echo "Deploying ${{ env.IMAGE_NAME }}:${{ needs.build.outputs.image-tag }} to...
          # Replace with your actual deployment command:
          # ssh staging-server "docker pull $IMAGE && docker compose up -d"

  # Job 3: Deploy to production (requires manual approval)
  deploy-production:
    needs: [build, deploy-staging]
    runs-on: ubuntu-latest
    environment: production
    if: startsWith(github.ref, 'refs/tags/v')
    steps:
      - name: Deploy to production
```

```
        run: |
          echo "Deploying ${{ env.IMAGE_NAME }}:${{ needs.build.outputs.image-tag }} to...
          # Replace with your actual deployment command
```

Key concepts:

- `needs: build` — The staging job waits for the build job to finish
- `needs: [build, deploy-staging]` — Production waits for both build AND staging
- `environment: production` — You can configure this environment in GitHub to require manual approval
- `if: startsWith(github.ref, 'refs/tags/v')` — Only deploy to production when a version tag is created

Setting Up Environment Protection

1. Go to GitHub → Repository Settings → Environments
2. Create environments: `staging` and `production`
3. For `production`, enable "Required reviewers" and add your team lead
4. Now, when the pipeline reaches the production step, it pauses and waits for approval

---

Complete Workflow File

Here is a production-ready workflow that combines everything from this chapter:

```yaml
# .github/workflows/docker.yml
#
# This workflow:
# - Builds a Docker image on every push and PR
# - Runs tests inside the container
# - Pushes to Docker Hub (on main branch and tags only)
# - Builds for both amd64 and arm64
# - Uses caching for faster builds
# - Generates proper semantic version tags

name: Docker CI/CD

on:
  push:
    branches: [main]
    tags: ['v*.*.*']
  pull_request:
```

```
    branches: [main]

env:
  IMAGE_NAME: myusername/myapp
  # Set to 'true' to enable multi-platform builds (slower but necessary)
  MULTI_PLATFORM: 'false'

jobs:
  # ---------------------------------------------
  # Job 1: Lint and test (fast feedback)
  # ---------------------------------------------
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Build test image
        run: docker build --target test -t myapp:test . 2>/dev/null || docker build -t ...

      - name: Run tests
        run: docker run --rm myapp:test npm test 2>/dev/null || echo "No test command f...

  # ---------------------------------------------
  # Job 2: Build and push Docker image
  # ---------------------------------------------
  build-and-push:
    runs-on: ubuntu-latest
    needs: test
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      # Set up QEMU for multi-platform builds
      - name: Set up QEMU
        if: env.MULTI_PLATFORM == 'true'
        uses: docker/setup-qemu-action@v3

      # Set up Buildx for advanced build features
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      # Log in to Docker Hub (skip on PRs)
      - name: Log in to Docker Hub
        if: github.event_name != 'pull_request'
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      # Log in to GHCR (skip on PRs)
```

```
      - name: Log in to GHCR
        if: github.event_name != 'pull_request'
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      # Generate tags based on Git context
      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: |
            ${{ env.IMAGE_NAME }}
            ghcr.io/${{ github.repository }}
          tags: |
            # Git short SHA
            type=sha,prefix=
            # Branch name
            type=ref,event=branch
            # PR number
            type=ref,event=pr
            # Semantic version tags (from Git tags like v1.2.3)
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=semver,pattern={{major}}
            # Latest tag for default branch
            type=raw,value=latest,enable={{is_default_branch}}

      # Build and push the image
      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          # Only push on main branch and tags, not on PRs
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          # Multi-platform: uncomment if needed
          platforms: ${{ env.MULTI_PLATFORM == 'true' && 'linux/amd64,linux/arm64' || '...
          # Cache layers in GitHub Actions cache
          cache-from: type=gha
          cache-to: type=gha,mode=max

      # Show what was built
      - name: Image digest
        run: echo "Image digest - ${{ steps.meta.outputs.tags }}"
```

How to use this workflow:

1. Copy the file to `.github/workflows/docker.yml` in your repository
2. Replace `myusername/myapp` with your Docker Hub username and image name

3. Add `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` as repository secrets
4. Push to `main` to trigger the workflow
5. Create a Git tag (`git tag v1.0.0 && git push --tags`) to trigger a release build

Common Mistakes

Mistake 1: Hardcoding Credentials in the Workflow

```yaml
# NEVER DO THIS
- name: Log in
  run: docker login -u myuser -p SuperSecret123
```

Anyone who can read your repository can see these credentials.

Fix: Use GitHub Secrets:

```yaml
- uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_TOKEN }}
```

Mistake 2: Pushing Images from Pull Requests

Pull requests can come from anyone, including external contributors. Pushing images from PRs could let attackers publish malicious images to your registry.

Fix: Only push from trusted branches:

```yaml
push: ${{ github.event_name != 'pull_request' }}
```

Mistake 3: Not Using Build Caching

Without caching, every CI build downloads all dependencies from scratch. This wastes time and money.

Fix: Add caching:

```yaml
cache-from: type=gha
cache-to: type=gha,mode=max
```

Mistake 4: Building Only for Your Architecture

You build on an amd64 CI runner, but your team uses Apple Silicon Macs. They pull the image and get an error about architecture mismatch.

Fix: Enable multi-platform builds with QEMU and Buildx.

Mistake 5: No Tests Before Pushing

Pushing an image that fails tests puts a broken image in your registry. Someone might pull and deploy it.

Fix: Always run tests before the push step. Use `needs: test` to enforce the dependency.

Best Practices

1. Never hardcode credentials — Use repository secrets for all sensitive values
2. Run tests before pushing — A broken image in the registry is worse than no image at all
3. Use caching — Saves minutes on every build, which adds up to hours over time
4. Build on PRs without pushing — Verify the build works before merging
5. Use the metadata action for tagging — It handles complex tagging logic correctly
6. Push to multiple registries — Docker Hub for public access, GHCR for GitHub integration
7. Pin action versions — Use `@v4` or `@v3`, not `@latest`, for reproducible pipelines
8. Enable multi-platform builds when your users run different architectures
9. Use environments with protection rules for production deployments
10. Keep workflows DRY — Extract common steps into reusable workflows or composite actions

---

Quick Summary

CI/CD automates the process of building, testing, tagging, and pushing Docker images every time you push code. GitHub Actions provides a free, integrated CI/CD system that defines workflows in YAML files. The `docker/build-push-action` handles the Docker-specific steps, while `docker/metadata-action` generates appropriate tags. Caching with GitHub Actions cache dramatically speeds up builds. Multi-platform builds using QEMU and Buildx produce images for both Intel and ARM processors. Environment protection rules add manual approval gates for production deployments.

Key Points

- CI/CD automates building, testing, and deploying — like an assembly line
- GitHub Actions workflows live in `.github/workflows/` as YAML files
- Secrets store credentials securely — never hardcode them
- docker/build-push-action builds and pushes images in one step
- docker/metadata-action generates smart tags from Git context
- Caching (`cache-from: type=gha`) reuses layers between builds
- Multi-platform builds use QEMU to build for amd64 and arm64
- Pull requests build but do not push — verify before merging
- Environments with protection rules gate production deployments
- Always test before pushing to avoid broken images in the registry

---

Practice Questions

1. What is the difference between Continuous Integration and Continuous Deployment? Use the assembly line analogy to explain.
2. Why should you never hardcode Docker Hub credentials in a GitHub Actions workflow file? What should you use instead?
3. What does the `if: github.event_name != 'pull_request'` condition do in the login and push steps? Why is this important?
4. Explain what `cache-from: type=gha` and `cache-to: type=gha,mode=max` do. Why is `mode=max` important for multi-stage builds?
5. What is QEMU and why is it needed for multi-platform Docker builds in CI? What is the trade-off?

Exercises

Exercise 1: Set Up a Basic CI Pipeline

1. Create a simple Node.js or Python application with a Dockerfile
2. Initialize a Git repository and push it to GitHub
3. Create `.github/workflows/docker.yml` with a workflow that builds the image on every push
4. Push a change and verify the workflow runs in the Actions tab
5. Introduce a build error (like a typo in the Dockerfile) and verify the workflow fails with a red X

Exercise 2: Push to Docker Hub

1. Create Docker Hub access token (Account Settings → Security → New Access Token)
2. Add `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` as repository secrets on GitHub
3. Extend your workflow to log in and push to Docker Hub
4. Push to `main` and verify the image appears on Docker Hub
5. Create a Git tag (`git tag v1.0.0 && git push --tags`) and verify the semantic version tags are created

Exercise 3: Add Caching and Multi-Platform

1. Add caching to your workflow using `cache-from` and `cache-to`
2. Push twice and compare the build times (check the Actions tab for durations)
3. Enable multi-platform builds by adding QEMU setup and `platforms: linux/amd64,linux/arm64`
4. Push and verify the build completes for both platforms
5. On Docker Hub, check that the image shows both architectures

---

What Is Next?

Your images are now built and pushed automatically. But are they secure? In the next chapter, we explore Docker Security — running containers as non-root users, scanning

images for vulnerabilities, managing secrets safely, and hardening your containers against attacks. Security is not optional; it is a requirement for any production deployment.

Chapter 22: Docker Security

What You Will Learn

By the end of this chapter, you will be able to:

- Understand why Docker security matters and common attack vectors
- Run containers as non-root users with the USER instruction
- Use read-only filesystems to prevent tampering
- Manage sensitive data with Docker secrets
- Scan images for vulnerabilities using Docker Scout and Trivy
- Choose trusted base images to reduce risk
- Minimize the attack surface using distroless and minimal images
- Prevent sensitive data from leaking into image layers
- Enable Docker Content Trust for image verification
- Follow a comprehensive security checklist

Why This Chapter Matters

Imagine you leave your house with the front door wide open, all windows unlocked, your car keys on the dashboard, and your bank statements on the porch. Nobody forced you to do this — you just did not think about security.

Many Docker deployments are exactly like this. Containers run as root (the all-powerful superuser), images contain known vulnerabilities, passwords are baked into image layers, and anyone can push to the registry. Each of these is an open door for attackers.

Docker containers are NOT sandboxes. They share the host's kernel. A container running as root with the wrong configuration can access the host system, read other containers' data, or become a launching point for attacks.

Security is not something you add later. It must be part of every Dockerfile, every deployment, and every decision you make.

Running as Non-Root

By default, processes inside a Docker container run as root — the superuser with unlimited permissions. This is the single most common Docker security mistake.

Why Root Is Dangerous

```
+-------------------------------------------------------------+
|              The Root Problem                               |
|                                                             |
|   Container running as root:                                |
|   +-----------------------------------------------------+   |
|   | root (UID 0)                                        |   |
|   |                                                     |   |
|   | Can read/write ALL files in the container          |   |
|   | Can install software                                |   |
|   | Can modify system settings                         |   |
|   | If the container "escapes," it is root on HOST      |   |
|   +-----------------------------------------------------+   |
|                                                             |
|   Container running as non-root:                            |
|   +-----------------------------------------------------+   |
|   | appuser (UID 1001)                                  |   |
|   |                                                     |   |
|   | Can only read/write its own files                   |   |
|   | Cannot install software                            |   |
|   | Cannot modify system settings                      |   |
|   | If it "escapes," it is a nobody on the HOST         |   |
|   +-----------------------------------------------------+   |
+-------------------------------------------------------------+
```

The USER Instruction

The `USER` instruction in a Dockerfile sets which user the container process runs as.

```dockerfile
# Dockerfile
FROM node:20-alpine

# Create a non-root user and group
```

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Set the working directory
WORKDIR /app

# Copy files and install dependencies as root (needed for npm install)
COPY package*.json ./
RUN npm ci --only=production

# Copy application code
COPY . .

# Change ownership of the app directory to the non-root user
RUN chown -R appuser:appgroup /app

# Switch to the non-root user
USER appuser

# Start the application
CMD ["node", "server.js"]
```

Line-by-line explanation:

- `addgroup -S appgroup` — Creates a system group called "appgroup." The `-S` flag means "system group" (no home directory needed)
- `adduser -S appuser -G appgroup` — Creates a system user called "appuser" in the "appgroup" group
- `RUN chown -R appuser:appgroup /app` — Changes ownership of `/app` so the non-root user can read the files
- `USER appuser` — All subsequent commands (and the final CMD) run as "appuser" instead of root

Verify It Works

```bash
# Build the image
docker build -t myapp-secure .

# Check which user the container runs as
docker run --rm myapp-secure whoami
```

Output:

```
appuser
```

```bash
# Try to do something only root can do
docker run --rm myapp-secure cat /etc/shadow
```

Output:

```
cat: can't open '/etc/shadow': Permission denied
```

The non-root user cannot access sensitive system files. This is exactly what we want.

Override at Runtime

Even if the Dockerfile uses `USER root`, you can force a non-root user at runtime:

```bash
docker run --user 1001:1001 myapp
```

This runs the container as UID 1001, regardless of what the Dockerfile specifies.

Read-Only Filesystem

A read-only filesystem prevents any process inside the container from writing to the filesystem. If an attacker gains access, they cannot modify application files, install malware, or alter configuration.

Using --read-only

```bash
docker run --read-only myapp
```

What happens: Any attempt to write to the filesystem fails:

```bash
docker run --read-only myapp touch /tmp/test.txt
```

Output:

```
touch: cannot touch '/tmp/test.txt': Read-only file system
```

Allowing Specific Writable Directories

Most applications need to write to some directories (temp files, logs, caches). Use `--tmpfs` to create writable temporary directories:

```bash
docker run --read-only \
  --tmpfs /tmp \
  --tmpfs /var/log \
  myapp
```

What this does:

- The entire filesystem is read-only
- `/tmp` and `/var/log` are writable, but stored in memory (not on disk)
- Data in `--tmpfs` directories is lost when the container stops

In Docker Compose

```yaml
services:
  web:
    image: myapp:1.0.0
    read_only: true
    tmpfs:
      - /tmp
      - /var/log
```

```
+-------------------------------------------------------------+
|              Read-Only Filesystem                            |
|                                                              |
|   +-----------------------------------------------------+   |
|   | Container Filesystem                                |   |
|   |                                                     |   |
|   |   /app          READ-ONLY  (application code)       |   |
|   |   /etc          READ-ONLY  (configuration)          |   |
|   |   /usr          READ-ONLY  (system files)           |   |
|   |   /tmp          WRITABLE   (temporary files)        |   |
|   |   /var/log      WRITABLE   (log files)              |   |
|   |                                                     |   |
|   |   Attacker cannot modify /app or install tools      |   |
|   +-----------------------------------------------------+   |
+-------------------------------------------------------------+
```

Docker Secrets

Secrets are sensitive data that your application needs but should never be stored in plain text inside an image. Examples include:

- Database passwords
- API keys
- TLS certificates
- Encryption keys

The Wrong Way: Environment Variables in Dockerfiles

```dockerfile
# NEVER DO THIS
ENV DATABASE_PASSWORD=SuperSecret123
```

This bakes the password into the image. Anyone who pulls the image can see it:

```bash
docker inspect myapp | grep DATABASE_PASSWORD
# Output: "DATABASE_PASSWORD=SuperSecret123"
```

Even if you delete the variable in a later layer, it is still visible in the image history:

```bash
docker history myapp
```

The Wrong Way: Copying Secret Files

```dockerfile
# NEVER DO THIS
COPY .env /app/.env
```

The `.env` file is now part of the image layer. Even if you delete it in a later `RUN` instruction, it exists in an earlier layer and can be extracted.

The Right Way: Docker Swarm Secrets

Docker Swarm has built-in secrets management (covered more in Chapter 24):

```bash
# Create a secret
echo "SuperSecret123" | docker secret create db_password -

# Use it in a service
docker service create \
  --name webapp \
  --secret db_password \
  myapp:1.0.0
```

Inside the container, the secret is available as a file at `/run/secrets/db_password`:

```bash
# In your application code
cat /run/secrets/db_password
# Output: SuperSecret123
```

The Right Way: Environment Variables at Runtime

Pass secrets at runtime, not build time:

```bash
docker run -e DATABASE_PASSWORD=SuperSecret123 myapp
```

Or use an environment file:

```bash
# .env file (NOT committed to Git, NOT copied into the image)
DATABASE_PASSWORD=SuperSecret123
API_KEY=abc123def456

docker run --env-file .env myapp
```

The Right Way: Docker Compose Secrets

```yaml
# docker-compose.yml
services:
  web:
    image: myapp:1.0.0
    secrets:
      - db_password
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

The secret file is mounted read-only at `/run/secrets/db_password` inside the container.

```
+-------------------------------------------------------------+
|              Secrets: Wrong vs Right                        |
|                                                             |
|   WRONG:                                                    |
|   +---------------------+                                   |
|   | Dockerfile          |  ENV PASSWORD=secret              |
|   | Image Layer         |  Password baked in forever        |
|   +---------------------+                                   |
|                                                             |
|   RIGHT:                                                    |
|   +---------------------+       +----------------+          |
|   | Dockerfile          |       | Runtime        |          |
|   | No secrets inside   | <---- | -e PASSWORD=.. |          |
|   +---------------------+       | --secret ...   |          |
|                                 +----------------+          |
|                                                             |
|   Secrets are injected at runtime, never stored in image.   |
+-------------------------------------------------------------+
```

Image Scanning

Image scanning analyzes your Docker image for known security vulnerabilities. Think of it as a virus scan for your Docker images.

What Scanners Look For

- CVEs (Common Vulnerabilities and Exposures) — Known security bugs in software packages
- Outdated packages — Libraries with newer, patched versions available
- Misconfigurations — Insecure Dockerfile patterns
- Malware — Known malicious code

Docker Scout

Docker Scout is Docker's built-in scanning tool.

```bash
# Scan a local image
docker scout cves myapp:1.0.0
```

Sample output:

```
    i New version 1.14.0 available (installed version is 1.13.1)
    ✓ Image stored for indexing
    ✗ Detected 3 vulnerable packages with a total of 12 vulnerabilities

   0C     3H     5M     4L  myapp:1.0.0

  Pkg          Installed  Fixed     Vulnerability     Severity
  curl         7.88.1     8.4.0     CVE-2023-38545   HIGH
  openssl      3.0.2      3.0.12    CVE-2023-5678    HIGH
  openssl      3.0.2      3.0.12    CVE-2023-5679    HIGH
  zlib         1.2.13     1.3       CVE-2023-45853   MEDIUM
```

Reading the output:

- `0C` — 0 Critical vulnerabilities
- `3H` — 3 High severity vulnerabilities
- `5M` — 5 Medium severity vulnerabilities
- `4L` — 4 Low severity vulnerabilities
- `Fixed` column — Shows which version fixes the vulnerability. Update to this version.

```bash
# Get recommendations for fixing vulnerabilities
docker scout recommendations myapp:1.0.0
```

Trivy

Trivy is a popular open-source scanner by Aqua Security. It is fast, comprehensive, and free.

```bash
# Install Trivy (macOS)
brew install trivy

# Scan a Docker image
trivy image myapp:1.0.0
```

Sample output:

```
myapp:1.0.0 (alpine 3.18.4)
Total: 5 (UNKNOWN: 0, LOW: 2, MEDIUM: 2, HIGH: 1, CRITICAL: 0)

+---------+------------------+----------+---------+---------+
| Library | Vulnerability    | Severity | Version | Fixed   |
+---------+------------------+----------+---------+---------+
| curl    | CVE-2023-38545   | HIGH     | 7.88.1  | 8.4.0   |
```

```
| zlib    | CVE-2023-45853  | MEDIUM   | 1.2.13  | 1.3     |
| libxml2 | CVE-2023-45322  | MEDIUM   | 2.11.5  | 2.11.6  |
| busybox | CVE-2023-42365  | LOW      | 1.36.1  | 1.36.2  |
| musl    | CVE-2023-43789  | LOW      | 1.2.4   | 1.2.5   |
+---------+-----------------+----------+---------+---------+
```

Scanning in CI/CD

Add scanning to your CI/CD pipeline so vulnerabilities are caught before deployment:

```yaml
# In your GitHub Actions workflow
      - name: Scan image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: myapp:1.0.0
          format: table
          exit-code: 1           # Fail the build if vulnerabilities found
          severity: CRITICAL,HIGH  # Only fail on critical and high
```

Setting `exit-code: 1` means the build fails if critical or high vulnerabilities are found, preventing the image from being pushed to the registry.

Trusted Base Images

The base image you choose in your `FROM` instruction is the foundation of your container's security. A compromised or outdated base image undermines everything you build on top.

Use Official Images

Official images are reviewed and maintained by Docker or the software vendor:

```dockerfile
# Good — official image
FROM node:20-alpine

# Risky — random user's image
FROM someguy/node:latest
```

Pin Specific Versions

```dockerfile
# Bad — could change at any time
FROM node:latest

# Better — pinned to minor version
FROM node:20-alpine

# Best — pinned to exact version with digest
FROM node:20.10.0-alpine3.19@sha256:abc123...
```

The digest (`@sha256:...`) is a unique hash of the image. Even if someone overwrites the `20.10.0-alpine3.19` tag, the digest ensures you get the exact same image every time.

Prefer Minimal Base Images

```
+-------------------------------------------------------------+
|              Base Image Comparison                          |
|                                                             |
|   Image              Size      Packages   Attack Surface    |
|   ---------------    ------    --------   ---------------   |
|   ubuntu:22.04       77 MB     ~400       Large             |
|   debian:bookworm    124 MB    ~300       Large             |
|   node:20            1.1 GB    ~600       Very large        |
|   node:20-slim       200 MB    ~150       Medium            |
|   node:20-alpine     130 MB    ~50        Small             |
|   gcr.io/distroless  ~20 MB    ~5         Tiny              |
|                                                             |
|   Fewer packages = fewer potential vulnerabilities.         |
+-------------------------------------------------------------+
```

Minimizing the Attack Surface

The attack surface is everything in your container that an attacker could potentially exploit. The smaller the attack surface, the fewer ways an attacker can get in.

Remove Unnecessary Tools

```dockerfile
# After installing build dependencies, remove them
RUN apt-get update && \
    apt-get install -y --no-install-recommends build-essential && \
    npm ci --only=production && \
    apt-get purge -y build-essential && \
```

```
    apt-get autoremove -y && \
    rm -rf /var/lib/apt/lists/*
```

Or better yet, use multi-stage builds so build tools never appear in the final image:

```dockerfile
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production (no build tools)
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER node
CMD ["node", "dist/server.js"]
```

Distroless Images

Distroless images from Google contain ONLY your application and its runtime dependencies. No shell, no package manager, no system utilities.

```dockerfile
# Multi-stage build with distroless final image
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

FROM gcr.io/distroless/nodejs20-debian12
WORKDIR /app
COPY --from=builder /app .
CMD ["server.js"]
```

Why distroless is more secure:

- No shell → Attackers cannot open a shell inside the container
- No package manager → Attackers cannot install additional tools
- No utilities (curl, wget, etc.) → Attackers cannot download payloads
- Dramatically fewer packages → Dramatically fewer vulnerabilities

The trade-off: Debugging is harder because you cannot `docker exec -it container sh`. For debugging, use a sidecar container or multi-stage build with a debug target.

No Sensitive Data in Images

Every layer of a Docker image is permanent and inspectable. Deleting a file in a later layer does NOT remove it from earlier layers.

The Layer Problem

```dockerfile
# This is INSECURE even though we delete the file
COPY credentials.json /app/credentials.json
RUN process-credentials.sh
RUN rm /app/credentials.json  # File is STILL in the layer above!
```

```
+-------------------------------------------------------------+
|              Image Layers Are Permanent                      |
|                                                              |
|   Layer 3:  RUN rm credentials.json   (file "removed")    |
|   Layer 2:  RUN process-credentials   (file used)         |
|   Layer 1:  COPY credentials.json     (file EXISTS here!) |
|                                                              |
|   An attacker can extract Layer 1 and read the file.       |
|   docker history and docker save reveal all layers.        |
+-------------------------------------------------------------+
```

How to Check for Leaked Secrets

```bash
# View all layers and commands
docker history myapp:1.0.0

# Export and inspect layers
docker save myapp:1.0.0 -o image.tar
tar xf image.tar
# Each layer is a separate tar file you can inspect
```

Rules for Keeping Secrets Out

1. Use `.dockerignore` to exclude sensitive files from the build context:

```
# .dockerignore
.env
*.pem
*.key
credentials.json
secrets/
.git
```

2. Use multi-stage builds to ensure build-time secrets do not appear in the final image:

```dockerfile
FROM alpine AS builder
# Use secrets only in the build stage
COPY credentials.json .
RUN process-with-credentials.sh

FROM alpine
# Final image has NO credentials
COPY --from=builder /app/output ./output
```

3. Use BuildKit secrets for build-time secrets that should never be in any layer:

```dockerfile
# syntax=docker/dockerfile:1
FROM node:20-alpine
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm ci --only=production
```

```bash
docker build --secret id=npmrc,src=.npmrc -t myapp .
```

The `.npmrc` file is available during the build but is NOT stored in any layer.

Docker Content Trust

Docker Content Trust (DCT) ensures that the images you pull are signed by the publisher and have not been tampered with.

The Problem Without DCT

```
+-------------------------------------------------------------+
|              Man-in-the-Middle Attack                       |
|                                                             |
```

```
|   You:  docker pull myapp:1.0                              |
|                |                                           |
|                v                                           |
|   Attacker:  Intercepts the pull and serves a              |
|              modified image with malware.                   |
|                |                                           |
|                v                                           |
|   You run the malware thinking it is your app.             |
+------------------------------------------------------------+
```

Enabling Docker Content Trust

```bash
# Enable for the current shell session
export DOCKER_CONTENT_TRUST=1

# Now Docker refuses to pull unsigned images
docker pull unsigned-image:latest
# Error: remote trust data does not exist

# Pull a signed image (works fine)
docker pull docker/trusttest:latest
```

Signing Your Images

```bash
# Enable content trust
export DOCKER_CONTENT_TRUST=1

# Push — Docker will ask you to create signing keys
docker push myusername/myapp:1.0.0
```

The first time, Docker generates a root key and a repository key. Store these keys safely — losing them means you cannot sign future images for that repository.

When to Use DCT

- Production environments — Always verify image authenticity
- Regulated industries — Compliance often requires image signing
- Open source projects — Lets users verify they are running the official image

---

Security Checklist

Use this checklist for every Docker deployment:

Dockerfile Security

```
[ ] Run as non-root user (USER instruction)
[ ] Use a minimal base image (alpine, slim, or distroless)
[ ] Pin base image versions (avoid :latest)
[ ] No secrets in ENV instructions
[ ] No secrets copied into layers
[ ] Use .dockerignore to exclude sensitive files
[ ] Use multi-stage builds to exclude build tools
[ ] Remove unnecessary packages after installation
[ ] Set HEALTHCHECK for monitoring
```

Runtime Security

```
[ ] Use --read-only filesystem where possible
[ ] Pass secrets via runtime environment or Docker secrets
[ ] Drop unnecessary Linux capabilities (--cap-drop ALL)
[ ] Limit memory and CPU (--memory, --cpus)
[ ] Use --no-new-privileges to prevent privilege escalation
[ ] Set restart policy appropriately
[ ] Do not mount the Docker socket into containers
[ ] Use network segmentation (separate Docker networks)
```

Registry Security

```
[ ] Use private repositories for proprietary images
[ ] Enable vulnerability scanning on the registry
[ ] Use access tokens instead of passwords
[ ] Enable Docker Content Trust for signed images
[ ] Set up image retention policies
[ ] Regularly rotate access tokens
```

CI/CD Security

```
[ ] Scan images for vulnerabilities in the pipeline
[ ] Fail builds on critical/high vulnerabilities
[ ] Never hardcode credentials in workflow files
[ ] Do not push images from pull requests
[ ] Pin CI/CD action versions
[ ] Use separate tokens for CI/CD (not personal credentials)
```

Advanced Runtime Hardening

```bash
# Drop ALL Linux capabilities and add back only what you need
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp

# Prevent privilege escalation
docker run --security-opt no-new-privileges myapp

# Limit resources
docker run --memory 512m --cpus 1.0 myapp

# Read-only filesystem with specific writable paths
docker run --read-only --tmpfs /tmp myapp
```

Common Mistakes

Mistake 1: Running as Root Because "It Is Easier"

```dockerfile
# The entire Dockerfile runs as root by default
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "server.js"]
# No USER instruction — runs as root!
```

Fix: Always add a USER instruction before the CMD.

Mistake 2: Baking Passwords into ENV

```dockerfile
# NEVER DO THIS
ENV DB_PASSWORD=MyPassword123
```

Fix: Pass at runtime with `-e` or use Docker secrets.

Mistake 3: Using a Full Operating System as Base Image

```dockerfile
# Unnecessary — includes compilers, editors, and hundreds of tools
FROM ubuntu:22.04
```

Fix: Use `alpine`, `slim`, or `distroless` variants.

Mistake 4: Not Scanning Images

Building an image and pushing it without scanning is like shipping a product without quality control.

Fix: Add `trivy image` or `docker scout cves` to your CI/CD pipeline.

Mistake 5: Mounting the Docker Socket

```bash
# EXTREMELY DANGEROUS — gives the container full control of Docker
docker run -v /var/run/docker.sock:/var/run/docker.sock myapp
```

This gives the container the ability to create, modify, and delete ANY container on the host. Only do this when absolutely necessary (like for monitoring tools) and never in production without understanding the risks.

Best Practices

1. Always run as non-root — Use the `USER` instruction in every Dockerfile
2. Use minimal base images — Fewer packages means fewer vulnerabilities
3. Scan images regularly — Add scanning to CI/CD and run it on a schedule
4. Never store secrets in images — Use runtime injection or Docker secrets
5. Use read-only filesystems — Prevent tampering with application files
6. Drop unnecessary capabilities — Use `--cap-drop ALL` and add back only what you need
7. Pin base image versions — Avoid unexpected changes from moving tags
8. Use multi-stage builds — Keep build tools out of production images
9. Enable Docker Content Trust — Verify image authenticity
10. Keep images updated — Rebuild regularly to pick up security patches

---

Quick Summary

Docker security starts with running containers as non-root users and using minimal base images to reduce the attack surface. Read-only filesystems prevent tampering, while Docker secrets keep sensitive data out of image layers. Image scanning with tools like Docker Scout and Trivy catches known vulnerabilities before deployment. Distroless images provide the smallest possible attack surface by including only your application and its runtime. Docker Content Trust verifies that images have not been tampered with. Following a security checklist ensures you do not miss any important hardening step.

Key Points

- Never run as root — Use the `USER` instruction to create and switch to a non-root user
- Read-only filesystems (`--read-only`) prevent processes from modifying the container
- Docker secrets inject sensitive data at runtime, never storing it in image layers
- Image scanning (Docker Scout, Trivy) finds known vulnerabilities in your images
- Minimal base images (alpine, slim, distroless) reduce the number of potential vulnerabilities
- Multi-stage builds keep build tools and secrets out of the final image
- `.dockerignore` prevents sensitive files from entering the build context
- Docker Content Trust verifies that images are signed and unmodified
- Drop capabilities (`--cap-drop ALL`) removes unnecessary Linux permissions
- Image layers are permanent — deleting a file in a later layer does not remove it from earlier layers

---

Practice Questions

1. Why is running a container as root dangerous? What could happen if a root container has a vulnerability that allows "container escape"?
2. You need to pass a database password to your container. Describe two WRONG ways and two RIGHT ways to do this.
3. What is a distroless image? Why is it more secure than a regular base image? What is the trade-off?

4. You run `docker history myapp` and see a layer that contains `COPY credentials.json /app/`. The next layer shows `RUN rm /app/credentials.json`. Is the credentials file still accessible? Why or why not?
5. What does `--cap-drop ALL` do? Why would you want to use it?

Exercises

Exercise 1: Secure a Dockerfile

Start with this insecure Dockerfile:

```dockerfile
FROM node:20
WORKDIR /app
ENV API_KEY=sk-secret-key-12345
COPY . .
RUN npm install
CMD ["node", "server.js"]
```

Fix all security issues:

1. Add a non-root user
2. Remove the secret from ENV
3. Use an alpine base image
4. Add a `.dockerignore` file
5. Use multi-stage build to minimize the final image

Exercise 2: Scan and Fix Vulnerabilities

1. Build a Docker image using `FROM node:20`
2. Scan it with `docker scout cves` or `trivy image`
3. Note the number and severity of vulnerabilities
4. Rebuild using `FROM node:20-alpine`
5. Scan again and compare the results
6. Try `FROM gcr.io/distroless/nodejs20-debian12` and scan a third time

Exercise 3: Read-Only Container

1. Run a container with `--read-only`: `docker run --read-only nginx`

2. It will likely fail. Read the error message.
3. Add `--tmpfs` flags for the directories nginx needs to write to
4. Verify the container runs successfully
5. Try to create a file in a non-tmpfs directory to confirm it is read-only

---

What Is Next?

Security is one piece of running containers in production. In the next chapter, we cover Docker in Production — resource limits to prevent containers from consuming all available memory, restart policies for self-healing, logging drivers for centralized log management, monitoring with Prometheus, and graceful shutdown handling. These are the operational skills that separate development experiments from production-ready deployments.

Chapter 23: Docker in Production

What You Will Learn

By the end of this chapter, you will be able to:

- Set resource limits on containers to prevent resource exhaustion
- Configure restart policies so containers self-heal after failures
- Choose and configure logging drivers for centralized log management
- Monitor containers with docker stats, cAdvisor, and Prometheus
- Handle graceful shutdown with proper SIGTERM handling
- Follow a production readiness checklist

Why This Chapter Matters

Running Docker on your laptop for development is forgiving. If a container uses too much memory, you close it. If it crashes, you restart it manually. If the logs are messy, you scroll through the terminal.

Production is a different world. A container that consumes all available memory can take down every other container on the server. A crashed container that does not restart means your application is offline at 3 AM. Logs that are not forwarded to a central system are lost when the container is removed.

Think of the difference like this: driving in a parking lot versus driving on a highway. In a parking lot, mistakes are harmless. On a highway, every detail matters — speed, fuel, tire pressure, and mirrors. This chapter teaches you the "highway driving" skills for Docker.

Resource Limits

By default, a container can use as much CPU and memory as it wants. On a shared server, one runaway container can starve every other container of resources.

Memory Limits

```bash
# Limit container to 512 MB of memory
docker run --memory 512m myapp

# Limit to 1 GB
docker run --memory 1g myapp

# Limit memory AND swap (swap is disk used as overflow memory)
docker run --memory 512m --memory-swap 1g myapp
```

What `--memory` does:

- Sets a hard limit on how much RAM the container can use
- If the container tries to use more, it is killed by the Linux kernel (OOM — Out of Memory)
- The container's exit code will be 137 (which means "killed by signal 9 / SIGKILL")

```
+-------------------------------------------------------------+
|              Memory Limits                                   |
|                                                              |
|   Server has 4 GB RAM                                        |
|                                                              |
|   Without limits:                                            |
|   +-----------+ +----------+ +-----------+                  |
|   | Container | | Container| | Container |                  |
|   | A: 3 GB!! | | B: 0.5GB | | C: DEAD   |                 |
|   +-----------+ +----------+ +-----------+                  |
|   Container A consumed all memory, C was killed.             |
|                                                              |
|   With limits (--memory 1g each):                            |
|   +-----------+ +----------+ +-----------+                  |
|   | Container | | Container| | Container |                  |
|   | A: 1 GB   | | B: 0.5GB | | C: 0.5GB  |                 |
|   +-----------+ +----------+ +-----------+                  |
|   Each container gets its fair share. None can hog.          |
+-------------------------------------------------------------+
```

CPU Limits

```bash
# Limit to 1 CPU core
docker run --cpus 1.0 myapp

# Limit to half a CPU core
docker run --cpus 0.5 myapp

# Limit to 2 CPU cores
docker run --cpus 2.0 myapp
```

What `--cpus` does:

- Sets how much CPU time the container can use
- `--cpus 1.0` means the container can use the equivalent of one full core
- `--cpus 0.5` means the container gets at most 50% of one core
- Unlike memory limits, exceeding the CPU limit does NOT kill the container — it just slows down

CPU Shares (Relative Priority)

```bash
# Higher number = higher priority when CPU is contended
docker run --cpu-shares 1024 high-priority-app
docker run --cpu-shares 512  low-priority-app
```

CPU shares only matter when the CPU is fully loaded. When CPU is available, both containers can use as much as they need.

Resource Limits in Docker Compose

```yaml
services:
  web:
    image: myapp:1.0.0
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M
        reservations:
          cpus: '0.25'
          memory: 128M
```

Explanation:

- `limits` — Maximum resources the container can use

- `reservations` — Minimum resources guaranteed to the container. Docker will not schedule the container on a host that cannot provide at least this much.

Checking Resource Usage

```bash
docker stats
```

Output:

```
CONTAINER ID   NAME    CPU %   MEM USAGE / LIMIT   MEM %   NET I/O         BLOCK I/O
a1b2c3d4e5f6   web     2.5%    125MiB / 512MiB     24.4%   1.2MB / 500kB   0B / 0B
f6e5d4c3b2a1   db      15.3%   350MiB / 1GiB       34.2%   2.5MB / 1.8MB   50MB / 10MB
```

This shows real-time resource usage for all running containers.

Restart Policies

A restart policy tells Docker what to do when a container stops. Without a restart policy, a crashed container stays dead until someone manually restarts it.

Available Restart Policies

Policy	What It Does
`no`	Never restart (default)
`always`	Always restart, no matter why it stopped
`unless-stopped`	Always restart UNLESS manually stopped with `docker stop`
`on-failure`	Restart only if the container exits with a non-zero exit code

Using Restart Policies

```bash
# Never restart (default behavior)
docker run --restart no myapp

# Always restart — even after Docker daemon restarts
docker run --restart always myapp

# Restart unless manually stopped
docker run --restart unless-stopped myapp

# Restart only on failure, up to 5 attempts
docker run --restart on-failure:5 myapp
```

When to Use Each Policy

```
+-------------------------------------------------------------+
|              Restart Policy Decision Tree                    |
|                                                             |
|   Is this a development container?                          |
|     YES --> "no" (you want to see crashes)                  |
|     NO  --> Continue                                        |
|                                                             |
|   Should it survive Docker daemon restarts (reboots)?       |
|     YES --> Continue                                        |
|     NO  --> "on-failure" (restart only on crashes)          |
|                                                             |
|   Should `docker stop` permanently stop it?                 |
|     YES --> "unless-stopped"                                |
|     NO  --> "always"                                        |
+-------------------------------------------------------------+
```

Detailed explanation of each:

`no` (default): The container stops and stays stopped. Use this for development, one-off tasks, and batch jobs.

```bash
docker run --restart no myapp
# Container crashes at 2 AM. It stays dead until morning.
```

`always`: Docker restarts the container whenever it stops, regardless of the exit code. It even restarts after the Docker daemon restarts (like after a server reboot).

```bash
docker run --restart always myapp
# Container crashes at 2 AM. Docker restarts it immediately.
# Server reboots at 3 AM. Docker restarts it after boot.
# You run `docker stop myapp`. Docker restarts it anyway!
```

`unless-stopped`: Like `always`, but respects `docker stop`. If you manually stop the container, it stays stopped even after a daemon restart.

```bash
docker run --restart unless-stopped myapp
# Container crashes at 2 AM. Docker restarts it.
# You run `docker stop myapp`. It stays stopped. Docker respects your intent.
```

`on-failure[:max-retries]`: Restarts only when the container exits with a non-zero exit code (meaning it crashed, not exited cleanly). The optional `max-retries` limits restart attempts.

```bash
docker run --restart on-failure:5 myapp
# Container crashes (exit code 1). Docker restarts it.
# After 5 consecutive failures, Docker gives up.
# Container exits with code 0 (clean exit). Docker does NOT restart it.
```

Restart Policies in Docker Compose

```yaml
services:
  web:
    image: myapp:1.0.0
    restart: unless-stopped

  worker:
    image: myapp-worker:1.0.0
    restart: on-failure:5

  one-off-task:
    image: myapp-migrate:1.0.0
    restart: "no"
```

Logging Drivers

By default, Docker captures container logs (stdout and stderr) and stores them as JSON files on the host. This works for development but creates problems in production:

- Log files grow without limit and fill up the disk
- Logs are lost when a container is removed
- You cannot search across multiple containers
- No central dashboard for monitoring

Logging drivers send container logs to external systems.

Default: json-file

```bash
# View default logging driver
docker info | grep "Logging Driver"
# Output: Logging Driver: json-file
```

```bash
# Configure max file size and rotation
docker run \
  --log-driver json-file \
  --log-opt max-size=10m \
  --log-opt max-file=3 \
  myapp
```

What this does:

- `max-size=10m` — Each log file can be at most 10 MB
- `max-file=3` — Keep at most 3 log files (total max: 30 MB)
- When the current file reaches 10 MB, it is rotated: `container-id.log` becomes `container-id.log.1`, and a new log file starts

> Critical: Without `max-size` and `max-file`, log files grow forever. A busy application can fill a 100 GB disk in days. Always set these limits.

syslog

Sends logs to a syslog server, which is the standard logging system on Linux:

```bash
docker run \
  --log-driver syslog \
  --log-opt syslog-address=udp://logserver:514 \
  --log-opt tag="myapp" \
  myapp
```

fluentd

Sends logs to Fluentd, a popular log collection and forwarding tool:

```bash
docker run \
  --log-driver fluentd \
  --log-opt fluentd-address=localhost:24224 \
  --log-opt tag="docker.myapp" \
  myapp
```

```
+-------------------------------------------------------------+
|              Centralized Logging Architecture               |
|                                                             |
|   Server 1              Server 2                            |
|   +---------+           +---------+                         |
|   | App A   |           | App C   |                         |
|   | App B   |           | App D   |                         |
|   +---------+           +---------+                         |
|       |                     |                               |
|       v                     v                               |
|   +-----------------------------------------+               |
|   |          Fluentd / Logstash             |               |
|   |      (collects and forwards logs)       |               |
|   +-----------------------------------------+               |
|       |                                                     |
|       v                                                     |
|   +-----------------------------------------+               |
|   |     Elasticsearch / Loki / CloudWatch   |               |
|   |          (stores and indexes)           |               |
|   +-----------------------------------------+               |
|       |                                                     |
|       v                                                     |
|   +-----------------------------------------+               |
|   |        Kibana / Grafana Dashboard        |               |
|   |          (search and visualize)          |               |
|   +-----------------------------------------+               |
+-------------------------------------------------------------+
```

Logging Drivers in Docker Compose

```yaml
services:
  web:
    image: myapp:1.0.0
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
```

Comparison of Logging Drivers

Driver	Where Logs Go	Use Case
json-file	Local JSON files	Development, small setups
syslog	Syslog server	Linux infrastructure
fluentd	Fluentd collector	Kubernetes, complex setups
awslogs	AWS CloudWatch	AWS deployments
gcplogs	Google Cloud Logging	GCP deployments
gelf	Graylog (GELF)	Graylog users
none	Nowhere (discarded)	When logs are not needed

Monitoring

Monitoring tells you what your containers are doing right now and alerts you when something goes wrong.

docker stats (Built-In)

The simplest monitoring tool is built into Docker:

```bash
# Real-time resource usage for all containers
docker stats

# Monitor specific containers
docker stats web db redis

# One-time snapshot (no streaming)
docker stats --no-stream
```

Output:

```
CONTAINER   CPU %   MEM USAGE / LIMIT   MEM %   NET I/O        BLOCK I/O
web         3.2%    128MiB / 512MiB     25.0%   5.2MB / 1.1MB  0B / 0B
db          12.1%   450MiB / 1GiB       44.0%   2.1MB / 890kB  100MB / 50MB
redis       0.5%    12MiB / 256MiB      4.7%    500kB / 200kB  0B / 0B
```

Column meanings:

- `CPU %` — Percentage of CPU being used
- `MEM USAGE / LIMIT` — Current memory usage and the limit (if set)
- `MEM %` — Memory usage as a percentage of the limit
- `NET I/O` — Network data received / sent
- `BLOCK I/O` — Disk data read / written

cAdvisor (Container Advisor)

cAdvisor is Google's open-source tool for monitoring container resource usage. It provides a web UI and exports metrics for Prometheus.

```bash
docker run -d \
  --name cadvisor \
  --volume /:/rootfs:ro \
  --volume /var/run:/var/run:ro \
  --volume /sys:/sys:ro \
  --volume /var/lib/docker/:/var/lib/docker:ro \
  --publish 8080:8080 \
  gcr.io/cadvisor/cadvisor:latest
```

After starting, open `http://localhost:8080` in your browser. You will see:

- CPU usage graphs per container
- Memory usage graphs per container
- Network traffic per container
- Filesystem usage per container

Prometheus + Grafana

For production monitoring, Prometheus collects metrics and Grafana visualizes them in dashboards.

```yaml
# docker-compose.monitoring.yml
services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    restart: unless-stopped

  grafana:
```

```
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    restart: unless-stopped

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
    ports:
      - "8080:8080"
    restart: unless-stopped
```

Prometheus configuration:

```yaml
# prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']

  - job_name: 'docker'
    static_configs:
      - targets: ['host.docker.internal:9323']
```

```
+-------------------------------------------------------------+
|              Monitoring Stack                                |
|                                                              |
|   Your Containers                                            |
|   +------+ +------+ +------+                                |
|   | Web  | | DB   | | Redis|                                |
|   +------+ +------+ +------+                                |
|       |        |        |                                    |
|       v        v        v                                    |
|   +----------------------------+                             |
|   |       cAdvisor             |  Collects metrics           |
|   +----------------------------+                             |
|               |                                              |
|               v                                              |
|   +----------------------------+                             |
|   |       Prometheus           |  Stores metrics             |
|   +----------------------------+  Time-series database      |
|               |                                              |
```

```
|               v                                          |
|   +---------------------------+                          |
|   |       Grafana             |  Visualizes metrics      |
|   +---------------------------+  Dashboards + alerts     |
+-------------------------------------------------------------+
```

Health Checks

Health checks let Docker (and monitoring tools) know if your application is actually working, not just running.

```dockerfile
# In your Dockerfile
HEALTHCHECK --interval=30s --timeout=5s --retries=3 --start-period=10s \
  CMD curl -f http://localhost:3000/health || exit 1
```

What each option means:

- `--interval=30s` — Check every 30 seconds
- `--timeout=5s` — If the check takes more than 5 seconds, it counts as a failure
- `--retries=3` — After 3 consecutive failures, mark the container as "unhealthy"
- `--start-period=10s` — Wait 10 seconds after starting before the first check (gives the app time to boot)
- `CMD curl -f http://localhost:3000/health || exit 1` — The actual health check command. Exit code 0 = healthy, exit code 1 = unhealthy.

```bash
# Check container health status
docker inspect --format='{{.State.Health.Status}}' myapp
# Output: healthy
```

```bash
# See health check history
docker inspect --format='{{json .State.Health}}' myapp | python3 -m json.tool
```

Graceful Shutdown

When Docker stops a container, it sends a SIGTERM signal first, waits for the process to shut down cleanly, and then sends SIGKILL (force kill) if it takes too long.

```
+-------------------------------------------------------------+
|              Docker Stop Sequence                           |
|                                                             |
|   docker stop myapp                                         |
|        |                                                    |
|        v                                                    |
|   1. Send SIGTERM to the main process                       |
|      "Please shut down gracefully"                          |
|        |                                                    |
|        v                                                    |
|   2. Wait (default: 10 seconds)                             |
|      Container can finish requests, close connections,      |
|      save state, flush buffers                              |
|        |                                                    |
|        v (if still running after timeout)                   |
|   3. Send SIGKILL                                           |
|      "You are dead now. No choice."                         |
|      Immediate termination. No cleanup.                     |
+-------------------------------------------------------------+
```

Why Graceful Shutdown Matters

Without graceful shutdown:

- HTTP requests in progress are dropped — Users see error pages
- Database transactions are interrupted — Data corruption possible
- Files being written are truncated — Partial data on disk
- Connections to other services are abandoned — Other services may retry and create duplicates

Handling SIGTERM in Node.js

```javascript
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200);
  res.end('Hello World\n');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});

// Handle graceful shutdown
process.on('SIGTERM', () => {
```

```
  console.log('SIGTERM received. Shutting down gracefully...');

  // Stop accepting new connections
  server.close(() => {
    console.log('All connections closed. Exiting.');
    process.exit(0);
  });

  // Force exit after 5 seconds if connections are not closed
  setTimeout(() => {
    console.error('Forced shutdown after timeout');
    process.exit(1);
  }, 5000);
});
```

Handling SIGTERM in Python

```python
# server.py
import signal
import sys
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello World\n')

server = HTTPServer(('', 8000), Handler)

def graceful_shutdown(signum, frame):
    print('SIGTERM received. Shutting down...')
    server.shutdown()
    sys.exit(0)

signal.signal(signal.SIGTERM, graceful_shutdown)

print('Server running on port 8000')
server.serve_forever()
```

The PID 1 Problem

In Docker, the main process runs as PID 1 (process ID 1). PID 1 has special behavior in Linux — it does NOT receive default signal handlers. This means:

```dockerfile
# PROBLEM: Shell form wraps your command in /bin/sh
CMD node server.js
# Actually runs: /bin/sh -c "node server.js"
# PID 1 is /bin/sh, not node. SIGTERM goes to /bin/sh,
# which does NOT forward it to node.
```

```dockerfile
# SOLUTION: Exec form runs your command directly
CMD ["node", "server.js"]
# node IS PID 1. SIGTERM goes directly to node.
```

Always use the exec form (JSON array) for CMD and ENTRYPOINT. The shell form wraps your command in `/bin/sh`, which does not forward signals.

Using tini as an Init Process

If your application spawns child processes, you may need a proper init system. tini is a tiny init process designed for containers:

```dockerfile
FROM node:20-alpine

# Install tini
RUN apk add --no-cache tini

WORKDIR /app
COPY . .
RUN npm ci --only=production

# Use tini as the entrypoint
ENTRYPOINT ["tini", "--"]
CMD ["node", "server.js"]
```

tini properly forwards signals and reaps zombie processes (cleaned up child processes that have exited but not been collected).

Adjusting the Stop Timeout

```bash
# Give the container 30 seconds to shut down (default is 10)
docker stop --time 30 myapp
```

In Docker Compose:

```yaml
services:
  web:
    image: myapp:1.0.0
    stop_grace_period: 30s
```

Production Checklist

Use this checklist before deploying any container to production.

Resource Management

```
[ ] Set memory limits (--memory)
[ ] Set CPU limits (--cpus)
[ ] Configure log rotation (max-size, max-file)
[ ] Monitor resource usage (docker stats, cAdvisor, Prometheus)
```

Reliability

```
[ ] Set appropriate restart policy (usually unless-stopped)
[ ] Add health checks (HEALTHCHECK in Dockerfile)
[ ] Handle SIGTERM for graceful shutdown
[ ] Use exec form for CMD (JSON array syntax)
[ ] Test container behavior under load
[ ] Test container recovery after crash
```

Logging and Monitoring

```
[ ] Configure logging driver (or at minimum, log rotation)
[ ] Set up centralized log aggregation for multi-server deployments
[ ] Add monitoring dashboards (Grafana)
[ ] Configure alerts for high CPU, memory, and error rates
[ ] Ensure application logs include timestamps and request IDs
```

Security (from Chapter 22)

```
[ ] Run as non-root user
[ ] Use minimal base images
[ ] Scan for vulnerabilities
[ ] No secrets in image layers
[ ] Read-only filesystem where possible
[ ] Drop unnecessary capabilities
```

Deployment

```
[ ] Pin image versions (no :latest in production)
[ ] Use Docker Compose or orchestration for multi-container apps
[ ] Test rollback procedure
[ ] Document deployment process
[ ] Set up CI/CD pipeline (from Chapter 21)
```

Common Mistakes

Mistake 1: No Memory Limits

```bash
# A memory leak causes the container to use ALL available memory
docker run myapp  # No --memory flag
```

The container grows to consume all available RAM, then the Linux OOM killer starts killing processes — possibly including other containers or system processes.

Fix:

```bash
docker run --memory 512m myapp
```

Mistake 2: Using restart: always for Everything

```bash
# A container that crashes on startup will restart in an infinite loop
docker run --restart always broken-app
```

If the application always fails (like a misconfigured database connection), Docker restarts it endlessly, consuming CPU and filling logs.

Fix: Use `on-failure:5` to limit retries, or `unless-stopped` so you can manually stop the loop with `docker stop`.

Mistake 3: No Log Rotation

Without `max-size` and `max-file`, Docker log files grow without limit:

```bash
# Check log file size
du -sh /var/lib/docker/containers/*/
# Could show: 50G for a single container!
```

Fix: Always set log rotation:

```bash
docker run --log-opt max-size=10m --log-opt max-file=3 myapp
```

Or set it globally in `/etc/docker/daemon.json`:

```json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

Mistake 4: Shell Form CMD

```dockerfile
# WRONG — SIGTERM is not forwarded to the application
CMD node server.js

# RIGHT — SIGTERM goes directly to the application
CMD ["node", "server.js"]
```

Mistake 5: No Health Checks

Without health checks, Docker considers a container "healthy" as long as the process is running — even if the application is frozen, deadlocked, or returning errors.

Fix: Add a HEALTHCHECK to your Dockerfile or docker-compose.yml.

Best Practices

1. Always set memory limits — Prevent one container from starving others
2. Use `unless-stopped` restart policy — Self-healing with manual override
3. Configure log rotation — Prevent disk space exhaustion
4. Add health checks — Let Docker know if your application is actually working
5. Handle SIGTERM — Shut down gracefully to avoid dropped requests and data loss
6. Use exec form for CMD — Ensure signals reach your application
7. Monitor with Prometheus + Grafana — Real-time visibility and alerting
8. Set the stop grace period — Give applications enough time to shut down cleanly
9. Use tini for multi-process containers — Proper signal forwarding and zombie reaping
10. Test failure scenarios — Kill containers intentionally and verify recovery

---

Quick Summary

Production Docker deployments require resource limits to prevent containers from consuming all available memory and CPU. Restart policies like `unless-stopped` enable self-healing while respecting manual overrides. Logging drivers and log rotation prevent disk exhaustion and enable centralized log management. Monitoring with docker stats, cAdvisor, and Prometheus provides visibility into container performance. Graceful shutdown through proper SIGTERM handling prevents dropped requests and data corruption. Health checks tell Docker whether an application is truly working, not just running. Together, these practices transform containers from development tools into production-ready infrastructure.

Key Points

- `--memory` and `--cpus` prevent resource exhaustion on shared servers
- Restart policies control what happens when a container stops: `no`, `always`, `unless-stopped`, `on-failure`
- Log rotation (`max-size`, `max-file`) is essential to prevent disk full errors
- Logging drivers (syslog, fluentd, awslogs) send logs to centralized systems
- docker stats provides real-time resource usage; cAdvisor and Prometheus provide historical data

- SIGTERM is Docker's polite shutdown signal; SIGKILL is the forced shutdown after timeout
- Exec form (`CMD ["node", "server.js"]`) ensures signals reach your application
- Health checks detect when an application is frozen or broken, not just when the process exits
- tini is a minimal init process that handles signals and zombie processes correctly
- A production checklist covers resources, reliability, logging, security, and deployment

---

Practice Questions

1. What happens when a container exceeds its memory limit set by `--memory`? How is this different from exceeding its CPU limit set by `--cpus`?
2. Explain the difference between the restart policies `always` and `unless-stopped`. When would you choose one over the other?
3. Why is log rotation important? What Docker options control log rotation, and what happens if you do not configure them?
4. What is the difference between `CMD node server.js` and `CMD ["node", "server.js"]` in a Dockerfile? Why does it matter for graceful shutdown?
5. Describe the Docker stop sequence. What signal is sent first? How long does Docker wait? What happens after the timeout?

Exercises

Exercise 1: Resource Limits

1. Run a container with `--memory 128m` using any image
2. Inside the container, try to allocate more memory than the limit (you can use a stress test tool)
3. Observe what happens when the limit is exceeded
4. Run `docker inspect` on the container to see the exit code (should be 137)
5. Repeat with `--cpus 0.5` and observe how CPU usage is throttled using `docker stats`

Exercise 2: Restart Policies

1. Create a container that deliberately crashes after 5 seconds: `docker run --restart on-failure:3 alpine sh -c "echo 'Starting...'; sleep 5; exit 1"`
2. Watch the container restart (use `docker ps` and `docker logs`)
3. After 3 restarts, verify Docker stops trying
4. Change to `--restart unless-stopped` and observe the difference
5. Stop it manually with `docker stop` and verify it does not restart

Exercise 3: Monitoring Stack

1. Start cAdvisor using the command shown in this chapter
2. Open `http://localhost:8080` and explore the container metrics
3. Run a few containers with resource limits
4. Observe CPU and memory usage in the cAdvisor dashboard
5. Bonus: Set up the full Prometheus + Grafana stack using the docker-compose file in this chapter

---

What Is Next?

You now know how to run containers reliably in production. But what happens when one server is not enough? When you need to run 10, 50, or 100 copies of your application across multiple machines? In the next chapter, we explore Docker Swarm — Docker's built-in orchestration tool for managing clusters of containers across multiple servers.

Chapter 24: Docker Swarm

What You Will Learn

By the end of this chapter, you will be able to:

- Understand what container orchestration is and why it is needed
- Initialize a Docker Swarm cluster with `docker swarm init`
- Explain the roles of manager and worker nodes
- Understand the difference between services and containers
- Create and manage services with `docker service create`
- Scale services up and down with `--replicas`
- Perform rolling updates with zero downtime
- Deploy multi-service applications with `docker stack deploy`
- Use the Swarm visualizer to see your cluster
- Drain nodes for maintenance
- Compare Docker Swarm with Kubernetes

Why This Chapter Matters

So far, you have run containers on a single machine. This works for development and small applications, but it breaks down when:

- Your application gets more traffic than one server can handle — You need to run copies on multiple machines
- A server crashes — Your application goes down and nobody restarts it
- You need to update — Stopping the old version and starting the new one causes downtime
- You need load balancing — Distributing traffic across multiple copies of your application

These are the problems that container orchestration solves. Think of orchestration like conducting an orchestra:

```
+-------------------------------------------------------------+
|              The Orchestra Analogy                          |
|                                                             |
|   Without a conductor (no orchestration):                   |
|   - Each musician plays at their own tempo                  |
|   - Nobody knows when to start or stop                     |
|   - If a musician gets sick, their part is silent           |
|   - Nobody coordinates the performance                      |
|                                                             |
|   With a conductor (orchestration):                         |
|   - Everyone plays in sync                                  |
|   - The conductor decides when each section plays           |
|   - If a musician is absent, another fills in               |
|   - The overall performance is coordinated                   |
+-------------------------------------------------------------+
```

Docker Swarm is Docker's built-in orchestration tool. It turns a group of Docker hosts into a single, coordinated cluster.

What Is Docker Swarm?

Docker Swarm (or "Swarm mode") is a clustering and orchestration feature built into Docker. When you enable Swarm mode, your Docker host becomes part of a swarm — a group of machines that work together to run your containers.

```
+-------------------------------------------------------------+
|              Single Host vs Swarm                           |
|                                                             |
|   Single Host:                                              |
|   +------------------+                                      |
|   | Docker Host      |                                      |
|   | +----+ +----+    |                                      |
|   | | C1 | | C2 |    |  Only one machine.                  |
|   | +----+ +----+    |  If it dies, everything dies.       |
|   +------------------+                                      |
|                                                             |
|   Swarm (3 hosts):                                          |
|   +----------+ +----------+ +----------+                   |
|   | Manager  | | Worker 1 | | Worker 2 |                   |
|   | +----+   | | +----+   | | +----+   |                   |
|   | | C1 |   | | | C2 |   | | | C3 |   |                   |
|   | +----+   | | +----+   | | +----+   |                   |
```

```
|   +----------+ +----------+ +----------+                  |
|                                                            |
|   Three machines. If one dies, the others take over.       |
+------------------------------------------------------------+
```

Key Features

- Built into Docker — No extra software to install
- Declarative service model — You declare what you want ("run 5 copies"), Swarm makes it happen
- Scaling — Add or remove copies with one command
- Rolling updates — Update containers one at a time with zero downtime
- Self-healing — If a container crashes, Swarm starts a replacement
- Load balancing — Incoming traffic is distributed across all copies
- Secure by default — All communication between nodes is encrypted

---

Initializing a Swarm

docker swarm init

Turn your Docker host into a Swarm manager with one command:

```bash
docker swarm init
```

Output:

```
Swarm initialized: current node (abc123def456) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-abc123...xyz789 192.168.1.100:2377

To add a manager to this swarm, run 'docker swarm join-token manager'
and follow the instructions.
```

What just happened:

- Your Docker host became a Swarm manager node

- A join token was generated for adding other machines to the swarm
- The Swarm API is now listening on port 2377
- An overlay network called `ingress` was created for load balancing

Verify the Swarm

```bash
docker node ls
```

Output:

```
ID                          HOSTNAME     STATUS    AVAILABILITY   MANAGER STATUS
abc123def456 *              my-server    Ready     Active         Leader
```

You have a one-node swarm. The `*` indicates this is the current node, and `Leader` means it is the primary manager.

Nodes: Managers and Workers

A Swarm cluster consists of two types of nodes:

Manager Nodes

Manager nodes make decisions for the cluster:

- Schedule which containers run on which nodes
- Maintain the cluster state
- Handle service creation, updates, and scaling
- Respond to API requests

```
+-------------------------------------------------------------+
|              Manager Node Responsibilities                   |
|                                                              |
|   +-----------------------------------------------------+   |
|   | Manager Node                                        |   |
|   |                                                     |   |
|   | - Accepts commands (docker service create)          |   |
|   | - Decides where to place containers                 |   |
|   | - Monitors container health                         |   |
|   | - Replaces failed containers                        |   |
```

```
|   | - Stores cluster configuration                    |   |
|   | - Can also run containers (optional)              |   |
|   +---------------------------------------------------+   |
+-------------------------------------------------------------+
```

Worker Nodes

Worker nodes execute the work:

- Run containers assigned by the manager
- Report container status back to the manager
- Cannot make scheduling decisions

Adding Worker Nodes

On the manager, get the join token:

```bash
docker swarm join-token worker
```

Output:

```
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-abc123...xyz789 192.168.1.100:2377
```

On each worker machine, run that command:

```bash
docker swarm join --token SWMTKN-1-abc123...xyz789 192.168.1.100:2377
```

Output:

```
This node joined a swarm as a worker.
```

Verify the Cluster

Back on the manager:

```bash
docker node ls
```

Output:

```
ID                          HOSTNAME       STATUS    AVAILABILITY   MANAGER STATUS
abc123def456 *              manager-1      Ready     Active         Leader
ghi789jkl012                worker-1       Ready     Active
mno345pqr678                worker-2       Ready     Active
```

You now have a three-node cluster: one manager and two workers.

How Many Managers?

For production, use an odd number of managers for fault tolerance:

Managers	Can Tolerate Failures	Recommended For
1	0	Development, testing
3	1	Small production
5	2	Large production
7	3	Very large production

The formula is: with N managers, the swarm tolerates (N-1)/2 failures. Three managers can lose one. Five managers can lose two.

Services vs Containers

In Swarm mode, you do not run containers directly. Instead, you create services. A service is a definition of what you want to run. Swarm creates the actual containers (called tasks) based on that definition.

```
+-------------------------------------------------------------+
|              Service vs Container                           |
|                                                             |
|   docker run:                                               |
|     "Run ONE container on THIS machine"                     |
|     You manage it. You restart it. You scale it.            |
|                                                             |
|   docker service create:                                    |
```

```
|     "I want 3 copies of this app running SOMEWHERE"        |
|     Swarm manages them. Swarm restarts them.               |
|     Swarm decides which machines run them.                 |
|                                                            |
|   +-----------------------------------------------------+   |
|   | Service: webapp (replicas: 3)                       |   |
|   |                                                     |   |
|   |   Task 1 (container) --> Worker 1                   |   |
|   |   Task 2 (container) --> Worker 2                   |   |
|   |   Task 3 (container) --> Manager 1                  |   |
|   +-----------------------------------------------------+   |
+-------------------------------------------------------------+
```

Think of a service as a job posting and tasks as the employees hired. The job posting says "I need 3 web developers." The employees are the actual people doing the work. If one quits, the job posting is still active — HR (Swarm) hires a replacement.

Creating Services

docker service create

```bash
docker service create \
  --name webapp \
  --replicas 3 \
  --publish 8080:80 \
  nginx:1.25
```

Line-by-line explanation:

- `docker service create` — Create a new service (instead of `docker run`)
- `--name webapp` — Give the service a name
- `--replicas 3` — Run 3 copies of this container
- `--publish 8080:80` — Map port 8080 on ALL nodes to port 80 in the containers
- `nginx:1.25` — The image to use

Output:

```
x9y8z7w6v5u4t3s2r1q0
```

This is the service ID.

Check the Service

```bash
docker service ls
```

Output:

```
ID             NAME      MODE         REPLICAS   IMAGE         PORTS
x9y8z7w6v5u4   webapp    replicated   3/3        nginx:1.25    *:8080->80/tcp
```

`REPLICAS: 3/3` means 3 out of 3 desired replicas are running.

See Where Tasks Are Running

```bash
docker service ps webapp
```

Output:

```
ID             NAME       IMAGE        NODE        DESIRED STATE   CURRENT STATE
a1b2c3d4e5f6   webapp.1   nginx:1.25   manager-1   Running         Running 30 seconds ago
g7h8i9j0k1l2   webapp.2   nginx:1.25   worker-1    Running         Running 28 seconds ago
m3n4o5p6q7r8   webapp.3   nginx:1.25   worker-2    Running         Running 29 seconds ago
```

Swarm distributed the 3 replicas across the 3 nodes automatically.

Routing Mesh (Built-In Load Balancing)

When you publish a port, Swarm creates a routing mesh. Any node in the swarm can receive traffic on that port, and Swarm routes it to a running container — even if that node is not running the container.

```
+-------------------------------------------------------------+
|              Swarm Routing Mesh                             |
|                                                             |
|   User request --> ANY node:8080                            |
|                                                             |
|   +----------+     +----------+     +----------+           |
|   | Manager  |     | Worker 1 |     | Worker 2 |           |
|   | Port 8080| --> | Port 8080| --> | Port 8080|           |
|   |          |     | [webapp] |     | [webapp] |           |
|   +----------+     +----------+     +----------+           |
|                                                             |
```

```
|   Request to Manager:8080 gets routed to Worker 1 or 2.   |
|   Request to Worker 1:8080 may go to Worker 2's container. |
|   The routing mesh handles this transparently.             |
+-------------------------------------------------------------+
```

This means you can point your load balancer at ANY node in the swarm, and traffic reaches the right container.

Scaling Services

Scaling is as simple as changing the replica count.

Scale Up

```bash
docker service scale webapp=5
```

Output:

```
webapp scaled to 5
overall progress: 5 out of 5 tasks
1/5: running
2/5: running
3/5: running
4/5: running
5/5: running
verify: Service converged
```

Swarm created 2 new containers and distributed them across available nodes.

Scale Down

```bash
docker service scale webapp=2
```

Swarm removes 3 containers, keeping only 2.

Verify

```bash
docker service ls
```

Output:

```
ID             NAME      MODE         REPLICAS   IMAGE         PORTS
x9y8z7w6v5u4   webapp    replicated   2/2        nginx:1.25    *:8080->80/tcp
```

Self-Healing

If a container crashes or a node goes down, Swarm automatically starts a replacement:

```bash
# Manually kill a container to simulate a crash
docker kill $(docker ps -q --filter "name=webapp")
```

Within seconds, Swarm detects the missing container and starts a new one. Run `docker service ps webapp` to see the replacement.

Rolling Updates

A rolling update replaces containers one at a time (or in batches), ensuring the service stays available throughout the update.

```
+-------------------------------------------------------------+
|              Rolling Update Process                          |
|                                                              |
|   Before update (all running v1):                            |
|   [v1] [v1] [v1] [v1]                                       |
|                                                              |
|   Step 1: Stop one v1, start one v2:                         |
|   [v2] [v1] [v1] [v1]                                       |
|                                                              |
|   Step 2: (wait delay) Stop another v1, start v2:            |
|   [v2] [v2] [v1] [v1]                                       |
|                                                              |
|   Step 3: Continue...                                        |
|   [v2] [v2] [v2] [v1]                                       |
|                                                              |
|   Step 4: Done!                                              |
|   [v2] [v2] [v2] [v2]                                       |
|                                                              |
```

```
|   At NO point was the service completely down.              |
+-------------------------------------------------------------+
```

Performing a Rolling Update

```bash
docker service update \
  --image nginx:1.26 \
  --update-delay 10s \
  --update-parallelism 1 \
  --update-failure-action rollback \
  webapp
```

Line-by-line explanation:

- `--image nginx:1.26` — Update to this new image version
- `--update-delay 10s` — Wait 10 seconds between updating each container
- `--update-parallelism 1` — Update 1 container at a time (default)
- `--update-failure-action rollback` — If the new container fails to start, automatically roll back to the previous version
- `webapp` — The service to update

Watch the Update Progress

```bash
docker service ps webapp
```

Output during update:

```
ID             NAME          IMAGE        NODE        DESIRED STATE   CURRENT STATE
s1t2u3v4w5x6   webapp.1     nginx:1.26   worker-1    Running         Running 5 seconds ago
a1b2c3d4e5f6    \_ webapp.1  nginx:1.25   worker-1    Shutdown        Shutdown 10 secon...
g7h8i9j0k1l2   webapp.2     nginx:1.25   worker-2    Running         Running 2 minutes ago
```

The `\_` prefix shows the previous version of that task. Swarm keeps a history so you can see what changed.

Rolling Back

If something goes wrong, roll back to the previous version:

```bash
docker service rollback webapp
```

Swarm replaces all containers with the previous image version.

Configuring Update Behavior at Creation

```bash
docker service create \
  --name webapp \
  --replicas 4 \
  --publish 8080:80 \
  --update-delay 10s \
  --update-parallelism 2 \
  --update-failure-action rollback \
  --rollback-delay 5s \
  --rollback-parallelism 1 \
  nginx:1.25
```

This creates the service with update and rollback policies built in. Every future update will use these settings.

Stack Deploy with Compose Files

For multi-service applications, you can deploy an entire stack using a Docker Compose file.

Compose File for Swarm

```yaml
# docker-compose.yml (Swarm-compatible)
version: "3.8"

services:
  web:
    image: myusername/webapp:1.0.0
    ports:
      - "8080:80"
    deploy:
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
        failure_action: rollback
      restart_policy:
```

```
        condition: on-failure
        delay: 5s
        max_attempts: 3
      resources:
        limits:
          cpus: '0.5'
          memory: 256M
    networks:
      - frontend

  api:
    image: myusername/api-server:1.0.0
    deploy:
      replicas: 2
      update_config:
        parallelism: 1
        delay: 10s
    networks:
      - frontend
      - backend

  db:
    image: postgres:16-alpine
    volumes:
      - db-data:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == manager
    secrets:
      - db_password
    networks:
      - backend

networks:
  frontend:
  backend:

volumes:
  db-data:

secrets:
  db_password:
    external: true
```

Deploy the Stack

```bash
# First, create the secret
echo "MyDatabasePassword" | docker secret create db_password -

# Deploy the stack
docker stack deploy -c docker-compose.yml myapp
```

Output:

```
Creating network myapp_frontend
Creating network myapp_backend
Creating service myapp_web
Creating service myapp_api
Creating service myapp_db
```

Manage the Stack

```bash
# List all stacks
docker stack ls
```

Output:

```
NAME      SERVICES   ORCHESTRATOR
myapp     3          Swarm
```

```bash
# List services in the stack
docker stack services myapp
```

Output:

```
ID             NAME        MODE         REPLICAS   IMAGE                          PORTS
a1b2c3d4e5f6   myapp_web   replicated   3/3        myusername/webapp:1.0.0        *:808...
g7h8i9j0k1l2   myapp_api   replicated   2/2        myusername/api-server:1.0.0
m3n4o5p6q7r8   myapp_db    replicated   1/1        postgres:16-alpine
```

```bash
# Remove the stack
docker stack rm myapp
```

Key Differences from Regular Compose

The `deploy` section is Swarm-only. It is ignored by `docker compose up`. Some key Swarm-specific settings:

- `deploy.replicas` — How many copies to run
- `deploy.update_config` — How to perform rolling updates
- `deploy.placement.constraints` — Control which nodes run which services
- `deploy.resources` — CPU and memory limits
- `secrets` — Swarm secrets management

Swarm Visualizer

The Docker Swarm Visualizer is a web application that shows your cluster in a visual dashboard.

```bash
docker service create \
  --name visualizer \
  --publish 9090:8080 \
  --constraint 'node.role == manager' \
  --mount type=bind,src=/var/run/docker.sock,dst=/var/run/docker.sock \
  dockersamples/visualizer:latest
```

Open `http://manager-ip:9090` in your browser. You will see:

```
+-------------------------------------------------------------+
|              Swarm Visualizer (Web UI)                      |
|                                                             |
|   +----------------+ +----------------+ +----------------+ |
|   | manager-1      | | worker-1       | | worker-2       | |
|   |                | |                | |                | |
|   | [webapp.1]     | | [webapp.2]     | | [webapp.3]     | |
|   | [visualizer]   | | [api.1]        | | [api.2]        | |
|   | [db.1]         | |                | |                | |
|   +----------------+ +----------------+ +----------------+ |
|                                                             |
|   Each box is a node. Each colored rectangle is a task.     |
+-------------------------------------------------------------+
```

Draining Nodes

Draining a node tells Swarm to stop running tasks on that node and reschedule them elsewhere. This is useful for maintenance — updating the operating system, replacing hardware, or decommissioning a server.

Drain a Node

```bash
docker node update --availability drain worker-2
```

What happens:

1. Swarm stops scheduling new tasks on `worker-2`
2. All running tasks on `worker-2` are stopped
3. Replacement tasks are started on other available nodes
4. `worker-2` is now idle — safe to perform maintenance

Verify

```bash
docker node ls
```

Output:

```
ID              HOSTNAME     STATUS    AVAILABILITY   MANAGER STATUS
abc123 *        manager-1    Ready     Active         Leader
ghi789          worker-1     Ready     Active
mno345          worker-2     Ready     Drain
```

`worker-2` shows `Drain` — it is not running any tasks.

Reactivate the Node

After maintenance:

```bash
docker node update --availability active worker-2
```

Swarm can now schedule tasks on `worker-2` again. Existing tasks are NOT moved back automatically — new tasks and rebalancing will use it.

Swarm vs Kubernetes

Docker Swarm and Kubernetes both solve the same problem — container orchestration — but with very different approaches.

```
+-----------------------------------------------------------+
|              Swarm vs Kubernetes                          |
|                                                           |
|   Docker Swarm:                                           |
|   - "Easy to learn, limited features"                    |
|   - Like a bicycle: simple, gets you where you need to go |
|                                                           |
|   Kubernetes:                                             |
|   - "Complex to learn, unlimited features"               |
|   - Like a car: powerful, but you need driving lessons    |
+-----------------------------------------------------------+
```

Comparison Table

Feature	Docker Swarm	Kubernetes
Setup difficulty	Very easy	Complex
Learning curve	Gentle	Steep
Built into Docker	Yes	No (separate install)
Scaling	Manual or simple	Auto-scaling built in
Rolling updates	Yes	Yes, more options
Service discovery	Built in	Built in, more flexible
Load balancing	Built in	Built in + Ingress
Storage management	Basic	Advanced (PV, PVC, SC)
Networking	Overlay networks	CNI plugins (many options)
Configuration	Docker Compose files	YAML manifests
Community & ecosystem	Smaller	Massive
Cloud provider support	Limited	AWS EKS, GKE, AKS
Best for	Small-medium teams	Large teams, complex apps

When to Choose Swarm

- You are already using Docker and want something simple
- Your team is small (1-10 people)
- You have a few services (1-20)
- You do not need auto-scaling
- You want to get started quickly without a learning curve

When to Choose Kubernetes

- You have a large team (10+ people)
- You run many services (20+)
- You need auto-scaling based on CPU, memory, or custom metrics
- You need advanced networking (service mesh, ingress controllers)
- You deploy to cloud providers (AWS, GCP, Azure all offer managed Kubernetes)
- You need a large ecosystem of tools and plugins

Learning Path

If you are new to orchestration, start with Docker Swarm. The concepts (services, replicas, rolling updates, nodes) are the same in Kubernetes — just with different commands and more options. Swarm is the training ground; Kubernetes is the next level.

Common Mistakes

Mistake 1: Running a Single Manager in Production

```bash
docker swarm init  # Only one manager
```

If this manager dies, the entire swarm is leaderless. No new tasks can be scheduled, no updates can be performed.

Fix: Use at least 3 manager nodes in production for fault tolerance.

Mistake 2: Not Using Update Policies

```bash
docker service update --image myapp:2.0 webapp
# No --update-delay, no --update-failure-action
```

Without update policies, all containers update simultaneously. If the new version has a bug, the entire service goes down.

Fix: Always configure update delay and rollback action:

```bash
docker service update \
  --image myapp:2.0 \
  --update-delay 10s \
  --update-failure-action rollback \
  webapp
```

Mistake 3: Storing Data in Containers Without Volumes

If a task (container) is rescheduled to a different node, any data written to the container's filesystem is lost.

Fix: Use volumes for all persistent data, especially databases.

Mistake 4: Exposing Manager Ports to the Internet

The Swarm management port (2377) and the node communication ports (7946, 4789) should only be accessible between swarm nodes.

Fix: Use firewall rules to restrict access to these ports to only swarm node IPs.

Mistake 5: Not Using Secrets for Sensitive Data

```yaml
# BAD: Password in plain text in the compose file
environment:
  DB_PASSWORD: MySecret123
```

Fix: Use Docker Swarm secrets:

```yaml
secrets:
  - db_password
```

Best Practices

1. Use 3 or 5 manager nodes in production for fault tolerance
2. Always configure rolling update policies — Set delay, parallelism, and rollback action
3. Use secrets for sensitive data — Never put passwords in compose files or environment variables
4. Drain nodes before maintenance — Safely move tasks before updating or rebooting
5. Use placement constraints — Pin databases to specific nodes with fast storage
6. Monitor your swarm — Use the visualizer during development, Prometheus in production
7. Set resource limits — Prevent one service from consuming all resources
8. Use overlay networks — Isolate service communication with dedicated networks
9. Keep manager nodes lightweight — Avoid running heavy workloads on managers
10. Test rollback procedures — Regularly verify that rollbacks work correctly

---

Quick Summary

Docker Swarm turns multiple Docker hosts into a coordinated cluster that manages containers as services. Manager nodes make scheduling decisions while worker nodes execute tasks. Services define the desired state (image, replicas, ports), and Swarm ensures that state is maintained — replacing crashed containers and distributing workloads. Rolling updates replace containers gradually for zero-downtime deployments. Stack deploy uses Docker Compose files to deploy multi-service applications. Draining nodes allows safe maintenance without service disruption. While Kubernetes offers more features and a larger ecosystem, Docker Swarm provides a simpler path to container orchestration that uses familiar Docker concepts and commands.

Key Points

- Docker Swarm turns multiple Docker hosts into a single orchestrated cluster
- Manager nodes schedule tasks and maintain state; worker nodes run containers
- Services define the desired state; tasks are the actual running containers

- `docker service create` creates a service; `docker service scale` changes the replica count
- Rolling updates replace containers gradually with `--update-delay` and `--update-parallelism`
- `docker stack deploy` deploys multi-service applications from Compose files
- Routing mesh lets any node receive traffic and route it to the right container
- Self-healing automatically replaces crashed containers
- Drain removes a node from the scheduling pool for safe maintenance
- Swarm is simpler than Kubernetes but has fewer features; start with Swarm, graduate to Kubernetes

---

Practice Questions

1. What is the difference between a manager node and a worker node in Docker Swarm? Can a manager node also run containers?
2. Explain the difference between `docker run` and `docker service create`. When would you use each?
3. You have a service with 4 replicas and you need to update the image. Describe how a rolling update works with `--update-delay 10s` and `--update-parallelism 1`.
4. What happens when you drain a node? What happens to the tasks that were running on it?
5. When would you choose Docker Swarm over Kubernetes? When would you choose Kubernetes over Docker Swarm?

Exercises

Exercise 1: Create Your First Swarm

1. Initialize a single-node swarm with `docker swarm init`
2. Create a service: `docker service create --name web --replicas 3 --publish 8080:80 nginx:latest`
3. Visit `http://localhost:8080` to verify it works
4. Run `docker service ps web` to see where tasks are running
5. Kill one container manually and watch Swarm replace it

6. Scale to 5 replicas and verify with `docker service ls`

Exercise 2: Rolling Update

1. Create a service running `nginx:1.24` with 4 replicas
2. Perform a rolling update to `nginx:1.25` with a 5-second delay
3. Watch the update progress with `docker service ps`
4. Roll back to `nginx:1.24` with `docker service rollback`
5. Verify all tasks are back on the previous version

Exercise 3: Stack Deploy

1. Create a `docker-compose.yml` with two services: a web app (nginx) and an API (any simple image)
2. Add `deploy` sections with replicas, update config, and resource limits
3. Deploy the stack with `docker stack deploy -c docker-compose.yml mystack`
4. List all services with `docker stack services mystack`
5. Update one service's image and watch the rolling update
6. Remove the stack with `docker stack rm mystack`

What Is Next?

You now know how to orchestrate containers across multiple machines. But orchestration only works well when your images are efficient. In the next chapter, we explore Optimizing Docker Images — reducing image sizes for faster pulls, smaller attack surfaces, and more efficient use of storage and bandwidth. A 1 GB image that could be 50 MB is not just wasteful — it is a bottleneck that slows down everything.

Chapter 25: Optimizing Docker Images

What You Will Learn

By the end of this chapter, you will be able to:

- Understand why image size matters for speed, cost, and security
- Analyze image layers and wasted space with `dive`
- Use distroless base images for minimal containers
- Choose Alpine variants for smaller images
- Apply multi-stage builds to eliminate build-time dependencies
- Write effective `.dockerignore` files
- Leverage BuildKit features for faster, smarter builds
- Implement layer caching strategies that speed up rebuilds
- Transform a bloated image into an optimized one with before/after examples

---

Why This Chapter Matters

A typical unoptimized Docker image for a Node.js application is around 1 GB. An optimized version of the same application can be 50-100 MB — ten to twenty times smaller.

Why does this matter?

```
+-------------------------------------------------------------+
|              Why Image Size Matters                         |
|                                                             |
|   1. FASTER PULLS                                           |
|      1 GB image over 100 Mbps = ~80 seconds                 |
|      50 MB image over 100 Mbps = ~4 seconds                 |
```

```
|      In a cluster with 50 nodes, that is 66 MINUTES saved. |
|                                                            |
|   2. LESS STORAGE                                          |
|      100 images x 1 GB = 100 GB on each server             |
|      100 images x 50 MB = 5 GB on each server              |
|                                                            |
|   3. SMALLER ATTACK SURFACE                                |
|      Fewer packages = fewer potential vulnerabilities       |
|      1 GB image: ~600 packages, ~200 known CVEs            |
|      50 MB image: ~20 packages, ~5 known CVEs              |
|                                                            |
|   4. FASTER CI/CD                                          |
|      Smaller images build and push faster                  |
|      Your pipeline finishes in 2 minutes instead of 10     |
|                                                            |
|   5. LOWER COSTS                                           |
|      Registry storage, bandwidth, and server disk space    |
|      all cost money. Smaller images = lower bills.         |
+------------------------------------------------------------+
```

Think of it like packing for a trip. You could bring your entire wardrobe in three suitcases, or you could pack only what you need in a carry-on. The carry-on is faster through the airport, costs less (no checked bag fees), and you are less likely to lose something.

Analyzing Images with Dive

Before optimizing, you need to understand what is inside your image. dive is a tool that lets you explore each layer of a Docker image and see exactly what files are added, modified, or removed.

Installing Dive

```bash
# macOS
brew install dive

# Linux
wget https://github.com/wagoodman/dive/releases/download/v0.12.0/dive_0.12.0_linux_amd6...
sudo dpkg -i dive_0.12.0_linux_amd64.deb
```

Using Dive

```bash
dive myapp:1.0.0
```

Dive opens an interactive terminal UI with two panels:

```
+-------------------------------------------------------------+
|              Dive Interface                                  |
|                                                              |
|   Left Panel: Image Layers                                   |
|   +-----------------------------------------------------+ |
|   | Layer 1: FROM node:20         (350 MB)              | |
|   | Layer 2: RUN npm install      (200 MB)              | |
|   | Layer 3: COPY . .             (450 MB) <-- WHY?     | |
|   | Layer 4: RUN npm run build    (50 MB)               | |
|   +-----------------------------------------------------+ |
|                                                              |
|   Right Panel: Files in Selected Layer                       |
|   +-----------------------------------------------------+ |
|   | /app/node_modules/           (200 MB)               | |
|   | /app/.git/                   (150 MB)  <-- WASTE     | |
|   | /app/test/                   (50 MB)   <-- WASTE     | |
|   | /app/docs/                   (30 MB)   <-- WASTE     | |
|   +-----------------------------------------------------+ |
|                                                              |
|   Image efficiency: 45%  (Wasted space: 230 MB)             |
+-------------------------------------------------------------+
```

What to Look For

1. Unexpectedly large layers — Why is COPY adding 450 MB?
2. Files that should not be there — `.git/`, `test/`, `docs/`, `node_modules/.cache`
3. Duplicate files — The same file appearing in multiple layers
4. Wasted space — Files added in one layer and deleted in another (they still take space)

Dive in CI/CD

You can use dive in your CI pipeline to enforce image efficiency:

```bash
# Fail if the image efficiency is below 90%
CI=true dive myapp:1.0.0 --ci --lowestEfficiency 0.90
```

Distroless Base Images

Distroless images from Google contain ONLY your application and its runtime dependencies. No shell, no package manager, no utilities like `curl`, `wget`, or `ls`.

Why Distroless?

```
+-------------------------------------------------------------+
|              What Is Inside Each Base Image                 |
|                                                             |
|   ubuntu:22.04 (77 MB):                                     |
|   +-----------------------------------------------------+ |
|   | Shell (bash), Package manager (apt), curl, wget,    | |
|   | ls, cat, grep, sed, awk, mount, sudo, systemd,     | |
|   | openssl, ca-certificates, libc, libstdc++, perl,    | |
|   | python3, coreutils, findutils, ...                  | |
|   +-----------------------------------------------------+ |
|                                                             |
|   distroless (20 MB):                                       |
|   +-----------------------------------------------------+ |
|   | ca-certificates, libc, libstdc++, tzdata            | |
|   | YOUR APPLICATION                                    | |
|   +-----------------------------------------------------+ |
|                                                             |
|   Everything else is GONE. Attackers have nothing to use.  |
+-------------------------------------------------------------+
```

Using Distroless

Distroless images are designed for multi-stage builds. You build in a full image and copy the result to a distroless image.

Node.js example:

```dockerfile
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

# Stage 2: Production (distroless)
FROM gcr.io/distroless/nodejs20-debian12
WORKDIR /app
COPY --from=builder /app .
CMD ["server.js"]
```

Go example (even smaller — no runtime needed):

```dockerfile
# Stage 1: Build
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 go build -o server .

# Stage 2: Production (static distroless)
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app/server /server
CMD ["/server"]
```

The Go example produces an image under 10 MB because the compiled binary includes everything it needs.

Available Distroless Images

Image	Contents	Size
`gcr.io/distroless/static-debian12`	CA certs, tzdata	~2 MB
`gcr.io/distroless/base-debian12`	+ glibc	~20 MB
`gcr.io/distroless/cc-debian12`	+ libstdc++	~25 MB
`gcr.io/distroless/nodejs20-debian12`	+ Node.js 20 runtime	~130 MB
`gcr.io/distroless/python3-debian12`	+ Python 3 runtime	~50 MB
`gcr.io/distroless/java21-debian12`	+ Java 21 runtime	~200 MB

The Trade-Off

Without a shell, you cannot:

- `docker exec -it container sh` — No shell to exec into
- Debug inside the container easily
- Install debugging tools on the fly

Solution: Use a debug variant during development:

```dockerfile
# For debugging (has a busybox shell)
FROM gcr.io/distroless/nodejs20-debian12:debug
```

Alpine Variants

Alpine Linux is a minimal Linux distribution designed for security and small size. Many official Docker images offer Alpine variants.

Size Comparison

```bash
docker images --format "table {{.Repository}}:{{.Tag}}\t{{.Size}}"
```

```
IMAGE                    SIZE
node:20                  1.1 GB
node:20-slim             200 MB
node:20-alpine           135 MB
python:3.12              1.0 GB
python:3.12-slim         150 MB
python:3.12-alpine       55 MB
```

Alpine variants are typically 5-20x smaller than their full counterparts.

Using Alpine

```dockerfile
# Instead of this (1.1 GB):
FROM node:20

# Use this (135 MB):
FROM node:20-alpine
```

Alpine Gotchas

Alpine uses musl libc instead of glibc. This can cause compatibility issues with some software:

1. Native npm packages may fail to compile. Fix by installing build tools:

```dockerfile
FROM node:20-alpine
RUN apk add --no-cache python3 make g++
```

2. Some Python packages with C extensions may not work. Use `python:3.12-slim` instead of Alpine in that case.
3. DNS resolution behaves slightly differently in Alpine. This rarely causes issues but can surprise you.

Rule of thumb: Try Alpine first. If something breaks, fall back to `-slim`. Only use the full image if `-slim` also has problems.

Multi-Stage Builds Recap

Multi-stage builds are the single most impactful optimization technique. We covered them in an earlier chapter, but here is a quick recap with an optimization focus.

The Problem

```dockerfile
# SINGLE STAGE — everything in one image
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci              # Installs ALL dependencies (including dev)
COPY . .
RUN npm run build       # Build step (generates dist/)
CMD ["node", "dist/server.js"]

# Final image contains:
# - All source code (not needed at runtime)
# - All dev dependencies (typescript, eslint, jest — not needed)
# - Build tools (not needed)
# - Test files (not needed)
# Result: ~500 MB
```

The Solution

```dockerfile
# MULTI-STAGE — separate build from runtime
# Stage 1: Build
FROM node:20-alpine AS builder
```

```
WORKDIR /app
COPY package*.json ./
RUN npm ci                    # All deps (including dev) for building
COPY . .
RUN npm run build             # Compile TypeScript, bundle, etc.

# Stage 2: Production
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production  # Only production dependencies
COPY --from=builder /app/dist ./dist
USER node
CMD ["node", "dist/server.js"]

# Final image contains:
# - Only production dependencies
# - Only compiled output (dist/)
# - No source code, no dev tools, no tests
# Result: ~150 MB
```

```
+-------------------------------------------------------------+
|              Multi-Stage Build Flow                          |
|                                                              |
|   Stage 1 (builder):        Stage 2 (production):            |
|   +-------------------+    +-------------------+             |
|   | node:20-alpine    |    | node:20-alpine    |             |
|   | Source code        |    | dist/ (from S1)   |             |
|   | ALL node_modules   |    | Prod node_modules |             |
|   | TypeScript         |    | (no source code)  |             |
|   | Test files         |    | (no dev deps)     |             |
|   | Build output (dist)|    |                   |             |
|   +-------------------+    +-------------------+             |
|          500 MB                   150 MB                     |
|                                                              |
|   Stage 1 is DISCARDED. Only Stage 2 becomes the image.    |
+-------------------------------------------------------------+
```

.dockerignore Optimization

The `.dockerignore` file tells Docker which files to exclude from the build context — the files sent to the Docker daemon when you run `docker build`.

Why .dockerignore Matters

Without `.dockerignore`, Docker sends EVERYTHING in your project directory to the daemon:

```bash
# Watch the build context size
docker build -t myapp .
# Output: Sending build context to Docker daemon  500MB
```

With a good `.dockerignore`:

```bash
docker build -t myapp .
# Output: Sending build context to Docker daemon  2.5MB
```

A Comprehensive .dockerignore

```
# .dockerignore

# Version control
.git
.gitignore

# Dependencies (will be installed inside the container)
node_modules
vendor
__pycache__
*.pyc

# Build output (will be built inside the container)
dist
build
out
*.o
*.a

# Development and test files
test
tests
__tests__
*.test.js
*.spec.js
coverage
.nyc_output

# Documentation
docs
*.md
LICENSE
CHANGELOG

# IDE and editor files
.vscode
.idea
```

```
*.swp
*.swo
*~

# Environment and secrets
.env
.env.*
*.pem
*.key
*.cert
credentials.json
secrets/

# Docker files (avoid recursive builds)
Dockerfile*
docker-compose*
.dockerignore

# OS files
.DS_Store
Thumbs.db

# CI/CD files
.github
.gitlab-ci.yml
.travis.yml
Jenkinsfile

# Logs
*.log
logs/
```

Before and After

Without .dockerignore	With .dockerignore
Build context: 500 MB	Build context: 2.5 MB
Includes .git (150 MB)	Excluded
Includes node_modules	Excluded (installed fresh)
Includes test files	Excluded
Includes docs	Excluded
Build time: 45 seconds	Build time: 15 seconds

BuildKit Features

BuildKit is Docker's modern build engine. It is faster, more efficient, and has features not available in the legacy builder.

Enabling BuildKit

```bash
# Enable for a single build
DOCKER_BUILDKIT=1 docker build -t myapp .

# Enable globally (add to ~/.bashrc or ~/.zshrc)
export DOCKER_BUILDKIT=1
```

Docker Desktop has BuildKit enabled by default. On Linux servers, you may need to enable it.

Parallel Stage Execution

BuildKit automatically builds independent stages in parallel:

```dockerfile
# These two stages build AT THE SAME TIME
FROM node:20-alpine AS frontend-builder
WORKDIR /frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ .
RUN npm run build

FROM python:3.12-alpine AS backend-builder
WORKDIR /backend
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY backend/ .

# Final stage uses output from both
FROM python:3.12-alpine
COPY --from=frontend-builder /frontend/dist /app/static
COPY --from=backend-builder /backend /app
CMD ["python", "/app/server.py"]
```

The legacy builder runs stages sequentially. BuildKit runs `frontend-builder` and `backend-builder` simultaneously, cutting build time nearly in half.

Inline Cache Metadata

BuildKit can embed cache metadata in pushed images, allowing future builds to pull cache from the registry:

```bash
docker build \
  --build-arg BUILDKIT_INLINE_CACHE=1 \
  -t myusername/myapp:latest \
  .

docker push myusername/myapp:latest

# Future builds can cache FROM the registry
docker build \
  --cache-from myusername/myapp:latest \
  -t myusername/myapp:v1.0.1 \
  .
```

Secret Mounts

BuildKit lets you use secrets during the build without storing them in any layer:

```dockerfile
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
# Mount .npmrc as a secret — available during this RUN but not in any layer
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm ci --only=production
COPY . .
CMD ["node", "server.js"]
```

```bash
docker build --secret id=npmrc,src=.npmrc -t myapp .
```

The `.npmrc` file (which may contain registry tokens) is available during `npm ci` but is NOT stored in the image.

Cache Mounts

BuildKit can cache directories between builds, dramatically speeding up package installations:

```dockerfile
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
```

```
# Cache npm's download cache between builds
RUN --mount=type=cache,target=/root/.npm \
    npm ci --only=production
COPY . .
CMD ["node", "server.js"]
```

Even if `package.json` changes, npm reuses cached downloads instead of downloading everything from scratch.

Python example:

```dockerfile
# syntax=docker/dockerfile:1
FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
COPY . .
CMD ["python", "server.py"]
```

Layer Caching Strategies

Docker caches each layer and reuses it if nothing has changed. Understanding how the cache works lets you structure your Dockerfile for maximum cache hits.

The Cache Rule

Docker reuses a cached layer if:

1. The instruction has not changed
2. ALL files referenced by the instruction have not changed
3. ALL previous layers are also cached (cache is sequential)

The moment one layer is invalidated, ALL subsequent layers are rebuilt.

```
+-------------------------------------------------------------+
|              Cache Invalidation                              |
|                                                              |
|   Layer 1: FROM node:20-alpine         CACHED              |
|   Layer 2: WORKDIR /app                CACHED              |
|   Layer 3: COPY package.json .         CACHED              |
|   Layer 4: RUN npm install             CACHED              |
```

```
|   Layer 5: COPY . .                    INVALIDATED        |
|   Layer 6: RUN npm run build           REBUILT (no cache)  |
|   Layer 7: CMD ["node", "server.js"]   REBUILT (no cache)  |
|                                                            |
|   Changing source code (Layer 5) does NOT invalidate       |
|   the npm install (Layer 4) because it comes BEFORE.       |
+------------------------------------------------------------+
```

Strategy 1: Copy Dependency Files First

```dockerfile
# GOOD — dependency installation is cached unless package.json changes
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./   # Changes rarely
RUN npm ci                                # Cached most of the time
COPY . .                                  # Changes often
RUN npm run build
```

```dockerfile
# BAD — npm install runs every time ANY file changes
FROM node:20-alpine
WORKDIR /app
COPY . .                                  # Invalidated on every change
RUN npm ci                                # Never cached
RUN npm run build
```

Strategy 2: Separate Rarely-Changing Layers

```dockerfile
FROM python:3.12-alpine

# Layer 1: System dependencies (change very rarely)
RUN apk add --no-cache libpq-dev

# Layer 2: Python dependencies (change occasionally)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Layer 3: Application code (changes often)
COPY . .

CMD ["python", "server.py"]
```

Order layers from least frequently changed to most frequently changed.

Strategy 3: Combine Related RUN Commands

```dockerfile
# BAD — three separate layers
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD — one layer, and the cleanup actually saves space
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*
```

When you delete files in a separate `RUN` instruction, the files still exist in the earlier layer. Combining them into one `RUN` means the deleted files never appear in the final layer.

Before and After: Complete Optimization Example

Let us take a real-world Node.js application and optimize it step by step.

Before: Unoptimized (1.1 GB)

```dockerfile
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Problems:

1. Uses `node:20` (full Debian — 1.1 GB base)
2. Copies everything (including `.git`, `node_modules`, tests)
3. No `.dockerignore`
4. Installs dev dependencies in the final image
5. Runs as root
6. No multi-stage build

After: Optimized (85 MB)

```dockerfile
# syntax=docker/dockerfile:1

# Stage 1: Install dependencies
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm \
    npm ci

# Stage 2: Build
FROM deps AS builder
COPY . .
RUN npm run build

# Stage 3: Production
FROM node:20-alpine AS production
WORKDIR /app

# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copy only production dependencies
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm \
    npm ci --only=production && \
    npm cache clean --force

# Copy built application from builder stage
COPY --from=builder /app/dist ./dist

# Set ownership and switch to non-root user
RUN chown -R appuser:appgroup /app
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD node -e "require('http').get('http://localhost:3000/health', (r) => r.statusCod...

EXPOSE 3000
CMD ["node", "dist/server.js"]
```

.dockerignore:

```
.git
node_modules
dist
test
docs
*.md
.env
.vscode
Dockerfile
docker-compose*
```

```
.dockerignore
coverage
.nyc_output
```

Size Comparison

```
+-------------------------------------------------------------+
|              Before vs After                                |
|                                                             |
|   Before:                            After:                 |
|   +---------------------------+    +---------------------+ |
|   | node:20 base     1,100 MB|    | node:20-alpine  135MB| |
|   | ALL node_modules   200 MB|    | Prod modules     40MB| |
|   | Source code          5 MB|    | dist/ only        5MB| |
|   | .git               150 MB|    | (no .git)            | |
|   | Tests               50 MB|    | (no tests)           | |
|   | Docs                30 MB|    | (no docs)            | |
|   +---------------------------+    +---------------------+ |
|   TOTAL:            ~1,535 MB      TOTAL:           ~85 MB  |
|                                                             |
|   Reduction: 94%                                            |
+-------------------------------------------------------------+
```

What Changed and Why

Optimization	Size Saved	How
Alpine base instead of full	~965 MB	`node:20-alpine` vs `node:20`
Multi-stage build	~255 MB	Dev deps and source code excluded
`.dockerignore`	~200 MB	`.git`, tests, docs not sent to daemon
Production-only dependencies	~160 MB	`npm ci --only=production`
Cache mounts	0 MB	Faster builds, no size change
Non-root user	0 MB	Security, no size change
Health check	0 MB	Reliability, no size change

Common Mistakes

Mistake 1: Not Using .dockerignore

Without `.dockerignore`, your `.git` directory (often 100+ MB) and `node_modules` (often 200+ MB) are sent to the Docker daemon and potentially copied into the image.

Fix: Always create a `.dockerignore` file. Start with the comprehensive example in this chapter.

Mistake 2: Using the Full Base Image

```dockerfile
FROM python:3.12    # 1 GB
```

Fix:

```dockerfile
FROM python:3.12-alpine    # 55 MB
```

Mistake 3: Installing Dev Dependencies in Production

```dockerfile
RUN npm install    # Installs EVERYTHING, including dev deps
```

Fix:

```dockerfile
RUN npm ci --only=production    # Only what your app needs to run
```

Mistake 4: Deleting Files in a Separate RUN Layer

```dockerfile
COPY big-file.tar.gz .        # Layer 1: 500 MB
RUN tar xf big-file.tar.gz    # Layer 2: 500 MB (extracted)
RUN rm big-file.tar.gz        # Layer 3: rm saves nothing! Layer 1 still has it.
```

Fix: Do it all in one layer:

```dockerfile
RUN curl -O https://example.com/big-file.tar.gz && \
    tar xf big-file.tar.gz && \
    rm big-file.tar.gz
```

Or use multi-stage builds.

Mistake 5: Copying Source Code Before Installing Dependencies

```dockerfile
COPY . .                # Invalidated on EVERY code change
RUN npm ci              # Reinstalls dependencies EVERY time
```

Fix:

```dockerfile
COPY package*.json ./   # Only invalidated when dependencies change
RUN npm ci              # Cached unless package.json changes
COPY . .                # Source code changes do not invalidate npm ci
```

Best Practices

1. Use Alpine or slim base images — Start small, only add what you need
2. Use multi-stage builds — Keep build tools and dev dependencies out of the final image
3. Write a comprehensive `.dockerignore` — Exclude `.git`, `node_modules`, tests, docs
4. Copy dependency files before source code — Maximize layer cache hits
5. Combine RUN instructions — Avoid files in intermediate layers that waste space
6. Use BuildKit cache mounts — Speed up package installation with persistent caches
7. Analyze with dive — Find wasted space and unexpected files
8. Use `--no-install-recommends` (apt) or `--no-cache` (apk) — Skip optional packages
9. Clean up in the same layer — Remove package manager caches in the same RUN instruction
10. Consider distroless for production — Minimum possible attack surface

---

Quick Summary

Optimizing Docker images reduces pull times, storage costs, CI/CD duration, and security vulnerabilities. Tools like dive reveal what is inside each layer and where space is wasted. Distroless and Alpine base images provide dramatically smaller foundations. Multi-stage

builds separate the build environment from the runtime environment, excluding dev dependencies and build tools. A well-crafted `.dockerignore` prevents large or sensitive files from entering the build context. BuildKit features like parallel stage execution, cache mounts, and secret mounts make builds faster and more secure. Layer ordering — placing rarely-changing instructions before frequently-changing ones — maximizes cache efficiency. Together, these techniques can reduce a 1.5 GB image to under 100 MB.

Key Points

- Image size directly impacts pull speed, storage costs, CI/CD time, and security
- dive analyzes image layers and shows wasted space
- Distroless images include only your app and runtime — no shell, no package manager
- Alpine variants are 5-20x smaller than full images
- Multi-stage builds keep build tools out of the final image
- `.dockerignore` prevents large files (`.git`, `node_modules`) from entering the build
- BuildKit enables parallel builds, cache mounts, and secret mounts
- Layer order matters — put rarely-changing instructions first for better caching
- Combine RUN commands to avoid files persisting in intermediate layers
- Copy dependency files before source code so dependency installation is cached

---

Practice Questions

1. Why does deleting a file in a separate RUN layer NOT reduce the image size? How can you avoid this problem?
2. What is the difference between `node:20`, `node:20-slim`, and `node:20-alpine`? When would you choose each?
3. What does `dive` show you about a Docker image? How would you use it to identify optimization opportunities?
4. Explain why `COPY package.json .` followed by `RUN npm ci` followed by `COPY . .` is more efficient than `COPY . .` followed by `RUN npm ci`. What caching behavior is different?
5. What are BuildKit cache mounts? How do they speed up Docker builds?

Exercises

Exercise 1: Analyze and Optimize

1. Build an unoptimized image using the "Before" Dockerfile from this chapter (use any Node.js app, or create a simple one)
2. Run `dive` on the image and note the total size and efficiency score
3. Apply each optimization one at a time:

a. Add `.dockerignore` → rebuild → check size b. Switch to Alpine base → rebuild → check size c. Add multi-stage build → rebuild → check size d. Add production-only dependencies → rebuild → check size

4. Record the size after each step to see the cumulative impact

Exercise 2: Compare Base Images

1. Build the same application using four different base images:
    - `FROM node:20`
    - `FROM node:20-slim`
    - `FROM node:20-alpine`
    - `FROM gcr.io/distroless/nodejs20-debian12` (with multi-stage)
2. Compare the sizes with `docker images`
3. Run `trivy image` on each to compare vulnerability counts
4. Try to `docker exec -it container sh` on each — which ones allow it?

Exercise 3: BuildKit Features

1. Build an image WITHOUT BuildKit: `DOCKER_BUILDKIT=0 docker build -t myapp-legacy .`
2. Build the same image WITH BuildKit: `DOCKER_BUILDKIT=1 docker build -t myapp-buildkit .`
3. Compare build times
4. Add cache mounts for your package manager and rebuild twice — note how the second build is faster
5. Use a multi-stage Dockerfile with two independent build stages and observe BuildKit building them in parallel

What Is Next?

Congratulations! You have now covered the full journey of Docker — from understanding what containers are, through building and optimizing images, to deploying them in production with orchestration. You know how to:

- Build efficient Docker images with multi-stage builds
- Manage configuration with environment variables and volumes
- Compose multi-container applications
- Push images to registries with proper tagging
- Automate builds with CI/CD
- Secure and harden containers
- Run containers reliably in production
- Orchestrate across multiple machines with Docker Swarm
- Optimize images for speed, security, and cost

The next step in your container journey is to explore Kubernetes for large-scale orchestration, or to deepen your Docker skills by applying everything in this book to a real project. Build something, deploy it, and iterate. The best learning happens by doing.

Chapter 26: What Is Kubernetes?

What You Will Learn

By the end of this chapter, you will be able to:

- Explain why managing many containers by hand becomes impossible at scale
- Describe what container orchestration means and why it exists
- Understand the Kubernetes architecture: control plane and worker nodes
- Name and explain every major Kubernetes component
- Compare Kubernetes with Docker Swarm
- Decide when Kubernetes is the right tool for your project

Why This Chapter Matters

In the previous chapters, you learned how to build Docker images, run containers, and use Docker Compose to manage multi-container applications on a single machine. That works great for development and small projects.

But what happens when your application grows? What happens when you need to run 50, 100, or 1,000 containers across multiple servers? What happens when a server crashes at 3 AM and your containers go down with it?

You need a system that manages containers for you automatically. That system is called a container orchestrator, and the most popular one is Kubernetes. This chapter introduces you to Kubernetes and explains why it has become the industry standard for running containers in production.

The Problem: Managing Containers at Scale

A Small Application

Imagine you have a simple web application:

- 1 web server container
- 1 database container
- 1 cache container

You can manage this with Docker Compose on a single machine. You run `docker compose up`, and everything works. Life is good.

A Growing Application

Now imagine your application becomes popular. You need:

- 20 web server containers (to handle more traffic)
- 3 database containers (primary plus replicas for reliability)
- 5 cache containers (to speed up responses)
- 10 background worker containers (to process jobs)
- 3 message queue containers (to coordinate workers)

That is 41 containers. Can you manage them with Docker Compose on a single machine? Maybe, but that single machine is now a single point of failure. If it crashes, your entire application goes down.

A Production Application

Now imagine a real production setup:

- 200+ containers running across 30 servers
- Containers need to be restarted automatically when they crash
- New versions need to be deployed without downtime
- Traffic needs to be distributed evenly across containers
- Some containers need more CPU, others need more memory
- You need to scale up during peak hours and scale down at night

Managing this by hand is impossible. You would need a team of people watching dashboards 24 hours a day, restarting crashed containers, moving workloads between servers, and updating configurations. One mistake could take down your entire application.

This is the problem that container orchestration solves.

What Is Container Orchestration?

Container orchestration is the automatic management of containerized applications across multiple servers. An orchestrator handles:

Task	What It Means
Scheduling	Deciding which server should run each container
Scaling	Adding or removing container copies based on demand
Self-healing	Restarting crashed containers automatically
Load balancing	Distributing traffic evenly across containers
Rolling updates	Deploying new versions without downtime
Service discovery	Letting containers find and talk to each other
Storage management	Attaching persistent storage to containers
Configuration	Managing secrets and settings across containers

The Orchestra Analogy

Think of a symphony orchestra:

- The musicians are your containers. Each one plays a specific part (web server, database, cache).
- The instruments are the servers. Multiple musicians can share an instrument section.
- The conductor is the orchestrator. The conductor does not play any instrument. Instead, the conductor:
    - Tells each musician when to start and stop
    - Makes sure all musicians play in harmony
    - Brings in more musicians when the music needs to be louder
    - Replaces a musician who gets sick (without stopping the performance)
    - Coordinates tempo changes (rolling updates) smoothly

Kubernetes is the conductor of your container orchestra.

What Is Kubernetes?

Kubernetes (often written as K8s — the "8" stands for the eight letters between "K" and "s") is an open-source container orchestration platform. It was originally designed by Google, based on over 15 years of experience running containers in production. Google donated it to the Cloud Native Computing Foundation (CNCF) in 2014, and it has since become the industry standard.

What Kubernetes Does

You tell Kubernetes: "I want 5 copies of my web server running at all times."

Kubernetes then:

1. Finds servers with enough resources to run those 5 copies
2. Starts the containers on those servers
3. Monitors them continuously
4. If one container crashes, starts a replacement automatically
5. If a server goes down, moves the containers to healthy servers
6. If you change the number to 10, starts 5 more containers
7. If you deploy a new version, replaces old containers one at a time

You describe the desired state (what you want), and Kubernetes makes it happen and keeps it that way. This is called declarative configuration.

Declarative vs. Imperative

This distinction is important:

```
Imperative (telling Kubernetes what to do step by step):
  "Start a container. Now start another one. Now stop the first one."

Declarative (telling Kubernetes what you want):
  "I want 3 containers running version 2.0 of my app."
```

With the declarative approach, Kubernetes figures out the steps to get from the current state to your desired state. If you currently have 2 containers running version 1.0, Kubernetes will:

1. Start 1 new container (to reach 3 total)
2. Replace each version 1.0 container with a version 2.0 container

3. Ensure 3 healthy version 2.0 containers are running

You do not need to write the steps. You just describe the end result.

Kubernetes Architecture

Kubernetes runs as a cluster — a group of machines working together. Every Kubernetes cluster has two types of machines:

1. Control plane nodes (the brain) — they make decisions about the cluster
2. Worker nodes (the muscle) — they run your actual containers

Here is how they fit together:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        KUBERNETES CLUSTER                                 │
│                                                                           │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                      CONTROL PLANE                               │   │
│   │                     (The Brain)                                  │   │
│   │                                                                  │   │
│   │   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐    │   │
│   │   │ API Server   │   │ Scheduler    │   │ Controller       │    │   │
│   │   │              │   │              │   │ Manager          │    │   │
│   │   │ Front door   │   │ Decides      │   │ Keeps desired    │    │   │
│   │   │ for all      │   │ where to     │   │ state matching   │    │   │
│   │   │ requests     │   │ run pods     │   │ actual state     │    │   │
│   │   └──────────────┘   └──────────────┘   └──────────────────┘    │   │
│   │                                                                  │   │
│   │   ┌──────────────────────────────────────────────────────────┐   │   │
│   │   │                         etcd                             │   │   │
│   │   │            Cluster database (stores all state)           │   │   │
│   │   └──────────────────────────────────────────────────────────┘   │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                  │                                        │
│                       kubectl commands                                   │
│                       travel through                                     │
│                       the API Server                                     │
│                                  │                                        │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────┐        │
│   │   WORKER NODE 1  │   │   WORKER NODE 2  │   │ WORKER NODE 3│        │
│   │   (The Muscle)   │   │   (The Muscle)   │   │ (The Muscle) │        │
│   │                  │   │                  │   │              │        │
│   │  ┌──────────────┐│   │  ┌──────────────┐│   │ ┌──────────┐ │        │
│   │  │   kubelet    │   │  │   kubelet    │   │ │  kubelet │ │        │
│   │  │ (node agent) │   │  │ (node agent) │   │ │(node agt)│ │        │
│   │  └──────────────┘   │  └──────────────┘   │ └──────────┘ │        │
│   │  ┌──────────────┐   │  ┌──────────────┐   │ ┌──────────┐ │        │
```

```
│   │ │  kube-proxy  ││   │ │  kube-proxy  ││   │ │kube-proxy│ │    │
│   │ │ (networking) ││   │ │ (networking) ││   │ │(network) │ │    │
│   │ └──────────────┘│   │ └──────────────┘│   │ └──────────┘ │    │
│   │                 │   │                 │   │              │    │
│   │ ┌──────────────┐│   │ ┌──────────────┐│   │ ┌──────────┐ │    │
│   │ │  Container   ││   │ │  Container   ││   │ │Container │ │    │
│   │ │  Runtime     ││   │ │  Runtime     ││   │ │ Runtime  │ │    │
│   │ │  (containerd)││   │ │  (containerd)││   │ │(cntrd)   │ │    │
│   │ └──────────────┘│   │ └──────────────┘│   │ └──────────┘ │    │
│   │                 │   │                 │   │              │    │
│   │   ┌────┐  ┌────┐   │   ┌────┐  ┌────┐   │   ┌────┐       │    │
│   │   │Pod │  │Pod │   │   │Pod │  │Pod │   │   │Pod │       │    │
│   │   │ A  │  │ B  │   │   │ C  │  │ D  │   │   │ E  │       │    │
│   │   └────┘  └────┘   │   └────┘  └────┘   │   └────┘       │    │
│   └────────────────────┘   └────────────────┘   └──────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

Let us examine each component.

Control Plane Components

The control plane is the brain of the cluster. It makes all the decisions but does not run your application containers. In production, the control plane usually runs on dedicated servers (often 3 or 5 for high availability).

API Server (kube-apiserver)

The API Server is the front door to the Kubernetes cluster. Every command, every request, every piece of communication goes through the API Server.

Think of it as the receptionist at a large company. Whether you are a visitor (a developer running kubectl), an employee (a component inside the cluster), or a delivery person (an external system), you go through the receptionist first. The receptionist validates your identity, checks if you are allowed in, and directs you to the right place.

What it does:

- Receives all requests (from kubectl, from other components, from external tools)
- Validates requests (checks if the YAML is correct, if you have permission)
- Stores the validated data in etcd
- Notifies other components about changes

```bash
# When you run this command:
kubectl get pods

# Here is what happens:
# 1. kubectl sends an HTTPS request to the API Server
# 2. The API Server checks your credentials
# 3. The API Server reads the pod data from etcd
# 4. The API Server sends the data back to kubectl
# 5. kubectl displays it in your terminal
```

Scheduler (kube-scheduler)

The Scheduler decides which worker node should run each new container (technically, each new Pod — we will explain Pods in Chapter 28).

Think of it as a hotel front desk clerk assigning rooms to guests. When a new guest arrives, the clerk checks:

- Which rooms are available?
- Does the guest need a room with a king bed (specific CPU/memory requirements)?
- Does the guest want a room on a high floor (specific node preferences)?
- Which floor has the fewest guests right now (load balancing)?

What it does:

- Watches for new Pods that have no assigned node
- Evaluates which nodes have enough resources (CPU, memory, disk)
- Considers constraints (some Pods must run on specific nodes)
- Assigns the Pod to the best available node

```
Scheduler Decision Process:

  New Pod needs:          Node A          Node B          Node C
  2 CPU, 4GB RAM         8 CPU free      1 CPU free      4 CPU free
                         16GB free       2GB free        8GB free

  Can it fit?            YES             NO              YES
  Current load:          3 pods          7 pods          2 pods

  Decision: ──────────────────────────────────────────► Node C
  (fits and has the lightest load)
```

etcd

etcd (pronounced "et-see-dee") is a distributed key-value database that stores all cluster data. It is the single source of truth for the entire cluster.

Think of it as the filing cabinet in that large company. Every employee record, every policy document, every piece of important information is stored there. If the filing cabinet is destroyed, the company loses everything.

What it stores:

- What nodes exist in the cluster
- What Pods are running and where
- What configurations and secrets exist
- The desired state of every resource
- The current state of every resource

Why it matters:

- If etcd loses data, the entire cluster loses its memory
- In production, etcd is always backed up and replicated across multiple machines
- etcd uses the Raft consensus protocol to ensure all copies agree on the data

Controller Manager (kube-controller-manager)

The Controller Manager runs a collection of controllers — background loops that continuously compare the desired state with the actual state and take action to fix any differences.

Think of it as a thermostat. You set the thermostat to 72 degrees Fahrenheit (the desired state). The thermostat continuously checks the actual temperature. If it is too cold, it turns on the heater. If it is too hot, it turns on the air conditioning. It never stops checking.

Some controllers inside the Controller Manager:

Controller	What It Does
Replication Controller	Ensures the correct number of Pod copies are running
Node Controller	Detects when a node goes down and responds
Job Controller	Manages one-time tasks that run to completion
Endpoint Controller	Connects Services to Pods

Example of the control loop:

```
Desired State: 3 replicas of web-server

Control Loop:
  1. Check: How many web-server pods are running?  → 3
     Action: Nothing. Everything is fine.

  2. Check: How many web-server pods are running?  → 2 (one crashed)
     Action: Start 1 new web-server pod.

  3. Check: How many web-server pods are running?  → 3
     Action: Nothing. Back to normal.

  (This loop runs continuously, every few seconds)
```

Worker Node Components

Worker nodes are the machines that actually run your application containers. A cluster can have 1 worker node or 1,000 — you add more as your application grows.

kubelet

The kubelet is an agent that runs on every worker node. It is the node's representative in the cluster.

Think of it as a site manager on a construction site. The headquarters (control plane) sends blueprints (Pod specifications). The site manager makes sure the buildings (containers) are built according to the blueprints and reports back on progress.

What it does:

- Receives Pod specifications from the API Server
- Tells the container runtime to start or stop containers
- Monitors containers and reports their health back to the control plane
- Mounts volumes and manages storage for Pods

kube-proxy

kube-proxy runs on every worker node and manages network rules. It makes sure that network traffic reaches the right containers.

Think of it as a mail room clerk. When mail (network traffic) arrives at the building (node), the mail room clerk knows which office (container) it should go to. If a container moves to a

different node, kube-proxy updates its routing rules so traffic follows the container to its new location.

What it does:

- Maintains network rules on each node
- Forwards traffic to the correct Pods
- Enables communication between Pods across different nodes
- Implements load balancing for Services

Container Runtime

The container runtime is the software that actually runs containers. Kubernetes does not run containers directly — it tells the container runtime to do it.

Think of it as the engine in a car. Kubernetes is the driver who decides where to go and when. The container runtime is the engine that actually makes the car move.

Common container runtimes:

- containerd — the most common runtime (Docker uses this under the hood)
- CRI-O — a lightweight runtime built specifically for Kubernetes

```
How a container starts:

  API Server                kubelet              containerd         Container
      |                        |                     |                  |
      |  "Run this Pod"        |                     |                  |
      |------------------------>|                     |                  |
      |                        |  "Start container"  |                  |
      |                        |--------------------->|                  |
      |                        |                     |  Pull image      |
      |                        |                     |  Create container|
      |                        |                     |------------------>|
      |                        |                     |                  | Running
      |                        |  "Container ready"  |                  |
      |                        |<---------------------|                  |
      |  "Pod is running"      |                     |                  |
      |<------------------------|                     |                  |
```

How Everything Works Together

Let us walk through what happens when you deploy an application to Kubernetes:

Step 1: You submit a request

```bash
kubectl apply -f my-app.yaml
```

This sends your YAML file to the API Server.

Step 2: API Server validates and stores

The API Server checks the YAML for errors, verifies you have permission, and stores the desired state in etcd.

Step 3: Controller Manager detects work to do

The Controller Manager notices: "The user wants 3 replicas of my-app, but 0 exist. I need to create 3 Pods."

Step 4: Scheduler assigns nodes

For each new Pod, the Scheduler finds the best worker node based on available resources.

Step 5: kubelet starts containers

The kubelet on each assigned node receives the Pod specification and tells the container runtime to pull the image and start the container.

Step 6: Continuous monitoring

The kubelet reports container health to the API Server. The Controller Manager continuously checks that the desired state matches the actual state. If a container crashes, the cycle repeats from Step 3.

```
The Full Flow:

  You (kubectl)
      │
      ▼
  ┌──────────┐     ┌──────────────┐     ┌──────────┐
  │API Server │────►│ Controller Mgr │────►│ Scheduler │
  └──────────┘     └──────────────┘     └──────────┘
      │                                      │
      │                                      │
      └──────────────►┌──────┐◄──────────────┘
                      │ etcd │
                      └──────┘
                         │
          ┌──────────────┼──────────────┐
          ▼              ▼              ▼
     ┌─────────┐    ┌─────────┐    ┌─────────┐
     │ kubelet │    │ kubelet │    │ kubelet │
     │ Node 1  │    │ Node 2  │    │ Node 3  │
```

```
        └──────────┘   └──────────┘   └──────────┘
             │              │              │
        ┌────┴─────┐   ┌────┴─────┐   ┌────┴─────┐
        │Container│   │Container│   │Container│
        │ Runtime │   │ Runtime │   │ Runtime │
        └──────────┘   └──────────┘   └──────────┘
```

kubectl: Your Way to Talk to Kubernetes

kubectl (pronounced "kube-control" or "kube-C-T-L") is the command-line tool you use to communicate with a Kubernetes cluster. Every command you type in kubectl is sent to the API Server.

Think of kubectl as your remote control for the Kubernetes cluster. You do not log into the cluster machines directly. Instead, you use kubectl from your own computer to send commands.

```bash
# Some basic kubectl commands (we will use these in later chapters):

kubectl get pods           # List all running Pods
kubectl get nodes          # List all nodes in the cluster
kubectl get services       # List all Services
kubectl describe pod my-pod  # Show detailed info about a Pod
kubectl logs my-pod        # View container logs
kubectl apply -f file.yaml # Apply a configuration file
kubectl delete pod my-pod  # Delete a Pod
```

Kubernetes vs. Docker Swarm

In Chapter 24, you learned about Docker Swarm. Both Swarm and Kubernetes are container orchestrators, but they differ significantly:

Feature	Docker Swarm	Kubernetes
Setup complexity	Very easy (built into Docker)	More complex (separate installation)
Learning curve	Gentle	Steep

Scaling	Good for small to medium	Excellent for any size
Auto-scaling	Manual scaling only	Supports automatic scaling
Self-healing	Basic (restarts containers)	Advanced (restarts, reschedules, replaces)
Rolling updates	Supported	Advanced (canary, blue-green, rollback)
Networking	Simple overlay networks	Advanced (network policies, ingress)
Storage	Basic volume support	Advanced (PV, PVC, storage classes)
Community	Smaller	Massive (largest open-source community)
Cloud support	Limited	All major clouds (AWS EKS, Google GKE, Azure AKS)
Ecosystem	Few add-ons	Thousands of tools and extensions
Production use	Small to medium deployments	Small to massive deployments

When to Use Docker Swarm

- You have a small team and a simple application
- You want to get up and running quickly
- You do not need advanced features like auto-scaling
- You are already comfortable with Docker Compose
- Your application runs on fewer than 10-20 nodes

When to Use Kubernetes

- You need to run containers at large scale (dozens to thousands of nodes)
- You need automatic scaling based on traffic
- You need advanced deployment strategies (canary releases, blue-green deployments)
- You want a rich ecosystem of tools and integrations
- Your cloud provider offers managed Kubernetes (EKS, GKE, AKS)
- Your team is willing to invest time in learning

The Industry Trend

Kubernetes has become the dominant container orchestrator. Most companies that run containers in production use Kubernetes. Major cloud providers offer managed Kubernetes services that handle the complex setup for you:

- Amazon EKS (Elastic Kubernetes Service)
- Google GKE (Google Kubernetes Engine)
- Azure AKS (Azure Kubernetes Service)
- DigitalOcean Kubernetes

These managed services let you focus on deploying your applications while the cloud provider manages the control plane.

```
Managed Kubernetes:

  ┌──────────────────────────────────────────┐
  │         Cloud Provider Manages           │
  │  ┌──────────────────────────────────┐    │
  │  │         Control Plane            │    │
  │  │  API Server, Scheduler, etcd,    │    │
  │  │  Controller Manager              │    │
  │  └──────────────────────────────────┘    │
  └──────────────────────────────────────────┘

  ┌──────────────────────────────────────────┐
  │           You Manage                     │
  │  ┌──────────┐  ┌──────────┐  ┌────────┐  │
  │  │ Worker   │  │ Worker   │  │ Worker │  │
  │  │ Node 1   │  │ Node 2   │  │ Node 3 │  │
  │  │ (Your    │  │ (Your    │  │ (Your  │  │
  │  │  apps)   │  │  apps)   │  │  apps) │  │
  │  └──────────┘  └──────────┘  └────────┘  │
  └──────────────────────────────────────────┘
```

Key Kubernetes Terminology

Here is a quick reference of terms you will encounter throughout the Kubernetes chapters:

Term	Meaning
Cluster	A group of machines running Kubernetes
Node	A single machine in the cluster (physical or virtual)
Control Plane	The set of components that manage the cluster

Worker Node	A machine that runs your application containers
Pod	The smallest deployable unit (one or more containers)
Deployment	Manages multiple copies of a Pod
Service	A stable network endpoint to access Pods
Namespace	A way to divide cluster resources between projects
kubectl	The command-line tool for interacting with Kubernetes
YAML manifest	A file describing the desired state of a resource
Desired state	What you want the cluster to look like
Actual state	What the cluster currently looks like

Common Mistakes

1. Thinking Kubernetes replaces Docker. Kubernetes does not replace Docker. Kubernetes uses a container runtime (like containerd) to run containers. Docker builds the images; Kubernetes orchestrates the containers made from those images. They work together.
2. Using Kubernetes for everything. Not every application needs Kubernetes. If you have a simple application running on one server, Docker Compose is simpler and faster. Kubernetes adds complexity. Use it when you actually need its features.
3. Ignoring the learning curve. Kubernetes has many concepts and components. Do not try to learn everything at once. Start with the basics (Pods, Deployments, Services) and gradually add more knowledge.
4. Confusing control plane and worker nodes. The control plane makes decisions. Worker nodes run your containers. In development (Minikube), both run on the same machine. In production, they are separate machines.
5. Thinking you need to manage your own cluster. Most companies use managed Kubernetes services (EKS, GKE, AKS). These handle the control plane for you. Unless you have specific reasons, use a managed service.

Best Practices

1. Start with managed Kubernetes. Unless you are learning Kubernetes administration specifically, use a managed service. It saves enormous amounts of time and effort.
2. Learn the fundamentals first. Master Pods, Deployments, and Services before moving to advanced topics like custom resources, operators, or service meshes.
3. Use declarative configuration. Always use YAML files with `kubectl apply` rather than imperative commands like `kubectl run`. YAML files can be version-controlled and reviewed.
4. Version-control your YAML files. Store all your Kubernetes manifests in Git. This gives you a history of changes and the ability to roll back.
5. Use namespaces to organize. Separate different environments (dev, staging, production) and different teams using namespaces.

Quick Summary

Kubernetes is a container orchestration platform that automates the deployment, scaling, and management of containerized applications across clusters of machines. It has a control plane (API Server, Scheduler, Controller Manager, etcd) that makes decisions and worker nodes (kubelet, kube-proxy, container runtime) that run containers. You interact with Kubernetes using kubectl, and you describe your desired state in YAML files. Kubernetes continuously works to make the actual state match your desired state.

Key Points

- Container orchestration automates the management of containers across multiple servers
- Kubernetes uses a declarative model: you describe what you want, and it makes it happen
- The control plane (API Server, Scheduler, Controller Manager, etcd) is the brain of the cluster
- Worker nodes (kubelet, kube-proxy, container runtime) run your application containers
- kubectl is the command-line tool for communicating with the cluster
- Kubernetes is more powerful but more complex than Docker Swarm
- Most companies use managed Kubernetes services from cloud providers
- Kubernetes does not replace Docker — they complement each other

Practice Questions

1. You have a web application with 3 containers running via Docker Compose. A colleague suggests migrating to Kubernetes. What questions would you ask before deciding?
2. Explain the role of the Scheduler. What information does it use to decide where to place a Pod?
3. What happens when a container crashes in Kubernetes? Walk through the sequence of events, naming each component involved.
4. A colleague says "We do not need Docker anymore because we have Kubernetes." Why is this statement incorrect?
5. What is the difference between the desired state and the actual state? Give an example of a situation where they might differ.

Exercises

Exercise 1: Architecture Diagram

Draw the Kubernetes architecture diagram from memory. Include:

- Control plane with all four components
- At least two worker nodes with all three components
- Arrows showing how kubectl communicates with the cluster

Check your diagram against the one in this chapter. Did you miss any components?

Exercise 2: Component Matching

Match each scenario with the Kubernetes component responsible:

Scenario	Component
A new Pod needs to be placed on a node	?
A Pod's container crashed and needs restarting	?
A developer runs `kubectl get pods`	?

Cluster state needs to be stored persistently	?
Network traffic needs to reach the right Pod	?
A container image needs to be pulled and started	?

Exercise 3: Kubernetes vs. Swarm Decision

For each scenario, decide whether Docker Swarm or Kubernetes is the better choice. Explain your reasoning:

1. A startup with 2 developers running 5 containers on a single server
2. An e-commerce company expecting 10x traffic during holiday sales
3. A team that already uses Docker Compose and wants minimal changes

---

What Is Next?

Now that you understand what Kubernetes is and how its components work together, you need a way to run it on your own machine. In the next chapter, you will install and set up a local Kubernetes environment using Minikube. You will run your first kubectl commands and see a real cluster in action.

Chapter 27: Setting Up Kubernetes Locally

What You Will Learn

By the end of this chapter, you will be able to:

- Install Minikube on Mac, Windows, or Linux
- Install kubectl and verify it works
- Start and stop a local Kubernetes cluster
- Run your first kubectl commands to inspect the cluster
- Understand what namespaces are and how to create them
- Switch between different Kubernetes contexts
- Open the Kubernetes Dashboard to see your cluster visually
- Troubleshoot common setup problems

Why This Chapter Matters

In the previous chapter, you learned what Kubernetes is and how its architecture works. You know about the control plane, worker nodes, the API Server, the Scheduler, etcd, and kubelet. That is a lot of theory.

Now it is time to get your hands dirty. You need a real Kubernetes cluster to practice with, but you do not need a fleet of expensive servers. You can run a fully functional Kubernetes cluster right on your laptop.

This chapter walks you through the setup step by step. By the end, you will have a working cluster and you will have run your first kubectl commands. Every chapter after this one builds on this setup.

Your Options for Running Kubernetes Locally

There are several ways to run Kubernetes on your own machine. Think of these as different ways to set up a practice gym — some are simple, some offer more features, but they all let you practice.

Tool	Description	Best For
Minikube	Runs a single-node cluster in a virtual machine or container	Learning and development
Docker Desktop	Includes a built-in Kubernetes option	Docker users who want a quick setup
kind	Kubernetes in Docker — runs cluster nodes as Docker containers	Testing and CI/CD
k3s	Lightweight Kubernetes distribution	Raspberry Pi, edge computing

We will focus on Minikube because it is the most popular tool for learning Kubernetes. It works on all operating systems and gives you a complete Kubernetes cluster with one command. We will also cover the Docker Desktop option for those who prefer a simpler setup.

Option A: Installing Minikube

Minikube creates a tiny Kubernetes cluster inside a virtual machine or a Docker container on your laptop. Think of it as a miniature data center that fits inside your computer.

Prerequisites

Before installing Minikube, you need two things:

1. A container or virtual machine manager. Minikube needs something to run the cluster inside. If you have Docker installed (which you do if you have been following this book), that is enough.

2. At least 2 CPUs, 2 GB of free memory, and 20 GB of free disk space. Kubernetes is not a lightweight tool. Make sure your computer meets these requirements.

Installing on macOS

Using Homebrew (recommended):

```bash
brew install minikube
```

```
==> Downloading https://ghcr.io/v2/homebrew/core/minikube/blobs/sha256:...
==> Installing minikube
==> Pouring minikube--1.32.0.arm64_sonoma.bottle.tar.gz
==> Summary
  /opt/homebrew/Cellar/minikube/1.32.0: 9 files, 93.2MB
```

What just happened? Homebrew is a package manager for macOS (like an app store for command-line tools). It downloaded the Minikube binary and placed it in your system path so you can run it from any terminal window.

Using direct download (if you do not have Homebrew):

```bash
# For Apple Silicon (M1, M2, M3 Macs):
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-darwin-arm64
sudo install minikube-darwin-arm64 /usr/local/bin/minikube

# For Intel Macs:
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-darwin-amd64
sudo install minikube-darwin-amd64 /usr/local/bin/minikube
```

Let us break down those commands:

- `curl -LO` — Downloads a file from the internet. `-L` follows redirects, `-O` saves the file with its original name.
- `sudo install` — Copies the downloaded file to `/usr/local/bin/` and makes it executable. `sudo` is needed because `/usr/local/bin/` is a system directory.

Installing on Windows

Using Windows Package Manager (winget):

```powershell
winget install Kubernetes.minikube
```

Using Chocolatey:

```powershell
choco install minikube
```

Using direct download:

1. Download the installer from https://minikube.sigs.k8s.io/docs/start/
2. Run the `.exe` installer
3. Follow the installation wizard

After installation, open a new terminal window (this is important — the old terminal will not know about the new command).

Installing on Linux

Using direct download:

```bash
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Using package manager (Debian/Ubuntu):

```bash
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
sudo dpkg -i minikube_latest_amd64.deb
```

Using package manager (Fedora/RHEL):

```bash
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-latest.x86_64...
sudo rpm -Uvh minikube-latest.x86_64.rpm
```

Verifying the Installation

After installing, verify it works:

```bash
minikube version
```

```
minikube version: v1.32.0
commit: 8220a6eb95f0a4d75f7f2d7b14cef975f050512d
```

If you see a version number, Minikube is installed correctly.

Option B: Enabling Kubernetes in Docker Desktop

If you already have Docker Desktop installed, you can enable Kubernetes with a few clicks. No additional installation required.

Steps

1. Open Docker Desktop
2. Click the gear icon (Settings)
3. Click Kubernetes in the left sidebar
4. Check the box that says Enable Kubernetes
5. Click Apply & Restart

```
Docker Desktop Settings:

  ┌──────────────────────────────────────────┐
  │  General                                 │
  │  Resources                               │
  │  Docker Engine                           │
  │  ► Kubernetes  ◄── Click this            │
  │  Software Updates                        │
  │                                          │
  │   ┌──────────────────────────────────┐   │
  │   │ [✓] Enable Kubernetes            │   │
  │   │                                  │   │
  │   │ Kubernetes is starting...        │   │
  │   └──────────────────────────────────┘   │
  │                                          │
  │  [Apply & Restart]                       │
  └──────────────────────────────────────────┘
```

Docker Desktop will download the Kubernetes components and start the cluster. This takes a few minutes the first time. When the Kubernetes icon in the bottom-left corner turns green, the cluster is ready.

Pros and Cons of Docker Desktop Kubernetes

Pros	Cons

No extra installation needed	Fewer configuration options than Minikube
Starts automatically with Docker Desktop	Uses more memory (Docker + Kubernetes)
Simple on/off toggle	Cannot easily create multiple clusters
Shares Docker's images and resources	Harder to reset if something breaks

---

Installing kubectl

kubectl (pronounced "kube-control" or "kube-C-T-L") is the command-line tool you use to talk to your Kubernetes cluster. Think of it as the remote control for your cluster. Without kubectl, you cannot give commands to Kubernetes.

> Note: If you installed Minikube, it comes with its own kubectl. You can use `minikube kubectl -- <command>` instead of `kubectl <command>`. However, installing kubectl separately is recommended because it is more convenient.

Installing kubectl on macOS

```bash
brew install kubectl
```

```
==> Downloading https://ghcr.io/v2/homebrew/core/kubernetes-cli/blobs/...
==> Installing kubernetes-cli
==> Summary
  /opt/homebrew/Cellar/kubernetes-cli/1.29.2: 236 files, 58.3MB
```

Installing kubectl on Windows

```powershell
winget install Kubernetes.kubectl
```

Or with Chocolatey:

```powershell
choco install kubernetes-cli
```

Installing kubectl on Linux

```bash
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/...
sudo install kubectl /usr/local/bin/kubectl
```

Let us break this down:

- The inner `curl` command (`curl -L -s https://dl.k8s.io/release/stable.txt`) fetches the latest stable version number (like "v1.29.2").
- The outer `curl` command uses that version number to download the matching kubectl binary.
- `sudo install` makes it executable and moves it to the right location.

Verifying kubectl

```bash
kubectl version --client
```

```
Client Version: v1.29.2
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
```

The `--client` flag tells kubectl to only show its own version, not try to connect to a cluster (which might not be running yet).

Starting Your First Cluster

Now comes the exciting part. Let us start a Kubernetes cluster.

Starting Minikube

```bash
minikube start
```

```
□  minikube v1.32.0 on Darwin 14.0 (arm64)
□  Automatically selected the docker driver. Other choices: qemu2, ssh
□  Using Docker Desktop driver with root privileges
□  Starting control plane node minikube in cluster minikube
```

```
    Pulling base image ...
    Downloading Kubernetes v1.28.3 preload ...
    Creating docker container (CPUs=2, Memory=4096MB, Disk=20000MB) ...
    Preparing Kubernetes v1.28.3 on Docker 24.0.7 ...
      ▪ Generating certificates and keys ...
      ▪ Booting up control plane ...
      ▪ Configuring RBAC rules ...
    Configuring bridge CNI (Container Networking Interface) ...
    Verifying Kubernetes components...
      ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
    Enabled addons: storage-provisioner, default-storageclass
    Done! kubectl is now configured to use "minikube" cluster and
     "default" namespace by default
```

What just happened? Let us walk through each line:

1. Minikube detected your operating system and chose Docker as the driver (the tool to create the virtual environment).
2. It downloaded the Kubernetes base image — a pre-built package containing all Kubernetes components.
3. It created a Docker container with 2 CPUs and 4 GB of memory to act as your cluster node.
4. It installed Kubernetes v1.28.3 inside that container.
5. It generated security certificates (like ID cards for cluster components to identify themselves).
6. It started the control plane (API Server, Scheduler, Controller Manager, etcd).
7. It configured networking so Pods can communicate.
8. It verified everything is working.
9. It configured kubectl to talk to this new cluster.

Your local Kubernetes cluster is now running. All the components from Chapter 26 — the API Server, Scheduler, Controller Manager, etcd, kubelet, and kube-proxy — are all running inside a single Docker container on your machine.

```
Your Laptop
┌──────────────────────────────────────────────────────┐
│                                                      │
│   Docker                                             │
│   ┌──────────────────────────────────────────────┐   │
│   │  Minikube Container                          │   │
│   │   ┌──────────────────────────────────────┐   │   │
│   │   │      Control Plane                   │   │   │
│   │   │  API Server  Scheduler               │   │   │
│   │   │  Controller  etcd                    │   │   │
│   │   │  Manager                             │   │   │
│   │   ├──────────────────────────────────────┤   │   │
```

```
│   │   │     Worker Node                │  │ │
│   │   │  kubelet  kube-proxy           │  │ │
│   │   │  Container Runtime             │  │ │
│   │   └────────────────────────────────┘  │ │
│   └──────────────────────────────────────┘ │
│                                            │
│   kubectl  ◄── You type commands here      │
└────────────────────────────────────────────┘
```

In a real production cluster, the control plane and worker nodes would be on separate machines. Minikube puts everything on one machine for simplicity.

Your First kubectl Commands

Now that the cluster is running, let us explore it.

Checking Cluster Information

```bash
kubectl cluster-info
```

```
Kubernetes control plane is running at https://127.0.0.1:55000
CoreDNS is running at https://127.0.0.1:55000/api/v1/namespaces/kube-system/services/ku...

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

What does this tell you?

- The control plane (API Server) is running on your local machine (`127.0.0.1`) on port 55000.
- CoreDNS is running — this is Kubernetes' built-in DNS service that lets Pods find each other by name (like a phone book for your cluster).

Listing Nodes

```bash
kubectl get nodes
```

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   2m    v1.28.3
```

Let us break down each column:

Column	Meaning
NAME	The name of the node. Minikube creates one node called "minikube".
STATUS	Whether the node is healthy. "Ready" means it can accept workloads.
ROLES	What role this node plays. "control-plane" means it runs the brain of the cluster. In Minikube, the same node also acts as a worker.
AGE	How long the node has been running.
VERSION	The version of Kubernetes running on this node.

In a real cluster, you would see multiple nodes:

```bash
# What a production cluster might look like:
kubectl get nodes
```

```
NAME              STATUS   ROLES           AGE   VERSION
master-1          Ready    control-plane   30d   v1.28.3
worker-1          Ready    <none>          30d   v1.28.3
worker-2          Ready    <none>          30d   v1.28.3
worker-3          Ready    <none>          25d   v1.28.3
```

Getting More Details About a Node

```bash
kubectl describe node minikube
```

```
Name:               minikube
Roles:              control-plane
Labels:             kubernetes.io/hostname=minikube
                    node-role.kubernetes.io/control-plane=
Annotations:        kubeadm.alpha.kubernetes.io/cri-socket: ...
CreationTimestamp:   Mon, 15 Jan 2024 10:30:00 -0500
Conditions:
```

```
  Type             Status
  ----             ------
  MemoryPressure   False
  DiskPressure     False
  PIDPressure      False
  Ready            True
Capacity:
  cpu:               2
  memory:            4046788Ki
  pods:              110
Allocatable:
  cpu:               2
  memory:            4046788Ki
  pods:              110
...
```

This shows detailed information about the node:

- Conditions — Is the node under memory pressure? Disk pressure? Is it ready? All False/True values look healthy here.
- Capacity — The total resources available: 2 CPUs, about 4 GB of memory, and room for up to 110 Pods.
- Allocatable — How much of those resources Kubernetes can actually use for your workloads.

Understanding Namespaces

What Is a Namespace?

A namespace is a way to divide your Kubernetes cluster into separate sections. Think of it as folders on your computer. You do not put every file on your desktop — you organize them into folders like "Work," "Personal," and "Projects." Namespaces do the same thing for Kubernetes resources.

```
Without Namespaces (messy):
┌──────────────────────────────────────────┐
│  Kubernetes Cluster                      │
│                                          │
│  frontend-pod    backend-pod             │
│  database-pod    test-frontend-pod       │
│  test-backend    staging-database        │
│  monitoring-pod  logging-pod             │
│                                          │
```

```
│  Everything mixed together!              │
└──────────────────────────────────────────┘

With Namespaces (organized):
┌──────────────────────────────────────────┐
│  Kubernetes Cluster                      │
│                                          │
│  ┌──────────────┐  ┌──────────────┐      │
│  │ production   │  │ staging      │      │
│  │              │  │              │      │
│  │ frontend-pod │  │ frontend-pod │      │
│  │ backend-pod  │  │ backend-pod  │      │
│  │ database-pod │  │ database-pod │      │
│  └──────────────┘  └──────────────┘      │
│                                          │
│  ┌──────────────┐  ┌──────────────┐      │
│  │ monitoring   │  │ testing      │      │
│  │              │  │              │      │
│  │ prometheus   │  │ test-runner  │      │
│  │ grafana      │  │ test-db      │      │
│  └──────────────┘  └──────────────┘      │
└──────────────────────────────────────────┘
```

Why Use Namespaces?

1. Organization — Keep different projects or environments separate.
2. Access control — Limit who can access resources in each namespace.
3. Resource limits — Set CPU and memory limits per namespace so one team cannot use all the resources.
4. Name reuse — You can have a Pod called "database" in the "production" namespace and another Pod called "database" in the "staging" namespace. They do not conflict.

Default Namespaces

Kubernetes creates several namespaces automatically. Let us see them:

```bash
kubectl get namespaces
```

```
NAME              STATUS   AGE
default           Active   5m
kube-node-lease   Active   5m
kube-public       Active   5m
kube-system       Active   5m
```

Namespace	Purpose
default	Where your resources go if you do not specify a namespace. This is your "home folder."
kube-system	Where Kubernetes' own components run (DNS, proxy, dashboard). Do not put your stuff here.
kube-public	Readable by everyone. Rarely used directly.
kube-node-lease	Used internally for node heartbeats. You can ignore this.

Seeing What Runs in kube-system

Curious what Kubernetes is running behind the scenes? Let us look:

```bash
kubectl get pods -n kube-system
```

```
NAME                                READY   STATUS    RESTARTS   AGE
coredns-5dd5756b68-abcde            1/1     Running   0          5m
etcd-minikube                       1/1     Running   0          5m
kube-apiserver-minikube             1/1     Running   0          5m
kube-controller-manager-minikube    1/1     Running   0          5m
kube-proxy-xyz12                    1/1     Running   0          5m
kube-scheduler-minikube             1/1     Running   0          5m
storage-provisioner                 1/1     Running   0          5m
```

Look at that! Those are all the components from Chapter 26 running as Pods: the API Server, Controller Manager, Scheduler, etcd, kube-proxy, and CoreDNS.

The `-n kube-system` flag tells kubectl to look in the `kube-system` namespace instead of the `default` namespace.

Creating Your Own Namespace

```bash
kubectl create namespace my-project
```

```
namespace/my-project created
```

Let us verify it:

```bash
kubectl get namespaces
```

```
NAME              STATUS   AGE
default           Active   10m
kube-node-lease   Active   10m
kube-public       Active   10m
kube-system       Active   10m
my-project        Active   5s
```

Your new namespace "my-project" is now ready to use.

Creating a Namespace with YAML

You can also define a namespace in a YAML file. This is the declarative approach we discussed in Chapter 26:

```yaml
# namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: my-project
```

Let us break down each line:

Line	Meaning
`apiVersion: v1`	Which version of the Kubernetes API to use. `v1` is the stable, core API.
`kind: Namespace`	What type of resource you are creating. In this case, a Namespace.
`metadata:`	Information about the resource.
`name: my-project`	The name you want to give this namespace.

Apply it with:

```bash
kubectl apply -f namespace.yaml
```

```
namespace/my-project created
```

Deleting a Namespace

```bash
kubectl delete namespace my-project
```

```
namespace "my-project" deleted
```

Warning: Deleting a namespace deletes everything inside it — all Pods, Services, ConfigMaps, and other resources. This is like deleting a folder on your computer. Everything inside is gone. Be careful with this command.

---

kubectl Context: Talking to the Right Cluster

What Is a Context?

A context tells kubectl which cluster to talk to and which namespace to use by default. Think of it as a phone contact — it stores the number (cluster address), the name (cluster name), and some preferences (default namespace).

If you only have one cluster (Minikube), you only have one context. But in the real world, developers often have access to multiple clusters — a development cluster, a staging cluster, and a production cluster. Contexts let you switch between them.

Viewing Your Current Context

```bash
kubectl config current-context
```

```
minikube
```

This tells you that kubectl is currently talking to the "minikube" cluster.

Viewing All Contexts

```bash
kubectl config get-contexts
```

```
CURRENT   NAME       CLUSTER    AUTHINFO   NAMESPACE
*         minikube   minikube   minikube   default
```

Column	Meaning
CURRENT	The asterisk (*) marks the active context.
NAME	The name of the context.
CLUSTER	Which cluster this context points to.
AUTHINFO	The credentials used to authenticate.
NAMESPACE	The default namespace for this context.

Switching Contexts

If you had multiple clusters, you would switch between them like this:

```bash
# Switch to a different context
kubectl config use-context my-production-cluster
```

```
Switched to context "my-production-cluster".
```

Setting a Default Namespace

Tired of typing `-n my-project` every time? Set a default namespace for your current context:

```bash
kubectl config set-context --current --namespace=my-project
```

```
Context "minikube" modified.
```

Now every kubectl command will use the `my-project` namespace by default, so you do not need to add `-n my-project` to every command.

To switch back to the default namespace:

```bash
kubectl config set-context --current --namespace=default
```

```
Context "minikube" modified.
```

```
Context Switching:

  You (kubectl)
      │
      │  current-context = "minikube"
      │
      ├──► minikube cluster  (development)
      │
      ├──► staging cluster   (testing)
      │
      └──► production cluster (live users)

  Only ONE context is active at a time.
  kubectl sends commands to the active context.
```

The Kubernetes Dashboard

The Kubernetes Dashboard is a web-based user interface that lets you see your cluster visually. It is like switching from a command-line file manager to a graphical file manager — same information, easier to browse.

Opening the Dashboard with Minikube

```bash
minikube dashboard
```

```
🔌  Enabling dashboard ...
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  Some dashboard features require the metrics-server addon.
    To enable all features please run:

    minikube addons enable metrics-server

🤔  Verifying dashboard health ...
🚀  Launching proxy ...
🤔  Verifying proxy health ...
🎉  Opening http://127.0.0.1:54321/api/v1/namespaces/kubernetes-dashboard/...
    in your default browser...
```

Your web browser will open automatically and show the Kubernetes Dashboard:

```
Kubernetes Dashboard:

  ┌──────────────────────────────────────────────────┐
  │  ☸ Kubernetes Dashboard                          │
  ├──────────┬───────────────────────────────────────┤
  │          │                                       │
  │ Cluster  │   Namespace: default                  │
  │  Nodes   │                                       │
  │          │   Workloads                            │
  │ Workloads│    ┌──────────┬────────┬────────────┐  │
  │  Pods    │    │ Resource │ Status │ Count      │  │
  │  Deploy  │    ├──────────┼────────┼────────────┤  │
  │  Replica │    │ Pods     │ 0/0    │ All good   │  │
  │  Sets    │    │ Deploys  │ 0/0    │ All good   │  │
  │          │    │ Services │ 1      │ kubernetes │  │
  │ Services │    └──────────┴────────┴────────────┘  │
  │          │                                       │
  │ Config   │                                       │
  │  ConfigMaps                                      │
  │  Secrets │                                       │
  │          │                                       │
  │ Storage  │                                       │
  │  PVCs    │                                       │
  └──────────┴───────────────────────────────────────┘
```

The Dashboard lets you:

- View all resources (Pods, Deployments, Services) in your cluster
- See resource status and health
- View container logs
- Execute commands inside containers
- Create resources from the UI (though we recommend using YAML files instead)

Press `Ctrl+C` in the terminal to stop the Dashboard proxy when you are done.

Docker Desktop Dashboard

If you are using Docker Desktop's Kubernetes, you do not get the Minikube Dashboard command. You can install the Dashboard manually or use Docker Desktop's own interface, which shows basic Kubernetes information.

Stopping and Starting Minikube

Stopping the Cluster

When you are done working, stop Minikube to free up your computer's resources:

```bash
minikube stop
```

```
✋  Stopping node "minikube"  ...
✋  Powering off "minikube" via SSH ...
✋  1 node stopped.
```

This does not delete anything. Your cluster, its configuration, and any resources you created are all saved. Think of it as putting your cluster to sleep.

Starting Again

```bash
minikube start
```

```
✋  minikube v1.32.0 on Darwin 14.0 (arm64)
✋  Using the docker driver based on existing profile
✋  Starting control plane node minikube in cluster minikube
✋  Done! kubectl is now configured to use "minikube" cluster
```

Notice it says "based on existing profile" — Minikube remembered your configuration and started the same cluster.

Checking Minikube Status

```bash
minikube status
```

```
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

All components are running. If any line says "Stopped," your cluster is not fully running.

Deleting the Cluster

If you want to completely remove the cluster and start fresh:

```bash
minikube delete
```

```
  Deleting "minikube" in docker ...
  Deleting container "minikube" ...
  Removing /Users/yourname/.minikube/machines/minikube ...
  Removed all traces of the "minikube" cluster.
```

This removes everything — the virtual machine, all resources, and all data. Use this only if you want a completely fresh start.

Common Mistakes

1. Forgetting to start the cluster. If you see "The connection to the server was refused," your cluster is not running. Run `minikube start` first.
2. Not enough resources. Minikube needs at least 2 CPUs and 2 GB of memory. If your computer is low on resources, Minikube will fail to start or run slowly. Close other applications to free up memory.
3. Wrong context. If you have multiple clusters, kubectl might be pointing to the wrong one. Always check with `kubectl config current-context` before running commands.
4. Docker not running. Minikube with the Docker driver needs Docker to be running. If you see "docker is not running," start Docker Desktop first.
5. Using kube-system namespace for your apps. The `kube-system` namespace is for Kubernetes' own components. Create your own namespaces for your applications. Putting your Pods in `kube-system` can cause confusion and potential conflicts.
6. Forgetting the namespace flag. If you created resources in a specific namespace but cannot find them, you are probably looking in the wrong namespace. Use `kubectl get pods -n your-namespace` or `kubectl get pods --all-namespaces` to search everywhere.
7. Not closing the Dashboard properly. The `minikube dashboard` command runs in the foreground. Use `Ctrl+C` to stop it. Do not just close the terminal window, or the proxy process may keep running.

Best Practices

1. Stop Minikube when not in use. Kubernetes consumes CPU and memory even when idle. Run `minikube stop` when you are done to free up resources.
2. Use namespaces from the start. Even in local development, get into the habit of creating namespaces for your projects. It will make transitioning to production easier.
3. Set a default namespace. Use `kubectl config set-context --current --namespace=my-project` to avoid typing `-n my-project` repeatedly.
4. Use YAML files for everything. Even for namespaces, use YAML files and `kubectl apply`. This creates a record of what you did and makes it reproducible.
5. Bookmark useful kubectl commands. You will use `kubectl get`, `kubectl describe`, and `kubectl logs` constantly. Practice them until they become muscle memory.

Quick Summary

To run Kubernetes locally, install Minikube and kubectl. Start a cluster with `minikube start`, which creates a single-node Kubernetes cluster inside a Docker container on your machine. Use kubectl to interact with the cluster: `kubectl get nodes` shows your nodes, `kubectl get namespaces` shows your namespaces, and `kubectl config current-context` shows which cluster you are connected to. Namespaces organize resources like folders organize files. The Kubernetes Dashboard provides a visual interface for browsing your cluster.

Key Points

- Minikube creates a single-node Kubernetes cluster on your laptop for learning and development
- kubectl is the command-line tool for communicating with any Kubernetes cluster
- `minikube start` starts the cluster; `minikube stop` pauses it; `minikube delete` removes it
- `kubectl get nodes` shows the machines in your cluster
- `kubectl cluster-info` shows where the control plane is running
- Namespaces divide your cluster into separate sections, like folders on a computer
- Every cluster has a `default` namespace and a `kube-system` namespace (for Kubernetes internals)
- Contexts tell kubectl which cluster and namespace to use
- `kubectl config current-context` shows which cluster you are connected to

- The Dashboard provides a web-based visual interface for your cluster

---

Practice Questions

1. What is the difference between `minikube stop` and `minikube delete`? When would you use each one?
2. You run `kubectl get pods` and see no results, but you know you created Pods earlier. What might be wrong? How would you investigate?
3. Explain what a Kubernetes namespace is using a real-life analogy. Why would you use multiple namespaces instead of putting everything in the default namespace?
4. You have three clusters: development, staging, and production. How do you switch kubectl between them? How do you verify which cluster you are currently connected to?
5. A teammate says "I ran `kubectl get pods` and I see all the Kubernetes system Pods mixed with my application Pods." What advice would you give them?

Exercises

Exercise 1: Cluster Setup and Exploration

1. Install Minikube (or enable Kubernetes in Docker Desktop)
2. Start a cluster with `minikube start`
3. Run `kubectl cluster-info` and note the control plane URL
4. Run `kubectl get nodes` and note the node name, status, and Kubernetes version
5. Run `kubectl get pods -n kube-system` and identify which Pod corresponds to which component from Chapter 26

Exercise 2: Namespace Management

1. Create three namespaces: `development`, `staging`, and `production`
2. Verify all three exist with `kubectl get namespaces`
3. Set your default namespace to `development` using `kubectl config set-context`
4. Verify the change with `kubectl config get-contexts`
5. Delete the `staging` namespace
6. Reset your default namespace back to `default`

Exercise 3: Context Practice

1. Run `kubectl config get-contexts` and note all the columns
2. Run `kubectl config view` and examine the output — try to identify the cluster server address, the user credentials, and the context definition
3. Create a YAML file for a namespace called "practice" and apply it with `kubectl apply -f`
4. Delete it with `kubectl delete -f` using the same YAML file

What Is Next?

You now have a working Kubernetes cluster on your machine and you know how to interact with it using kubectl. But the cluster is empty — there are no applications running in it yet.

In the next chapter, you will deploy your first application to Kubernetes. You will learn about Pods (the smallest unit in Kubernetes), Deployments (which manage groups of Pods), and Services (which let you access your Pods over the network). You will write YAML manifests and watch Kubernetes bring your application to life.

Chapter 28: Pods, Deployments, and Services

What You Will Learn

By the end of this chapter, you will be able to:

- Explain what a Pod is and why it is the smallest unit in Kubernetes
- Write a Pod YAML manifest from scratch
- Use kubectl to create, inspect, and troubleshoot Pods
- Explain why you rarely create Pods directly in production
- Understand what a Deployment is and how it manages Pods for you
- Scale Pods up and down using Deployments
- Perform rolling updates and rollbacks
- Explain what a Service is and why Pods need one
- Distinguish between ClusterIP, NodePort, and LoadBalancer services
- Deploy a complete application with replicas and a Service

Why This Chapter Matters

In the previous chapter, you set up a local Kubernetes cluster. You ran kubectl commands and explored namespaces. But the cluster was empty — no applications were running.

This chapter changes that. Pods, Deployments, and Services are the three most fundamental building blocks of Kubernetes. Nearly everything you do with Kubernetes involves at least one of these three. If Kubernetes is a house, Pods are the bricks, Deployments are the construction crew, and Services are the doors.

You cannot use Kubernetes effectively without understanding these three concepts deeply. By the end of this chapter, you will have a real application running in your cluster and accessible from your browser.

Pods: The Smallest Unit in Kubernetes

What Is a Pod?

A Pod is the smallest thing you can deploy in Kubernetes. A Pod is a wrapper around one or more containers. In most cases, a Pod contains exactly one container.

Think of a Pod as a lunch box. The food inside is your container (your application). The lunch box provides a wrapper that gives the food:

- A place to exist (an IP address)
- Protection (isolation from other Pods)
- Shared resources (all containers in the same Pod share the same network and storage)

```
A Pod with one container (most common):

  ┌──────────────────────┐
  │  Pod                 │
  │  ┌────────────────┐  │
  │  │  Container     │  │
  │  │  (your app)    │  │
  │  └────────────────┘  │
  │                      │
  │  IP: 10.244.0.5      │
  └──────────────────────┘

A Pod with multiple containers (advanced use case):

  ┌──────────────────────────────┐
  │  Pod                         │
  │  ┌────────────┐ ┌──────────┐│
  │  │  Container │ │ Container│
  │  │  (main app)│ │ (log     │
  │  │            │ │  shipper)│
  │  └────────────┘ └──────────┘
  │                              │
  │  IP: 10.244.0.6              │
  │  Shared network and storage  │
  └──────────────────────────────┘
```

Why Not Just Use Containers?

You might wonder: "Why add a Pod wrapper around a container? Why not just run containers directly?"

The Pod exists because Kubernetes needs a higher-level concept that provides:

1. A unique IP address. Every Pod gets its own IP address. Containers inside the same Pod share this IP.
2. Shared storage. Containers in the same Pod can share files through volumes.
3. Lifecycle management. Kubernetes manages Pods, not individual containers. It starts them, monitors them, and restarts them as a unit.
4. Co-location of tightly coupled containers. Sometimes two containers need to work together closely (like a web app and a log collector). Putting them in the same Pod ensures they always run on the same node.

Writing Your First Pod YAML

Kubernetes uses YAML files to describe what you want. These files are called manifests. Let us write a manifest for a simple Pod running an Nginx web server.

Create a file called `pod.yaml`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-first-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx-container
      image: nginx:1.25
      ports:
        - containerPort: 80
```

Let us break this down line by line:

Line	Meaning
`apiVersion: v1`	Which version of the Kubernetes API to use. Pods are part of the core API, so this is `v1`.
`kind: Pod`	What type of resource you are creating.
`metadata:`	Information about this Pod.

`name: my-first-pod`	The name of the Pod. Must be unique within the namespace.
`labels:`	Key-value pairs attached to the Pod. Labels are like tags — they help you organize and select Pods.
`app: nginx`	A label that says this Pod belongs to the "nginx" app.
`spec:`	The specification — what the Pod should contain.
`containers:`	A list of containers to run inside this Pod.
`- name: nginx-container`	The name of this container. The dash (`-`) means this is an item in a list.
`image: nginx:1.25`	The Docker image to use. Kubernetes will pull this image (just like `docker pull`).
`ports:`	Which ports the container listens on.
`containerPort: 80`	This container listens on port 80 (the standard HTTP port).

The Structure of Every Kubernetes YAML

Almost every Kubernetes YAML file has the same four top-level sections:

```yaml
apiVersion:  # Which API version to use
kind:        # What type of resource (Pod, Deployment, Service, etc.)
metadata:    # Name, labels, annotations — info ABOUT the resource
spec:        # The actual specification — what the resource SHOULD be
```

```
Every Kubernetes YAML:

  ┌──────────────────────┐
  │ apiVersion: v1       │◄── Which API to talk to
  ├──────────────────────┤
  │ kind: Pod            │◄── What to create
  ├──────────────────────┤
  │ metadata:            │◄── Info about it (name, labels)
  │   name: my-pod       │
  │   labels:            │
  │     app: myapp       │
  ├──────────────────────┤
  │ spec:                │◄── What it should contain/do
  │   containers:        │
  │     - name: ...      │
  │       image: ...     │
  └──────────────────────┘
```

Creating the Pod

Now let us create the Pod in your cluster:

```bash
kubectl apply -f pod.yaml
```

```
pod/my-first-pod created
```

What just happened?

1. kubectl read your YAML file.
2. It sent the file to the Kubernetes API Server.
3. The API Server validated the YAML (checked for errors).
4. The API Server stored the desired state in etcd.
5. The Scheduler found a node with enough resources and assigned the Pod to it.
6. The kubelet on that node pulled the `nginx:1.25` image and started the container.

Inspecting Your Pod

List all Pods:

```bash
kubectl get pods
```

```
NAME           READY   STATUS    RESTARTS   AGE
my-first-pod   1/1     Running   0          30s
```

Column	Meaning
NAME	The Pod name from your YAML.
READY	How many containers are ready out of total. `1/1` means 1 container running out of 1 total.
STATUS	The current state. `Running` means everything is fine.
RESTARTS	How many times containers have been restarted. `0` means no crashes.
AGE	How long the Pod has been running.

Get more details:

```bash
kubectl get pods -o wide
```

```
NAME           READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE
my-first-pod   1/1     Running   0          1m    10.244.0.5   minikube   <none>
```

The `-o wide` flag shows extra columns, including the Pod's IP address and which node it is running on.

Describe the Pod (full details):

```bash
kubectl describe pod my-first-pod
```

```
Name:             my-first-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Mon, 15 Jan 2024 10:45:00 -0500
Labels:           app=nginx
Status:           Running
IP:               10.244.0.5
Containers:
  nginx-container:
    Image:          nginx:1.25
    Port:           80/TCP
    State:          Running
      Started:      Mon, 15 Jan 2024 10:45:02 -0500
    Ready:          True
    Restart Count:  0
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  2m    default-scheduler  Successfully assigned default/my-first-po...
  Normal  Pulling    2m    kubelet            Pulling image "nginx:1.25"
  Normal  Pulled     1m    kubelet            Successfully pulled image "nginx:1.25"
  Normal  Created    1m    kubelet            Created container nginx-container
  Normal  Started    1m    kubelet            Started container nginx-container
```

The Events section at the bottom is extremely useful for troubleshooting. It shows you the step-by-step history of what happened to this Pod:

1. The Scheduler assigned it to the minikube node
2. The kubelet pulled the nginx image
3. The container was created

4. The container was started

View container logs:

```bash
kubectl logs my-first-pod
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt
to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
...
2024/01/15 15:45:02 [notice] 1#1: nginx/1.25.3
2024/01/15 15:45:02 [notice] 1#1: built by gcc 12.2.0
```

This is the same output you would see with `docker logs`. It shows what the Nginx container printed when it started.

Run a command inside the Pod:

```bash
kubectl exec -it my-first-pod -- /bin/bash
```

```
root@my-first-pod:/#
```

This is similar to `docker exec`. The `--` separates kubectl flags from the command to run. `-i` means interactive (keep stdin open), `-t` means allocate a terminal. Type `exit` to leave.

Deleting a Pod

```bash
kubectl delete pod my-first-pod
```

```
pod "my-first-pod" deleted
```

Or delete using the YAML file:

```bash
kubectl delete -f pod.yaml
```

```
pod "my-first-pod" deleted
```

Why You Rarely Create Pods Directly

Here is an important lesson: in production, you almost never create Pods directly.

Why? Because Pods are mortal. They do not self-heal.

If a Pod crashes, it is gone. Kubernetes does not automatically create a replacement when you create Pods directly. If the node where your Pod is running goes down, the Pod goes down with it.

```
Pod created directly:

  Pod crashes ──► Pod is gone ──► Nobody creates a new one
                                   You must do it manually!

Pod created by a Deployment:

  Pod crashes ──► Deployment notices ──► Deployment creates
                                          a new Pod automatically!
```

This is like the difference between hiring a single worker with no manager versus hiring through a staffing agency. If the single worker calls in sick, nobody shows up. If you hire through an agency (a Deployment), they send a replacement.

So what do you use instead of creating Pods directly? You use a Deployment.

Deployments: Managing Pods at Scale

What Is a Deployment?

A Deployment is a Kubernetes resource that manages Pods for you. Think of it as a manager who ensures a certain number of workers (Pods) are always on the job.

You tell the Deployment: "I want 3 copies of my web server running at all times."

The Deployment then:

1. Creates 3 Pods
2. Monitors them continuously
3. If one Pod crashes, creates a replacement
4. If a node goes down, reschedules the Pods to healthy nodes

5. Handles rolling updates when you deploy a new version
6. Supports rollback if the new version has problems

How Deployments Work

A Deployment does not manage Pods directly. It creates an intermediate resource called a ReplicaSet, and the ReplicaSet manages the Pods.

```
Deployment ──creates──► ReplicaSet ──creates──► Pods

  ┌──────────────┐      ┌──────────────┐      ┌─────────┐
  │ Deployment   │──────►│ ReplicaSet   │──────►│ Pod 1   │
  │              │      │              │      │ Pod 2   │
  │ replicas: 3  │      │ replicas: 3  │      │ Pod 3   │
  └──────────────┘      └──────────────┘      └─────────┘
```

You do not need to create ReplicaSets yourself. Deployments handle them automatically. Think of the ReplicaSet as the Deployment's assistant.

Writing a Deployment YAML

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

This is longer than the Pod YAML. Let us break it down section by section:

Top section (same as any Kubernetes resource):

Line	Meaning
`apiVersion: apps/v1`	Deployments are in the `apps` API group, version `v1`.
`kind: Deployment`	We are creating a Deployment.
`metadata: name: nginx-deployment`	The name of this Deployment.

Spec section (what makes a Deployment special):

Line	Meaning
`replicas: 3`	Run 3 copies (replicas) of the Pod at all times.
`selector: matchLabels: app: nginx`	The Deployment manages Pods that have the label `app: nginx`. This is how the Deployment finds its Pods.
`template:`	The Pod template — a blueprint for creating Pods.

The template section is basically a Pod definition embedded inside the Deployment. It has its own `metadata` (with labels) and its own `spec` (with containers).

```
Deployment YAML structure:

┌──────────────────────────────────────┐
│  Deployment                          │
│  ┌──────────────────────────────────┐│
│  │  apiVersion, kind, metadata      │
│  ├──────────────────────────────────┤│
│  │  spec:                           │
│  │    replicas: 3                   │
│  │    selector: app=nginx           │
│  │  ┌──────────────────────────┐    │
│  │  │  template (Pod spec)     │    │
│  │  │  ┌────────────────────┐   │    │
│  │  │  │  metadata:         │   │    │
│  │  │  │    labels:         │   │    │
│  │  │  │      app: nginx    │   │    │
│  │  │  ├────────────────────┤   │    │
│  │  │  │  spec:             │   │    │
│  │  │  │    containers:     │   │    │
│  │  │  │    - nginx:1.25    │   │    │
│  │  │  └────────────────────┘   │    │
│  │  └──────────────────────────┘    │
│  └──────────────────────────────────┘│
└──────────────────────────────────────┘
```

Important: The labels in `selector.matchLabels` must match the labels in `template.metadata.labels`. This is how the Deployment knows which Pods belong to it. If they do not match, Kubernetes will reject the YAML with an error.

Creating the Deployment

```bash
kubectl apply -f deployment.yaml
```

```
deployment.apps/nginx-deployment created
```

Inspecting the Deployment

```bash
kubectl get deployments
```

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           30s
```

Column	Meaning
READY	How many Pods are ready out of the desired number. `3/3` means all 3 are running.
UP-TO-DATE	How many Pods have been updated to the latest version.
AVAILABLE	How many Pods are available to serve traffic.

List the Pods created by the Deployment:

```bash
kubectl get pods
```

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c79c4bf97-abc12   1/1     Running   0          45s
nginx-deployment-7c79c4bf97-def34   1/1     Running   0          45s
nginx-deployment-7c79c4bf97-ghi56   1/1     Running   0          45s
```

Notice the Pod names. They follow the pattern: `deployment-name` + `replicaset-hash` + `random-suffix`. The random suffix ensures each Pod has a unique name.

List the ReplicaSet (created automatically):

```bash
kubectl get replicasets
```

```
NAME                          DESIRED   CURRENT   READY   AGE
nginx-deployment-7c79c4bf97   3         3         3       1m
```

The ReplicaSet was created by the Deployment. You did not write any YAML for it.

Self-Healing in Action

Now let us see self-healing in action. Delete one of the Pods:

```bash
kubectl delete pod nginx-deployment-7c79c4bf97-abc12
```

```
pod "nginx-deployment-7c79c4bf97-abc12" deleted
```

Immediately check the Pods again:

```bash
kubectl get pods
```

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c79c4bf97-def34   1/1     Running   0          2m
nginx-deployment-7c79c4bf97-ghi56   1/1     Running   0          2m
nginx-deployment-7c79c4bf97-jkl78   1/1     Running   0          3s
```

The Deployment noticed that only 2 Pods existed but the desired state was 3. It created a new Pod (`jkl78`) to replace the deleted one. The entire process took seconds and required no manual intervention.

This is the power of Deployments. They ensure your desired state is always maintained.

Scaling: Changing the Number of Replicas

Scale up (more Pods):

```bash
kubectl scale deployment nginx-deployment --replicas=5
```

```
deployment.apps/nginx-deployment scaled
```

```bash
kubectl get pods
```

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c79c4bf97-def34   1/1     Running   0          5m
nginx-deployment-7c79c4bf97-ghi56   1/1     Running   0          5m
nginx-deployment-7c79c4bf97-jkl78   1/1     Running   0          3m
nginx-deployment-7c79c4bf97-mno90   1/1     Running   0          10s
nginx-deployment-7c79c4bf97-pqr12   1/1     Running   0          10s
```

Two new Pods appeared. Kubernetes found resources on the cluster and started them.

Scale down (fewer Pods):

```bash
kubectl scale deployment nginx-deployment --replicas=2
```

```
deployment.apps/nginx-deployment scaled
```

```bash
kubectl get pods
```

```
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c79c4bf97-def34   1/1     Running   0          6m
nginx-deployment-7c79c4bf97-ghi56   1/1     Running   0          6m
```

Kubernetes terminated 3 Pods, leaving only 2. It chose which ones to stop based on internal criteria (like how long they have been running and their current load).

Rolling Updates: Deploying a New Version

One of the most powerful features of Deployments is rolling updates. When you deploy a new version of your application, Kubernetes replaces old Pods with new Pods one at a time, ensuring your application is never fully down.

Update the image version:

```bash
kubectl set image deployment/nginx-deployment nginx=nginx:1.26
```

```
deployment.apps/nginx-deployment image updated
```

Let us break down this command:

- `set image` — Change the container image.
- `deployment/nginx-deployment` — The Deployment to update.
- `nginx=nginx:1.26` — Change the container named "nginx" to use the image `nginx:1.26`.

Watch the rollout progress:

```bash
kubectl rollout status deployment/nginx-deployment
```

```
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 2
new replicas have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas
are pending termination...
deployment "nginx-deployment" successfully rolled out
```

What happened behind the scenes:

```
Rolling Update Process:

  Step 1: Original state
  ┌──────────┐  ┌──────────┐
  │ Pod v1.25│  │ Pod v1.25│
  └──────────┘  └──────────┘

  Step 2: Create new Pod, keep old ones running
  ┌──────────┐  ┌──────────┐  ┌──────────┐
  │ Pod v1.25│  │ Pod v1.25│  │ Pod v1.26│
  └──────────┘  └──────────┘  └──────────┘
                                   NEW

  Step 3: New Pod is healthy, terminate one old Pod
  ┌──────────┐  ┌──────────┐
  │ Pod v1.25│  │ Pod v1.26│
  └──────────┘  └──────────┘

  Step 4: Create another new Pod
  ┌──────────┐  ┌──────────┐  ┌──────────┐
  │ Pod v1.25│  │ Pod v1.26│  │ Pod v1.26│
  └──────────┘  └──────────┘  └──────────┘
                                   NEW

  Step 5: Terminate last old Pod
  ┌──────────┐  ┌──────────┐
  │ Pod v1.26│  │ Pod v1.26│
  └──────────┘  └──────────┘
```

```
                    ALL UPDATED
```

Kubernetes created new Pods with the updated image and gradually terminated old Pods. At no point were all Pods down simultaneously. Your users would not have experienced any downtime.

Check the rollout history:

```bash
kubectl rollout history deployment/nginx-deployment
```

```
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

Each revision represents a version of your Deployment. Revision 1 was `nginx:1.25`, Revision 2 is `nginx:1.26`.

Rollback: Undoing a Bad Update

What if the new version has a bug? You can roll back to the previous version:

```bash
kubectl rollout undo deployment/nginx-deployment
```

```
deployment.apps/nginx-deployment rolled back
```

This performs another rolling update, but in reverse — replacing the new version Pods with old version Pods. Your users experience zero downtime.

Roll back to a specific revision:

```bash
kubectl rollout undo deployment/nginx-deployment --to-revision=1
```

```
deployment.apps/nginx-deployment rolled back
```

This is like an "undo" button for your deployments. In a real production scenario, this can save you from a disaster.

Services: Giving Pods a Stable Address

The Problem: Pod IPs Change

You know that every Pod gets an IP address. But here is the problem: Pod IPs are not permanent.

When a Pod crashes and a new one replaces it, the new Pod gets a different IP address. When you scale up, the new Pods get new IPs. When you do a rolling update, all the Pods get new IPs.

If another application is trying to reach your Pods, how does it know what IP to use? It cannot hard-code the IP because the IP keeps changing.

```
The Problem:

  App ──► Pod (10.244.0.5)
                │
                ▼ Pod crashes
  App ──► ???   New Pod (10.244.0.9)
                │
                │ Where did it go?
```

What Is a Service?

A Service is a stable network endpoint that provides a permanent address for a group of Pods. Think of it as a phone number that never changes.

You and your friends might change phone companies, move to different cities, and get new phones. But if you have a permanent office phone number, people can always reach you by calling that number. The phone number stays the same even if the person answering changes.

A Service works the same way. It has a permanent IP address and DNS name. When traffic comes to the Service, it forwards it to one of the healthy Pods behind it. If Pods change, the Service keeps working — it simply updates its list of Pods.

```
The Solution — a Service:

  ┌──────────────────────────────────────────┐
  │                                          │
```

```
│  App ──► Service (10.96.0.1)              │
│           "my-service"                    │
│              │                            │
│          ┌───┼───┐                        │
│          ▼   ▼   ▼                        │
│       Pod1  Pod2  Pod3                    │
│                                           │
│  Pods come and go.                        │
│  Service IP stays the same.               │
└───────────────────────────────────────────┘
```

How Services Find Their Pods

Services use labels to find Pods. Remember the `labels:` section in our Pod template? The Service uses a selector that matches those labels.

```yaml
# In the Deployment template:
labels:
  app: nginx    # ◄── Pod has this label

# In the Service:
selector:
  app: nginx    # ◄── Service looks for Pods with this label
```

When a new Pod is created with the label `app: nginx`, the Service automatically discovers it and starts sending traffic to it. When a Pod with that label is deleted, the Service stops sending traffic to it. This happens automatically — you do not need to update anything.

Service Types

Kubernetes has three main types of Services, each for a different use case:

```
Service Types:

  ClusterIP (internal only):
  ┌──────────────────────────────────┐
  │  Kubernetes Cluster              │
  │                                  │
  │  App A ──► Service ──► Pods      │
  │            (internal)            │
  │                                  │
  │  Only accessible from inside     │
  │  the cluster.                    │
  └──────────────────────────────────┘

  NodePort (accessible from outside):
```

```
┌──────────────────────────────────┐
│  Kubernetes Cluster              │
│                                  │
│  Service ──► Pods                │
│  Port 30080                      │
│     ▲                            │
└─────│────────────────────────────┘
      │
Your Browser ──► Node IP:30080

LoadBalancer (cloud provider assigns an external IP):
┌──────────────────────────────────┐
│  Cloud Provider                  │
│  ┌────────────────────────┐      │
│  │ Load Balancer          │      │
│  │ External IP: 34.56.78.90 │    │
│  └────────────────────────┘      │
│              │                   │
│  ────────────▼──────────────     │
│  ┌────────────────────────┐      │
│  │  Kubernetes Cluster    │      │
│  │  Service ──► Pods      │      │
│  └────────────────────────┘      │
└──────────────────────────────────┘
```

Type	What It Does	When to Use
ClusterIP	Creates an internal IP. Only accessible from inside the cluster.	Communication between services inside the cluster (e.g., app talks to database).
NodePort	Opens a port (30000-32767) on every node. Accessible from outside.	Development and testing. Quick way to access your app from your browser.
LoadBalancer	Asks the cloud provider to create an external load balancer.	Production on cloud providers (AWS, GCP, Azure). The standard way to expose apps to the internet.

ClusterIP Service (Default)

This is the default type. It creates an internal-only IP address.

```yaml
# service-clusterip.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-internal
spec:
```

```
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

Line	Meaning
`type: ClusterIP`	Internal-only Service. This is the default — you can omit this line and it will still be ClusterIP.
`selector: app: nginx`	Send traffic to Pods with the label `app: nginx`.
`port: 80`	The port the Service listens on.
`targetPort: 80`	The port on the Pod to forward traffic to.

Use ClusterIP when you have a database or internal API that only other Pods need to access.

NodePort Service

This opens a port on every node in the cluster so you can access the Service from outside.

```yaml
# service-nodeport.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-external
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Line	Meaning
`type: NodePort`	Make this Service accessible from outside the cluster.
`nodePort: 30080`	The port to open on the node. Must be between 30000 and 32767. If you omit this, Kubernetes picks a random port in that range.

```
NodePort traffic flow:

  Your Browser
      │
      │ http://node-ip:30080
      ▼
  ┌──────────┐
  │   Node   │
  │ Port     │
  │ 30080    │
  │    │     │
  │    ▼     │
  │ Service  │
  │ Port 80  │
  │    │     │
  │ ┌──┼──┐  │
  │ ▼  ▼  ▼  │
  │ P1 P2 P3 │  ◄── Pods on port 80
  └──────────┘
```

LoadBalancer Service

On cloud providers, this creates a real load balancer with an external IP:

```yaml
# service-loadbalancer.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-public
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

On Minikube, LoadBalancer does not work the same way as on a cloud provider because there is no cloud load balancer. Minikube provides a workaround with `minikube service <service-name>`, which opens the Service in your browser.

Complete Example: Nginx with 3 Replicas and a NodePort Service

Let us put everything together. We will deploy Nginx with 3 replicas and expose it via a NodePort Service so you can access it from your browser.

Step 1: Create the Deployment

```yaml
# nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-web
  labels:
    app: nginx-web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-web
  template:
    metadata:
      labels:
        app: nginx-web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

Step 2: Create the Service

```yaml
# nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-web-service
spec:
  type: NodePort
  selector:
    app: nginx-web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Step 3: Apply Both Files

```bash
kubectl apply -f nginx-deployment.yaml
```

```
deployment.apps/nginx-web created
```

```bash
kubectl apply -f nginx-service.yaml
```

```
service/nginx-web-service created
```

Step 4: Verify Everything Is Running

```bash
kubectl get all
```

```
NAME                              READY   STATUS    RESTARTS   AGE
pod/nginx-web-5d4f8b7c69-abc12  1/1     Running   0          30s
pod/nginx-web-5d4f8b7c69-def34  1/1     Running   0          30s
pod/nginx-web-5d4f8b7c69-ghi56  1/1     Running   0          30s

NAME                        TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
service/kubernetes          ClusterIP   10.96.0.1      <none>        443/TCP        1h
service/nginx-web-service   NodePort    10.96.45.123   <none>        80:30080/TCP   30s

NAME                        READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-web   3/3     3            3           30s

NAME                                   DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-web-5d4f8b7c69   3         3         3       30s
```

The `kubectl get all` command shows everything: Pods, Services, Deployments, and ReplicaSets.

Step 5: Access the Application

On Minikube:

```bash
minikube service nginx-web-service
```

```
|-----------|-------------------|-------------|---------------------------|
| NAMESPACE |       NAME        | TARGET PORT |            URL            |
```

```
|-----------|--------------------|-------------|---------------------------|
| default   | nginx-web-service |          80 | http://192.168.49.2:30080 |
|-----------|--------------------|-------------|---------------------------|
□  Opening service default/nginx-web-service in default browser...
```

Your browser opens and you see the Nginx welcome page. You have a web server running in Kubernetes!

On Docker Desktop Kubernetes:

```bash
curl http://localhost:30080
```

Or open `http://localhost:30080` in your browser.

Step 6: Test Self-Healing

Delete one of the Pods and watch Kubernetes replace it:

```bash
kubectl delete pod nginx-web-5d4f8b7c69-abc12
```

```bash
kubectl get pods
```

```
NAME                              READY   STATUS    RESTARTS   AGE
nginx-web-5d4f8b7c69-def34      1/1     Running   0          5m
nginx-web-5d4f8b7c69-ghi56      1/1     Running   0          5m
nginx-web-5d4f8b7c69-jkl78      1/1     Running   0          5s
```

A new Pod was created within seconds. The Service automatically routes traffic to the remaining healthy Pods — your users would not notice a thing.

Step 7: Clean Up

```bash
kubectl delete -f nginx-deployment.yaml
kubectl delete -f nginx-service.yaml
```

```
deployment.apps "nginx-web" deleted
service "nginx-web-service" deleted
```

Common Mistakes

1. Creating Pods directly instead of using Deployments. Direct Pods do not self-heal. Always use Deployments unless you have a specific reason not to (like running a one-time batch job).
2. Mismatched labels between Deployment selector and template. The `selector.matchLabels` in the Deployment must match the `template.metadata.labels`. If they do not match, Kubernetes rejects the YAML.
3. Mismatched labels between Service selector and Pod labels. If the Service selector does not match any Pod labels, the Service will have no endpoints and traffic will go nowhere. Use `kubectl describe service <name>` to check the Endpoints field.
4. Forgetting `containerPort` does not publish a port. Unlike Docker's `-p` flag, `containerPort` in a Pod spec is informational only. It documents which port the container listens on. To actually access the Pod from outside, you need a Service.
5. Using NodePort in production. NodePort is great for development, but in production you should use LoadBalancer or Ingress. NodePort exposes a high-numbered port on every node, which is not user-friendly and can be a security concern.
6. Not checking Pod events when a Pod is stuck. If a Pod is in `Pending`, `ImagePullBackOff`, or `CrashLoopBackOff` status, run `kubectl describe pod <name>` and read the Events section. It tells you exactly what went wrong.

Best Practices

1. Always use Deployments. Even if you only need one Pod, create a Deployment with `replicas: 1`. This gives you self-healing, rolling updates, and rollback for free.
2. Use meaningful labels. Labels like `app: nginx-web`, `environment: production`, and `team: backend` make it easy to organize and filter resources.
3. Store YAML files in version control. Put your Deployment and Service YAML files in Git. This creates a history of changes and lets you reproduce your setup on any cluster.
4. Use `kubectl apply` instead of `kubectl create`. The `apply` command is idempotent — you can run it multiple times, and it will update the resource if it exists or create it if it does not. The `create` command fails if the resource already exists.
5. Set resource requests and limits. In production, always specify how much CPU and memory your containers need. We will cover this in later chapters, but it prevents one application from consuming all the resources on a node.

Quick Summary

Pods are the smallest deployable units in Kubernetes, wrapping one or more containers with a shared IP and storage. Deployments manage Pods by maintaining a desired number of replicas, handling self-healing, rolling updates, and rollbacks. Services provide stable network endpoints for accessing Pods, with three types: ClusterIP for internal communication, NodePort for external access during development, and LoadBalancer for production on cloud providers. Together, these three resources form the foundation of every Kubernetes application.

Key Points

- A Pod is the smallest unit in Kubernetes — a wrapper around one or more containers
- Pods are mortal — they do not self-heal when created directly
- A Deployment manages Pods, ensuring the desired number of replicas are always running
- Deployments provide self-healing, scaling, rolling updates, and rollback
- `kubectl scale` changes the number of Pod replicas
- `kubectl set image` triggers a rolling update to a new version
- `kubectl rollout undo` reverts to a previous version
- A Service provides a stable IP and DNS name for accessing a group of Pods
- ClusterIP is for internal communication (default)
- NodePort exposes a port on every node (development/testing)
- LoadBalancer creates an external load balancer (production on cloud)
- Services find their Pods using label selectors

Practice Questions

1. What is the difference between a Pod and a container? Why does Kubernetes use Pods instead of managing containers directly?

2. You create a Pod directly (without a Deployment). The Pod crashes at 2 AM. What happens? How would the outcome differ if you had used a Deployment?
3. Explain what happens step by step when you run `kubectl set image deployment/myapp myapp=myapp:v2`. Which components are involved?
4. You have a Deployment with `replicas: 3` and a Service with `selector: app: myapp`. You notice the Service is not routing traffic to any Pods. What is the most likely cause? How would you diagnose the issue?
5. When would you use a ClusterIP Service versus a NodePort Service? Give a real-world example of each.

Exercises

Exercise 1: Pod Lifecycle

1. Create a Pod YAML for an Nginx container (you can use the one from this chapter)
2. Apply it and verify it is running
3. View the Pod's logs
4. Run `kubectl exec` to get a shell inside the Pod
5. Delete the Pod and verify it does not come back

Exercise 2: Deployment and Scaling

1. Create a Deployment YAML with `replicas: 2` using the `httpd:2.4` (Apache) image on port 80
2. Apply it and verify 2 Pods are running
3. Scale the Deployment to 5 replicas using `kubectl scale`
4. Delete one of the Pods and watch Kubernetes recreate it
5. Update the image to `httpd:2.4-alpine` and watch the rolling update
6. Roll back to the previous version

Exercise 3: Complete Application

1. Create a Deployment for Nginx with 3 replicas
2. Create a NodePort Service for the Deployment on port 30080
3. Access the application from your browser (use `minikube service` or `curl localhost:30080`)

4. Scale the Deployment to 1 replica — does the Service still work?
5. Scale back to 3 replicas
6. Clean up all resources using `kubectl delete -f`

What Is Next?

You now know how to deploy applications to Kubernetes with Deployments and expose them with Services. But real applications need more than just containers — they need configuration (database URLs, feature flags), secrets (passwords, API keys), and persistent storage (so data survives Pod restarts).

In the next chapter, you will learn about ConfigMaps (for configuration), Secrets (for sensitive data), and PersistentVolumes (for storage). These three resources turn simple containers into production-ready applications.

Chapter 29: ConfigMaps, Secrets, and Persistent Volumes

What You Will Learn

By the end of this chapter, you will be able to:

- Explain what a ConfigMap is and why you need one
- Create ConfigMaps from literal values, from files, and from YAML manifests
- Mount ConfigMaps as environment variables and as files inside a container
- Explain what a Secret is and how it differs from a ConfigMap
- Create Secrets and understand base64 encoding
- Mount Secrets as environment variables and as files
- Explain why Kubernetes Secrets are not truly secret by default
- Understand PersistentVolumes (PV) and PersistentVolumeClaims (PVC)
- Create a PVC and mount it in a Pod to store persistent data
- Build a complete application with ConfigMap, Secret, and PVC

Why This Chapter Matters

In the previous chapter, you deployed Nginx to Kubernetes with a Deployment and Service. That works for a simple web server. But real applications need more:

- Configuration. Your application needs to know the database URL, the log level, and the feature flags. These settings change between environments (development, staging, production). You do not want to rebuild the Docker image every time a setting changes.
- Secrets. Your application needs passwords, API keys, and TLS certificates. These are sensitive. You cannot put them in your Docker image or in plain-text configuration files

that end up in Git.
- Persistent storage. When a Pod is deleted and recreated, all data inside the container is lost. If your database runs in a Pod and the Pod restarts, all your data is gone. You need storage that survives Pod restarts.

ConfigMaps, Secrets, and PersistentVolumes solve these three problems. Every production Kubernetes application uses at least one of them. Most use all three.

ConfigMaps: Externalized Configuration

What Is a ConfigMap?

A ConfigMap is a Kubernetes resource that stores configuration data as key-value pairs. Think of it as a settings file that lives outside your application.

In traditional software development, configuration is often baked into the application:

```
Without ConfigMap:

  ┌──────────────────────────────┐
  │  Docker Image                │
  │  ┌────────────────────────┐  │
  │  │  Application Code      │  │
  │  │  + config.json         │  │
  │  │  + database_url        │  │
  │  │  + log_level           │  │
  │  └────────────────────────┘  │
  └──────────────────────────────┘

  Problem: To change a setting, you must
  rebuild the image and redeploy!
```

With a ConfigMap, configuration lives separately:

```
With ConfigMap:

  ┌────────────────────┐     ┌──────────────────┐
  │  Docker Image      │     │  ConfigMap       │
  │  ┌──────────────┐  │     │                  │
  │  │  Application │◄─────────│  database_url=...  │
  │  │  Code        │  │     │  log_level=debug   │
  │  │  (no config) │  │     │  feature_x=true    │
  │  └──────────────┘  │     └──────────────────┘
  └────────────────────┘
```

```
Benefit: Change settings without rebuilding
the image or restarting the container!
```

Why Use ConfigMaps?

1. Separate configuration from code. This is a fundamental best practice in software engineering. Your Docker image should contain only the application code. Configuration should be injected at runtime.
2. Different settings per environment. The same Docker image can run in development with debug logging and in production with error-only logging. You just change the ConfigMap.
3. No rebuilds for config changes. Changing a database URL or feature flag does not require building a new Docker image.

Creating a ConfigMap from Literal Values

The simplest way to create a ConfigMap is from the command line:

```bash
kubectl create configmap app-config \
  --from-literal=DATABASE_HOST=postgres \
  --from-literal=DATABASE_PORT=5432 \
  --from-literal=LOG_LEVEL=debug
```

```
configmap/app-config created
```

Let us break this down:

- `kubectl create configmap` — Create a new ConfigMap resource.
- `app-config` — The name of the ConfigMap.
- `--from-literal=KEY=VALUE` — Add a key-value pair. You can add as many as you need.

Verify the ConfigMap:

```bash
kubectl get configmaps
```

```
NAME               DATA   AGE
app-config         3      10s
kube-root-ca.crt   1      1h
```

View the ConfigMap contents:

```bash
kubectl describe configmap app-config
```

```
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
DATABASE_HOST:
----
postgres
DATABASE_PORT:
----
5432
LOG_LEVEL:
----
debug

Events:  <none>
```

You can see all three key-value pairs stored in the ConfigMap.

Creating a ConfigMap from a File

You can also create a ConfigMap from an existing file. This is useful when you have a configuration file like `app.properties` or `config.json`.

Create a file called `app.properties`:

```
# app.properties
database.host=postgres
database.port=5432
database.name=myapp
log.level=debug
feature.new_ui=true
```

Create the ConfigMap from this file:

```bash
kubectl create configmap app-file-config --from-file=app.properties
```

```
configmap/app-file-config created
```

The entire file becomes a single entry in the ConfigMap, with the filename as the key and the file contents as the value.

```bash
kubectl describe configmap app-file-config
```

```
Name:         app-file-config
Namespace:    default

Data
====
app.properties:
----
# app.properties
database.host=postgres
database.port=5432
database.name=myapp
log.level=debug
feature.new_ui=true

Events:  <none>
```

Creating a ConfigMap from YAML

The declarative approach (recommended for production):

```yaml
# configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DATABASE_HOST: postgres
  DATABASE_PORT: "5432"
  LOG_LEVEL: debug
  APP_NAME: my-application
```

Let us break this down:

Line	Meaning
`apiVersion: v1`	ConfigMaps are part of the core API.
`kind: ConfigMap`	We are creating a ConfigMap.
`metadata: name: app-config`	The name of the ConfigMap.

`data:`	The key-value pairs.
`DATABASE_HOST: postgres`	A configuration entry. The key is `DATABASE_HOST`, the value is `postgres`.
`DATABASE_PORT: "5432"`	Note the quotes around `5432`. YAML treats unquoted numbers as integers, but ConfigMap values must be strings. Always quote numbers.

Apply it:

```bash
kubectl apply -f configmap.yaml
```

```
configmap/app-config created
```

Using ConfigMaps as Environment Variables

The most common way to use a ConfigMap is to inject its values as environment variables into your container.

```yaml
# pod-with-configmap-env.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-config
spec:
  containers:
    - name: myapp
      image: busybox
      command: ["sh", "-c", "echo DB=$DATABASE_HOST:$DATABASE_PORT LOG=$LOG_LEVEL && sl...
      envFrom:
        - configMapRef:
            name: app-config
```

Line	Meaning
`envFrom:`	Load environment variables from an external source.
`configMapRef:`	The source is a ConfigMap.
`name: app-config`	Use the ConfigMap named "app-config".

This injects all key-value pairs from the ConfigMap as environment variables. The container can access them like normal environment variables.

```bash
kubectl apply -f pod-with-configmap-env.yaml
```

```bash
kubectl logs app-with-config
```

```
DB=postgres:5432 LOG=debug
```

The application received the configuration values as environment variables.

Injecting specific values (instead of all):

If you only want some values from the ConfigMap, use `env` with `valueFrom`:

```yaml
# pod-with-selective-env.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-selective-config
spec:
  containers:
    - name: myapp
      image: busybox
      command: ["sh", "-c", "echo Host=$DB_HOST Level=$LOG && sleep 3600"]
      env:
        - name: DB_HOST
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: DATABASE_HOST
        - name: LOG
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: LOG_LEVEL
```

This gives you more control. You can rename the environment variables (the ConfigMap key is `DATABASE_HOST`, but inside the container it becomes `DB_HOST`) and only inject what you need.

Mounting ConfigMaps as Files

Sometimes your application reads configuration from a file instead of environment variables. You can mount a ConfigMap as a file inside the container.

```yaml
# pod-with-configmap-volume.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-config-file
spec:
  containers:
    - name: myapp
      image: busybox
      command: ["sh", "-c", "cat /etc/config/DATABASE_HOST && sleep 3600"]
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-config
```

Let us break down the volume parts:

Section	Meaning
`volumes:`	Define a volume. A volume is a directory that is accessible to containers.
`name: config-volume`	Give this volume a name.
`configMap: name: app-config`	The volume's content comes from the ConfigMap "app-config".
`volumeMounts:`	Attach the volume to the container.
`name: config-volume`	Which volume to mount (matches the name above).
`mountPath: /etc/config`	Where to mount it inside the container.

Each key in the ConfigMap becomes a file inside `/etc/config/`:

```
/etc/config/
├── DATABASE_HOST    (contains "postgres")
├── DATABASE_PORT    (contains "5432")
├── LOG_LEVEL        (contains "debug")
└── APP_NAME         (contains "my-application")
```

```
ConfigMap as Volume:

  ┌──────────────────────────────┐
  │  Pod                         │
```

```
│   ┌──────────────────────────┐   │
│   │  Container               │   │
│   │                          │   │
│   │  /etc/config/            │   │
│   │    ├── DATABASE_HOST     │   │
│   │    ├── DATABASE_PORT     │◄──┼── ConfigMap
│   │    ├── LOG_LEVEL         │   │   "app-config"
│   │    └── APP_NAME          │   │
│   └──────────────────────────┘   │
└──────────────────────────────────┘
```

Secrets: Sensitive Data

What Is a Secret?

A Secret is like a ConfigMap, but designed for sensitive data — passwords, API keys, TLS certificates, and database credentials.

Think of it this way: a ConfigMap is like a bulletin board in your office. Anyone can see it, and that is fine because it contains general information. A Secret is like a locked drawer — it contains sensitive information that should be handled with care.

```
ConfigMap vs. Secret:

  ConfigMap                     Secret
  ┌────────────────────┐        ┌────────────────────┐
  │ LOG_LEVEL=debug    │        │ DB_PASSWORD=s3cr3t│
  │ APP_NAME=myapp     │        │ API_KEY=abc123xyz  │
  │ FEATURE_X=true     │        │ TLS_CERT=...       │
  │                    │        │                    │
  │ Non-sensitive      │        │ Sensitive!         │
  │ Plain text in etcd │        │ Base64 in etcd     │
  └────────────────────┘        └────────────────────┘
```

Creating a Secret from the Command Line

```bash
kubectl create secret generic db-credentials \
  --from-literal=DB_USERNAME=admin \
  --from-literal=DB_PASSWORD=supersecretpassword123
```

```
secret/db-credentials created
```

Let us break this down:

- `kubectl create secret generic` — Create a new Secret of type "generic" (the most common type for arbitrary key-value pairs).
- `db-credentials` — The name of the Secret.
- `--from-literal=KEY=VALUE` — Add a key-value pair.

Verify the Secret:

```bash
kubectl get secrets
```

```
NAME              TYPE     DATA   AGE
db-credentials    Opaque   2      10s
```

View the Secret details:

```bash
kubectl describe secret db-credentials
```

```
Name:         db-credentials
Namespace:    default
Type:         Opaque

Data
====
DB_PASSWORD:  23 bytes
DB_USERNAME:  5 bytes
```

Notice that `kubectl describe` does not show the Secret values — it only shows their sizes. This is a safety feature to prevent accidental exposure in terminal output.

Base64 Encoding

Kubernetes stores Secret values as base64-encoded strings. Base64 is not encryption — it is just an encoding format that converts any data (including binary data like TLS certificates) into a text-safe format.

What is base64? Think of base64 as writing a message in pig latin. It looks different, but anyone who knows the rules can read it. Base64 is not secure. It is just a way to represent binary data as text.

You can see the base64-encoded values:

```bash
kubectl get secret db-credentials -o yaml
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_PASSWORD: c3VwZXJzZWNyZXRwYXNzd29yZDEyMw==
  DB_USERNAME: YWRtaW4=
```

You can decode these values easily:

```bash
echo "c3VwZXJzZWNyZXRwYXNzd29yZDEyMw==" | base64 --decode
```

```
supersecretpassword123
```

Important: Base64 encoding is NOT encryption. Anyone with access to the cluster can decode Secrets. This is a common source of confusion — see the section on "Why Secrets Are Not Truly Secret" below.

Creating a Secret from YAML

```yaml
# secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_USERNAME: YWRtaW4=
  DB_PASSWORD: c3VwZXJzZWNyZXRwYXNzd29yZDEyMw==
```

The values under `data:` must be base64-encoded. To encode a value:

```bash
echo -n "admin" | base64
```

```
YWRtaW4=
```

The `-n` flag prevents echo from adding a newline character at the end. This is important — without `-n`, your encoded value will include a hidden newline, and your password will not match.

Using `stringData` (easier):

If you do not want to manually encode values, use `stringData` instead of `data`:

```yaml
# secret-plain.yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: supersecretpassword123
```

With `stringData`, you write plain text. Kubernetes encodes it for you automatically. This is more convenient but has a risk: if you store this YAML file in Git, the passwords are visible in plain text. Never commit Secret YAML files with `stringData` to version control.

Using Secrets as Environment Variables

```yaml
# pod-with-secret-env.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-secrets
spec:
  containers:
    - name: myapp
      image: busybox
      command: ["sh", "-c", "echo Connected as $DB_USERNAME && sleep 3600"]
      envFrom:
        - secretRef:
            name: db-credentials
```

This works exactly like ConfigMap environment variables. All key-value pairs from the Secret become environment variables inside the container.

Selective environment variables:

```yaml
env:
  - name: DATABASE_PASSWORD
```

```
    valueFrom:
      secretKeyRef:
        name: db-credentials
        key: DB_PASSWORD
```

Mounting Secrets as Files

```yaml
# pod-with-secret-volume.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-secret-files
spec:
  containers:
    - name: myapp
      image: busybox
      command: ["sh", "-c", "cat /etc/secrets/DB_PASSWORD && sleep 3600"]
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: db-credentials
```

Each key becomes a file:

```
/etc/secrets/
├── DB_USERNAME    (contains "admin")
└── DB_PASSWORD    (contains "supersecretpassword123")
```

Notice the `readOnly: true` on the volume mount. This is a best practice for Secrets — it prevents the container from accidentally modifying the secret files.

Why Secrets Are Not Truly Secret by Default

This is an important warning that trips up many beginners:

Kubernetes Secrets are only base64-encoded, not encrypted. By default:

1. Secrets are stored unencrypted in etcd (the cluster database).
2. Anyone with API access to the cluster can read Secrets.
3. Secrets are passed to Pods as plain text (after decoding).
4. Secrets can be seen in Pod environment variables via `kubectl exec`.

```
The Security Reality:

   ┌──────────────────┐
   │   etcd           │
   │                  │
   │  password:       │◄── Stored as base64 (NOT encrypted)
   │  YWJjMTIz        │    Anyone with cluster access can read it
   └──────────────────┘
           │
           │
           ▼

   ┌──────────────────┐
   │   Pod            │
   │                  │
   │  $DB_PASSWORD    │◄── Available as plain text
   │  = abc123        │    Can be seen with kubectl exec
   └──────────────────┘
```

How to make Secrets more secure:

1. Enable encryption at rest — Configure Kubernetes to encrypt Secrets in etcd.
2. Use RBAC — Restrict who can read Secrets using Role-Based Access Control.
3. Use external secret managers — Tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault provide real encryption and access control. They integrate with Kubernetes through operators.
4. Never commit Secrets to Git — Use tools like Sealed Secrets or External Secrets Operator to safely manage Secrets in version control.

For learning purposes, Kubernetes Secrets are fine. But for production, take the extra steps to secure them properly.

Persistent Volumes: Storage That Survives

The Problem: Containers Are Ephemeral

By default, all data inside a container is lost when the container stops. This is the same behavior you learned about in the Docker chapters.

In Kubernetes, the problem is worse because Pods are constantly being created and destroyed — during scaling, updates, or self-healing. If your database is running in a Pod and the Pod restarts, all your data is gone.

```
Without Persistent Storage:

  Pod (running database)
  ┌──────────────────────┐
  │  PostgreSQL          │
  │  ┌──────────────┐    │
  │  │  Data:       │    │
  │  │  users       │    │
  │  │  orders      │    │
  │  │  products    │    │
  │  └──────────────┘    │
  └──────────────────────┘
          │
          ▼ Pod restarts
  ┌──────────────────────┐
  │  PostgreSQL          │
  │  ┌──────────────┐    │
  │  │  Data:       │    │
  │  │  (empty!)    │    │
  │  │              │    │
  │  └──────────────┘    │
  └──────────────────────┘
  All data is gone!
```

The Solution: PersistentVolumes and PersistentVolumeClaims

Kubernetes solves this with two resources:

1. PersistentVolume (PV) — A piece of storage in the cluster. Think of it as a hard drive available for use. PVs are created by administrators or automatically by the storage system.
2. PersistentVolumeClaim (PVC) — A request for storage. Think of it as a storage request form. Your Pod says "I need 1 GB of storage," and Kubernetes finds a matching PV and assigns it.

The Landlord Analogy

Think of storage like renting an apartment:

- The PersistentVolume is the apartment itself — it exists, has a certain size, and is available for rent.
- The PersistentVolumeClaim is your rental application — you specify how much space you need and what type of apartment you want.
- Kubernetes is the real estate agent — it matches your application (PVC) with an available apartment (PV).

```
Storage Flow:

  1. PV exists (storage is available)
  ┌──────────────────┐
  │  PersistentVolume  │
  │  Size: 5Gi         │
  │  Status: Available  │
  └──────────────────┘

  2. Pod creates a PVC (requests storage)
  ┌──────────────────┐
  │  PVC               │
  │  Request: 1Gi      │
  │  Status: Pending    │
  └──────────────────┘

  3. Kubernetes binds PVC to PV
  ┌──────────────────┐     ┌──────────────────┐
  │  PVC               │──►│  PV                │
  │  Request: 1Gi      │   │  Size: 5Gi         │
  │  Status: Bound     │   │  Status: Bound     │
  └──────────────────┘     └──────────────────┘

  4. Pod uses the PVC
  ┌──────────────────────────────┐
  │  Pod                          │
  │  ┌──────────────┐             │
  │  │  Container    │             │
  │  │  /data ──────►│── PVC ──► PV │
  │  └──────────────┘             │
  └──────────────────────────────┘

  Data survives Pod restarts!
```

Dynamic Provisioning

In modern Kubernetes (including Minikube), you usually do not need to create PVs manually. Kubernetes can dynamically provision storage — when you create a PVC, Kubernetes automatically creates a matching PV. This is like an apartment building that constructs a new unit whenever someone submits a rental application.

Minikube comes with a default StorageClass that handles dynamic provisioning automatically. You only need to create a PVC.

Creating a PersistentVolumeClaim

```yaml
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Let us break this down:

Line	Meaning
`kind: PersistentVolumeClaim`	We are requesting storage.
`name: my-data-pvc`	The name of this storage claim.
`accessModes:`	How the storage can be accessed.
`ReadWriteOnce`	One node can read and write. This is the most common mode.
`resources: requests: storage: 1Gi`	We want 1 gigabyte of storage.

Access modes explained:

Mode	Abbreviation	Meaning
ReadWriteOnce	RWO	Can be mounted as read-write by a single node
ReadOnlyMany	ROX	Can be mounted as read-only by multiple nodes
ReadWriteMany	RWX	Can be mounted as read-write by multiple nodes

`ReadWriteOnce` is the most commonly used mode. It works with most storage types.

Apply the PVC:

```bash
kubectl apply -f pvc.yaml
```

```
persistentvolumeclaim/my-data-pvc created
```

Check the PVC status:

```bash
kubectl get pvc
```

```
NAME          STATUS   VOLUME                                     CAPACITY   ACCESS MOD...
my-data-pvc   Bound    pvc-a1b2c3d4-e5f6-7890-abcd-ef1234567890   1Gi        RWO       ...
```

The status is `Bound`, which means Kubernetes created a PV and linked it to our PVC. The storage is ready to use.

Mounting a PVC in a Pod

```yaml
# pod-with-pvc.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-storage
spec:
  containers:
    - name: myapp
      image: busybox
      command: ["sh", "-c", "echo 'Hello from persistent storage!' > /data/hello.txt &&...
      volumeMounts:
        - name: data-volume
          mountPath: /data
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: my-data-pvc
```

Section	Meaning
`volumes:`	Define a volume for the Pod.
`persistentVolumeClaim: claimName: my-data-pvc`	This volume uses storage from our PVC.
`volumeMounts: mountPath: /data`	Mount the volume at `/data` inside the container.

```bash
kubectl apply -f pod-with-pvc.yaml
```

```bash
kubectl logs app-with-storage
```

```
Hello from persistent storage!
```

Now delete the Pod and recreate it:

```bash
kubectl delete pod app-with-storage
kubectl apply -f pod-with-pvc.yaml
```

```bash
kubectl exec app-with-storage -- cat /data/hello.txt
```

```
Hello from persistent storage!
```

The data survived the Pod deletion! The PVC kept the storage alive, and when the new Pod mounted the same PVC, the data was still there.

```
Data Persistence:

  Pod v1 writes data ──► PVC ──► Disk
       │
       ▼ Pod deleted

  Pod v2 starts ──► PVC ──► Disk (data still there!)
       │
       ▼ reads data

  "Hello from persistent storage!"
```

Complete Example: App with ConfigMap, Secret, and PVC

Let us build a complete example that ties everything together. We will create:

1. A ConfigMap for application settings
2. A Secret for the database password
3. A PVC for persistent data storage

4. A Pod that uses all three

Step 1: ConfigMap for Application Settings

```yaml
# app-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-config
data:
  APP_NAME: "My Web App"
  APP_ENV: development
  LOG_LEVEL: debug
  DB_HOST: localhost
  DB_PORT: "5432"
  DB_NAME: webapp_db
```

```bash
kubectl apply -f app-configmap.yaml
```

```
configmap/webapp-config created
```

Step 2: Secret for Database Password

```yaml
# app-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: webapp-secret
type: Opaque
stringData:
  DB_USERNAME: webapp_user
  DB_PASSWORD: MyS3cur3P@ssw0rd
```

```bash
kubectl apply -f app-secret.yaml
```

```
secret/webapp-secret created
```

Step 3: PVC for Data Storage

```yaml
# app-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: webapp-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

```bash
kubectl apply -f app-pvc.yaml
```

```
persistentvolumeclaim/webapp-data created
```

Step 4: Pod Using All Three

```yaml
# app-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: webapp
  labels:
    app: webapp
spec:
  containers:
    - name: webapp
      image: busybox
      command:
        - sh
        - -c
        - |
          echo "=== Application Configuration ==="
          echo "App: $APP_NAME"
          echo "Environment: $APP_ENV"
          echo "Log Level: $LOG_LEVEL"
          echo "Database: $DB_HOST:$DB_PORT/$DB_NAME"
          echo "DB User: $DB_USERNAME"
          echo "DB Pass: [hidden]"
          echo ""
          echo "=== Writing to persistent storage ==="
          echo "App started at $(date)" >> /data/app.log
          cat /data/app.log
          echo ""
          echo "=== Config file from volume ==="
```

```
          ls /etc/app-config/
          sleep 3600
      envFrom:
        - configMapRef:
            name: webapp-config
        - secretRef:
            name: webapp-secret
      volumeMounts:
        - name: data-volume
          mountPath: /data
        - name: config-volume
          mountPath: /etc/app-config
          readOnly: true
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: webapp-data
    - name: config-volume
      configMap:
        name: webapp-config
```

```bash
kubectl apply -f app-pod.yaml
```

```bash
kubectl logs webapp
```

```
=== Application Configuration ===
App: My Web App
Environment: development
Log Level: debug
Database: localhost:5432/webapp_db
DB User: webapp_user
DB Pass: [hidden]

=== Writing to persistent storage ===
App started at Mon Jan 15 16:00:00 UTC 2024

=== Config file from volume ===
APP_ENV
APP_NAME
DB_HOST
DB_NAME
DB_PORT
LOG_LEVEL
```

Let us look at what this Pod uses:

```
Complete Application:

  ┌──────────────────────────────────────────┐
  │  Pod "webapp"                            │
  │  ┌──────────────────────────────────┐    │
  │  │  Container "webapp"              │    │
  │  │                                  │    │
  │  │  Environment Variables:          │    │
  │  │  ├── APP_NAME ◄── ConfigMap      │    │
  │  │  ├── LOG_LEVEL ◄── ConfigMap     │    │
  │  │  ├── DB_HOST ◄── ConfigMap       │    │
  │  │  ├── DB_USERNAME ◄── Secret      │    │
  │  │  └── DB_PASSWORD ◄── Secret      │    │
  │  │                                  │    │
  │  │  Volume Mounts:                  │    │
  │  │  ├── /data ◄── PVC (persistent)  │    │
  │  │  └── /etc/app-config ◄── ConfigMap│    │
  │  └──────────────────────────────────┘    │
  └──────────────────────────────────────────┘
```

Now delete the Pod and recreate it:

```bash
kubectl delete pod webapp
kubectl apply -f app-pod.yaml
```

```bash
kubectl logs webapp
```

```
=== Application Configuration ===
App: My Web App
Environment: development
...

=== Writing to persistent storage ===
App started at Mon Jan 15 16:00:00 UTC 2024
App started at Mon Jan 15 16:05:00 UTC 2024
...
```

Notice that the app.log now has two entries — the data from the first run survived! The ConfigMap and Secret values are also correctly injected.

Clean Up

```bash
kubectl delete pod webapp
kubectl delete configmap webapp-config
```

```
kubectl delete secret webapp-secret
kubectl delete pvc webapp-data
```

Common Mistakes

1. Forgetting to quote numbers in ConfigMaps. YAML treats `5432` as an integer, but ConfigMap values must be strings. Always write `"5432"` with quotes.
2. Committing Secret YAML files to Git. Never store Secret manifests with `stringData` (or decoded `data`) in version control. Use tools like Sealed Secrets or External Secrets Operator for production.
3. Thinking Secrets are encrypted. Secrets are base64-encoded, not encrypted. Anyone with cluster access can decode them. For real security, enable encryption at rest and use RBAC.
4. Forgetting `-n` when encoding base64. When creating base64 values manually, always use `echo -n "value" | base64`. Without `-n`, a newline character is included, which corrupts your Secret value.
5. Deleting a PVC that is still in use. If a Pod is using a PVC, deleting the PVC puts it in a "Terminating" state. It will not actually be deleted until all Pods stop using it. Delete the Pod first, then the PVC.
6. Mounting a volume over an existing directory. When you mount a volume at `/etc/config`, any existing files in that directory are hidden. Be careful not to overwrite important system directories.
7. Using ReadWriteMany when you only need ReadWriteOnce. Not all storage backends support ReadWriteMany. Use `ReadWriteOnce` unless you specifically need multiple nodes to write to the same volume.

Best Practices

1. Always use ConfigMaps for non-sensitive configuration. Do not hardcode values in your Docker image. Make your images configurable through environment variables or config files.
2. Use Secrets for all sensitive data. Even if you think "it is just a development password," build the habit of using Secrets. It makes transitioning to production easier.

3. Use `stringData` during development, `data` in production. `stringData` is more convenient for writing YAML by hand. But in production, use a secret management tool that handles encoding.
4. Always set `readOnly: true` on Secret volume mounts. This prevents the container from accidentally modifying the secret files.
5. Use meaningful names for PVCs. Names like `postgres-data` or `uploads-storage` make it clear what the storage is for. Names like `pvc-1` do not.
6. Always specify storage size in PVCs. Even if dynamic provisioning adjusts the size, specifying a size documents your intent and helps with capacity planning.

Quick Summary

ConfigMaps store non-sensitive configuration as key-value pairs and can be injected into Pods as environment variables or mounted as files. Secrets store sensitive data like passwords and API keys using base64 encoding (not encryption). PersistentVolumeClaims request storage from the cluster, and the claimed storage survives Pod restarts. Together, these three resources let you separate configuration, secrets, and data from your application code, following best practices for containerized applications.

Key Points

- ConfigMaps store non-sensitive configuration data as key-value pairs
- ConfigMaps can be created from literals, files, or YAML manifests
- ConfigMaps can be injected as environment variables (`envFrom`) or mounted as files (volumes)
- Secrets store sensitive data (passwords, API keys, certificates)
- Secrets are base64-encoded, not encrypted — they are not truly secret by default
- Secrets can be injected as environment variables or mounted as files (with `readOnly: true`)
- Use `echo -n "value" | base64` to encode values (the `-n` prevents a trailing newline)
- PersistentVolumeClaims (PVC) request storage that survives Pod restarts
- `ReadWriteOnce` is the most common access mode for PVCs
- Minikube supports dynamic provisioning — PVs are created automatically when you create a PVC

- Never commit Secret YAML files with plain-text values to Git

---

Practice Questions

1. What is the difference between a ConfigMap and a Secret? When would you use each one?
2. You create a Secret with `stringData: PASSWORD: abc123` and apply it. Then you run `kubectl get secret -o yaml`. Will you see `abc123` in the output? Why or why not?
3. Explain the relationship between a PersistentVolume and a PersistentVolumeClaim using a real-life analogy.
4. Your application stores uploaded files at `/uploads` inside the container. After every Pod restart, the files disappear. How would you fix this?
5. A colleague says "Our passwords are safe because we store them in Kubernetes Secrets." Is this statement accurate? What additional steps would you recommend?

Exercises

Exercise 1: ConfigMap Practice

1. Create a ConfigMap called `game-config` with these values:
    - `GAME_NAME: Space Invaders`
    - `MAX_PLAYERS: "4"`
    - `DIFFICULTY: medium`
2. Create a Pod that uses this ConfigMap as environment variables and prints all three values
3. Modify the ConfigMap to change `DIFFICULTY` to `hard`
4. Delete and recreate the Pod — does it pick up the new value?

Exercise 2: Secret Handling

1. Encode the string `my-secret-password` in base64 using the command line
2. Create a Secret YAML file using the encoded value
3. Create a Pod that mounts the Secret as a file at `/etc/secrets/password`
4. Verify the file contents using `kubectl exec`

5. Delete the Secret and recreate it with `stringData` instead — verify the Pod still works the same way

Exercise 3: Persistent Storage

1. Create a PVC requesting 200Mi of storage
2. Create a Pod that mounts this PVC at `/data`
3. Write a file to `/data/test.txt` using `kubectl exec`
4. Delete the Pod (not the PVC)
5. Create a new Pod mounting the same PVC
6. Verify that `/data/test.txt` still exists with the correct content

What Is Next?

You now have all the building blocks: Deployments to manage Pods, Services to expose them, ConfigMaps for configuration, Secrets for sensitive data, and PersistentVolumes for storage.

In the next chapter, you will put everything together to deploy a real application — a Node.js API backed by PostgreSQL — to Kubernetes. You will create namespaces, ConfigMaps, Secrets, PVCs, Deployments, and Services, all working together as a complete system. This is the capstone of your Kubernetes journey.

Chapter 30: Deploying a Real Application to Kubernetes

What You Will Learn

By the end of this chapter, you will be able to:

- Plan and deploy a multi-component application to Kubernetes
- Build a Docker image and make it available to your cluster
- Create a Kubernetes namespace for your project
- Configure an application using ConfigMaps and Secrets
- Set up persistent storage for a database
- Deploy PostgreSQL as a backend database with a ClusterIP Service
- Deploy a Node.js API with a NodePort Service
- Inspect all resources running in a namespace
- Test your application end to end with curl
- Scale your application by adding more replicas
- Perform a rolling update to deploy a new version
- Roll back a deployment when something goes wrong
- Understand what Ingress is and why you would use it
- Monitor resource usage with kubectl top

Why This Chapter Matters

In the previous chapters, you learned about Pods, Deployments, Services, ConfigMaps, Secrets, and PersistentVolumes — each one individually. That is like learning individual ingredients in cooking. You know what flour is. You know what eggs are. You know what

sugar does.

But knowing ingredients is different from baking a cake. This chapter is where you bake the cake.

You will deploy a complete, real-world application: a Node.js REST API backed by a PostgreSQL database. You will use every Kubernetes concept from the previous chapters, all working together. By the end, you will have a fully functional application running in your local Kubernetes cluster, and you will have the confidence to deploy applications to any Kubernetes environment.

The Application Architecture

Here is what we are building:

```
Application Architecture:

  Your Browser / curl
        │
        │ HTTP request (port 30001)
        ▼
  ┌──────────────────────────────────────────────────┐
  │  Kubernetes Cluster                              │
  │  Namespace: myapp                                │
  │                                                  │
  │   ┌──────────────────────────────────────────┐   │
  │   │  NodePort Service (port 30001)           │   │
  │   │  "api-service"                           │   │
  │   └──────────────────────────────────────────┘   │
  │                    │                             │
  │                    │                             │
  │          ┌─────────┼─────────┐                   │
  │          ▼         ▼         ▼                   │
  │      ┌────────┐ ┌────────┐ ┌────────┐           │
  │      │ API Pod │ │ API Pod │ │ API Pod │           │
  │      │ (v1.0)  │ │ (v1.0)  │ │ (v1.0)  │           │
  │      └────────┘ └────────┘ └────────┘           │
  │          │         │         │                   │
  │          └─────────┼─────────┘                   │
  │                    │                             │
  │   ┌────────────────▼─────────────────────────┐   │
  │   │  ClusterIP Service                       │   │
  │   │  "postgres-service"                      │   │
  │   └──────────────────────────────────────────┘   │
  │                    │                             │
  │                ┌───▼────┐                        │
  │                │ Postgres │                       │
  │                │ Pod      │                       │
```

```
│                │    │       │                    │
│                │    ▼       │                    │
│                │ ┌──────┐   │                    │
│                │ │ PVC  │   │                    │
│                │ │ Data │   │                    │
│                │ └──────┘   │                    │
│                └────────────┘                    │
│                                                  │
│  ConfigMap: app-config (DB host, port, name)     │
│  Secret: db-secret (DB password)                 │
└──────────────────────────────────────────────────┘
```

The components:

Component	Purpose
Namespace	Isolates our app from other workloads in the cluster
ConfigMap	Stores application configuration (database host, port, name)
Secret	Stores the database password securely
PVC	Provides persistent storage for PostgreSQL data
PostgreSQL Deployment	Runs the database
PostgreSQL ClusterIP Service	Lets the API Pods reach the database (internal only)
API Deployment	Runs 3 replicas of the Node.js API
API NodePort Service	Exposes the API to the outside world

The Application Code

Before deploying to Kubernetes, we need a simple Node.js application. Here is a minimal REST API that connects to PostgreSQL.

Project Structure

```
myapp/
├── server.js
├── package.json
└── Dockerfile
```

server.js

```javascript
// server.js
const http = require('http');
const { Pool } = require('pg');

const PORT = process.env.PORT || 3000;
const APP_VERSION = process.env.APP_VERSION || '1.0.0';

// Database connection using environment variables
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT || '5432'),
  database: process.env.DB_NAME || 'myapp',
  user: process.env.DB_USERNAME || 'postgres',
  password: process.env.DB_PASSWORD || 'password',
});

// Initialize database table
async function initDB() {
  try {
    await pool.query(`
      CREATE TABLE IF NOT EXISTS messages (
        id SERIAL PRIMARY KEY,
        text VARCHAR(255) NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      )
    `);
    console.log('Database initialized');
  } catch (err) {
    console.error('Database init error:', err.message);
  }
}

const server = http.createServer(async (req, res) => {
  res.setHeader('Content-Type', 'application/json');

  // Health check endpoint
  if (req.url === '/health' && req.method === 'GET') {
    res.writeHead(200);
    res.end(JSON.stringify({
      status: 'healthy',
      version: APP_VERSION,
    }));
    return;
  }

  // Get all messages
  if (req.url === '/messages' && req.method === 'GET') {
    try {
      const result = await pool.query(
```

```
        'SELECT * FROM messages ORDER BY created_at DESC'
      );
      res.writeHead(200);
      res.end(JSON.stringify(result.rows));
    } catch (err) {
      res.writeHead(500);
      res.end(JSON.stringify({ error: err.message }));
    }
    return;
  }

  // Create a message
  if (req.url === '/messages' && req.method === 'POST') {
    let body = '';
    req.on('data', chunk => { body += chunk; });
    req.on('end', async () => {
      try {
        const { text } = JSON.parse(body);
        const result = await pool.query(
          'INSERT INTO messages (text) VALUES ($1) RETURNING *',
          [text]
        );
        res.writeHead(201);
        res.end(JSON.stringify(result.rows[0]));
      } catch (err) {
        res.writeHead(500);
        res.end(JSON.stringify({ error: err.message }));
      }
    });
    return;
  }

  // Default: 404
  res.writeHead(404);
  res.end(JSON.stringify({ error: 'Not found' }));
});

initDB().then(() => {
  server.listen(PORT, () => {
    console.log(`Server v${APP_VERSION} running on port ${PORT}`);
  });
});
```

Let us understand what this application does:

- It is a simple REST API with three endpoints:
  - `GET /health` — Returns the app version and health status
  - `GET /messages` — Lists all messages from the database
  - `POST /messages` — Saves a new message to the database
- It reads all configuration from environment variables (DB_HOST, DB_PORT, etc.)
- It connects to PostgreSQL using the `pg` library

package.json

```json
{
  "name": "myapp",
  "version": "1.0.0",
  "description": "Simple API for Kubernetes demo",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "pg": "^8.11.0"
  }
}
```

Dockerfile

```dockerfile
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY server.js ./

EXPOSE 3000

CMD ["npm", "start"]
```

This Dockerfile follows the best practices from earlier chapters:

- Uses a small base image (`node:20-alpine`)
- Copies `package.json` first for better layer caching
- Installs only production dependencies
- Copies the application code last (so code changes do not trigger a full reinstall)

Step 1: Build and Load the Docker Image

Before Kubernetes can run your application, it needs the Docker image. In a production environment, you would push the image to a registry (Docker Hub, ECR, GCR). For local development with Minikube, we have a shortcut.

Option A: Minikube's Docker Environment

Minikube has its own Docker daemon running inside the cluster. You can point your local Docker client to Minikube's Docker daemon and build images directly there:

```bash
eval $(minikube docker-env)
```

```
# This command sets several environment variables that tell your
# Docker client to connect to Minikube's Docker daemon instead
# of your local Docker daemon.
```

Now build the image:

```bash
docker build -t myapp-api:1.0.0 ./myapp/
```

```
[+] Building 15.2s (10/10) FINISHED
 => [1/5] FROM docker.io/library/node:20-alpine
 => [2/5] WORKDIR /app
 => [3/5] COPY package*.json ./
 => [4/5] RUN npm install --production
 => [5/5] COPY server.js ./
 => exporting to image
 => => naming to docker.io/library/myapp-api:1.0.0
```

Option B: Using minikube image load

If you prefer to build with your regular Docker and then load the image into Minikube:

```bash
docker build -t myapp-api:1.0.0 ./myapp/
minikube image load myapp-api:1.0.0
```

```
# This copies the image from your local Docker into Minikube's
# Docker daemon. It may take a minute for large images.
```

Option C: Docker Desktop Kubernetes

If you are using Docker Desktop's Kubernetes, you do not need to do anything special. Docker Desktop shares images between Docker and Kubernetes automatically. Just build the image normally:

```bash
docker build -t myapp-api:1.0.0 ./myapp/
```

Step 2: Create the Namespace

We will put all our resources in a dedicated namespace called `myapp`. This keeps everything organized and isolated.

```yaml
# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: myapp
```

```bash
kubectl apply -f k8s/namespace.yaml
```

```
namespace/myapp created
```

Set this namespace as the default so we do not have to type `-n myapp` with every command:

```bash
kubectl config set-context --current --namespace=myapp
```

```
Context "minikube" modified.
```

Step 3: Create the ConfigMap

The ConfigMap stores our application's non-sensitive configuration:

```yaml
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: app-config
  namespace: myapp
data:
  DB_HOST: postgres-service
  DB_PORT: "5432"
  DB_NAME: myapp
  APP_VERSION: "1.0.0"
```

Notice that `DB_HOST` is set to `postgres-service`. This is the name of the Kubernetes Service we will create for PostgreSQL. Inside the cluster, Pods can reach each other using Service names as hostnames — Kubernetes' built-in DNS handles the resolution.

```bash
kubectl apply -f k8s/configmap.yaml
```

```
configmap/app-config created
```

Step 4: Create the Secret

The Secret stores the database credentials:

```yaml
# k8s/secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
  namespace: myapp
type: Opaque
stringData:
  DB_USERNAME: appuser
  DB_PASSWORD: K8sD3moP@ss2024
  POSTGRES_USER: appuser
  POSTGRES_PASSWORD: K8sD3moP@ss2024
  POSTGRES_DB: myapp
```

We include both the `DB_*` variables (for our Node.js app) and the `POSTGRES_*` variables (which the official PostgreSQL Docker image uses to initialize the database).

```bash
kubectl apply -f k8s/secret.yaml
```

```
secret/db-secret created
```

Reminder: In a real project, do not commit this file to Git with plain-text passwords. Use Sealed Secrets, External Secrets Operator, or your cloud provider's secret management tool.

Step 5: Create the PVC for Database Storage

PostgreSQL needs persistent storage so data survives Pod restarts:

```yaml
# k8s/postgres-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
  namespace: myapp
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

```bash
kubectl apply -f k8s/postgres-pvc.yaml
```

```
persistentvolumeclaim/postgres-data created
```

Verify it:

```bash
kubectl get pvc
```

```
NAME            STATUS   VOLUME                                     CAPACITY   ACCESS M...
postgres-data   Bound    pvc-abc12345-def6-7890-abcd-ef1234567890   1Gi        RWO     ...
```

Status is `Bound` — Minikube dynamically provisioned a PersistentVolume and linked it to our claim.

Step 6: Deploy PostgreSQL

Now we deploy the database. PostgreSQL runs as a Deployment with 1 replica (databases are typically not scaled horizontally without special tools) and a ClusterIP Service (because only the API Pods need to access it, not the outside world).

PostgreSQL Deployment

```yaml
# k8s/postgres-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: myapp
  labels:
    app: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: db-secret
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
              subPath: pgdata
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-data
```

Let us examine the new parts:

Section	Meaning
`envFrom: secretRef: name: db-secret`	Injects all Secret values as environment variables. PostgreSQL uses `POSTGRES_USER`, `POSTGRES_PASSWORD`, and `POSTGRES_DB` to create the initial database and user.
`mountPath: /var/lib/postgresql/data`	Where PostgreSQL stores its data files.
`subPath: pgdata`	Creates a subdirectory inside the PVC. PostgreSQL requires that its data directory is empty on first start. The `subPath` prevents the PVC's root directory (which may contain a "lost+found" folder) from interfering.
`resources: requests:`	How much CPU and memory the container needs at minimum. The Scheduler uses this to find a node with enough resources.
`resources: limits:`	The maximum CPU and memory the container can use. If it exceeds the memory limit, Kubernetes kills it.

Understanding resource values:

- `250m` means 250 millicores = 0.25 CPU cores (one quarter of a CPU)
- `256Mi` means 256 mebibytes of memory (approximately 268 megabytes)

```bash
kubectl apply -f k8s/postgres-deployment.yaml
```

```
deployment.apps/postgres created
```

PostgreSQL Service

```yaml
# k8s/postgres-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
  namespace: myapp
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

We use ClusterIP because the database should only be accessible from inside the cluster. The API Pods will connect to it using the DNS name `postgres-service` (which is the Service name).

```bash
kubectl apply -f k8s/postgres-service.yaml
```

```
service/postgres-service created
```

Verify PostgreSQL Is Running

```bash
kubectl get pods
```

```
NAME                        READY   STATUS    RESTARTS   AGE
postgres-7f9b8c6d45-x2k9m  1/1     Running   0          30s
```

Check the logs to make sure PostgreSQL initialized successfully:

```bash
kubectl logs deployment/postgres
```

```
PostgreSQL init process complete; ready for start up.
2024-01-15 17:00:00.000 UTC [1] LOG:  starting PostgreSQL 16.1
2024-01-15 17:00:00.000 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2024-01-15 17:00:00.000 UTC [1] LOG:  database system is ready to accept connections
```

PostgreSQL is running and ready.

Step 7: Deploy the Node.js API

API Deployment

```yaml
# k8s/api-deployment.yaml
apiVersion: apps/v1
```

```yaml
kind: Deployment
metadata:
  name: api
  namespace: myapp
  labels:
    app: api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: myapp-api:1.0.0
          imagePullPolicy: Never
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: app-config
            - secretRef:
                name: db-secret
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "200m"
```

Important details:

Line	Meaning
`replicas: 3`	Run 3 copies of the API for load distribution and reliability.
`imagePullPolicy: Never`	Do not try to pull this image from Docker Hub. We built it locally. Without this, Kubernetes would try to download `myapp-api:1.0.0` from the internet and fail.
`envFrom: configMapRef`	Injects ConfigMap values (DB_HOST, DB_PORT, etc.) as environment variables.
`envFrom: secretRef`	Injects Secret values (DB_USERNAME, DB_PASSWORD) as environment variables.

The API container receives all the configuration it needs from the ConfigMap and Secret. It does not contain any hardcoded database URLs or passwords.

```bash
kubectl apply -f k8s/api-deployment.yaml
```

```
deployment.apps/api created
```

API Service

```yaml
# k8s/api-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: api-service
  namespace: myapp
spec:
  type: NodePort
  selector:
    app: api
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30001
```

We use NodePort so you can access the API from your browser. Port 30001 on your node will forward traffic to port 3000 on the API Pods.

```bash
kubectl apply -f k8s/api-service.yaml
```

```
service/api-service created
```

Step 8: Inspect Everything

Now let us see all the resources we created:

```bash
kubectl get all -n myapp
```

```
NAME                            READY   STATUS    RESTARTS   AGE
pod/api-6d8f9b7c54-abc12       1/1     Running   0          1m
pod/api-6d8f9b7c54-def34       1/1     Running   0          1m
pod/api-6d8f9b7c54-ghi56       1/1     Running   0          1m
pod/postgres-7f9b8c6d45-x2k9m  1/1     Running   0          3m

NAME                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/api-service        NodePort    10.96.123.45    <none>        3000:30001/TCP   1m
service/postgres-service   ClusterIP   10.96.67.89     <none>        5432/TCP         3m

NAME                       READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api        3/3     3            3           1m
deployment.apps/postgres   1/1     1            1           3m

NAME                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/api-6d8f9b7c54       3         3         3       1m
replicaset.apps/postgres-7f9b8c6d45  1         1         1       3m
```

Let us also check our other resources:

```bash
kubectl get configmaps -n myapp
```

```
NAME               DATA   AGE
app-config         4      5m
kube-root-ca.crt   1      5m
```

```bash
kubectl get secrets -n myapp
```

```
NAME        TYPE     DATA   AGE
db-secret   Opaque   5      5m
```

```bash
kubectl get pvc -n myapp
```

```
NAME            STATUS   VOLUME                                     CAPACITY   ACCESS M...
postgres-data   Bound    pvc-abc12345-def6-7890-abcd-ef1234567890   1Gi        RWO     ...
```

Everything is running. Let us count what we have:

```
Resources in namespace "myapp":

  Namespace:    1  (myapp)
  ConfigMap:    1  (app-config)
```

```
  Secret:       1  (db-secret)
  PVC:          1  (postgres-data)
  Deployments:  2  (api, postgres)
  Pods:         4  (3 API + 1 PostgreSQL)
  Services:     2  (api-service, postgres-service)
  ReplicaSets:  2  (created automatically by Deployments)
  ─────────────────
  Total:       10  Kubernetes resources
```

Step 9: Test the Application

Get the Application URL

On Minikube:

```bash
minikube service api-service -n myapp --url
```

```
http://192.168.49.2:30001
```

On Docker Desktop:

The URL is `http://localhost:30001`.

Test the Health Endpoint

```bash
curl http://192.168.49.2:30001/health
```

```json
{
  "status": "healthy",
  "version": "1.0.0"
}
```

The API is running and reports itself as healthy with version 1.0.0.

Create a Message

```bash
curl -X POST http://192.168.49.2:30001/messages \
  -H "Content-Type: application/json" \
  -d '{"text": "Hello from Kubernetes!"}'
```

```json
{
  "id": 1,
  "text": "Hello from Kubernetes!",
  "created_at": "2024-01-15T17:10:00.000Z"
}
```

List All Messages

```bash
curl http://192.168.49.2:30001/messages
```

```json
[
  {
    "id": 1,
    "text": "Hello from Kubernetes!",
    "created_at": "2024-01-15T17:10:00.000Z"
  }
]
```

Add More Messages

```bash
curl -X POST http://192.168.49.2:30001/messages \
  -H "Content-Type: application/json" \
  -d '{"text": "Kubernetes is managing my app!"}'

curl -X POST http://192.168.49.2:30001/messages \
  -H "Content-Type: application/json" \
  -d '{"text": "Data persists across Pod restarts!"}'
```

```bash
curl http://192.168.49.2:30001/messages
```

```json
[
  {
    "id": 3,
    "text": "Data persists across Pod restarts!",
    "created_at": "2024-01-15T17:12:00.000Z"
```

```
  },
  {
    "id": 2,
    "text": "Kubernetes is managing my app!",
    "created_at": "2024-01-15T17:11:00.000Z"
  },
  {
    "id": 1,
    "text": "Hello from Kubernetes!",
    "created_at": "2024-01-15T17:10:00.000Z"
  }
]
```

The application is fully functional. Data is stored in PostgreSQL, which uses a PersistentVolume. Even if the PostgreSQL Pod restarts, the data will survive.

Step 10: Scale the Application

Your API is getting popular. Let us add more replicas:

```bash
kubectl scale deployment api -n myapp --replicas=5
```

```
deployment.apps/api scaled
```

```bash
kubectl get pods -n myapp -l app=api
```

```
NAME                    READY   STATUS    RESTARTS   AGE
api-6d8f9b7c54-abc12    1/1     Running   0          10m
api-6d8f9b7c54-def34    1/1     Running   0          10m
api-6d8f9b7c54-ghi56    1/1     Running   0          10m
api-6d8f9b7c54-jkl78    1/1     Running   0          15s
api-6d8f9b7c54-mno90    1/1     Running   0          15s
```

The `-l app=api` flag filters Pods by label, showing only API Pods. Two new Pods started in 15 seconds.

The Service automatically discovers the new Pods (because they have the label `app: api`) and starts routing traffic to them. No configuration change needed.

```
After Scaling:

  api-service (NodePort 30001)
        │
        ├──────┬──────┬──────┬──────┐
        ▼      ▼      ▼      ▼      ▼
  Pod1   Pod2   Pod3   Pod4   Pod5

  Traffic distributed across
  all 5 Pods automatically
```

Scale back down when traffic decreases:

```bash
kubectl scale deployment api -n myapp --replicas=3
```

```
deployment.apps/api scaled
```

Step 11: Rolling Update (Deploy a New Version)

Let us say you have improved your application. Version 2.0.0 adds a timestamp to the health endpoint. In a real project, you would modify the code, rebuild the image, and push it. For this example, we will simulate a rolling update.

Build the New Version

Modify `server.js` to change `APP_VERSION` to `2.0.0` (or set it via the ConfigMap). Then rebuild:

```bash
# If using Minikube docker-env:
docker build -t myapp-api:2.0.0 ./myapp/

# If using minikube image load:
docker build -t myapp-api:2.0.0 ./myapp/
minikube image load myapp-api:2.0.0
```

Update the ConfigMap

```bash
kubectl patch configmap app-config -n myapp \
  --type merge -p '{"data":{"APP_VERSION":"2.0.0"}}'
```

```
configmap/app-config patched
```

Trigger the Rolling Update

```bash
kubectl set image deployment/api -n myapp api=myapp-api:2.0.0
```

```
deployment.apps/api image updated
```

Watch the Rolling Update

```bash
kubectl rollout status deployment/api -n myapp
```

```
Waiting for deployment "api" rollout to finish: 1 out of 3 new replicas
have been updated...
Waiting for deployment "api" rollout to finish: 2 out of 3 new replicas
have been updated...
Waiting for deployment "api" rollout to finish: 1 old replicas are
pending termination...
deployment "api" successfully rolled out
```

Verify the New Version

```bash
curl http://192.168.49.2:30001/health
```

```json
{
  "status": "healthy",
  "version": "2.0.0"
}
```

The application has been updated to version 2.0.0 with zero downtime. During the update, at least 2 of the 3 Pods were always running, so users experienced no interruption.

Check the Rollout History

```bash
kubectl rollout history deployment/api -n myapp
```

```
deployment.apps/api
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

Step 12: Rollback

Something is wrong with version 2.0.0. Users are reporting errors. You need to roll back immediately.

```bash
kubectl rollout undo deployment/api -n myapp
```

```
deployment.apps/api rolled back
```

```bash
kubectl rollout status deployment/api -n myapp
```

```
deployment "api" successfully rolled out
```

```bash
curl http://192.168.49.2:30001/health
```

```json
{
  "status": "healthy",
  "version": "1.0.0"
}
```

Back to version 1.0.0. The rollback performed another rolling update in reverse — replacing version 2.0.0 Pods with version 1.0.0 Pods. The entire process took seconds and happened with zero downtime.

```
Rollback Flow:
```

```
  v2.0.0  v2.0.0  v2.0.0   ◄── Current (broken)
     │
     ▼  kubectl rollout undo
  v1.0.0  v2.0.0  v2.0.0   ◄── Rolling back...
     │
     ▼
  v1.0.0  v1.0.0  v2.0.0   ◄── Almost done...
     │
     ▼
  v1.0.0  v1.0.0  v1.0.0   ◄── Rollback complete!
```

Ingress: A Brief Introduction

So far, you have been accessing your application through NodePort (a high-numbered port like 30001). This works for development, but in production you want users to access your application through a real domain name like `api.mycompany.com` on port 80 (HTTP) or 443 (HTTPS).

This is where Ingress comes in.

What Is Ingress?

An Ingress is a Kubernetes resource that manages external HTTP and HTTPS access to your Services. Think of it as a front desk at a building — visitors tell the front desk which department they want (which Service), and the front desk directs them to the right floor.

```
Without Ingress:

  User ──► NodePort 30001 ──► API Service
  User ──► NodePort 30002 ──► Web Service
  User ──► NodePort 30003 ──► Admin Service

  Ugly URLs: http://mysite.com:30001, :30002, :30003

With Ingress:

  User ──► api.mysite.com    ──► Ingress ──► API Service
  User ──► www.mysite.com    ──► Ingress ──► Web Service
  User ──► admin.mysite.com  ──► Ingress ──► Admin Service

  Clean URLs: http://api.mysite.com (port 80)
```

A Simple Ingress Example

```yaml
# This is for reference — we will not apply this in our local setup
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  namespace: myapp
spec:
  rules:
    - host: api.mysite.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 3000
```

This tells Kubernetes: "When someone visits `api.mysite.com`, send the traffic to the `api-service` Service on port 3000."

Ingress requires an Ingress Controller (like Nginx Ingress Controller or Traefik) to be installed in the cluster. Managed Kubernetes services (EKS, GKE, AKS) provide their own Ingress Controllers. We will not set one up here, but you now know what Ingress is and when you would use it.

Monitoring with kubectl top

Kubernetes can show you how much CPU and memory your Pods are using with the `kubectl top` command.

Enable Metrics (Minikube)

First, enable the metrics server:

```bash
minikube addons enable metrics-server
```

```
▪  metrics-server is an addon maintained by Kubernetes. For any concerns
    contact minikube on GitHub.
▪  The 'metrics-server' addon is enabled
```

Wait about a minute for the metrics server to start collecting data.

View Node Resource Usage

```bash
kubectl top nodes
```

```
NAME       CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
minikube   250m         12%    1200Mi          30%
```

This shows your Minikube node is using 250 millicores (12% of its CPU) and 1200 mebibytes of memory (30% of total).

View Pod Resource Usage

```bash
kubectl top pods -n myapp
```

```
NAME                        CPU(cores)   MEMORY(bytes)
api-6d8f9b7c54-abc12       2m           45Mi
api-6d8f9b7c54-def34       1m           42Mi
api-6d8f9b7c54-ghi56       2m           44Mi
postgres-7f9b8c6d45-x2k9m  5m           120Mi
```

Each API Pod is using about 2 millicores of CPU and 44 megabytes of memory. PostgreSQL is using more — 5 millicores and 120 megabytes. These numbers help you tune the resource requests and limits in your Deployment YAML.

Cleaning Up

When you are done, clean up all resources:

Option 1: Delete the Namespace (removes everything)

```bash
kubectl delete namespace myapp
```

```
namespace "myapp" deleted
```

This deletes the namespace and everything inside it — all Pods, Deployments, Services, ConfigMaps, Secrets, and PVCs. One command to clean up everything.

Option 2: Delete Resources Individually

```bash
kubectl delete -f k8s/api-service.yaml -n myapp
kubectl delete -f k8s/api-deployment.yaml -n myapp
kubectl delete -f k8s/postgres-service.yaml -n myapp
kubectl delete -f k8s/postgres-deployment.yaml -n myapp
kubectl delete -f k8s/postgres-pvc.yaml -n myapp
kubectl delete -f k8s/secret.yaml -n myapp
kubectl delete -f k8s/configmap.yaml -n myapp
kubectl delete -f k8s/namespace.yaml
```

Reset the Default Namespace

```bash
kubectl config set-context --current --namespace=default
```

```
Context "minikube" modified.
```

Congratulations!

You have successfully deployed a real, multi-component application to Kubernetes. Let us recap everything you accomplished:

1. Built a Docker image for a Node.js API
2. Created a Kubernetes namespace to isolate your project
3. Used a ConfigMap to externalize application configuration
4. Used a Secret to protect database credentials
5. Used a PersistentVolumeClaim to give PostgreSQL durable storage
6. Deployed PostgreSQL with a ClusterIP Service for internal access

7. Deployed a Node.js API with 3 replicas and a NodePort Service
8. Tested the application end to end with curl
9. Scaled the API from 3 to 5 replicas and back
10. Performed a rolling update to deploy a new version with zero downtime
11. Rolled back when the new version had issues
12. Learned about Ingress for production-grade URL routing
13. Monitored resource usage with kubectl top

You have gone from knowing nothing about Kubernetes to deploying and managing a real application. That is a significant achievement.

What to Learn Next

Kubernetes is a vast ecosystem. Here are the most important topics to explore after mastering the basics:

Topic	What It Is	Why It Matters
Helm	A package manager for Kubernetes (like npm for Node.js or apt for Linux)	Lets you install complex applications with one command and manage configuration templates
Terraform	Infrastructure as Code tool	Lets you create and manage Kubernetes clusters and cloud resources using code
Service Mesh (Istio, Linkerd)	Advanced networking layer	Provides traffic management, security, and observability between services
CI/CD Pipelines	Automated build and deploy	Automates building images, running tests, and deploying to Kubernetes when you push code
Horizontal Pod Autoscaler	Automatic scaling	Automatically adjusts the number of Pod replicas based on CPU, memory, or custom metrics
Network Policies	Firewall rules for Pods	Controls which Pods can communicate with each other
RBAC	Role-Based Access Control	Controls who can do what in the cluster

Prometheus + Grafana	Monitoring and dashboards	Collects metrics from your applications and displays them in visual dashboards

---

Common Mistakes

1. Forgetting `imagePullPolicy: Never` for local images. Without this, Kubernetes tries to pull the image from Docker Hub, fails, and the Pod enters `ImagePullBackOff` status.
2. Not setting the namespace in YAML files. If you forget to set `namespace: myapp` in your YAML, resources get created in the `default` namespace. Always include the namespace in the metadata section.
3. Wrong Service name in DB_HOST. The ConfigMap's `DB_HOST` must exactly match the PostgreSQL Service name. If the Service is called `postgres-service` but you set `DB_HOST=postgres`, the API cannot find the database.
4. Not using `subPath` for PostgreSQL volumes. PostgreSQL requires its data directory to be empty on initialization. Without `subPath: pgdata`, the PVC's root directory might contain a `lost+found` folder that causes PostgreSQL to fail.
5. Deleting the PVC without backing up data. Deleting a PVC permanently destroys the data on the underlying PersistentVolume (depending on the reclaim policy). Always back up important data before deleting PVCs.
6. Scaling the database Deployment. Do not run `kubectl scale deployment postgres --replicas=3`. PostgreSQL is not designed to be scaled this way. Multiple PostgreSQL instances writing to separate volumes would result in separate, inconsistent databases. Database scaling requires specialized tools like Patroni or CrunchyData Postgres Operator.

Best Practices

1. Use namespaces for every project. Namespaces keep resources organized and make cleanup easy (`kubectl delete namespace` removes everything).
2. Always set resource requests and limits. Without them, a single Pod can consume all resources on a node and starve other Pods.
3. Use health checks (readiness and liveness probes). We did not cover these in detail, but production Deployments should define probes so Kubernetes knows when a container is

healthy and ready to receive traffic.

4. Use declarative YAML files for everything. Store all your manifests in Git. This makes your infrastructure reproducible and auditable.
5. Tag your images with specific versions. Use `myapp:1.0.0`, not `myapp:latest`. The `latest` tag makes it unclear which version is running and can cause unexpected behavior during rollbacks.
6. Separate database Deployments from application Deployments. Databases have different lifecycle requirements (single replica, persistent storage, careful upgrades). Keep them in separate YAML files.

Quick Summary

Deploying a real application to Kubernetes involves creating multiple interconnected resources: a Namespace for isolation, ConfigMaps for settings, Secrets for credentials, PVCs for storage, Deployments for running Pods, and Services for networking. The database uses a ClusterIP Service for internal access, while the API uses a NodePort Service for external access. Kubernetes provides scaling, rolling updates, and rollback out of the box, enabling zero-downtime deployments and quick recovery from bad releases.

Key Points

- A complete Kubernetes application typically requires: Namespace, ConfigMap, Secret, PVC, Deployments, and Services
- Use `imagePullPolicy: Never` for locally built images in Minikube
- Kubernetes DNS lets Pods reach Services by name (e.g., `postgres-service`)
- Use ClusterIP for internal services (databases) and NodePort or LoadBalancer for external services (APIs)
- `kubectl get all -n <namespace>` shows all resources in a namespace
- `kubectl scale` adjusts the number of Pod replicas instantly
- `kubectl set image` triggers a rolling update with zero downtime
- `kubectl rollout undo` rolls back to the previous version immediately
- Ingress provides production-grade HTTP routing with domain names
- `kubectl top` shows CPU and memory usage for nodes and Pods

- Always set resource requests and limits in production
- Use specific image tags (not `latest`) for reproducible deployments

---

Practice Questions

1. You deploy an API with 3 replicas and a database with 1 replica. Why should you not scale the database Deployment to 3 replicas the same way you scaled the API?
2. Your API Pods are in `ImagePullBackOff` status. You built the image locally with Docker. What did you forget to do? How do you fix it?
3. After a rolling update, users report errors. Walk through the steps to roll back the deployment. What commands would you run, and what would you check afterwards?
4. Explain why the PostgreSQL Service uses ClusterIP while the API Service uses NodePort. What would happen if you used NodePort for the database?
5. You change a value in the ConfigMap, but the running Pods still use the old value. Why? What do you need to do to make the Pods pick up the new configuration?

Exercises

Exercise 1: Deploy Your Own Application

1. Create a simple application in any language (Python Flask, Go, Ruby Sinatra, or even a static HTML site with Nginx)
2. Write a Dockerfile and build the image
3. Create the full set of Kubernetes manifests (Namespace, ConfigMap, Deployment, Service)
4. Deploy it to your local cluster
5. Access it from your browser
6. Scale it to 5 replicas and verify all Pods are running

Exercise 2: Rolling Update Practice

1. Deploy the Nginx image version `1.24` with 3 replicas
2. Create a NodePort Service for it
3. Perform a rolling update to `nginx:1.25`

4. Check the rollout history
5. Roll back to `nginx:1.24`
6. Verify the rollback was successful

Exercise 3: Full Stack Application

1. Using the Node.js + PostgreSQL example from this chapter as a guide, deploy a different stack:
    - A Python Flask or Go API
    - A Redis or MySQL database
    - ConfigMaps for application settings
    - Secrets for database credentials
    - PVC for database storage
2. Test all endpoints
3. Perform a rolling update and rollback
4. Clean up by deleting the namespace

What Is Next?

You have completed the Kubernetes section of this book. You can now build Docker images, compose multi-container applications, and deploy them to Kubernetes with confidence.

The skills you have learned — containerization with Docker and orchestration with Kubernetes — are the foundation of modern cloud-native development. Companies around the world use these tools every day to run applications at scale.

Here is your roadmap for continued learning:

1. Helm — Learn to package and share Kubernetes applications as reusable charts
2. CI/CD — Set up automated pipelines (GitHub Actions, GitLab CI, Jenkins) that build, test, and deploy to Kubernetes on every code push
3. Terraform — Manage your Kubernetes clusters and cloud infrastructure as code
4. Monitoring — Set up Prometheus and Grafana to watch your applications in real time
5. Service Mesh — Explore Istio or Linkerd for advanced traffic management and security between microservices

Congratulations on finishing this journey. You started with "What is Docker?" and ended with a fully deployed, scalable application on Kubernetes. Keep building, keep learning, and keep deploying.

Appendix A: Docker Commands Cheat Sheet

What You Will Learn

In this appendix, you will learn:

- Every Docker command organized by category
- What each command does in plain English
- Quick examples you can copy and paste
- The most common flags and options for each command

Why This Chapter Matters

Think of this appendix as your toolbox label maker. When you are building a house, you do not memorize every tool. You learn where to find them. This cheat sheet works the same way. Keep it open while you work with Docker. Over time, the commands you use most will become second nature. For the rest, you will always know exactly where to look.

```
+-------------------------------------------------------+
|              YOUR DOCKER TOOLBOX                      |
|                                                       |
|  +-------------+  +-------------+  +-------------+   |
|  | Containers  |  |   Images    |  |   Volumes   |   |
|  | run, stop,  |  | build, pull |  | create, ls  |   |
|  | rm, logs... |  | push, tag.. |  | rm, prune.. |   |
|  +-------------+  +-------------+  +-------------+   |
|                                                       |
|  +-------------+  +-------------+  +-------------+   |
|  |  Networks   |  |   Compose   |  |   System    |   |
|  | create, ls  |  | up, down,   |  | info, prune |   |
|  | connect...  |  | ps, logs... |  | df, events  |   |
|  +-------------+  +-------------+  +-------------+   |
```

```
|                                                          |
|  +-------------+  +-------------+                        |
|  |   Swarm     |  |  kubectl    |                        |
|  | init, join  |  | get, apply  |                        |
|  | service...  |  | logs, exec  |                        |
|  +-------------+  +-------------+                        |
+----------------------------------------------------------+
```

Container Commands

Containers are running instances of images. Think of images as recipes and containers as the actual dishes you cook from those recipes. These commands let you cook, serve, and clean up.

---

docker run

What it does: Creates and starts a new container from an image.

Think of it as saying "take this recipe and cook a dish right now."

```bash
# Run a simple container
docker run hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

```bash
# Run in the background (detached mode) with a name
docker run -d --name my-web -p 8080:80 nginx
```

```
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6
```

```bash
# Run interactively with a terminal
docker run -it ubuntu bash
```

```
root@a1b2c3d4e5f6:/#
```

Common flags:

Flag	What It Does
`-d`	Run in background (detached)
`-it`	Interactive mode with terminal
`--name`	Give the container a name
`-p 8080:80`	Map port 8080 on your machine to port 80 in the container
`-v /host:/container`	Mount a folder from your machine into the container
`-e KEY=VALUE`	Set an environment variable
`--rm`	Automatically remove container when it stops
`--network`	Connect to a specific network
`--restart`	Set restart policy (no, always, on-failure, unless-stopped)

---

docker ps

What it does: Lists running containers. Add `-a` to see all containers, including stopped ones.

Think of it as checking which dishes are currently being served.

```bash
# Show running containers
docker ps
```

```
CONTAINER ID   IMAGE   COMMAND                  CREATED          STATUS          PORTS ...
a1b2c3d4e5f6   nginx   "/docker-entrypoint.…"   5 minutes ago    Up 5 minutes    0.0.0....
```

```bash
# Show ALL containers (running and stopped)
docker ps -a
```

```
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          ...
a1b2c3d4e5f6   nginx         "/docker-entrypoint.…"   5 minutes ago    Up 5 minutes    ...
b2c3d4e5f6g7   hello-world   "/hello"                 10 minutes ago   Exited (0) 10 mi...
```

Common flags:

Flag	What It Does
`-a`	Show all containers (including stopped)
`-q`	Show only container IDs
`--filter`	Filter output (e.g., `--filter status=running`)
`--format`	Custom output format

---

docker stop

What it does: Gracefully stops a running container. It sends a polite "please shut down" signal, then waits 10 seconds before forcing it.

Think of it as asking a restaurant to close for the night — they finish serving current customers first.

```bash
# Stop a container by name
docker stop my-web
```

```
my-web
```

```bash
# Stop a container with a custom timeout (5 seconds)
docker stop -t 5 my-web
```

```
my-web
```

```bash
# Stop multiple containers at once
docker stop container1 container2 container3
```

---

docker rm

What it does: Removes a stopped container. This is like washing the dish and putting it away — it is gone.

```bash
# Remove a stopped container
docker rm my-web
```

```
my-web
```

```bash
# Force remove a running container
docker rm -f my-web
```

```
my-web
```

```bash
# Remove all stopped containers
docker rm $(docker ps -aq -f status=exited)
```

Common flags:

Flag	What It Does
`-f`	Force remove (even if running)
`-v`	Also remove associated volumes

---

docker logs

What it does: Shows the output (logs) from a container. Like reading the diary of what the container has been doing.

```bash
# Show all logs
docker logs my-web
```

```
/docker-entrypoint.sh: Configuration complete; ready for start up
2024-01-15 10:30:00 [notice] nginx/1.25.0
```

```
2024-01-15 10:30:00 [notice] start worker processes
```

```bash
# Follow logs in real time (like "tail -f")
docker logs -f my-web
```

```bash
# Show last 20 lines
docker logs --tail 20 my-web
```

```bash
# Show logs with timestamps
docker logs -t my-web
```

Common flags:

Flag	What It Does
`-f`	Follow log output in real time
`--tail N`	Show only the last N lines
`-t`	Show timestamps
`--since`	Show logs since a timestamp (e.g., `--since 2h`)

---

docker exec

What it does: Runs a command inside a running container. Like reaching into a running machine to flip a switch.

```bash
# Open a shell inside a running container
docker exec -it my-web bash
```

```
root@a1b2c3d4e5f6:/#
```

```bash
# Run a single command
docker exec my-web cat /etc/nginx/nginx.conf
```

```bash
# Run a command as a different user
docker exec -u www-data my-web whoami
```

```
www-data
```

Common flags:

Flag	What It Does
`-it`	Interactive mode with terminal
`-u`	Run as a specific user
`-w`	Set working directory
`-e`	Set environment variables

---

docker inspect

What it does: Shows detailed information about a container in JSON format. Like reading the full technical specification of a running machine.

```bash
# Inspect a container
docker inspect my-web
```

```json
[
    {
        "Id": "a1b2c3d4e5f6...",
        "State": {
            "Status": "running",
            "Running": true
        },
        "NetworkSettings": {
            "IPAddress": "172.17.0.2"
        }
    }
]
```

```bash
# Get just the IP address
docker inspect --format '{{.NetworkSettings.IPAddress}}' my-web
```

```
172.17.0.2
```

---

docker stats

What it does: Shows live resource usage (CPU, memory, network) for containers. Like a heart rate monitor for your containers.

```bash
# Show stats for all running containers
docker stats
```

```
CONTAINER ID   NAME     CPU %   MEM USAGE / LIMIT     MEM %   NET I/O         BLOCK I/O...
a1b2c3d4e5f6   my-web   0.00%   3.5MiB / 7.77GiB     0.04%   1.2kB / 648B    0B / 0B   ...
```

```bash
# Show stats for a specific container (one snapshot, no streaming)
docker stats --no-stream my-web
```

---

docker top

What it does: Shows the processes running inside a container. Like peeking through a window to see who is working inside.

```bash
docker top my-web
```

```
UID    PID    PPID   C   STIME   TTY   TIME       CMD
root   1234   1233   0   10:30   ?     00:00:00   nginx: master process
www    1235   1234   0   10:30   ?     00:00:00   nginx: worker process
```

---

docker cp

What it does: Copies files between your computer and a container. Like passing a note through a window.

```bash
# Copy a file FROM your computer TO a container
docker cp ./index.html my-web:/usr/share/nginx/html/index.html
```

```bash
# Copy a file FROM a container TO your computer
docker cp my-web:/var/log/nginx/access.log ./access.log
```

---

docker rename

What it does: Renames a container. Simple as that.

```bash
docker rename my-web my-website
```

---

docker wait

What it does: Blocks (waits) until a container stops, then prints the exit code. Useful in scripts where you need to know when something finishes.

```bash
docker wait my-web
```

```
0
```

The number `0` means the container exited successfully. Any other number means something went wrong.

---

docker pause / docker unpause

What it does: Freezes all processes in a container (pause) and unfreezes them (unpause). Like pressing the pause button on a video — everything stops exactly where it is.

```bash
# Freeze the container
docker pause my-web

# Unfreeze the container
docker unpause my-web
```

docker attach

What it does: Connects your terminal to a running container's main process. Unlike `exec`, this connects to the process that is already running, not a new one.

```bash
docker attach my-web
```

> Warning: Pressing `Ctrl+C` while attached will stop the container. Use `Ctrl+P` then `Ctrl+Q` to detach without stopping.

docker diff

What it does: Shows what files have changed inside a container compared to its original image. Like comparing a messy room to how it looked when it was clean.

```bash
docker diff my-web
```

```
C /var
C /var/log
C /var/log/nginx
A /var/log/nginx/access.log
A /var/log/nginx/error.log
```

- `A` = Added
- `C` = Changed
- `D` = Deleted

docker export

What it does: Exports a container's filesystem as a tar archive. Like packing everything in a container into a suitcase.

```bash
docker export my-web > my-web-backup.tar
```

> Note: This exports the filesystem only, not the image layers or metadata. For full image backup, use `docker save` instead.

---

docker commit

What it does: Creates a new image from a container's current state. Like taking a snapshot of a messy room and saving it as the "new normal."

```bash
docker commit my-web my-custom-nginx:v1
```

```
sha256:a1b2c3d4e5f6...
```

> Note: In practice, use a Dockerfile instead of `commit`. Dockerfiles are repeatable and version-controlled. Think of `commit` as a quick-and-dirty photo, while a Dockerfile is a proper blueprint.

Image Commands

Images are the blueprints (recipes) for containers. These commands let you create, share, and manage those blueprints.

```
+-----------------------------------------------------+
|                   IMAGE LIFECYCLE                   |
|                                                     |
|   Build ──> Tag ──> Push ──> (Registry) ──> Pull    |
|     │                                        │      |
|     ▼                                        ▼      |
|   images ──> inspect ──> history            run     |
|     │                                               |
|     ▼                                               |
|   save/load (backup)    rmi/prune (cleanup)         |
+-----------------------------------------------------+
```

---

docker build

What it does: Builds a new image from a Dockerfile. Like following a recipe to create a blueprint.

```bash
# Build an image from the current directory
docker build -t my-app:1.0 .
```

```
[+] Building 15.2s (10/10) FINISHED
 => [1/5] FROM node:18-alpine
 => [2/5] WORKDIR /app
 => [3/5] COPY package*.json ./
 => [4/5] RUN npm install
 => [5/5] COPY . .
 => exporting to image
 => => naming to docker.io/library/my-app:1.0
```

```bash
# Build with a specific Dockerfile
docker build -t my-app:1.0 -f Dockerfile.prod .
```

```bash
# Build with build arguments
docker build --build-arg NODE_ENV=production -t my-app:1.0 .
```

Common flags:

Flag	What It Does
`-t`	Tag (name) the image
`-f`	Specify a Dockerfile by name
`--build-arg`	Pass variables to the build
`--no-cache`	Build from scratch, ignore cache
`--target`	Build a specific stage in a multi-stage build

---

docker pull

What it does: Downloads an image from a registry (like Docker Hub). Like ordering a recipe book from a store.

```bash
docker pull nginx
```

```
Using default tag: latest
latest: Pulling from library/nginx
a1b2c3d4e5f6: Pull complete
g7h8i9j0k1l2: Pull complete
Digest: sha256:abc123...
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

```bash
# Pull a specific version
docker pull nginx:1.25

# Pull from a different registry
docker pull ghcr.io/myorg/my-app:2.0
```

---

docker push

What it does: Uploads an image to a registry. Like publishing your recipe book so others can use it.

```bash
# You must be logged in first
docker login

# Tag your image for the registry
docker tag my-app:1.0 myusername/my-app:1.0

# Push it
docker push myusername/my-app:1.0
```

```
The push refers to repository [docker.io/myusername/my-app]
a1b2c3d4e5f6: Pushed
g7h8i9j0k1l2: Pushed
1.0: digest: sha256:abc123... size: 1234
```

---

docker images

What it does: Lists all images on your machine. Like looking at your shelf of recipe books.

```bash
docker images
```

```
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
my-app        1.0       a1b2c3d4e5f6   2 hours ago     175MB
nginx         latest    g7h8i9j0k1l2   2 weeks ago     187MB
node          18        m3n4o5p6q7r8   3 weeks ago     1.1GB
```

Common flags:

Flag	What It Does
`-a`	Show all images (including intermediate)
`-q`	Show only image IDs
`--filter`	Filter output (e.g., `--filter dangling=true`)

---

docker rmi

What it does: Removes an image. Like throwing away a recipe book you no longer need.

```bash
# Remove by name
docker rmi my-app:1.0
```

```
Untagged: my-app:1.0
Deleted: sha256:a1b2c3d4e5f6...
```

```bash
# Force remove
docker rmi -f my-app:1.0

# Remove multiple images
docker rmi image1 image2 image3
```

---

docker tag

What it does: Gives an image a new name (tag). Like putting a new label on an existing recipe book. The image itself does not change.

```bash
docker tag my-app:1.0 myusername/my-app:latest
docker tag my-app:1.0 myusername/my-app:1.0
```

docker history

What it does: Shows the layers that make up an image. Like seeing the step-by-step history of how a recipe was created.

```bash
docker history my-app:1.0
```

```
IMAGE          CREATED        CREATED BY                                      SIZE
a1b2c3d4e5f6   2 hours ago    CMD ["node", "server.js"]                       0B
b2c3d4e5f6g7   2 hours ago    COPY . .                                        5.2MB
c3d4e5f6g7h8   2 hours ago    RUN npm install                                 45MB
d4e5f6g7h8i9   2 hours ago    COPY package*.json ./                           120kB
e5f6g7h8i9j0   2 hours ago    WORKDIR /app                                    0B
f6g7h8i9j0k1   3 weeks ago    /bin/sh -c #(nop) CMD ["node"]                  0B
```

docker save

What it does: Saves an image to a tar file. Like making a backup copy of your recipe book.

```bash
docker save my-app:1.0 > my-app-backup.tar

# Or with gzip compression
docker save my-app:1.0 | gzip > my-app-backup.tar.gz
```

docker load

What it does: Loads an image from a tar file. Like restoring a recipe book from a backup.

```bash
docker load < my-app-backup.tar
```

```
Loaded image: my-app:1.0
```

docker image inspect

What it does: Shows detailed information about an image in JSON format.

```bash
docker image inspect nginx
```

```bash
# Get just the image size
docker image inspect --format '{{.Size}}' nginx
```

```
187234567
```

docker image prune

What it does: Removes unused images (dangling images with no tag). Like cleaning out old recipe books you never use.

```bash
# Remove dangling images
docker image prune
```

```
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:a1b2c3d4e5f6...
Total reclaimed space: 234MB
```

```bash
# Remove ALL unused images (not just dangling)
docker image prune -a
```

Volume Commands

Volumes are Docker's way of storing data that persists even when containers are deleted. Think of volumes as external hard drives you plug into your containers.

```
+-------------------------------------------+
|              VOLUMES                      |
|                                           |
|  Container A ──┐                          |
|                ├──> [my-data-volume]       |
|  Container B ──┘     (persists forever)    |
|                                           |
|  Container C ────> [logs-volume]          |
|                    (persists forever)     |
+-------------------------------------------+
```

---

docker volume create

What it does: Creates a new named volume.

```bash
docker volume create my-data
```

```
my-data
```

```bash
# Use it when running a container
docker run -d --name db -v my-data:/var/lib/postgresql/data postgres
```

---

docker volume ls

What it does: Lists all volumes.

```bash
docker volume ls
```

```
DRIVER    VOLUME NAME
local     my-data
local     logs-volume
local     db-backup
```

---

docker volume inspect

What it does: Shows detailed information about a volume.

```bash
docker volume inspect my-data
```

```json
[
    {
        "CreatedAt": "2024-01-15T10:30:00Z",
        "Driver": "local",
        "Labels": {},
        "Mountpoint": "/var/lib/docker/volumes/my-data/_data",
        "Name": "my-data",
        "Options": {},
        "Scope": "local"
    }
]
```

---

docker volume rm

What it does: Removes a volume. The data inside is gone forever.

```bash
docker volume rm my-data
```

```
my-data
```

> Warning: This permanently deletes all data in the volume. Make sure no containers are using it.

---

docker volume prune

What it does: Removes all volumes that are not currently attached to any container.

```bash
docker volume prune
```

```
WARNING! This will remove all local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
old-data
```

```
unused-logs
Total reclaimed space: 1.2GB
```

Network Commands

Networks let containers talk to each other. Think of networks as private phone lines between containers.

```
+-----------------------------------------------------+
|           DOCKER NETWORKS                           |
|                                                     |
|   ┌──────────── my-network ────────────┐            |
|   │                                    │            |
|   │  [web-app] ◄────► [database]       │            |
|   │      │                             │            |
|   │      ▼                             │            |
|   │  [cache]                           │            |
|   │                                    │            |
|   └────────────────────────────────────┘             |
|                                                     |
|   ┌──────────── other-network ─────────┐            |
|   │  [monitoring] ◄────► [alerting]    │            |
|   └────────────────────────────────────┘             |
+-----------------------------------------------------+
```

---

docker network create

What it does: Creates a new network.

```bash
docker network create my-network
```

```
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6
```

```bash
# Create a network with a specific subnet
docker network create --subnet 172.20.0.0/16 my-custom-network
```

---

docker network ls

What it does: Lists all networks.

```bash
docker network ls
```

```
NETWORK ID     NAME          DRIVER    SCOPE
a1b2c3d4e5f6   bridge        bridge    local
g7h8i9j0k1l2   host          host      local
m3n4o5p6q7r8   none          null      local
s9t0u1v2w3x4   my-network    bridge    local
```

---

docker network inspect

What it does: Shows detailed information about a network, including which containers are connected.

```bash
docker network inspect my-network
```

```json
[
    {
        "Name": "my-network",
        "Driver": "bridge",
        "Containers": {
            "a1b2c3d4...": {
                "Name": "web-app",
                "IPv4Address": "172.18.0.2/16"
            },
            "e5f6g7h8...": {
                "Name": "database",
                "IPv4Address": "172.18.0.3/16"
            }
        }
    }
]
```

---

docker network rm

What it does: Removes a network.

```bash
docker network rm my-network
```

---

docker network connect

What it does: Connects a running container to a network. Like plugging a phone into a new phone line.

```bash
docker network connect my-network my-container
```

---

docker network disconnect

What it does: Disconnects a container from a network.

```bash
docker network disconnect my-network my-container
```

---

docker network prune

What it does: Removes all networks that no container is using.

```bash
docker network prune
```

```
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
old-network
test-network
```

Docker Compose Commands

Docker Compose lets you define and run multi-container applications with a single YAML file. Think of it as an orchestra conductor — one person (one command) coordinates many

musicians (containers).

```
+-----------------------------------------------------+
|         docker-compose.yml                          |
|                                                     |
|   services:                                         |
|     web:        ------->  [web container]           |
|     database:   ------->  [db container]            |
|     cache:      ------->  [redis container]         |
|                                                     |
|   One file controls everything!                     |
+-----------------------------------------------------+
```

---

docker compose up

What it does: Creates and starts all containers defined in your `docker-compose.yml`.

```bash
# Start everything
docker compose up

# Start in the background
docker compose up -d

# Rebuild images before starting
docker compose up --build

# Start only specific services
docker compose up -d web database
```

```
[+] Running 3/3
 ✔ Network myapp_default      Created
 ✔ Container myapp-database-1 Started
 ✔ Container myapp-web-1      Started
```

---

docker compose down

What it does: Stops and removes all containers, networks created by `up`.

```bash
# Stop and remove containers
docker compose down
```

```
# Also remove volumes (careful — deletes data!)
docker compose down -v

# Also remove images
docker compose down --rmi all
```

```
[+] Running 3/3
 ✔ Container myapp-web-1      Removed
 ✔ Container myapp-database-1 Removed
 ✔ Network myapp_default      Removed
```

---

docker compose ps

What it does: Lists containers managed by the current Compose file.

```bash
docker compose ps
```

```
NAME                 IMAGE      COMMAND                  SERVICE     STATUS          PORTS
myapp-web-1          nginx      "/docker-entrypoint.…"   web         Up 5 minutes    0....
myapp-database-1     postgres   "docker-entrypoint.s…"   database    Up 5 minutes    54...
```

---

docker compose logs

What it does: Shows logs from all services or a specific service.

```bash
# All services
docker compose logs

# Specific service
docker compose logs web

# Follow in real time
docker compose logs -f web
```

---

docker compose exec

What it does: Runs a command in a running service container.

```bash
# Open a shell in the web service
docker compose exec web bash

# Run a database query
docker compose exec database psql -U postgres -d mydb -c "SELECT * FROM users;"
```

---

docker compose build

What it does: Builds (or rebuilds) images for services defined with a `build` section.

```bash
# Build all services
docker compose build

# Build a specific service
docker compose build web

# Build without cache
docker compose build --no-cache
```

---

docker compose pull

What it does: Pulls the latest images for services defined with an `image` section.

```bash
docker compose pull
```

```
[+] Pulling 2/2
 ✔ database Pulled
 ✔ cache    Pulled
```

---

docker compose restart

What it does: Restarts one or more services.

```bash
# Restart all services
docker compose restart
```

```
# Restart a specific service
docker compose restart web
```

---

docker compose stop

What it does: Stops services without removing them. You can start them again later with `docker compose start`.

```bash
docker compose stop
```

---

docker compose config

What it does: Validates and displays the final Compose configuration. Very useful for debugging.

```bash
docker compose config
```

This shows the fully resolved YAML with all variables substituted and defaults applied.

---

docker compose top

What it does: Shows running processes for each service.

```bash
docker compose top
```

```
myapp-web-1
UID    PID    PPID   C   STIME   TTY   TIME       CMD
root   1234   1233   0   10:30   ?     00:00:00   nginx: master process

myapp-database-1
UID       PID    PPID   C   STIME   TTY   TIME       CMD
postgres  5678   5677   0   10:30   ?     00:00:01   postgres
```

System Commands

These commands help you understand and manage Docker itself — the engine under the hood.

---

docker info

What it does: Shows system-wide information about your Docker installation.

```bash
docker info
```

```
Client:
 Version:    24.0.7
Server:
 Containers: 5
  Running: 2
  Paused: 0
  Stopped: 3
 Images: 12
 Server Version: 24.0.7
 Storage Driver: overlay2
 Operating System: Ubuntu 22.04
 Total Memory: 7.77GiB
 CPUs: 4
```

---

docker version

What it does: Shows the Docker client and server version.

```bash
docker version
```

```
Client:
 Version:           24.0.7
 API version:       1.43

Server:
 Engine:
  Version:          24.0.7
  API version:      1.43
```

---

docker system df

What it does: Shows how much disk space Docker is using. Like checking how full your hard drive is.

```bash
docker system df
```

```
TYPE            TOTAL   ACTIVE   SIZE      RECLAIMABLE
Images          12      3        4.2GB     3.1GB (73%)
Containers      5       2        120MB     80MB (66%)
Local Volumes   4       2        500MB     200MB (40%)
Build Cache     0       0        0B        0B
```

```bash
# Show detailed breakdown
docker system df -v
```

---

docker system prune

What it does: Removes all unused data — stopped containers, dangling images, unused networks, and build cache. The big cleanup button.

```bash
docker system prune
```

```
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all dangling images
  - all dangling build cache

Are you sure you want to continue? [y/N] y
Total reclaimed space: 2.3GB
```

```bash
# Also remove unused volumes (be very careful!)
docker system prune --volumes

# Remove everything unused, including tagged images
docker system prune -a
```

---

docker events

What it does: Shows real-time events from the Docker daemon. Like watching a live security camera feed of everything Docker does.

```bash
docker events
```

```
2024-01-15T10:30:00.000000000Z container start a1b2c3d4...
2024-01-15T10:30:05.000000000Z container die b2c3d4e5...
2024-01-15T10:30:06.000000000Z network connect s9t0u1v2...
```

```bash
# Filter events by type
docker events --filter type=container

# Filter events since a specific time
docker events --since 1h
```

Swarm Commands

Docker Swarm turns a group of Docker machines into a single cluster. Think of it as connecting multiple kitchens so they can cook together for a much bigger restaurant.

```
+-----------------------------------------------------+
|              DOCKER SWARM CLUSTER                    |
|                                                      |
|  ┌──────────────┐    ┌──────────────┐               |
|  │ Manager      │    │ Worker       │               |
|  │ Node         │    │ Node 1       │               |
|  │              │    │              │               |
|  │ ┌──────────┐ │    │ ┌──────────┐ │               |
|  │ │ Service  │ │    │ │ Service  │ │               |
|  │ │ Replica  │ │    │ │ Replica  │ │               |
|  │ └──────────┘ │    │ └──────────┘ │               |
|  └──────────────┘    └──────────────┘               |
|                                                      |
|                      ┌──────────────┐               |
|                      │ Worker       │               |
|                      │ Node 2       │               |
|                      │              │               |
|                      │ ┌──────────┐ │               |
|                      │ │ Service  │ │               |
|                      │ │ Replica  │ │               |
```

```
|                    | └──────────┘ |             |
|                    └──────────────┘             |
+-------------------------------------------------+
```

docker swarm init

What it does: Initializes the current machine as a Swarm manager.

```bash
docker swarm init
```

```
Swarm initialized: current node is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-abc123... 192.168.1.100:2377
```

docker swarm join

What it does: Joins a machine to an existing Swarm as a worker.

```bash
docker swarm join --token SWMTKN-1-abc123... 192.168.1.100:2377
```

```
This node joined a swarm as a worker.
```

docker swarm leave

What it does: Removes the current machine from the Swarm.

```bash
docker swarm leave

# Force leave if it is a manager
docker swarm leave --force
```

docker node

What it does: Manages nodes (machines) in the Swarm.

```bash
# List all nodes
docker node ls
```

```
ID             HOSTNAME       STATUS   AVAILABILITY   MANAGER STATUS
a1b2c3d4e5 *   manager-1      Ready    Active         Leader
f6g7h8i9j0     worker-1       Ready    Active
k1l2m3n4o5     worker-2       Ready    Active
```

```bash
# Inspect a node
docker node inspect worker-1

# Drain a node (stop scheduling new tasks on it)
docker node update --availability drain worker-1
```

---

docker service

What it does: Manages services in the Swarm. A service is a definition of how to run containers across the cluster.

```bash
# Create a service with 3 replicas
docker service create --name web --replicas 3 -p 8080:80 nginx
```

```
a1b2c3d4e5f6g7h8i9j0
```

```bash
# List services
docker service ls
```

```
ID             NAME   MODE         REPLICAS   IMAGE
a1b2c3d4e5f6   web    replicated   3/3        nginx:latest
```

```bash
# Scale a service
docker service scale web=5
```

```
# See which nodes are running the service
docker service ps web

# Update a service
docker service update --image nginx:1.25 web

# Remove a service
docker service rm web
```

---

docker stack

What it does: Manages stacks (multi-service applications) in the Swarm using Compose files.

```bash
# Deploy a stack from a Compose file
docker stack deploy -c docker-compose.yml myapp
```

```
Creating network myapp_default
Creating service myapp_web
Creating service myapp_database
```

```bash
# List stacks
docker stack ls

# List services in a stack
docker stack services myapp

# Remove a stack
docker stack rm myapp
```

---

kubectl Quick Reference

kubectl is the command-line tool for Kubernetes. If Docker Swarm is a few kitchens working together, Kubernetes is an entire restaurant chain managed from a central headquarters.

```
+-----------------------------------------------------+
|           KUBERNETES QUICK REFERENCE                |
|                                                     |
```

```
|   kubectl get      = "Show me what exists"         |
|   kubectl describe = "Tell me everything about it" |
|   kubectl apply    = "Make it happen"               |
|   kubectl delete   = "Remove it"                   |
|   kubectl logs     = "Show me the output"          |
|   kubectl exec     = "Run something inside"        |
|   kubectl scale    = "Change the count"            |
|   kubectl rollout  = "Manage updates"              |
+----------------------------------------------------+
```

---

kubectl get

What it does: Lists resources. The most-used kubectl command.

```bash
# List all pods
kubectl get pods
```

```
NAME                     READY   STATUS    RESTARTS   AGE
web-app-7d4b8c9f-x2k9   1/1     Running   0          5m
web-app-7d4b8c9f-m3p7   1/1     Running   0          5m
database-5c6d7e8f-j4k2  1/1     Running   0          5m
```

```bash
# List services
kubectl get services

# List deployments
kubectl get deployments

# List everything
kubectl get all

# List pods with more detail
kubectl get pods -o wide

# List pods in all namespaces
kubectl get pods --all-namespaces
```

---

kubectl describe

What it does: Shows detailed information about a resource. When something goes wrong, this is your best friend.

```bash
kubectl describe pod web-app-7d4b8c9f-x2k9
```

```
Name:         web-app-7d4b8c9f-x2k9
Namespace:    default
Status:       Running
IP:           10.244.0.5
Containers:
  web-app:
    Image:    my-app:1.0
    Port:     3000/TCP
    State:    Running
Events:
  Normal  Scheduled  5m   default-scheduler  Successfully assigned...
  Normal  Pulled     5m   kubelet            Container image already present
  Normal  Started    5m   kubelet            Started container web-app
```

---

kubectl apply

What it does: Creates or updates resources from a YAML file. The standard way to deploy things in Kubernetes.

```bash
# Apply a single file
kubectl apply -f deployment.yaml
```

```
deployment.apps/web-app created
```

```bash
# Apply all YAML files in a directory
kubectl apply -f ./k8s/

# Apply from a URL
kubectl apply -f https://example.com/deployment.yaml
```

---

kubectl delete

What it does: Removes resources.

```bash
# Delete a specific pod
kubectl delete pod web-app-7d4b8c9f-x2k9

# Delete using a YAML file
kubectl delete -f deployment.yaml

# Delete all pods with a specific label
kubectl delete pods -l app=web-app
```

---

kubectl logs

What it does: Shows logs from a pod.

```bash
# Show logs
kubectl logs web-app-7d4b8c9f-x2k9

# Follow logs in real time
kubectl logs -f web-app-7d4b8c9f-x2k9

# Show logs from a specific container (multi-container pod)
kubectl logs web-app-7d4b8c9f-x2k9 -c sidecar

# Show logs from a previous (crashed) instance
kubectl logs web-app-7d4b8c9f-x2k9 --previous
```

---

kubectl exec

What it does: Runs a command inside a pod.

```bash
# Open a shell
kubectl exec -it web-app-7d4b8c9f-x2k9 -- bash

# Run a single command
kubectl exec web-app-7d4b8c9f-x2k9 -- cat /app/config.json
```

---

kubectl scale

What it does: Changes the number of replicas in a deployment.

```bash
kubectl scale deployment web-app --replicas=5
```

```
deployment.apps/web-app scaled
```

---

kubectl rollout

What it does: Manages rollouts (updates) for deployments.

```bash
# Check rollout status
kubectl rollout status deployment/web-app
```

```
deployment "web-app" successfully rolled out
```

```bash
# View rollout history
kubectl rollout history deployment/web-app

# Undo the last rollout
kubectl rollout undo deployment/web-app

# Undo to a specific revision
kubectl rollout undo deployment/web-app --to-revision=2
```

Quick Reference Table

Here is a side-by-side comparison of the most common tasks:

Task	Docker Command	kubectl Command
Run something	`docker run nginx`	`kubectl run nginx --image=nginx`
List running	`docker ps`	`kubectl get pods`
Stop	`docker stop my-web`	`kubectl delete pod my-pod`
View logs	`docker logs my-web`	`kubectl logs my-pod`

Shell access	`docker exec -it my-web bash`	`kubectl exec -it my-pod -- bash`
Deploy app	`docker compose up`	`kubectl apply -f app.yaml`
Scale	`docker compose up --scale web=3`	`kubectl scale deploy web --replicas=3`
Clean up	`docker system prune`	`kubectl delete all --all`

Key Points

- Container commands let you create, run, monitor, and clean up containers. The most important ones are `run`, `ps`, `stop`, `rm`, `logs`, and `exec`.
- Image commands handle blueprints. You will use `build`, `pull`, `push`, `images`, and `rmi` the most.
- Volume commands manage persistent data. Always use named volumes for important data.
- Network commands control how containers communicate. Custom networks give you automatic DNS.
- Compose commands orchestrate multi-container apps. `up`, `down`, `ps`, and `logs` are your daily tools.
- System commands keep Docker healthy. Run `docker system prune` regularly to free up disk space.
- Swarm commands manage container clusters. Great for simpler deployments.
- kubectl commands manage Kubernetes clusters. More powerful but more complex than Swarm.
- When in doubt about any command, add `--help` to see all available options (e.g., `docker run --help`).

Appendix B: Complete Project — Microservices App

What You Will Learn

In this appendix, you will learn:

- How to build a complete microservices application from scratch
- How to write Dockerfiles for Node.js, Python, and Java services
- How to wire everything together with Docker Compose
- How to deploy the same application to Kubernetes
- How all the concepts from this book come together in a real project

Why This Chapter Matters

Throughout this book, you have learned Docker concepts one at a time — like learning individual musical notes. Now it is time to play a full song. This appendix walks you through building, connecting, and deploying a complete application made of multiple services, just like professional teams do every day.

By the end, you will have a working microservices application that you built, tested locally, and deployed to Kubernetes. This is not a toy example. It is the same architecture pattern used by companies like Netflix, Uber, and Spotify.

```
+-------------------------------------------------------------------+
|                    WHAT WE ARE BUILDING                           |
|                                                                   |
|   Browser / Client                                                |
|        |                                                          |
|        ▼                                                          |
|   ┌──────────────────────────────────────────┐                    |
|   │           API Gateway (Nginx)            │                    |
```

```
|    |           Port 80                        |                      |
|    └──────────┬──────────┬──────────┘                      |
|         │          │          │                                |
|         ▼          ▼          ▼                                |
|    ┌─────────┐  ┌─────────┐  ┌─────────┐                     |
|    │  User   │  │ Product │  │  Order  │                     |
|    │ Service │  │ Service │  │ Service │                     |
|    │(Node.js)│  │(Python) │  │ (Java)  │                     |
|    │Port 3001│  │Port 5000│  │Port 8080│                     |
|    └────┬────┘  └──┬──┬───┘  └──┬──┬──┘                     |
|         │          │  │         │  │                         |
|         ▼          ▼  ▼         ▼  ▼                         |
|    ┌──────────────────────────────────┐                      |
|    │           PostgreSQL Database    │                      |
|    │           Port 5432              │                      |
|    └──────────────────────────────────┘                      |
|                      │                                       |
|    ┌──────────────────────────────────┐                      |
|    │           Redis Cache            │                      |
|    │           Port 6379              │                      |
|    └──────────────────────────────────┘                      |
+--------------------------------------------------------------+
```

Project Structure

First, let us set up the folder structure. Think of each service as its own mini-project living in its own folder.

```bash
mkdir -p microservices-app/{user-service,product-service,order-service,api-gateway,k8s,db}
cd microservices-app
```

Your project will look like this:

```
microservices-app/
├── docker-compose.yml
├── api-gateway/
│   └── nginx.conf
├── user-service/
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── product-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
```

```
├── order-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/
│       └── main/
│           └── java/
│               └── com/
│                   └── orders/
│                       └── OrderServiceApplication.java
├── db/
│   └── init.sql
└── k8s/
    ├── namespace.yaml
    ├── postgres.yaml
    ├── redis.yaml
    ├── user-service.yaml
    ├── product-service.yaml
    ├── order-service.yaml
    └── api-gateway.yaml
```

Step 1: The Database

Every application needs data. We will use PostgreSQL and initialize it with tables for all three services.

db/init.sql

```sql
-- Create databases for each service
CREATE DATABASE userdb;
CREATE DATABASE productdb;
CREATE DATABASE orderdb;

-- Connect to userdb and create tables
\c userdb;

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO users (username, email) VALUES
    ('alice', 'alice@example.com'),
    ('bob', 'bob@example.com'),
```

```
    ('charlie', 'charlie@example.com');

-- Connect to productdb and create tables
\c productdb;

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL,
    stock INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO products (name, description, price, stock) VALUES
    ('Laptop', 'High-performance laptop', 999.99, 50),
    ('Headphones', 'Wireless noise-canceling headphones', 149.99, 200),
    ('Keyboard', 'Mechanical keyboard with RGB', 79.99, 150),
    ('Mouse', 'Ergonomic wireless mouse', 49.99, 300);

-- Connect to orderdb and create tables
\c orderdb;

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 1,
    total_price DECIMAL(10, 2) NOT NULL,
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO orders (user_id, product_id, quantity, total_price, status) VALUES
    (1, 1, 1, 999.99, 'completed'),
    (2, 2, 2, 299.98, 'pending'),
    (1, 3, 1, 79.99, 'shipped');
```

This single SQL file creates three separate databases — one for each service. This is a common pattern in microservices: each service owns its own data.

```
+-----------------------------------------------+
|         ONE POSTGRES, THREE DATABASES         |
|                                               |
|  ┌──────────────┐ ┌──────────────┐ ┌──────────┐  |
|  │  userdb      │ │ productdb    │ │ orderdb  │  |
|  │              │ │              │ │          │  |
|  │  users       │ │  products    │ │  orders  │  |
|  │  table       │ │  table       │ │  table   │  |
|  └──────────────┘ └──────────────┘ └──────────┘  |
|                                               |
|         PostgreSQL Server (port 5432)         |
```

```
+----------------------------------------------+
```

Step 2: User Service (Node.js)

The User Service manages user accounts. We will build it with Node.js and Express.

user-service/package.json

```json
{
  "name": "user-service",
  "version": "1.0.0",
  "description": "User microservice",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.3",
    "redis": "^4.6.10"
  }
}
```

user-service/server.js

```javascript
const express = require('express');
const { Pool } = require('pg');
const { createClient } = require('redis');

const app = express();
app.use(express.json());

// PostgreSQL connection
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  database: process.env.DB_NAME || 'userdb',
  user: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD || 'postgres',
});
```

```
// Redis connection
let redisClient;

async function connectRedis() {
  redisClient = createClient({
    url: `redis://${process.env.REDIS_HOST || 'localhost'}:${process.env.REDIS_PORT || ...
  });
  redisClient.on('error', (err) => console.log('Redis error:', err.message));
  await redisClient.connect();
  console.log('Connected to Redis');
}

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'healthy', service: 'user-service' });
});

// Get all users
app.get('/api/users', async (req, res) => {
  try {
    // Check Redis cache first
    const cached = await redisClient.get('users:all');
    if (cached) {
      console.log('Returning users from cache');
      return res.json(JSON.parse(cached));
    }

    // Query database
    const result = await pool.query('SELECT * FROM users ORDER BY id');

    // Cache for 60 seconds
    await redisClient.setEx('users:all', 60, JSON.stringify(result.rows));

    console.log('Returning users from database');
    res.json(result.rows);
  } catch (error) {
    console.error('Error fetching users:', error.message);
    res.status(500).json({ error: 'Failed to fetch users' });
  }
});

// Get user by ID
app.get('/api/users/:id', async (req, res) => {
  try {
    const { id } = req.params;

    // Check cache
    const cached = await redisClient.get(`users:${id}`);
    if (cached) {
      return res.json(JSON.parse(cached));
    }

    const result = await pool.query('SELECT * FROM users WHERE id = $1', [id]);
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'User not found' });
```

```
    }

    await redisClient.setEx(`users:${id}`, 60, JSON.stringify(result.rows[0]));
    res.json(result.rows[0]);
  } catch (error) {
    console.error('Error fetching user:', error.message);
    res.status(500).json({ error: 'Failed to fetch user' });
  }
});

// Create a new user
app.post('/api/users', async (req, res) => {
  try {
    const { username, email } = req.body;
    const result = await pool.query(
      'INSERT INTO users (username, email) VALUES ($1, $2) RETURNING *',
      [username, email]
    );

    // Invalidate cache
    await redisClient.del('users:all');

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error('Error creating user:', error.message);
    res.status(500).json({ error: 'Failed to create user' });
  }
});

// Start server
const PORT = process.env.PORT || 3001;

async function start() {
  await connectRedis();
  app.listen(PORT, () => {
    console.log(`User Service running on port ${PORT}`);
  });
}

start();
```

user-service/Dockerfile

```dockerfile
# Stage 1: Install dependencies
FROM node:18-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# Stage 2: Production image
```

```
FROM node:18-alpine
WORKDIR /app

# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copy dependencies and source code
COPY --from=deps /app/node_modules ./node_modules
COPY server.js .
COPY package.json .

# Switch to non-root user
USER appuser

EXPOSE 3001

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3001/health || exit 1

CMD ["node", "server.js"]
```

Let us break down why this Dockerfile is structured this way:

```
+-----------------------------------------------------+
|        MULTI-STAGE BUILD EXPLAINED                  |
|                                                     |
|  Stage 1 (deps):                                    |
|  ┌──────────────────────────────────┐               |
|  │  Full Node.js + npm              │               |
|  │  Install ALL dependencies        │               |
|  │  This stage is thrown away        |               |
|  └──────────────────────────────────┘               |
|                    |                                |
|                    | copy node_modules               |
|                    ▼                                |
|  Stage 2 (production):                              |
|  ┌──────────────────────────────────┐               |
|  │  Slim Node.js (no npm needed)    │               |
|  │  Only production dependencies    │               |
|  │  Non-root user (security)        │               |
|  │  Health check built in           │               |
|  │  MUCH smaller final image        │               |
|  └──────────────────────────────────┘               |
+-----------------------------------------------------+
```

Step 3: Product Service (Python Flask)

The Product Service manages the product catalog. We will build it with Python and Flask.

product-service/requirements.txt

```
flask==3.0.0
psycopg2-binary==2.9.9
redis==5.0.1
gunicorn==21.2.0
```

product-service/app.py

```python
import os
import json
from flask import Flask, request, jsonify
import psycopg2
from psycopg2.extras import RealDictCursor
import redis

app = Flask(__name__)

# Database connection
def get_db_connection():
    return psycopg2.connect(
        host=os.environ.get('DB_HOST', 'localhost'),
        port=os.environ.get('DB_PORT', 5432),
        database=os.environ.get('DB_NAME', 'productdb'),
        user=os.environ.get('DB_USER', 'postgres'),
        password=os.environ.get('DB_PASSWORD', 'postgres'),
        cursor_factory=RealDictCursor
    )

# Redis connection
redis_client = redis.Redis(
    host=os.environ.get('REDIS_HOST', 'localhost'),
    port=int(os.environ.get('REDIS_PORT', 6379)),
    decode_responses=True
)


@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'service': 'product-service'})


@app.route('/api/products', methods=['GET'])
def get_products():
    # Check cache
    cached = redis_client.get('products:all')
```

```
    if cached:
        app.logger.info('Returning products from cache')
        return jsonify(json.loads(cached))

    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('SELECT * FROM products ORDER BY id')
    products = cur.fetchall()
    cur.close()
    conn.close()

    # Convert Decimal to float for JSON serialization
    products_list = []
    for p in products:
        product = dict(p)
        product['price'] = float(product['price'])
        product['created_at'] = str(product['created_at'])
        products_list.append(product)

    # Cache for 60 seconds
    redis_client.setex('products:all', 60, json.dumps(products_list))

    app.logger.info('Returning products from database')
    return jsonify(products_list)


@app.route('/api/products/<int:product_id>', methods=['GET'])
def get_product(product_id):
    # Check cache
    cached = redis_client.get(f'products:{product_id}')
    if cached:
        return jsonify(json.loads(cached))

    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('SELECT * FROM products WHERE id = %s', (product_id,))
    product = cur.fetchone()
    cur.close()
    conn.close()

    if not product:
        return jsonify({'error': 'Product not found'}), 404

    product = dict(product)
    product['price'] = float(product['price'])
    product['created_at'] = str(product['created_at'])

    redis_client.setex(f'products:{product_id}', 60, json.dumps(product))
    return jsonify(product)


@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.get_json()
    conn = get_db_connection()
```

```
    cur = conn.cursor()
    cur.execute(
        'INSERT INTO products (name, description, price, stock) '
        'VALUES (%s, %s, %s, %s) RETURNING *',
        (data['name'], data.get('description', ''),
         data['price'], data.get('stock', 0))
    )
    product = dict(cur.fetchone())
    conn.commit()
    cur.close()
    conn.close()

    product['price'] = float(product['price'])
    product['created_at'] = str(product['created_at'])

    # Invalidate cache
    redis_client.delete('products:all')

    return jsonify(product), 201


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

product-service/Dockerfile

```dockerfile
# Stage 1: Install dependencies
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2: Production image
FROM python:3.11-slim
WORKDIR /app

# Create non-root user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# Copy installed packages from builder
COPY --from=builder /install /usr/local

# Copy application code
COPY app.py .

# Switch to non-root user
USER appuser

EXPOSE 5000
```

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/h...

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--timeout", "120", "app:a...
```

Step 4: Order Service (Java Spring Boot)

The Order Service handles order creation and tracking. We will build it with Java and Spring Boot.

order-service/pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.1</version>
    </parent>

    <groupId>com.orders</groupId>
    <artifactId>order-service</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

order-service/src/main/resources/application.yml

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:orderdb}
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: none
    show-sql: false
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: ${REDIS_PORT:6379}

management:
  endpoints:
    web:
      exposure:
        include: health
  endpoint:
    health:
      show-details: always
```

order-service/src/main/java/com/orders/OrderServiceApplication.java

```java
package com.orders;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

order-service/src/main/java/com/orders/model/Order.java

```java
package com.orders.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", nullable = false)
    private Long userId;

    @Column(name = "product_id", nullable = false)
    private Long productId;

    @Column(nullable = false)
    private Integer quantity = 1;

    @Column(name = "total_price", nullable = false)
    private BigDecimal totalPrice;

    @Column(length = 50)
    private String status = "pending";

    @Column(name = "created_at")
    private LocalDateTime createdAt = LocalDateTime.now();
```

```
    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Long getUserId() { return userId; }
    public void setUserId(Long userId) { this.userId = userId; }

    public Long getProductId() { return productId; }
    public void setProductId(Long productId) { this.productId = productId; }

    public Integer getQuantity() { return quantity; }
    public void setQuantity(Integer quantity) { this.quantity = quantity; }

    public BigDecimal getTotalPrice() { return totalPrice; }
    public void setTotalPrice(BigDecimal totalPrice) { this.totalPrice = totalPrice; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
}
```

order-service/src/main/java/com/orders/repository/OrderRepository.java

```java
package com.orders.repository;

import com.orders.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByUserIdOrderByCreatedAtDesc(Long userId);
    List<Order> findByStatusOrderByCreatedAtDesc(String status);
}
```

order-service/src/main/java/com/orders/controller/OrderController.java

```java
package com.orders.controller;

import com.orders.model.Order;
```

```
import com.orders.repository.OrderRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
public class OrderController {

    @Autowired
    private OrderRepository orderRepository;

    @GetMapping("/health")
    public Map<String, String> health() {
        Map<String, String> status = new HashMap<>();
        status.put("status", "healthy");
        status.put("service", "order-service");
        return status;
    }

    @GetMapping("/api/orders")
    public List<Order> getAllOrders() {
        return orderRepository.findAll();
    }

    @GetMapping("/api/orders/{id}")
    public ResponseEntity<?> getOrder(@PathVariable Long id) {
        return orderRepository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/api/orders/user/{userId}")
    public List<Order> getOrdersByUser(@PathVariable Long userId) {
        return orderRepository.findByUserIdOrderByCreatedAtDesc(userId);
    }

    @PostMapping("/api/orders")
    public ResponseEntity<Order> createOrder(@RequestBody Order order) {
        order.setStatus("pending");
        Order saved = orderRepository.save(order);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @PutMapping("/api/orders/{id}/status")
    public ResponseEntity<?> updateStatus(
            @PathVariable Long id,
            @RequestBody Map<String, String> body) {
        return orderRepository.findById(id)
                .map(order -> {
                    order.setStatus(body.get("status"));
```

```
                    orderRepository.save(order);
                    return ResponseEntity.ok(order);
                })
                .orElse(ResponseEntity.notFound().build());
    }
}
```

order-service/Dockerfile

```dockerfile
# Stage 1: Build the application
FROM maven:3.9-eclipse-temurin-17-alpine AS builder
WORKDIR /app
COPY pom.xml .
# Download dependencies first (cached if pom.xml doesn't change)
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn package -DskipTests -B

# Stage 2: Production image
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copy the built JAR from the builder stage
COPY --from=builder /app/target/*.jar app.jar

# Switch to non-root user
USER appuser

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:8080/actuator/health || exit 1

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Notice that the Java Dockerfile has a longer `start-period` in the health check. Java applications take longer to start up than Node.js or Python.

```
+-----------------------------------------------------+
|     JAVA MULTI-STAGE BUILD                          |
|                                                     |
|  Stage 1 (builder) ~500MB:                          |
|  ┌──────────────────────────────────┐               |
|  │  Full JDK + Maven               |               |
|  │  All source code                 |               |
|  │  All build tools                 |               |
```

```
|   |  Compiles to a single JAR file   |              |
|   └──────────────────────────────────┘              |
|                     |                               |
|                     | copy app.jar only              |
|                     ▼                               |
|  Stage 2 (production) ~200MB:                       |
|   ┌──────────────────────────────────┐              |
|   |  JRE only (no compiler needed)   |              |
|   |  Just the JAR file               |              |
|   |  Non-root user                   |              |
|   |  60% smaller than Stage 1        |              |
|   └──────────────────────────────────┘              |
+-----------------------------------------------------+
```

Step 5: API Gateway (Nginx)

The API Gateway is the front door. All requests from the outside world come through here, and Nginx routes them to the correct service.

api-gateway/nginx.conf

```nginx
upstream user_service {
    server user-service:3001;
}

upstream product_service {
    server product-service:5000;
}

upstream order_service {
    server order-service:8080;
}

server {
    listen 80;
    server_name localhost;

    # Health check for the gateway itself
    location /health {
        return 200 '{"status": "healthy", "service": "api-gateway"}';
        add_header Content-Type application/json;
    }

    # Route user requests to the User Service
    location /api/users {
```

```
        proxy_pass http://user_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # Route product requests to the Product Service
    location /api/products {
        proxy_pass http://product_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # Route order requests to the Order Service
    location /api/orders {
        proxy_pass http://order_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # Default: return info about available endpoints
    location / {
        return 200 '{
            "service": "microservices-api-gateway",
            "endpoints": {
                "users": "/api/users",
                "products": "/api/products",
                "orders": "/api/orders",
                "health": "/health"
            }
        }';
        add_header Content-Type application/json;
    }
}
```

Here is how routing works:

```
+-----------------------------------------------------+
|              HOW THE GATEWAY ROUTES                 |
|                                                     |
|  Request: GET /api/users                            |
|     └──> Nginx sees "/api/users"                    |
|          └──> Forward to user-service:3001          |
|                                                     |
|  Request: GET /api/products/1                       |
|     └──> Nginx sees "/api/products"                 |
|          └──> Forward to product-service:5000       |
|                                                     |
|  Request: POST /api/orders                          |
|     └──> Nginx sees "/api/orders"                   |
|          └──> Forward to order-service:8080         |
```

```
|                                                      |
|  The client only talks to port 80.                   |
|  It never knows about the internal services.         |
+------------------------------------------------------+
```

Step 6: Docker Compose — Bringing It All Together

Now we wire everything together with a single `docker-compose.yml` file. This is the orchestra conductor.

docker-compose.yml

```yaml
version: '3.8'

services:
  # ═══════════════════════════════════════════════
  # Infrastructure Services
  # ═══════════════════════════════════════════════

  postgres:
    image: postgres:16-alpine
    container_name: microservices-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - microservices-network

  redis:
    image: redis:7-alpine
    container_name: microservices-redis
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
```

```
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - microservices-network

  # ──────────────────────────────────────────────
  # Application Services
  # ──────────────────────────────────────────────

  user-service:
    build:
      context: ./user-service
      dockerfile: Dockerfile
    container_name: microservices-user-service
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: userdb
      DB_USER: postgres
      DB_PASSWORD: postgres
      REDIS_HOST: redis
      REDIS_PORT: 6379
      PORT: 3001
    ports:
      - "3001:3001"
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - microservices-network
    restart: unless-stopped

  product-service:
    build:
      context: ./product-service
      dockerfile: Dockerfile
    container_name: microservices-product-service
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: productdb
      DB_USER: postgres
      DB_PASSWORD: postgres
      REDIS_HOST: redis
      REDIS_PORT: 6379
    ports:
      - "5000:5000"
    depends_on:
      postgres:
        condition: service_healthy
```

```
      redis:
        condition: service_healthy
    networks:
      - microservices-network
    restart: unless-stopped

  order-service:
    build:
      context: ./order-service
      dockerfile: Dockerfile
    container_name: microservices-order-service
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: orderdb
      DB_USER: postgres
      DB_PASSWORD: postgres
      REDIS_HOST: redis
      REDIS_PORT: 6379
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - microservices-network
    restart: unless-stopped

  # ──────────────────────────────────────────────
  # API Gateway
  # ──────────────────────────────────────────────

  api-gateway:
    image: nginx:1.25-alpine
    container_name: microservices-api-gateway
    ports:
      - "80:80"
    volumes:
      - ./api-gateway/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - user-service
      - product-service
      - order-service
    networks:
      - microservices-network
    restart: unless-stopped

# ──────────────────────────────────────────────
# Volumes (persistent data)
# ──────────────────────────────────────────────

volumes:
  postgres-data:
```

```
  redis-data:

# ─────────────────────────────────────────────────
# Networks
# ─────────────────────────────────────────────────

networks:
  microservices-network:
    driver: bridge
```

Let us understand the startup order:

```
+-----------------------------------------------------+
|            STARTUP ORDER (depends_on)               |
|                                                     |
|  Step 1:  postgres + redis start                    |
|           (they have no dependencies)               |
|                 |                                   |
|                 | wait until healthy                 |
|                 ▼                                   |
|  Step 2:  user-service + product-service            |
|           + order-service start                     |
|           (they need postgres and redis)            |
|                 |                                   |
|                 | wait until running                 |
|                 ▼                                   |
|  Step 3:  api-gateway starts                        |
|           (it needs all three services)             |
+-----------------------------------------------------+
```

Step 7: Build and Test Locally

Now let us build and run everything.

Build all images

```bash
docker compose build
```

```
[+] Building 45.2s (35/35) FINISHED
 => [user-service] Building...
 => [product-service] Building...
 => [order-service] Building...
```

Start all services

```bash
docker compose up -d
```

```
[+] Running 6/6
 ✔ Network microservices-app_microservices-network  Created
 ✔ Container microservices-postgres                 Started
 ✔ Container microservices-redis                    Started
 ✔ Container microservices-user-service             Started
 ✔ Container microservices-product-service          Started
 ✔ Container microservices-order-service            Started
 ✔ Container microservices-api-gateway              Started
```

Check that everything is running

```bash
docker compose ps
```

```
NAME                            IMAGE                     STATUS                   PORTS
microservices-api-gateway       nginx:1.25-alpine         Up 30 seconds            0.0.0...
microservices-order-service     order-service:latest       Up 30 seconds (healthy)  0.0....
microservices-postgres          postgres:16-alpine         Up 45 seconds (healthy)  0.0....
microservices-product-service   product-service:latest     Up 30 seconds (healthy)  0.0....
microservices-redis             redis:7-alpine             Up 45 seconds (healthy)  0.0....
microservices-user-service      user-service:latest        Up 30 seconds (healthy)  0.0....
```

Test the API Gateway

```bash
# Check the gateway itself
curl http://localhost/health
```

```json
{"status": "healthy", "service": "api-gateway"}
```

Test the User Service

```bash
# Get all users (through the gateway)
curl http://localhost/api/users
```

```json
[
  {"id": 1, "username": "alice", "email": "alice@example.com", "created_at": "2024-01-1...
  {"id": 2, "username": "bob", "email": "bob@example.com", "created_at": "2024-01-15T10...
  {"id": 3, "username": "charlie", "email": "charlie@example.com", "created_at": "2024-...
]
```

```bash
# Create a new user
curl -X POST http://localhost/api/users \
  -H "Content-Type: application/json" \
  -d '{"username": "diana", "email": "diana@example.com"}'
```

```json
{"id": 4, "username": "diana", "email": "diana@example.com", "created_at": "2024-01-15T...
```

Test the Product Service

```bash
# Get all products
curl http://localhost/api/products
```

```json
[
  {"id": 1, "name": "Laptop", "description": "High-performance laptop", "price": 999.99...
  {"id": 2, "name": "Headphones", "description": "Wireless noise-canceling headphones",...
  {"id": 3, "name": "Keyboard", "description": "Mechanical keyboard with RGB", "price":...
  {"id": 4, "name": "Mouse", "description": "Ergonomic wireless mouse", "price": 49.99,...
]
```

Test the Order Service

```bash
# Get all orders
curl http://localhost/api/orders
```

```json
[
  {"id": 1, "userId": 1, "productId": 1, "quantity": 1, "totalPrice": 999.99, "status":...
  {"id": 2, "userId": 2, "productId": 2, "quantity": 2, "totalPrice": 299.98, "status":...
  {"id": 3, "userId": 1, "productId": 3, "quantity": 1, "totalPrice": 79.99, "status": ...
]
```

```bash
# Create a new order
curl -X POST http://localhost/api/orders \
  -H "Content-Type: application/json" \
  -d '{"userId": 3, "productId": 4, "quantity": 2, "totalPrice": 99.98}'
```

```json
{"id": 4, "userId": 3, "productId": 4, "quantity": 2, "totalPrice": 99.98, "status": "p...
```

View logs

```bash
# See logs from all services
docker compose logs

# Follow logs from a specific service
docker compose logs -f user-service
```

Stop everything

```bash
docker compose down
```

Step 8: Kubernetes Manifests

Now let us deploy the same application to Kubernetes. We need to translate our Compose file into Kubernetes YAML manifests.

```
+-----------------------------------------------------+
|    COMPOSE vs KUBERNETES TRANSLATION                |
|                                                     |
|    Compose                 Kubernetes               |
|    _______                 __________               |
|    service:          -->   Deployment + Service     |
|    ports:            -->   Service (type: NodePort) |
|    environment:      -->   ConfigMap or Secret      |
|    volumes:          -->   PersistentVolumeClaim    |
|    depends_on:       -->   (handled by readiness    |
|                             probes and retry logic) |
|    networks:         -->   (built-in networking)    |
+-----------------------------------------------------+
```

k8s/namespace.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: microservices
  labels:
    app: microservices-demo
```

k8s/postgres.yaml

```yaml
# ConfigMap for initialization SQL
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-init
  namespace: microservices
data:
  init.sql: |
    CREATE DATABASE userdb;
    CREATE DATABASE productdb;
    CREATE DATABASE orderdb;

    \c userdb;
    CREATE TABLE users (
        id SERIAL PRIMARY KEY,
        username VARCHAR(100) NOT NULL UNIQUE,
        email VARCHAR(255) NOT NULL UNIQUE,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
    INSERT INTO users (username, email) VALUES
        ('alice', 'alice@example.com'),
        ('bob', 'bob@example.com'),
        ('charlie', 'charlie@example.com');

    \c productdb;
    CREATE TABLE products (
        id SERIAL PRIMARY KEY,
        name VARCHAR(255) NOT NULL,
        description TEXT,
        price DECIMAL(10, 2) NOT NULL,
        stock INTEGER DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
    INSERT INTO products (name, description, price, stock) VALUES
        ('Laptop', 'High-performance laptop', 999.99, 50),
        ('Headphones', 'Wireless noise-canceling headphones', 149.99, 200),
        ('Keyboard', 'Mechanical keyboard with RGB', 79.99, 150),
        ('Mouse', 'Ergonomic wireless mouse', 49.99, 300);
```

```
    \c orderdb;
    CREATE TABLE orders (
        id SERIAL PRIMARY KEY,
        user_id INTEGER NOT NULL,
        product_id INTEGER NOT NULL,
        quantity INTEGER NOT NULL DEFAULT 1,
        total_price DECIMAL(10, 2) NOT NULL,
        status VARCHAR(50) DEFAULT 'pending',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
    INSERT INTO orders (user_id, product_id, quantity, total_price, status) VALUES
        (1, 1, 1, 999.99, 'completed'),
        (2, 2, 2, 299.98, 'pending'),
        (1, 3, 1, 79.99, 'shipped');
---
# Secret for database credentials
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
  namespace: microservices
type: Opaque
stringData:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
---
# PersistentVolumeClaim for data storage
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: microservices
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
# Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: microservices
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
```

```
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: postgres-secret
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data
            - name: init-scripts
              mountPath: /docker-entrypoint-initdb.d
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres"]
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres"]
            initialDelaySeconds: 15
            periodSeconds: 20
      volumes:
        - name: postgres-data
          persistentVolumeClaim:
            claimName: postgres-pvc
        - name: init-scripts
          configMap:
            name: postgres-init
---
# Service (so other pods can reach PostgreSQL)
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: microservices
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
  type: ClusterIP
```

k8s/redis.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: redis
  namespace: microservices
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 15
            periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: microservices
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
  type: ClusterIP
```

k8s/user-service.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-service-config
  namespace: microservices
data:
```

```
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: userdb
  DB_USER: postgres
  REDIS_HOST: redis
  REDIS_PORT: "6379"
  PORT: "3001"
---
apiVersion: v1
kind: Secret
metadata:
  name: user-service-secret
  namespace: microservices
type: Opaque
stringData:
  DB_PASSWORD: postgres
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  namespace: microservices
spec:
  replicas: 2
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: user-service:1.0
          ports:
            - containerPort: 3001
          envFrom:
            - configMapRef:
                name: user-service-config
            - secretRef:
                name: user-service-secret
          readinessProbe:
            httpGet:
              path: /health
              port: 3001
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 3001
            initialDelaySeconds: 10
            periodSeconds: 20
          resources:
```

```
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 250m
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: user-service
  namespace: microservices
spec:
  selector:
    app: user-service
  ports:
    - port: 3001
      targetPort: 3001
  type: ClusterIP
```

k8s/product-service.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: product-service-config
  namespace: microservices
data:
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: productdb
  DB_USER: postgres
  REDIS_HOST: redis
  REDIS_PORT: "6379"
---
apiVersion: v1
kind: Secret
metadata:
  name: product-service-secret
  namespace: microservices
type: Opaque
stringData:
  DB_PASSWORD: postgres
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: microservices
spec:
```

```yaml
  replicas: 2
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: product-service
          image: product-service:1.0
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: product-service-config
            - secretRef:
                name: product-service-secret
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 20
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 250m
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: microservices
spec:
  selector:
    app: product-service
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
```

k8s/order-service.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: order-service-config
  namespace: microservices
data:
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: orderdb
  DB_USER: postgres
  REDIS_HOST: redis
  REDIS_PORT: "6379"
---
apiVersion: v1
kind: Secret
metadata:
  name: order-service-secret
  namespace: microservices
type: Opaque
stringData:
  DB_PASSWORD: postgres
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: microservices
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: order-service:1.0
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: order-service-config
            - secretRef:
                name: order-service-secret
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
```

```
          livenessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 20
          resources:
            requests:
              cpu: 200m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
  namespace: microservices
spec:
  selector:
    app: order-service
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP
```

k8s/api-gateway.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: microservices
data:
  default.conf: |
    upstream user_service {
        server user-service:3001;
    }
    upstream product_service {
        server product-service:5000;
    }
    upstream order_service {
        server order-service:8080;
    }
    server {
        listen 80;
        server_name localhost;

        location /health {
```

```
            return 200 '{"status": "healthy", "service": "api-gateway"}';
            add_header Content-Type application/json;
        }
        location /api/users {
            proxy_pass http://user_service;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
        location /api/products {
            proxy_pass http://product_service;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
        location /api/orders {
            proxy_pass http://order_service;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
        location / {
            return 200 '{"service": "microservices-api-gateway", "endpoints": {"users":...
            add_header Content-Type application/json;
        }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
  namespace: microservices
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api-gateway
  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
          volumeMounts:
            - name: nginx-config
              mountPath: /etc/nginx/conf.d
          readinessProbe:
            httpGet:
              path: /health
              port: 80
            initialDelaySeconds: 5
```

```
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 20
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 100m
              memory: 128Mi
      volumes:
        - name: nginx-config
          configMap:
            name: nginx-config
---
apiVersion: v1
kind: Service
metadata:
  name: api-gateway
  namespace: microservices
spec:
  selector:
    app: api-gateway
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  type: NodePort
```

Step 9: Deploy to Kubernetes

Now let us deploy the same application to Kubernetes, step by step.

Build and tag images for Kubernetes

If you are using Minikube, you need to point your Docker client to Minikube's Docker daemon:

```bash
# For Minikube users
eval $(minikube docker-env)
```

Build the images:

```bash
docker build -t user-service:1.0 ./user-service/
docker build -t product-service:1.0 ./product-service/
docker build -t order-service:1.0 ./order-service/
```

Create the namespace

```bash
kubectl apply -f k8s/namespace.yaml
```

```
namespace/microservices created
```

Deploy infrastructure first

```bash
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/redis.yaml
```

```
configmap/postgres-init created
secret/postgres-secret created
persistentvolumeclaim/postgres-pvc created
deployment.apps/postgres created
service/postgres created
deployment.apps/redis created
service/redis created
```

Wait for infrastructure to be ready:

```bash
kubectl wait --for=condition=ready pod -l app=postgres -n microservices --timeout=120s
kubectl wait --for=condition=ready pod -l app=redis -n microservices --timeout=120s
```

```
pod/postgres-7d4b8c9f-x2k9 condition met
pod/redis-5c6d7e8f-j4k2 condition met
```

Deploy application services

```bash
kubectl apply -f k8s/user-service.yaml
kubectl apply -f k8s/product-service.yaml
```

```
kubectl apply -f k8s/order-service.yaml
kubectl apply -f k8s/api-gateway.yaml
```

```
configmap/user-service-config created
secret/user-service-secret created
deployment.apps/user-service created
service/user-service created
configmap/product-service-config created
secret/product-service-secret created
deployment.apps/product-service created
service/product-service created
configmap/order-service-config created
secret/order-service-secret created
deployment.apps/order-service created
service/order-service created
configmap/nginx-config created
deployment.apps/api-gateway created
service/api-gateway created
```

Verify everything is running

```bash
kubectl get all -n microservices
```

```
NAME                                    READY   STATUS    RESTARTS   AGE
pod/api-gateway-6b8c9d-m3p7             1/1     Running   0          60s
pod/api-gateway-6b8c9d-x2k9             1/1     Running   0          60s
pod/order-service-7d4b8c-j4k2           1/1     Running   0          65s
pod/order-service-7d4b8c-n5o6           1/1     Running   0          65s
pod/postgres-5c6d7e-p7q8                1/1     Running   0          90s
pod/product-service-8e9f0a-r9s0         1/1     Running   0          70s
pod/product-service-8e9f0a-t1u2         1/1     Running   0          70s
pod/redis-4b5c6d-v3w4                   1/1     Running   0          85s
pod/user-service-9f0a1b-x5y6            1/1     Running   0          75s
pod/user-service-9f0a1b-z7a8            1/1     Running   0          75s

NAME                        TYPE        CLUSTER-IP      PORT(S)        AGE
service/api-gateway         NodePort    10.96.100.1     80:30080/TCP   60s
service/order-service       ClusterIP   10.96.100.2     8080/TCP       65s
service/postgres            ClusterIP   10.96.100.3     5432/TCP       90s
service/product-service     ClusterIP   10.96.100.4     5000/TCP       70s
service/redis               ClusterIP   10.96.100.5     6379/TCP       85s
service/user-service        ClusterIP   10.96.100.6     3001/TCP       75s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api-gateway         2/2     2            2           60s
deployment.apps/order-service       2/2     2            2           65s
deployment.apps/postgres            1/1     1            1           90s
deployment.apps/product-service     2/2     2            2           70s
```

```
deployment.apps/redis             1/1     1            1           85s
deployment.apps/user-service      2/2     2            2           75s
```

Test the Kubernetes deployment

```bash
# For Minikube
minikube service api-gateway -n microservices --url
```

```
http://192.168.49.2:30080
```

```bash
# Test the endpoints
curl http://192.168.49.2:30080/api/users
curl http://192.168.49.2:30080/api/products
curl http://192.168.49.2:30080/api/orders
```

Scale services

```bash
# Scale the user service to 4 replicas
kubectl scale deployment user-service --replicas=4 -n microservices
```

```
deployment.apps/user-service scaled
```

```bash
# Verify
kubectl get pods -l app=user-service -n microservices
```

```
NAME                            READY   STATUS    RESTARTS   AGE
user-service-9f0a1b-x5y6       1/1     Running   0          5m
user-service-9f0a1b-z7a8       1/1     Running   0          5m
user-service-9f0a1b-b9c0       1/1     Running   0          10s
user-service-9f0a1b-d1e2       1/1     Running   0          10s
```

View logs in Kubernetes

```bash
# Logs from a specific pod
kubectl logs user-service-9f0a1b-x5y6 -n microservices

# Follow logs
kubectl logs -f user-service-9f0a1b-x5y6 -n microservices

# Logs from all pods of a deployment
kubectl logs deployment/user-service -n microservices
```

Clean up Kubernetes deployment

```bash
# Delete everything in the namespace
kubectl delete namespace microservices
```

```
namespace "microservices" deleted
```

The Complete Architecture — One Final Look

Here is the full picture of what you built:

```
+=================================================================+
||                 MICROSERVICES APPLICATION                      ||
||                                                                ||
||   ┌──────────────────────────────────────────────────────────┐ ||
||   │                      INTERNET                            │ ||
||   └──────────────────────────┬───────────────────────────────┘ ||
||                              │                                  ||
||                              ▼                                  ||
||   ┌──────────────────────────────────────────────────────────┐ ||
||   │              API GATEWAY (Nginx)                          │ ||
||   │              Port 80 / NodePort 30080                     │ ||
||   │                                                           │ ||
||   │   /api/users  ──>  user-service:3001                      │ ||
||   │   /api/products ──> product-service:5000                  │ ||
||   │   /api/orders ──>  order-service:8080                     │ ||
||   └──────────────────────────────────────────────────────────┘ ||
||       │                 │                 │                   ||
||       ▼                 ▼                 ▼                   ||
||   ┌──────────┐      ┌──────────┐      ┌──────────┐            ||
||   │  USER    │      │ PRODUCT  │      │  ORDER   │            ||
||   │ SERVICE  │      │ SERVICE  │      │ SERVICE  │            ||
||   │          │      │          │      │          │            ||
```

```
||    | Node.js  |      |  Python  |       |   Java   |              ||
||    | Express  |      |  Flask   |       |  Spring  |              ||
||    | 2 pods   |      |  2 pods  |       |  2 pods  |              ||
||    └──────────┘      └──────────┘       └──────────┘              ||
||        │                │                  │                       ||
||        └────────────────┴──────────────────┘                       ||
||                         │                                          ||
||                         ┴                                          ||
||               ┌─────────┴─────────┐                                ||
||               │                   │                                ||
||               ▼                   ▼                                ||
||    ┌──────────────────┐  ┌──────────────────┐                     ||
||    │   PostgreSQL     │  │      Redis       │                     ||
||    │   3 databases    │  │      Cache       │                     ||
||    │   (userdb,       │  │                  │                     ||
||    │    productdb,    │  │   Speeds up      │                     ||
||    │    orderdb)      │  │   read queries   │                     ||
||    └──────────────────┘  └──────────────────┘                     ||
||                                                                    ||
+====================================================================+
```

Congratulations

You have built a complete microservices application from scratch. Let us take a moment to appreciate what you accomplished:

- You wrote three services in three different programming languages (Node.js, Python, Java)
- Each service has its own optimized, multi-stage Dockerfile
- You connected them all through a shared database and Redis cache
- You set up an API Gateway to route traffic
- You deployed everything locally with Docker Compose
- You translated the entire application to Kubernetes manifests
- You deployed to Kubernetes with proper health checks, resource limits, and scaling

This is not a toy project. This is the same architecture pattern used by professional engineering teams around the world.

What to Learn Next

Your Docker and Kubernetes journey does not end here. Here are the roads ahead:

Helm — Package Manager for Kubernetes

Helm lets you package your Kubernetes manifests into reusable charts. Instead of applying seven YAML files one by one, you run a single command. Think of Helm as what `apt-get` or `brew` is for your computer, but for Kubernetes.

```
helm install my-app ./my-chart
```

Terraform — Infrastructure as Code

Terraform lets you define your entire cloud infrastructure (servers, networks, databases, Kubernetes clusters) in code. Instead of clicking buttons in a cloud console, you write configuration files and run `terraform apply`. Your infrastructure becomes versioned, reviewable, and repeatable.

```
terraform apply    # Creates your entire cloud setup
terraform destroy  # Tears it all down cleanly
```

Service Mesh (Istio, Linkerd)

As your microservices grow, you need advanced networking features: traffic encryption between services, load balancing, circuit breakers, and observability. A service mesh handles all of this without changing your application code. Think of it as a smart highway system between your services.

Cloud Providers

Once you are comfortable with Docker and Kubernetes locally, take your skills to the cloud:

- AWS — ECS (Elastic Container Service) or EKS (Elastic Kubernetes Service)
- Google Cloud — GKE (Google Kubernetes Engine)
- Azure — AKS (Azure Kubernetes Service)
- DigitalOcean — DOKS (DigitalOcean Kubernetes Service)

Each provider offers managed Kubernetes clusters, so you do not have to manage the control plane yourself.

Monitoring and Observability

Learn Prometheus for metrics collection, Grafana for dashboards, and tools like Jaeger for distributed tracing. When you have dozens of services, you need to see what is happening inside your system at all times.

```
+-----------------------------------------------------+
|             YOUR LEARNING ROADMAP                   |
|                                                     |
|   You Are Here                                      |
|       |                                             |
|       ├── Helm (package K8s apps)                   |
|       |                                             |
|       ├── Terraform (infrastructure as code)        |
|       |                                             |
|       ├── Service Mesh (advanced networking)        |
|       |                                             |
|       ├── Cloud Providers (AWS, GCP, Azure)         |
|       |                                             |
|       ├── Monitoring (Prometheus, Grafana)           |
|       |                                             |
|       └── CI/CD Pipelines (GitHub Actions,           |
|           GitLab CI, Jenkins)                       |
|                                                     |
|   Each step builds on what you already know.        |
|   You have a strong foundation. Keep building.      |
+-----------------------------------------------------+
```

Key Points

- A microservices architecture splits an application into small, independent services that each do one thing well.
- Each service gets its own Dockerfile, its own database, and its own deployment configuration.
- An API Gateway (Nginx) routes external traffic to the correct internal service.
- Docker Compose orchestrates multi-container applications locally with a single YAML file.
- Kubernetes manifests define the same application for production deployment, with added benefits like scaling, self-healing, and rolling updates.
- Multi-stage Docker builds keep images small and secure by separating build tools from runtime.

- Health checks (in Docker) and readiness/liveness probes (in Kubernetes) ensure traffic only reaches healthy services.
- Secrets and ConfigMaps in Kubernetes separate sensitive data from application code.
- Scaling in Kubernetes is as simple as changing the replica count.
- The skills you learned in this book — images, containers, volumes, networks, Compose, and Kubernetes — all come together in real-world applications exactly like this one.